

R for Public Health: A Problem-Based Learning Course

**From Zero to Advanced R for Epidemiology, Biostatistics, and
Health Data Science**

Compiled by: Bikram Adhikari

2026-07-23

Table of contents

- Welcome** **1**
 - Why this course exists 1
 - Who this is for 1
 - How each lesson works 2
 - The dataset you'll live in 3
 - Course map 3
 - What you'll be able to do 4

- I. Level 1: Getting Started with R** **5**
 - Level 1 — Getting Started with R** **7**
 - What you'll build toward 7
 - Topics in this level 7

 - 1. What Is R?** **9**
 - 1.1. The R Concept 10
 - 1.2. Demonstration 11
 - 1.3. Explore 11
 - 1.4. Solve the Problem 12
 - 1.5. Interpret the Result 12
 - 1.6. Practice Problem 13
 - 1.7. Challenge Problem 13
 - 1.8. Key Takeaways 13

 - 2. What Is RStudio?** **15**
 - 2.1. The R Concept 15

Table of contents

2.2. Demonstration	16
2.3. Explore	17
2.4. Solve the Problem	17
2.5. Interpret the Result	18
2.6. Practice Problem	18
2.7. Challenge Problem	18
2.8. Key Takeaways	19
3. The RStudio Interface	21
3.1. The R Concept	21
3.2. Demonstration	22
3.3. Explore	23
3.4. Solve the Problem	23
3.5. Interpret the Result	24
3.6. Practice Problem	24
3.7. Challenge Problem	24
3.8. Key Takeaways	25
4. Installing and Loading Packages	27
4.1. The R Concept	27
4.2. Demonstration	28
4.3. Explore	29
4.4. Solve the Problem	29
4.5. Interpret the Result	30
4.6. Practice Problem	30
4.7. Challenge Problem	31
4.8. Key Takeaways	31
5. Using the Console	33
5.1. The R Concept	33
5.2. Demonstration	34
5.3. Explore	34
5.4. Solve the Problem	35
5.5. Interpret the Result	35

Table of contents

5.6. Practice Problem	36
5.7. Challenge Problem	36
5.8. Key Takeaways	36
6. Creating an R Script	39
6.1. The R Concept	39
6.2. Demonstration	40
6.3. Explore	41
6.4. Solve the Problem	41
6.5. Interpret the Result	42
6.6. Practice Problem	42
6.7. Challenge Problem	42
6.8. Key Takeaways	43
7. Comments	45
7.1. The R Concept	46
7.2. Demonstration	46
7.3. Explore	47
7.4. Solve the Problem	47
7.5. Interpret the Result	48
7.6. Practice Problem	49
7.7. Challenge Problem	49
7.8. Key Takeaways	49
8. Basic Arithmetic	51
8.1. The R Concept	51
8.2. Demonstration	52
8.3. Explore	53
8.4. Solve the Problem	53
8.5. Interpret the Result	54
8.6. Practice Problem	54
8.7. Challenge Problem	54
8.8. Key Takeaways	55

Table of contents

9. The Assignment Operator	57
9.1. The R Concept	57
9.2. Demonstration	58
9.3. Explore	59
9.4. Solve the Problem	59
9.5. Interpret the Result	60
9.6. Practice Problem	61
9.7. Challenge Problem	61
9.8. Key Takeaways	61
10. Objects	63
10.1. The R Concept	63
10.2. Demonstration	64
10.3. Explore	65
10.4. Solve the Problem	65
10.5. Interpret the Result	65
10.6. Practice Problem	66
10.7. Challenge Problem	66
10.8. Key Takeaways	66
11. Naming Objects	69
11.1. The R Concept	70
11.2. Demonstration	70
11.3. Explore	71
11.4. Solve the Problem	71
11.5. Interpret the Result	72
11.6. Practice Problem	72
11.7. Challenge Problem	73
11.8. Key Takeaways	73
12. Basic Data Types	75
12.1. The R Concept	76
12.2. Demonstration	76
12.3. Explore	77

Table of contents

12.4. Solve the Problem	77
12.5. Interpret the Result	78
12.6. Practice Problem	78
12.7. Challenge Problem	78
12.8. Key Takeaways	79
13. Numeric Data	81
13.1. The R Concept	81
13.2. Demonstration	82
13.3. Explore	83
13.4. Solve the Problem	83
13.5. Interpret the Result	84
13.6. Practice Problem	84
13.7. Challenge Problem	85
13.8. Key Takeaways	85
14. Character Data	87
14.1. The R Concept	88
14.2. Demonstration	88
14.3. Explore	89
14.4. Solve the Problem	89
14.5. Interpret the Result	90
14.6. Practice Problem	90
14.7. Challenge Problem	91
14.8. Key Takeaways	91
15. Logical Data	93
15.1. The R Concept	93
15.2. Demonstration	94
15.3. Explore	95
15.4. Solve the Problem	95
15.5. Interpret the Result	96
15.6. Practice Problem	97
15.7. Challenge Problem	97

Table of contents

15.8. Key Takeaways	97
16. Missing Values	99
16.1. The R Concept	99
16.2. Demonstration	100
16.3. Explore	101
16.4. Solve the Problem	101
16.5. Interpret the Result	102
16.6. Practice Problem	103
16.7. Challenge Problem	103
16.8. Key Takeaways	103
17. Basic Functions	105
17.1. The R Concept	105
17.2. Demonstration	106
17.3. Explore	107
17.4. Solve the Problem	107
17.5. Interpret the Result	108
17.6. Practice Problem	109
17.7. Challenge Problem	109
17.8. Key Takeaways	110
18. Getting Help in R	113
18.1. The R Concept	113
18.2. Demonstration	114
18.3. Explore	115
18.4. Solve the Problem	115
18.5. Interpret the Result	115
18.6. Practice Problem	116
18.7. Challenge Problem	116
18.8. Key Takeaways	117
19. Working Directories	119
19.1. The R Concept	120

19.2. Demonstration	120
19.3. Explore	121
19.4. Solve the Problem	121
19.5. Interpret the Result	122
19.6. Practice Problem	123
19.7. Challenge Problem	123
19.8. Key Takeaways	123
20. Importing Simple Data	125
20.1. The R Concept	126
20.2. Demonstration	126
20.3. Explore	127
20.4. Solve the Problem	127
20.5. Interpret the Result	128
20.6. Practice Problem	129
20.7. Challenge Problem	129
20.8. Key Takeaways	130
20.9. Where you've arrived	131
 II. Level 2: Data Structures in R	 133
Level 2 — Data Structures in R	135
What you'll build toward	135
Topics in this level	136
 21. Vectors and c()	 137
21.1. The R Concept	137
21.2. Demonstration	138
21.3. Explore	139
21.4. Solve the Problem	139
21.5. Interpret the Result	140
21.6. Practice Problem	140
21.7. Challenge Problem	140

Table of contents

21.8. Key Takeaways	141
22. Sequences with seq()	143
22.1. The R Concept	143
22.2. Demonstration	144
22.3. Explore	144
22.4. Solve the Problem	145
22.5. Interpret the Result	145
22.6. Practice Problem	146
22.7. Challenge Problem	146
22.8. Key Takeaways	146
23. Repeating Values with rep()	149
23.1. The R Concept	149
23.2. Demonstration	150
23.3. Explore	150
23.4. Solve the Problem	151
23.5. Interpret the Result	151
23.6. Practice Problem	152
23.7. Challenge Problem	152
23.8. Key Takeaways	152
24. Positive Indexing	155
24.1. The R Concept	155
24.2. Demonstration	156
24.3. Explore	157
24.4. Solve the Problem	157
24.5. Interpret the Result	157
24.6. Practice Problem	158
24.7. Challenge Problem	158
24.8. Key Takeaways	158
25. Negative Indexing	161
25.1. The R Concept	161

Table of contents

25.2. Demonstration	162
25.3. Explore	162
25.4. Solve the Problem	163
25.5. Interpret the Result	163
25.6. Practice Problem	164
25.7. Challenge Problem	164
25.8. Key Takeaways	164
26. Logical Indexing	167
26.1. The R Concept	167
26.2. Demonstration	168
26.3. Explore	169
26.4. Solve the Problem	169
26.5. Interpret the Result	170
26.6. Practice Problem	170
26.7. Challenge Problem	171
26.8. Key Takeaways	171
27. Named Vectors	173
27.1. The R Concept	173
27.2. Demonstration	174
27.3. Explore	175
27.4. Solve the Problem	175
27.5. Interpret the Result	176
27.6. Practice Problem	177
27.7. Challenge Problem	177
27.8. Key Takeaways	177
28. Factors	179
28.1. The R Concept	179
28.2. Demonstration	180
28.3. Explore	181
28.4. Solve the Problem	182
28.5. Interpret the Result	182

Table of contents

28.6. Practice Problem	183
28.7. Challenge Problem	184
28.8. Key Takeaways	184
29. Creating Data Frames	187
29.1. The R Concept	188
29.2. Demonstration	188
29.3. Explore	189
29.4. Solve the Problem	189
29.5. Interpret the Result	190
29.6. Practice Problem	191
29.7. Challenge Problem	191
29.8. Key Takeaways	191
30. Accessing Columns	193
30.1. The R Concept	193
30.2. Demonstration	194
30.3. Explore	195
30.4. Solve the Problem	195
30.5. Interpret the Result	196
30.6. Practice Problem	196
30.7. Challenge Problem	197
30.8. Key Takeaways	197
31. Accessing Rows	199
31.1. The R Concept	199
31.2. Demonstration	200
31.3. Explore	201
31.4. Solve the Problem	201
31.5. Interpret the Result	201
31.6. Practice Problem	202
31.7. Challenge Problem	202
31.8. Key Takeaways	203

32. Matrices	205
32.1. The R Concept	205
32.2. Demonstration	206
32.3. Explore	207
32.4. Solve the Problem	207
32.5. Interpret the Result	208
32.6. Practice Problem	209
32.7. Challenge Problem	209
32.8. Key Takeaways	209
33. Lists	211
33.1. The R Concept	212
33.2. Demonstration	212
33.3. Explore	213
33.4. Solve the Problem	214
33.5. Interpret the Result	215
33.6. Practice Problem	215
33.7. Challenge Problem	216
33.8. Key Takeaways	216
34. Tibbles	219
34.1. The R Concept	220
34.2. Demonstration	220
34.3. Explore	221
34.4. Solve the Problem	222
34.5. Interpret the Result	222
34.6. Practice Problem	223
34.7. Challenge Problem	223
34.8. Key Takeaways	223
35. Understanding Structure with <code>str()</code>	225
35.1. The R Concept	226
35.2. Demonstration	226
35.3. Explore	227

Table of contents

35.4. Solve the Problem	227
35.5. Interpret the Result	228
35.6. Practice Problem	229
35.7. Challenge Problem	229
35.8. Key Takeaways	229
36. Checking Dimensions: <code>nrow()</code>, <code>ncol()</code>, <code>dim()</code>	231
36.1. The R Concept	232
36.2. Demonstration	232
36.3. Explore	233
36.4. Solve the Problem	233
36.5. Interpret the Result	234
36.6. Practice Problem	234
36.7. Challenge Problem	234
36.8. Key Takeaways	235
37. <code>length()</code> and <code>names()</code>	237
37.1. The R Concept	238
37.2. Demonstration	238
37.3. Explore	239
37.4. Solve the Problem	239
37.5. Interpret the Result	240
37.6. Practice Problem	241
37.7. Challenge Problem	241
37.8. Key Takeaways	241
37.9. Where you've arrived	241
III. Level 3: Data Import and Export	245
Level 3 — Data Import and Export	247
What you'll build toward	247
Topics in this level	247
A preview mission	248

38. Reading CSV Files with readr	249
38.1. The R Concept	249
38.2. Demonstration	250
38.3. Explore	251
38.4. Solve the Problem	252
38.5. Interpret the Result	252
38.6. Practice Problem	253
38.7. Challenge Problem	253
38.8. Key Takeaways	253
39. Writing CSV Files	255
39.1. The R Concept	255
39.2. Demonstration	256
39.3. Explore	257
39.4. Solve the Problem	257
39.5. Interpret the Result	257
39.6. Practice Problem	258
39.7. Challenge Problem	258
39.8. Key Takeaways	259
40. Reading Excel Files	261
40.1. The R Concept	261
40.2. Demonstration	262
40.3. Explore	263
40.4. Solve the Problem	263
40.5. Interpret the Result	264
40.6. Practice Problem	264
40.7. Challenge Problem	265
40.8. Key Takeaways	265
41. Writing Excel Files	267
41.1. The R Concept	267
41.2. Demonstration	268
41.3. Explore	269

Table of contents

41.4. Solve the Problem	269
41.5. Interpret the Result	270
41.6. Practice Problem	270
41.7. Challenge Problem	271
41.8. Key Takeaways	271
42. Reading RDS Files	273
42.1. The R Concept	274
42.2. Demonstration	274
42.3. Explore	275
42.4. Solve the Problem	275
42.5. Interpret the Result	276
42.6. Practice Problem	276
42.7. Challenge Problem	277
42.8. Key Takeaways	277
43. Saving R Objects	279
43.1. The R Concept	279
43.2. Demonstration	280
43.3. Explore	281
43.4. Solve the Problem	281
43.5. Interpret the Result	282
43.6. Practice Problem	282
43.7. Challenge Problem	283
43.8. Key Takeaways	283
44. File Paths and Relative Paths	285
44.1. The R Concept	285
44.2. Demonstration	286
44.3. Explore	287
44.4. Solve the Problem	287
44.5. Interpret the Result	287
44.6. Practice Problem	288
44.7. Challenge Problem	288

44.8. Key Takeaways	289
45. RStudio Projects	291
45.1. The R Concept	291
45.2. Demonstration	292
45.3. Explore	293
45.4. Solve the Problem	293
45.5. Interpret the Result	294
45.6. Practice Problem	294
45.7. Challenge Problem	295
45.8. Key Takeaways	295
46. Importing Messy, Real-World Datasets	297
46.1. The R Concept	298
46.2. Demonstration	298
46.3. Explore	299
46.4. Solve the Problem	299
46.5. Interpret the Result	300
46.6. Practice Problem	301
46.7. Challenge Problem	301
46.8. Key Takeaways	301
47. Handling Missing Values on Import	303
47.1. The R Concept	304
47.2. Demonstration	304
47.3. Explore	305
47.4. Solve the Problem	305
47.5. Interpret the Result	306
47.6. Practice Problem	307
47.7. Challenge Problem	307
47.8. Key Takeaways	307
48. Checking Imported Data	311
48.1. The R Concept	311

Table of contents

48.2. Demonstration	312
48.3. Explore	313
48.4. Solve the Problem	313
48.5. Interpret the Result	314
48.6. Practice Problem	315
48.7. Challenge Problem	315
48.8. Key Takeaways	316
49. Understanding Variable Types After Import	319
49.1. The R Concept	320
49.2. Demonstration	320
49.3. Explore	321
49.4. Solve the Problem	322
49.5. Interpret the Result	322
49.6. Practice Problem	323
49.7. Challenge Problem	323
49.8. Key Takeaways	324
49.9. Where you've arrived	325
IV. Level 4: Data Cleaning with Base R	327
Level 4 — Data Cleaning with Base R	329
What you'll build toward	329
Topics in this level	329
A preview mission	330
50. Identifying Missing Values Across a Dataset	331
50.1. The R Concept	331
50.2. Demonstration	332
50.3. Explore	333
50.4. Solve the Problem	333
50.5. Interpret the Result	334
50.6. Practice Problem	334

Table of contents

50.7. Challenge Problem	335
50.8. Key Takeaways	335
51. Removing Missing Values	337
51.1. The R Concept	337
51.2. Demonstration	338
51.3. Explore	339
51.4. Solve the Problem	339
51.5. Interpret the Result	340
51.6. Practice Problem	340
51.7. Challenge Problem	341
51.8. Key Takeaways	341
52. Replacing Missing Values	343
52.1. The R Concept	343
52.2. Demonstration	344
52.3. Explore	345
52.4. Solve the Problem	345
52.5. Interpret the Result	346
52.6. Practice Problem	346
52.7. Challenge Problem	347
52.8. Key Takeaways	347
53. Detecting Duplicates	349
53.1. The R Concept	349
53.2. Demonstration	350
53.3. Explore	351
53.4. Solve the Problem	351
53.5. Interpret the Result	352
53.6. Practice Problem	352
53.7. Challenge Problem	353
53.8. Key Takeaways	353

Table of contents

54. Removing Duplicates	355
54.1. The R Concept	355
54.2. Demonstration	356
54.3. Explore	357
54.4. Solve the Problem	357
54.5. Interpret the Result	358
54.6. Practice Problem	358
54.7. Challenge Problem	359
54.8. Key Takeaways	359
55. Renaming Variables	361
55.1. The R Concept	361
55.2. Demonstration	362
55.3. Explore	363
55.4. Solve the Problem	363
55.5. Interpret the Result	363
55.6. Practice Problem	364
55.7. Challenge Problem	364
55.8. Key Takeaways	365
56. Changing Variable Types: Character to Numeric	367
56.1. The R Concept	368
56.2. Demonstration	368
56.3. Explore	369
56.4. Solve the Problem	369
56.5. Interpret the Result	370
56.6. Practice Problem	371
56.7. Challenge Problem	371
56.8. Key Takeaways	371
57. Changing Variable Types: Numeric to Character	373
57.1. The R Concept	373
57.2. Demonstration	374
57.3. Explore	375

Table of contents

57.4. Solve the Problem	375
57.5. Interpret the Result	376
57.6. Practice Problem	376
57.7. Challenge Problem	377
57.8. Key Takeaways	377
58. Changing Variable Types: Character to Factor	379
58.1. The R Concept	380
58.2. Demonstration	380
58.3. Explore	381
58.4. Solve the Problem	381
58.5. Interpret the Result	382
58.6. Practice Problem	383
58.7. Challenge Problem	383
58.8. Key Takeaways	383
59. Creating New Variables	385
59.1. The R Concept	385
59.2. Demonstration	386
59.3. Explore	387
59.4. Solve the Problem	387
59.5. Interpret the Result	388
59.6. Practice Problem	388
59.7. Challenge Problem	389
59.8. Key Takeaways	389
60. Recoding Variables	391
60.1. The R Concept	391
60.2. Demonstration	392
60.3. Explore	392
60.4. Solve the Problem	393
60.5. Interpret the Result	393
60.6. Practice Problem	394
60.7. Challenge Problem	394

Table of contents

60.8. Key Takeaways	394
61. Combining Categories	397
61.1. The R Concept	397
61.2. Demonstration	398
61.3. Explore	399
61.4. Solve the Problem	399
61.5. Interpret the Result	400
61.6. Practice Problem	400
61.7. Challenge Problem	401
61.8. Key Takeaways	401
62. Creating Binary Variables	403
62.1. The R Concept	403
62.2. Demonstration	404
62.3. Explore	405
62.4. Solve the Problem	405
62.5. Interpret the Result	406
62.6. Practice Problem	406
62.7. Challenge Problem	407
62.8. Key Takeaways	407
63. Conditional Statements: if	409
63.1. The R Concept	409
63.2. Demonstration	410
63.3. Explore	411
63.4. Solve the Problem	411
63.5. Interpret the Result	412
63.6. Practice Problem	412
63.7. Challenge Problem	413
63.8. Key Takeaways	413
64. ifelse()	415
64.1. The R Concept	415

Table of contents

64.2. Demonstration	416
64.3. Explore	417
64.4. Solve the Problem	417
64.5. Interpret the Result	417
64.6. Practice Problem	418
64.7. Challenge Problem	418
64.8. Key Takeaways	419
65. if_else()	421
65.1. The R Concept	421
65.2. Demonstration	422
65.3. Explore	423
65.4. Solve the Problem	423
65.5. Interpret the Result	424
65.6. Practice Problem	425
65.7. Challenge Problem	425
65.8. Key Takeaways	425
66. case_when()	427
66.1. The R Concept	427
66.2. Demonstration	428
66.3. Explore	429
66.4. Solve the Problem	430
66.5. Interpret the Result	430
66.6. Practice Problem	431
66.7. Challenge Problem	431
66.8. Key Takeaways	432
66.9. Where you've arrived	433
V. Level 5: Tidyverse	435
Level 5 — Tidyverse	437
What you'll build toward	437

Table of contents

Topics in this level	437
A preview mission	438
67.select()	439
67.1. The R Concept	439
67.2. Demonstration	440
67.3. Explore	441
67.4. Solve the Problem	441
67.5. Interpret the Result	442
67.6. Practice Problem	443
67.7. Challenge Problem	443
67.8. Key Takeaways	443
68.filter()	445
68.1. The R Concept	446
68.2. Demonstration	446
68.3. Explore	447
68.4. Solve the Problem	447
68.5. Interpret the Result	448
68.6. Practice Problem	448
68.7. Challenge Problem	449
68.8. Key Takeaways	449
69.arrange()	451
69.1. The R Concept	451
69.2. Demonstration	452
69.3. Explore	453
69.4. Solve the Problem	453
69.5. Interpret the Result	454
69.6. Practice Problem	454
69.7. Challenge Problem	455
69.8. Key Takeaways	455

70.mutate()	457
70.1. The R Concept	457
70.2. Demonstration	458
70.3. Explore	459
70.4. Solve the Problem	459
70.5. Interpret the Result	460
70.6. Practice Problem	461
70.7. Challenge Problem	461
70.8. Key Takeaways	461
71.summarise()	463
71.1. The R Concept	463
71.2. Demonstration	464
71.3. Explore	465
71.4. Solve the Problem	465
71.5. Interpret the Result	466
71.6. Practice Problem	466
71.7. Challenge Problem	466
71.8. Key Takeaways	467
72.group_by()	469
72.1. The R Concept	470
72.2. Demonstration	470
72.3. Explore	471
72.4. Solve the Problem	471
72.5. Interpret the Result	472
72.6. Practice Problem	473
72.7. Challenge Problem	473
72.8. Key Takeaways	473
73.count()	475
73.1. The R Concept	475
73.2. Demonstration	476
73.3. Explore	477

Table of contents

73.4. Solve the Problem	477
73.5. Interpret the Result	478
73.6. Practice Problem	479
73.7. Challenge Problem	479
73.8. Key Takeaways	479
74.distinct()	481
74.1. The R Concept	481
74.2. Demonstration	482
74.3. Explore	483
74.4. Solve the Problem	483
74.5. Interpret the Result	484
74.6. Practice Problem	484
74.7. Challenge Problem	485
74.8. Key Takeaways	485
75.rename()	487
75.1. The R Concept	487
75.2. Demonstration	488
75.3. Explore	488
75.4. Solve the Problem	489
75.5. Interpret the Result	489
75.6. Practice Problem	490
75.7. Challenge Problem	490
75.8. Key Takeaways	491
76.across()	493
76.1. The R Concept	494
76.2. Demonstration	494
76.3. Explore	495
76.4. Solve the Problem	495
76.5. Interpret the Result	495
76.6. Practice Problem	496
76.7. Challenge Problem	496

76.8. Key Takeaways	497
77. The Native Pipe >	499
77.1. The R Concept	499
77.2. Demonstration	500
77.3. Explore	501
77.4. Solve the Problem	501
77.5. Interpret the Result	502
77.6. Practice Problem	503
77.7. Challenge Problem	503
77.8. Key Takeaways	503
78. Chaining Operations	505
78.1. The R Concept	506
78.2. Demonstration	506
78.3. Explore	507
78.4. Solve the Problem	507
78.5. Interpret the Result	508
78.6. Practice Problem	509
78.7. Challenge Problem	509
78.8. Key Takeaways	510
VI. Level 6: Data Reshaping	513
Level 6 — Data Reshaping	515
What you'll build toward	515
Topics in this level	516
A preview mission	516
79. Wide Format vs. Long Format	517
79.1. The R Concept	517
79.2. Demonstration	518
79.3. Explore	519

Table of contents

79.4. Solve the Problem	519
79.5. Interpret the Result	520
79.6. Practice Problem	520
79.7. Challenge Problem	521
79.8. Key Takeaways	521
80.pivot_longer()	523
80.1. The R Concept	523
80.2. Demonstration	524
80.3. Explore	525
80.4. Solve the Problem	525
80.5. Interpret the Result	526
80.6. Practice Problem	527
80.7. Challenge Problem	528
80.8. Key Takeaways	528
81.pivot_wider()	531
81.1. The R Concept	531
81.2. Demonstration	532
81.3. Explore	534
81.4. Solve the Problem	534
81.5. Interpret the Result	535
81.6. Practice Problem	536
81.7. Challenge Problem	536
81.8. Key Takeaways	537
82.separate()	539
82.1. The R Concept	539
82.2. Demonstration	540
82.3. Explore	541
82.4. Solve the Problem	541
82.5. Interpret the Result	542
82.6. Practice Problem	542
82.7. Challenge Problem	543

82.8. Key Takeaways	543
83.unite()	545
83.1. The R Concept	545
83.2. Demonstration	546
83.3. Explore	547
83.4. Solve the Problem	547
83.5. Interpret the Result	548
83.6. Practice Problem	548
83.7. Challenge Problem	549
83.8. Key Takeaways	549
84.Pivoting Multiple Measurements at Once	551
84.1. The R Concept	552
84.2. Demonstration	552
84.3. Explore	553
84.4. Solve the Problem	554
84.5. Interpret the Result	555
84.6. Practice Problem	555
84.7. Challenge Problem	556
84.8. Key Takeaways	556
85.Reshaping Mini-Capstone: Change Across Visits	559
85.1. The R Concept	560
85.2. Demonstration	560
85.3. Explore	561
85.4. Solve the Problem	561
85.5. Interpret the Result	562
85.6. Practice Problem	563
85.7. Challenge Problem	563
85.8. Key Takeaways	564
86.Keys and Relationships	567
86.1. The R Concept	567

Table of contents

86.2. Demonstration	568
86.3. Explore	569
86.4. Solve the Problem	569
86.5. Interpret the Result	570
86.6. Practice Problem	571
86.7. Challenge Problem	571
86.8. Key Takeaways	571
87.left_join()	573
87.1. The R Concept	573
87.2. Demonstration	574
87.3. Explore	575
87.4. Solve the Problem	575
87.5. Interpret the Result	576
87.6. Practice Problem	577
87.7. Challenge Problem	577
87.8. Key Takeaways	577
88.inner_join()	579
88.1. The R Concept	579
88.2. Demonstration	580
88.3. Explore	581
88.4. Solve the Problem	581
88.5. Interpret the Result	582
88.6. Practice Problem	583
88.7. Challenge Problem	583
88.8. Key Takeaways	583
89.right_join() and full_join()	585
89.1. The R Concept	585
89.2. Demonstration	586
89.3. Explore	587
89.4. Solve the Problem	587
89.5. Interpret the Result	588

Table of contents

89.6. Practice Problem	589
89.7. Challenge Problem	589
89.8. Key Takeaways	589
90.semi_join() and anti_join()	591
90.1. The R Concept	592
90.2. Demonstration	592
90.3. Explore	593
90.4. Solve the Problem	593
90.5. Interpret the Result	594
90.6. Practice Problem	595
90.7. Challenge Problem	595
90.8. Key Takeaways	596
91.Level 6 Capstone: Reshaping and Joining Together	599
91.1. The R Concept	600
91.2. Demonstration	600
91.3. Explore	601
91.4. Solve the Problem	601
91.5. Interpret the Result	602
91.6. Practice Problem	603
91.7. Challenge Problem	604
91.8. Key Takeaways	604
VII.Level 7: Descriptive Statistics	607
Level 7 — Descriptive Statistics	609
What you'll build toward	609
Topics in this level	609
A preview mission	610
92.Mean, Median, and Mode	611
92.1. The R Concept	612

Table of contents

92.2. Demonstration	612
92.3. Explore	613
92.4. Solve the Problem	613
92.5. Interpret the Result	614
92.6. Practice Problem	615
92.7. Challenge Problem	615
92.8. Key Takeaways	616
93. Range, Variance, and Standard Deviation	619
93.1. The R Concept	620
93.2. Demonstration	620
93.3. Explore	621
93.4. Solve the Problem	621
93.5. Interpret the Result	622
93.6. Practice Problem	623
93.7. Challenge Problem	623
93.8. Key Takeaways	623
94. Quantiles and the Interquartile Range	627
94.1. The R Concept	628
94.2. Demonstration	628
94.3. Explore	629
94.4. Solve the Problem	629
94.5. Interpret the Result	630
94.6. Practice Problem	631
94.7. Challenge Problem	631
94.8. Key Takeaways	631
95. The summary() Function	633
95.1. The R Concept	633
95.2. Demonstration	634
95.3. Explore	634
95.4. Solve the Problem	634
95.5. Interpret the Result	635

Table of contents

95.6. Practice Problem	636
95.7. Challenge Problem	636
95.8. Key Takeaways	636
96. Frequency Tables and Proportions	639
96.1. The R Concept	639
96.2. Demonstration	640
96.3. Explore	641
96.4. Solve the Problem	641
96.5. Interpret the Result	642
96.6. Practice Problem	642
96.7. Challenge Problem	643
96.8. Key Takeaways	643
97. Cross-Tabulations	645
97.1. The R Concept	646
97.2. Demonstration	646
97.3. Explore	647
97.4. Solve the Problem	647
97.5. Interpret the Result	648
97.6. Practice Problem	649
97.7. Challenge Problem	649
97.8. Key Takeaways	649
98. Correlation with <code>cor()</code>	651
98.1. The R Concept	652
98.2. Demonstration	652
98.3. Explore	653
98.4. Solve the Problem	653
98.5. Interpret the Result	654
98.6. Practice Problem	655
98.7. Challenge Problem	655
98.8. Key Takeaways	655

Table of contents

99. Level 7 Capstone: A Descriptive Statistics Report	657
99.1. The R Concept	658
99.2. Demonstration	658
99.3. Explore	659
99.4. Solve the Problem	659
99.5. Interpret the Result	661
99.6. Practice Problem	662
99.7. Challenge Problem	662
99.8. Key Takeaways	662
 VIII Level 8: Data Visualization with ggplot2	 665
Level 8 — Data Visualization with ggplot2	667
What you'll build toward	667
Topics in this level	667
A preview mission	668
 100 The Grammar of Graphics	 669
100.1 The R Concept	669
100.2 Demonstration	670
100.3 Explore	670
100.4 Solve the Problem	671
100.5 Interpret the Result	671
100.6 Practice Problem	672
100.7 Challenge Problem	672
100.8 Key Takeaways	672
 101 Adding Trend Lines with geom_smooth()	 675
101.1 The R Concept	676
101.2 Demonstration	676
101.3 Explore	677
101.4 Solve the Problem	677
101.5 Interpret the Result	677

Table of contents

101.6Practice Problem	678
101.7Challenge Problem	678
101.8Key Takeaways	679
102Bar Charts with geom_bar()	681
102.1The R Concept	681
102.2Demonstration	682
102.3Explore	682
102.4Solve the Problem	683
102.5Interpret the Result	683
102.6Practice Problem	684
102.7Challenge Problem	684
102.8Key Takeaways	684
103Histograms and Density Plots	687
103.1The R Concept	687
103.2Demonstration	688
103.3Explore	688
103.4Solve the Problem	689
103.5Interpret the Result	689
103.6Practice Problem	690
103.7Challenge Problem	690
103.8Key Takeaways	691
104Boxplots with geom_boxplot()	693
104.1The R Concept	693
104.2Demonstration	694
104.3Explore	694
104.4Solve the Problem	695
104.5Interpret the Result	695
104.6Practice Problem	696
104.7Challenge Problem	696
104.8Key Takeaways	696

Table of contents

105	Line Plots with <code>geom_line()</code>	699
105.1	The R Concept	699
105.2	Demonstration	700
105.3	Explore	701
105.4	Solve the Problem	701
105.5	Interpret the Result	702
105.6	Practice Problem	703
105.7	Challenge Problem	703
105.8	Key Takeaways	703
106	Titles, Labels, and Themes	707
106.1	The R Concept	707
106.2	Demonstration	708
106.3	Explore	709
106.4	Solve the Problem	709
106.5	Interpret the Result	710
106.6	Practice Problem	710
106.7	Challenge Problem	711
106.8	Key Takeaways	711
107	Colors, Grouping, and Legends	713
107.1	The R Concept	714
107.2	Demonstration	714
107.3	Explore	715
107.4	Solve the Problem	715
107.5	Interpret the Result	716
107.6	Practice Problem	716
107.7	Challenge Problem	717
107.8	Key Takeaways	717
108	Faceting into Multiple Panels	719
108.1	The R Concept	719
108.2	Demonstration	720
108.3	Explore	720

Table of contents

108.4Solve the Problem	721
108.5Interpret the Result	721
108.6Practice Problem	722
108.7Challenge Problem	722
108.8Key Takeaways	723
109Saving Plots with ggsave()	725
109.1The R Concept	725
109.2Demonstration	726
109.3Explore	727
109.4Solve the Problem	727
109.5Interpret the Result	728
109.6Practice Problem	728
109.7Challenge Problem	729
109.8Key Takeaways	729
110Level 8 Capstone: One Figure, Two Variables, One Story	731
110.1The R Concept	732
110.2Demonstration	732
110.3Explore	733
110.4Solve the Problem	733
110.5Interpret the Result	734
110.6Practice Problem	735
110.7Challenge Problem	735
110.8Key Takeaways	735
IX. Level 9: Functions and Programming	737
Level 9 — Functions and Programming	739
What you'll build toward	739
Topics in this level	739
A preview mission	740

Table of contents

111What Is a Function?	741
111.1The R Concept	741
111.2Demonstration	742
111.3Explore	743
111.4Solve the Problem	743
111.5Interpret the Result	744
111.6Practice Problem	744
111.7Challenge Problem	745
111.8Key Takeaways	745
112Writing Your First Function	747
112.1The R Concept	747
112.2Demonstration	748
112.3Explore	749
112.4Solve the Problem	749
112.5Interpret the Result	750
112.6Practice Problem	751
112.7Challenge Problem	751
112.8Key Takeaways	751
113Default Argument Values	753
113.1The R Concept	753
113.2Demonstration	754
113.3Explore	755
113.4Solve the Problem	755
113.5Interpret the Result	756
113.6Practice Problem	757
113.7Challenge Problem	757
113.8Key Takeaways	757
114Conditional Logic Inside Functions	759
114.1The R Concept	760
114.2Demonstration	761
114.3Explore	761

Table of contents

114.4Solve the Problem	762
114.5Interpret the Result	763
114.6Practice Problem	763
114.7Challenge Problem	764
114.8Key Takeaways	764
115Functions with Multiple Arguments	767
115.1The R Concept	768
115.2Demonstration	768
115.3Explore	769
115.4Solve the Problem	769
115.5Interpret the Result	770
115.6Practice Problem	771
115.7Challenge Problem	771
115.8Key Takeaways	771
116Variable Scope	773
116.1The R Concept	773
116.2Demonstration	774
116.3Explore	775
116.4Solve the Problem	775
116.5Interpret the Result	776
116.6Practice Problem	777
116.7Challenge Problem	777
116.8Key Takeaways	778
117Anonymous Functions	781
117.1The R Concept	781
117.2Demonstration	782
117.3Explore	783
117.4Solve the Problem	783
117.5Interpret the Result	784
117.6Practice Problem	784
117.7Challenge Problem	785

Table of contents

117.8Key Takeaways	785
118Applying a Function Across Columns: <code>sapply()</code> and <code>lapply()</code>	787
118.1The R Concept	788
118.2Demonstration	788
118.3Explore	789
118.4Solve the Problem	789
118.5Interpret the Result	790
118.6Practice Problem	791
118.7Challenge Problem	791
118.8Key Takeaways	792
119The Tidyverse Way: <code>purrr::map()</code>	795
119.1The R Concept	796
119.2Demonstration	796
119.3Explore	797
119.4Solve the Problem	797
119.5Interpret the Result	798
119.6Practice Problem	799
119.7Challenge Problem	799
119.8Key Takeaways	799
120Basic Error Handling: <code>stop()</code> and <code>warning()</code>	803
120.1The R Concept	804
120.2Demonstration	804
120.3Explore	805
120.4Solve the Problem	805
120.5Interpret the Result	806
120.6Practice Problem	807
120.7Challenge Problem	807
120.8Key Takeaways	807
121Level 9 Capstone: A Complete Data Quality Report	811
121.1The R Concept	812

121.2	Demonstration	812
121.3	Explore	813
121.4	Solve the Problem	814
121.5	Interpret the Result	815
121.6	Practice Problem	816
121.7	Challenge Problem	816
121.8	Key Takeaways	817
X.	Level 10: Loops and Automation	819
	Level 10 — Loops and Automation	821
	What you'll build toward	821
	Topics in this level	821
	A preview mission	822
122	for Loops	823
122.1	The R Concept	823
122.2	Demonstration	824
122.3	Explore	824
122.4	Solve the Problem	825
122.5	Interpret the Result	825
122.6	Practice Problem	826
122.7	Challenge Problem	826
122.8	Key Takeaways	827
123	Accumulating Results in a Loop	829
123.1	The R Concept	829
123.2	Demonstration	830
123.3	Explore	831
123.4	Solve the Problem	831
123.5	Interpret the Result	832
123.6	Practice Problem	832
123.7	Challenge Problem	833

Table of contents

123.8Key Takeaways	833
124while Loops	835
124.1The R Concept	836
124.2Demonstration	836
124.3Explore	837
124.4Solve the Problem	837
124.5Interpret the Result	838
124.6Practice Problem	839
124.7Challenge Problem	839
124.8Key Takeaways	839
125Looping Over a Data Frame's Columns	841
125.1The R Concept	841
125.2Demonstration	842
125.3Explore	843
125.4Solve the Problem	843
125.5Interpret the Result	844
125.6Practice Problem	844
125.7Challenge Problem	845
125.8Key Takeaways	845
126Nested Loops	847
126.1The R Concept	847
126.2Demonstration	848
126.3Explore	849
126.4Solve the Problem	849
126.5Interpret the Result	850
126.6Practice Problem	851
126.7Challenge Problem	851
126.8Key Takeaways	851
127vapply() and tapply()	853
127.1The R Concept	854

Table of contents

127.2	Demonstration	854
127.3	Explore	855
127.4	Solve the Problem	855
127.5	Interpret the Result	856
127.6	Practice Problem	857
127.7	Challenge Problem	857
127.8	Key Takeaways	858
128	When to Loop, and When to Vectorize	861
128.1	The R Concept	862
128.2	Demonstration	862
128.3	Explore	863
128.4	Solve the Problem	863
128.5	Interpret the Result	864
128.6	Practice Problem	865
128.7	Challenge Problem	865
128.8	Key Takeaways	865
129	Level 10 Capstone: Automating Five Clinic Files	869
129.1	The R Concept	870
129.2	Demonstration	870
129.3	Explore	871
129.4	Solve the Problem	871
129.5	Interpret the Result	872
129.6	Practice Problem	873
129.7	Challenge Problem	873
129.8	Key Takeaways	873
XI	Level 11: Statistical Analysis in R	875
Level 11	— Statistical Analysis in R	877
	What you'll build toward	877
	Topics in this level	877

Table of contents

A preview mission	878
130Probability and the Normal Distribution	879
130.1The R Concept	880
130.2Demonstration	880
130.3Explore	881
130.4Solve the Problem	881
130.5Interpret the Result	882
130.6Practice Problem	882
130.7Challenge Problem	883
130.8Key Takeaways	883
131The Central Limit Theorem	885
131.1The R Concept	886
131.2Demonstration	886
131.3Explore	887
131.4Solve the Problem	887
131.5Interpret the Result	888
131.6Practice Problem	889
131.7Challenge Problem	889
131.8Key Takeaways	889
132Confidence Intervals	893
132.1The R Concept	893
132.2Demonstration	894
132.3Explore	895
132.4Solve the Problem	895
132.5Interpret the Result	896
132.6Practice Problem	897
132.7Challenge Problem	897
132.8Key Takeaways	897
133The One-Sample t-Test	899
133.1The R Concept	899

133.2	Demonstration	900
133.3	Explore	900
133.4	Solve the Problem	901
133.5	Interpret the Result	901
133.6	Practice Problem	902
133.7	Challenge Problem	902
133.8	Key Takeaways	903
134	The Two-Sample t-Test	905
134.1	The R Concept	906
134.2	Demonstration	906
134.3	Explore	907
134.4	Solve the Problem	907
134.5	Interpret the Result	908
134.6	Practice Problem	909
134.7	Challenge Problem	909
134.8	Key Takeaways	909
135	The Paired t-Test	911
135.1	The R Concept	912
135.2	Demonstration	912
135.3	Explore	913
135.4	Solve the Problem	913
135.5	Interpret the Result	914
135.6	Practice Problem	915
135.7	Challenge Problem	915
135.8	Key Takeaways	915
136	The Chi-Square Test of Independence	917
136.1	The R Concept	918
136.2	Demonstration	918
136.3	Explore	919
136.4	Solve the Problem	919
136.5	Interpret the Result	920

Table of contents

136.6Practice Problem	921
136.7Challenge Problem	921
136.8Key Takeaways	921
137Testing a Correlation for Significance	923
137.1The R Concept	924
137.2Demonstration	924
137.3Explore	924
137.4Solve the Problem	925
137.5Interpret the Result	926
137.6Practice Problem	926
137.7Challenge Problem	927
137.8Key Takeaways	927
138Simple Linear Regression with <code>lm()</code>	929
138.1The R Concept	930
138.2Demonstration	930
138.3Explore	931
138.4Solve the Problem	931
138.5Interpret the Result	932
138.6Practice Problem	933
138.7Challenge Problem	933
138.8Key Takeaways	933
139One-Way ANOVA	935
139.1The R Concept	936
139.2Demonstration	936
139.3Explore	936
139.4Solve the Problem	937
139.5Interpret the Result	937
139.6Practice Problem	938
139.7Challenge Problem	939
139.8Key Takeaways	939

140 Logistic Regression for a Binary Outcome	941
140.1 The R Concept	942
140.2 Demonstration	942
140.3 Explore	943
140.4 Solve the Problem	943
140.5 Interpret the Result	944
140.6 Practice Problem	945
140.7 Challenge Problem	945
140.8 Key Takeaways	945
 141 Level 11 Capstone: Testing This Course's Earlier Findings	 949
141.1 The R Concept	949
141.2 Demonstration	950
141.3 Explore	951
141.4 Solve the Problem	951
141.5 Interpret the Result	952
141.6 Practice Problem	953
141.7 Challenge Problem	953
141.8 Key Takeaways	954
 XII. Level 12 — Advanced Data Analysis	 957
 Level 12 — Advanced Data Analysis	 959
What this level covers	959
 XIII Level 13 — Advanced R Programming	 961
 Level 13 — Advanced R Programming	 963
What's coming	963

Table of contents

XIV	Level 14 — Reproducible Research	965
	Level 14 — Reproducible Research	967
	What's coming	967
XV	Level 15 — Real-World Capstone Projects	969
	Level 15 — Real-World Capstone Projects	971
	What's coming	971

Welcome

Why this course exists

Most R textbooks teach syntax first and hope the motivation follows. This course does the opposite. Every page opens with a problem you might actually be handed as a public health intern, a research assistant, an epidemiology student, or a data analyst on a health department team — and you learn exactly the R you need, at the moment you need it, to solve it.

You will not find a chapter that starts with “Today we will learn the `filter()` function.” Instead you will find a supervisor asking you to find every participant over 65 with hypertension, and `filter()` will show up as the tool that gets you out of that jam.

Who this is for

This course assumes **zero prior programming experience**. If you have never written a line of code in your life but you can think through a problem logically, you are the target reader. It is built for students and professionals in:

Public health, epidemiology, biostatistics, clinical research, health data analysis, survey research, and population health.

Welcome

By the final capstone level, the material assumes you have worked through everything before it — the course is cumulative, not a reference manual to jump around in on your first pass.

How each lesson works

Every lesson in this book follows the same rhythm, because the rhythm itself is what builds problem-solving skill:

1. **Real-World Problem** — a short, realistic scenario from health research.
2. **Your Mission** — the concrete task you need to accomplish.
3. **Think Before Coding** — a few questions to reason through before touching the keyboard.
4. **The R Concept** — the one new idea you need, explained plainly: what it does, its syntax, its arguments, and what usually goes wrong.
5. **Demonstration** — runnable R code showing the concept in isolation.
6. **Explore** — small modifications you make to that code to test your understanding.
7. **Solve the Problem** — you return to the opening scenario and solve it completely.
8. **Interpret the Result** — the output translated into a plain-language sentence a supervisor or reader would understand.
9. **Common Mistakes** — errors beginners reliably make with this concept, and why they happen.
10. **Practice Problem** — a new, similar problem for you to solve with support.

11. Challenge Problem — a harder variant you solve with less scaffolding.

12. Key Takeaways — two to four sentences worth remembering.

13. Knowledge Check — multiple-choice, true/false, predict-the-output, find-the-error, and write-the-code questions, with answers hidden behind a toggle so you can self-test honestly.

The dataset you'll live in

Rather than switching example datasets every page, most of this course reuses a single running dataset: a simulated **community health survey of 100 participants**, with age, sex, county, systolic and diastolic blood pressure, BMI, and smoking status — including realistic missing values. You'll meet it for the first time in Level 1 and return to it constantly, so that learning R and learning to think about a real dataset happen together. The file lives at `data/participants.csv`.

By Level 15 you'll be running full analyses — and eventually independent capstone projects — on data structured just like it.

Course map

Level	Focus
1	Getting started: R, RStudio, objects, basic types
2	Data structures: vectors, data frames, lists, tibbles
3	Importing and exporting real data files
4	Cleaning messy data in base R
5	The tidyverse: <code>dplyr</code> , pipes, and friends
6	Reshaping and joining datasets

Welcome

Level	Focus
7	Descriptive statistics
8	Data visualization with <code>ggplot2</code>
9	Writing functions and <code>purrr</code>
10	Loops and automation
11	Statistical analysis: t-tests through survival models
12	Advanced analysis: confounding, mixed models, surveys
13	Advanced R programming: OOP, debugging, packages
14	Reproducible research with Quarto
15	Capstone research projects, start to finish

What you'll be able to do

By the end, starting from zero, you will be able to navigate R and RStudio, write and debug your own R code, understand R's core data structures, import and clean real-world health data, manipulate data fluently with the tidyverse, build publication-quality figures, write your own functions and automate repetitive analysis, run and interpret the statistical methods most common in health research, work with longitudinal, survey, and time-to-event data, and produce a fully reproducible research report from raw data to final results — in Quarto, from scratch.

Let's start with the question every beginner has: what is R, actually, and why are we bothering?

Part I.

Level 1: Getting Started with R

Level 1 — Getting Started with R

Where this level is headed

You’ve just started an internship, a course, or a new role that expects you to “just use R” for the data work — and you’ve never written a line of code. This level takes you from a completely blank slate to confidently running your first real analysis: computing the average age of 100 study participants from an actual survey file.

What you’ll build toward

Every page in this level adds one small piece, and by the last page you’ll combine all of them to solve the same running problem: **your supervisor needs the average age of the participants in a community health survey, and you need to get that number out of R.**

Along the way you’ll learn what R and RStudio actually are, how to work in the console and in scripts, how R represents numbers, text, true/false values, and missing data, how to use built-in functions, how to get help when you’re stuck, and how to import a real data file.

Topics in this level

- What is R? What is RStudio?
- The RStudio interface

Level 1 — Getting Started with R

- Installing and loading packages
- The Console vs. an R script
- Comments
- Basic arithmetic
- The assignment operator and objects
- Naming objects well
- Data types: numeric, character, logical
- Missing values (NA)
- Basic functions
- Getting help in R
- Working directories
- Importing your first real dataset

By the end of this level, you'll import `data/participants.csv` — a simulated community health survey — and report the mean age of its 100 participants, correctly handling the missing values hiding in it. Let's begin.

1. What Is R?

The Problem

You’ve just started as a public health intern at your local health department. On your first day, your supervisor says: “We collected survey data from 100 participants in our community health study. I need the average age reported by Friday. Everyone here uses R for this — go ahead and get started.”

You’ve used Excel before. You’ve never heard of R. You don’t even know if it’s software you install, a website, or a programming language.

Your Mission

Understand what R actually is, why a health department would use it instead of Excel, and what it means to “write code” to answer a data question.

Think Before Coding

- When you’ve calculated an average before (in Excel, or by hand), what steps did you follow?
- What happens in Excel if you need to repeat the exact same calculation on next month’s data — do you redo all your clicks?
- What might be an advantage of *writing down* the steps of a calculation, rather than just clicking through a menu?

1. What Is R?

1.1. The R Concept

R is a free, open-source programming language built specifically for statistics, data analysis, and graphics. It was created by statisticians, for statisticians — which is why it’s the default choice in public health, epidemiology, biostatistics, and academic research more broadly.

A few things make R different from a spreadsheet:

- **You write instructions, not clicks.** Instead of selecting a range of cells and clicking “AVERAGE,” you write a line of code that says, in effect, “take this data and compute its mean.” That line can be saved, reused, and rerun on next month’s data in seconds.
- **It’s reproducible.** Anyone with your code and your data can get the exact same result. A colleague reviewing your Excel file has to guess which cells you clicked; a colleague reading your R script can see exactly what you did.
- **It’s built for data, not documents.** R has no concept of a “sheet” with 500 unrelated things on it. It’s designed around clean, structured data and the operations researchers actually need: summarizing, filtering, modeling, plotting.
- **It’s free and extensible.** R itself is free, and thousands of free add-ons (called *packages*) extend it — for reading messy Excel files, running survival analysis, mapping disease outbreaks, and far more.

R is not the app with the blue icon you click to open. That’s a separate program called **RStudio**, which you’ll meet on the next page. R is the *language and engine underneath* — RStudio is the workspace you’ll actually sit in.

1.2. Demonstration

Every R session, however small, starts with running code. Let's run the simplest possible line: basic arithmetic.

```
# R can act as a calculator right out of the box  
2 + 2
```

```
[1] 4
```

That `[1] 4` is R's answer. The `[1]` just means “this is result number 1” — you'll see why that notation matters once you work with longer lists of numbers.

Here's a second line, doing something closer to what your supervisor actually wants: averaging a few numbers.

```
# The average of four ages, computed directly  
(34 + 51 + 62 + 29) / 4
```

```
[1] 44
```

1.3. Explore

Try changing the numbers in the calculation above to four different ages. Then try adding a fifth age to the sum — remember to also change the number you divide by. What happens if you forget to update the divisor?

1. What Is R?

1.4. Solve the Problem

You don't yet have the tools to import your supervisor's actual 100-participant file — that's coming in a later page. But you can already answer the *conceptual* question your supervisor implicitly asked: **is R the right tool for this, and what will doing this work in R actually look like?**

Yes — R is exactly the right tool. Once your data is loaded into R (Level 1, later pages), computing a mean will take a single line of code, `mean()`, no matter how many participants there are. Whether it's 100 or 100,000 rows, the code doesn't get any harder.

1.5. Interpret the Result

When you ran `2 + 2` and got `[1] 4`, you weren't just testing a calculator — you confirmed that R is installed, running, and ready to take instructions. That's the same confirmation a much bigger analysis needs: R evaluates your code and hands back an answer, every time, the same way.

Common Mistakes

- **Confusing R and RStudio.** People often say “open R” when they mean “open RStudio.” You'll always work inside RStudio, which runs R for you behind the scenes.
- **Assuming R is only for expert statisticians.** R is used by beginners and experts alike — the language scales with you.
- **Expecting a menu-driven experience.** If you're used to Excel's toolbar, the biggest adjustment is trusting that a line of text *is* the instruction, not a shortcut to one.

1.6. Practice Problem

List three tasks in your own field (or one you're interested in — clinical research, survey analysis, health policy) where you currently rely on Excel or manual calculation, and that could instead be done with reusable, written instructions.

1.7. Challenge Problem

Write two or three sentences, in your own words, explaining to a classmate who has never coded why a health department might choose R over Excel for analyzing a 10,000-row patient dataset. Be specific about at least one concrete advantage.

1.8. Key Takeaways

- R is a free, open-source programming language built for statistics and data analysis.
- Working in R means *writing instructions* rather than clicking through menus — which makes your work reusable and reproducible.
- R and RStudio are not the same thing; R is the language, RStudio is the workspace (next page).

Knowledge Check

1. **Multiple choice.** R is best described as:

- a) A spreadsheet program
- b) A programming language for statistics and data analysis
- c) A database management system

1. *What Is R?*

d) A word processor

2. **True or false.** R and RStudio are two names for the exact same program.

3. **Predict the output.** What will the following code print?

```
10 * 3 - 5
```

4. **Write the code.** Write one line of R code that calculates the average of the numbers 12, 18, and 24 without using any function — just arithmetic.

Show answers

1. **b)** A programming language for statistics and data analysis.
2. **False.** R is the underlying language; RStudio is a separate application (an IDE) that makes R easier to use.
3. `[1] 25` ($10 \times 3 = 30$, then $30 - 5 = 25$).
4. `(12 + 18 + 24) / 3`, which evaluates to `[1] 18`.

2. What Is RStudio?

The Problem

You’ve installed R on your laptop, as instructed. When you open it, a plain, small window appears — no toolbars, no file browser, no obvious way to see the data you’re working with. Meanwhile, your classmates all have a colorful window with four organized sections. They tell you: “You need RStudio too — nobody actually works in plain R.”

Your Mission

Understand what RStudio is, how it relates to R, and why almost every R user works through it instead of through R’s bare-bones window.

Think Before Coding

- If R is the “engine” that runs your code, what job might a separate application be doing around that engine?
- What features would make it easier to write, organize, and rerun code — beyond just a blank box to type into?

2.1. The R Concept

RStudio is an *Integrated Development Environment* (IDE) for R. That’s a general term for an application that wraps a programming language in helpful tools: a place to write and organize code, a way to see your data and objects, a place plots appear, and a way to browse files — all without leaving the app.

2. What Is RStudio?

Concretely, RStudio gives you:

- A **Source** pane, where you write and save scripts (reusable files of R code).
- A **Console** pane, where code actually runs and results appear.
- An **Environment** pane, listing every object you’ve created (like a running “what’s currently loaded” panel).
- A combined **Files / Plots / Packages / Help / Viewer** pane, for browsing your project folder, viewing charts, managing installed add-ons, and reading documentation.

You’ll tour each of these panes in detail on the next page. For now, the key idea is: **R does the computing; RStudio gives you a workspace to do that computing in.** Installing R alone gives you the engine with no dashboard. Installing RStudio (in addition to R, not instead of it) gives you the dashboard, the windows, and the seatbelt.

i A note on Posit

RStudio is made by a company called Posit. You may see “RStudio” and “Posit” used somewhat interchangeably in newer documentation — they refer to the same IDE.

2.2. Demonstration

Once RStudio is open, you can confirm both R and RStudio are working by asking R to report its own version.

```
# Ask R which version of itself is running
R.version.string
```

```
[1] "R version 4.4.1 (2024-06-14)"
```


2.3. Explore

You can also check which packages come built into R by default without installing anything extra.

```
# A quick built-in function - more on functions later in this level  
getRversion()
```

```
[1] '4.4.1'
```

2.3. Explore

If RStudio is installed on your machine, open it and locate the version number yourself: click **Help > About RStudio** in the menu bar. Compare that to the R version reported by the code above — notice they’re reported separately, because they’re genuinely two different pieces of software with two different version numbers.

2.4. Solve the Problem

Your classmates were right: you need both pieces of software, installed separately, working together.

1. **R** is installed first — it’s the calculation engine, downloaded from CRAN (the Comprehensive R Archive Network).
2. **RStudio** is installed second — it’s the application you actually open day-to-day, and it automatically finds and uses the R installation on your machine.

From this point in the course forward, “opening R” always means opening RStudio.

2. What Is RStudio?

2.5. Interpret the Result

Seeing `R.version.string` print a real version number confirms that RStudio has successfully connected to an R installation on your computer. If that line had produced an error instead, it would mean R itself wasn't installed correctly — a problem to fix before writing any further code.

Common Mistakes

- **Installing only RStudio.** RStudio is not R — without R installed underneath it, RStudio has no engine to run and will show an error on startup.
- **Installing them in the wrong order.** Install R first, then RStudio; RStudio looks for an existing R installation when it starts.
- **Expecting to “code in RStudio.”** You write code in RStudio, but RStudio hands that code to R to actually execute — RStudio itself doesn't understand R code on its own.

2.6. Practice Problem

Without running any code, describe in one sentence the difference between what R does and what RStudio does — as if explaining it to a classmate who just asked “wait, aren't those the same thing?”

2.7. Challenge Problem

Look up (or recall) one other IDE used for a different programming language (for example, PyCharm for Python, or VS Code for many languages). Name one feature it likely shares with RStudio, based on what you now know an IDE provides.

2.8. Key Takeaways

- RStudio is an IDE — a workspace built around R, not a replacement for R.
- You must install R first, then RStudio, in that order.
- From here on, “using R” in this course means working inside RStudio.

Knowledge Check

1. **Multiple choice.** RStudio is:
 - a) A different programming language than R
 - b) An IDE that runs on top of R
 - c) A cloud storage service
 - d) A type of R package
2. **True or false.** You can run RStudio without ever installing R.
3. **Find the error.** A student says: “I installed RStudio, so I don’t need R.” What’s wrong with this statement?
4. **Write the code.** Write the line of R code from this page that asks R to report its own version as a string.

Show answers

1. **b)** An IDE that runs on top of R.
2. **False.** RStudio requires a working R installation underneath it; it has no engine of its own.
3. RStudio needs R installed separately — it’s the interface, not the engine. Without R, RStudio has nothing to run your code with.
4. `R.version.string`

3. The RStudio Interface

The Problem

You open RStudio for the first time. There are four rectangular sections on your screen, each full of tabs, buttons, and small icons you don't recognize. Your supervisor mentions "check the Environment pane" and "look in the Console" as if these are obvious places — but you can't tell which section is which.

Your Mission

Identify the four main panes in RStudio and know which one to use for which task, so instructions like "check the Environment pane" stop being confusing.

Think Before Coding

- If you were designing a coding workspace from scratch, what separate "zones" would you want: one for writing, one for running, one for seeing results, one for managing files?
- Which of those zones do you think should be permanent and savable, and which should be temporary?

3.1. The R Concept

RStudio's default layout has four panes, usually arranged as four quadrants:

3. The RStudio Interface

1. **Source pane** (top-left) — where you write and edit R scripts (`.R` files) and Quarto documents (`.qmd` files, like the one you’re reading now was written in). Code here is *saved* and *reusable*. If this pane isn’t visible, it’s because no file is currently open.
2. **Console pane** (bottom-left, or left side if no file is open) — where code actually *executes*. You can type a line directly here and press Enter to run it immediately, but nothing here is saved automatically.
3. **Environment / History pane** (top-right) — the Environment tab lists every object currently loaded in your session (data you’ve imported, values you’ve calculated). The History tab logs every command you’ve run.
4. **Files / Plots / Packages / Help / Viewer pane** (bottom-right) — a multi-purpose pane: **Files** browses your project folder, **Plots** displays any chart you create, **Packages** lists installed add-ons, **Help** shows documentation, and **Viewer** displays rendered HTML output.

The distinction that trips up beginners most: **the Source pane is for permanent work, and the Console is for temporary work**. Anything you want to keep — and reuse tomorrow — belongs in a script in the Source pane, not typed directly into the Console.

3.2. Demonstration

You don’t need special code to “see” panes — but you can generate something in each of the three panes that respond to code, to connect the concept to something concrete.

```
# Running this creates an object -> it will appear in the Environment pane
intern_name <- "Jordan"
```

```
# Running this prints text -> the output appears in the Console pane
print(intern_name)
```

3.3. Explore

```
[1] "Jordan"
```

If you ran that code inside a script in the Source pane (rather than typing directly in the Console), you would see: the code itself remains visible and saved in the Source pane, the printed result `[1] "Jordan"` appears in the Console, and `intern_name` appears listed in the Environment pane with its current value.

3.3. Explore

If you have RStudio open, run the two lines above and locate `intern_name` in your Environment pane. Then click the **History** tab (next to Environment) and confirm the same line of code is logged there.

3.4. Solve the Problem

Now “check the Environment pane” and “look in the Console” have concrete meanings:

- “Check the Environment pane” means: look at the top-right section, which lists `intern_name` (and every other object you’ve created).
- “Look in the Console” means: look at the pane where your code’s printed output — like `[1] "Jordan"` — actually appears.

You now know exactly where to look for each instruction your supervisor gives you.

3. The RStudio Interface

3.5. Interpret the Result

Seeing `intern_name` listed in the Environment pane after running the code confirms the object was successfully created and is currently available for you to use in later code — the Environment pane is effectively a live inventory of everything you’ve built so far in your session.

Common Mistakes

- **Doing all work in the Console and never saving a script.** Anything typed only in the Console vanishes when you close RStudio (or is easy to lose track of in the History). Important work belongs in a saved script.
- **Confusing the Environment pane with your saved files.** The Environment pane shows what’s currently loaded in memory — closing RStudio without saving clears it, even though your script file itself is unaffected.
- **Not knowing where plots go.** New users often expect a plot to appear in the Console; it will instead appear in the Plots tab of the bottom-right pane.

3.6. Practice Problem

Without running any code, match each task to the pane that handles it: (1) writing a reusable analysis script, (2) viewing a bar chart you just created, (3) seeing a list of the data objects you’ve loaded so far, (4) reading the documentation for a function.

3.7. Challenge Problem

Explain, in your own words, why “type your entire analysis directly into the Console and never save a script” is a risky habit for someone doing

3.8. Key Takeaways

public health research where results may need to be reproduced or audited later.

3.8. Key Takeaways

- RStudio has four main panes: Source (write/save code), Console (run code), Environment/History (see objects and command history), and Files/Plots/Packages/Help/Viewer (browse, view, and reference).
- Permanent work belongs in the Source pane as a saved script; the Console is for quick, temporary checks.
- The Environment pane is your live inventory of everything currently loaded in your session.

Knowledge Check

1. **Multiple choice.** Where would you look to see a chart you just created?
 - a) Source pane
 - b) Console pane
 - c) Plots tab
 - d) History tab
2. **True or false.** Code typed only into the Console is automatically saved when you close RStudio.
3. **Find the error.** A student writes their entire semester's analysis directly in the Console, never opening a script file. What problem will they eventually run into?
4. **Write the code.** Write one line of R code that would create an object called `city` holding the text "Riverdale", which would then appear in the Environment pane.

3. *The RStudio Interface*

Show answers

1. **c)** The Plots tab, in the bottom-right pane.
2. **False.** Console work is not saved automatically; only scripts saved in the Source pane persist as files.
3. They will lose their work (or lose track of exactly what they ran) as soon as RStudio closes or the History becomes too long to search — with no reusable, reviewable record of their analysis.
4. `city <- "Riverdale"`

4. Installing and Loading Packages

The Problem

Your supervisor emails you an Excel file (`.xlsx`) containing the community health survey data, not a CSV. You try to open it in R the way you’ve seen in a tutorial, and you get an error: `could not find function "read_excel"`. The tutorial clearly used that exact function.

Your Mission

Understand what an R package is, and install and load one so its functions become available to you.

Think Before Coding

- If `read_excel` isn’t a function R comes with automatically, where might it come from?
- Have you ever used an app that required you to download a separate “add-on” or “extension” before a feature worked? How is that similar to what might be happening here?

4.1. The R Concept

Base R — what you get immediately after installing R — includes a solid set of built-in functions, but it doesn’t include everything. **Packages** are add-on collections of functions, data, and documentation that extend what R can do. Anyone can write and publish one; most official packages are hosted on **CRAN** (the Comprehensive R Archive Network).

4. Installing and Loading Packages

Working with a package is a two-step process, and beginners often only do one of the two steps:

1. **Install it — once per computer.**

```
install.packages("readxl")
```

This downloads the package and stores it on your machine permanently. You do not need to repeat this every time you use R — only when installing for the first time, or updating later.

2. **Load it — once per R session.**

```
library(readxl)
```

This tells your *current* R session “make this package’s functions available right now.” Every time you close and reopen RStudio, you need to load the package again with `library()` — but you do **not** need to reinstall it.

Notice the quoting difference: `install.packages()` requires the package name in quotes (`"readxl"`), because it’s just a piece of text identifying what to download. `library()` conventionally doesn’t require quotes, though quoting it also works.

4.2. Demonstration

```
# Step 1: install the package (only needs to be run once, ever, per computer)
install.packages("readxl")
```

```
# Step 2: load the package (needs to be run once per R session)
library(readxl)
```

Once loaded, functions from that package — like `read_excel()` — become available exactly as if they were built into R.

4.3. Explore

```
# Now this function exists and is ready to use
survey_data <- read_excel("data/survey.xlsx")
```

You can check which packages are already installed on your machine without loading anything:

```
# Lists every package installed locally (long output - often dozens or hundreds of rows)
installed.packages()[, "Package"] |> head()
```

4.3. Explore

What do you think happens if you run `library(readxl)` *without* ever running `install.packages("readxl")` first, on a computer where it's never been installed? Try to predict the error message before you test it — it will say something to the effect of “there is no package called ‘readxl’”

4.4. Solve the Problem

The could not find function “`read_excel`” error happened because the tutorial’s code assumed the `readxl` package was already loaded — but loading it requires two things to have happened first: the package must be *installed* on your computer, and `library()` must have been *run in your current session*. Since you’d done neither, R had no idea what `read_excel` meant.

The fix:

```
install.packages("readxl") # only needed once
library(readxl)             # needed every session
survey_data <- read_excel("data/survey.xlsx")
```

4. Installing and Loading Packages

4.5. Interpret the Result

Once `library(readxl)` runs without error, it means the package's functions — including `read_excel()` — are now available in your current session. If `install.packages()` had failed (for example, due to no internet connection), `library()` would fail too, since there'd be nothing installed to load.

Common Mistakes

- **Forgetting to load the package each new session.** Installing is permanent; loading is not. A fresh RStudio session needs `library()` again even if you installed the package last week.
- **Reinstalling every time.** Running `install.packages()` at the top of every script is unnecessary and slow — install once, then just `library()` in future sessions.
- **Quoting inconsistency.** Forgetting the quotes in `install.packages(readxl)` (missing quotes) causes an error, since R tries to interpret `readxl` as an object rather than a package name.
- **Typos in package names.** `install.packages("read_xl")` will fail silently with a “package not available” message if the actual package is spelled `readxl` (no underscore).

4.6. Practice Problem

Write the two lines of code needed to install and then load a package called `janitor` (a real, commonly used package for cleaning messy data), assuming it has never been installed on this computer before.

4.7. Challenge Problem

A classmate says their script works perfectly on their own laptop but produces `could not find function` errors on yours, even though you're both running the identical script. List two possible explanations related to what you learned on this page, and how you'd check each one.

4.8. Key Takeaways

- Packages extend R with additional functions, data, and tools beyond what comes built in.
- `install.packages("name")` downloads a package once per computer; `library(name)` loads it once per session.
- Forgetting `library()` — not forgetting to reinstall — is the far more common cause of “could not find function” errors in day-to-day work.

Knowledge Check

1. **Multiple choice.** Which line only needs to be run once, ever, on a given computer?
 - a) `library(readxl)`
 - b) `install.packages("readxl")`
 - c) Both need to run every session
 - d) Neither needs quotes
2. **True or false.** If you restart RStudio, previously installed packages must be reinstalled before you can use them again.
3. **Find the error.**

```
install.packages(readxl)
```

4. *Installing and Loading Packages*

4. **Write the code.** Write the two lines needed to install and load a package called `lubridate`.

Show answers

1. **b)** `install.packages("readxl")` — installation is a one-time, per-computer action.
2. **False.** Installed packages remain installed; you only need to run `library()` again to load them into the new session.
3. The package name is missing quotes. It should be `install.packages("readxl")`.
4. `install.packages("lubridate")` then `library(lubridate)`.

5. Using the Console

The Problem

You're in a meeting and your supervisor asks, out loud: "Quick — if we enrolled 120 participants and 18 dropped out, what percentage completed the study?" Everyone's looking at you. You don't want to open a whole new script file, name it, and save it just to answer one quick question.

Your Mission

Use the RStudio Console to run a one-off calculation immediately, without creating or saving a file.

Think Before Coding

- What's the actual arithmetic needed to turn "102 out of 120 completed" into a percentage?
- Does this calculation need to be saved for later, or is it truly a one-time question?

5.1. The R Concept

The **Console** is the pane where R code runs immediately and directly — you type a line, press Enter, and see the result right away. Nothing you type there is automatically saved anywhere as a file. This makes the Console perfect for:

- Quick, one-off calculations

5. Using the Console

- Checking what a function does before using it in a real script
- Peeking at an object's value in the middle of a longer analysis

It's a poor choice for anything you'll want to rerun, hand to a colleague, or revisit next month — for that, you want a script (next page).

You can tell you're in the Console because it has a `>` prompt at the start of the line, waiting for input. Type your code after the `>` and press **Enter** to run it.

5.2. Demonstration

Typed directly into the Console:

```
> 102 / 120 * 100
```

```
[1] 85
```

You can also press the **up arrow** key in the Console to recall the previous command, edit it, and rerun it — useful when your supervisor immediately follows up with “okay, now what if it's 110 out of 120?”

```
> 110 / 120 * 100
```

```
[1] 91.66667
```

5.3. Explore

In the Console, calculate what percentage 18 out of 120 represents — the dropout rate rather than the completion rate. Then use the up-arrow to recall that line and change 18 to 25. Notice how much faster this is than retyping the whole line.

5.4. Solve the Problem

Back in the meeting, you type directly into the Console:

```
> 102 / 120 * 100  
[1] 85
```

You answer immediately: “85% of participants completed the study.” No file needed, no saving required — exactly the right use of the Console.

5.5. Interpret the Result

85 means that of the 120 enrolled participants, 85 out of every 100 (equivalently, 102 out of 120) completed the study — an 85% completion rate, which is generally considered a strong retention rate in longitudinal health research.

Common Mistakes

- **Doing important, reusable analysis only in the Console.** If you’ll need this calculation again — for a report, for next month’s update — put it in a script instead (next page).
- **Losing track of what was run.** The Console’s History tab keeps a record, but it’s not as organized or intentional as a saved script with comments explaining each step.
- **Forgetting the calculation entirely once the Console is cleared or RStudio is closed.** Nothing in the Console is saved unless you explicitly copy it elsewhere.

5. Using the Console

5.6. Practice Problem

In the Console, calculate the percentage of a study where 340 out of 400 participants returned for a 6-month follow-up visit.

5.7. Challenge Problem

Your supervisor now asks a follow-up: “What if we’d also included 40 participants who transferred to another clinic?” Using the up-arrow to build on your previous answer, calculate the new completion percentage out of a total of 440 enrolled, assuming the same 340 completed.

5.8. Key Takeaways

- The Console runs code immediately but does not save it as a file.
- Use the Console for quick, one-off questions — not for analysis you’ll need to reproduce or share.
- The up-arrow key recalls previous commands in the Console, which speeds up rapid back-and-forth questions.

Knowledge Check

1. **Multiple choice.** The Console is best suited for:
 - a) Long-term, reusable analysis scripts
 - b) Quick, one-off calculations and checks
 - c) Storing your final report
 - d) Installing RStudio itself
2. **True or false.** Code typed into the Console is automatically saved as a file when you close RStudio.

3. **Predict the output.**

```
45 / 60 * 100
```

4. **Write the code.** Write the Console command to calculate what percentage 7 is of 28.

Show answers

1. **b)** Quick, one-off calculations and checks.
2. **False.** Console input is not saved as a file; only scripts you explicitly save are preserved.
3. `[1] 75`
4. `7 / 28 * 100`, which evaluates to `[1] 25`.

6. Creating an R Script

The Problem

Last week you calculated a completion rate directly in the Console, like you learned on the previous page. Today your supervisor asks: “Can you rerun that same calculation, but with this month’s updated numbers?” You have no idea what you typed — it’s gone, buried somewhere in the Console history, if it’s even still there.

Your Mission

Create a saved R script so your work can be reopened, rerun, and reused — this month, next month, or a year from now.

Think Before Coding

- What’s the difference between typing code that disappears versus writing code into a document you can save and reopen?
- If a colleague asked to see exactly how you calculated last month’s number, what would you want to hand them?

6.1. The R Concept

An **R script** is a plain text file, saved with a `.R` extension, that holds a sequence of R commands. Unlike the Console, a script is a permanent, editable, shareable file — closer to a Word document than to a chat window.

6. Creating an R Script

To create one in RStudio:

File > New File > R Script (or the keyboard shortcut Ctrl+Shift+N on Windows, Cmd+Shift+N on Mac)

This opens a blank file in the **Source pane**. You write your code there, line by line, and then run it using:

- **Ctrl+Enter** (Cmd+Return on Mac) — runs just the current line (or a highlighted selection), sending it to the Console.
- The **Source** button — runs the entire script from top to bottom at once.

Crucially, writing a line in a script does **not** run it automatically — you still need to explicitly run it, exactly like the Console, but now the instructions themselves live in a saved file you control.

6.2. Demonstration

Here's a short script, as it would appear in the Source pane:

```
# completion_rate.R
# Calculates study completion rate for the community health survey

enrolled <- 120
completed <- 102

completion_rate <- completed / enrolled * 100
completion_rate
```

Running this line by line (Ctrl+Enter on each line) sends each command to the Console in turn, and you'd see:

```
[1] 85
```


Saved as `completion_rate.R`, this file can be reopened any time, and every line rerun exactly as written.

6.3. Explore

Open a new R script and type the same four lines above. Change the values of `enrolled` and `completed` to this month's actual numbers, then run the whole script using the **Source** button rather than line-by-line. Confirm you get an updated result without retyping any of the structure.

6.4. Solve the Problem

Because last month's calculation lived only in the Console, it's gone. Going forward, you save every calculation as a script:

```
# completion_rate.R
enrolled <- 130
completed <- 109

completion_rate <- completed / enrolled * 100
completion_rate

[1] 83.84615
```

Now, next month, you'll simply reopen `completion_rate.R`, update the two numbers, and rerun — no need to remember or retype anything.

6. Creating an R Script

6.5. Interpret the Result

An 83.8% completion rate for this month, computed from a saved, reusable file rather than a one-time Console calculation, means this exact process can be repeated identically every month going forward — which is the entire point of moving from Console to script.

Common Mistakes

- **Forgetting to save the script.** A script only protects your work once it's actually saved (Ctrl+S / Cmd+S) — an unsaved script is just as fragile as unsaved Console work.
- **Confusing “writing” a line with “running” it.** Typing a line of code in a script does nothing until you run it (Ctrl+Enter or Source) — the code just sits there as text until executed.
- **Running lines out of order.** If a script depends on earlier lines (like `enrolled` and `completed` existing before `completion_rate` is calculated), running lines out of sequence will cause errors or wrong results.

6.6. Practice Problem

Create a script called `dropout_rate.R` that calculates the dropout rate (as a percentage) for a study where 45 out of 300 enrolled participants dropped out. Include the object names `enrolled`, `dropped`, and `dropout_rate`.

6.7. Challenge Problem

Extend your `dropout_rate.R` script to also calculate the *completion* rate in the same script, using the same `enrolled` and `dropped` objects (hint: `completed participants = enrolled - dropped`). Make sure both results print when you run the whole script with Source.

6.8. Key Takeaways

- An R script (.R file) is a saved, reusable sequence of commands — unlike Console input, it persists after RStudio closes.
- Writing a line of code and running it are two separate steps; use Ctrl+Enter for one line or Source for the whole file.
- Save early and often — an unsaved script offers no real advantage over the Console.

Knowledge Check

1. **Multiple choice.** What file extension does a standard R script use?
 - a) .rscript
 - b) .R
 - c) .rmd
 - d) .txt
2. **True or false.** Typing a line of code into a script automatically runs it.
3. **Find the error.** A student writes a full script, closes RStudio without saving, and reopens it the next day expecting their code to still be there.
4. **Write the code.** Write the keyboard action (not R code) used to run just the current line in a script.

Show answers

1. **b) .R**
2. **False.** Writing code in a script only stores it as text; you must explicitly run it with Ctrl+Enter (a line) or Source (the whole script).
3. The script was never saved, so none of it persists — RStudio does not auto-save scripts by default; the student must save the file (Ctrl+S) before closing.

6. *Creating an R Script*

4. Ctrl+Enter (Windows/Linux) or Cmd+Return (Mac).

7. Comments

The Problem

Three months ago you wrote `completion_rate.R` and it worked perfectly. Today your supervisor asks you to hand it to a new intern. You reopen it and find:

```
enrolled <- 130
completed <- 109
completion_rate <- completed / enrolled * 100
completion_rate
```

Even you have to pause for a second to remember what this script was for and why these particular numbers were chosen. The new intern will have no context at all.

Your Mission

Add comments to your script so that anyone reading it later — including future you — immediately understands its purpose and logic.

Think Before Coding

- What information would you want written down if you were the new intern encountering this script for the first time?
- Should comments explain *what* the code does, or *why* it does it — or both?

7. Comments

7.1. The R Concept

A **comment** in R is any text following a **#** symbol on a line. R completely ignores everything after **#** — comments exist purely for humans reading the code, not for R itself.

```
# This entire line is ignored by R
enrolled <- 130 # so is everything after the # on this line
```

Comments are useful for:

- Explaining the **purpose** of a script at the top (what problem does this solve?)
- Explaining **why** a particular value or approach was chosen, when it isn't obvious
- Temporarily disabling a line of code without deleting it, by placing **#** in front of it

Good comments explain *why*, not just *what* — **# calculate completion rate** next to `completion_rate <- completed / enrolled * 100` is less useful than **# per supervisor's request, using enrollment totals as of this month's data pull**, since the code itself already shows *what* is being calculated.

7.2. Demonstration

```
# completion_rate.R
# Purpose: Calculate the study completion rate for the monthly report
# Data source: Enrollment tracking spreadsheet, updated by the study coordinator
# Written by: [your name], [date]

enrolled <- 130 # total participants enrolled this month
completed <- 109 # participants who completed all required visits
```

7.3. Explore

```
# Completion rate as a percentage of enrolled participants
completion_rate <- completed / enrolled * 100
completion_rate
```

```
[1] 83.84615
```

Running this script produces exactly the same result as before — comments have zero effect on what R computes.

7.3. Explore

Try adding a # in front of the line `completion_rate <- completed / enrolled * 100` and run the script. What happens to the final line, `completion_rate`, when that calculation never runs? Predict the error before you try it.

7.4. Solve the Problem

You rewrite `completion_rate.R` with clear comments before handing it to the new intern:

```
# completion_rate.R
# Purpose: Calculate the percentage of enrolled participants who completed the study
# Used for: Monthly report to the health department director
# Update instructions: replace 'enrolled' and 'completed' with this month's totals

enrolled <- 130    # total participants enrolled as of this month
completed <- 109   # participants who finished all study visits
```

7. Comments

```
completion_rate <- completed / enrolled * 100
completion_rate
```

Now the new intern can read the top three comment lines and immediately understand what the script is for, what to update, and where the numbers come from — without asking you a single question.

7.5. Interpret the Result

The calculation itself still returns 83.84615, unchanged. What changed is that the script is now self-documenting: anyone opening it — a new intern, a supervisor, or you in six months — can understand its purpose without additional explanation.

Common Mistakes

- **Over-commenting the obvious.** `x <- 5 # set x to 5` re-states what’s already visible in the code and adds no value; better comments explain *why* 5 was chosen.
- **Under-commenting the non-obvious.** Leaving a script with zero explanation for where data comes from or why a specific approach was used forces every future reader (including you) to reverse-engineer your intent.
- **Forgetting comments don’t need to “close.”** Unlike some languages, R comments run from `#` to the end of that line only — there’s no special closing symbol, and a comment never spans multiple lines automatically.
- **Commenting out code and forgetting about it.** A `#`-disabled line left in a script for months can confuse future readers about whether it’s meant to run or not — clean up disabled code once you’re sure you don’t need it.

7.6. Practice Problem

Take the `dropout_rate.R` script you wrote in the previous page's practice problem and add: a purpose comment at the top, a comment explaining where the numbers came from, and inline comments on the `enrolled` and `dropped` lines.

7.7. Challenge Problem

Write a short script (3–4 lines of actual code) that calculates a body mass index (BMI) from a weight in kilograms and a height in meters, using the formula `weight / height^2`. Document it fully with comments so that someone with zero statistics background could still understand and reuse it.

7.8. Key Takeaways

- Comments start with `#` and are completely ignored by R — they exist only for human readers.
- Good comments explain *why*, not just *what* — the code itself already shows what's happening.
- `#` can also temporarily disable a line of code without deleting it.

Knowledge Check

1. **Multiple choice.** What symbol starts a comment in R?

- a) `//`
- b) `#`
- c) `/* */`
- d) `--`

7. Comments

2. **True or false.** Comments affect how R executes a script.
3. **Predict the output.**

```
x <- 10 # setting x
# x <- 999
x
```

4. **Write the code.** Write a two-line script that stores the number of study sites (5) in an object called `n_sites`, with a comment explaining what the object represents.

Show answers

1. **b) #**
2. **False.** Comments are entirely ignored by R and have no effect on execution or results.
3. `[1] 10` — the second line is commented out, so `x` is never reassigned to 999.
4.

```
n_sites <- 5 # number of participating study sites
n_sites
```

8. Basic Arithmetic

The Problem

A clinic reports patient weights in pounds, but your study protocol requires kilograms for calculating drug dosages. You also need to compute each patient's BMI, which requires squaring their height. You're not sure R can handle exponents, or whether the order you write things in will matter.

Your Mission

Perform the arithmetic conversions and calculations needed to convert units and compute BMI correctly in R.

Think Before Coding

- What's the conversion formula from pounds to kilograms (roughly divide by 2.2)?
- What's the BMI formula, and which operation do you need for "height squared"?
- If you wrote $2 + 3 * 4$ by hand, would you get 20 or 14 — and why does the order matter?

8.1. The R Concept

R supports all standard arithmetic operators, and follows the usual mathematical order of operations (PEMDAS/BODMAS):

8. Basic Arithmetic

Operator	Meaning	Example	Result
+	addition	5 + 3	8
-	subtraction	5 - 3	2
*	multiplication	5 * 3	15
/	division	5 / 3	1.6667
^	exponent (power)	5 ^ 2	25
%%	modulo (remainder)	5 %% 3	2
%%/	integer division	5 %/% 3	1

Parentheses () override the default order, exactly as in standard math notation. When in doubt, use them — they make your intent explicit even when they're not strictly required.

8.2. Demonstration

```
# Order of operations matters: multiplication happens before addition
2 + 3 * 4
```

```
[1] 14
```

```
# Parentheses force addition to happen first
(2 + 3) * 4
```

```
[1] 20
```

```
# Converting 154 pounds to kilograms
weight_lb <- 154
weight_kg <- weight_lb / 2.20462
weight_kg
```

```
[1] 69.85342
```

8.3. Explore

```
# BMI = weight (kg) / height (m) squared
height_m <- 1.75
bmi <- weight_kg / height_m^2
bmi

[1] 22.80887
```

8.3. Explore

Modify `height_m` to 1.60 and rerun the BMI calculation. Then try writing the BMI formula *without* parentheses around `height_m^2` as `weight_kg / height_m^2` versus incorrectly as `(weight_kg / height_m)^2` — compare the two results and notice how differently they turn out.

8.4. Solve the Problem

For a patient weighing 180 pounds and standing 1.68 meters tall:

```
weight_lb <- 180
weight_kg <- weight_lb / 2.20462
height_m <- 1.68

bmi <- weight_kg / height_m^2
bmi

[1] 28.92108
```

8.5. Interpret the Result

A BMI of approximately 28.9 falls in the “overweight” category by standard WHO classification (25–29.9), which is the kind of clinical interpretation that would go directly into a patient chart or research dataset alongside this calculated value.

Common Mistakes

- **Misjudging order of operations.** Writing `weight_kg / height_m^2` relies on `^` happening before `/` — which R does correctly by default — but when unsure, add parentheses: `weight_kg / (height_m^2)`.
- **Using `x` instead of `*` for multiplication.** R has no concept of `x` as a multiplication symbol; only `*` works.
- **Confusing `/` and `%/%`.** `7 / 2` gives 3.5 (regular division); `7 %/% 2` gives 3 (integer division, dropping the remainder). Using the wrong one silently produces a different kind of number.
- **Forgetting units before converting.** A common real error is applying a kg formula to a still-in-pounds value, producing a BMI that looks plausible but is completely wrong.

8.6. Practice Problem

A dosing protocol calls for 5 mg of medication per kilogram of body weight. Calculate the correct total dose (in mg) for a patient weighing 198 pounds.

8.7. Challenge Problem

Body surface area (BSA) is sometimes estimated using the Mosteller formula: `sqrt((height_cm * weight_kg) / 3600)`. Using `height_m <-`

1.72 and `weight_kg <- 75`, calculate BSA — remembering to first convert height into centimeters (hint: `sqrt()` is a built-in function you can use directly, and you'll learn more about functions like it later in this level).

8.8. Key Takeaways

- R supports `+`, `-`, `*`, `/`, `^`, `%%`, and `sqrt()`, following standard order of operations.
- Parentheses `()` override default order and make calculations easier to read correctly.
- Getting units right *before* applying a formula matters as much as getting the formula's operators right.

Knowledge Check

1. **Multiple choice.** What does `%%` compute?
 - a) Percentage
 - b) Integer division
 - c) Remainder after division (modulo)
 - d) Square root
2. **True or false.** `2 + 3 * 4` and `(2 + 3) * 4` give the same result in R.
3. **Predict the output.**

`10 %% 3`
4. **Find the error.** A student writes `weight_lb x 2.20462` intending to convert kilograms to pounds. What's wrong?

Show answers

8. *Basic Arithmetic*

1. **c)** Remainder after division (modulo).
2. **False.** $2 + 3 * 4 = 14$ (multiplication first); $(2 + 3) * 4 = 20$ (parentheses force addition first).
3. `[1] 1` (10 divided by 3 is 3 remainder 1).
4. R doesn't recognize `x` as a multiplication symbol — it should be `weight_lb * 2.20462`.

9. The Assignment Operator

The Problem

You calculated a patient’s BMI in the last page, but now you need to use that same BMI value in three more calculations later in your script — comparing it to thresholds, flagging obesity, and adding it to a summary table. Retyping the entire BMI formula three more times feels wasteful, and error-prone if you make a typo on the second or third copy.

Your Mission

Store a calculated value using R’s assignment operator so you can reuse it by name, instead of recalculating or retyping it.

Think Before Coding

- If you compute something once, what would it mean to give that result a “name” you could refer back to?
- In algebra, when you write $x = 5$, is that the same idea as what you’d want here — storing a value under a label?

9.1. The R Concept

The **assignment operator**, `<-`, stores a value under a name you choose:

```
name <- value
```

9. The Assignment Operator

Read `<-` as “gets” — `bmi <- 22.8` means “bmi gets the value 22.8.” Once assigned, you can use `bmi` anywhere in your later code exactly as if you’d typed 22.8 directly.

R also accepts `=` for assignment in most contexts, and you’ll see both in other people’s code. This course uses `<-` throughout, which is the long-standing convention in the R community — it visually distinguishes “storing a value” from “specifying a function argument” (which uses `=`, as you’ll see in later lessons), a distinction that keeps longer scripts easier to read.

Keyboard shortcut: **Alt** + **-** (Windows/Linux) or **Option** + **-** (Mac) types `<-` automatically, including the spaces around it.

Nothing prints to the Console when you assign a value — assignment is a silent action. To see the value, type the object’s name on its own line afterward.

9.2. Demonstration

```
# Assign a value - nothing prints
bmi <- 22.8

# Now print it by typing its name alone
bmi

[1] 22.8

# Reuse the stored value in further calculations
is_high_bmi <- bmi >= 25
is_high_bmi

[1] FALSE
```

9.3. Explore

```
# Overwriting: assigning a new value to an existing name replaces it
bmi <- 27.4
bmi
```

```
[1] 27.4
```

9.3. Explore

Assign `age <- 45`, then create `age_in_5_years <- age + 5` using the stored object rather than typing 45 again. Print both objects. Then reassign `age <- 52` — does `age_in_5_years` automatically update? Predict the answer before running it.

9.4. Solve the Problem

```
weight_kg <- 78
height_m <- 1.70

bmi <- weight_kg / height_m^2
bmi
```

```
[1] 26.98962
```

```
# Reuse bmi for further checks, without retyping the formula
is_overweight <- bmi >= 25 & bmi < 30
is_overweight
```

```
[1] TRUE
```

9. The Assignment Operator

```
# Reuse it again in a rounded summary
bmi_rounded <- round(bmi, 1)
bmi_rounded
```

```
[1] 27
```

The BMI value is now calculated once and reused three times, with zero risk of retyping it incorrectly.

9.5. Interpret the Result

Storing `bmi` once and reusing it means every downstream calculation — the overweight flag, the rounded summary — is guaranteed to be based on the *exact* same value, with no chance of a copy-paste typo silently introducing an inconsistency between them.

Common Mistakes

- **Expecting objects to update automatically.** `age_in_5_years` does **not** recalculate when `age` changes later — it was computed once, at assignment time, using whatever `age` was *at that moment*. You'd need to rerun the calculation line to update it.
- **Forgetting to print after assigning.** `bmi <- 22.8` shows nothing in the Console; you must run `bmi` on its own line (or wrap the assignment in parentheses: `(bmi <- 22.8)`) to see the value.
- **Using `<` and `-` with a space between them.** `< -` (with a space) is not the same as `<-` — R will likely produce an error or unexpected comparison. Use the keyboard shortcut to avoid this.
- **Mixing up `<-` direction.** `5 <- bmi` attempts to assign `bmi`'s value into 5, which is invalid — the name always goes on the left.

9.6. Practice Problem

Assign a patient's systolic blood pressure of 142 to an object called `sbp`. Then create a second object, `elevated_bp`, that stores whether `sbp` is greater than or equal to 130 (a common elevated-BP threshold). Print both objects.

9.7. Challenge Problem

Assign `enrolled <- 250` and `completed <- 211`. Using only these two objects (not the raw numbers again), calculate `completion_rate` as a percentage, then calculate `dropout_rate` as `100 - completion_rate`, reusing `completion_rate` rather than recomputing it from scratch.

9.8. Key Takeaways

- `<-` assigns a value to a name; use the object's name afterward to see or reuse it.
- Assignment is silent — nothing prints unless you explicitly ask for the object by name.
- Objects do not automatically update when the values they were built from change — they hold a fixed snapshot from the moment they were assigned.

Knowledge Check

1. **Multiple choice.** What does `bmi <- 22.8` do?
 - a) Compares `bmi` to 22.8
 - b) Stores the value 22.8 under the name `bmi`
 - c) Prints 22.8 to the Console

9. The Assignment Operator

d) Deletes the object `bmi`

2. **True or false.** If `y <- x + 1` and then `x` is reassigned to a new value, `y` automatically updates too.

3. **Predict the output.**

```
score <- 88  
score
```

4. **Find the error.**

```
90 <- passing_score
```

Show answers

1. **b)** Stores the value 22.8 under the name `bmi`.
2. **False.** `y` was calculated once, using `x`'s value at that moment — it does not recalculate automatically when `x` changes.
3. `[1] 88`
4. The assignment is backwards — the object name must be on the left of `<-`. It should be `passing_score <- 90`.

10. Objects

The Problem

Halfway through a script, you’ve now created `weight_kg`, `height_m`, `bmi`, `is_overweight`, and `bmi_rounded`. Your supervisor walks by and asks: “What exactly is stored in R right now?” You glance at your script but realize the script shows what you *ran*, not necessarily what currently *exists* in memory.

Your Mission

Understand what an R “object” is in general, and learn to inspect everything currently stored in your session.

Think Before Coding

- Is there a difference between the *code that created* something and the *thing that now exists* as a result?
- If you assigned ten different values across a long script, how would you check, right now, what’s actually available to use?

10.1. The R Concept

In R, virtually everything you create and store is called an **object**. A single number, a block of text, a list of a hundred ages, an entire imported spreadsheet — all of these become objects once assigned a name with `<-`. This is a deliberately broad, unifying idea: whether an object holds one value or a million, you interact with it the same way — by name.

10. Objects

Two ways to see what objects currently exist in your session:

1. **The Environment pane** (top-right in RStudio) — lists every object live, updating automatically as you create, modify, or remove them.
2. `ls()` — a function that lists all object names in your current session, directly in the Console.

```
ls()
```

To remove an object you no longer need:

```
rm(bmi_rounded)
```

And to clear everything at once (useful for starting fresh, though rarely needed mid-analysis):

```
rm(list = ls())
```

10.2. Demonstration

```
weight_kg <- 78
height_m <- 1.70
bmi <- weight_kg / height_m^2
is_overweight <- bmi >= 25 & bmi < 30
bmi_rounded <- round(bmi, 1)
```

```
# See every object currently in this session
```

```
ls()
```

```
[1] "bmi"           "bmi_rounded"   "height_m"      "is_overweight" "weight_kg"
```

Note that `ls()` returns names in alphabetical order, not the order you created them — useful to know so you don't expect chronological order.

10.3. Explore

Run `rm(bmi_rounded)` and then `ls()` again. Confirm `bmi_rounded` is gone from the list, while the other four objects remain. Then try printing `bmi_rounded` directly — what error message does R give you?

10.4. Solve the Problem

```
ls()
```

```
[1] "bmi"           "bmi_rounded"   "height_m"      "is_overweight" "weight_kg"
```

You can now answer your supervisor precisely: “Right now, I have five objects stored — `bmi`, `bmi_rounded`, `height_m`, `is_overweight`, and `weight_kg`.” You didn’t need to scroll back through your whole script to answer confidently — `ls()` (or the Environment pane) gave you the live, current state directly.

10.5. Interpret the Result

The output of `ls()` is not the *code* that built these objects — it’s the *current inventory* of what those lines of code produced and left behind in memory. This is an important distinction: a script shows history; `ls()` and the Environment pane show present state.

Common Mistakes

- **Assuming the script and the Environment always match.** If you delete a line from your script but already ran it, the object it created still exists in your session until you explicitly remove it or restart R.

10. Objects

- **Using an object that “shouldn’t” exist anymore.** Because R doesn’t automatically clean up after you, leftover objects from earlier experimentation can accidentally get reused later in a script, producing confusing results.
- **Confusing `rm()` with deleting a file.** `rm(bmi)` removes the *object* from your current session’s memory — it has no effect on any saved script or data file.

10.6. Practice Problem

Create three objects: `n_enrolled <- 250`, `n_completed <- 211`, and `site_name <- "Riverdale Clinic"`. Run `ls()` and confirm all three appear. Then remove `site_name` with `rm()` and run `ls()` again to confirm it’s gone.

10.7. Challenge Problem

Without running any code, predict what `ls()` would return immediately after a fresh RStudio session starts, before you’ve created any objects. Then verify your prediction is correct — reason through what an empty environment should report.

10.8. Key Takeaways

- Nearly everything you create in R — numbers, text, entire datasets — is stored as an object once assigned a name.
- `ls()` and the Environment pane both show what currently exists in your session, which can differ from what your script currently contains.

- `rm()` removes an object from memory but never touches any saved file.

Knowledge Check

1. **Multiple choice.** What does `ls()` show?
 - a) The lines of code in your current script
 - b) The names of all objects currently in your session
 - c) A list of installed packages
 - d) The contents of your working directory folder
2. **True or false.** Deleting a line of code from your script automatically removes the corresponding object from your session.
3. **Predict the output.**

```
a <- 1
b <- 2
rm(a)
ls()
```
4. **Write the code.** Write the command that removes every object currently in your session at once.

Show answers

1. **b)** The names of all objects currently in your session.
2. **False.** Removing a line from your script has no effect on objects already created in memory; you must explicitly run `rm()` or restart R to clear them.
3. `[1] "b"` — only `b` remains after `a` was removed.
4. `rm(list = ls())`

11. Naming Objects

The Problem

You inherit a script from a former intern that includes lines like:

```
x <- 78
y <- 1.70
z <- x / y^2
temp2 <- z >= 25
```

There's no way to tell, just by reading it, that `x` is a weight, `y` is a height, `z` is a BMI, or that `temp2` flags overweight status. You have to trace through the whole script just to understand what one line means.

Your Mission

Learn R's naming rules and adopt naming habits that make your code understandable at a glance — to your supervisor, a collaborator, or yourself in six months.

Think Before Coding

- If you saw `bmi <- weight_kg / height_m^2` versus `z <- x / y^2`, which would you understand faster without any comments at all?
- What information would a good object name communicate about what it holds?

11. Naming Objects

11.1. The R Concept

R's actual naming **rules** (violating these causes an error) are:

- Names can contain letters, numbers, dots (`.`), and underscores (`_`).
- Names cannot start with a number (`2nd_visit` is invalid; `second_visit` is fine).
- Names are case-sensitive: `Age`, `age`, and `AGE` are three different objects.
- A handful of words are reserved by R itself (like `TRUE`, `FALSE`, `if`, `for`) and can't be used as object names.

Beyond the hard rules, there are **style conventions** — not enforced by R, but strongly recommended for readability:

- Use **snake_case** (lowercase words separated by underscores): `bmi_rounded`, `is_overweight`, `n_enrolled`. This is the convention used throughout the tidyverse and this course.
- Make names descriptive but not excessively long: `bmi` is fine; `body_mass_index_calculated_value` is overkill.
- Use consistent prefixes for related concepts: `n_enrolled`, `n_completed` (counts), `is_overweight`, `is_smoker` (logical flags).

11.2. Demonstration

```
# Poor naming - technically valid R, but unreadable
x <- 78
y <- 1.70
z <- x / y^2
temp2 <- z >= 25
```

11.3. Explore

```
# Clear naming - communicates meaning without needing comments
weight_kg <- 78
height_m <- 1.70
bmi <- weight_kg / height_m^2
is_overweight <- bmi >= 25
```

Both blocks compute exactly the same thing and produce identical results — the difference is entirely about how quickly a human reader understands them.

```
# This will cause an error - names can't start with a number
2nd_visit_bp <- 130
```

Error: unexpected symbol in "2nd_visit_bp"

11.3. Explore

Try creating an object called `TRUE_value <- 5` — does it work? Then try creating one literally named `TRUE <- 5` — what error do you get, and why do you think R protects this particular word?

11.4. Solve the Problem

Rewriting the inherited script with clear names:

```
weight_kg <- 78
height_m <- 1.70
bmi <- weight_kg / height_m^2
is_overweight <- bmi >= 25
```

11. Naming Objects

Now any reader — including the next intern who inherits *this* script — can understand the purpose of every line without needing you to explain it in person.

11.5. Interpret the Result

The computed values are unchanged (`bmi` is still approximately 26.99, `is_overweight` is still `TRUE`), but the script itself has become documentation. Clear names reduce the need for comments explaining *what* a value represents, freeing comments to explain *why*, as discussed on the comments page.

Common Mistakes

- **Starting a name with a number.** `1st_dose <- 50` fails; use `first_dose` or `dose_1` instead.
- **Assuming names are case-insensitive.** `Age` and `age` are two entirely separate objects in R — a frequent source of “object not found” errors when a name is typed with different capitalization than it was created with.
- **Using generic, uninformative names.** `data`, `df`, `temp`, `x1`, `x2` are common but tell a reader nothing about the actual content — reserve short generic names for genuinely temporary, throwaway calculations only.
- **Using reserved words.** Trying to name an object `TRUE`, `if`, `for`, or `function` will fail, since these are reserved by R’s syntax itself.

11.6. Practice Problem

Rename the following poorly-named objects to follow this course’s `snake_case` convention, keeping their meaning clear: `PatientAge`, `bp.systolic`, `SmokerFlag`.

11.7. Challenge Problem

A colleague's script uses `df1`, `df2`, and `df3` for three different datasets: baseline enrollment, six-month follow-up, and twelve-month follow-up. Propose clearer names for all three, following the conventions on this page.

11.8. Key Takeaways

- R naming rules: letters, numbers, dots, and underscores are allowed; names can't start with a number; names are case-sensitive.
- This course (and the tidyverse broadly) uses **snake_case**: lowercase words joined with underscores.
- Descriptive names reduce the need for comments and make scripts understandable at a glance.

Knowledge Check

1. **Multiple choice.** Which of these is a valid R object name?

- a) 2nd_dose
- b) dose_2
- c) dose-2
- d) TRUE

2. **True or false.** `Age` and `age` refer to the same object in R.

3. **Find the error.**

```
1st_bp_reading <- 130
```

4. **Write the code.** Rename the object `x` to something descriptive, assuming it currently stores a participant's diastolic blood pressure of 82.

11. Naming Objects

Show answers

1. **b)** `dose_2` — the others either start with a number, use an invalid hyphen, or are a reserved word.
2. **False.** R is case-sensitive; `Age` and `age` are two distinct objects.
3. Object names cannot start with a number. It should be `bp_reading_1 <- 130` or similar.
4. `diastolic_bp <- 82`

12. Basic Data Types

The Problem

You import a participant's age from a form and try to calculate next year's age:

```
age <- "34"  
age_next_year <- age + 1
```

```
Error in age + 1 : non-numeric argument to binary operator
```

R refuses to do the math, even though "34" clearly looks like a number to you.

Your Mission

Understand that R distinguishes between different fundamental *types* of data, and learn to check which type you're actually working with.

Think Before Coding

- Does "34" (with quotation marks) look different from 34 (without quotes) when you type it?
- What other kinds of information, besides numbers, might a health dataset contain — names, yes/no answers, dates?

12. Basic Data Types

12.1. The R Concept

Every value in R has a **data type**, and R treats different types differently, even when they look similar to a human. The three you'll use constantly, starting today:

- **Numeric** — actual numbers you can do math with: 34, 22.8, -5.
- **Character** — text, always wrapped in quotes: "34", "Female", "Riverdale". Note that "34" is text, *not* a number, purely because of the quotes.
- **Logical** — TRUE or FALSE values (no quotes), used for yes/no conditions.

You can check any object's type with `class()`:

```
class(34)      # numeric
class("34")   # character
class(TRUE)    # logical
```

The error at the top of this page happened because `age <- "34"` created a *character* object — text that happens to look like a number — and R correctly refuses to perform arithmetic on text.

12.2. Demonstration

```
class(34)

[1] "numeric"

class("34")

[1] "character"
```

```
class(TRUE)

[1] "logical"

# The type mismatch that caused the original error
age <- "34"
class(age)

[1] "character"
```

12.3. Explore

Run `class(age + 1)` after setting `age <- "34"` and confirm you see the same error as in the opening problem. Then try `class(as.numeric(age) + 1)` — you’re using a *conversion function* here, which you’ll study properly on the numeric-data page next, but notice how wrapping `age` changes the outcome.

12.4. Solve the Problem

The fix is to make sure `age` is stored as a numeric value, not text:

```
age <- 34          # no quotes -> numeric
class(age)

[1] "numeric"

age_next_year <- age + 1
age_next_year

[1] 35
```

12.5. Interpret the Result

`age_next_year` now correctly equals 35, because `age` was stored as an actual number rather than a piece of text. The lesson generalizes: any time R refuses to do arithmetic you expect it to handle, `class()` is the first diagnostic tool to reach for.

Common Mistakes

- **Adding quotes around numbers by habit.** `age <- "34"` looks harmless but silently creates text, not a number — a very common source of confusing downstream errors.
- **Assuming R will “figure out” what you meant.** Unlike Excel, R does not automatically guess that `"34"` should behave like a number in a math context — it enforces the type strictly.
- **Forgetting quotes around genuine text.** `sex <- Female` (no quotes) will error, because R interprets `Female` as an object name it can't find, not as the text `"Female"`.

12.6. Practice Problem

For each of the following, predict the result of `class()` before running it: `class(120)`, `class("120")`, `class(FALSE)`, `class("TRUE")`.

12.7. Challenge Problem

A dataset column for blood type contains values like `"O+"`, `"AB-"`, `"A+"`. Explain, in your own words, why this column should be stored as character data rather than numeric, even though blood types are sometimes informally coded with numbers elsewhere.

12.8. Key Takeaways

- Every R value has a type; the three fundamentals are numeric, character, and logical.
- `class()` reveals an object's current type — your first stop when arithmetic or comparisons behave unexpectedly.
- Quotes make the difference between a number (34) and text that merely looks like a number ("34").

Knowledge Check

1. **Multiple choice.** What is the class of "25" (with quotes)?
 - a) numeric
 - b) character
 - c) logical
 - d) integer
2. **True or false.** R will automatically convert "25" to a number if you try to do math with it.
3. **Find the error.**

```
n_visits <- "4"
total_visits <- n_visits + 2
```

4. **Predict the output.**

```
class(TRUE)
```

Show answers

1. **b)** character — the quotes make it text, regardless of what it looks like.
2. **False.** R will raise an error (**non-numeric argument to binary operator**) rather than silently converting it.

12. Basic Data Types

3. `n_visits` is stored as character text ("`4`"), not a number, so `+` 2 fails. Fix: `n_visits <- 4` (no quotes).
4. `[1] "logical"`

13. Numeric Data

The Problem

A data entry error means one participant's age was typed into the spreadsheet as "45 " — with a trailing space — and imported into R as text. Your supervisor wants to know: is this actually going to cause a problem, and how do you fix it once you spot it?

Your Mission

Understand R's numeric data type in more depth, including how to check for it, and how to convert text that looks numeric into an actual number.

Think Before Coding

- If `class()` tells you an object's type, what function might tell you *specifically* whether it's numeric — a yes/no answer rather than a label?
- What tool would you need to turn "45" (text) into 45 (a real number) so you can do arithmetic with it?

13.1. The R Concept

R's **numeric** type covers ordinary decimal numbers — what R internally calls “double precision” floating point numbers, which is more than enough precision for virtually all health and research data. (R also has a stricter “integer” type, marked with an L suffix like 5L, but you will rarely need

13. Numeric Data

to use it directly as a beginner — numeric handles both whole numbers and decimals fine.)

Two functions matter here:

- `is.numeric(x)` — returns TRUE or FALSE, directly answering “is this numeric?”
- `as.numeric(x)` — attempts to *convert* `x` into a numeric value. If `x` is text that clearly represents a number (like "45"), this works cleanly. If the text isn't a valid number (like "forty-five"), R will produce NA and a warning, rather than crashing.

```
is.numeric(45)      # TRUE
is.numeric("45")    # FALSE - it's character, even though it looks numeric
as.numeric("45")    # converts "45" into the number 45
```

13.2. Demonstration

```
age_text <- "45"
is.numeric(age_text)
```

```
[1] FALSE
```

```
age_numeric <- as.numeric(age_text)
age_numeric
```

```
[1] 45
```

```
is.numeric(age_numeric)
```

```
[1] TRUE
```

13.3. Explore

```
# What happens with genuinely invalid text
as.numeric("forty-five")
```

```
[1] NA
Warning message:
NAs introduced by coercion
```

13.3. Explore

Try `as.numeric("45 ")` (with a trailing space, matching the opening problem) — does the extra space cause a problem, or does R handle it gracefully? Then try `as.numeric("$45")` — does a dollar sign behave the same way?

13.4. Solve the Problem

```
age_raw <- "45 " # as it came in from the spreadsheet, with a trailing space
is.numeric(age_raw)
```

```
[1] FALSE
```

```
age_fixed <- as.numeric(age_raw)
age_fixed
```

```
[1] 45
```

```
is.numeric(age_fixed)
```

```
[1] TRUE
```

13. Numeric Data

R's `as.numeric()` handles the trailing space gracefully in this case, correctly converting "45 " into the number 45. The fix is a single line, run once the problem is identified.

13.5. Interpret the Result

The participant's age is now correctly stored as the number 45, ready to be used in any calculation (like computing the sample's mean age) rather than being silently excluded or causing errors because it was still text.

Common Mistakes

- **Assuming `as.numeric()` always succeeds silently.** If the text isn't a valid number (extra letters, stray symbols like currency signs), `as.numeric()` returns NA with a warning rather than an error — easy to miss if you're not watching for warnings.
- **Not checking before converting an entire column.** Running `as.numeric()` on a whole column of messy real-world data can silently turn several entries into NA if even a few values aren't clean numbers — always check the result afterward.
- **Confusing `is.numeric()` with `as.numeric()`.** One *checks* (`is.`), the other *converts* (`as.`) — mixing them up is a common beginner slip, since they sound similar.

13.6. Practice Problem

You have `bp_text <- "128"`. Confirm with `is.numeric()` that it's currently not numeric, convert it with `as.numeric()`, and confirm the conversion worked.

13.7. Challenge Problem

A column of self-reported weights includes the entries "70", "68.5", "unknown", and "72 kg". Predict what `as.numeric()` would produce for each of these four values individually, and explain why two of them would likely become NA.

13.8. Key Takeaways

- `is.numeric()` checks whether a value is numeric; `as.numeric()` attempts to convert a value into a number.
- `as.numeric()` can gracefully handle some messiness (like stray whitespace) but returns NA with a warning for text that isn't a valid number.
- Always check the result after converting real-world data — silent NAs from a failed conversion are easy to miss.

Knowledge Check

1. **Multiple choice.** What does `as.numeric("hello")` return?

- a) An error that stops your script
- b) 0
- c) NA, with a warning
- d) The text "hello" unchanged

2. **True or false.** `is.numeric("45")` returns TRUE.

3. **Predict the output.**

```
as.numeric("100")
```

4. **Write the code.** Convert `dose_text <- "250"` into a numeric object called `dose_numeric`, then confirm it's numeric.

13. Numeric Data

Show answers

1. **c)** NA, with a warning (“NAs introduced by coercion”) — R does not stop your script, but flags the failed conversion.
2. **False.** "45" is character data (it's quoted); `is.numeric()` returns FALSE regardless of what the text looks like.
3. `[1] 100`
4.

```
dose_numeric <- as.numeric(dose_text)
is.numeric(dose_numeric)
```

14. Character Data

The Problem

You need to record each participant's county of residence and generate a short label like "Participant from Riverdale" for a summary report. You try:

```
county <- Riverdale
```

```
Error: object 'Riverdale' not found
```

R seems to think `Riverdale` should already exist somewhere, rather than treating it as the text you meant to store.

Your Mission

Understand R's character (text) data type, how to create and combine strings of text, and why quotes matter so much.

Think Before Coding

- What's different between `Riverdale` and `"Riverdale"` from R's point of view?
- If you wanted to glue two pieces of text together — like a label and a name — what kind of tool would you need?

14. Character Data

14.1. The R Concept

Character data is text, and it must always be wrapped in quotes — either single (`'text'`) or double (`"text"`); this course uses double quotes consistently. Without quotes, R assumes you're referring to an *object name*, not literal text — which is exactly why `county <- Riverdale` failed: R went looking for an existing object called `Riverdale` and found none.

Useful functions for working with character data:

- `nchar(x)` — counts the number of characters in a string.
- `paste(...)` — combines multiple pieces of text (and even numbers) into one string, separated by a space by default.
- `paste0(...)` — same as `paste()`, but with no separator between pieces — handy when you want text glued together with no gap.

```
nchar("Riverdale")           # 9
paste("Participant from", "Riverdale") # "Participant from Riverdale"
paste0("ID_", 42)             # "ID_42"
```

14.2. Demonstration

```
county <- "Riverdale"
class(county)

[1] "character"

nchar(county)

[1] 9

label <- paste("Participant from", county)
label
```


14.3. Explore

```
[1] "Participant from Riverdale"

participant_id <- paste0("ID_", 42)
participant_id
```

```
[1] "ID_42"
```

14.3. Explore

Try `paste("Participant from", county, sep = "-")` — the `sep` argument changes the default separator. What does the output look like now? Then try combining three pieces of text at once: `paste("Age:", 45, "County:", county)`.

14.4. Solve the Problem

```
county <- "Riverdale" # quotes required - this is text, not an object reference
label <- paste("Participant from", county)
label
```

```
[1] "Participant from Riverdale"
```

The original error is fixed simply by adding quotes: R now correctly stores the literal text `"Riverdale"` rather than searching for a nonexistent object with that name.

14.5. Interpret the Result

`label` now holds a complete, human-readable sentence built by combining fixed text (`"Participant from"`) with a stored value (`county`) — exactly the kind of thing you’d use to generate personalized summaries or labels across many participants at once (a technique you’ll extend to entire columns of data later in the course).

Common Mistakes

- **Forgetting quotes around text.** `county <- Riverdale` causes an “object not found” error, since R interprets unquoted text as an object reference, not literal text.
- **Mixing up "TRUE" and TRUE.** `"TRUE"` (quoted) is character data — just the text “TRUE” — while `TRUE` (unquoted) is logical data. They look similar but behave very differently.
- **Expecting `paste()` to add punctuation automatically.** `paste("Age:", 45)` produces `"Age: 45"` with the default single-space separator — if you want something else (a comma, no space), you need the `sep` argument.
- **Confusing `paste()` and `paste0()`.** `paste("ID_", 42)` produces `"ID_ 42"` (note the space), while `paste0("ID_", 42)` produces `"ID_42"` — a small but common source of unwanted spaces in generated labels.

14.6. Practice Problem

Create an object `city <- "Oakford"`. Use `paste0()` to build a string reading exactly `"Site: Oakford"` (no extra space after the colon).

14.7. Challenge Problem

You have `first_name <- "Amara"` and `last_name <- "Okafor"`. Build a single string `full_id` that reads "Okafor, Amara" using `paste()` with an appropriate separator.

14.8. Key Takeaways

- Character data must be quoted; unquoted text is interpreted as an object name and will error if that name doesn't exist.
- `paste()` combines text with a space separator by default (customizable via `sep`); `paste0()` combines text with no separator.
- `nchar()` counts the characters in a string — useful for spotting unexpectedly long or short text entries during data checks.

Knowledge Check

1. **Multiple choice.** Which of these correctly stores the text "Female"?
 - a) `sex <- Female`
 - b) `sex <- "Female"`
 - c) `sex = Female`
 - d) `sex <- Female()`
2. **True or false.** `paste0()` adds a space between combined pieces of text by default.
3. **Predict the output.**

```
paste0("Visit_", 3)
```
4. **Find the error.**

14. Character Data

```
status <- Active
```

Show answers

1. **b)** `sex <- "Female"` — quotes are required for text.
2. **False.** `paste0()` uses no separator by default; `paste()` uses a single space by default.
3. `[1] "Visit_3"`
4. `Active` is unquoted, so R looks for an object named `Active` and fails. It should be `status <- "Active"`.

15. Logical Data

The Problem

Your supervisor asks: “Which participants are current smokers, and which are 65 or older?” You need to flag each participant with a clear yes/no answer for both questions, so these flags can later be used to filter and count people — but you’re not sure what kind of value a “yes/no” answer should even be stored as in R.

Your Mission

Understand R’s logical data type (**TRUE/FALSE**) and the comparison operators used to create it.

Think Before Coding

- What are all the possible answers to a genuine yes/no question — is there really a third option?
- If you wanted R to check “is this participant’s age 65 or more,” what symbol would represent “is greater than or equal to”?

15.1. The R Concept

Logical data has exactly two possible values: **TRUE** or **FALSE** (always written in capital letters, never quoted — “**TRUE**” is character text, not logical). Logical values are most often produced by **comparison operators**:

15. Logical Data

Operator	Meaning
==	equal to
!=	not equal to
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to

You can also combine logical values using:

Operator	Meaning
&	and (both must be TRUE)
\	or (at least one must be TRUE)
!	not (flips TRUE to FALSE, and vice versa)

The single most common beginner mistake is confusing = (assignment) with == (equality comparison) — they look almost identical but do fundamentally different things.

15.2. Demonstration

```
age <- 68
is_senior <- age >= 65
is_senior

[1] TRUE

smoker_status <- "Yes"
is_smoker <- smoker_status == "Yes"
is_smoker
```

15.3. Explore

```
[1] TRUE
```

```
# Combining two conditions with &
high_risk <- is_senior & is_smoker
high_risk
```

```
[1] TRUE
```

```
# Flipping a logical value with !
is_nonsmoker <- !is_smoker
is_nonsmoker
```

```
[1] FALSE
```

15.3. Explore

Change `age` to 50 and rerun `is_senior <- age >= 65`. What do you get, and does `high_risk` change automatically? (Recall from the objects page: it won't, unless you rerun the calculation.) Try `age == "68"` (comparing a number to quoted text) — does it still work as expected?

15.4. Solve the Problem

```
age <- 71
smoker_status <- "Yes"

is_senior <- age >= 65
is_smoker <- smoker_status == "Yes"

is_senior
```

15. Logical Data

```
[1] TRUE
```

```
is_smoker
```

```
[1] TRUE
```

Both flags now clearly and unambiguously answer your supervisor’s two questions for this participant, ready to be reused later (for example, to count or filter many participants at once — a technique covered in Level 5).

15.5. Interpret the Result

`is_senior` being `TRUE` means this participant meets the “65 or older” criterion; `is_smoker` being `TRUE` means they reported current smoking. Stored as logical values rather than text, these flags can be directly combined (as with `high_risk` above) or later used to filter a whole dataset down to exactly the participants who match both conditions.

Common Mistakes

- **Confusing = and ==.** `age = 65` *assigns* 65 to `age`; `age == 65` *checks whether* `age` equals 65. Using the wrong one produces very different, sometimes silently wrong, results.
- **Quoting TRUE/FALSE unnecessarily.** `is_smoker <- "TRUE"` creates character text, not a logical value — drop the quotes: `is_smoker <- TRUE`.
- **Misusing & versus |.** `age >= 65 & age <= 75` requires *both* conditions; using `|` instead would match ages 65 and under *or* 75 and over — the opposite of what’s usually intended for an age range.
- **Forgetting operator precedence.** Comparisons are evaluated before `&/|`, but when combining several conditions, parentheses make your intent explicit and prevent subtle bugs.

15.6. Practice Problem

A participant has `systolic_bp <- 145`. Create a logical object `has_elevated_bp` that is `TRUE` if `systolic_bp` is greater than or equal to 130.

15.7. Challenge Problem

A participant has `age <- 58` and `bmi <- 31.2`. Create a logical object `needs_followup` that is `TRUE` only if the participant is *both* under 65 *and* has a BMI of 30 or above (using `&`), then create a second object `either_condition` using `|` instead, and explain in one sentence why the two results differ.

15.8. Key Takeaways

- Logical values are `TRUE` or `FALSE` (unquoted); comparison operators (`==`, `!=`, `>`, `<`, `>=`, `<=`) produce them.
- `=` assigns a value; `==` checks equality — mixing these up is one of the most common beginner errors.
- `&` (and), `|` (or), and `!` (not) combine or invert logical values, and parentheses help make combined conditions unambiguous.

Knowledge Check

1. **Multiple choice.** What does `age == 65` do?
 - a) Assigns 65 to age
 - b) Checks whether age equals 65, returning `TRUE` or `FALSE`
 - c) Rounds age to 65
 - d) Creates a character string

15. Logical Data

2. **True or false.** "TRUE" and TRUE are the same value in R.
3. **Predict the output.**

```
x <- 10  
y <- 20  
(x < y) & (y < 30)
```

4. **Find the error.**

```
is_adult <- age = 18
```

Show answers

1. **b)** Checks whether `age` equals 65, returning TRUE or FALSE.
2. **False.** "TRUE" (quoted) is character text; TRUE (unquoted) is logical — they have different classes.
3. `[1] TRUE` — both `10 < 20` and `20 < 30` are true, and `&` requires both.
4. The code uses `=` where `==` was intended for comparison (and the logic is also reversed). It likely should read `is_adult <- age >= 18`.

16. Missing Values

The Problem

Three participants in your community health survey never reported their age — the field was simply left blank on the paper form. Your supervisor still wants the average age of all 100 participants. You wonder: do you just skip those three rows, treat the blank as zero, or something else entirely?

Your Mission

Understand how R represents missing data, and learn to detect it correctly before it silently distorts your calculations.

Think Before Coding

- Is a missing age the same thing as an age of zero? What would happen to an average if missing values were secretly treated as zero?
- If you don't know a value, should R be able to tell you whether one number equals another “unknown” value?

16.1. The R Concept

R represents a missing value with the special symbol **NA** (no quotes — it's not the text “NA”, it's a distinct, reserved value meaning “this data point does not exist”). **NA** can appear inside numeric, character, or logical data alike.

16. Missing Values

Critically, NA is *contagious* in calculations: almost any arithmetic involving an NA produces NA as the result, because R has no way to know what the missing value “would have been.”

```
5 + NA      # NA - unknown plus 5 is still unknown
mean(c(20, 30, NA))  # NA - one unknown value makes the whole average unknown
```

To detect missing values, use `is.na(x)` — never `x == NA`, which will not work the way you’d expect (comparing anything to NA, including NA itself, returns NA, not TRUE).

```
is.na(NA)    # TRUE
NA == NA     # NA, not TRUE - this is a common trap
```

16.2. Demonstration

```
age <- NA
age
```

```
[1] NA
```

```
is.na(age)
```

```
[1] TRUE
```

```
# NA is contagious in arithmetic
age + 1
```

```
[1] NA
```

```
# The trap: don't use == to check for NA
age == NA
```

```
100
```

16.3. Explore

```
[1] NA
```

```
ages <- c(45, 62, NA, 38, 29)
mean(ages)
```

```
[1] NA
```

16.3. Explore

Run `sum(is.na(ages))` on the vector above — what do you think this counts? (Hint: `is.na()` produces `TRUE/FALSE` for each element, and `TRUE` behaves like 1 when summed.) Then try `mean(ages, na.rm = TRUE)` — you'll study `na.rm` properly on the very next page, but see what changes.

16.4. Solve the Problem

```
ages <- c(45, 62, NA, 38, 29, NA, 51, NA, 67, 33) # 3 missing, matching the problem
```

```
# How many are missing?
sum(is.na(ages))
```

```
[1] 3
```

```
# The naive average fails, exactly as predicted
mean(ages)
```

```
[1] NA
```

16. Missing Values

You’ve now correctly identified that three participants have missing ages, and confirmed that a plain `mean()` call refuses to guess an answer around them — which is the right, safe default behavior. The actual fix for computing a usable average despite the missing values is covered on the very next page, using `mean()`’s `na.rm` argument.

16.5. Interpret the Result

`sum(is.na(ages))` returning 3 confirms exactly three participants have missing age data, matching what your supervisor described. `mean(ages)` returning NA is not a bug — it’s R correctly refusing to compute an average when it doesn’t have complete information, forcing you to make an explicit, informed decision about how to handle the gap.

Common Mistakes

- **Treating missing as zero.** A missing age is not the same as an age of 0 — silently substituting zero would badly distort any average or model built on that variable.
- **Using `== NA` instead of `is.na()`.** `age == NA` always returns NA, never TRUE, even when `age` genuinely is NA — this is one of the most common and confusing beginner traps in R.
- **Assuming NA always causes an error.** It doesn’t — R happily computes with NA and simply propagates it forward as an “unknown” result, which can silently hide a missing-data problem if you’re not specifically checking with `is.na()`.
- **Ignoring warnings when data is imported.** Real datasets often encode missingness inconsistently (blank cells, "999", "N/A" as text) — recognizing and converting these to real NA values is a recurring theme in Level 3 and Level 4.

16.6. Practice Problem

Given `bp_readings <- c(120, 135, NA, 142, NA, 128)`, use `is.na()` and `sum()` to count how many blood pressure readings are missing.

16.7. Challenge Problem

Given `smoker <- c("Yes", "No", NA, "Yes", NA)`, confirm with `is.na()` which positions are missing, then explain why `smoker == NA` would not have correctly identified them, even though `smoker` contains genuine NA values.

16.8. Key Takeaways

- NA represents a genuinely missing value and is distinct from zero, an empty string, or any other placeholder.
- NA is contagious — most calculations involving it return NA, which is a safety feature, not a bug.
- Always use `is.na()` to detect missing values; `== NA` will not work correctly.

Knowledge Check

1. **Multiple choice.** What does `is.na(x)` return when `x` is NA?
 - a) NA
 - b) TRUE
 - c) FALSE
 - d) An error
2. **True or false.** `NA == NA` returns TRUE.

16. Missing Values

3. Predict the output.

```
x <- c(10, NA, 20)
sum(is.na(x))
```

4. Find the error.

```
if (age == NA) {
  print("Missing age")
}
```

Show answers

1. **b)** TRUE — `is.na()` is specifically designed to correctly detect NA values.
2. **False.** Comparing NA to anything, including another NA, returns NA, not TRUE — this is why `is.na()` exists.
3. `[1] 1` — exactly one NA is present in the vector.
4. `age == NA` will never evaluate to TRUE, even if `age` is missing. It should be `if (is.na(age))`.

17. Basic Functions

The Problem

You are working as a public health intern. Your supervisor hands you a list of the ages of 100 study participants and asks: “Can you calculate the average age for me? I need it for the report by end of day.” You have never used R before, and you’re staring at a list of 100 numbers wondering how you’re supposed to add them all up and divide by 100 without doing it by hand.

Your Mission

Calculate and report the mean age of the study participants using R — correctly, even though a few participants didn’t report their age.

Think Before Coding

- What information do you actually need to answer this question — where would the ages be stored?
- How would you calculate a mean manually, in plain arithmetic terms (what do you add, and what do you divide by)?
- If R already has tools built to do exactly this, would you rather write that arithmetic out by hand, or ask R to do it directly?

17.1. The R Concept

A **function** in R is a named, reusable set of instructions that takes some input (called **arguments**), does something with it, and returns a result.

17. Basic Functions

You’ve already used a few without a formal introduction — `class()`, `is.na()`, `paste()`, `round()` — every one of those is a function.

The general pattern is always:

```
function_name(argument1, argument2, ...)
```

Today’s function is `mean()`, which calculates the arithmetic average of a set of numbers:

```
mean(x)
```

Here, `x` is the argument — the numbers you want averaged, typically stored as a **vector** (a single object holding multiple values, created with `c()` — you’ll study vectors properly in Level 2, but you need a first taste of them right now to solve this problem).

```
ages <- c(34, 51, 29, 62)  # a vector holding four ages
mean(ages)
```

Common mistakes with functions in general: forgetting the parentheses (`mean` alone just shows you the function’s code, it doesn’t run it), mismatching parentheses, and — as you’re about to see — not knowing about optional arguments that change a function’s behavior, like `na.rm`.

17.2. Demonstration

```
# A small vector of ages, standing in for your 100 participants
ages <- c(34, 51, 29, 62, 45)
```

```
mean(ages)
```

```
[1] 44.2
```

17.3. Explore

Now, what happens with missing data, as flagged on the previous page?

```
ages_with_missing <- c(34, 51, NA, 62, 45)
mean(ages_with_missing)
```

```
[1] NA
```

17.3. Explore

Add a second NA to `ages_with_missing` and rerun `mean()` — does the result change, or is it still simply NA? Now look up `mean()`'s help page (you'll learn the proper way to do this on the very next page) and notice it lists an argument called `na.rm`, short for “NA remove,” which defaults to `FALSE`.

```
mean(ages_with_missing, na.rm = TRUE)
```

```
[1] 48
```

Setting `na.rm = TRUE` tells `mean()` to exclude any NA values before calculating — averaging only the participants who actually reported an age, rather than refusing to answer at all.

17.4. Solve the Problem

Your supervisor's 100 participants, represented here as a smaller stand-in vector for demonstration (in Level 3, you'll import the real, full 100-row file directly):

17. Basic Functions

```
# Ages of study participants - a few did not report their age
participant_ages <- c(45, 62, 38, NA, 29, 71, 54, NA, 33, 48)

mean(participant_ages, na.rm = TRUE)

[1] 47.5

# How many participants were actually included in that average?
sum(!is.na(participant_ages))

[1] 8
```

You now have both numbers your supervisor actually needs: the mean age itself, and — just as important for an honest report — how many of the 100 participants that mean was actually based on.

17.5. Interpret the Result

The mean age of the study participants who reported their age was **47.5 years**, based on 8 of the 10 participants in this example (2 had missing age data and were excluded). In your final report, you would note this exclusion explicitly — for example: “Mean age was 47.5 years (n = 8; 2 participants had missing age data)” — rather than silently reporting a number without disclosing that some participants were left out of the calculation.

Common Mistakes

- **Forgetting `na.rm = TRUE` when data has missing values.** `mean(x)` alone silently returns `NA` the moment even a single value is missing — a frequent source of “why is my result `NA`?” confusion for beginners.

17.6. Practice Problem

- **Forgetting the parentheses entirely.** Typing `mean` without `()` shows you R's internal code for the function, rather than running it — always include the parentheses, even with no visible arguments inside.
- **Assuming `na.rm = TRUE` fixes the underlying data problem.** It only excludes missing values from *this specific calculation* — it doesn't fill them in, explain why they're missing, or make them not-missing elsewhere in your dataset.
- **Reporting a mean without reporting how many observations it's based on.** A mean of 47.5 means something different if it's based on 98 of 100 participants versus 8 of 100 — always report the sample size alongside any summary statistic.

17.6. Practice Problem

A second batch of participant systolic blood pressure readings is `bp <- c(118, 132, NA, 145, 128, NA, 121)`. Calculate the mean blood pressure, correctly excluding the missing values, and report how many readings that mean is based on.

17.7. Challenge Problem

Your supervisor now asks for the mean age *and* the mean systolic blood pressure, but adds: “Actually, I only want this for participants who don't have missing data in either variable — a complete-case analysis.” Using `participant_ages` and a matching-length `bp_readings <- c(118, 132, 140, NA, 128, 155, 121, 130, NA, 142)`, think through (in words, not yet code — this is coming properly in Level 4) what “removing a participant only if either value is missing” would actually require, beyond just calling `na.rm = TRUE` separately on each variable.

17.8. Key Takeaways

- A function takes arguments in parentheses and returns a result; `mean()` calculates the arithmetic average of a numeric vector.
- By default, `mean()` returns `NA` if any value is missing — a safety behavior, not a bug — and `na.rm = TRUE` tells it to exclude missing values instead.
- Always report the sample size a summary statistic is based on, especially when missing data caused some observations to be excluded.

Knowledge Check

1. **Multiple choice.** What does `na.rm = TRUE` do inside `mean()`?
 - a) Removes all values from the vector
 - b) Excludes missing values before calculating the mean
 - c) Converts missing values to zero
 - d) Rounds the result
2. **True or false.** `mean(c(10, 20, NA))` returns 15 by default (without `na.rm`).
3. **Predict the output.**

```
x <- c(5, 10, NA, 15)
mean(x, na.rm = TRUE)
```
4. **Write the code.** Given `readings <- c(22, NA, 28, 31, NA)`, write the code to calculate the mean, correctly excluding missing values, and separately count how many readings were missing.

Show answers

1. **b)** Excludes missing values before calculating the mean.
2. **False.** By default, `mean()` returns `NA` if any value in the vector is missing; it does not silently skip it.

17.8. Key Takeaways

3. `[1]` 10 — the mean of 5, 10, 15 (the NA is excluded).
4. `mean(readings, na.rm = TRUE)`
`sum(is.na(readings))`

18. Getting Help in R

The Problem

You just learned that `mean()` has an argument called `na.rm`. But you also notice `round()` takes a second argument you've been supplying without really understanding — `round(bmi, 1)` — and you're not sure what that 1 actually controls, or what other arguments `round()` might accept that you don't know about yet.

Your Mission

Learn to use R's built-in help system to answer your own questions about any function, without needing to search the internet or ask a colleague first.

Think Before Coding

- If a function's documentation were built directly into R, what symbol might you type before a function's name to pull it up?
- What information would you expect a function's help page to contain — just examples, or something more structured?

18.1. The R Concept

Every function that ships with R (and every well-made package function) has a **help page** describing exactly what it does. Three ways to access it:

18. Getting Help in R

```
?round      # opens the help page for round() in the Help pane
help(round)   # identical to ?round
args(round)   # shows just the function's arguments, without the full page
```

A typical help page has a consistent structure worth learning to skim:

- **Description** — a short summary of what the function does.
- **Usage** — the function's exact syntax, including argument names and their default values.
- **Arguments** — an explanation of each argument.
- **Value** — what the function returns.
- **Examples** — runnable code demonstrating typical use, at the very bottom (often the fastest place to start reading).

For functions with special characters in their name (like `if` or `+`), you need to wrap the name in backticks or quotes: `?`+`` or `help("if")`.

18.2. Demonstration

```
?round
```

Opens the help page in the Help pane, showing (abbreviated):

```
round(x, digits = 0)
```

Arguments:

```
x          a numeric vector.
```

```
digits     integer indicating the number of decimal places to round to.
```

```
args(round)
```

```
function (x, digits = 0)
```

```
NULL
```

This confirms: `round()`'s second argument is called `digits`, and it defaults to 0 if you don't supply it — which is exactly why `round(bmi, 1)` rounds to 1 decimal place, since 1 is being matched to the `digits` argument by its position.

18.3. Explore

Run `?mean` and scroll to the Arguments section. Confirm you can now see, in R's own documentation, the `na.rm` argument you used on the previous page, along with its default value. Then try `example(round)` — a function that runs the worked examples straight from the help page.

18.4. Solve the Problem

`?round`

Reading the help page's Usage section — `round(x, digits = 0)` — answers your question directly: the second argument to `round()` is named `digits`, and it controls how many decimal places the result is rounded to. `round(bmi, 1)` rounds `bmi` to 1 decimal place; had you written `round(bmi, 0)` (or just `round(bmi)`, relying on the default), it would round to the nearest whole number instead.

18.5. Interpret the Result

Knowing that `round()`'s second argument is specifically called `digits` — not just “some number you put there” — means you can now write clearer code by naming it explicitly: `round(bmi, digits = 1)`, which is more self-explanatory to a future reader than the shorter, positional `round(bmi, 1)`.

18. Getting Help in R

Common Mistakes

- **Jumping to a search engine before checking `?function`.** Built-in help is often faster and always version-matched to the R you're actually running — search results may reference an outdated or different version of a function.
- **Being intimidated by a help page's dense formatting.** Skip straight to the Examples section at the bottom first — it's usually the fastest way to understand a function's practical use before reading the more formal Arguments section.
- **Forgetting quotes or backticks for special names.** `?if` alone will error; you need `?`if`` or `help("if")`.
- **Not realizing argument defaults are shown in Usage.** `round(x, digits = 0)` tells you `digits` defaults to 0 if you don't specify it — this is often the fastest way to discover a function's default behavior.

18.6. Practice Problem

Use `?sum` to answer: does `sum()` have an `na.rm` argument, similar to `mean()`? Confirm your answer by reading the Arguments section.

18.7. Challenge Problem

You encounter an unfamiliar function, `quantile()`, in a colleague's script. Use `?quantile` to identify what its default argument (besides `x`) is called, and what value it defaults to when not specified.

18.8. Key Takeaways

- `?function_name` (or `help(function_name)`) opens R's built-in documentation in the Help pane.
- A help page's Usage section shows argument names and their defaults; the Examples section (often the fastest starting point) shows runnable code.
- Checking built-in help first is usually faster and more reliable than searching the web, especially for base R functions.

Knowledge Check

1. **Multiple choice.** Which command opens the help page for `sd()`?
 - a) `sd?`
 - b) `?sd`
 - c) `help sd`
 - d) `sd.help()`
2. **True or false.** A function's help page always lists its default argument values, if it has any.
3. **Predict the output.** What section of a help page would you check to find out what a function *returns*?
4. **Write the code.** Write the command to view the help page for the function `paste`, using the `help()` form rather than `?`.

Show answers

1. **b) `?sd`**
2. **True.** The Usage section of a help page shows every argument along with any default value it has.
3. The **Value** section describes what the function returns.
4. `help(paste)`

19. Working Directories

The Problem

Your supervisor emails you the full 100-participant survey file, `participants.csv`, and says “just read it in.” You try:

```
participants <- read.csv("participants.csv")
```

```
Error in file(file, "rt") : cannot open the connection  
In addition: Warning message:  
cannot open file 'participants.csv': No such file or directory
```

The file is right there on your computer — you can see it in File Explorer or Finder — so why can’t R find it?

Your Mission

Understand what a working directory is, check yours, and fix a file-not-found error caused by R looking in the wrong folder.

Think Before Coding

- If you tell someone “open the file called `report.docx`” without telling them which folder to look in, what would they need to assume?
- What folder do you think R is currently “standing in,” by default, when you give it a filename with no folder path attached?

19. Working Directories

19.1. The R Concept

Your **working directory** is the folder R currently treats as its “home base” — when you give a filename with no additional path information, R looks for it there first. If the file isn’t in that exact folder, you get a “cannot open the connection” / “No such file or directory” error, even though the file genuinely exists somewhere on your computer.

Two essential functions:

```
getwd()      # shows your current working directory
setwd("/path/to/folder") # changes your working directory
```

Once your working directory matches the folder your file is actually in, a plain filename like `"participants.csv"` will work. If you’d rather not change the working directory, you can instead give R the file’s full (absolute) path, or a path relative to your current working directory (like `"data/participants.csv"` if the file sits inside a `data` subfolder of your working directory).

i A better long-term fix

Manually managing `setwd()` is fragile — it hardcodes a path that only works on *your* computer. Level 3 introduces **RStudio Projects**, which solve this properly by automatically setting the working directory to your project folder every time you open it, so your scripts work identically on any computer. For now, understanding `getwd()/setwd()` and relative paths is the essential foundation.

19.2. Demonstration

```
getwd()
```


19.3. Explore

```
[1] "/Users/jordan/Documents"

# The file actually lives in a subfolder called "data"
setwd("/Users/jordan/Documents/health-survey-project")
getwd()

[1] "/Users/jordan/Documents/health-survey-project"

# Now a relative path works, because "data" exists inside the current working directory
participants <- read.csv("data/participants.csv")
```

19.3. Explore

Run `getwd()` right now, in your own RStudio session, and note what it returns. Then try `list.files()` — a function that lists every file in your current working directory — to see whether the file you’re looking for is actually there.

19.4. Solve the Problem

```
getwd()
```

```
[1] "/Users/jordan/Documents"
```

The working directory was `Documents`, but `participants.csv` actually lives inside a `data` subfolder of the project folder — not directly inside `Documents`. Two valid fixes:

```
# Option 1: change the working directory to the project folder,
# then use a relative path to the data subfolder
setwd("/Users/jordan/Documents/health-survey-project")
participants <- read.csv("data/participants.csv")
```

19. Working Directories

```
# Option 2: leave the working directory alone, and give the full path directly.
participants <- read.csv("/Users/jordan/Documents/health-survey-project/data/
```

Either approach resolves the original error, because R now knows exactly which folder to look in.

19.5. Interpret the Result

Once `read.csv()` runs without error, `participants` now holds the actual survey data in R, ready to be inspected and analyzed — the “cannot open the connection” error is gone because R’s working directory (or the path you supplied) now correctly points to the folder containing the file.

Common Mistakes

- **Assuming R can “see” your whole computer.** R only looks in its current working directory by default — being able to see a file in File Explorer doesn’t mean R’s working directory is set to that same folder.
- **Hardcoding a path that only works on one computer.** `setwd("/Users/jordan/Documents/...")` will break the moment a colleague with a different username or operating system tries to run your script — a key reason Level 3 introduces RStudio Projects as a better long-term solution.
- **Forgetting forward slashes.** Even on Windows, R paths use forward slashes (/), not backslashes (\) — a raw backslash in an R string has special meaning and will cause errors or wrong behavior.
- **Confusing relative and absolute paths.** `"data/participants.csv"` (relative — depends on your current working directory) behaves very differently from `"/Users/jordan/.../data/participants.csv"` (absolute — always points to the same place, regardless of working directory).

19.6. Practice Problem

Suppose `getwd()` returns `"/Users/jordan/Documents"`, and your file `clinic_data.csv` actually lives at `"/Users/jordan/Documents/projects/clinic_data.csv"`. Write the `read.csv()` call using a *relative* path (not changing the working directory), and separately, write it using the *absolute* path.

19.7. Challenge Problem

A classmate's script works perfectly on their laptop but fails with a "cannot open the connection" error on yours, even though you're both using the identical script and you both have the exact same data file. Explain what's most likely different between your two setups, and propose the most robust long-term fix (hint: think about what changes between two different computers, and preview why Level 3's RStudio Projects might solve this permanently).

19.8. Key Takeaways

- The working directory is the folder R looks in by default for files given without a full path; check it with `getwd()`.
- `setwd()` changes the working directory, but hardcoded paths break across different computers — a fragile short-term fix.
- Relative paths (like `"data/participants.csv"`) depend on the current working directory; absolute paths (the full location) don't, but aren't portable across computers either.

Knowledge Check

1. **Multiple choice.** What does `getwd()` do?

19. Working Directories

- a) Downloads a file
 - b) Shows your current working directory
 - c) Creates a new working directory
 - d) Installs a package
2. **True or false.** If you can see a file in File Explorer or Finder, R can automatically find it regardless of its working directory.
3. **Predict the output.** If `getwd()` returns `"/Users/amy/project"` and a file lives at `"/Users/amy/project/data/survey.csv"`, what relative path would correctly load it with `read.csv()`?
4. **Find the error.**

```
setwd("C:\\Users\\Amy\\Documents")
```

Show answers

- 1. **b)** Shows your current working directory.
- 2. **False.** R only looks in its current working directory by default, regardless of what's visible in a file browser.
- 3. `read.csv("data/survey.csv")`
- 4. The path uses backslashes (`\`), which have special meaning in R strings. It should use forward slashes: `setwd("C:/Users/Amy/Documents")`.

20. Importing Simple Data

The Problem

Everything in this level so far has used small, hand-typed vectors as stand-ins for real data. But your supervisor's actual request — the one that opened this entire level — was about a genuine file: `data/participants.csv`, a community health survey of 100 real participants. It's time to stop practicing on toy examples and finally import and analyze the real thing.

Your Mission

Import `data/participants.csv` into R, verify it loaded correctly, and finally calculate and report the actual mean age of the 100 study participants your supervisor asked for on day one.

Think Before Coding

- Based on the working-directory page, what do you need to confirm before R can find this file?
- Based on the missing-values page, what do you expect might happen if you call `mean()` directly on the imported age column, without checking for NA first?
- What would you want to check *immediately* after importing a file, before trusting any calculation based on it?

20. Importing Simple Data

20.1. The R Concept

`read.csv()` is a base R function that reads a CSV (comma-separated values) file and returns it as a **data frame** — R's standard structure for rectangular, spreadsheet-like data (rows and columns; you'll study data frames properly in Level 2, but you need one to hold this dataset today).

```
my_data <- read.csv("path/to/file.csv")
```

A few defaults worth knowing right away: `read.csv()` assumes the file uses a comma as its separator and, by default, treats the first row as column headers (`header = TRUE`) — this is different from the more general `read.table()`, whose default is `header = FALSE`, a common source of confusion if you ever switch between the two functions.

Once imported, three functions help you sanity-check what you just loaded:

```
head(my_data)      # previews the first 6 rows
str(my_data)       # shows the structure: column names, types, and a peek at v
nrow(my_data)      # counts the rows
```

To access a single column of a data frame by name, use the `$` operator — `my_data$column_name` — which you'll use constantly from this point forward.

20.2. Demonstration

```
participants <- read.csv("data/participants.csv")

head(participants)
```

20.3. Explore

```
  id age   sex   county systolic_bp diastolic_bp  bmi smoker
1  1  46 Female Oakford      125          79 35.1    No
2  2  53 Female Riverdale    123          80 27.7    No
3  3  59 Female Oakford      130          77 28.3    No
4  4  49 Female Millbrook    117          72 30.4    No
5  5  61 Female Millbrook    139          79 21.5    No
6  6  18 Female Riverdale    117          64 24.0    No
```

```
str(participants)
```

```
'data.frame':  100 obs. of  8 variables:
 $ id          : int  1 2 3 4 5 6 7 8 9 10 ...
 $ age         : int  46 53 59 49 61 18 62 68 33 36 ...
 $ sex         : chr  "Female" "Female" "Female" "Female" ...
 $ county      : chr  "Oakford" "Riverdale" "Oakford" "Millbrook" ...
 $ systolic_bp : int  125 123 130 117 139 117 110 100 149 NA ...
 $ diastolic_bp: int  79 80 77 72 79 64 64 53 88 86 ...
 $ bmi         : num  35.1 27.7 28.3 30.4 21.5 24 19.7 29.7 22.4 NA ...
 $ smoker      : chr  "No" "No" "No" "No" ...
```

20.3. Explore

Run `nrow(participants)` — does it match the “100 study participants” your supervisor described? Then run `sum(is.na(participants$age))` to count how many participants have a missing age, connecting straight back to what you learned on the missing-values page.

20.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")
```

20. Importing Simple Data

```
# Sanity checks first, always  
nrow(participants)
```

```
[1] 100
```

```
sum(is.na(participants$age))
```

```
[1] 6
```

```
# Now, finally: the calculation your supervisor asked for on day one  
mean(participants$age, na.rm = TRUE)
```

```
[1] 47.42553
```

```
round(mean(participants$age, na.rm = TRUE), 1)
```

```
[1] 47.4
```

20.5. Interpret the Result

The mean age of the study participants was **47.4 years**, based on the 94 participants (of 100 enrolled) who reported their age — 6 participants had missing age data and were correctly excluded via `na.rm = TRUE` rather than silently treated as zero or causing the whole calculation to fail. A complete answer to your supervisor would read: “*The mean age of study participants was 47.4 years ($n = 94$ of 100; 6 participants had missing age data).*” That single sentence closes the exact problem you opened this entire level with — and every concept along the way (objects, functions, data types, missing values, working directories) was a piece you needed to get here.

Common Mistakes

20.6. Practice Problem

- **Skipping the sanity checks.** Jumping straight to `mean()` without first confirming `nrow()` matches expectations, or checking for NAs, risks reporting a confidently wrong number.
- **Forgetting `na.rm = TRUE` after import.** This is the exact trap from the missing-values page, now showing up with real, messier data — always check `sum(is.na(...))` on a new dataset before trusting any summary statistic built from it.
- **Wrong working directory.** If `read.csv("data/participants.csv")` fails with “cannot open the connection,” revisit the working-directory page — the fix is almost always confirming `getwd()` and the file’s actual location relative to it.
- **Not previewing before analyzing.** Skipping `head()` and `str()` means you might not notice a column was imported as the wrong type (a common issue explored further in Level 3 and Level 4) before it silently affects your results.

20.6. Practice Problem

Using the same `participants` data frame, calculate the mean systolic blood pressure (`systolic_bp`), correctly handling any missing values, and report how many participants that mean is based on.

20.7. Challenge Problem

Calculate the mean BMI (`bmi`) for the dataset, then separately calculate what percentage of the 100 participants are current smokers (hint: `sum(participants$smoker == "Yes")` counts smokers, and you already know how to turn a count into a percentage from the arithmetic page). Write both results as a sentence you could put directly into a report.

20.8. Key Takeaways

- `read.csv()` imports a CSV file into a data frame; always follow it with `head()`, `str()`, and `nrow()` to confirm it loaded as expected.
- The `$` operator (`data$column`) accesses a single column of a data frame — you'll use this constantly going forward.
- Real datasets almost always contain missing values — checking for them and handling them explicitly (`na.rm = TRUE`) is not optional, it's the baseline standard for honest reporting.

Knowledge Check

1. **Multiple choice.** What does `read.csv()` return?
 - a) A single number
 - b) A character vector
 - c) A data frame
 - d) A logical value
2. **True or false.** `read.csv()` assumes the first row of the file contains column names by default.
3. **Predict the output.** If `participants` has 100 rows and 6 participants have a missing `age`, what would `sum(!is.na(participants$age))` return?
4. **Write the code.** Write the full sequence of code to import `data/participants.csv`, check how many rows it has, and calculate the mean `bmi`, correctly excluding missing values.

Show answers

1. **c)** A data frame.
2. **True.** `read.csv()` defaults to `header = TRUE`, treating the first row as column names (unlike `read.table()`, which defaults to `header = FALSE`).

20.9. Where you've arrived

3. `[1] 94` — the count of participants whose age is *not* missing (100 total minus 6 missing).
4.

```
participants <- read.csv("data/participants.csv")
nrow(participants)
mean(participants$bmi, na.rm = TRUE)
```

20.9. Where you've arrived

You started this level not knowing what R even was. You've now installed and loaded a package, written and commented a saved script, learned R's core data types and how missing values behave, used a built-in function with an optional argument, looked up documentation yourself, fixed a working-directory error, and imported a genuine 100-participant dataset to answer the exact question your supervisor asked on the very first page of this level.

That's not a coincidence — it's the whole idea of this course. Level 2 picks up right where this leaves off: you'll go far deeper into data frames, vectors, and the other structures R uses to hold real research data, using this same `participants.csv` file throughout.

Part II.

Level 2: Data Structures in R

Level 2 — Data Structures in R

Where this level is headed

In Level 1 you worked with single values — one age, one BMI, one label. But `participants.csv` holds 100 rows and 8 columns at once, and your supervisor’s questions keep needing you to work with *many* values together: every participant’s age, just the smokers’ blood pressure, one participant’s entire record. This level builds the containers R gives you for that — vectors, factors, data frames, matrices, lists, and tibbles — and the tools to reach into exactly the piece you need.

What you’ll build toward

By the end of this level you’ll be comfortable creating and indexing vectors, converting messy category columns into well-ordered factors, building and slicing data frames by row and column, understanding matrices and lists as R’s other core structures, and using `str()`, dimension functions, `length()`, and `names()` to orient yourself in any object — familiar or brand new.

Every page keeps using the same running dataset from Level 1: `data/participants.csv`, the 100-participant community health survey.

Topics in this level

- Vectors and `c()`
- Sequences with `seq()` and repetition with `rep()`
- Positive, negative, and logical indexing
- Named vectors
- Factors for categorical data
- Data frames: creating them, and accessing rows and columns
- Matrices and lists
- Tibbles
- Inspecting structure with `str()`, `dimensions`, `length()`, and `names()`

By the last page, you'll be able to look at any R object — a vector, a factor, a data frame, a list, a matrix you've never seen before — and quickly answer “what is this, and how big is it?” without guessing.

21. Vectors and `c()`

The Problem

You need to record the systolic blood pressure readings for five participants seen this morning: 118, 132, 145, 121, and 138. Following Level 1's habits, you consider creating five separate objects — `bp1`, `bp2`, `bp3`, `bp4`, `bp5` — but you already sense this won't scale. What happens when you have 100 participants, and you want their mean, all at once?

Your Mission

Store multiple related values in a single object, so you can work with all of them together instead of one at a time.

Think Before Coding

- If you had five separate objects `bp1` through `bp5`, could you compute their mean with `mean()` directly, the way you did in Level 1?
- What would it mean for a single object to hold five numbers instead of just one?

21.1. The R Concept

A **vector** is R's most fundamental data structure: a single object that holds one or more values of the *same type* (all numeric, all character, or all logical — never mixed, as you'll see below). In fact, every column you saw in `participants` back in Level 1 — `age`, `sex`, `bmi` — is itself a vector. Even a single number like `age <- 34` is technically a vector of length one;

21. Vectors and `c()`

you’ve been using vectors since your very first line of R code without a name for it yet.

You build a vector explicitly with `c()` (short for “combine” or “concatenate”):

```
c(value1, value2, value3, ...)
```

```
bp_readings <- c(118, 132, 145, 121, 138)
```

Once created, a vector behaves as a single object you can pass to any function — `mean()`, `sum()`, `length()` — and R applies the operation across every value inside it.

i What happens if you mix types?

`c(1, "two", 3)` doesn’t error — instead, R silently converts every element to the same type (here, character: `"1"`, `"two"`, `"3"`), a behavior called **coercion**. This is a common source of confusing bugs: mixing a number into a vector of mostly text (or vice versa) doesn’t fail loudly, it just quietly changes what you have.

21.2. Demonstration

```
bp_readings <- c(118, 132, 145, 121, 138)
bp_readings
```

```
[1] 118 132 145 121 138
```

```
class(bp_readings)
```

```
[1] "numeric"
```

```
138
```

```
length(bp_readings)
```

```
[1] 5
```

```
mean(bp_readings)
```

```
[1] 130.8
```

21.3. Explore

Create a character vector of five county names: `c("Riverdale", "Oakford", "Millbrook", "Riverdale", "Cedar Falls")`. Run `class()` on it to confirm it's "character". Then try `c(118, "high", 145)` and run `class()` on the result — notice that every element, including the numbers, has been coerced to character.

21.4. Solve the Problem

```
bp_readings <- c(118, 132, 145, 121, 138)
```

```
mean(bp_readings)
```

```
[1] 130.8
```

```
max(bp_readings)
```

```
[1] 145
```

Five separate objects (`bp1` through `bp5`) would have made this calculation painful and error-prone; one vector makes it a single line of code, and this exact approach scales identically whether you have 5 readings or 5,000.

21.5. Interpret the Result

The mean systolic blood pressure across these five participants was 130.8 mmHg, with a maximum reading of 145 mmHg. Because all five values live in one vector, any summary function you already know from Level 1 works on all of them at once — no repetition required.

Common Mistakes

- **Forgetting `c()` entirely.** `bp_readings <- 118, 132, 145` is invalid syntax — you must wrap multiple values in `c(...)`.
- **Not noticing silent coercion.** Mixing a number into mostly-text values (or vice versa) doesn't error; it quietly converts everything to one type, which can break downstream calculations without an obvious warning.
- **Assuming vectors can hold mixed types like a spreadsheet row can.** A vector is uniform by design — if you need to hold genuinely different types together (a name *and* an age *and* a flag), you want a list or a data frame instead, both covered later in this level.

21.6. Practice Problem

Create a numeric vector called `ages` holding the ages 45, 62, 38, 71, and 29. Confirm its class and length, then calculate its mean.

21.7. Challenge Problem

Create a character vector called `smoking_status` holding "Yes", "No", "No", "Yes", "No" for the same five participants. Then create a logical vector `is_smoker` from it using `smoking_status == "Yes"` (a technique

from Level 1's logical-data page). Confirm the class of each vector you created.

21.8. Key Takeaways

- A vector holds one or more values of the same type; `c()` combines individual values into one.
- Every column in a data frame, and even a single value, is technically a vector — the concept is foundational to everything else in R.
- Mixing types inside `c()` silently coerces every element to one common type, rather than raising an error.

Knowledge Check

1. **Multiple choice.** What does `c(4, 8, 15)` create?
 - a) Three separate objects
 - b) A numeric vector of length 3
 - c) A character vector
 - d) A data frame
2. **True or false.** A vector can hold both numbers and text without any conversion happening.
3. **Predict the output.**

```
x <- c(TRUE, FALSE, TRUE)
class(x)
```

4. **Find the error.**

```
ages <- 45, 62, 38
```

Show answers

21. Vectors and `c()`

1. **b)** A numeric vector of length 3.
2. **False.** Mixing types causes coercion — every element becomes the same type (usually character, if any text is present).
3. `[1] "logical"`
4. Missing `c()` — it should be `ages <- c(45, 62, 38)`.

22. Sequences with `seq()`

The Problem

You want to assign a unique participant ID to each of the 100 rows in your survey — 1 through 100 — and separately, you need age-group cutpoints for a chart: 0, 10, 20, 30, all the way to 90. Typing `c(1, 2, 3, 4, 5, ..., 100)` by hand would take forever and invite typos.

Your Mission

Generate a sequence of numbers automatically, without typing every value individually.

Think Before Coding

- If you only need every whole number from 1 to 100, is there a shorter way to say that than listing all 100 values?
- If you instead need every 10th number from 0 to 90, what two pieces of information would you need to specify — a starting point, an ending point, and what else?

22.1. The R Concept

The colon operator `:` is the fastest way to generate a sequence of whole numbers stepping by 1:

```
1:100      # every integer from 1 to 100
10:1       # counts backward: 10, 9, 8, ..., 1
```

22. Sequences with `seq()`

For more control — a different step size, or a specific number of evenly spaced values — use `seq()`:

```
seq(from, to, by = ...)          # step by a fixed amount
seq(from, to, length.out = ...)  # produce a fixed number of evenly spaced values

seq(0, 90, by = 10)              # 0, 10, 20, ..., 90
seq(0, 1, length.out = 5)       # 0, 0.25, 0.5, 0.75, 1 - 5 evenly spaced values
```

22.2. Demonstration

```
participant_id <- 1:100
length(participant_id)

[1] 100

head(participant_id)

[1] 1 2 3 4 5 6

age_breaks <- seq(0, 90, by = 10)
age_breaks

[1] 0 10 20 30 40 50 60 70 80 90
```

22.3. Explore

Try `seq(0, 100, by = 15)` — since 100 isn't evenly divisible by 15, where does the sequence actually stop? Then try `seq(0, 100, length.out = 5)` and notice it guarantees exactly 5 values, adjusting the step size to fit, rather than guaranteeing a specific step size.

22.4. Solve the Problem

```
# Participant IDs, 1 through 100
participant_id <- 1:100

# Age-group cutpoints for a chart
age_breaks <- seq(0, 90, by = 10)
age_breaks

[1] 0 10 20 30 40 50 60 70 80 90
```

Both sequences were generated in a single line each, with zero risk of a typo or a skipped number partway through 100 values.

22.5. Interpret the Result

`participant_id` now holds exactly the numbers 1 through 100, ready to be attached as an ID column; `age_breaks` holds the ten cutpoints (0 through 90, in steps of 10) you'd feed into a histogram or age-grouping function later in the course — both produced reliably from a short, readable line of code rather than manual typing.

Common Mistakes

- **Assuming `seq()` always lands exactly on to.** `seq(0, 100, by = 15)` stops at 90 (the last multiple of 15 that doesn't exceed 100) rather than forcing 100 to appear — check the actual output rather than assuming.
- **Confusing `by` and `length.out`.** `by` fixes the step size (and lets the count of values vary); `length.out` fixes the count of values (and lets the step size vary) — mixing these up gives a very different result than intended.

22. Sequences with `seq()`

- **Using `:` when a non-integer step is needed.** `1:5` only steps by whole numbers; for something like 0, 0.5, 1, 1.5, you need `seq(0, 1.5, by = 0.5)`, not `:`.

22.6. Practice Problem

Create a sequence of even numbers from 2 to 20 using `seq()`. Then create the same sequence a second way using `seq(from, to, length.out = ...)`, and confirm both approaches give identical results.

22.7. Challenge Problem

You need BMI category cutpoints at 18.5, 25, 30, and 40 — but these aren't evenly spaced, so neither `:` nor `seq()` can generate them automatically. Write the `c()` call that creates this exact vector, and explain in one sentence why `seq()` wouldn't be the right tool here.

22.8. Key Takeaways

- `:` quickly generates a sequence of whole numbers stepping by 1 (or -1 , counting backward).
- `seq()` gives more control: `by` sets a fixed step size, `length.out` sets a fixed number of evenly spaced values.
- Irregularly spaced values (like clinical cutpoints that don't follow a fixed pattern) still require `c()`, not `seq()`.

Knowledge Check

1. **Multiple choice.** What does `1:5` produce?

22.8. Key Takeaways

- a) `c(1, 5)`
 - b) `c(1, 2, 3, 4, 5)`
 - c) `c(5, 4, 3, 2, 1)`
 - d) An error
2. **True or false.** `seq(0, 100, by = 30)` always includes exactly the number 100 in its output.
3. **Predict the output.**
- ```
seq(5, 25, by = 5)
```
4. **Write the code.** Write the code to generate exactly 4 evenly spaced values between 0 and 30, inclusive.

Show answers

- 1. **b)** `c(1, 2, 3, 4, 5)`
- 2. **False.** `seq()` with `by` stops at the last value that doesn't exceed to — 100 is only included if it's an exact multiple of the step from the starting point (here, 0, 30, 60, 90 — 100 is not reached).
- 3. `[1] 5 10 15 20 25`
- 4. `seq(0, 30, length.out = 4)`, which produces 0, 10, 20, 30.



## 23. Repeating Values with `rep()`

### The Problem

Your study enrolled participants at two clinics: the first 50 rows in your dataset came from the Riverdale clinic, and the last 50 came from the Oakford clinic. You need a `site` label for every one of the 100 rows — but typing "Riverdale" fifty times and "Oakford" fifty times is exactly the kind of repetitive task you'd hope R could handle for you.

### Your Mission

Generate a vector of repeated values automatically, without typing each repetition individually.

### Think Before Coding

- Is “Riverdale, Riverdale, Riverdale, ..., Oakford, Oakford, Oakford” the same pattern as “Riverdale, Oakford, Riverdale, Oakford, ...”? Which one matches your actual dataset?
- What two pieces of information would you need to describe a repeating pattern: what to repeat, and what else?

### 23.1. The R Concept

`rep()` repeats a value or vector according to one of two different logics, and the difference between them matters a great deal:

### 23. Repeating Values with `rep()`

```
rep(x, times = n) # repeats the WHOLE vector n times, in sequence
rep(x, each = n) # repeats EACH element n times before moving to the next

rep(c("A", "B"), times = 3) # "A" "B" "A" "B" "A" "B"
rep(c("A", "B"), each = 3) # "A" "A" "A" "B" "B" "B"
```

Notice both calls use the same inputs but produce entirely different orderings — `times` repeats the *pattern*, `each` repeats *every element* before advancing.

## 23.2. Demonstration

```
rep(c("Riverdale", "Oakford"), times = 2)
```

```
[1] "Riverdale" "Oakford" "Riverdale" "Oakford"
```

```
rep(c("Riverdale", "Oakford"), each = 2)
```

```
[1] "Riverdale" "Riverdale" "Oakford" "Oakford"
```

```
Repeating a single value
rep("Enrolled", times = 5)
```

```
[1] "Enrolled" "Enrolled" "Enrolled" "Enrolled" "Enrolled"
```

## 23.3. Explore

Try `rep(c("Riverdale", "Oakford"), each = 2, times = 2)` — `each` and `times` can be combined. Predict the output before running it: does `each` or `times` get applied first?

## 23.4. Solve the Problem

```
50 participants from Riverdale, followed by 50 from Oakford
site <- rep(c("Riverdale", "Oakford"), each = 50)
```

```
length(site)
```

```
[1] 100
```

```
table(site)
```

```
site
Oakford Riverdale
 50 50
```

Using `each = 50` (not `times = 50`) correctly produces 50 consecutive "Riverdale" values followed by 50 consecutive "Oakford" values — matching exactly how the real dataset was collected, clinic by clinic.

## 23.5. Interpret the Result

`table(site)` confirms 50 participants are labeled "Oakford" and 50 are labeled "Riverdale", generated correctly in one line rather than 100 individual entries — and critically, in the right *order* to match the two-clinic-block structure of the actual data collection.

### Common Mistakes

- **Using `times` when you meant `each` (or vice versa).** `rep(c("A", "B"), times = 50)` alternates A/B one hundred times; `rep(c("A", "B"), each = 50)` produces fifty A's followed by fifty B's — these look similar in code but produce completely different data.

### 23. Repeating Values with `rep()`

- **Assuming `rep()` output length always matches your target automatically.** Always check `length()` after building a repeated vector, especially before attaching it as a new column to a data frame with a fixed number of rows.
- **Forgetting `rep()` only repeats an existing pattern.** If the real-world groups aren't evenly sized (say 62 Riverdale participants and 38 Oakford), a simple `rep(..., each = 50)` will be wrong — you'd need `rep(c("Riverdale", "Oakford"), times = c(62, 38))`, where `times` can also take a vector specifying how many times each element repeats individually.

#### 23.6. Practice Problem

Create a vector `dose_group` representing 3 participants assigned to "Placebo" and 3 assigned to "Treatment", in that block order (not alternating), using `rep()` with the appropriate argument.

#### 23.7. Challenge Problem

A study has 3 sites with uneven enrollment: 20 participants at Site A, 35 at Site B, and 45 at Site C (100 total). Use `rep()` with a vector passed to `times` (as hinted in the Common Mistakes box) to generate the full 100-length `site` vector in the correct block order.

#### 23.8. Key Takeaways

- `rep(x, times = n)` repeats the entire pattern `n` times; `rep(x, each = n)` repeats every individual element `n` times before moving to the next.



## 23.8. Key Takeaways

- These two produce very different orderings from the same inputs — always double-check which one matches your actual data structure.
- `times` can also accept a vector, letting each element repeat a different number of times — useful for unevenly sized groups.

### Knowledge Check

1. **Multiple choice.** What does `rep(c("A", "B"), each = 2)` produce?
  - a) "A" "B" "A" "B"
  - b) "A" "A" "B" "B"
  - c) "A" "A" "A" "B"
  - d) An error
2. **True or false.** `rep(x, times = n)` and `rep(x, each = n)` always produce the same output.
3. **Predict the output.**

```
rep(1:3, times = 2)
```

4. **Find the error.** A student wants 10 consecutive "Male" labels followed by 10 consecutive "Female" labels, but writes:

```
sex <- rep(c("Male", "Female"), times = 10)
```

Show answers

1. b) "A" "A" "B" "B"
2. **False.** `times` repeats the whole pattern in sequence; `each` repeats every element before moving on — they produce different orderings.
3. `[1] 1 2 3 1 2 3`
4. The student used `times` when they meant `each` — `rep(c("Male", "Female"), times = 10)` alternates Male/Female 10 times (20 values total, alternating), not 10 Males followed by 10 Females. It should be `rep(c("Male", "Female"), each = 10)`.



## 24. Positive Indexing

### The Problem

Your supervisor asks: “What was the age of the very first participant enrolled? And can you show me the ages of the first ten participants, so I can spot-check the data entry?” You have a 100-value **age** vector, but you’ve only ever printed the *whole* thing at once — you don’t yet know how to pull out just one value, or just a few, from a specific position.

### Your Mission

Extract specific elements from a vector by their position, using square brackets.

### Think Before Coding

- If someone asked you for “the 3rd item on this list,” how would you count to find it — starting from 0 or from 1?
- What if you wanted several positions at once, like the 1st, 2nd, and 3rd — could you reuse something you learned two pages ago (sequences) to describe “1 through 10”?

### 24.1. The R Concept

**Indexing** means extracting one or more elements from a vector using square brackets, `[ ]`, containing the position(s) you want:

```
x[position]
```

## 24. Positive Indexing

Critically, **R is 1-indexed** — the first element is at position 1, not 0 (unlike some other programming languages you may encounter later, such as Python). This is one of the most important structural facts to internalize early.

```
ages <- c(46, 53, 59, 49, 61)
ages[1] # the FIRST element - not the second
ages[3] # the third element
ages[1:3] # the first three elements, using the sequence range from the 1
```

You can also pass a vector of specific, non-consecutive positions: `ages[c(1, 3, 5)]` returns the 1st, 3rd, and 5th elements.

### 24.2. Demonstration

```
ages <- c(46, 53, 59, 49, 61, 18, 62, 68, 33, 36)
```

```
ages[1]
```

```
[1] 46
```

```
ages[1:10]
```

```
[1] 46 53 59 49 61 18 62 68 33 36
```

```
ages[c(2, 5, 9)]
```

```
[1] 53 61 33
```

## 24.3. Explore

Try `ages[11]` — one position past the end of a 10-element vector. What does R return, and why do you think it doesn't produce an error? Then try `ages[0]` — an unusual edge case worth seeing once.

## 24.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")
```

```
Age of the very first participant enrolled
participants$age[1]
```

```
[1] 46
```

```
Ages of the first ten participants
participants$age[1:10]
```

```
[1] 46 53 59 49 61 18 62 68 33 36
```

## 24.5. Interpret the Result

The first participant enrolled was 46 years old, and the first ten participants ranged across ages including 18, 33, and 68 — exactly the kind of quick spot-check your supervisor asked for, pulled directly from specific positions in the `age` column without scrolling through the entire 100-row dataset.

### Common Mistakes

## 24. Positive Indexing

- **Assuming R is 0-indexed.** `ages[0]` does not return the first element — it returns an empty vector of length zero, since position 0 doesn't correspond to any real element in R's 1-indexed system.
- **Using `()` instead of `[]`.** `ages(1)` is not valid indexing syntax — parentheses are for function calls, not for extracting elements from a vector.
- **Indexing past the end silently.** `ages[11]` on a 10-element vector doesn't error — it returns `NA`, since R interprets it as “the value at a position that doesn't exist.” This can silently introduce missing values into further calculations if you're not paying attention.

### 24.6. Practice Problem

Using `participants$bmi`, extract the BMI of the 15th participant, and separately, the BMIs of the first 5 participants.

### 24.7. Challenge Problem

Extract the ages of the 1st, 50th, and 100th participants in a single line of code (hint: you'll need `c()` inside the square brackets). Then explain, in one sentence, why `participants$age[1, 50, 100]` (with commas but no `c()`) would not work the way you might expect.

### 24.8. Key Takeaways

- Square brackets `[ ]` extract elements from a vector by position; R is 1-indexed, so the first element is at position 1.
- A range `(1:10)` or an arbitrary set of positions `(c(2, 5, 9))` can both be used inside `[ ]`.

- Indexing past the end of a vector doesn't error — it silently returns NA, which is worth watching for.

### Knowledge Check

1. **Multiple choice.** What does `x[1]` return for `x <- c(10, 20, 30)`?
  - a) 20
  - b) 10
  - c) An error, since indexing starts at 0
  - d) The entire vector
2. **True or false.** R is 0-indexed, like Python.
3. **Predict the output.**

```
x <- c(10, 20, 30, 40)
x[c(1, 4)]
```

4. **Find the error.**

```
ages(3)
```

Show answers

1. **b)** 10 — the first element, since R is 1-indexed.
2. **False.** R is 1-indexed; the first element is at position 1, not 0.
3. `[1] 10 40`
4. Parentheses are used for function calls, not indexing. It should be `ages[3]`.





## 25. Negative Indexing

### The Problem

Your supervisor tells you: “Actually, participant #1 was a pilot test entry from before the study officially started — exclude that one row from all your calculations, but keep the other 99.” You know how to pull *specific* elements out of a vector using positive indexing, but now you need the opposite: everything *except* one particular position.

### Your Mission

Exclude one or more elements from a vector by position, rather than selecting elements to include.

### Think Before Coding

- If positive numbers inside [ ] mean “give me these positions,” what might the opposite sign mean?
- If you needed to exclude both the 1st and 2nd elements, how might you extend a single negative index to several?

### 25.1. The R Concept

Putting a **minus sign** in front of a position inside [ ] tells R to exclude that position, returning everything else:

```
x[-position]
```

## 25. Negative Indexing

```
ages <- c(46, 53, 59, 49, 61)
ages[-1] # everything except the 1st element
ages[-c(1, 2)] # everything except the 1st and 2nd elements
```

This mirrors positive indexing exactly, just inverted — and it composes with everything you learned on the last page, including ranges: `ages[-(1:5)]` excludes the first five elements.

### 25.2. Demonstration

```
ages <- c(46, 53, 59, 49, 61, 18, 62, 68, 33, 36)
```

```
ages[-1]
```

```
[1] 53 59 49 61 18 62 68 33 36
```

```
ages[-c(1, 2)]
```

```
[1] 59 49 61 18 62 68 33 36
```

```
ages[-(1:3)]
```

```
[1] 49 61 18 62 68 33 36
```

### 25.3. Explore

Try mixing signs: `ages[c(-1, 2)]`. What happens? R will refuse and produce an error — predict what that error might say before you run it, based on the idea that “include position 2” and “exclude position 1” are contradictory instructions to combine in one call.

## 25.4. Solve the Problem

```

participants <- read.csv("data/participants.csv")

Exclude participant 1 (the pilot test entry) from the age calculation
valid_ages <- participants$age[-1]

length(valid_ages)

[1] 99

mean(valid_ages, na.rm = TRUE)

[1] 47.36735

```

## 25.5. Interpret the Result

`valid_ages` now holds 99 values instead of 100, correctly excluding the pilot test entry, and the mean age recalculated on this corrected set is 47.4 years (rounded) — very close to, but not identical to, the full-sample mean from Level 1, since removing one participant necessarily shifts the average slightly.

### Common Mistakes

- **Mixing positive and negative indices in one call.** `x[c(-1, 2)]` produces an error (can't mix positive and negative subscripts) — you must choose one approach or the other for a single indexing operation.
- **Forgetting parentheses around a range you want to negate.** `-1:3` is interpreted as “the sequence from  $-1$  to  $3$ ,” not “exclude positions 1 through 3” — you need `-(1:3)` with explicit parentheses to negate the whole range.

## 25. Negative Indexing

- **Assuming negative indexing modifies the original vector.** `ages[-1]` returns a *new* vector with one element excluded; it does not delete anything from `ages` itself unless you explicitly reassign the result back: `ages <- ages[-1]`.

### 25.6. Practice Problem

Using `participants$bmi`, create a vector `bmi_excluding_first_three` that excludes the first three participants' BMI values, using negative indexing with a range.

### 25.7. Challenge Problem

Your supervisor now says participants #1 and #100 were both pilot test entries — exclude both from `participants$age` in a single line of negative indexing, and compute the mean of the remaining 98 ages.

### 25.8. Key Takeaways

- A negative sign inside `[ ]` excludes that position (or positions) rather than selecting it.
- Positive and negative indices cannot be mixed in a single indexing call.
- Indexing (positive or negative) returns a new vector — it doesn't modify the original unless you explicitly reassign the result.

#### Knowledge Check

1. **Multiple choice.** What does `x[-2]` return for `x <- c(10, 20, 30)`?

## 25.8. Key Takeaways

- a) 20
- b) `c(10, 30)`
- c) `c(20, 30)`
- d) An error

2. **True or false.** `x[c(-1, 2)]` is valid R code that excludes position 1 and includes position 2.

3. **Predict the output.**

```
x <- c(5, 10, 15, 20, 25)
x[-(1:2)]
```

4. **Find the error.**

```
x[-1:3]
```

Show answers

- 1. **b)** `c(10, 30)` — position 2 (20) is excluded.
- 2. **False.** Mixing positive and negative indices in the same call raises an error.
- 3. `[1] 15 20 25`
- 4. Without parentheses, `-1:3` is read as the sequence from  $-1$  to 3, not “exclude 1 through 3.” It should be `x[-(1:3)]`.



## 26. Logical Indexing

### The Problem

Your supervisor asks: “What are the blood pressure readings just for the participants who smoke?” You don’t know which row positions correspond to smokers — position-based indexing (positive or negative) requires knowing exact row numbers in advance, and you don’t have that memorized for 100 participants. You need a way to select rows based on a *condition*, not a position.

### Your Mission

Extract exactly the elements of a vector that satisfy a condition, using a logical vector as an index.

### Think Before Coding

- From Level 1, what does a comparison like `smoker_status == "Yes"` produce — a number, or something else?
- If you had a `TRUE/FALSE` value for every single participant indicating whether they smoke, could you use that entire vector as the index inside `[ ]`, instead of a single number?

### 26.1. The R Concept

Square brackets don’t just accept positions — they also accept a **logical vector** of the same length as the vector being indexed. Wherever that

## 26. Logical Indexing

logical vector is `TRUE`, the corresponding element is kept; wherever it's `FALSE`, it's dropped.

```
x[logical_vector]
```

The logical vector is usually produced on the spot using a comparison, exactly as you learned in Level 1:

```
ages <- c(46, 53, 59, 49, 61)
ages[ages > 55] # keeps only elements where the condition is TRUE
ages[ages >= 50 & ages <= 60] # combining conditions with &, from Level 1
```

This is, in effect, the base-R version of “filtering” — the exact idea you’ll formalize with the tidyverse’s `filter()` function in Level 5, but built here from tools you already have.

### 26.2. Demonstration

```
ages <- c(46, 53, 59, 49, 61, 18, 62, 68, 33, 36)
```

```
ages > 55
```

```
[1] FALSE FALSE TRUE FALSE TRUE FALSE TRUE TRUE FALSE FALSE
```

```
ages[ages > 55]
```

```
[1] 59 61 62 68
```

```
ages[ages < 40]
```

```
[1] 18 33 36
```



## 26.3. Explore

Try `ages[ages > 55 & ages < 65]` — combining two conditions with `&`, from Level 1. Then try applying a logical index built from *one* vector to index a *different* vector of the same length — for instance, using a `smoker` vector to subset a separate `bp` vector — which is exactly the technique you need to solve today’s problem.

## 26.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

Blood pressure readings, but only for participants who smoke
smoker_bp <- participants$systolic_bp[participants$smoker == "Yes"]

length(smoker_bp)

[1] 25

mean(smoker_bp, na.rm = TRUE)

[1] 130.5417
```

Notice the two different vectors involved: `participants$smoker == "Yes"` produces a `TRUE/FALSE` value for every row, and that logical vector is then used to index `participants$systolic_bp` — a completely different column, but the same length, so each `TRUE/FALSE` lines up correctly, row for row.

## 26.5. Interpret the Result

Of the 100 participants, 25 reported being current smokers, and their mean systolic blood pressure — based on the 24 of those 25 with a non-missing reading — was approximately 130.5 mmHg — a figure you could now compare against the mean for non-smokers to explore a potential association between smoking and blood pressure (exactly the kind of comparison Level 11 will teach you to test formally).

### Common Mistakes

- **Length mismatch between the logical vector and the vector being indexed.** `participants$age[participants$smoker == "Yes"]` only works correctly because both vectors have the same length (100) and are aligned row-for-row — indexing with a logical vector of a different length gives nonsensical, silently recycled results.
- **Using `=` instead of `==`.** Just as in Level 1, `participants$smoker = "Yes"` inside an index would attempt an assignment, not a comparison, and will not filter as intended.
- **Forgetting NA values propagate into logical indexing too.** If `smoker` itself contains NA, positions corresponding to that NA will return NA rather than being cleanly excluded — worth checking with `is.na()` first if a column has missing values.

## 26.6. Practice Problem

Using `participants`, extract the ages of every participant with a BMI of 30 or greater (`participants$bmi >= 30`), and calculate the mean age of this group.

## 26.7. Challenge Problem

Extract the BMI values of participants who are *both* over 65 years old *and* current smokers, combining two conditions with `&`. Report how many participants meet both criteria and their mean BMI.

## 26.8. Key Takeaways

- Indexing a vector with a same-length logical vector keeps only the elements where that logical vector is `TRUE`.
- The logical vector can come from a comparison on a *different* column, as long as both vectors are the same length and aligned row for row — this is how you filter one variable based on another.
- This technique is the base-R foundation for what Level 5's `filter()` does more conveniently for entire data frames.

### Knowledge Check

1. **Multiple choice.** What does `x[x > 10]` return for `x <- c(5, 15, 8, 20)`?
  - a) `c(15, 20)`
  - b) `c(5, 8)`
  - c) `TRUE, TRUE, FALSE, FALSE`
  - d) An error
2. **True or false.** You can use a logical vector built from one column to index a completely different column, as long as both are the same length.
3. **Predict the output.**

```
bp <- c(118, 132, 145, 121)
bp[bp >= 130]
```

## 26. Logical Indexing

4. **Write the code.** Using `participants`, write the code to extract the `age` values of all participants living in "Millbrook" county.

Show answers

1. **a)** `c(15, 20)` — the elements where `x > 10` is `TRUE`.
2. **True.** As long as the logical vector and the target vector are the same length and correctly aligned, this works — it's exactly how the flagship example on this page filtered blood pressure by smoking status.
3. `[1] 132 145`
4. `participants$age[participants$county == "Millbrook"]`

## 27. Named Vectors

### The Problem

You've calculated the mean systolic blood pressure for each of the five counties in your survey — Cedar Falls, Millbrook, Oakford, Riverdale, and Springvale — and stored the five results in a plain numeric vector. A week later, you look at that vector again: 128.16, 133.30, 129.71, 124.36, 131.46. Which number belonged to which county? You'd have to recompute everything to remember.

### Your Mission

Attach labels directly to the values in a vector, so each number is self-documenting and can be looked up by name instead of position.

### Think Before Coding

- If a plain vector only lets you retrieve a value by position (`x[2]`), what extra piece of information would let you retrieve it by a meaningful label instead?
- Have you seen anything like this before — a set of labeled values you can look up by name rather than counting positions? (Hint: a data frame's *columns* already work this way.)

### 27.1. The R Concept

A vector can have a **name attached to every element**, either by supplying names directly inside `c()`, or by assigning them afterward with

## 27. Named Vectors

`names()`:

```
c(name1 = value1, name2 = value2, ...)
```

```
x <- c(10, 20, 30)
names(x) <- c("a", "b", "c")
```

Once named, you can index by name using `[ ]` with a quoted string, in addition to the position-based indexing you already know:

```
x["b"] # retrieves the element named "b", regardless of its position
```

`names()` works both ways: calling it alone shows the current names; assigning to it (`names(x) <- ...`) sets new ones.

### 27.2. Demonstration

```
mean_bp_by_county <- c(
 "Cedar Falls" = 128.16,
 "Millbrook" = 133.30,
 "Oakford" = 129.71,
 "Riverdale" = 124.36,
 "Springvale" = 131.46
)
```

```
mean_bp_by_county
```

| Cedar Falls | Millbrook | Oakford | Riverdale | Springvale |
|-------------|-----------|---------|-----------|------------|
| 128.16      | 133.30    | 129.71  | 124.36    | 131.46     |

```
mean_bp_by_county["Millbrook"]
```

```
Millbrook
133.3
```

```
names(mean_bp_by_county)
```

```
[1] "Cedar Falls" "Millbrook" "Oakford" "Riverdale" "Springvale"
```

## 27.3. Explore

Try `mean_bp_by_county[["Millbrook"]]` (double brackets) instead of single brackets — notice the output no longer includes the name label, just the bare number. This distinction between `[ ]` and `[[ ]]` will matter even more once you reach lists later in this level. Then try retrieving a county that doesn't exist, like `mean_bp_by_county["Oak Grove"]` — what does R return?

## 27.4. Solve the Problem

```
Building the named vector directly from the data, using tools from earlier in this level
participants <- read.csv("data/participants.csv")
```

```
counties <- unique(participants$county)
mean_bp_by_county <- sapply(counties, function(cty) {
 mean(participants$systolic_bp[participants$county == cty], na.rm = TRUE)
})
names(mean_bp_by_county) <- counties
```

```
A week later, you can still look this up instantly and unambiguously
mean_bp_by_county["Millbrook"]
```

```
Millbrook
133.30
```

**i** A quick note on `sapply()`

This solution uses `sapply()` to avoid repeating the same filter-and-mean calculation five times by hand — you haven't formally learned it yet (that's covered in Level 10), so don't worry about the syntax here. The point of this page is what happens *after*: attaching names so the result is self-documenting.

## 27.5. Interpret the Result

`mean_bp_by_county["Millbrook"]` instantly and unambiguously returns 133.30 mmHg — the mean systolic blood pressure among Millbrook participants — without you needing to remember whether Millbrook was the first, second, or third value in the vector. The name *is* the lookup key, which is far more durable than remembering positional order.

### Common Mistakes

- **Misspelling or mis-casing a name when looking it up.** `mean_bp_by_county["millbrook"]` (lowercase) returns NA, not an error — names are matched exactly and are case-sensitive, just like object names in Level 1.
- **Confusing `[ ]` and `[[ ]]` on named vectors.** `x["b"]` returns a named single-element vector; `x[["b"]]` returns just the bare value with no name attached — a subtle difference that becomes more important with lists, later in this level.
- **Forgetting names can become out of sync.** If you reorder or filter a named vector's values without also reordering `names()`, the labels will silently no longer match the values they're attached to — always build both together, as in the solution above.



## 27.6. Practice Problem

Run `table(participants$county)` to get the real participant counts per county, then build a named vector `n_participants_by_county` from that output. Retrieve just the Springvale count by name.

## 27.7. Challenge Problem

You have `mean_bp_by_county` from this page. Use `which.max()` (look it up with `?which.max` if needed, applying what you learned about getting help in Level 1) to identify which county has the *highest* mean blood pressure, and extract both its name and its value in one expression.

## 27.8. Key Takeaways

- Vectors can carry names alongside their values, either set inside `c(name = value)` or assigned afterward with `names()`.
- Named elements can be retrieved by name using `x["name"]`, which is more durable and self-documenting than remembering positional order.
- `x["name"]` keeps the name attached to the result; `x[["name"]]` returns just the bare value.

### Knowledge Check

1. **Multiple choice.** How do you retrieve the element named "b" from vector `x`, keeping its name?
  - a) `x[2]`
  - b) `x["b"]`
  - c) `x(b)`

## 27. Named Vectors

d) `x.b`

2. **True or false.** Name-based indexing in R is case-sensitive.

3. **Predict the output.**

```
x <- c(low = 10, high = 90)
x["low"]
```

4. **Find the error.** A student wants the mean BP for Riverdale and writes:

```
mean_bp_by_county["riverdale"]
```

but gets NA.

Show answers

1. **b)** `x["b"]`
2. **True.** Names must match exactly, including capitalization.
3. `low \n 10` (a named single-element vector showing `low: 10`).
4. The name was written in lowercase ("**riverdale**") but the actual name is "**Riverdale**" (capitalized) — names are case-sensitive, so the lookup silently returns NA instead of erroring.

## 28. Factors

### The Problem

You've categorized every participant's BMI into "Underweight", "Normal", "Overweight", or "Obese", stored as character text. When you try to summarize or plot this variable, R sorts the categories alphabetically: Normal, Obese, Overweight, Underweight — a meaningless order that has nothing to do with actual BMI severity. You need the categories to appear in their true clinical order instead.

### Your Mission

Convert a character variable into a factor with an explicitly defined, meaningful order.

### Think Before Coding

- Why would R default to alphabetical order for text categories — what other order would it possibly guess?
- If you were going to tell R the “correct” order yourself, what information would you need to supply?

### 28.1. The R Concept

A **factor** is R's data structure for categorical variables — data that takes on a limited set of distinct values (called **levels**). Unlike plain character data, a factor can have its levels placed in a specific, meaningful order that R will then respect everywhere: in tables, summaries, and plots.

## 28. Factors

```
factor(x, levels = c(...))

bmi_category <- c("Overweight", "Normal", "Obese", "Normal")
bmi_factor <- factor(
 bmi_category,
 levels = c("Underweight", "Normal", "Overweight", "Obese")
)
```

Without the `levels` argument, `factor()` still works, but defaults to alphabetical order — exactly the problem described above. Supplying `levels` explicitly, in the order you want, is what fixes it.

For categories with a genuine ranking (like BMI category, or a Likert-style survey response), you can additionally mark the factor as **ordered**:

```
factor(x, levels = c(...), ordered = TRUE)
```

An ordered factor supports `<` and `>` comparisons between categories (e.g., “Overweight” `>` “Normal”), which a plain factor or character vector cannot do meaningfully.

### 28.2. Demonstration

```
bmi_category <- c("Overweight", "Normal", "Obese", "Normal", "Underweight")

Without specifying levels - defaults to alphabetical order
factor(bmi_category)
```

```
[1] Overweight Normal Obese Normal Underweight
Levels: Normal Obese Overweight Underweight
```

### 28.3. Explore

```
With explicit, clinically meaningful order
bmi_factor <- factor(
 bmi_category,
 levels = c("Underweight", "Normal", "Overweight", "Obese")
)
bmi_factor

[1] Overweight Normal Obese Normal Underweight
Levels: Underweight Normal Overweight Obese

table(bmi_factor)

bmi_factor
Underweight Normal Overweight Obese
 1 2 1 1
```

Notice the table now lists categories in the correct clinical order, not alphabetically.

### 28.3. Explore

Try creating an ordered version: `factor(bmi_category, levels = c("Underweight", "Normal", "Overweight", "Obese"), ordered = TRUE)`. Then try comparing two elements of it, like `bmi_factor_ordered[1] > bmi_factor_ordered[2]` — a comparison that would be meaningless on the original character vector but works logically once ordered.

## 28.4. Solve the Problem

```

participants <- read.csv("data/participants.csv")

Build the BMI category variable using base R conditional logic
(case_when(), a cleaner tidyverse tool for this, is covered in Level 4)
participants$bmi_category <- ifelse(
 participants$bmi < 18.5, "Underweight",
 ifelse(participants$bmi < 25, "Normal",
 ifelse(participants$bmi < 30, "Overweight", "Obese"))
)

participants$bmi_category <- factor(
 participants$bmi_category,
 levels = c("Underweight", "Normal", "Overweight", "Obese")
)

table(participants$bmi_category)

```

| Underweight | Normal | Overweight | Obese |
|-------------|--------|------------|-------|
| 3           | 28     | 39         | 25    |

## 28.5. Interpret the Result

Among the 95 participants with a recorded BMI, 3 are underweight, 28 have a normal BMI, 39 are overweight, and 25 fall into the obese category — and critically, this table now prints in the correct clinical severity order rather than alphabetically, which will also carry through correctly to any plot you build from this variable in Level 8.

### Common Mistakes

- **Forgetting the levels argument entirely.** `factor(bmi_category)` without specifying `levels` still runs, but silently defaults to alphabetical order — exactly the original problem, just hidden one step further away.
- **Misspelling a level.** If a level listed in `levels =` doesn't exactly match a value in the data (a typo, extra whitespace, different capitalization), those non-matching values silently become NA in the resulting factor.
- **Trying to do arithmetic on a factor.** Factors are stored internally as integers behind the scenes, but `mean(bmi_factor)` does not give a sensible clinical answer — it either errors or averages the internal integer codes, not the category meanings. Factors are for categories, not calculations.
- **Confusing an ordered factor's comparisons with the original character data's.** `"Obese" > "Normal"` on plain character text compares alphabetically (and happens to be TRUE here, misleadingly) — only an explicitly `ordered = TRUE` factor guarantees the comparison reflects your intended clinical ranking, not alphabetical coincidence.

## 28.6. Practice Problem

Create a factor `smoking_factor` from `c("No", "Yes", "No", "Yes", "Yes")` with levels ordered as "No" then "Yes" (rather than the alphabetical default, which happens to already match here — explain in one sentence why specifying `levels` explicitly is still good practice even when it doesn't change anything visible).

## 28.7. Challenge Problem

Blood pressure categories are often defined as "Normal" (systolic < 120), "Elevated" (120–129), "Hypertension Stage 1" (130–139), and "Hypertension Stage 2" (140+). Using `ifelse()` (nested, as in the solution above) and `participants$systolic_bp`, build an ordered factor `bp_category` with these four levels in clinically increasing order, then produce a `table()` of it.

## 28.8. Key Takeaways

- `factor()` converts character (or numeric) data into a categorical variable with defined levels; without an explicit `levels` argument, it defaults to alphabetical order.
- `ordered = TRUE` additionally allows meaningful `</>` comparisons between categories, based on your specified order rather than alphabetical coincidence.
- Factors are for categories, not calculations — arithmetic functions like `mean()` don't produce meaningful results on them.

### Knowledge Check

1. **Multiple choice.** What order does `factor(x)` use by default, if `levels` isn't specified?
  - a) The order values first appear in the data
  - b) Alphabetical order
  - c) Numerical order
  - d) Random order
2. **True or false.** An ordered factor allows meaningful `<` and `>` comparisons between categories.
3. **Predict the output.**



## 28.8. Key Takeaways

```
f <- factor(c("Low", "High", "Medium"), levels = c("Low", "Medium", "High"))
table(f)
```

4. **Find the error.** A student writes `factor(bmi_category, levels = c("UnderwEight", "Normal", "Overweight", "Obese"))` where the data actually contains "Underweight" (correct spelling). What happens to those values?

Show answers

1. **b)** Alphabetical order.
2. **True.** That's precisely the purpose of `ordered = TRUE` — it encodes a meaningful ranking, not just category membership.

3. **f**

| Low | Medium | High |
|-----|--------|------|
| 1   | 1      | 1    |

(printed in the specified level order: Low, Medium, High)

4. Because "UnderwEight" (in `levels`) doesn't exactly match "Underweight" (in the data), every actual "Underweight" value becomes `NA` in the resulting factor — a silent, easy-to-miss data loss caused by a typo in the `levels` argument.



## 29. Creating Data Frames

### The Problem

You’ve been recruiting participants for a small pilot study, and so far you’ve recorded each person’s data in separate vectors as you went: an `id` vector, an `age` vector, a `sex` vector, and a `systolic_bp` vector. Your supervisor asks to see “the data” — but four loose, separate vectors floating around your R session isn’t a dataset, it’s four disconnected lists that happen to be the same length.

### Your Mission

Combine several related vectors into a single, rectangular table — a data frame — where each vector becomes a column.

### Think Before Coding

- If four vectors are each the same length, and each position across them describes the same person, what would it mean to line them up side by side?
- You’ve already used a data frame — `participants`, imported with `read.csv()` in Level 1. What did each column of that object actually turn out to be, based on what you learned two pages ago about vectors?

## 29.1. The R Concept

A **data frame** is R's standard structure for rectangular, spreadsheet-like data: rows represent observations (here, participants), and columns represent variables (age, sex, blood pressure), with every column being a vector of the same length. In fact, `read.csv()` from Level 1 was doing exactly this the whole time — converting a CSV file's rows and columns into a data frame automatically.

You build one directly with `data.frame()`:

```
data.frame(column1 = vector1, column2 = vector2, ...)

pilot_data <- data.frame(
 id = c(1, 2, 3),
 age = c(45, 62, 38),
 sex = c("Female", "Male", "Female")
)
```

Every vector you pass in must be the **same length** — mismatched lengths raise an error, since R has no way to decide how a shorter vector's values should line up against a longer one.

## 29.2. Demonstration

```
pilot_data <- data.frame(
 id = c(1, 2, 3, 4),
 age = c(45, 62, 38, 71),
 sex = c("Female", "Male", "Female", "Male"),
 systolic_bp = c(120, 145, 118, 152)
)

pilot_data
```

```

 id age sex systolic_bp
1 1 45 Female 120
2 2 62 Male 145
3 3 38 Female 118
4 4 71 Male 152

```

```
class(pilot_data)
```

```
[1] "data.frame"
```

## 29.3. Explore

Try building a data frame from vectors of mismatched length — for example, `id = c(1, 2, 3)` alongside `age = c(45, 62)` (only two values). Predict the error message before running it: R will complain that “arguments imply differing number of rows.”

## 29.4. Solve the Problem

```

id <- c(1, 2, 3, 4)
age <- c(45, 62, 38, 71)
sex <- c("Female", "Male", "Female", "Male")
systolic_bp <- c(120, 145, 118, 152)

pilot_data <- data.frame(id, age, sex, systolic_bp)
pilot_data

```

```

 id age sex systolic_bp
1 1 45 Female 120
2 2 62 Male 145

```

## 29. Creating Data Frames

|   |   |    |        |     |
|---|---|----|--------|-----|
| 3 | 3 | 38 | Female | 118 |
| 4 | 4 | 71 | Male   | 152 |

Now, instead of four disconnected vectors, you have a single object — `pilot_data` — that represents one coherent dataset, ready to hand to your supervisor or feed into any function that expects a data frame (which is most of the rest of this course).

### 29.5. Interpret the Result

`pilot_data` bundles four separate vectors into one rectangular table where each row now represents a complete participant record — exactly the structure your supervisor expects when they ask to see “the data,” and exactly what `read.csv()` was building for you automatically back in Level 1.

#### Common Mistakes

- **Mismatched vector lengths.** Every column passed to `data.frame()` must have the same number of elements — R will not guess how to pad a shorter vector.
- **Forgetting column names inside `data.frame()`.** `data.frame(id, age, sex)` (as shown in the solution) works and reuses the object names as column names automatically — but if you instead pass unnamed literal vectors like `data.frame(c(1,2,3))`, you’ll get an unhelpful auto-generated column name like `c.1..2..3..`
- **Assuming character columns become factors automatically.** In older versions of R, character columns were silently converted to factors inside `data.frame()` by default; modern R (4.0 and later) keeps them as character unless you explicitly convert them — worth knowing if you ever read documentation or code written for older R versions.

## 29.6. Practice Problem

Create a data frame `visit_log` with columns `participant_id = c(101, 102, 103)`, `visit_date = c("2026-01-05", "2026-01-06", "2026-01-06")`, and `completed = c(TRUE, TRUE, FALSE)`. Confirm its class and print it.

## 29.7. Challenge Problem

Someone hands you four vectors: `name <- c("Amara", "Priya")`, `age <- c(34, 41)`, `bp <- c(118, 132, 140)`, and `smoker <- c(TRUE, FALSE)`. Attempting `data.frame(name, age, bp, smoker)` will fail. Identify which vector is the problem and explain, in one sentence, exactly why data frames require this constraint.

## 29.8. Key Takeaways

- A data frame bundles multiple same-length vectors into one rectangular table, with each vector becoming a column.
- `data.frame(col1 = vec1, col2 = vec2, ...)` builds one directly; every column must have the same number of elements.
- `read.csv()` (Level 1) was building a data frame the entire time — importing a file is really just constructing a data frame from its rows and columns.

### Knowledge Check

1. **Multiple choice.** What must be true of every vector passed into `data.frame()`?
  - a) They must all be numeric

## 29. Creating Data Frames

- b) They must all be the same length
  - c) They must all have names
  - d) They must all be factors
2. **True or false.** `read.csv()` and `data.frame()` produce objects of the same class.
3. **Predict the output.**

```
df <- data.frame(x = c(1, 2), y = c("a", "b"))
class(df)
```

4. **Find the error.**

```
data.frame(id = c(1, 2, 3), age = c(45, 62))
```

Show answers

- 1. **b)** They must all be the same length.
- 2. **True.** `read.csv()` returns a `data.frame`, exactly like a direct `data.frame()` call.
- 3. `[1] "data.frame"`
- 4. `id` has 3 values but `age` has only 2 — mismatched lengths raise an error (“arguments imply differing number of rows”).



## 30. Accessing Columns

### The Problem

You’ve been using `participants$age` since Level 1 to grab a single column, and it’s worked fine. But a colleague’s script instead uses `participants[["age"]]`, and another uses `participants["age"]` — and confusingly, that last one behaves differently from the other two in a way that broke their code. You need to understand what’s actually going on before you get bitten by the same thing.

### Your Mission

Learn the different ways to extract a single column from a data frame, and understand exactly how they differ.

### Think Before Coding

- You already know a data frame’s column is really just a vector (from earlier in this level). If you ask for “the column itself,” should you expect back a vector, or something else?
- Could there be a difference between “give me this one column, still wrapped as a mini data frame” versus “give me this one column’s raw values directly”?

### 30.1. The R Concept

There are three common ways to pull a single column out of a data frame, and they don’t all return the same *type* of result:

### 30. Accessing Columns

```
df$column # returns the column as a bare vector
df[["column"]] # also returns the column as a bare vector - equivalent to $
df["column"] # returns a one-column DATA FRAME, not a bare vector
```

The first two (`$` and `[[ ]]`) behave the same way for this purpose — both hand you the raw vector, ready for `mean()`, indexing, or anything else you'd do with a vector. The third, single-bracket form (`["column"]`), instead returns a *data frame* containing just that one column — useful when you specifically want to keep the rectangular structure (for example, to pass a single-column subset into a function that expects a data frame), but it will not behave like a plain vector if you try to use it as one.

```
class(participants$age) # "integer" or "numeric" - a bare vector
class(participants["age"]) # "data.frame" - still a data frame!
```

## 30.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
class(participants$age)
```

```
[1] "integer"
```

```
class(participants[["age"]])
```

```
[1] "integer"
```

```
class(participants["age"])
```

```
[1] "data.frame"
```

```
This works, because $ returns a bare vector
mean(participants$age, na.rm = TRUE)
```

### 30.3. Explore

```
[1] 47.42553
```

```
This does NOT work as expected - participants["age"] is still a data frame
mean(participants["age"], na.rm = TRUE)
```

Warning message:

```
argument is not numeric or logical: returning NA
[1] NA
```

### 30.3. Explore

Try `head(participants["age"])` versus `head(participants$age)` — notice the first prints like a small data frame (with a column header and row numbers), while the second prints like a plain vector of numbers with no header. This is the same distinction from a different angle.

### 30.4. Solve the Problem

The colleague's bug is now explainable: they used single-bracket `participants["age"]` somewhere expecting a plain vector, but got a one-column data frame instead — which is why a function like `mean()` failed on it. The fix is switching to `$` or `[[ ]]`:

```
Broken - passes a one-column data frame to mean()
mean(participants["age"], na.rm = TRUE)
```

```
Fixed - passes a bare vector to mean()
mean(participants[["age"]], na.rm = TRUE)
```

```
[1] 47.42553
```

## 30.5. Interpret the Result

Both `$` and `[[ ]]` correctly return the mean age of 47.4 years, because both hand `mean()` a plain numeric vector. Single-bracket `["age"]` silently returns a data frame instead, which most vector-oriented functions like `mean()` cannot process directly — this is a genuinely common source of confusing bugs, and now you know exactly why it happens.

### Common Mistakes

- **Assuming `["column"]` and `[[ "column" ]]` are interchangeable.** They are not: single brackets preserve the data frame structure (one column, still a data frame); double brackets (or `$`) extract the bare vector.
- **Using `$` with a column name stored in a variable.** `$` requires the literal column name typed directly — `participants$col_name` where `col_name <- "age"` does *not* work as you might expect (it looks for a column literally named `col_name`). For column names stored in variables, use `participants[[col_name]]` instead.
- **Typos with `$` failing silently.** `participants$Age` (wrong capitalization) doesn't error — it returns `NULL`, which can cause confusing downstream failures far from the actual mistake.

## 30.6. Practice Problem

Using `participants`, extract the `bmi` column three different ways — `$`, `[[ ]]`, and `[ ]` — and use `class()` on each result to confirm which two match and which one differs.

## 30.7. Challenge Problem

You have a column name stored in a variable: `col_name <- "systolic_bp"`. Write the code to extract that column from `participants` as a bare vector, using `col_name` rather than typing `systolic_bp` literally (hint: this requires one of the three approaches on this page, not `$`).

## 30.8. Key Takeaways

- `df$column` and `df[["column"]]` both return the column as a bare vector, ready for vector functions like `mean()`.
- `df["column"]` (single brackets) returns a one-column data frame instead — structurally different, and not directly usable by most vector functions.
- `[[]]` (unlike `$`) also works with a column name stored in a variable, which `$` cannot do.

### Knowledge Check

1. **Multiple choice.** Which of these returns a bare vector, not a data frame?
  - a) `df["age"]`
  - b) `df[["age"]]`
  - c) Both return the same thing
  - d) Neither returns a vector
2. **True or false.** `df$col_name` works correctly even when `col_name` is a variable holding the string `"age"`, rather than the literal word `age`.
3. **Predict the output.**

### 30. Accessing Columns

```
df <- data.frame(x = c(1, 2, 3))
class(df["x"])
```

#### 4. Find the error.

```
mean(participants["bmi"], na.rm = TRUE)
```

Show answers

1. **b)** `df[["age"]]` returns a bare vector; `df["age"]` returns a one-column data frame.
2. **False.** `$` requires the literal column name; it does not substitute the value of a variable. Use `df[[col_name]]` for that.
3. `[1] "data.frame"`
4. `participants["bmi"]` returns a one-column data frame, not a bare vector, so `mean()` cannot compute on it directly (it returns NA with a warning). It should be `mean(participants[["bmi"]], na.rm = TRUE)` or `mean(participants$bmi, na.rm = TRUE)`.

## 31. Accessing Rows

### The Problem

Your supervisor flags an unusual result and asks: “Can you pull up participant #42’s entire record? I want to see everything we have on them — age, county, blood pressure, all of it.” You know how to grab a single *column* across all rows. Now you need the opposite: a single *row*, across all columns.

### Your Mission

Extract one or more complete rows from a data frame, keeping every column.

### Think Before Coding

- A data frame is two-dimensional — rows and columns. If vector indexing used one number in [ ], how many numbers (or positions) might a data frame need?
- Do you think leaving one of those two positions blank might mean something specific, like “give me all of them”?

### 31.1. The R Concept

Data frames are indexed with **two** positions inside the brackets, separated by a comma: `df[rows, columns]`. Leaving either side blank means “all of them”:

### 31. Accessing Rows

```
df[row_number,] # one full row, every column (note the comma!)
df[, column_number] # one full column, every row
df[row_number, column_number] # a single specific cell
```

This comma is what distinguishes row-and-column indexing from the single-bracket column shortcut you saw on the last page (`df["age"]`, no comma, meant something entirely different — a column by name).

```
participants[42,] # participant #42's complete record, every column
participants[1:5,] # the first 5 participants, every column
participants[42, "age"] # just participant #42's age - a specific cell
```

## 31.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
participants[42,]
```

|    | id | age | sex  | county     | systolic_bp | diastolic_bp | bmi  | smoker |
|----|----|-----|------|------------|-------------|--------------|------|--------|
| 42 | 42 | 53  | Male | Springvale | 135         | 86           | 30.4 | Yes    |

```
participants[1:3,]
```

|   | id | age | sex    | county    | systolic_bp | diastolic_bp | bmi  | smoker |
|---|----|-----|--------|-----------|-------------|--------------|------|--------|
| 1 | 1  | 46  | Female | Oakford   | 125         | 79           | 35.1 | No     |
| 2 | 2  | 53  | Female | Riverdale | 123         | 80           | 27.7 | No     |
| 3 | 3  | 59  | Female | Oakford   | 130         | 77           | 28.3 | No     |

```
participants[42, "age"]
```

```
[1] 53
```



### 31.3. Explore

Try `participants[42]` — the same thing, but *without* the comma. Notice this doesn't give you the row at all; it behaves like column-selection instead (specifically, it tries to select the 42nd *column*, which doesn't exist in an 8-column data frame, producing an error or unexpected result). The comma is not optional decoration — it fundamentally changes what R thinks you're asking for.

### 31.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

Participant #42's entire record, every column
participants[42,]

 id age sex county systolic_bp diastolic_bp bmi smoker
42 42 53 Male Springvale 135 86 30.4 Yes
```

Your supervisor now has the complete picture for participant #42 in one line: 53 years old, male, from Springvale, with a systolic BP of 135, diastolic of 86, BMI of 30.4, and a current smoker.

### 31.5. Interpret the Result

`participants[42, ]` returns a one-row data frame containing every variable recorded for that single participant — exactly the “everything we have on them” your supervisor asked for, rather than a bare vector, since a row genuinely mixes multiple data types (numbers and text together) that only a data frame (not a vector) can hold.

### 31. Accessing Rows

#### Common Mistakes

- **Forgetting the comma.** `participants[42, ]` (row 42, all columns) and `participants[42]` (attempting to select the 42nd column) are completely different operations — the comma is essential, not optional.
- **Confusing row *position* with the id column's value.** `participants[42, ]` returns the 42nd *row* in the data frame — this only matches `id == 42` because the data happens to be sorted and unfiltered; after any filtering or reordering (Level 5 onward), row position and id value can diverge completely.
- **Leaving a trailing comma off by habit from vector indexing.** Vectors use one index (`x[5]`); data frames need two, separated by a comma, even when you want “all columns” (in which case the space after the comma is simply left blank, not omitted).

### 31.6. Practice Problem

Extract the complete records for participants 10 through 15 (using a range) from `participants`.

### 31.7. Challenge Problem

Extract just participant #7's `county` and `smoker` values in a single expression (hint: the column position can also be a vector of column names, like `c("county", "smoker")`, inside the same two-position bracket syntax).

## 31.8. Key Takeaways

- Data frames use two-position indexing, `df[rows, columns]`, separated by a comma; leaving one side blank means “all.”
- Omitting the comma entirely changes the meaning of the expression — it no longer refers to rows at all.
- A single row returns a one-row data frame (since rows can mix types); a single column returns a vector (via `$` or `[[ ]]`) or a one-column data frame (via `[ ]`, from the previous page).

### Knowledge Check

1. **Multiple choice.** What does `df[5, ]` return?
  - a) The 5th column
  - b) The 5th row, all columns
  - c) The first 5 rows
  - d) An error
2. **True or false.** `df[5, ]` and `df[5]` do the same thing.
3. **Predict the output.**

```
df <- data.frame(x = c(10, 20, 30), y = c("a", "b", "c"))
df[2,]
```

4. **Find the error.** A student wants participant #10's record and writes:

```
participants[10]
```

Show answers

1. **b)** The 5th row, all columns.
2. **False.** `df[5, ]` returns row 5 (all columns); `df[5]` attempts to select column 5 — the comma changes the meaning entirely.

### 31. Accessing Rows

3. `x y`  
`2 20 b`

4. Missing the comma — `participants[10]` attempts to select the 10th column, not row 10. It should be `participants[10, ]`.

## 32. Matrices

### The Problem

A lab tracks each participant's systolic blood pressure across four consecutive monthly visits. You could store this as a data frame — but every value here is the exact same type (numeric blood pressure readings), and you want to do genuinely mathematical operations across the whole grid, like computing each participant's visit-to-visit change. A data frame's flexibility (mixed column types) isn't actually needed here, and a simpler, more strictly numeric grid would do.

### Your Mission

Create and index a matrix — a rectangular grid where every single value shares the same type.

### Think Before Coding

- If every value in your grid is a number, what would you lose by using a data frame instead of something simpler?
- A data frame's columns can each be a different type. What might it mean for a structure to *require* every value, across the whole grid, to be the same type?

### 32.1. The R Concept

A **matrix** is a two-dimensional grid where, unlike a data frame, *every* value must be the same type (all numeric, or all character — never mixed).

## 32. Matrices

Matrices are common in more mathematical or statistical contexts (like a correlation matrix in Level 11) where genuine matrix arithmetic matters.

```
matrix(data, nrow = ..., ncol = ..., byrow = FALSE)

bp_matrix <- matrix(
 c(120, 125, 118, 130, 132, 128, 145, 140, 138),
 nrow = 3, ncol = 3, byrow = TRUE
)
```

The `byrow` argument controls how the flat vector of values gets arranged into the grid: `byrow = TRUE` fills row by row; the default, `byrow = FALSE`, fills column by column instead — a frequent source of confusion, since both produce a valid-looking matrix with the *same numbers in different positions*.

Indexing a matrix uses the same two-position bracket syntax as a data frame: `m[row, col]`.

### 32.2. Demonstration

```
bp_matrix <- matrix(
 c(120, 125, 118,
 130, 132, 128,
 145, 140, 138),
 nrow = 3, ncol = 3, byrow = TRUE
)
bp_matrix
```

```
 [,1] [,2] [,3]
[1,] 120 125 118
[2,] 130 132 128
[3,] 145 140 138
```

### 32.3. Explore

```
Row 1, column 2 - a specific reading
bp_matrix[1, 2]

[1] 125

All of row 2 (one participant's readings across all visits)
bp_matrix[2,]

[1] 130 132 128

class(bp_matrix)

[1] "matrix" "array"
```

### 32.3. Explore

Rebuild the same matrix with `byrow = FALSE` instead (or simply omit `byrow` entirely, since `FALSE` is the default) and compare the printed result to the one above — same nine numbers, arranged completely differently. This is exactly the kind of mistake that produces a matrix that *looks* fine but silently has every value in the wrong position.

### 32.4. Solve the Problem

```
Three participants (rows), four monthly visits (columns)
bp_matrix <- matrix(
 c(120, 125, 118, 122,
 130, 132, 128, 126,
 145, 140, 138, 135),
 nrow = 3, ncol = 4, byrow = TRUE
)
bp_matrix
```

## 32. Matrices

```
 [,1] [,2] [,3] [,4]
[1,] 120 125 118 122
[2,] 130 132 128 126
[3,] 145 140 138 135
```

```
Change from visit 1 to visit 4, for every participant at once
bp_matrix[, 4] - bp_matrix[, 1]
```

```
[1] 2 -4 -10
```

### 32.5. Interpret the Result

The change vector 2, -4, -10 shows that participant 1's blood pressure rose slightly by visit 4, while participants 2 and 3 saw decreases of 4 and 10 mmHg, respectively — a calculation performed across the entire matrix in a single line, exactly the kind of operation a matrix's uniform numeric structure makes easy.

#### Common Mistakes

- **Confusing `byrow = TRUE` and `byrow = FALSE`.** The same flat vector of numbers produces a completely different arrangement depending on this argument — always verify the printed matrix matches your intended layout before trusting any calculation on it.
- **Assuming a matrix can mix types like a data frame.** `matrix(c(1, "two", 3), nrow = 1)` silently coerces everything to character, exactly like `c()` did in the vectors page — matrices inherit this same all-one-type constraint.
- **Mixing up matrix indexing with data frame indexing habits.** Both use `m[row, col]`, but a matrix has no column *names* by default (just `[,1]`, `[,2]`, etc.) unless you explicitly assign them with `colnames()` — attempting `bp_matrix["visit_1"]` without first setting names will fail.



## 32.6. Practice Problem

Create a 2×3 matrix of BMI values `c(22.1, 24.5, 27.3, 19.8, 31.2, 26.0)` filled by row, and extract the entire second row.

## 32.7. Challenge Problem

Using the `bp_matrix` from the solution above, calculate the mean blood pressure for each participant across their four visits (hint: look up `rowMeans()` using the help-page skills from Level 1 — `?rowMeans`).

## 32.8. Key Takeaways

- A matrix is a two-dimensional grid where every value must share the same type — unlike a data frame, which allows different types per column.
- `byrow` controls whether the input vector fills the matrix row by row or column by column, and mixing this up silently misplaces every value.
- Matrix indexing uses the same `m[row, col]` syntax as data frames, making genuinely mathematical, whole-grid operations (like `bp_matrix[, 4] - bp_matrix[, 1]`) straightforward.

### Knowledge Check

1. **Multiple choice.** What must be true of every value in a matrix?
  - a) All values must be unique
  - b) All values must be the same type
  - c) All values must be positive
  - d) All values must be named

## 32. Matrices

2. **True or false.** `byrow = TRUE` and `byrow = FALSE` always produce the same matrix from the same input vector.

3. **Predict the output.**

```
m <- matrix(1:6, nrow = 2, byrow = TRUE)
m[1,]
```

4. **Find the error.**

```
matrix(c(1, "two", 3), nrow = 1)
```

A student expects this to raise an error because it mixes a number and text. What actually happens instead?

Show answers

1. **b)** All values must be the same type.
2. **False.** They arrange the same values differently — row by row versus column by column — producing different matrices unless the data happens to be symmetric.
3. `[1] 1 2 3` (the first row, since `byrow = TRUE` fills 1, 2, 3 into row 1 and 4, 5, 6 into row 2).
4. It doesn't error — R silently coerces everything to character, exactly like `c()` does, producing `matrix(c("1", "two", "3"), nrow = 1)`.

## 33. Lists

### The Problem

You've written a small summary of your analysis: the study name ("Community Health Survey", a single piece of text), the full `participants` data frame, a numeric vector of the three counties with the highest mean blood pressure, and a logical flag for whether the analysis is complete. These four things have completely different shapes and types — there's no vector, matrix, or single data frame that could hold a data frame, a vector, and a single value all together.

### Your Mission

Bundle several different, unrelated objects — of any type or shape — into a single container.

### Think Before Coding

- Every structure you've learned so far in this level (vectors, matrices, data frames) enforces some kind of consistency — same type, or same length. What if you needed a container with *no* such restriction at all?
- If you already know `[[ ]]` extracts a bare value from a named vector, could something similar work for extracting one whole, distinct item out of a bundle of very different items?

### 33.1. The R Concept

A **list** is R's most flexible structure: it can hold any combination of objects — vectors, data frames, matrices, single values, even other lists — with no requirement that they share a type, length, or shape.

```
list(name1 = object1, name2 = object2, ...)
```

```
analysis_summary <- list(
 study_name = "Community Health Survey",
 data = participants,
 top_counties = c("Millbrook", "Springvale", "Oakford"),
 complete = TRUE
)
```

Accessing list elements uses the same double-bracket idea from named vectors, but it matters even more here: `[[ ]]` extracts one element exactly as it is (a data frame stays a data frame, a vector stays a vector); `[ ]` returns a *smaller list* containing that element, still wrapped in list packaging — usually not what you want when you're after the actual content.

```
analysis_summary[["study_name"]] # the bare character value
analysis_summary["study_name"] # a one-element LIST containing that value
analysis_summary$study_name # $ also works for named list elements, like
```

### 33.2. Demonstration

```
analysis_summary <- list(
 study_name = "Community Health Survey",
 n_participants = 100,
 top_counties = c("Millbrook", "Springvale", "Oakford"),
 complete = TRUE
)
```

### 33.3. Explore

```
analysis_summary

$study_name
[1] "Community Health Survey"

$n_participants
[1] 100

$top_counties
[1] "Millbrook" "Springvale" "Oakford"

$complete
[1] TRUE

analysis_summary[["top_counties"]]

[1] "Millbrook" "Springvale" "Oakford"

class(analysis_summary["top_counties"])

[1] "list"

class(analysis_summary[["top_counties"]])

[1] "character"
```

### 33.3. Explore

Try `analysis_summary$data <- participants` to add the entire `participants` data frame as a fifth element of the list — demonstrating that a list can hold something as large and complex as an entire dataset

### 33. Lists

alongside small individual values. Then run `length(analysis_summary)` — what do you think it counts: total values across everything, or top-level elements?

#### 33.4. Solve the Problem

```
analysis_summary <- list(
 study_name = "Community Health Survey",
 data = participants,
 top_counties = c("Millbrook", "Springvale", "Oakford"),
 complete = TRUE
)
```

```
length(analysis_summary)
```

```
[1] 4
```

```
analysis_summary$study_name
```

```
[1] "Community Health Survey"
```

```
nrow(analysis_summary$data)
```

```
[1] 100
```

The list `analysis_summary` now holds all four genuinely different pieces — a piece of text, an entire 100-row data frame, a 3-element character vector, and a single logical value — as one coherent object, with each piece still fully accessible and usable in its original form.

## 33.5. Interpret the Result

`length(analysis_summary)` returns 4, confirming the list has four *top-level* elements — it does not count all 100 rows inside the nested data frame, or every character in `study_name`; it counts the list’s own direct components. This distinction between “top-level elements” and “everything nested inside” becomes important as you work with more complex, real-world list structures — including, later in the course, the output of statistical models, which are almost always returned as lists.

### Common Mistakes

- **Confusing `[ ]` and `[[ ]]` on a list.** `analysis_summary["study_name"]` returns a one-element *list* (still wrapped); `analysis_summary[["study_name"]]` returns the bare character value inside — nearly every bug involving “why does this look like text but won’t behave like text” traces back to this exact distinction.
- **Assuming `length()` counts everything nested inside.** `length(analysis_summary)` counts the 4 top-level elements, not the 100 rows of the nested data frame or the 3 elements of the nested vector — each nested object has its own separate `length()` or `nrow()`.
- **Forgetting that list elements can be added or changed with `$` or `[[ ]]`, just like data frame columns.** `analysis_summary$new_field <- "value"` adds a new list element on the fly, exactly like adding a new column to a data frame.

## 33.6. Practice Problem

Create a list `patient_record` with elements `id = 42`, `age = 53`, `vitals = c(135, 86)` (systolic and diastolic BP), and `notes = "Follow-up needed"`. Extract just the `vitals` element as a bare vector (not wrapped in a list).

### 33.7. Challenge Problem

Add a fifth element to `patient_record` called `flagged`, set to `TRUE`, using `$`. Then write the code to check `length(patient_record)` before and after adding it, confirming it increases by exactly one — regardless of how large or small the new element itself is.

### 33.8. Key Takeaways

- A list can hold any combination of objects — different types, different lengths, different shapes — with no consistency requirement at all.
- `[[ ]]` (or `$`, for named elements) extracts an element exactly as it is; `[ ]` returns a smaller list still wrapping that element.
- `length()` on a list counts top-level elements only, not everything nested inside them — a distinction that matters once lists hold complex objects like entire data frames.

#### Knowledge Check

1. **Multiple choice.** What can a list hold?
  - a) Only vectors of the same type
  - b) Only numeric values
  - c) Any combination of objects, regardless of type or shape
  - d) Only data frames
2. **True or false.** `mylist[["x"]]` and `mylist["x"]` always return the exact same thing.
3. **Predict the output.**

```
l <- list(a = 1, b = c(2, 3), c = "text")
length(l)
```



### 33.8. Key Takeaways

4. **Find the error.** A student expects `mylist["score"]` to behave like a bare number so they can do math with it directly, but gets an error.

Show answers

1. **c)** Any combination of objects, regardless of type or shape.
2. **False.** `[[ ]]` returns the bare element; `[ ]` returns a one-element list still wrapping it — they are structurally different.
3. `[1] 3` — three top-level elements (`a`, `b`, `c`), regardless of how many values are nested inside `b`.
4. `mylist["score"]` returns a one-element list, not a bare number — arithmetic functions can't operate on a list directly. The fix is `mylist[["score"]]` or `mylist$score`.



## 34. Tibbles

### The Problem

You accidentally type `participants$Age` (capital A) instead of `participants$age`. Instead of a helpful error telling you the column doesn't exist, base R's data frame silently returns `NULL` — and worse, in some cases base R's partial matching would have quietly matched a *different*, similarly-named column, giving you a wrong answer with no warning at all. You'd like R to be stricter and catch mistakes like this immediately.

### Your Mission

Learn about tibbles — the tidyverse's modern alternative to the base R data frame — and understand what they do differently.

### Think Before Coding

- If a base R data frame lets a typo pass silently, what would you want a “safer” version of the same structure to do instead?
- When you print a 100-row data frame with `head()`, you only see 6 rows by default — what would happen if you printed the *whole* 100-row data frame with no `head()` at all? Would that always be convenient?

## 34.1. The R Concept

A **tibble** is a modern reimagining of the data frame, provided by the `tibble` package (part of the `tidyverse`, loaded automatically with `library(tidyverse)`, or on its own with `library(tibble)`). A tibble *is* still fundamentally a data frame — everything you’ve learned about `$`, `[ ]`, `[[ ]]`, and two-position row/column indexing still applies — but it behaves more predictably in a few specific ways:

- **No partial matching with `$`.** Base R data frames sometimes let `df$ag` match a column called `age` if no exact match exists; tibbles refuse this and return `NULL` with a warning instead, surfacing the typo rather than hiding it.
- **Cleaner, safer printing.** Printing a tibble shows only the first 10 rows and however many columns fit on screen by default, along with each column’s type directly under its name — avoiding the base R habit of flooding your Console with every row of a large dataset.
- **Never silently converts to a vector when selecting one column.** Some older base R subsetting operations that return a vector for a data frame instead consistently return a smaller tibble, reducing an entire class of “wait, why is this suddenly not a data frame anymore” bugs.

Converting an existing data frame into a tibble uses `as_tibble()`:

```
library(tibble) # or library(tidyverse), which loads tibble along with dplyr
as_tibble(participants)
```

## 34.2. Demonstration

```
library(tibble)
```

### 34.3. Explore

```
participants_tbl <- as_tibble(participants)
participants_tbl

A tibble: 100 x 8
 id age sex county systolic_bp diastolic_bp bmi smoker
 <int> <int> <chr> <chr> <int> <int> <dbl> <chr>
1 1 46 Female Oakford 125 79 35.1 No
2 2 53 Female Riverdale 123 80 27.7 No
3 3 59 Female Oakford 130 77 28.3 No
4 4 49 Female Millbrook 117 72 30.4 No
5 5 61 Female Millbrook 139 79 21.5 No
6 6 18 Female Riverdale 117 64 24.0 No
... with 94 more rows

class(participants_tbl)

[1] "tbl_df" "tbl" "data.frame"
```

Notice `class()` shows *three* labels — a tibble is still a "data.frame" underneath (the third label), which is why everything you already know about data frames continues to work.

### 34.3. Explore

Try `participants_tbl$Age` (wrong capitalization, matching the opening problem) and compare the result to what base R's `participants$Age` would give — both actually return `NULL` for a fully wrong name in modern R, but try `participants_tbl$ag` (a genuine partial match attempt) versus how older base R behavior historically handled this, to see why the tidyverse considers this worth being strict about.

### 34.4. Solve the Problem

```
library(tibble)

participants_tbl <- as_tibble(participants)

A typo now returns NULL cleanly, with no risk of a silent partial match
participants_tbl$Age

NULL

The correct column still works exactly as expected
participants_tbl$age |> mean(na.rm = TRUE)

[1] 47.42553
```

### 34.5. Interpret the Result

The typo `$Age` returns `NULL` rather than any data, which — while still requiring you to notice the problem — at least doesn’t silently hand back a plausible-looking but wrong result. The correctly spelled `$age` continues to behave exactly as it did with the base data frame, since a tibble is a data frame with extra safety, not a replacement structure with new rules to relearn.

#### Common Mistakes

- **Forgetting to load `tibble` (or `tidyverse`) before using `as_tibble()`.** Just like any package function from Level 1, this fails with `could not find function` if the package hasn’t been loaded in the current session.

### 34.6. Practice Problem

- **Assuming a tibble and a data frame are unrelated structures.** They're not — a tibble *is* a data frame (notice `"data.frame"` in its `class()` output), just with extra guardrails layered on top; everything you learned in this level about data frames still applies.
- **Expecting `print()` to show every row by default.** A tibble intentionally truncates to the first 10 rows when printed directly — use `View(participants_tbl)` in RStudio, or `print(participants_tbl, n = Inf)`, to see everything.

## 34.6. Practice Problem

Convert `participants` into a tibble called `participants_tbl`, confirm its class includes `"tbl_df"`, and print it to observe the truncated, cleaner display compared to printing the base `participants` data frame directly.

## 34.7. Challenge Problem

Explain, in your own words, why a tidyverse-heavy course (like the one starting in Level 5) would generally prefer tibbles over base data frames, connecting your answer to at least one specific safety behavior described on this page.

## 34.8. Key Takeaways

- A tibble is a data frame with extra safety guardrails: no partial matching on `$`, cleaner truncated printing, and more predictable subsetting behavior.
- `class()` on a tibble includes `"data.frame"`, confirming it's built on top of the same structure you've been using all level — not a separate thing to relearn from scratch.

### 34. Tibbles

- `as_tibble()` converts an existing data frame into a tibble; the tidyverse (starting in Level 5) will produce tibbles by default from most of its functions.

#### Knowledge Check

1. **Multiple choice.** What is a tibble, fundamentally?
  - a) A completely separate structure unrelated to data frames
  - b) A data frame with additional safety behaviors
  - c) A type of vector
  - d) A type of list
2. **True or false.** `class(as_tibble(df))` includes "data.frame" in its output.
3. **Predict the output.** If `participants_tbl` is a tibble with no column named `Age` (only lowercase `age` exists), what does `participants_tbl$Age` return?
4. **Write the code.** Convert a data frame called `survey_data` into a tibble, and confirm its class using `class()`.

Show answers

1. **b)** A data frame with additional safety behaviors — it is still fundamentally a data frame underneath.
2. **True.** A tibble's class includes "tbl\_df", "tbl", and "data.frame" — all three.
3. **NULL** — with no partial matching, an incorrect column name simply returns nothing, rather than risking a silent wrong match.
4. 

```
survey_tbl <- as_tibble(survey_data)
class(survey_tbl)
```



## 35. Understanding Structure with `str()`

### The Problem

A colleague sends you an R object saved from their analysis — you're told it's called `model_output`, but nothing else. Printing it directly floods your Console with a dense, confusing wall of text you don't recognize. Before you can use it, you need a fast way to answer: what *kind* of object is this, how is it organized, and what's actually inside it?

### Your Mission

Use `str()` to get a compact, readable overview of the structure of *any* R object — vector, factor, data frame, list, or matrix — without printing all of its contents.

### Think Before Coding

- You've already used `str()` once, in Level 1, on the `participants` data frame. What did it show you then?
- Do you think a function built to summarize a data frame's structure might also work on completely different types of objects, like a list or a plain vector?

## 35. Understanding Structure with `str()`

### 35.1. The R Concept

`str()` (short for “structure”) is one of the most broadly useful functions in R precisely because it works on *any* object type and adapts its output to what it finds — a vector, a factor, a data frame, a list, a matrix, or the complicated nested output of a statistical model later in this course.

```
str(object)
```

For a data frame, `str()` shows the number of rows and columns, each column’s name and type, and a short preview of its first few values — exactly what you saw in Level 1. But it works identically well on other structures:

```
str(c(1, 2, 3)) # a simple numeric vector
str(factor(c("Low", "High"))) # a factor, showing its levels
str(list(a = 1, b = "text")) # a list, showing each named element's type
```

This makes `str()` your first move whenever you encounter an unfamiliar object — before printing it in full, before guessing, before asking a colleague.

### 35.2. Demonstration

```
str(c(45, 62, 38))
```

```
num [1:3] 45 62 38
```

```
str(factor(c("Low", "High", "Low"), levels = c("Low", "High")))
```

```
Factor w/ 2 levels "Low","High": 1 2 1
```

```
analysis_summary <- list(
 study_name = "Community Health Survey",
 n_participants = 100,
 complete = TRUE
)
str(analysis_summary)
```

```
List of 3
 $ study_name : chr "Community Health Survey"
 $ n_participants : num 100
 $ complete : logi TRUE
```

### 35.3. Explore

Run `str(participants)` again, as you did back in Level 1, and now also try `str(matrix(1:6, nrow = 2))`. Notice `str()` reports something meaningfully different for each of the five structure types you've learned this level — vector, factor, data frame, list, and matrix — always adapted to that specific structure.

### 35.4. Solve the Problem

```
model_output <- list(
 method = "Two-sample t-test",
 estimate = 4.2,
 p_value = 0.031,
 significant = TRUE,
 groups = c("Smoker", "Non-smoker")
)

str(model_output)
```

### 35. Understanding Structure with `str()`

```
List of 5
 $ method : chr "Two-sample t-test"
 $ estimate : num 4.2
 $ p_value : num 0.031
 $ significant: logi TRUE
 $ groups : chr [1:2] "Smoker" "Non-smoker"
```

In one call, without printing a confusing wall of unexplained text, you now know exactly what your colleague’s object is: a list with five named elements, including a method label, two numeric results, a logical significance flag, and a two-element character vector of group names.

## 35.5. Interpret the Result

`str()` reveals that `model_output` is a `list` of 5 elements, each with its own type — information you could not have gotten as quickly, or as clearly, by simply printing the object and reading through its default (and, for many object types including actual statistical model results later in the course, often much messier) display.

### Common Mistakes

- **Only ever using `str()` on data frames.** Its real value is that it works identically well on *any* object — vectors, factors, lists, matrices — making it your general-purpose “what is this?” tool, not a data-frame-only trick.
- **Printing a large or unfamiliar object directly instead of checking `str()` first.** For a big list (especially the kind returned by statistical functions later in the course), a direct `print()` can flood your Console with far more than you need; `str()` gives you the shape first.

- **Confusing `str()` (structure) with `class()` (type label only).** `class()` gives you one short label ("`list`", "`data.frame`"); `str()` gives you that plus a breakdown of everything inside — use `class()` for a quick check, `str()` when you need the fuller picture.

## 35.6. Practice Problem

Run `str()` on `participants$sex` (a character vector) and separately on `participants$bmi_category` (the ordered factor you built earlier in this level, if you completed that page's solution). Compare the two outputs.

## 35.7. Challenge Problem

Build a small nested list: `study <- list(name = "Pilot Study", data = participants[1:5, ], sites = c("Riverdale", "Oakford"))`. Run `str(study)` and identify, from its output alone, how many rows and columns the nested data frame (`data`) has — without separately calling `nrow()` or `ncol()`.

## 35.8. Key Takeaways

- `str()` works on any R object — vectors, factors, data frames, lists, matrices — and adapts its output to that structure.
- It's the fastest way to understand an unfamiliar object without printing its full, potentially overwhelming contents.
- `class()` gives a one-line type label; `str()` gives a fuller structural breakdown — reach for `str()` whenever `class()` alone isn't enough to understand what you're looking at.

### Knowledge Check

35. *Understanding Structure with `str()`*

1. **Multiple choice.** Which of these object types does `str()` work on?
  - a) Only data frames
  - b) Only vectors
  - c) Any R object type
  - d) Only lists
2. **True or false.** `str()` and `class()` always return exactly the same amount of information.
3. **Predict the output.**  

```
str(c("a", "b", "c"))
```
4. **Find the error.** A student assumes `str()` only works on data frames and writes a workaround to manually inspect a list element by element instead of just calling `str()` on the whole list. What did they overlook about `str()`'s design?

Show answers

1. **c)** Any R object type.
2. **False.** `class()` gives a short type label; `str()` gives a fuller breakdown of the object's contents and structure — they serve different, complementary purposes.
3. `chr [1:3] "a" "b" "c"`
4. `str()` is explicitly designed to work on any object type, including lists — calling `str(mylist)` directly would have given the same (or better) insight in one line, with no manual element-by-element inspection needed.

## 36. Checking Dimensions: `nrow()`, `ncol()`, `dim()`

### The Problem

Before you send your cleaned dataset back to your supervisor, you want to double check one basic fact: does it still have exactly 100 rows and 8 columns, matching what you started with in Level 1? If a filtering step earlier in your script accidentally dropped rows, or a merge accidentally added duplicate columns, you want to catch that *before* it reaches your supervisor’s inbox, not after.

### Your Mission

Check the dimensions — rows and columns — of a data frame or matrix, quickly and reliably.

### Think Before Coding

- You already know a data frame is genuinely two-dimensional (rows and columns). What two separate numbers would fully describe its size?
- Would you expect a single plain vector — which has no rows or columns, just individual elements — to have “dimensions” in the same sense?

## 36. Checking Dimensions: `nrow()`, `ncol()`, `dim()`

### 36.1. The R Concept

Three related functions report the size of a two-dimensional object (a data frame or a matrix):

```
nrow(x) # number of rows
ncol(x) # number of columns
dim(x) # both at once, as a vector: c(rows, columns)

nrow(participants) # 100
ncol(participants) # 8
dim(participants) # c(100, 8)
```

Crucially, these are built specifically for two-dimensional structures. A plain vector has no rows or columns — just a length — so `nrow()` and `ncol()` both return `NULL` for a vector, and `dim(vector)` also returns `NULL`, rather than an error. This is a useful, quick way to distinguish “is this actually two-dimensional?” without needing `str()` or `class()`.

### 36.2. Demonstration

```
participants <- read.csv("data/participants.csv")

nrow(participants)

[1] 100

ncol(participants)

[1] 8

dim(participants)
```



```
[1] 100 8

A plain vector has no rows or columns
ages <- c(45, 62, 38)
nrow(ages)

NULL
```

### 36.3. Explore

Run `dim(ages)` (the same plain vector) and confirm it also returns `NULL`. Then build a matrix, `m <- matrix(1:12, nrow = 3)`, and run `nrow(m)`, `ncol(m)`, and `dim(m)` — confirming these functions work identically well on matrices, not just data frames.

### 36.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

Quick sanity check before handing off the dataset
dim(participants)

[1] 100 8

nrow(participants) == 100 & ncol(participants) == 8

[1] TRUE
```

The final line combines this page's functions with Level 1's logical comparisons into a single, explicit confirmation: `TRUE` means both dimensions match expectations exactly, and the dataset is safe to send along.

## 36.5. Interpret the Result

`dim(participants)` returning `c(100, 8)` confirms the dataset still has exactly 100 rows and 8 columns — no rows silently dropped, no columns silently duplicated — giving you concrete, checkable confidence before sharing the file onward, rather than just assuming everything is fine.

### Common Mistakes

- **Expecting `nrow()/ncol()` to work on plain vectors.** Both return `NULL` for a vector — for a vector’s size, you want `length()` (covered on the next page), not `nrow()`.
- **Confusing the order of `dim()`’s output.** `dim(x)` always returns `c(rows, columns)`, in that order — not columns first.
- **Skipping this check entirely and assuming a script “probably” worked.** A single silently dropped row (from a bad filter condition, explored more in Level 5) is easy to miss by eye in a 100-row dataset, but immediately obvious from a `dim()` check that no longer matches what you expect.

## 36.6. Practice Problem

Import `participants.csv` again into an object called `check_data`, and write a single line confirming both that it has exactly 100 rows and exactly 8 columns, using `&` to combine both conditions (as in the solution above).

## 36.7. Challenge Problem

Build a matrix `m <- matrix(1:20, nrow = 4)`. Without running any code, predict what `dim(m)` will return, then verify your prediction. Explain, in one sentence, why `nrow()` and `ncol()` work identically for ma-

trices and data frames despite them being different structures underneath (a distinction from the matrices page earlier in this level).

## 36.8. Key Takeaways

- `nrow()`, `ncol()`, and `dim()` report the size of two-dimensional structures (data frames and matrices); `dim()` returns both values at once as `c(rows, columns)`.
- These functions return `NULL` — not an error — when applied to a plain vector, which has no row/column structure.
- Checking dimensions before and after a data-processing step is a fast, concrete way to catch silently dropped rows or unexpectedly added columns.

### Knowledge Check

1. **Multiple choice.** What does `dim(df)` return?
  - a) Just the number of rows
  - b) Just the number of columns
  - c) A vector: `c(rows, columns)`
  - d) The column names
2. **True or false.** `nrow()` returns an error when applied to a plain vector.
3. **Predict the output.**

```
m <- matrix(1:15, nrow = 5)
dim(m)
```
4. **Write the code.** Write a single line confirming that a data frame `df` has more than 50 rows.

Show answers

36. *Checking Dimensions: `nrow()`, `ncol()`, `dim()`*

1. **c)** A vector: `c(rows, columns)`.
2. **False.** It returns `NULL`, not an error, since a vector has no row/column structure.
3. `[1] 5 3`
4. `nrow(df) > 50`

## 37. `length()` and `names()`

### The Problem

A new intern joins your team and, trying to be helpful, checks the size of the `participants` dataset by running `length(participants)`. They report back: “It only has 8 things in it!” — sounding alarmed, since they expected 100. You need to explain what actually just happened, and show them the right tool for what they were really trying to check.

### Your Mission

Understand what `length()` actually measures on different object types, and use `names()` to inspect or set labels on vectors, lists, and data frames alike.

### Think Before Coding

- You learned two pages ago that a data frame is really a collection of columns, each one a vector. If `length()` measures “how many things are in this container,” what would “things” mean for a data frame specifically — rows, or columns?
- If `names()` can retrieve labels from a named vector (earlier in this level), might it also retrieve something meaningful from a data frame or a list?

### 37. `length()` and `names()`

#### 37.1. The R Concept

`length()` measures the number of *top-level elements* in an object — but what counts as a “top-level element” depends entirely on the structure:

```
length(vector) # number of individual values
length(list_obj) # number of top-level list elements (as you saw last-but)
length(data_frame) # number of COLUMNS - because a data frame is a list of
```

This last one is the trap the intern fell into: a data frame is, structurally, a special kind of list where each column is one element — so `length(participants)` returns 8 (the number of columns), not 100 (the number of rows). For row count, you need `nrow()` from the previous page, not `length()`.

`names()` retrieves (or sets) the labels attached to an object’s top-level elements:

```
names(vector) # element names, if any (as with named vectors)
names(list_obj) # the list's element names
names(data_frame) # the column names
```

#### 37.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
length(participants)
```

```
[1] 8
```

```
nrow(participants)
```

```
[1] 100
```

### 37.3. Explore

```
names(participants)

[1] "id" "age" "sex" "county" "systolic_bp"
[6] "diastolic_bp" "bmi" "smoker"

length() on the age COLUMN (a vector) behaves as expected
length(participants$age)

[1] 100
```

### 37.3. Explore

Run `length(participants$age)` versus `length(participants)` side by side, and notice they answer two genuinely different questions: “how many values are in this one column?” versus “how many columns does this whole data frame have?” Then try `names(participants$age)` — since `age` is a plain, unnamed vector, what do you think this returns?

### 37.4. Solve the Problem

```
What the intern actually needed:
nrow(participants) # 100 - the number of rows (participants)

[1] 100

What length() actually measured:
length(participants) # 8 - the number of columns (variables)

[1] 8
```

### 37. `length()` and `names()`

```
names(participants) # the 8 column names, confirming what's being counted

[1] "id" "age" "sex" "county" "systolic_bp"
[6] "diastolic_bp" "bmi" "smoker"
```

You can now explain it clearly to the intern: “`length()` on a data frame counts columns, not rows, because a data frame is really a list where each column is one element. For row count, always reach for `nrow()` instead.”

## 37.5. Interpret the Result

The dataset genuinely does have 100 participants (rows) and 8 variables (columns) — both facts are true at once, and `length()` and `nrow()` simply answer different questions about the same object. Neither function is “wrong”; the intern’s surprise came from expecting `length()` to answer a question it was never built to answer for a two-dimensional structure.

### Common Mistakes

- **Using `length()` when you mean `nrow()`.** This is, by far, the single most common version of this mistake in real R code — always reach for `nrow()` for row counts on a data frame, and reserve `length()` for vectors, lists, or explicitly checking a data frame’s column count.
- **Forgetting a data frame is structurally a list.** This single fact explains both why `length(df)` counts columns and why `df$column` and list-style `[[ ]]` access work identically on data frames — they share the same underlying structure.
- **Assuming `names()` always returns column names.** On a plain, unnamed vector (like `participants$age`), `names()` returns `NULL` — there’s nothing to name individual elements of an ordinary numeric column.



## 37.6. Practice Problem

Confirm, using `length()` and `nrow()` together in the same check (with `&`, as in the dimensions page), that `participants` still has 8 columns and more than 90 rows.

## 37.7. Challenge Problem

Build a list `patient <- list(id = 1, age = 45, vitals = c(120, 80))`. Predict, before running any code, what `length(patient)` will return, and separately what `length(patient$vitals)` will return — explain why these two numbers are different even though both trace back to the same object.

## 37.8. Key Takeaways

- `length()` counts top-level elements — for a plain vector, that's individual values; for a data frame (structurally a list of columns), that's the number of *columns*, not rows.
- For row counts on a data frame, always use `nrow()`, not `length()`.
- `names()` retrieves the labels on a vector's elements, a list's elements, or a data frame's columns — what it returns depends on what labels actually exist on that particular object.

## 37.9. Where you've arrived

You started this level holding one value at a time. You can now build and index vectors every which way, convert messy categories into properly ordered factors, construct and slice data frames by row and column,

### 37. `length()` and `names()`

work with matrices and lists, understand tibbles as data frames with extra guardrails, and — on this very last page — correctly distinguish what `length()` actually measures on different kinds of objects, instead of falling into the same trap the intern did.

Level 3 picks up from here with the other side of this same coin: getting real, messy files *into* R reliably in the first place — CSV, Excel, and `.rds` files, project-relative paths, and the sanity checks you’d run on a dataset the moment it lands in your session.

#### Knowledge Check

1. **Multiple choice.** What does `length(participants)` return, if `participants` has 100 rows and 8 columns?

- a) 100
- b) 8
- c) 800
- d) NULL

2. **True or false.** A data frame is, structurally, a special kind of list where each column is one element.

3. **Predict the output.**

```
df <- data.frame(a = 1:5, b = 6:10, c = 11:15)
length(df)
```

4. **Find the error.** A student wants to confirm their cleaned dataset still has 250 rows and writes:

```
stopifnot(length(cleaned_data) == 250)
```

but the check fails unexpectedly even though the data looks correct.

Show answers

1. **b) 8** — `length()` on a data frame counts columns, not rows.

### 37.9. Where you've arrived

2. **True.** This is exactly why `length()` behaves the way it does on data frames, and why `$` and list-style indexing both work on them.
3. `[1] 3` — three columns (`a`, `b`, `c`), regardless of how many rows each contains.
4. The student used `length()` (which counts columns) when they meant `nrow()` (which counts rows) — the fix is `stopifnot(nrow(cleaned_data) == 250)`.



## **Part III.**

# **Level 3: Data Import and Export**



## Level 3 — Data Import and Export

### Where this level is headed

Real research data doesn't live inside your R script — it lives in CSV exports from REDCap, Excel files from a partner clinic, or `.rds` files a colleague sent you. Level 1 got you as far as a single clean `read.csv()` call; this level is about everything that happens when the file *isn't* clean — extra header rows, inconsistent missing-value codes, mismatched capitalization — and how to get data in and out of R without silently corrupting anything.

### What you'll build toward

By the end of this level you'll read and write CSV, Excel, and `.rds` files; understand file paths well enough to never fight `setwd()` again, using RStudio Projects instead; and — the real heart of this level — import a genuinely messy dataset, `data/participants_messy.csv`, and catch every way it could quietly corrupt your analysis before it does.

### Topics in this level

- Reading and writing CSV files (with `readr`, the tidyverse's faster, more predictable alternative to base R)
- Reading and writing Excel files
- Reading RDS files and saving R objects

### *Level 3 — Data Import and Export*

- File paths, relative paths, and RStudio Projects
- Importing messy, real-world datasets
- Handling missing values on import
- Checking imported data and understanding variable types after import

## **A preview mission**

Your mission across this level: take `data/participants_messy.csv` — a raw survey export exactly as it arrived from the field team, complete with metadata header lines, inconsistent capitalization, and three different disguises for a missing value — and load it into R without a single column silently corrupting along the way.

By the last page, you'll have a repeatable checklist you can run on *any* newly imported dataset, for the rest of this course and beyond.



## 38. Reading CSV Files with readr

### The Problem

Base R's `read.csv()`, which you learned in Level 1, works — but it's slow on large files, and it silently makes some decisions you might not want (like converting text columns in ways that vary subtly across R versions). A colleague mentions they always use `read_csv()` instead, from a package called **readr**, and that it “just tells you upfront what it did.” You want to know what's actually different.

### Your Mission

Import a CSV file using `readr::read_csv()`, and understand how it differs from base R's `read.csv()`.

### Think Before Coding

- From Level 1, what did `read.csv()` return, and what functions did you use afterward to check it imported correctly?
- If a function “tells you upfront what it did” while importing, what kind of information would be most useful to see immediately?

### 38.1. The R Concept

`read_csv()`, from the **readr** package (part of the tidyverse), reads CSV files with a few meaningful differences from base R's `read.csv()`:

### 38. Reading CSV Files with *readr*

```
library(readr)
my_data <- read_csv("path/to/file.csv")
```

- It returns a **tibble** (Level 2), not a plain data frame — with all the safety benefits you already know about.
- It prints a **column specification** message immediately after import, showing exactly what type it guessed for each column — surfacing potential problems immediately, rather than requiring you to run `str()` separately to find out.
- It's meaningfully **faster** on large files, which matters once your datasets grow beyond a few hundred rows.
- Unlike base R historically, it **never** converts character columns to factors automatically, and consistently treats blank fields as missing across all column types (a subtlety you'll see matters a lot on the missing-values page later in this level).

```
participants <- read_csv("data/participants.csv")
```

```
Rows: 100 Columns: 8
 Column specification
Delimiter: ","
chr (3): sex, county, smoker
dbl (5): id, age, systolic_bp, diastolic_bp, bmi
```

Use ``spec()`` to retrieve the full column specification for this data.  
Specify the column types or set ``show_col_types = FALSE`` to quiet this mess

## 38.2. Demonstration

```
library(readr)

participants <- read_csv("data/participants.csv")
```

### 38.3. Explore

```
Rows: 100 Columns: 8
 Column specification
Delimiter: ","
chr (3): sex, county, smoker
dbl (5): id, age, systolic_bp, diastolic_bp, bmi

class(participants)

[1] "spec_tbl_df" "tbl_df" "tbl" "data.frame"

participants

A tibble: 100 x 8
 id age sex county systolic_bp diastolic_bp bmi smoker
 <dbl> <dbl> <chr> <chr> <dbl> <dbl> <dbl> <chr>
1 1 46 Female Oakford 125 79 35.1 No
2 2 53 Female Riverdale 123 80 27.7 No
3 3 59 Female Oakford 130 77 28.3 No
... with 97 more rows
```

### 38.3. Explore

Compare `class(read.csv("data/participants.csv"))` (base R, from Level 1) against `class(read_csv("data/participants.csv"))` (this page). Notice `read_csv()`'s class includes `"tbl_df"` and `"data.frame"` both — a tibble is, structurally, still a data frame, exactly as you learned in Level 2. Then try adding `show_col_types = FALSE` to suppress the column specification message once you're confident you don't need to see it every time.

## 38.4. Solve the Problem

```
library(readr)

participants <- read_csv("data/participants.csv")

Rows: 100 Columns: 8
 Column specification
Delimiter: ","
chr (3): sex, county, smoker
dbl (5): id, age, systolic_bp, diastolic_bp, bmi
```

The column specification message immediately confirms, without running any additional code, that `age`, `systolic_bp`, `diastolic_bp`, and `bmi` were all correctly detected as numeric (`dbl`), and `sex`, `county`, and `smoker` were correctly detected as text (`chr`) — exactly what you’d expect, visible upfront rather than requiring a separate `str()` call to discover.

## 38.5. Interpret the Result

Seeing the column specification message print immediately after import means `read_csv()` guessed every column’s type correctly on this particular file — a genuinely useful sanity check that base R’s `read.csv()` doesn’t surface automatically. Later in this level, on a messier file, you’ll see this same message actively catch a column that *didn’t* import the way you’d expect.

### Common Mistakes

- **Forgetting to load `readr` (or `tidyverse`).** Just like any package, `read_csv()` requires `library(readr)` (or `library(tidyverse)`) first — otherwise you’ll get the familiar `could not find function` error from Level 1.

### 38.6. Practice Problem

- **Ignoring the column specification message.** Treating it as noise to scroll past, rather than as a free, automatic check on whether each column imported as the type you expected.
- **Assuming `read.csv()` and `read_csv()` are interchangeable in every respect.** They return different object classes (data frame vs. tibble) and differ in a few default behaviors — mixing them up mid-script can cause subtle inconsistencies.

## 38.6. Practice Problem

Import `data/participants.csv` using `read_csv()`, and without running `str()`, answer from the column specification message alone: how many character columns are there, and how many numeric columns?

## 38.7. Challenge Problem

Import `data/participants.csv` twice — once with `read.csv()` (base R) and once with `read_csv()` (readr) — into two separate objects. Compare `class()` of each, and explain in one sentence why both are still valid ways to get a “data frame” into R despite the different class labels.

## 38.8. Key Takeaways

- `readr::read_csv()` is a modern, faster alternative to base R’s `read.csv()`, returning a tibble and printing a column specification immediately after import.
- The column specification message is a free, automatic type-check — always worth reading, not scrolling past.
- A tibble from `read_csv()` is still fundamentally a data frame, exactly as you learned in Level 2.

### 38. Reading CSV Files with *readr*

#### Knowledge Check

1. **Multiple choice.** What does `read_csv()` return?
  - a) A plain base R data frame only
  - b) A tibble
  - c) A matrix
  - d) A list
2. **True or false.** `read_csv()` prints a column specification message showing the guessed type of every column.
3. **Predict the output.** If a CSV has one numeric column and one text column, what would `read_csv()`'s specification message show for column types?
4. **Write the code.** Write the code to load *readr* and import `data/participants.csv` using `read_csv()`.

Show answers

1. **b)** A tibble.
2. **True.** This message shows each column's guessed type immediately after import.
3. Something like `dbl (1): ...` for the numeric column and `chr (1): ...` for the text column.
4. 

```
library(readr)
participants <- read_csv("data/participants.csv")
```

## 39. Writing CSV Files

### The Problem

You've spent an hour cleaning and recoding the `participants` dataset — fixing capitalization, adding a BMI category column, filtering out a pilot test row. Your supervisor asks you to email the cleaned version to a colleague at another clinic, who doesn't use R at all. You need to get this data *out* of R and into a plain file they can open in Excel.

### Your Mission

Save a data frame or tibble from R to a CSV file on disk.

### Think Before Coding

- If `read_csv()` reads a file *into* R, what do you imagine the mirror-image function for writing *out* of R might be called?
- Your colleague doesn't use R — what file format would be usable by literally anyone, regardless of what software they have?

### 39.1. The R Concept

`write_csv()` (from `readr`) saves a data frame or tibble to a CSV file:

```
write_csv(data, "path/to/output.csv")
```

### 39. Writing CSV Files

Base R has an equivalent, `write.csv()`, but with one especially important difference worth knowing: base R's `write.csv()` includes row names as an extra, unnamed first column by default, unless you explicitly turn that off (`row.names = FALSE`) — a common source of a mysterious extra column when colleagues open your file. `readr`'s `write_csv()` never adds row names, which is one of several reasons this course prefers it.

```
write_csv(participants, "participants_cleaned.csv") # readr - no extra
write.csv(participants, "participants_cleaned.csv", row.names = FALSE) # base
```

## 39.2. Demonstration

```
library(readr)

A small cleaned example
cleaned_sample <- data.frame(
 id = c(1, 2, 3),
 age = c(46, 53, 59),
 county = c("Oakford", "Riverdale", "Oakford")
)

write_csv(cleaned_sample, "cleaned_sample.csv")

Reading it back in confirms it saved correctly
read_csv("cleaned_sample.csv")

A tibble: 3 x 3
 id age county
<dbl> <dbl> <chr>
1 1 46 Oakford
2 2 53 Riverdale
3 3 59 Oakford
```



### 39.3. Explore

Try the base R equivalent without `row.names = FALSE`: `write.csv(cleaned_sample, "test_with_rownames.csv")`, then open the resulting file (or read it back with `read_csv()`) and look for an unexpected, unnamed first column containing 1, 2, 3 — the row names base R added automatically. Compare that to `write_csv()`'s output, which never has this issue.

### 39.4. Solve the Problem

```
library(readr)

participants <- read_csv("data/participants.csv")

... cleaning steps happen here (Level 4 covers this properly) ...

write_csv(participants, "participants_cleaned.csv")
```

The file `participants_cleaned.csv` is now saved in your working directory, ready to attach to an email — a plain CSV that opens correctly in Excel, Google Sheets, or any other spreadsheet software your colleague uses, with no R installation required on their end at all.

### 39.5. Interpret the Result

`write_csv()` completing with no error message means the file was written successfully to disk. Unlike an R object living only in your current session (gone the moment you close RStudio, unless saved), a CSV file on disk persists permanently and can be opened by anyone — this is the entire point of *exporting* data, as opposed to just holding it in memory.

## 39. Writing CSV Files

### Common Mistakes

- **Forgetting `row.names = FALSE` with base R's `write.csv()`.** Without it, an unwanted, unnamed row-number column gets added to the output file — a frequent source of confusion for whoever opens it next.
- **Overwriting the original raw data file by accident.** Writing your cleaned output to the *same* filename as your original import (`data/participants.csv`) permanently destroys your original raw data — always write cleaned output to a clearly different filename.
- **Forgetting the file extension.** `write_csv(data, "output")` (no `.csv`) still creates a file, just without the extension most software expects — always include `.csv` explicitly.

### 39.6. Practice Problem

Create a small data frame with 3 participants' `id` and `bmi_category`, and write it to a file called `bmi_summary.csv` using `write_csv()`. Then read it back in to confirm it saved correctly.

### 39.7. Challenge Problem

Explain, in one or two sentences, why writing your cleaned output to a new filename (like `participants_cleaned.csv`) rather than overwriting `data/participants.csv` is a form of the same “reproducibility” principle from Level 1 — specifically, what would you lose if the original raw file were gone and only the cleaned version remained?

## 39.8. Key Takeaways

- `write_csv()` (readr) saves a data frame or tibble to disk as a CSV file, without adding an unwanted row-name column.
- Base R's `write.csv()` requires `row.names = FALSE` explicitly to avoid that same extra column.
- Always write cleaned or processed output to a new filename — never overwrite your original raw data file.

### Knowledge Check

1. **Multiple choice.** What does `write_csv()` do differently from base R's `write.csv()` by default?
  - a) It adds row names automatically
  - b) It never adds row names
  - c) It only works on numeric data
  - d) It requires a `.txt` extension
2. **True or false.** It's safe to save your cleaned data by overwriting the original raw data file.
3. **Predict the output.** After `write_csv(df, "out.csv")` runs successfully with no error, where does the resulting file live?
4. **Write the code.** Write the code to save a tibble called `results` to a file named `results_final.csv`.

Show answers

1. **b)** It never adds row names, unlike base R's `write.csv()` without `row.names = FALSE`.
2. **False.** Always preserve the original raw file; write cleaned output to a new filename.
3. In your current working directory, unless a different path was specified.

### 39. *Writing CSV Files*

```
4. write_csv(results, "results_final.csv")
```

## 40. Reading Excel Files

### The Problem

A partner clinic sends you their portion of the community health survey as `clinic_data.xlsx`, not a CSV. You try `read_csv("clinic_data.xlsx")`, following the pattern from earlier in this level — and it fails, producing garbled, nonsensical output instead of a clean error. CSV-reading functions have no idea what to do with Excel's actual file format.

### Your Mission

Import an Excel file (`.xlsx`) into R using the appropriate package and function.

### Think Before Coding

- A CSV file is plain text you could open in Notepad; is an `.xlsx` file the same kind of plain text, or something more complex under the hood?
- From the packages page in Level 1, what two steps are always required before using a function from a package you haven't used yet in this session?

### 40.1. The R Concept

Excel files (`.xlsx`, and the older `.xls`) are **not** plain text like CSV — they're a compressed, structured file format that can include multiple

## 40. Reading Excel Files

sheets, formatting, and formulas. Reading one requires a dedicated package: `readxl`.

```
library(readxl)
my_data <- read_excel("path/to/file.xlsx")
```

A few arguments matter especially often with real Excel files:

```
read_excel("file.xlsx", sheet = "Sheet2") # a specific sheet by name (or position)
read_excel("file.xlsx", skip = 2) # skip metadata rows before the first data row
read_excel("file.xlsx", na = c("", "N/A")) # custom missing-value codes, covering both empty and "N/A" cells
```

`read_excel()` (like `read_csv()`) returns a tibble, and — importantly — it does not modify the original Excel file at all; reading is always non-destructive.

### 40.2. Demonstration

```
library(readxl)

List every sheet in a workbook before importing, if you're not sure which one to use
excel_sheets("data/clinic_data.xlsx")

[1] "Enrollment" "Follow-up"

clinic_data <- read_excel("data/clinic_data.xlsx", sheet = "Enrollment")

clinic_data

A tibble: 25 x 6
 id age sex county systolic_bp smoker
 <dbl> <dbl> <chr> <chr> <dbl> <chr>
1 101 42 Male Oakford 129 Yes
```

```

2 102 26 Male Springvale 124 No
3 103 29 Female Cedar Falls 144 No
... with 22 more rows

```

## 40.3. Explore

Try `read_excel("data/clinic_data.xlsx")` with no `sheet` argument at all — by default, `read_excel()` reads the *first* sheet in the workbook. If that happens to not be the sheet you wanted (say, “Follow-up” instead of “Enrollment”), you’d get valid-looking but entirely wrong data with no error at all — which is exactly why checking `excel_sheets()` first is good practice on an unfamiliar file.

## 40.4. Solve the Problem

```

library(readxl)

CSV-reading functions can't parse Excel's file format at all
clinic_data <- read_excel("data/clinic_data.xlsx", sheet = "Enrollment")

nrow(clinic_data)

[1] 25

str(clinic_data)

tibble [25 x 6] (S3: tbl_df/tbl/data.frame)
 $ id : num [1:25] 101 102 103 104 105 ...
 $ age : num [1:25] 42 26 29 51 53 ...
 $ sex : chr [1:25] "Male" "Male" "Female" "Male" ...

```

## 40. Reading Excel Files

```
$ county : chr [1:25] "Oakford" "Springvale" "Cedar Falls" "Springvale"
$ systolic_bp: num [1:25] 129 124 144 129 167 ...
$ smoker : chr [1:25] "Yes" "No" "No" "No" ...
```

### 40.5. Interpret the Result

`clinic_data` now holds the 25-participant Enrollment sheet from the partner clinic's Excel file, correctly imported as a tibble — with `readxl` handling Excel's internal file structure automatically, something no CSV-reading function (base R or `readr`) could ever have done.

#### Common Mistakes

- **Trying to read an Excel file with `read_csv()` or `read.csv()`.** These functions only understand plain-text, comma-delimited files — Excel's actual binary/XML structure requires `readxl` (or a similar dedicated package).
- **Not checking which sheet you're actually reading.** `read_excel()` silently defaults to the first sheet in the workbook if `sheet` isn't specified — always check `excel_sheets()` first on an unfamiliar file to avoid analyzing the wrong data entirely.
- **Forgetting `library(readxl)`.** The same package-loading rule from Level 1 applies here exactly as everywhere else.

### 40.6. Practice Problem

List all the sheets in `data/clinic_data.xlsx` using `excel_sheets()`, then import the "Follow-up" sheet specifically and check its `nrow()`.



## 40.7. Challenge Problem

You're told a partner's file has a title row and a blank row before the real header starts (three rows total before the data). Write the `read_excel()` call that correctly skips past them, using the `skip` argument.

## 40.8. Key Takeaways

- Excel files require the `readxl` package (`read_excel()`) — CSV-reading functions cannot parse Excel's file format at all.
- `excel_sheets()` lists every sheet in a workbook before you import, helping you avoid silently reading the wrong (default first) sheet.
- `read_excel()` returns a tibble and never modifies the original file, exactly like `read_csv()`.

### Knowledge Check

1. **Multiple choice.** Which package provides `read_excel()`?
  - a) `readr`
  - b) `readxl`
  - c) `dplyr`
  - d) Base R, no package needed
2. **True or false.** `read_csv()` can correctly parse an `.xlsx` file if you just change the file extension in the function call.
3. **Predict the output.** If you call `read_excel("file.xlsx")` with no `sheet` argument on a workbook with 3 sheets, which sheet gets read?
4. **Write the code.** Write the code to load `readxl` and list every sheet name in `"data/clinic_data.xlsx"` without importing any data yet.

#### 40. Reading Excel Files

Show answers

1. **b)** `readxl`
2. **False.** `read_csv()` only understands plain-text CSV format; Excel files require `readxl`.
3. The first sheet in the workbook, by default.
4. `library(readxl)`  
`excel_sheets("data/clinic_data.xlsx")`

## 41. Writing Excel Files

### The Problem

You’ve calculated a summary table — mean blood pressure, BMI, and age by county — and your supervisor asks you to “put it in Excel so I can share it in tomorrow’s meeting.” They specifically want an actual `.xlsx` file, not a CSV, because they plan to add their own formatting and a second sheet with notes once you send it over.

### Your Mission

Save a data frame or tibble from R directly to an Excel file.

### Think Before Coding

- You already know `read_excel()` reads Excel files, using the `readxl` package. Do you expect the same package to also *write* Excel files, or might that require a different one?
- If your supervisor wants to eventually add a second sheet of notes, might it matter whether the function you use supports writing more than one sheet at once?

### 41.1. The R Concept

Writing Excel files uses a **different** package from reading them: `readxl` (Level 3’s earlier page) only reads; `writexl` is a common, lightweight package for writing:

## 41. Writing Excel Files

```
library(writexl)
write_xlsx(data, "path/to/output.xlsx")
```

`write_xlsx()` can also write **multiple sheets at once**, by passing a named list instead of a single data frame:

```
write_xlsx(
 list(Summary = summary_table, Notes = notes_table),
 "report.xlsx"
)
```

Each name in the list becomes a sheet name in the resulting workbook — directly answering the “second sheet with notes” part of today’s problem.

## 41.2. Demonstration

```
library(writexl)

county_summary <- data.frame(
 county = c("Cedar Falls", "Millbrook", "Oakford", "Riverdale", "Springvale"),
 mean_bp = c(128.16, 133.30, 129.71, 124.36, 131.46)
)

write_xlsx(county_summary, "county_summary.xlsx")

Confirm it saved correctly by reading it back
library(readxl)
read_excel("county_summary.xlsx")

A tibble: 5 x 2
 county mean_bp
 <chr> <dbl>
```

|               |      |
|---------------|------|
| 1 Cedar Falls | 128. |
| 2 Millbrook   | 133. |
| 3 Oakford     | 130. |
| 4 Riverdale   | 124. |
| 5 Springvale  | 131. |

## 41.3. Explore

Try writing two data frames at once as separate sheets: `write_xlsx(list(Summary = county_summary, Notes = data.frame(note = "Preliminary results, pending final QC")), "report_with_notes.xlsx")`. Then use `excel_sheets()` (from the reading-Excel page) on the resulting file to confirm both sheet names appear correctly.

## 41.4. Solve the Problem

```
library(writexl)

county_summary <- data.frame(
 county = c("Cedar Falls", "Millbrook", "Oakford", "Riverdale", "Springvale"),
 mean_bp = c(128.16, 133.30, 129.71, 124.36, 131.46)
)

notes <- data.frame(
 note = "Preliminary blood pressure summary by county - pending final QC review"
)

write_xlsx(
 list(Summary = county_summary, Notes = notes),
 "county_report.xlsx"
)
```

## 41. Writing Excel Files

The file `county_report.xlsx` is now ready to send — a real Excel workbook with a “Summary” sheet and a “Notes” sheet, exactly as your supervisor requested, ready for them to add their own formatting on top.

### 41.5. Interpret the Result

`write_xlsx()` completing without error confirms `county_report.xlsx` now exists on disk with two properly separated sheets — your supervisor can open it in Excel, see the “Summary” sheet’s numbers immediately, and switch to “Notes” without you needing to send two separate files or explain how to combine them.

#### Common Mistakes

- **Confusing which package reads versus writes.** `readxl::read_excel()` reads; `writexl::write_xlsx()` writes — they are two separate packages, despite the similar names, and neither can do the other’s job.
- **Forgetting the list structure for multiple sheets.** `write_xlsx(data)` (a single data frame) writes one sheet with a generic default name; to control sheet names or write several sheets, you need a *named list*, as shown in the solution.
- **Expecting `write_xlsx()` to preserve advanced Excel formatting.** It writes clean, plain data — colors, formulas, and conditional formatting from a previous version of a file are not preserved or added; that layer of polish is left for your supervisor (or a more specialized package) to add afterward.

### 41.6. Practice Problem

Create a small data frame summarizing mean age by sex (`sex = c("Female", "Male")`, `mean_age = c(48.2, 46.7)`), and write it to a

file called `age_by_sex.xlsx` using `write_xlsx()`.

## 41.7. Challenge Problem

Write two data frames — your county summary and a second small data frame with just the overall sample size (`data.frame(n_total = 100)`) — into one workbook called `final_report.xlsx`, with sheet names "By County" and "Overall". Then read both sheets back with `read_excel()` to confirm each imported correctly.

## 41.8. Key Takeaways

- `writexl::write_xlsx()` saves a data frame or tibble to a real Excel file — a separate package from `readxl`, which only reads.
- Passing a named list instead of a single data frame writes multiple sheets at once, with each name becoming a sheet name.
- Writing Excel files preserves the data cleanly but not advanced formatting — that layer is typically added afterward by whoever opens it in Excel.

### Knowledge Check

1. **Multiple choice.** Which package's function writes Excel files, as covered on this page?
  - a) `readxl::read_excel()`
  - b) `writexl::write_xlsx()`
  - c) `readr::write_csv()`
  - d) Base R's `write.csv()`
2. **True or false.** `readxl` can both read and write Excel files.

#### 41. Writing Excel Files

3. **Predict the output.** What determines the sheet names when writing multiple sheets with `write_xlsx()`?
4. **Write the code.** Write the code to save a data frame `results` to a single-sheet Excel file named `results.xlsx`.

Show answers

1. **b)** `writexl::write_xlsx()`
2. **False.** `readxl` only reads Excel files; writing requires a separate package like `writexl`.
3. The names in the list passed to `write_xlsx()` — each name becomes that sheet's name in the workbook.
4. `write_xlsx(results, "results.xlsx")`



## 42. Reading RDS Files

### The Problem

A colleague sends you `participants_cleaned.rds` instead of a CSV, saying “this already has the BMI category factor set up correctly, and I didn’t want that to get lost.” You open it the way you’d open a CSV, `read_csv("participants_cleaned.rds")`, and get garbled nonsense — `.rds` is not a text format at all, and your colleague’s comment about the factor “not getting lost” is a clue about *why* they chose it over a CSV in the first place.

### Your Mission

Read an `.rds` file — R’s own native format for saving a single R object exactly as it is.

### Think Before Coding

- From Level 2, what happens to a factor’s carefully specified level order if it’s exported to a CSV (a plain-text format with no concept of “factor”) and then re-imported?
- If R had a file format specifically designed to save *any* R object — including its exact type, factor levels, and structure — would that format need to be readable by non-R software at all?

## 42.1. The R Concept

`.rds` is R's native binary format for saving a *single* R object — a vector, a data frame, a factor, a list, anything — exactly as it exists in R, including all its type information. Unlike CSV, which flattens everything to plain text (losing factor level order, for instance), `.rds` preserves an object perfectly.

```
my_object <- readRDS("path/to/file.rds")
```

The trade-off: `.rds` files are R-specific — a colleague using Python or Excel cannot open one directly. This makes `.rds` the right choice for *saving your own intermediate work* (to reopen later, in R), and CSV or Excel the right choice for *sharing data with people or software outside R*.

## 42.2. Demonstration

```
Reading a previously saved R object back in
participants_cleaned <- readRDS("data/participants_cleaned.rds")

class(participants_cleaned$bmi_category)

[1] "factor"

levels(participants_cleaned$bmi_category)

[1] "Underweight" "Normal" "Overweight" "Obese"
```

Notice the factor's carefully specified level order (from Level 2's factors page) survived perfectly — exactly as your colleague intended when choosing `.rds` over CSV.

## 42.3. Explore

Compare this to what would happen with CSV: if that same `bmi_category` factor had instead been saved with `write_csv()` and reopened with `read_csv()`, the column would come back as plain character text, with the clinically meaningful order (Underweight, Normal, Overweight, Obese) completely lost — you'd need to rebuild the factor and its level order from scratch. This is precisely the loss your colleague avoided by using `.rds`.

## 42.4. Solve the Problem

```
.rds is not plain text - trying to read it as CSV produces garbage
read_csv("data/participants_cleaned.rds") # do not run - wrong tool entirely

The correct approach:
participants_cleaned <- readRDS("data/participants_cleaned.rds")

str(participants_cleaned)

'data.frame': 100 obs. of 9 variables:
 $ id : int 1 2 3 4 5 6 7 8 9 10 ...
 $ age : int 46 53 59 49 61 18 62 68 33 36 ...
 $ sex : chr "Female" "Female" "Female" "Female" ...
 $ county : chr "Oakford" "Riverdale" "Oakford" "Millbrook" ...
 $ systolic_bp : int 125 123 130 117 139 117 110 100 149 NA ...
 $ diastolic_bp: int 79 80 77 72 79 64 64 53 88 86 ...
 $ bmi : num 35.1 27.7 28.3 30.4 21.5 24 19.7 29.7 22.4 NA ...
 $ smoker : chr "No" "No" "No" "No" ...
 $ bmi_category: Factor w/ 4 levels "Underweight",...: 4 3 3 4 2 2 2 3 2 NA ...
```

## 42.5. Interpret the Result

`readRDS()` correctly restores `participants_cleaned` exactly as your colleague saved it, including `bmi_category` as a proper 4-level ordered factor — no reconstruction needed, unlike what would have happened if the same data had round-tripped through a CSV file instead.

### Common Mistakes

- **Trying to open .rds files with CSV or Excel tools.** `.rds` is a binary R-specific format — `read_csv()`, `read_excel()`, and non-R software cannot open it at all.
- **Assuming .rds is a good format for sharing data outside R.** It's the opposite: `.rds` is ideal for your own R-to-R workflow, but useless for a colleague using Python, Stata, or Excel — use CSV or Excel for external sharing (as covered earlier this level).
- **Forgetting .rds stores exactly one object.** Unlike a CSV file (naturally one table) or an Excel workbook (multiple sheets), a single `.rds` file holds exactly one R object — if you need to save several related objects together, a list (Level 2) is often the natural container to save as one `.rds` file.

## 42.6. Practice Problem

Assume a colleague sends you `study_summary.rds`, described as “a list with three elements: the cleaned data, a summary table, and today’s date.” Write the code to read it in and confirm, using tools from Level 2, that it does indeed have exactly 3 top-level elements.

## 42.7. Challenge Problem

Explain, in your own words, why `.rds` is the right choice for saving your own intermediate analysis steps within a single project, but the *wrong* choice for the final deliverable you'd email to a non-R collaborator — referencing what you learned about CSV and Excel earlier in this level.

## 42.8. Key Takeaways

- `.rds` is R's native format for saving a single R object exactly as it is, including factor levels and any other type information CSV would flatten away.
- `.rds` files are R-specific — use CSV or Excel instead when sharing data with non-R collaborators or software.
- A single `.rds` file always holds exactly one object; use a list to bundle several related objects into that one file.

### Knowledge Check

1. **Multiple choice.** What does `.rds` preserve that CSV does not?
  - a) Column names
  - b) Row count
  - c) Factor levels and exact R object type
  - d) File size
2. **True or false.** A colleague using Python can open an `.rds` file directly with no special tools.
3. **Predict the output.** If `x` was saved as `saveRDS(x, "x.rds")` where `x` was a factor with 3 levels, what would `levels(readRDS("x.rds"))` return?

## 42. Reading RDS Files

4. **Write the code.** Write the code to read `"data/model_results.rds"` into an object called `model_results`.

Show answers

1. **c)** Factor levels and the exact R object type — CSV flattens everything to plain text.
2. **False.** `.rds` is R-specific; Python and other non-R tools cannot open it directly without special conversion libraries.
3. The same 3 levels, in the same order, exactly as they existed before saving.
4. `model_results <- readRDS("data/model_results.rds")`

## 43. Saving R Objects

### The Problem

You’ve just spent twenty minutes building `participants_cleaned` — the dataset with a properly leveled `bmi_category` factor from Level 2, filtered and recoded exactly right. Tomorrow you’ll continue the analysis, and you don’t want to rerun all of today’s cleaning steps from scratch, nor do you want to risk losing the factor’s level order by saving it as a CSV (as you just learned two pages ago).

### Your Mission

Save an R object to disk in a way that preserves it exactly, ready to reload tomorrow with zero reconstruction needed.

### Think Before Coding

- You already know `readRDS()` reads an `.rds` file back into R. What would the natural, mirror-image function for *writing* one be called?
- Between a CSV (Level 3’s writing-CSV page) and an `.rds` file, which one is built specifically to avoid the factor-flattening problem you just saw?

### 43.1. The R Concept

`saveRDS()` writes a single R object to disk as an `.rds` file, preserving its exact type and structure — the save-side counterpart to `readRDS()`:

### 43. Saving R Objects

```
saveRDS(object, "path/to/file.rds")
```

```
saveRDS(participants_cleaned, "data/participants_cleaned.rds")
```

Reloading it later, in a future session, restores the object *exactly* as it was — same factor levels, same column types, same everything — with `readRDS()`:

```
participants_cleaned <- readRDS("data/participants_cleaned.rds")
```

#### **i** `saveRDS()` vs. `save()`

You may also encounter `save()` in older code, which can bundle *multiple* named objects into one `.RData` file, reloaded with `load()`. This course prefers `saveRDS()/readRDS()` because they work with exactly one object at a time, assigned explicitly to a name you choose on reload — `load()` instead restores objects under their *original* names automatically, which can silently overwrite an existing object in your session with the same name.

## 43.2. Demonstration

```
participants_cleaned <- participants
participants_cleaned$bmi_category <- factor(
 ifelse(participants_cleaned$bmi < 18.5, "Underweight",
 ifelse(participants_cleaned$bmi < 25, "Normal",
 ifelse(participants_cleaned$bmi < 30, "Overweight", "Obese"))),
 levels = c("Underweight", "Normal", "Overweight", "Obese")
)
```

```
saveRDS(participants_cleaned, "data/participants_cleaned.rds")
```



### 43.3. Explore

```
Confirm it round-trips perfectly
reloaded <- readRDS("data/participants_cleaned.rds")
identical(participants_cleaned, reloaded)
```

```
[1] TRUE
```

### 43.3. Explore

Try saving the same object as a CSV instead (`write_csv(participants_cleaned, "test.csv")`), then reload it with `read_csv("test.csv")` and check `class(reloaded_csv$bmi_category)`. Compare that to `class(reloaded$bmi_category)` from the `.rds` round-trip above — one preserves "factor", the other silently becomes "character".

### 43.4. Solve the Problem

```
Today's cleaning work, ready to preserve exactly
saveRDS(participants_cleaned, "data/participants_cleaned.rds")

Tomorrow, in a fresh R session:
participants_cleaned <- readRDS("data/participants_cleaned.rds")

class(participants_cleaned$bmi_category)

[1] "factor"
```

Tomorrow's session starts exactly where today's left off — the factor's level order intact, with zero re-cleaning required.

## 43.5. Interpret the Result

`identical()` returning `TRUE` confirms the object that came back out of `readRDS()` is, in every respect, identical to the object that went into `saveRDS()` — no information lost, no type quietly changed, unlike the CSV round-trip you tested in the Explore step.

### Common Mistakes

- **Using `.rds` when you actually need to share the file with someone outside R.** `.rds` preserves everything perfectly, but only within R — for external sharing, go back to CSV or Excel (earlier this level).
- **Confusing `saveRDS()/readRDS()` with `save()/load()`.** The latter restores objects under their original saved names automatically, which can silently clobber an existing object of the same name in your current session — `saveRDS()/readRDS()` avoid this by requiring you to explicitly assign the reloaded object to a name yourself.
- **Forgetting to actually rerun `saveRDS()` after further changes.** An `.rds` file is a frozen snapshot at the moment you saved it — if you keep working on `participants_cleaned` afterward, the file on disk won't reflect those later changes until you save it again.

## 43.6. Practice Problem

Create a small list called `analysis_notes` with elements `date = "2026-07-23"` and `analyst = "your name"`. Save it with `saveRDS()` to `notes.rds`, then reload it into a new object and confirm with `identical()` that the round-trip preserved it exactly.

## 43.7. Challenge Problem

Explain, in your own words, why a research team maintaining a multi-step cleaning pipeline (raw import → clean → recode → analyze) would want to `saveRDS()` the output of each intermediate step, rather than only saving the very final result. What would you gain if step 3 of 5 turned out to have a bug?

## 43.8. Key Takeaways

- `saveRDS()` writes a single R object to disk, exactly preserving its type and structure; `readRDS()` restores it later, identically.
- `saveRDS()/readRDS()` are preferred over `save()/load()` because they require explicit naming on reload, avoiding silent overwrites.
- Use `.rds` for your own R-to-R intermediate work; use CSV or Excel (earlier this level) when the destination is outside R.

### Knowledge Check

1. **Multiple choice.** What does `saveRDS()` do?
  - a) Saves a plot to a PNG file
  - b) Saves a single R object to disk, preserving its exact structure
  - c) Saves multiple objects under their original names
  - d) Converts a data frame to a CSV
2. **True or false.** `readRDS()` requires you to explicitly assign the result to an object name.
3. **Predict the output.**

```
x <- factor(c("Low", "High"), levels = c("Low", "High"))
saveRDS(x, "x.rds")
```

#### 43. Saving R Objects

```
y <- readRDS("x.rds")
identical(x, y)
```

4. **Write the code.** Write the code to save a data frame `cleaned_data` to `"data/cleaned_data.rds"`.

Show answers

1. **b)** Saves a single R object to disk, preserving its exact structure.
2. **True.** Unlike `load()`, which restores objects under their original names automatically, `readRDS()` requires you to assign its result explicitly.
3. `[1] TRUE`
4. `saveRDS(cleaned_data, "data/cleaned_data.rds")`

## 44. File Paths and Relative Paths

### The Problem

Your script has `participants <- read.csv("/Users/jordan/Documents/health-survey-project/data/pa` — the absolute path from Level 1's working-directory page. You send the script to a classmate, whose computer is a different operating system with a completely different folder structure. It fails immediately, because that exact path only exists on your machine.

### Your Mission

Understand the difference between absolute and relative file paths deeply enough to write scripts that work on *any* computer, not just your own.

### Think Before Coding

- If two people both have a project folder called `health-survey-project`, but it lives at a different absolute location on each of their computers, what's the one thing likely to stay the same *inside* that folder?
- From Level 1, what did `getwd()` tell you, and what did a path like `"data/participants.csv"` depend on?

### 44.1. The R Concept

An **absolute path** describes a file's exact location starting from the very root of the file system — `/Users/jordan/Documents/health-survey-project/data/participants.csv` (Mac/Linux) or `C:/Users/Jordan/Documents/health-survey-project/data/participants.csv`

## 44. File Paths and Relative Paths

(Windows). It's precise, but only valid on the one computer where that exact folder structure exists.

A **relative path** describes a file's location *relative to the current working directory* — `data/participants.csv` says “look inside a folder called `data`, relative to wherever we currently are,” without caring what's above that in the folder structure. As long as two people both have the same *internal* project structure (a `data` folder inside their project folder, wherever that project folder happens to live), the same relative path works for both of them.

```
Absolute - only works on Jordan's computer
read.csv("/Users/jordan/Documents/health-survey-project/data/participants.csv")

Relative - works on anyone's computer, as long as their working directory
is set to the project folder, and it contains a "data" subfolder
read.csv("data/participants.csv")
```

This is exactly why every code example since Level 1 has used `"data/participants.csv"` — a relative path, deliberately, so the exact same code works on your computer, a classmate's, or a grading server, without a single change.

### 44.2. Demonstration

```
getwd()

[1] "/Users/jordan/Documents/health-survey-project"

Relative path - depends on the working directory shown above
participants <- read.csv("data/participants.csv")
nrow(participants)
```

### 44.3. Explore

```
[1] 100
```

```
The SAME relative path, run by a classmate with a totally different absolute location
but an identical internal project structure, works identically:
getwd() -> "/home/priya/school/health-survey-project"
read.csv("data/participants.csv") -> works exactly the same way
```

### 44.3. Explore

Run `getwd()` on your own machine right now, and compare it to a classmate's, if possible. Your absolute paths are almost certainly completely different — but if you both run `read.csv("data/participants.csv")` from within your respective project folders, both succeed identically.

### 44.4. Solve the Problem

```
Broken - hardcoded to one specific computer
participants <- read.csv("/Users/jordan/Documents/health-survey-project/data/participants.csv")

Fixed - portable relative path, works on any computer with the same internal structure
participants <- read.csv("data/participants.csv")
```

Sending this fixed version to your classmate now works immediately on their machine too, with no editing required on their end — the entire point of preferring relative paths in shared or collaborative code.

### 44.5. Interpret the Result

Switching from an absolute to a relative path doesn't change what the code *does* on your own machine — `nrow(participants)` still returns 100

#### 44. File Paths and Relative Paths

either way — but it changes whether the code is *portable*, which matters enormously the moment more than one person, or more than one computer, is involved.

##### Common Mistakes

- **Hardcoding an absolute path in shared code.** This is one of the most common reasons a script “works on my machine” but fails immediately for anyone else.
- **Assuming a relative path works from any starting point.** A relative path is only correct *relative to a specific working directory* — if your working directory isn’t set to the project folder, “data/participants.csv” will fail even though the file genuinely exists somewhere on disk.
- **Mixing backslashes into paths on Windows.** As covered in Level 1, R paths always use forward slashes (/), regardless of operating system — even on Windows.

#### 44.6. Practice Problem

You and a classmate both have a project folder structure with a `data` subfolder and a `scripts` subfolder inside your respective (differently located) project folders. Write the relative path you’d use, from within a script that assumes the working directory is the project folder, to read a file called `survey_2024.csv` sitting in `data`.

#### 44.7. Challenge Problem

Explain, in your own words, why a relative path alone still isn’t a *complete* solution to portability — what has to be true about the working directory for a relative path to work correctly? (This sets up the next page’s solution.)



## 44.8. Key Takeaways

- Absolute paths describe a file’s exact location from the root of the file system; they only work on the one computer where that exact structure exists.
- Relative paths describe a location relative to the current working directory; the same relative path works across different computers, as long as the internal project structure matches.
- Relative paths still depend on the working directory being set correctly — they solve *portability of the path itself*, but not yet the working-directory problem from Level 1 entirely (the next page finishes that job).

### Knowledge Check

1. **Multiple choice.** Which of these is a relative path?
  - a) `/Users/jordan/data/file.csv`
  - b) `C:/Users/Jordan/data/file.csv`
  - c) `data/file.csv`
  - d) All of the above are equally relative
2. **True or false.** A relative path will work identically on any computer, regardless of the current working directory.
3. **Predict the output.** If `getwd()` returns `"/home/priya/project"` and the file is at `"/home/priya/project/data/survey.csv"`, what relative path correctly loads it?
4. **Find the error.** A student shares a script containing `read.csv("/Users/alex/Desktop/data.csv")` with a classmate, who gets a “file not found” error even though they have the identical data file.

Show answers

#### 44. *File Paths and Relative Paths*

1. `c) data/file.csv`
2. **False.** A relative path depends entirely on the current working directory being set to the right location first.
3. `read.csv("data/survey.csv")`
4. The script hardcodes an absolute path specific to Alex's computer (`/Users/alex/Desktop/...`); the classmate's computer has a different folder structure, so that exact path doesn't exist for them. The fix is a relative path plus a correctly set working directory (or, better, an RStudio Project — next page).

## 45. RStudio Projects

### The Problem

You’ve switched to relative paths, as the last page recommended. But you still have to remember to run `setwd()` at the start of every session, pointing to wherever your project folder happens to live on *your* machine — and if you forget, or open the wrong folder, every relative path in your script breaks. You want the working directory to just always be correct, automatically, without you thinking about it.

### Your Mission

Set up an RStudio Project so your working directory is always correct automatically, finally solving the working-directory problem for good.

### Think Before Coding

- If RStudio could somehow “remember” which folder belongs to which project, what would it need to automatically do the moment you opened that project?
- Would you rather manually run `setwd()` every single session, or have this happen automatically the moment you open your work?

### 45.1. The R Concept

An **RStudio Project** is a folder containing a special `.Rproj` file that marks it as a self-contained project. Opening that `.Rproj` file (by double-clicking it, or via **File > Open Project** in RStudio) automatically:

## 45. RStudio Projects

- Sets your working directory to the project folder itself — no `setwd()` needed, ever, for anyone who opens that project.
- Keeps your Environment, open scripts, and history organized separately from other projects.
- Makes every relative path inside your scripts (like `"data/participants.csv"`) work correctly and identically for *anyone* who opens the same project folder, on any computer.

Creating one: **File > New Project > New Directory** (or **Existing Directory**, if you already have a folder like this course's R-PBL/), then choose “New Project,” and RStudio creates a `.Rproj` file inside it.

```
Once inside an RStudio Project, this just works - no setwd() ever needed
participants <- read.csv("data/participants.csv")
```

**i** An even more robust alternative: the `here` package

Some teams go one step further with the `here` package, which finds your project's root folder automatically (by looking for the `.Rproj` file) and builds paths from there — `here::here("data", "participants.csv")` works correctly even when called from a script buried several subfolders deep. RStudio Projects alone solve the vast majority of path problems you'll encounter in this course; `here` is worth knowing about as you take on larger, multi-folder projects later on.

### 45.2. Demonstration

```
BEFORE an RStudio Project: manual setwd() needed every session
setwd("/Users/jordan/Documents/health-survey-project") # different for every
participants <- read.csv("data/participants.csv")
```

### 45.3. Explore

```
AFTER opening R-PBL.Rproj: working directory is already correct, automatically
getwd()

[1] "/Users/jordan/Documents/health-survey-project"

No setwd() anywhere in the script - it's simply never needed again
participants <- read.csv("data/participants.csv")
nrow(participants)

[1] 100
```

### 45.3. Explore

If you have RStudio installed, try creating a new project from an existing folder (**File > New Project > Existing Directory**) using any folder on your computer. Notice a new `.Rproj` file appears inside it, and that closing and reopening RStudio *through that .Rproj file* automatically returns you to the correct working directory every time.

### 45.4. Solve the Problem

The fix isn't a line of code at all — it's a one-time setup step: turn your project folder into an RStudio Project by creating a `.Rproj` file inside it (**File > New Project**), then always open your work by double-clicking that `.Rproj` file, rather than opening RStudio generically and navigating to your files afterward.

```
From that point forward, every script in this project can simply use:
participants <- read.csv("data/participants.csv")

...with zero setwd() calls, on any computer, for any collaborator who opens the project.
```

## 45.5. Interpret the Result

Once you're working inside an RStudio Project, `getwd()` always reports the project's own folder — automatically, the moment you open it — which means every relative path from the previous page now works reliably, permanently, for you and for anyone else who opens the same project.

### Common Mistakes

- **Opening RStudio generically instead of through the `.Rproj` file.** If you launch RStudio first and then navigate to your files manually, the working directory is *not* automatically set to your project folder — always open the `.Rproj` file directly (or use **File > Open Project**) to get the automatic benefit.
- **Still including `setwd()` inside a script, out of habit, even within an RStudio Project.** This actually works against the whole point — a hardcoded `setwd()` will override the project's automatic, portable behavior and reintroduce the exact one-computer-only fragility this page just fixed.
- **Forgetting the `.Rproj` file needs to actually be committed or shared alongside the rest of the project folder.** If you email just your scripts and data but not the `.Rproj` file, a collaborator won't get the automatic working-directory benefit — they'd need to create their own Project pointing at the folder you sent.

## 45.6. Practice Problem

Describe, step by step, exactly what you would click in RStudio to turn an existing folder called `my-analysis` into an RStudio Project (you do not need to actually have the folder — describe the menu steps from memory).

## 45.7. Challenge Problem

A team of four analysts share a single RStudio Project folder via a shared drive. Explain, in one or two sentences, why every one of their scripts can safely use relative paths like `read.csv("data/participants.csv")` with zero `setwd()` calls anywhere, even though all four analysts have the shared folder mounted at a different absolute path on their own machines.

## 45.8. Key Takeaways

- An RStudio Project (a `.Rproj` file inside a folder) automatically sets the working directory to that folder the moment it's opened — no manual `setwd()` needed, ever again.
- Always open your work by opening the `.Rproj` file directly, not by launching RStudio generically first.
- Combined with relative paths (previous page), RStudio Projects make a script fully portable across computers and collaborators — the complete, permanent fix to the working-directory problem first raised in Level 1.

### Knowledge Check

1. **Multiple choice.** What does opening a `.Rproj` file automatically do?
  - a) Installs all required packages
  - b) Sets the working directory to that project's folder
  - c) Cleans your data automatically
  - d) Converts your scripts to Quarto documents
2. **True or false.** Including `setwd()` inside a script within an RStudio Project is still good practice.

#### 45. RStudio Projects

3. **Predict the output.** If a `.Rproj` file lives at `/home/priya/project/analysis.Rproj` what will `getwd()` return immediately after opening it?
4. **Find the error.** A team's shared script includes `setwd("/Users/jordan/Documents/...")` at the top, even though the whole team works inside a shared RStudio Project. What's wrong with this line?

Show answers

1. **b)** Sets the working directory to that project's folder, automatically.
2. **False.** A hardcoded `setwd()` overrides the Project's automatic, portable working directory, reintroducing the exact one-computer-only fragility RStudio Projects are meant to eliminate.
3. `/home/priya/project`
4. The hardcoded `setwd()` only works on Jordan's computer and breaks the working directory for every other team member — it should be removed entirely, letting the RStudio Project handle the working directory automatically.



## 46. Importing Messy, Real-World Datasets

### The Problem

The field team sends you their raw export, `participants_messy.csv` — and it is nothing like the clean `participants.csv` you’ve used since Level 1. Opening it in a text editor, you see two lines of description text before the actual header row, county names spelled with inconsistent capitalization and stray spaces, and it’s not yet clear whether all the missing values even look the same way twice. Your usual one-line `read.csv()` call doesn’t feel safe to trust blindly here.

### Your Mission

Import a genuinely messy CSV file correctly, handling extra header rows and inconsistent text formatting along the way.

### Think Before Coding

- If a file has two lines of description text before the real column headers, what do you think happens if you `read.csv()` it as if the first line were the header?
- “Riverdale”, “riverdale”, and ” Riverdale” all refer to the same county to a human reading them — would R’s `table()` function agree, or would it count them as three different things?

## 46.1. The R Concept

Real exports are rarely as clean as a textbook example. Two of the most common problems, and their fixes:

**Extra rows before the real header** — use the `skip` argument to tell `read.csv()` (or `read_csv()`) how many lines to skip before it starts looking for the header row:

```
read.csv("file.csv", skip = 2) # ignore the first 2 lines entirely
```

**Inconsistent capitalization and stray whitespace** in text columns — these don't stop a file from importing (unlike a genuinely wrong data type, which you'll see two pages from now), but they silently create *duplicate-looking categories* that are actually the same real-world value. `trimws()` removes leading/trailing whitespace; wrapping a column in `tolower()` (or `toupper()`) standardizes case. Full systematic cleaning of variables like this belongs to Level 4 — for now, the goal is simply to *notice* the problem immediately after import, using `table()` or `unique()`.

```
table(participants_messy$county) # reveals how many distinct-looking values
```

## 46.2. Demonstration

```
Naively reading without skip - the first "row" of data becomes garbage column
broken_attempt <- read.csv("data/participants_messy.csv")
names(broken_attempt)
```

```
[1] "Community.Health.Survey...Raw.Export"
```

```
Correctly skipping the 2 metadata lines before the real header
participants_messy <- read.csv("data/participants_messy.csv", skip = 2)
names(participants_messy)
```

### 46.3. Explore

```
[1] "id" "age" "sex" "county" "systolic_bp"
[6] "diastolic_bp" "bmi" "smoker"
```

```
Checking for hidden duplicate categories in a text column
table(participants_messy$county)
```

```

Riverdale CEDAR FALLS Cedar Falls MILLBROOK Millbrook OAKFORD
 1 1 1 2 3 2
Oakford Riverdale Springvale Springvale oakford riverdale
 1 1 1 2 1 3
springvale
 1
```

### 46.3. Explore

Compare `length(unique(participants_messy$county))` (how many distinct-looking values R currently sees) to the number of *actual* counties you know exist in this dataset (5, from Level 2). The gap between those two numbers is exactly the size of the capitalization/whitespace problem this page surfaced — the real fix (standardizing them into one clean set of 5) is Level 4’s job, but recognizing the gap immediately after import is this page’s job.

### 46.4. Solve the Problem

```
participants_messy <- read.csv("data/participants_messy.csv", skip = 2)
```

```
nrow(participants_messy)
```

```
[1] 20
```

## 46. Importing Messy, Real-World Datasets

```
length(unique(participants_messy$county))
```

```
[1] 13
```

Thirteen distinct-*looking* county values, for what you know should really be 5 actual counties — confirming, immediately and quantitatively, that this column needs cleanup before any county-level summary (like the mean-BP-by-county calculation from Level 2) could be trusted on this file as-is.

### 46.5. Interpret the Result

Importing with `skip = 2` correctly recovers all 8 real column names instead of a single garbled one; `table()` on `county` reveals 13 distinct-looking entries where only 5 real counties exist — proof that capitalization and whitespace inconsistencies are silently fragmenting a single true category into several apparent ones, exactly the kind of problem that would quietly wreck a `group_by(county)` summary (Level 5) if it went unnoticed.

#### Common Mistakes

- **Not checking `names()` immediately after importing an unfamiliar file.** A garbled single-column-name result (like "Community.Health.Survey...Raw.Export") is an immediate, obvious sign that extra header rows need `skip`.
- **Assuming inconsistent capitalization is harmless.** It's not — "Riverdale" and "riverdale" are treated as completely different, unrelated text values by every function you've learned so far, including `table()`, `==`, and (in Level 5) `group_by()`.
- **Fixing messiness by hand, row by row.** Even a 20-row file like this one is tedious to fix manually; Level 4 teaches systematic, scalable ways to standardize an entire column at once, applicable whether the file has 20 rows or 20,000.

## 46.6. Practice Problem

Import `data/participants_messy.csv` with the correct `skip` value, then run `table(participants_messy$sex)` — is this column also affected by the same capitalization problem, or does it look clean?

## 46.7. Challenge Problem

Without changing anything else, use `trimws(tolower(participants_messy$county))` to preview what the county column *would* look like if whitespace and case were standardized. Run `length(unique(...))` on this cleaned-up preview and confirm it now correctly equals 5.

## 46.8. Key Takeaways

- The `skip` argument tells `read.csv()` (or `read_csv()`) to ignore a fixed number of lines before looking for the real header row — essential for files with metadata or title lines at the top.
- `table()` or `unique()` on a text column immediately after import is the fastest way to spot hidden, inconsistent duplicate categories caused by capitalization or whitespace.
- Recognizing messiness is this page’s job; systematically fixing it at scale is Level 4’s.

### Knowledge Check

1. **Multiple choice.** What does the `skip` argument do in `read.csv()`?
  - a) Skips columns with missing data
  - b) Skips a specified number of lines before reading the header and data

#### 46. *Importing Messy, Real-World Datasets*

- c) Skips duplicate rows
  - d) Skips the entire import
2. **True or false.** "Riverdale" and "riverdale" are treated as the same value by `table()`.
  3. **Predict the output.** If a CSV has 3 metadata lines before the real header, what argument value would you pass to `skip`?
  4. **Find the error.** A student runs `read.csv("participants_messy.csv")` with no `skip` argument and gets a data frame with only one very oddly named column.

Show answers

1. **b)** Skips a specified number of lines before reading the header and data.
2. **False.** They are treated as two entirely different text values — capitalization matters in R string comparisons.
3. `skip = 3`
4. The file has metadata lines before the real header, and without `skip`, R tried to treat the first metadata line as the header — the fix is adding the correct `skip` value (here, `skip = 2`).

## 47. Handling Missing Values on Import

### The Problem

You import `participants_messy.csv` and run `mean(participants_messy$age, na.rm = TRUE)` — it runs without error and gives you a number. But looking closer at the raw file, you notice one participant’s age was entered as `999`, a placeholder some field teams use for “not reported,” rather than a genuine age. R has no way to know `999` means “missing” instead of “a 999-year-old participant” — unless you tell it.

### Your Mission

Correctly recognize custom, non-standard missing-value codes during import, instead of letting them masquerade as real data.

### Think Before Coding

- Level 1 taught you that `NA` (the literal text, unquoted) is R’s missing-value symbol. What do you think `read.csv()` does, by default, if it sees the literal text `"NA"` in a cell?
- What if a field team instead used `"999"`, `"N/A"` (with a slash), or a completely blank cell to mean the same thing — would R recognize all of these as missing by default?

## 47.1. The R Concept

By default, `read.csv()` only recognizes the literal text "NA" as a missing value. Any *other* placeholder — "999", "N/A", "Unknown", "-", or similar — is imported as if it were genuine data, silently, with no warning at all. The fix is the `na.strings` argument, where you explicitly list every code your data uses for “missing”:

```
read.csv("file.csv", na.strings = c("NA", "999", "N/A", "Unknown"))
```

There’s one more subtlety worth knowing precisely: **blank (empty) fields are automatically treated as missing for numeric columns by default, but not for character columns** — an empty cell in a numeric column becomes NA automatically, but an empty cell in a text column becomes "" (an empty string, not NA) unless you add "" to `na.strings` explicitly.

```
read.csv("file.csv", na.strings = c("NA", "999", "N/A", "Unknown", ""))
```

## 47.2. Demonstration

```
Default import - only "NA" is recognized as missing
default_import <- read.csv("data/participants_messy.csv", skip = 2)
```

```
The 999 "missing" age is NOT caught - it looks like real data
default_import$age
```

```
[1] 46 53 59 45 61 18 62 999 33 36 62 39 NA 41 35 45 79 26
```

```
mean(default_import$age, na.rm = TRUE)
```

```
[1] 98.63158
```



### 47.3. Explore

That mean is wildly, obviously wrong — nearly 99 years old on average is implausible for this sample, and it’s because the fake 999 “age” got averaged in as if it were a real 999-year-old participant.

```
Correctly specifying every missing-value code actually used in this file
participants_messy <- read.csv(
 "data/participants_messy.csv", skip = 2,
 na.strings = c("NA", "999", "N/A", "Unknown", "")
)
```

```
participants_messy$age
```

```
[1] 46 53 59 45 61 18 62 NA 33 36 62 39 NA 41 35 45 79 26 80 55
```

```
mean(participants_messy$age, na.rm = TRUE)
```

```
[1] 48.61111
```

### 47.3. Explore

Run `sum(is.na(default_import$age))` versus `sum(is.na(participants_messy$age))` — the first correctly catches only the one genuine "NA" text value; the second, with `na.strings` set properly, also catches the disguised 999. The difference between these two counts is exactly the size of the silent problem `na.strings` fixes.

### 47.4. Solve the Problem

```
participants_messy <- read.csv(
 "data/participants_messy.csv", skip = 2,
 na.strings = c("NA", "999", "N/A", "Unknown", "")
)
```

## 47. Handling Missing Values on Import

```
)
```

```
Now genuinely missing in every relevant column, not disguised as real data
sum(is.na(participants_messy$age))
```

```
[1] 2
```

```
mean(participants_messy$age, na.rm = TRUE)
```

```
[1] 48.61111
```

The corrected mean age, 48.6 years, is far more plausible than the earlier 98.6 — and critically, it’s now based on genuinely reported ages only, with both real missing values (the blank/"NA" entries) and the disguised one (999) correctly excluded.

### 47.5. Interpret the Result

Specifying `na.strings` explicitly changed the calculated mean age from an implausible 98.6 years to a realistic 48.6 years — not because the underlying data changed, but because R finally recognized 999 for what it actually was: a missing value in disguise, not a genuine data point. This is exactly the kind of silent, unflagged error that a “sanity check” (Level 1’s `mean()` page) is meant to catch — and here, the sanity check *worked*, because 98.6 was obviously implausible enough to notice.

#### Common Mistakes

- **Trusting the default `na.strings = "NA"` on unfamiliar data.** Any dataset from an external source may use its own missing-value codes — always ask (or check the file directly) what codes were actually used before importing for real analysis.

## 47.6. Practice Problem

- **Not noticing an implausible result.** A mean age of 96.75 should immediately raise suspicion — always sanity-check summary statistics against what's plausible for your actual population, exactly as you did in Level 1.
- **Forgetting blank fields behave differently by column type.** An empty cell becomes `NA` automatically in a numeric column, but stays as `" "` (not `NA`) in a character column unless `" "` is explicitly added to `na.strings` — this asymmetry is easy to overlook and can leave blank text fields undetected by `is.na()`.

## 47.6. Practice Problem

Import `data/participants_messy.csv` with the correct `skip` and `na.strings` arguments (as in the solution), then check `sum(is.na(participants_messy$bmi))` — how many BMI values are genuinely missing once `"Unknown"` is correctly recognized as a missing-value code?

## 47.7. Challenge Problem

The `smoker` column has one blank entry in this file. Without `" "` in `na.strings`, what would `is.na(participants_messy$smoker)` report for that row — `TRUE` or `FALSE`? Import the file both with and without `" "` in `na.strings` to confirm your prediction, and explain in one sentence why this matters for a calculation like `sum(participants_messy$smoker == "Yes")`.

## 47.8. Key Takeaways

- `read.csv()` only recognizes the literal text `"NA"` as missing by default; any other placeholder code must be listed explicitly in

## 47. Handling Missing Values on Import

`na.strings`.

- Blank fields become NA automatically in numeric columns, but stay as "" in character columns unless "" is added to `na.strings` — an easy-to-miss asymmetry.
- An implausible summary statistic (like a mean age of 96.75) is often your first, best clue that a disguised missing-value code has slipped through undetected.

### Knowledge Check

1. **Multiple choice.** What does `read.csv()` recognize as missing by default, with no `na.strings` argument?
  - a) Any blank field
  - b) Only the literal text "NA"
  - c) "999" and "N/A" automatically
  - d) Nothing — you must always specify `na.strings`
2. **True or false.** A blank field in a character column automatically becomes NA by default, with no `na.strings` needed.
3. **Predict the output.** If a numeric column has values 10, 20, "", what does the third value become after `read.csv()`, by default?
4. **Write the code.** Write the `read.csv()` call to import "survey.csv", treating "NA", "-99", and blank fields all as missing.

Show answers

1. **b)** Only the literal text "NA".
2. **False.** Blank fields in numeric columns become NA automatically, but blank fields in character columns become "", not NA, unless "" is explicitly added to `na.strings`.
3. **NA** — blank fields in numeric columns are automatically converted to NA.

#### 47.8. Key Takeaways

```
4. read.csv("survey.csv", na.strings = c("NA", "-99", ""))
```



## 48. Checking Imported Data

### The Problem

You’ve now learned, across this level, that a file can have extra header rows, disguised missing-value codes, and inconsistent capitalization — each discovered somewhat by accident, one page at a time. Before your next real analysis, you want a single, repeatable checklist you run on *any* freshly imported dataset, so you catch all of these problems immediately, together, rather than stumbling onto them one at a time.

### Your Mission

Build a systematic, repeatable checklist for verifying any newly imported dataset before trusting it for analysis.

### Think Before Coding

- Across this entire level, which functions have you used to catch a problem *immediately after* importing a file?
- If you had to explain to a new intern “always run these steps first, no matter what file you just imported,” what would you tell them to check, in order?

### 48.1. The R Concept

There’s no single function that catches everything — but combining tools from this level and Level 2 into one consistent post-import routine catches nearly every common problem:

## 48. Checking Imported Data

```
dim(data) # does row/column count match expectations?
names(data) # are column names sensible, not garbled? (tbl)
str(data) # did every column import as the expected type?
head(data) # do the first few rows look right, at a glance
apply(data, function(x) sum(is.na(x))) # how much missing data per column?
apply(data[apply(data, is.character)], function(x) length(unique(x))) # how many unique values per column?
```

**i** Don't worry about the `apply()` syntax yet

The last two lines use `apply()` to run a check across *every* column at once, rather than one at a time — you haven't formally learned it yet (Level 10 covers it properly), but you can already read what it's *for*: applying the same check to every column automatically instead of retyping it once per column.

Running this checklist takes seconds and turns “I hope this imported correctly” into “I’ve verified this imported correctly,” every time.

## 48.2. Demonstration

```
participants_messy <- read.csv(
 "data/participants_messy.csv", skip = 2,
 na.strings = c("NA", "999", "N/A", "Unknown", "")
)

dim(participants_messy)

[1] 20 8

names(participants_messy)

[1] "id" "age" "sex" "county" "systolic_bp"
[6] "diastolic_bp" "bmi" "smoker"
```



```
str(participants_messy)
```

```
'data.frame': 20 obs. of 8 variables:
 $ id : int 1 2 3 4 5 6 7 8 9 10 ...
 $ age : int 46 53 59 45 61 18 62 NA 33 36 ...
 $ sex : chr "Female" "Female" "Female" "Female" ...
 $ county : chr "Oakford" "riverdale" "OAKFORD " "Millbrook" ...
 $ systolic_bp : int 125 123 130 117 139 117 110 100 149 131 ...
 $ diastolic_bp: int 79 80 77 72 79 64 64 53 88 86 ...
 $ bmi : num 35.1 27.7 28.3 30.4 21.5 24 19.7 29.7 22.4 NA ...
 $ smoker : chr "No" "No" "No" "No" ...
```

## 48.3. Explore

Run `sapply(participants_messy, function(x) sum(is.na(x)))` and look for which columns have any missing data at all. Then run `sapply(participants_messy[sapply(participants_messy, is.character)], function(x) length(unique(x)))` and compare county's count to sex's and smoker's — one of these three text columns should stand out as far messier than the other two.

## 48.4. Solve the Problem

```
participants_messy <- read.csv(
 "data/participants_messy.csv", skip = 2,
 na.strings = c("NA", "999", "N/A", "Unknown", "")
)
```

```
Full checklist, run every time
dim(participants_messy)
```

## 48. Checking Imported Data

```
[1] 20 8
```

```
sapply(participants_messy, function(x) sum(is.na(x)))
```

| id  | age    | sex | county | systolic_bp | diastolic_bp |
|-----|--------|-----|--------|-------------|--------------|
| 0   | 2      | 0   | 0      | 2           | 0            |
| bmi | smoker |     |        |             |              |
| 2   | 1      |     |        |             |              |

```
sapply(participants_messy[sapply(participants_messy, is.character)],
 function(x) length(unique(x)))
```

| sex | county | smoker |
|-----|--------|--------|
| 2   | 13     | 2      |

In four short commands, you now know: the data has the expected 20 rows and 8 columns; `age`, `systolic_bp`, and `bmi` have some missing values (2, 2, and 2 respectively) that need `na.rm = TRUE` in any summary; and `county` has 13 apparent categories where `sex` and `smoker` have a clean, expected 2 each — immediately flagging `county` as the column needing cleanup before any analysis grouped by it.

## 48.5. Interpret the Result

This four-step checklist — dimensions, missingness per column, and distinct-category counts per text column — took seconds to run and surfaced every major issue this level has covered: the right row/column count, exactly which columns have missing data and how much, and precisely which text column (`county`) needs standardizing before it can be trusted. This is the routine worth running on *every* new dataset you import, for the rest of this course.

### Common Mistakes

- **Running this checklist once and considering the data “clean.”** This checklist *diagnoses* problems — it doesn’t fix them. county’s 13 categories still need Level 4’s systematic cleaning tools before analysis.
- **Skipping the checklist on “simple-looking” files.** Even a clean-looking 20-row file hid a disguised missing-value code (999) and 13 fragmented category values — appearances after a quick glance are not a substitute for actually running the checks.
- **Checking dimensions and stopping there.** `dim()` confirms row/column *count* is right, but says nothing about whether the *contents* imported correctly — always follow it with the type and missingness checks too.

## 48.6. Practice Problem

Run this full checklist on the clean `data/participants.csv` file (imported the normal way, no `skip` or special `na.strings` needed) instead of the messy file. Confirm every column shows sensible missingness counts and that `sex`, `county`, and `smoker` all show plausible, small numbers of distinct categories.

## 48.7. Challenge Problem

Imagine a new intern asks you: “Why bother with all these checks if `read.csv()` already ran successfully with no error message?” Write a two-to-three sentence answer, using at least one specific example from this level, explaining why a successful import is not the same as a *correct* import.

## 48.8. Key Takeaways

- A successful, error-free import is not the same as a correct one — always verify dimensions, column types, missingness, and category counts afterward.
- `dim()`, `names()`, `str()`, and per-column missingness/uniqueness checks together form a fast, repeatable routine worth running on every new dataset.
- This checklist diagnoses problems; fixing them (like standardizing county's 13 fragmented categories into 5) is Level 4's job.

### Knowledge Check

1. **Multiple choice.** What does a successful `read.csv()` call with no error message guarantee?
  - a) The data is completely clean and ready for analysis
  - b) Only that R was able to read the file into a data frame - nothing about its correctness
  - c) That there are no missing values
  - d) That all column names are unique
2. **True or false.** `dim()` alone is sufficient to confirm a dataset imported correctly.
3. **Predict the output.** If a data frame has 50 rows and one column, `age`, with 5 missing values, what would `sum(is.na(df$age))` return?
4. **Write the code.** Write the single line of code to count the number of distinct values in a column called `region`, in a data frame called `survey`.

Show answers

#### 48.8. Key Takeaways

1. **b)** Only that R was able to read the file into a data frame — it says nothing about whether the contents are correct.
2. **False.** `dim()` confirms row/column count only; it says nothing about column types, missing data, or hidden category duplicates.
3. `[1] 5`
4. `length(unique(survey$region))`



## 49. Understanding Variable Types After Import

### The Problem

Before you knew to specify `na.strings` properly, you imported `participants_messy.csv` with just `skip = 2` and tried `mean(participants_messy$systolic_bp, na.rm = TRUE)`. It failed:

```
Warning message:
argument is not numeric or logical: returning NA
```

`systolic_bp` should obviously be a numeric column — blood pressure readings are numbers. So why is R treating it like it isn't?

### Your Mission

Understand exactly how a single unexpected text value can force an entire numeric-looking column to import as character, and learn to catch this immediately.

### Think Before Coding

- From Level 2's data types page, what happens when you mix even one piece of text into an otherwise numeric vector with `c()`?
- If a CSV column has 19 number-looking values and exactly one cell containing the text "N/A" — and R doesn't yet know "N/A" means missing — what type would R have to make the *entire* column, to hold that one piece of text?

## 49.1. The R Concept

This is the same **coercion** rule from Level 2's vectors page, just showing up during import instead of inside `c()`. A CSV column is read in as one vector — and if even a single cell in an otherwise-numeric column contains genuine text that isn't a recognized missing-value code, R has no choice but to make the *entire column* character, exactly the way `c(1, 2, "N/A", 4)` would become `c("1", "2", "N/A", "4")`.

```
c(125, 123, "N/A", 117) # the whole vector becomes character - just like a
```

The fix isn't really about variable types at all — it's about **making sure every genuine missing-value code is included in `na.strings` before import**, exactly as the previous two pages covered. Once R correctly recognizes "N/A" (or "Unknown", or any other disguised code) as missing rather than as literal text, the column parses as numeric the way it should.

```
Without na.strings covering "N/A" - forced to character by that one text v
read.csv("file.csv")$systolic_bp
```

```
With na.strings covering "N/A" - correctly parses as numeric
read.csv("file.csv", na.strings = c("NA", "N/A"))$systolic_bp
```

## 49.2. Demonstration

```
Importing without recognizing "N/A" as missing
broken <- read.csv("data/participants_messy.csv", skip = 2)
```

```
class(broken$systolic_bp)
```

```
[1] "character"
```



### 49.3. Explore

```
mean(broken$systolic_bp, na.rm = TRUE)
```

Warning message:

argument is not numeric or logical: returning NA

```
[1] NA
```

```
Importing with na.strings correctly covering every missing-value code
fixed <- read.csv(
 "data/participants_messy.csv", skip = 2,
 na.strings = c("NA", "999", "N/A", "Unknown", "")
)
```

```
class(fixed$systolic_bp)
```

```
[1] "integer"
```

```
mean(fixed$systolic_bp, na.rm = TRUE)
```

```
[1] 122.4444
```

### 49.3. Explore

Run `class(broken$bmi)` and `class(fixed$bmi)` side by side — `bmi` has the same problem, caused by the disguised `"Unknown"` value instead of `"N/A"`. Confirm that fixing `na.strings` resolves both columns' type problems simultaneously, since both were victims of the exact same underlying issue.

## 49.4. Solve the Problem

```
participants_messy <- read.csv(
 "data/participants_messy.csv", skip = 2,
 na.strings = c("NA", "999", "N/A", "Unknown", "")
)

class(participants_messy$systolic_bp)

[1] "integer"

mean(participants_messy$systolic_bp, na.rm = TRUE)

[1] 122.4444
```

The original error — `argument is not numeric or logical` — is gone entirely, not because you worked around it, but because the true root cause (a disguised missing-value code silently forcing the column to character) was fixed at the source, during import.

## 49.5. Interpret the Result

`class(participants_messy$systolic_bp)` now correctly reports `"integer"`, and the mean systolic blood pressure among participants with a valid reading is 122.4 mmHg — a plausible, trustworthy number, now that the column is genuinely numeric rather than text that merely looked numeric to a human reader.

### Common Mistakes

- **Trying to fix a wrong column type *after* import, rather than fixing the cause during import.** You *can* patch this after the fact with `as.numeric()` (Level 1), but it's more fragile — better to prevent the coercion in the first place with correct `na.strings`, so every function downstream sees the right type immediately.
- **Assuming a “numeric-looking” column is safely numeric without checking `class()`.** A column can contain 999 perfectly numeric-looking values and one stray piece of text, and the entire column still silently becomes character — always verify with `class()` or `str()`, never assume from appearances.
- **Not connecting today's error message to yesterday's fix.** argument is not numeric or logical: returning NA is exactly the symptom of the disguised-missing-value problem from the previous two pages — recognizing this connection saves you from re-diagnosing the same root cause from scratch every time it appears in a new column.

## 49.6. Practice Problem

Without `na.strings` set to catch "Unknown", what is `class(broken$bmi)`? After correctly setting `na.strings`, what does it become? Confirm both with code.

## 49.7. Challenge Problem

A colleague says: “I fixed it — I just wrapped the column in `as.numeric()` after importing, and now `mean()` works.” Explain, in two or three sentences, what their fix does and does not solve, compared to fixing `na.strings` at import time (hint: think about what happens to the disguised text value itself under each approach, and whether every future column in future files would need the same manual patch).

## 49.8. Key Takeaways

- A single unrecognized text value anywhere in an otherwise-numeric CSV column forces the *entire* column to import as character — the same coercion rule from Level 2’s vectors page, just triggered during import.
- The right fix is making `na.strings` complete and correct at import time, not patching the column’s type afterward.
- `argument is not numeric or logical: returning NA` is a strong signal that a column you expected to be numeric actually imported as character — check `class()` immediately.

### Knowledge Check

1. **Multiple choice.** Why does a numeric-looking CSV column sometimes import as character?
  - a) R randomly assigns types
  - b) A single unrecognized text value (like a disguised missing-value code) forces the whole column to character
  - c) CSV files cannot contain numeric columns
  - d) The column has too many rows

2. **True or false.** Fixing `na.strings` at import time is a more robust solution than patching a column’s type with `as.numeric()` after the fact.

3. **Predict the output.**

```
x <- c(10, 20, "missing", 40)
class(x)
```

4. **Find the error.** A student sees `argument is not numeric or logical: returning NA` from `mean()` and responds by wrapping the column in `as.character()` to “match its type.” What’s wrong with this response?

Show answers

1. **b)** A single unrecognized text value forces the whole column to character, exactly like `c()` coercion from Level 2.
2. **True.** Fixing `na.strings` addresses the root cause during import; patching afterward with `as.numeric()` works but must be repeated manually for every affected column, in every file, going forward.
3. `[1] "character"`
4. `as.character()` moves in the wrong direction entirely — the goal is a *numeric* column for `mean()` to work; the actual fix is recognizing the disguised missing-value code via `na.strings` at import (or `as.numeric()` afterward as a less robust patch), not converting further into character.

## 49.9. Where you've arrived

You started this level with a single clean `read.csv()` call from Level 1 and a vague sense that real data might not always be so cooperative. You've now read and written CSV, Excel, and `.rds` files, permanently fixed the working-directory problem with RStudio Projects, and — across the last four pages — taken a genuinely messy file apart piece by piece: extra header rows, disguised missing-value codes, fragmented text categories, and the coercion trap that ties them all together.

Level 4 picks up exactly here, with the systematic, scalable tools — `case_when()`, `ifelse()`, and more — for actually *fixing* the messiness you now know how to detect.



## **Part IV.**

# **Level 4: Data Cleaning with Base R**





## Level 4 — Data Cleaning with Base R

### Where this level is headed

Level 3 taught you to *detect* messiness on import — disguised missing codes, fragmented categories, wrong types. This level teaches you to actually *fix* it: systematically finding and handling missing values, catching and removing duplicate records, renaming and retyping variables, building new variables from existing ones, and making decisions — one condition, or many at once — with `if`, `ifelse()`, `if_else()`, and `case_when()`.

### What you'll build toward

By the end of this level you'll take `participants.csv` — and the messier files from Level 3 — and turn them into datasets you'd actually trust for analysis: missing data handled deliberately, duplicates gone, variables named and typed the way your analysis needs, new clinically meaningful variables built from the ones you have, and categorical variables recoded and collapsed the way a real research question demands.

### Topics in this level

- Identifying, removing, and replacing missing values
- Detecting and removing duplicate records

## *Level 4 — Data Cleaning with Base R*

- Renaming variables and converting between character, numeric, and factor types
- Creating new variables and recoding existing ones
- Combining categories and building binary indicator variables
- Conditional logic: `if`, `ifelse()`, `if_else()`, and `case_when()`

### **A preview mission**

Your mission across this level: take a genuinely messy patient dataset — wrong types, duplicated rows, inconsistent missing-data codes — and turn it into something you’d trust enough to actually analyze, using nothing but base R.

By the last page, you’ll have built `case_when()` fluency — the single tool that replaces most of the nested `ifelse()` logic you’d otherwise be tempted to write, and the bridge into Level 5’s tidyverse.

## 50. Identifying Missing Values Across a Dataset

### The Problem

Your supervisor asks a broader question than any single page in Level 1 or Level 3 answered: “Before you analyze anything, give me a complete picture — which variables in `participants.csv` have missing data, and how much?” Checking `sum(is.na(participants$age))` one column at a time, as you did in Level 1, would work — but with 8 columns, you want a way to see all of them at once.

### Your Mission

Systematically identify missing data across an entire dataset in one step, rather than checking one column at a time.

### Think Before Coding

- From Level 1, what does `is.na(x)` return for a single vector `x`?
- If a data frame is really a collection of columns (Level 2), could you imagine applying `is.na()` and `sum()` to *every* column at once, rather than typing the same check eight separate times?

### 50.1. The R Concept

`colSums(is.na(df))` applies `is.na()` to every column of a data frame at once, then sums each column’s TRUE/FALSE results — giving you a

## 50. Identifying Missing Values Across a Dataset

missing-value count per column in a single line:

```
colSums(is.na(df))
```

For an even quicker, broader first look, **summary()** on a whole data frame reports basic statistics for every column simultaneously, including an NA's count for any column that has missing values:

```
summary(df)
```

Between the two: `colSums(is.na(df))` gives you exactly the missing-count-per-column answer your supervisor asked for; `summary()` gives you that plus a broader statistical snapshot (min, max, mean for numeric columns; counts for character columns) in one glance.

### 50.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
colSums(is.na(participants))
```

| id  | age    | sex | county | systolic_bp | diastolic_bp |
|-----|--------|-----|--------|-------------|--------------|
| 0   | 6      | 0   | 0      | 4           | 0            |
| bmi | smoker |     |        |             |              |
| 5   | 0      |     |        |             |              |

```
summary(participants)
```

| id             | age           | sex              | county           |
|----------------|---------------|------------------|------------------|
| Min. : 1.00    | Min. :18.00   | Length:100       | Length:100       |
| 1st Qu.: 25.75 | 1st Qu.:36.25 | Class :character | Class :character |
| Median : 50.50 | Median :49.00 | Mode :character  | Mode :character  |
| Mean : 50.50   | Mean :47.43   | NA's :6          |                  |

```
3rd Qu.: 75.25 3rd Qu.:58.75
Max. :100.00 Max. :81.00
```

## 50.3. Explore

Run `colSums(is.na(participants))` and confirm the total missing cells matches what you'd get by adding the six individual `sum(is.na(participants$column))` calls from earlier levels:  $6 + 4 + 5 = 15$ . Then try `sum(colSums(is.na(participants)))` — a single number summarizing the *entire* dataset's missingness at once.

## 50.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")
```

```
colSums(is.na(participants))
```

|     |        |     |        |             |              |
|-----|--------|-----|--------|-------------|--------------|
| id  | age    | sex | county | systolic_bp | diastolic_bp |
| 0   | 6      | 0   | 0      | 4           | 0            |
| bmi | smoker |     |        |             |              |
| 5   | 0      |     |        |             |              |

You can now answer your supervisor completely and precisely: “Three variables have missing data — `age` (6 participants), `systolic_bp` (4 participants), and `bmi` (5 participants). The other five variables — `id`, `sex`, `county`, `diastolic_bp`, and `smoker` — are fully complete.”

## 50.5. Interpret the Result

`colSums(is.na(participants))` reveals the missingness pattern across the *entire* dataset in one line: 15 missing cells total, concentrated in exactly three numeric variables, with no missingness at all in the categorical variables or `diastolic_bp`. This kind of complete picture — rather than checking columns one at a time and hoping you didn't miss one — is exactly what a supervisor or a methods section of a paper expects before any analysis proceeds.

### Common Mistakes

- **Checking missingness column by column and losing track.** With more than a handful of variables, it's easy to forget to check one — `colSums(is.na(df))` checks all of them, guaranteed, every time.
- **Confusing `colSums()` with `rowSums()`.** `colSums(is.na(df))` counts missing values *per column* (down each column); `rowSums(is.na(df))` counts missing values *per row* (across each row) — genuinely different questions, both useful, easy to mix up by name alone.
- **Treating `summary()`'s output as the full picture.** It's an excellent quick overview, but for character columns it only reports **Length**, **Class**, and **Mode** — not the counts or categories `table()` would show. Use both tools, not just one.

## 50.6. Practice Problem

Run `rowSums(is.na(participants))` and then `table(rowSums(is.na(participants)))` — this tells you how many participants have 0 missing values, how many have exactly 1, and so on. How many participants have zero missing data across all 8 variables?

## 50.7. Challenge Problem

Using `colSums(is.na(participants))`, identify which single variable has the *most* missing data, and calculate what percentage of the 100 participants that represents (using arithmetic from Level 1).

## 50.8. Key Takeaways

- `colSums(is.na(df))` reports the number of missing values in every column of a data frame at once, replacing repetitive one-column-at-a-time checks.
- `summary(df)` gives a broader statistical snapshot of every column simultaneously, including an **NA's** count where relevant.
- `rowSums(is.na(df))` answers a different, complementary question: how much missing data does each individual *row* (participant) have.

### Knowledge Check

1. **Multiple choice.** What does `colSums(is.na(df))` return?
  - a) The total number of rows in `df`
  - b) The number of missing values in each column
  - c) The number of missing values in each row
  - d) A logical vector the same size as `df`
2. **True or false.** `colSums()` and `rowSums()` answer the same question.
3. **Predict the output.** If a data frame has 3 columns and column `x` has 2 missing values while `y` and `z` have none, what would `colSums(is.na(df))` show for `x`?
4. **Write the code.** Write the single line of code to check missing-value counts across every column of a data frame called `survey_data`.

## 50. Identifying Missing Values Across a Dataset

Show answers

1. **b)** The number of missing values in each column.
2. **False.** `colSums()` summarizes down each column (per variable); `rowSums()` summarizes across each row (per observation) — different, complementary questions.
3. 2
4. `colSums(is.na(survey_data))`



# 51. Removing Missing Values

## The Problem

Your supervisor wants a “complete case” analysis: every calculation should use only participants who have *no* missing data across `age`, `systolic_bp`, and `bmi` at all — not just excluding missing values one column at a time with `na.rm = TRUE`, but removing entire *rows* wherever any of the three variables is missing. `na.rm = TRUE` (Level 1) only ever worked within a single `mean()` call — it never actually removed a row from the dataset itself.

## Your Mission

Remove rows with missing data from a data frame, either across specific columns or completely.

## Think Before Coding

- From Level 2, how would you build a logical vector flagging which *rows* have a missing value in a specific column?
- If a row is missing `age` but has every other variable, should it be excluded from an analysis of `bmi` that has nothing to do with `age`? Does your answer depend on the specific question being asked?

## 51.1. The R Concept

`na.omit()` removes any row with *at least one* missing value, across *every* column of a data frame:

## 51. Removing Missing Values

```
na.omit(df)
```

`complete.cases()` gives you more control — it returns a logical vector flagging which rows have no missing data, which you can then use with the row-indexing you learned in Level 2, optionally restricted to just the columns you care about:

```
complete.cases(df) # TRUE for rows complete across ALL columns
complete.cases(df[, c("age", "bmi")]) # TRUE for rows complete in just these columns
df[complete.cases(df),] # keep only the fully complete rows
```

The distinction matters: `na.omit()` is quick but *all-or-nothing* across every column; `complete.cases()` lets you decide exactly which columns' completeness actually matters for the analysis at hand — precisely your supervisor's request.

### 51.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
nrow(participants)
```

```
[1] 100
```

```
na.omit() - removes a row if ANY column has a missing value
complete_all <- na.omit(participants)
nrow(complete_all)
```

```
[1] 85
```

```
complete.cases() restricted to just the 3 columns that matter for this analysis
keep <- complete.cases(participants[, c("age", "systolic_bp", "bmi")])
complete_relevant <- participants[keep,]
nrow(complete_relevant)
```

```
[1] 85
```

### 51.3. Explore

These two approaches happen to agree here (85 rows) because `age`, `systolic_bp`, and `bmi` are the *only* columns with any missingness at all in this dataset (Level 4's first page confirmed `id`, `sex`, `county`, `diastolic_bp`, and `smoker` are fully complete). Try `nrow(na.omit(participants[, c("sex", "county")]))` — since neither column has any missing data, what would you expect, and does the result confirm it?

### 51.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

Complete-case analysis restricted to exactly the variables your supervisor cares about
keep <- complete.cases(participants[, c("age", "systolic_bp", "bmi")])
participants_complete <- participants[keep,]

nrow(participants_complete)

[1] 85

mean(participants_complete$age)

[1] 47.22353
```

Notice `mean()` no longer needs `na.rm = TRUE` at all here — every remaining row is already guaranteed complete for these three variables, so there's nothing left for `na.rm` to exclude.

## 51.5. Interpret the Result

85 of the 100 participants have complete data across `age`, `systolic_bp`, and `bmi` — the other 15 are missing at least one of the three. Restricting to exactly these three columns (rather than `na.omit()`'s all-or-nothing approach across every column) matters in general, though here the two approaches happen to produce the same 85 rows since no other columns in this particular dataset have any missing values.

### Common Mistakes

- **Using `na.omit()` on a whole data frame when only a few columns matter.** If a dataset had missingness in a column irrelevant to your analysis, `na.omit()` would still drop those rows unnecessarily — `complete.cases()` restricted to relevant columns avoids discarding usable data.
- **Losing track of how many rows were dropped.** Always compare `nrow()` before and after — silently going from 100 to 85 rows without noticing is a common way to accidentally misreport a sample size in a report.
- **Forgetting that removing rows changes more than one column at once.** Once you filter to complete cases for `age`, `systolic_bp`, and `bmi`, every *other* column (like `smoker`) also loses those same 15 rows — even though `smoker` itself had no missing data.

## 51.6. Practice Problem

Using `complete.cases()`, create a subset of `participants` that is complete for `bmi` alone, and report how many rows remain (it should match the earlier missingness count from the previous page).

## 51.7. Challenge Problem

Compare `nrow(na.omit(participants))` to `nrow(participants[complete.cases(participants[, c("age", "systolic_bp", "bmi")]), ])`. They should match in this dataset — explain, in one or two sentences, *why* they match here, and describe a hypothetical change to the dataset that would make them produce different results.

## 51.8. Key Takeaways

- `na.omit()` removes any row with missing data in *any* column; `complete.cases()` lets you restrict that check to just the columns relevant to your analysis.
- Always compare row counts before and after removing missing data, and report how many rows (and why) were excluded.
- Removing rows for missingness in one variable also removes those same rows' data for every other variable — a side effect worth being deliberate about.

### Knowledge Check

1. **Multiple choice.** What does `na.omit(df)` do?
  - a) Replaces missing values with 0
  - b) Removes any row with at least one missing value, across all columns
  - c) Removes only missing values in numeric columns
  - d) Removes columns with missing data
2. **True or false.** `complete.cases()` can be restricted to check completeness in just a subset of columns.

### 51. Removing Missing Values

3. **Predict the output.** If `df` has 50 rows and 5 have missing `age`, and no other column has missing data, what would `nrow(na.omit(df))` return?
4. **Write the code.** Write the code to keep only the rows of `survey_data` that are complete for the column `income`.

Show answers

1. **b)** Removes any row with at least one missing value, across all columns.
2. **True.** `complete.cases(df[, c("col1", "col2")])` restricts the completeness check to just those columns.
3. 45
4. `survey_data[complete.cases(survey_data[, "income"]), ]`  
(or equivalently, `survey_data[!is.na(survey_data$income), ]`)

## 52. Replacing Missing Values

### The Problem

Your supervisor pushes back on the complete-case approach from the last page: “Dropping 15 participants loses too much data for a small pilot study like this one. Instead, fill in the missing BMI values with the sample’s average BMI, so we keep all 100 participants in the analysis.” This is a genuinely different strategy from removing rows — replacing a missing value with a reasonable stand-in, rather than discarding the participant entirely.

### Your Mission

Replace missing values in a variable with a reasonable substitute, without discarding the rest of that participant’s data.

### Think Before Coding

- From Level 2, how would you build a logical vector identifying exactly which positions in a vector are NA?
- If you can identify *which* positions are missing, could you assign a new value to just those positions, leaving every other value untouched?

### 52.1. The R Concept

You can assign directly to the subset of a vector identified by `is.na()`, replacing only the missing positions and leaving everything else exactly as

## 52. Replacing Missing Values

it was:

```
x[is.na(x)] <- replacement_value
```

The most common simple replacement is the variable's own mean or median — this is called **mean imputation** (or median imputation), and it's a reasonable, simple starting strategy, though it does understate the true variability in the data (a limitation Level 12's multiple imputation addresses more rigorously later in the course).

```
df$bmi[is.na(df$bmi)] <- mean(df$bmi, na.rm = TRUE)
```

Notice the `na.rm = TRUE` is still required *inside* the `mean()` call on the right-hand side — you're calculating the mean from the *non-missing* values, in order to fill in the missing ones.

### 52.2. Demonstration

```
bmi_values <- c(22.1, NA, 27.5, 24.0, NA, 30.2)
```

```
The mean, calculated only from the non-missing values
mean(bmi_values, na.rm = TRUE)
```

```
[1] 25.95
```

```
Replace only the NA positions with that mean
bmi_values[is.na(bmi_values)] <- mean(bmi_values, na.rm = TRUE)
bmi_values
```

```
[1] 22.10 25.95 27.50 24.00 25.95 30.20
```

Notice the two originally missing positions are now both filled with 25.95 — the mean of the four originally non-missing values — while every real, originally observed value is completely untouched.



## 52.3. Explore

Try running `mean(bmi_values)` again *after* the replacement above (no `na.rm` needed now, since there are no more NAs) — compare it to the original 25.95. It's identical, which makes sense: replacing missing values with the mean can never change the mean itself, only reduce the apparent spread of the data (a limitation worth remembering).

## 52.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

Preserve the original column so you can still compare before/after if needed
participants$bmi_imputed <- participants$bmi
participants$bmi_imputed[is.na(participants$bmi_imputed)] <- mean(participants$bmi, na.rm =
sum(is.na(participants$bmi_imputed))

[1] 0

nrow(participants)

[1] 100
```

All 100 participants now remain in the dataset — none dropped — with the 5 originally missing BMI values replaced by the sample mean, kept in a clearly separate `bmi_imputed` column so the original, unmodified `bmi` column is still available for comparison or a different analysis strategy later.

## 52.5. Interpret the Result

`sum(is.na(participants$bmi_imputed))` returning 0 confirms every missing BMI value has been filled in, and `nrow(participants)` still showing 100 confirms no participant was dropped — directly satisfying your supervisor’s request to retain the full sample, at the cost of slightly understating how much BMI actually varies across the 5 imputed participants.

### Common Mistakes

- **Overwriting the original column directly.** Replacing `participants$bmi` in place destroys the record of which values were originally missing — always create a new column (like `bmi_imputed`) so you can compare, audit, or try a different imputation strategy later.
- **Forgetting `na.rm = TRUE` inside the replacement calculation itself.** `mean(participants$bmi)` without `na.rm = TRUE` returns NA (Level 1), and assigning NA to replace NA accomplishes nothing.
- **Treating mean imputation as a universally correct default.** It’s a simple, defensible starting point for a small amount of missing data, but it artificially shrinks the variability of the imputed variable — Level 12 covers more rigorous approaches (multiple imputation) for when this matters more.

## 52.6. Practice Problem

Using `participants$age`, create a new column `age_imputed` with the missing age values replaced by the median age (`median(participants$age, na.rm = TRUE)`) rather than the mean. Confirm `sum(is.na(participants$age_imputed))` is 0.

## 52.7. Challenge Problem

Explain, in two or three sentences, why replacing missing values with the mean is a reasonable default for a *small* fraction of missing data (like 5 out of 100 BMI values here) but becomes more questionable as the fraction of missing data grows larger — say, if 40 of 100 values were missing instead.

## 52.8. Key Takeaways

- `x[is.na(x)] <- value` replaces only the missing positions in a vector, leaving every other value untouched.
- Mean (or median) imputation is a simple strategy that keeps every row in the dataset, at the cost of slightly understating the imputed variable's true variability.
- Always store an imputed variable in a new column, preserving the original (with its real missing values intact) for comparison or a different strategy later.

### Knowledge Check

1. **Multiple choice.** What does `x[is.na(x)] <- mean(x, na.rm = TRUE)` do?
  - a) Removes missing values from `x`
  - b) Replaces missing values in `x` with the mean of its non-missing values
  - c) Replaces every value in `x` with the mean
  - d) Converts `x` to character
2. **True or false.** Mean imputation increases the apparent variability of a variable.
3. **Predict the output.**

## 52. Replacing Missing Values

```
x <- c(10, NA, 20, NA, 30)
x[is.na(x)] <- mean(x, na.rm = TRUE)
x
```

### 4. Find the error.

```
participants$bmi[is.na(participants$bmi)] <- mean(participants$bmi)
```

Show answers

1. **b)** Replaces missing values in `x` with the mean of its non-missing values.
2. **False.** It *decreases* apparent variability — imputed values all equal the mean, reducing the overall spread compared to the true, unobserved data.
3. `[1] 10 20 20 20 30` (the mean of 10, 20, 30 is 20, filled into both NA positions).
4. The right-hand side `mean(participants$bmi)` has no `na.rm = TRUE`, so it returns NA — and assigning NA to replace NA accomplishes nothing. It should be `mean(participants$bmi, na.rm = TRUE)`.

## 53. Detecting Duplicates

### The Problem

You learn that two research assistants both entered data for the same visit last week, without realizing the other had already done it. You suspect this means at least one participant now appears *twice* in `participants.csv` — but with 100 rows, you can’t just scroll through and eyeball it. You need R to find any duplicate rows for you.

### Your Mission

Detect duplicate rows in a data frame automatically, rather than checking by hand.

### Think Before Coding

- If two rows have exactly identical values in every column, what would make them “the same” row to R?
- Would you expect R to flag *both* copies of a duplicate as duplicates, or only the second (repeat) occurrence?

### 53.1. The R Concept

`duplicated(x)` returns a logical vector, `TRUE` for any element (or, for a data frame, any *row*) that is an exact repeat of an earlier one:

```
duplicated(df)
```

### 53. Detecting Duplicates

Critically, `duplicated()` marks only the *second and later* occurrences as `TRUE` — the *first* occurrence of a value is always `FALSE`, even though it's the one being repeated. This asymmetry is deliberate: it directly tells you which rows you could safely remove while keeping exactly one copy of each.

```
sum(duplicated(df)) # how many duplicate rows exist
df[duplicated(df),] # view just the duplicate rows themselves
```

#### 53.2. Demonstration

```
A simple example to see the FALSE/TRUE pattern clearly
ages <- c(45, 62, 45, 38, 62, 62)
duplicated(ages)
```

```
[1] FALSE FALSE TRUE FALSE TRUE TRUE
```

Notice: the *first* 45 and the *first* 62 are both `FALSE` (not yet a repeat of anything); the *second* 45 and every repeat of 62 after the first are `TRUE`.

```
participants <- read.csv("data/participants.csv")
```

```
Simulating the two-research-assistants scenario: duplicate participant #1's
participants_dup <- rbind(participants, participants[1,])
```

```
nrow(participants_dup)
```

```
[1] 101
```

```
sum(duplicated(participants_dup))
```

```
[1] 1
```

## 53.3. Explore

Run `which(duplicated(participants_dup))` — this tells you the exact row *position* of the duplicate, rather than just how many exist. Then run `participants_dup[duplicated(participants_dup), ]` to view the duplicated row’s actual contents.

## 53.4. Solve the Problem

```
participants_dup <- rbind(participants, participants[1,])
```

```
How many duplicate rows exist?
sum(duplicated(participants_dup))
```

```
[1] 1
```

```
Which row is it, and what does it contain?
participants_dup[duplicated(participants_dup),]
```

```
 id age sex county systolic_bp diastolic_bp bmi smoker
1.1 1 46 Female Oakford 125 79 35.1 No
```

You’ve now confirmed the research assistants’ suspicion precisely: participant #1’s row was entered twice, and you know exactly what it contains — ready to remove on the next page.

## 53.5. Interpret the Result

`duplicated()` correctly flags the second copy of participant #1 as a duplicate (`TRUE`) while leaving the first, original entry as `FALSE` — exactly the behavior you need to identify and later remove the redundant copy without losing the original.

### Common Mistakes

- **Expecting both copies of a duplicate to be flagged `TRUE`.** Only the second (and any later) occurrence is `TRUE`; the first is always `FALSE`. If you want to see *all* copies of anything that has a duplicate, you'd need a different check (like `duplicated(x) | duplicated(x, fromLast = TRUE)`).
- **Confusing exact row duplicates with duplicate IDs.** `duplicated(df)` by default checks whether *every column* matches exactly — two rows with the same `id` but a corrected `age` value would *not* be flagged as duplicates by this check (a distinct, trickier problem covered in this page's challenge).
- **Assuming duplicates can't happen in "clean" data.** They're extremely common in real research data entry — merged files, re-submitted forms, and exactly this page's two-research-assistants scenario are all frequent causes.

## 53.6. Practice Problem

Using `participants_dup` from this page, run `table(duplicated(participants_dup))` and interpret both counts: how many rows are marked `FALSE` (unique or first occurrence) and how many are marked `TRUE` (a repeat).



## 53.7. Challenge Problem

Suppose instead the second research assistant’s entry for participant #1 had a typo — `age` recorded as 47 instead of the correct 46, with every other column identical. Would `duplicate()` still catch this as a duplicate? Explain why or why not, and describe (in words) what you’d need to check instead to catch a “duplicate participant with conflicting values” like this one.

## 53.8. Key Takeaways

- `duplicate(df)` flags rows that are exact repeats of an earlier row; the first occurrence is always `FALSE`, later copies are `TRUE`.
- `sum(duplicate(df))` counts duplicate rows; `df[duplicate(df), ]` shows you their actual contents.
- `duplicate()` only catches *exact* row matches — a repeated participant with even one differing value (like a corrected typo) will not be flagged by this check alone.

### Knowledge Check

1. **Multiple choice.** For `c(5, 5, 5)`, what does `duplicate()` return?
  - a) `TRUE TRUE TRUE`
  - b) `FALSE TRUE TRUE`
  - c) `FALSE FALSE TRUE`
  - d) `FALSE FALSE FALSE`
2. **True or false.** `duplicate()` marks the *first* occurrence of a repeated value as `TRUE`.
3. **Predict the output.**

### 53. Detecting Duplicates

```
x <- c("a", "b", "a", "a")
sum(duplicated(x))
```

4. **Write the code.** Write the code to view just the duplicate rows of a data frame called `survey_data`.

Show answers

1. **b)** FALSE TRUE TRUE — the first 5 is not yet a repeat.
2. **False.** The first occurrence is always FALSE; only subsequent repeats are TRUE.
3. `[1] 2` — the 3rd and 4th "a" are both repeats of the 1st.
4. `survey_data[duplicated(survey_data), ]`

## 54. Removing Duplicates

### The Problem

You've confirmed participant #1 appears twice in the dataset, thanks to the last page's `duplicated()` check. Simply *knowing* about the duplicate isn't enough — your supervisor needs the dataset fixed, with exactly one copy of every participant, before any analysis proceeds.

### Your Mission

Remove duplicate rows from a data frame, keeping exactly one copy of each.

### Think Before Coding

- From the last page, `duplicated()` returns `FALSE` for the first occurrence of a row and `TRUE` for every repeat. If you kept only the rows where this is `FALSE`, what would remain?
- Is there a shortcut function you might expect, given that Level 2 already introduced a function called `unique()` for vectors?

### 54.1. The R Concept

Two equivalent approaches remove duplicate rows, keeping the first occurrence of each:

```
df[!duplicated(df),] # keep only rows where duplicated() is FALSE
unique(df) # shortcut: does exactly the same thing for a whole data frame
```

## 54. Removing Duplicates

`unique()`, which you first met on vectors in Level 2, works the same way on entire data frames — it's really just a more convenient wrapper around `!duplicated(df)`.

```
nrow(df) # before
nrow(unique(df)) # after - should be nrow(df) - sum(duplicated(df))
```

### 54.2. Demonstration

```
participants <- read.csv("data/participants.csv")

participants_dup <- rbind(participants, participants[1,])
nrow(participants_dup)

[1] 101

Approach 1: explicit logical indexing
participants_fixed <- participants_dup[!duplicated(participants_dup),]
nrow(participants_fixed)

[1] 100

Approach 2: the unique() shortcut - identical result
participants_fixed2 <- unique(participants_dup)
nrow(participants_fixed2)

[1] 100

identical(participants_fixed, participants_fixed2)

[1] TRUE
```

## 54.3. Explore

Confirm `identical(participants_fixed, participants)` — after removing the duplicate you added, does the dataset exactly match the original, untouched `participants`? This confirms the fix is complete and introduced no side effects beyond removing the one duplicate row.

## 54.4. Solve the Problem

```
participants_dup <- rbind(participants, participants[1,])
```

```
nrow(participants_dup)
```

```
[1] 101
```

```
participants_clean <- unique(participants_dup)
```

```
nrow(participants_clean)
```

```
[1] 100
```

```
sum(duplicated(participants_clean))
```

```
[1] 0
```

Exactly 100 rows remain — one copy of every participant, with the accidental double-entry from the two research assistants fully resolved.

## 54.5. Interpret the Result

`nrow(participants_clean)` returning to 100 (down from the temporarily inflated 101), combined with `sum(duplicated(participants_clean))` confirming 0 remaining duplicates, gives you concrete proof the dataset is now exactly as it should be — ready to hand back to your supervisor with confidence, not just a hopeful assumption.

### Common Mistakes

- **Removing duplicates before confirming which rows they actually are.** Always run the detection step (previous page) first, so you know *what* you're removing and can sanity-check the before/after row counts make sense.
- **Assuming `unique()` and `duplicated()` disagree in some way.** They don't — `unique(df)` is defined to produce exactly the same result as `df[!duplicated(df), ]`; either is correct, and which one you reach for is purely a matter of style.
- **Not rechecking after removal.** Running `sum(duplicated(participants_clean))` afterward and confirming it's 0 is a cheap, valuable habit — don't just assume the removal worked because no error appeared.

## 54.6. Practice Problem

Create `test_dup <- rbind(participants, participants[50, ], participants[50, ])` (duplicating row 50 *twice*, so it appears three times total). Confirm `nrow(test_dup)` is 102, then use `unique()` to reduce it back to 100, and verify with `sum(duplicated(...))`.

## 54.7. Challenge Problem

Recall the previous page’s challenge: a duplicate participant with one *differing* value (like a corrected age) would not be caught by `duplicated()` or `unique()`, since they only catch exact row matches. Describe, in words, what additional check based on `id` alone (ignoring the other columns) would be needed to catch this trickier kind of duplicate — you don’t need to write the code, just describe the logic.

## 54.8. Key Takeaways

- `df[!duplicated(df), ]` and `unique(df)` are equivalent ways to remove duplicate rows, keeping the first occurrence of each.
- Always detect and confirm duplicates *before* removing them, and reconfirm with `sum(duplicated(...))` afterward that none remain.
- Both approaches only catch exact row matches — a “duplicate” participant with even one differing value requires a different, ID-based check (not covered by `duplicated()` alone).

### Knowledge Check

1. **Multiple choice.** What does `unique(df)` do?
  - a) Removes all rows with missing data
  - b) Removes duplicate rows, keeping the first occurrence of each
  - c) Removes the first occurrence of any repeated row
  - d) Sorts the data frame
2. **True or false.** `df[!duplicated(df), ]` and `unique(df)` produce the same result.
3. **Predict the output.** If `df` has 20 rows and 3 are exact duplicates of earlier rows, what would `nrow(unique(df))` return?

#### 54. Removing Duplicates

4. **Write the code.** Write the code to remove duplicate rows from a data frame called `survey_data`, using the `unique()` shortcut.

Show answers

1. **b)** Removes duplicate rows, keeping the first occurrence of each.
2. **True.** They are equivalent — `unique()` is a convenience wrapper around the same logic.
3. 17
4. `survey_data <- unique(survey_data)`



## 55. Renaming Variables

### The Problem

Your department's official data dictionary requires blood pressure columns to be named `sbp` and `dbp` (standard clinical abbreviations), not `systolic_bp` and `diastolic_bp`. Every script you write from now on needs to reference the new names — but you don't want to reimport or manually retype the entire dataset to fix just two column names.

### Your Mission

Rename one or more columns in a data frame without touching any of the data itself.

### Think Before Coding

- From Level 2, what function retrieves (or sets) a data frame's column names?
- If `names(df)` returns a character vector of column names, could you change just *one* element of that vector, the same way you'd change one element of any other vector (Level 2's indexing)?

### 55.1. The R Concept

`names(df)` (Level 2) can be assigned to directly, and — combined with logical or position-based indexing — you can rename specific columns without disturbing any others:

## 55. Renaming Variables

```
names(df)[names(df) == "old_name"] <- "new_name"
```

Read this carefully: `names(df) == "old_name"` produces a logical vector, `TRUE` at the one position matching `"old_name"`; that logical vector then indexes into `names(df)` on the left-hand side, replacing just that one name.

For renaming several columns at once, you can also assign a full replacement vector for `names()`, or rename by *position* if you know exactly which column index you need:

```
names(df)[5] <- "sbp" # rename the 5th column specifically, by position
```

### 55.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
names(participants)
```

```
[1] "id" "age" "sex" "county" "systolic_bp"
[6] "diastolic_bp" "bmi" "smoker"
```

```
names(participants)[names(participants) == "systolic_bp"] <- "sbp"
names(participants)[names(participants) == "diastolic_bp"] <- "dbp"
```

```
names(participants)
```

```
[1] "id" "age" "sex" "county" "sbp" "dbp" "bmi" "smoker"
```

```
The data itself is completely unchanged - only the column labels
head(participants$sbp)
```

```
[1] 125 123 130 117 139 117
```

## 55.3. Explore

Try renaming by position instead: `names(participants)[5] <- "systolic_bp_renamed"`. This works too, but is riskier — if the column order ever changes (say, after a join in Level 6), a position-based rename would silently rename the *wrong* column. Name-based renaming (matching on the current name, as in the solution) is more robust against exactly this kind of accidental reordering.

## 55.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

names(participants)[names(participants) == "systolic_bp"] <- "sbp"
names(participants)[names(participants) == "diastolic_bp"] <- "dbp"

names(participants)

[1] "id" "age" "sex" "county" "sbp" "dbp" "bmi" "smoker"
```

Every script you write from this point forward can now use `participants$sbp` and `participants$dbp`, matching your department's data dictionary exactly, with the underlying blood pressure values themselves completely unchanged.

## 55.5. Interpret the Result

The column names now read `sbp` and `dbp` instead of `systolic_bp` and `diastolic_bp`, but every value in every row is identical to before — renaming a column is purely a labeling change, with zero effect on the data

## 55. Renaming Variables

itself, exactly as you'd want when the *only* thing that needed to change was compliance with a naming standard.

### Common Mistakes

- **Renaming by position when the column order might change.**  
A position-based rename (`names(df)[5] <- ...`) silently renames whichever column is currently 5th — if a join or reordering operation shifts columns around later, this becomes a subtle, hard-to-spot bug.
- **Misspelling the old name in a name-based rename.**  
`names(df)[names(df) == "Systolic_BP"] <- "sbp"` (wrong capitalization) matches nothing — the logical vector is all `FALSE`, so no column gets renamed, with no error or warning at all.
- **Forgetting that renaming doesn't reorder columns.** After renaming, `sbp` and `dbp` still sit in the same position in the data frame as `systolic_bp` and `diastolic_bp` did — renaming and reordering are two entirely separate operations.

## 55.6. Practice Problem

Rename the `bmi` column to `bmi_kg_m2` (a more explicit, unit-labeled name), using the name-based approach, and confirm with `names()` that it worked.

## 55.7. Challenge Problem

Rename `sex` to `sex_reported` and `smoker` to `smoking_status` in a single block of code (two separate rename lines are fine), then use `str()` to confirm both the new names and the underlying data are exactly as expected.

## 55.8. Key Takeaways

- `names(df)[names(df) == "old_name"] <- "new_name"` renames a column by matching its current name, leaving the underlying data completely untouched.
- Name-based renaming is more robust than position-based renaming, which silently breaks if column order changes later.
- Renaming never reorders or modifies the data itself — it only changes the label used to refer to a column.

### Knowledge Check

1. **Multiple choice.** What does `names(df)[names(df) == "age"] <- "participant_age"` do?
  - a) Deletes the `age` column
  - b) Renames the `age` column to `participant_age`
  - c) Changes every value in the `age` column
  - d) Creates a new column called `participant_age` alongside `age`
2. **True or false.** Renaming a column can change the values stored in it.
3. **Predict the output.** After `names(df)[names(df) == "bmi"] <- "bmi_kg_m2"`, what would `df$bmi` return?
4. **Find the error.**

```
names(df)[names(df) == "Age"] <- "participant_age"
```

assuming the actual column is named `age` (lowercase).

Show answers

1. **b)** Renames the `age` column to `participant_age`.
2. **False.** Renaming only changes the column label; the values themselves are completely unaffected.

## 55. Renaming Variables

3. An error (`$ operator is invalid for atomic vectors-style issue`) or `NULL` — depending on context, but effectively: `bmi` no longer exists as a name, since it was renamed to `bmi_kg_m2`.
4. The capitalization doesn't match ("`Age`" vs. the actual "`age`"), so the logical comparison is all `FALSE` and nothing gets renamed — no error is raised, which makes this mistake easy to miss.

## 56. Changing Variable Types: Character to Numeric

### The Problem

A partner clinic sends you self-reported weights, but their form allowed free text: the column contains entries like "70", "68.5 kg", "72kg", and "unknown". Level 3 briefly covered `as.numeric()`, but that was for clean text like "45" — this column has units and inconsistent spacing mixed into otherwise-numeric text, and a plain `as.numeric()` call will not handle it gracefully.

### Your Mission

Convert a messy character column, containing units and inconsistent formatting, into a clean numeric column.

### Think Before Coding

- From Level 1, what does `as.numeric("70")` return? What about `as.numeric("70kg")`?
- If a piece of text has non-numeric characters mixed in that you need to strip away *before* converting, what kind of tool would remove specific characters from a string?

## 56.1. The R Concept

`as.numeric()` alone only works on text that's *already* a clean, valid number — anything else (extra units, stray letters) produces `NA` with a warning, exactly as you saw in Level 1. The fix is to **clean the text first**, then convert:

`gsub(pattern, replacement, x)` removes (or replaces) a pattern of characters throughout a string — here, used to strip out anything that isn't a digit or a decimal point:

```
gsub("[^0-9.]", "", x) # remove everything except digits and decimal points

weight_text <- "68.5 kg"
weight_clean <- gsub("[^0-9.]", "", weight_text) # "68.5"
weight_numeric <- as.numeric(weight_clean) # 68.5
```

The pattern `"[^0-9.]"` is a small piece of **regular expression** syntax: `[0-9.]` means “any digit or decimal point,” and the leading `^` inside the brackets means “not” — so the whole pattern matches “anything that is *not* a digit or decimal point,” which `gsub()` then deletes (replacing it with `""`, empty text).

## 56.2. Demonstration

```
weight_raw <- c("70", "68.5 kg", "72kg", "unknown")

Strip out anything that isn't a digit or decimal point
weight_stripped <- gsub("[^0-9.]", "", weight_raw)
weight_stripped

[1] "70" "68.5" "72" ""
```



### 56.3. Explore

```
weight_numeric <- as.numeric(weight_stripped)
```

```
Warning message:
NAs introduced by coercion
```

```
weight_numeric
```

```
[1] 70.0 68.5 72.0 NA
```

Notice "unknown" correctly became NA — stripping every character from it left an empty string "", which `as.numeric()` correctly cannot interpret as a number, and appropriately converts to NA with a warning rather than guessing.

### 56.3. Explore

Try `gsub("[^0-9.]", "", "72 KG")` (uppercase, with a space) and confirm it still correctly strips down to "72" — the pattern removes *any* non-digit, non-decimal character, regardless of what that character actually is.

### 56.4. Solve the Problem

```
weight_raw <- c("70", "68.5 kg", "72kg", "unknown", "65.2 kg")
```

```
weight_clean <- as.numeric(gsub("[^0-9.]", "", weight_raw))
weight_clean
```

```
[1] 70.0 68.5 72.0 NA 65.2
```

## 56. Changing Variable Types: Character to Numeric

```
mean(weight_clean, na.rm = TRUE)
```

```
[1] 68.925
```

The messy, unit-contaminated text column is now a clean numeric vector, with the genuinely unreadable "unknown" entry correctly and appropriately converted to NA rather than silently guessed at.

### 56.5. Interpret the Result

The mean weight among the four participants with an interpretable value is 68.925 kg — a real, usable number, obtained only because the units and inconsistent spacing were stripped away *before* attempting the numeric conversion, rather than attempting `as.numeric()` directly on the original, unit-contaminated text.

#### Common Mistakes

- **Calling `as.numeric()` directly on unit-contaminated text.** `as.numeric("68.5 kg")` fails (returns NA with a warning) because of the trailing " kg" — the units must be stripped first.
- **Writing a pattern that's too aggressive or too narrow.** `"^[0-9.]"` removes anything that isn't a digit or period; if your data used commas as decimal separators (common outside the US), you'd need a different pattern, or to convert commas to periods first.
- **Not checking for new NAs after conversion.** Always compare `sum(is.na(x))` before and after a cleaning step like this — genuinely unreadable entries (like "unknown") should become NA, but you want to confirm that's the *only* thing generating new missingness, not a pattern mistake affecting otherwise-valid entries.

## 56.6. Practice Problem

Clean and convert `height_raw <- c("165cm", "172.5 cm", "180", "n/a")` into a numeric vector, then calculate the mean height among the valid entries.

## 56.7. Challenge Problem

A dosage column contains `c("5mg", "10 mg", "7.5mg", "twice daily")` — the last entry isn't a dose at all, just free-text scheduling information mistakenly entered in the wrong column. Clean and convert this column, and explain why `"twice daily"` correctly becomes `NA` even though it doesn't look like a disguised missing-value code the way `"unknown"` did.

## 56.8. Key Takeaways

- `as.numeric()` alone cannot handle text with embedded units or extra characters — it must be cleaned first, typically with `gsub()`, before conversion.
- `gsub("[^0-9.]", "", x)` strips away everything except digits and decimal points, a common first step for numeric-looking but unit-contaminated text.
- Always check for newly introduced `NA`s after a cleaning-and-converting step, to confirm they're the entries you expected (genuinely unreadable text), not a pattern mistake.

### Knowledge Check

1. **Multiple choice.** What does `gsub("[^0-9.]", "", "72kg")` return?

56. *Changing Variable Types: Character to Numeric*

- a) "72kg"
  - b) "72"
  - c) "kg"
  - d) NA
2. **True or false.** `as.numeric("68.5 kg")` correctly returns 68.5, ignoring the units automatically.
3. **Predict the output.**
- ```
as.numeric(gsub("[^0-9.]", "", "15.2 mmHg"))
```
4. **Write the code.** Write the code to clean and convert `dose_raw` `<- c("5mg", "10mg", "unclear")` into a numeric vector.

Show answers

1. **b)** "72" — everything except digits and the decimal point is stripped away.
2. **False.** `as.numeric()` alone fails on unit-contaminated text; the units must be removed first with something like `gsub()`.
3. `[1] 15.2`
4. `as.numeric(gsub("[^0-9.]", "", dose_raw))`

57. Changing Variable Types: Numeric to Character

The Problem

Your team is about to merge `participants.csv` with a partner clinic's system (a full topic in Level 6), and their system identifies every participant with a zero-padded code like "P001", "P002", ..., "P100" — not a plain integer like 1, 2, ..., 100. You need to convert your numeric `id` column into these specific text-based codes before the merge can work.

Your Mission

Convert a numeric variable into character text, including generating a formatted, zero-padded ID code.

Think Before Coding

- From Level 1, what does `as.character(42)` return, and how is it different from 42 itself, even though they print similarly?
- If you need 1 to become "P001" (not just "1"), what extra step beyond a simple type conversion would that require?

57.1. The R Concept

`as.character(x)` performs the basic conversion — a number becomes its text representation:

57. Changing Variable Types: Numeric to Character

```
as.character(42)    # "42"
```

But a zero-padded, prefixed code like "P001" needs more than a bare conversion — it needs **formatting**. `sprintf()` (or the more R-idiomatic `formatC()`) builds a formatted string from a template:

```
sprintf("P%03d", 1)      # "P001" - %03d means "an integer, zero-padded to 3 d
sprintf("P%03d", 42)     # "P042"
sprintf("P%03d", 100)    # "P100"
```

Reading "P%03d": P is literal text that appears as-is; %03d is a placeholder meaning “insert an integer here, padded with zeros to at least 3 digits wide.”

57.2. Demonstration

```
as.character(42)
```

```
[1] "42"
```

```
class(as.character(42))
```

```
[1] "character"
```

```
# Basic conversion alone doesn't zero-pad
```

```
as.character(1)
```

```
[1] "1"
```

```
# sprintf() builds the exact formatted code needed
```

```
sprintf("P%03d", 1)
```

```
[1] "P001"
```

```
sprintf("P%03d", c(1, 42, 100))
```

```
[1] "P001" "P042" "P100"
```

57.3. Explore

Try `sprintf("P%02d", 1)` (2 digits instead of 3) and `sprintf("P%04d", 1)` (4 digits) — compare the outputs to see exactly how the number after `%0` controls the padding width. What happens if the number itself already has more digits than the specified width, like `sprintf("P%02d", 100)`?

57.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

participants$patient_code <- sprintf("P%03d", participants$id)

head(participants$patient_code)

[1] "P001" "P002" "P003" "P004" "P005" "P006"

class(participants$patient_code)

[1] "character"

tail(participants$patient_code)

[1] "P095" "P096" "P097" "P098" "P099" "P100"
```

Every participant now has a properly formatted, zero-padded code exactly matching the partner clinic's system, ready for the merge in Level 6.

57.5. Interpret the Result

`participants$patient_code` now holds character strings like "P001" and "P100", generated directly from the numeric `id` column — the original `id` column remains untouched and numeric, while `patient_code` provides the specific text format the partner system requires, without you needing to type all 100 codes by hand.

Common Mistakes

- **Using `as.character()` alone when zero-padding is required.** `as.character(1)` gives "1", not "001" — a bare type conversion doesn't add formatting; `sprintf()` (or `formatC()`) is needed for that.
- **Getting the width wrong in `sprintf()`.** `sprintf("P%02d", 100)` — asking for 2-digit padding on a number that's already 3 digits — doesn't truncate anything; it simply uses as many digits as the number actually needs, so the padding number should be at least as large as your largest expected ID.
- **Forgetting the new code is a completely separate column.** `participants$patient_code` doesn't replace `participants$id` — both coexist, since you'll likely still want the numeric `id` for calculations, and the character `patient_code` specifically for the external merge.

57.6. Practice Problem

Create a `visit_code` column formatted as "V" followed by a 2-digit zero-padded visit number, for visit numbers `c(1, 5, 12)` (expected output: "V01", "V05", "V12").

57.7. Challenge Problem

Your partner clinic's system actually uses a different prefix depending on county: "R" for Riverdale, "O" for Oakford, and so on. Without yet knowing the tidyverse tools from Level 5, describe in words (you don't need full working code) how you might combine `substr()` or manual character logic with `sprintf()` to build a per-county prefix — this is intentionally a preview of a genuinely awkward base-R problem that Level 5's `case_when()` and `paste0()` will make far more elegant.

57.8. Key Takeaways

- `as.character()` performs a basic numeric-to-text conversion; it does not add formatting like zero-padding.
- `sprintf("P%03d", x)` builds a formatted, zero-padded string from a numeric template — the 03 controls the minimum digit width.
- Store a formatted ID code in a new column, keeping the original numeric column intact for any calculations that still need it.

Knowledge Check

1. **Multiple choice.** What does `as.character(7)` return?
 - a) 7
 - b) "7"
 - c) "07"
 - d) 07
2. **True or false.** `as.character()` alone can zero-pad a number to a fixed width.
3. **Predict the output.**

```
sprintf("ID%03d", 5)
```

57. *Changing Variable Types: Numeric to Character*

4. **Write the code.** Write the code to convert the numeric vector `c(3, 27, 150)` into zero-padded 3-digit codes prefixed with "S" (e.g., "S003", "S027", "S150").

Show answers

1. b) "7"
2. **False.** `as.character()` performs a plain conversion with no formatting; `sprintf()` or `formatC()` is needed for zero-padding.
3. [1] "ID005"
4. `sprintf("S%03d", c(3, 27, 150))`

58. Changing Variable Types: Character to Factor

The Problem

Level 11 will teach you to model whether a participant smokes as a predictor of blood pressure. Statistical modeling functions in R generally expect categorical predictors to be **factors**, not plain character text — and beyond that, you'd like "No" (not "Yes") treated as the baseline, reference category for that future model, since “compared to non-smokers” is the natural way to phrase the result.

Your Mission

Convert a character column into a factor, with explicit control over which category serves as the reference level.

Think Before Coding

- From Level 2, what argument to `factor()` controls the order of its categories?
- If a factor's *first* level is typically treated as its reference or baseline category by modeling functions, which category would you want listed first for `smoker`?

58.1. The R Concept

`as.factor(x)` performs a basic character-to-factor conversion, but — exactly like Level 2’s factors page warned — it defaults to **alphabetical** level order, which is not necessarily the order you want for a reference category. Building the factor explicitly with `factor()` and a specified `levels` argument gives you full control:

```
factor(x, levels = c("reference_category", "other_category"))
```

```
smoker_factor <- factor(participants$smoker, levels = c("No", "Yes"))
```

Whichever category is listed *first* in `levels` becomes the factor’s reference level — the one every other category is implicitly compared against in a model like the ones you’ll build starting in Level 11.

i `relevel()` as an alternative

If a factor already exists with the wrong reference level, `relevel(x, ref = "No")` changes just the reference level without rebuilding the whole factor from scratch. Both approaches — specifying `levels` up front, or `relevel()` afterward — arrive at the same result.

58.2. Demonstration

```
smoker_raw <- c("Yes", "No", "No", "Yes", "No")
```

```
# Basic conversion - defaults to alphabetical order ("No" happens to come first)
as.factor(smoker_raw)
```

```
[1] Yes No  No  Yes No
Levels: No Yes
```

58.3. Explore

```
# Explicit, deliberate order - not relying on alphabetical coincidence
smoker_factor <- factor(smoker_raw, levels = c("No", "Yes"))
smoker_factor
```

```
[1] Yes No  No  Yes No
Levels: No Yes
```

```
levels(smoker_factor)
```

```
[1] "No"  "Yes"
```

58.3. Explore

Try a category pair where alphabetical order would *not* match your intended reference level — for instance, `treatment_group <- c("Treatment", "Control", "Treatment", "Control")`. Run `as.factor(treatment_group)` and notice "Control" happens to come first alphabetically anyway; now imagine categories named "Drug A" and "Placebo" instead, where alphabetical order ("Drug A" first) would *not* match the reference level a researcher would actually want ("Placebo" as baseline) — exactly the situation explicit `levels` protects against.

58.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

participants$smoker_factor <- factor(participants$smoker, levels = c("No", "Yes"))

levels(participants$smoker_factor)
```

58. Changing Variable Types: Character to Factor

```
[1] "No"  "Yes"
```

```
table(participants$smoker_factor)
```

```
No Yes  
75  25
```

"No" is now explicitly the first, reference level — guaranteed by the deliberate `levels` argument, not by alphabetical coincidence — ready for Level 11's modeling functions to correctly interpret smoking status as "smoker vs. the reference non-smoker group."

58.5. Interpret the Result

`levels(participants$smoker_factor)` confirms "No" is listed first, and `table()` confirms 75 non-smokers and 25 current smokers in the sample — the factor is now both correctly typed for modeling and explicitly, deliberately ordered, rather than relying on alphabetical order to happen to produce the right reference category.

Common Mistakes

- **Relying on `as.factor()`'s alphabetical default without checking whether it matches your intended reference category.** It happens to work for "No"/"Yes" (alphabetically, "No" comes first anyway), but this is coincidence, not something to depend on for every categorical variable.
- **Assuming factor order doesn't matter until you actually build a model.** It matters immediately for how `table()`, plots (Level 8), and summaries display the category — not just for eventual regression output.

- **Forgetting a factor still displays its labels, even though it's stored as integers internally.** `smoker_factor` prints as "Yes"/"No" text, exactly as expected — the underlying integer codes (Level 2) are an implementation detail, not something you need to manage directly.

58.6. Practice Problem

Convert `participants$sex` into a factor with "Male" as the reference (first) level, even though "Female" would come first alphabetically. Confirm with `levels()` that "Male" is listed first.

58.7. Challenge Problem

A colleague's model output describes an effect as “relative to Riverdale,” implying `county` was converted to a factor with Riverdale as the reference level. Write the `factor()` call that would produce this, given the five counties in `participants$county` (Cedar Falls, Millbrook, Oakford, Riverdale, Springvale) — you determined the correct full set of counties back in Level 2.

58.8. Key Takeaways

- `as.factor()` performs a basic conversion but defaults to alphabetical level order, which may not match your intended reference category.
- `factor(x, levels = c("reference", ...))` gives explicit, deliberate control over which category is the reference (first) level.
- Getting the reference level right matters immediately for tables and plots, not just for eventual regression models in Level 11.

58. Changing Variable Types: Character to Factor

Knowledge Check

1. **Multiple choice.** What determines a factor's reference level, when built with `factor(x, levels = c(...))`?
 - a) Whichever category is most frequent
 - b) Whichever category is listed first in `levels`
 - c) Whichever category is alphabetically last
 - d) It's always chosen randomly
2. **True or false.** `as.factor()` always produces the same reference level as an explicit `factor(x, levels = ...)` call.
3. **Predict the output.**

```
f <- factor(c("High", "Low"), levels = c("Low", "High"))
levels(f)
```

4. **Write the code.** Write the code to convert a character vector `outcome <- c("Improved", "Not Improved")` into a factor with "Not Improved" as the reference (first) level.

Show answers

1. b) Whichever category is listed first in `levels`.
2. **False.** `as.factor()` defaults to alphabetical order, which only matches an explicit `levels` specification by coincidence.
3. `[1] "Low" "High"`
4. `factor(outcome, levels = c("Not Improved", "Improved"))`

59. Creating New Variables

The Problem

Your supervisor asks: “Pulse pressure — the difference between systolic and diastolic blood pressure — is actually a better predictor of cardiovascular risk in older adults than either number alone. Can you add that as a variable?” This isn’t a variable that exists anywhere in the raw data; it needs to be *calculated* from two columns that do.

Your Mission

Create a brand-new column in a data frame, calculated from one or more existing columns.

Think Before Coding

- From Level 1, how would you calculate the difference between two single numbers?
- From Level 2, if `df$col1` and `df$col2` are both vectors of the same length, what would `df$col1 - df$col2` produce — one number, or a whole vector of differences?

59.1. The R Concept

Assigning to a column name that **doesn’t yet exist** in a data frame creates it, exactly as assigning to a brand-new object name creates that object (Level 1):

59. Creating New Variables

```
df$new_column <- <some calculation using existing columns>
```

Because columns are vectors of matching length (Level 2), arithmetic between two columns happens **element-wise** — row 1's calculation uses row 1's values from each column, row 2 uses row 2's values, and so on, automatically, all at once.

```
df$pulse_pressure <- df$systolic_bp - df$diastolic_bp
```

59.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
# Before: no such column exists
```

```
"pulse_pressure" %in% names(participants)
```

```
[1] FALSE
```

```
participants$pulse_pressure <- participants$systolic_bp - participants$diastolic_bp
```

```
"pulse_pressure" %in% names(participants)
```

```
[1] TRUE
```

```
head(participants[, c("systolic_bp", "diastolic_bp", "pulse_pressure")])
```

	systolic_bp	diastolic_bp	pulse_pressure
1	125	79	46
2	123	80	43
3	130	77	53
4	117	72	45
5	139	79	60
6	117	64	53

59.3. Explore

Run `summary(participants$pulse_pressure)` to see the range of pulse pressure values across the sample. Then check `sum(is.na(participants$pulse_pressure))` — since `pulse_pressure` is calculated from two columns that each have some missing values, what would you expect this count to be, compared to the missingness in `systolic_bp` or `diastolic_bp` alone?

59.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

participants$pulse_pressure <- participants$systolic_bp - participants$diastolic_bp

mean(participants$pulse_pressure, na.rm = TRUE)

[1] 49.35417

sum(is.na(participants$pulse_pressure))

[1] 4
```

The new `pulse_pressure` variable is calculated for every participant with both a systolic and diastolic reading, with a mean of about 49.4 mmHg across the sample — ready to hand back to your supervisor as a genuinely new, derived clinical variable.

59.5. Interpret the Result

`sum(is.na(participants$pulse_pressure))` returning 4 (matching exactly the 4 participants missing `systolic_bp`, since `diastolic_bp` has no missing values at all) confirms that a calculation is only as complete as its *least* complete input — any row missing either ingredient produces a missing result, which propagates the same way any arithmetic with NA does (Level 1).

Common Mistakes

- **Expecting a new variable to be complete even when its inputs aren't.** A derived variable inherits missingness from every column it depends on — `pulse_pressure` is missing wherever `systolic_bp` (or, in general, either ingredient) is missing.
- **Overwriting an existing column by accident.** Assigning to a column name that *does* already exist replaces it silently, with no warning — always double check `names(df)` if you're not sure whether a name is already taken.
- **Forgetting the calculation is element-wise.** `df$systolic_bp - df$diastolic_bp` computes 100 separate subtractions, one per row, all at once — not a single overall subtraction of two summary numbers.

59.6. Practice Problem

Create a new column `bp_ratio` calculated as `systolic_bp / diastolic_bp`, and report its mean (excluding missing values).

59.7. Challenge Problem

Create a new column `age_at_next_checkup` representing each participant's age one year from now (`age + 1`). Then explain, in one sentence, why this new column will have exactly the same number of missing values as `age` itself — no more, no fewer.

59.8. Key Takeaways

- Assigning to a data frame column name that doesn't yet exist creates it; assigning to one that already exists overwrites it.
- Arithmetic between two columns happens element-wise, row by row, automatically — the same rule from Level 2's vectors applies directly to data frame columns.
- A derived variable is missing wherever any of its input columns are missing, inheriting incompleteness from its ingredients.

Knowledge Check

1. **Multiple choice.** What does `df$new_col <- df$a - df$b` do if `new_col` doesn't already exist?
 - a) Raises an error
 - b) Creates a new column calculated element-wise from `a` and `b`
 - c) Overwrites column `a`
 - d) Does nothing until `new_col` is explicitly declared first
2. **True or false.** A new column calculated from two existing columns will always have zero missing values, regardless of its inputs.
3. **Predict the output.** If `df$x <- c(10, NA, 30)` and `df$y <- c(1, 2, 3)`, what would `df$x - df$y` produce?

59. Creating New Variables

4. **Write the code.** Write the code to create a new column `bmi_times_10` equal to `participants$bmi` multiplied by 10.

Show answers

1. **b)** Creates a new column, calculated element-wise from `a` and `b`.
2. **False.** A derived column is missing wherever any of its input columns are missing — it cannot be more complete than its least complete ingredient.
3. `c(9, NA, 27)`
4. `participants$bmi_times_10 <- participants$bmi * 10`

60. Recoding Variables

The Problem

Your supervisor wants the final report to use clearer language than the raw data's terse "Yes"/"No" smoking codes — specifically, "Current smoker" and "Non-smoker", which read better in a table meant for a general audience. You need to replace every "Yes" with "Current smoker" and every "No" with "Non-smoker", throughout the entire column.

Your Mission

Recode a variable's category labels — one existing value replaced with a new one, one-to-one — without touching any other column.

Think Before Coding

- From Level 2, if you built a logical vector flagging every row where `smoker == "Yes"`, could you use that same logical vector to assign a new label to just those rows?
- Is this the same idea as Level 4's replacing-missing-values page — assigning a new value only to a targeted subset of positions?

60.1. The R Concept

Recoding a variable's labels uses exactly the same targeted-assignment pattern from the replacing-missing-values page, just with a real category condition instead of `is.na()`:

60. Recoding Variables

```
df$new_col <- df$old_col # start from a copy
df$new_col[df$old_col == "Yes"] <- "Current smoker"
df$new_col[df$old_col == "No"] <- "Non-smoker"
```

For a two-category recode like this one, `ifelse()` (covered properly two pages from now) is often a faster one-line alternative — but building the recode with explicit logical indexing first makes the underlying mechanics completely transparent, which is the point of this page.

60.2. Demonstration

```
smoker_raw <- c("Yes", "No", "No", "Yes", "No")

smoker_label <- smoker_raw # start from a copy, not the original
smoker_label[smoker_raw == "Yes"] <- "Current smoker"
smoker_label[smoker_raw == "No"] <- "Non-smoker"

smoker_label

[1] "Current smoker" "Non-smoker"      "Non-smoker"      "Current smoker"
[5] "Non-smoker"
```

60.3. Explore

Try `table(smoker_raw, smoker_label)` — a cross-tabulation confirming every "Yes" became "Current smoker" and every "No" became "Non-smoker", with no mismatches. This kind of before/after cross-check is a valuable habit any time you recode a variable.

60.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

participants$smoking_status <- participants$smoker
participants$smoking_status[participants$smoker == "Yes"] <- "Current smoker"
participants$smoking_status[participants$smoker == "No"] <- "Non-smoker"

table(participants$smoker, participants$smoking_status)
```

	Current smoker	Non-smoker
No	0	75
Yes	25	0

Every "Yes" correctly recoded to "Current smoker" (25 participants) and every "No" to "Non-smoker" (75 participants), confirmed by the cross-tabulation showing no unexpected mismatches — the report-ready labels your supervisor asked for.

60.5. Interpret the Result

The cross-tabulation's diagonal pattern — "Yes" always paired with "Current smoker", "No" always paired with "Non-smoker", and zero off-diagonal counts — confirms the recode is exactly correct, with every original category mapped to precisely one new label and nothing left ambiguous or mismatched.

Common Mistakes

- **Recoding directly into the original column, losing the original labels.** Starting from a copy (`df$new_col <- df$old_col`, then modifying the copy) preserves the original `smoker` column for any downstream code that still expects "Yes"/"No".

60. Recoding Variables

- **Forgetting to recode every category.** If a variable has more than two categories and you only recode some of them, the un-recoded values silently remain in their original form, mixed in among the newly recoded ones.
- **Not verifying the recode with a cross-tabulation.** `table(old, new)` is a fast, nearly foolproof way to catch a mismatched or incomplete recode before it reaches a report.

60.6. Practice Problem

Recode `participants$sex` into a new column `sex_label`, converting "Female" to "F" and "Male" to "M". Verify with a cross-tabulation.

60.7. Challenge Problem

Your supervisor changes the request: instead of "Current smoker"/"Non-smoker", they want "Current smoker", "Former smoker", and "Never smoker" — a three-category recode that the raw "Yes"/"No" data cannot actually support (it was never collected with a former-smoker distinction). Explain, in a sentence or two, why no amount of recoding code can fix this — what would actually need to change, further upstream, to support this request?

60.8. Key Takeaways

- Recoding uses the same targeted-assignment pattern as replacing missing values, applied to a real category condition instead of `is.na()`.
- Always recode into a new column, preserving the original for any code that still depends on the original labels.

- `table(old_column, new_column)` is a fast, reliable way to verify a recode mapped every category correctly and completely.

Knowledge Check

1. **Multiple choice.** What is the safest way to recode a variable?
 - a) Overwrite the original column directly
 - b) Recode into a new column, preserving the original
 - c) Recode only some categories and leave the rest as-is
 - d) Recode without checking the result
2. **True or false.** `table(old_column, new_column)` is a useful way to verify a recode was applied correctly.
3. **Predict the output.** If `x <- c("A", "B", "A")` and you recode "A" to "Group 1" and "B" to "Group 2", what would the recoded vector be?
4. **Find the error.** A student recodes `smoker` but only handles "Yes", forgetting to also recode "No". What happens to the "No" values in the new column?

Show answers

1. **b)** Recode into a new column, preserving the original.
2. **True.** It's a fast, reliable cross-check that every original category mapped correctly to its new label.
3. `c("Group 1", "Group 2", "Group 1")`
4. Since the new column started as a copy of the original, the unrecoded "No" values remain exactly as "No" — silently unrecoded and inconsistent with the newly labeled "Yes" values.

61. Combining Categories

The Problem

Your supervisor wants a simpler regional comparison than five separate counties allow: “Group Cedar Falls and Millbrook as ‘North Region,’ and Oakford, Riverdale, and Springvale as ‘South Region.’ I don’t need county-level detail for this particular report — I need exactly two regions.” This is a genuinely different operation from recoding: several *different* original categories all need to collapse into the *same* new category.

Your Mission

Collapse several existing categories into a smaller number of broader groups.

Think Before Coding

- From the recoding page, you mapped each old category to exactly one new label, one-to-one. What’s different about mapping *several* old categories to the *same* new label?
- Could you use `%in%` (checking whether a value is one of several options) instead of a single `==` comparison, to flag “any of these counties belongs to the North Region”?

61.1. The R Concept

Combining categories extends the recoding pattern from the last page, using `%in%` to check membership in a *set* of values rather than equality

61. Combining Categories

with just one:

```
x %in% c("value1", "value2")    # TRUE if x matches ANY of these, not just one

df$new_col[df$old_col %in% c("CategoryA", "CategoryB")] <- "Combined Group"
```

This is the natural extension of `==` (from the recoding page) to “matches one of several things” rather than “matches exactly this one thing” — precisely what’s needed when multiple original categories all collapse into the same broader group.

61.2. Demonstration

```
county <- c("Cedar Falls", "Millbrook", "Oakford", "Riverdale", "Springvale")

# Check membership in a set, rather than equality with a single value
county %in% c("Cedar Falls", "Millbrook")
```

```
[1] TRUE TRUE FALSE FALSE FALSE TRUE
```

```
region <- county    # start from a copy, same pattern as recoding
region[county %in% c("Cedar Falls", "Millbrook")] <- "North Region"
region[county %in% c("Oakford", "Riverdale", "Springvale")] <- "South Region"

region
```

```
[1] "North Region" "North Region" "South Region" "South Region" "South Region"
[6] "North Region"
```

61.3. Explore

Run `table(county, region)` — a cross-tabulation confirming every one of the five counties mapped into exactly one of the two intended regions, with no county left unmapped and no county mapped to both.

61.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

participants$region <- participants$county
participants$region[participants$county %in% c("Cedar Falls", "Millbrook")] <- "North Region"
participants$region[participants$county %in% c("Oakford", "Riverdale", "Springvale")] <- "South Region"

table(participants$region)

North Region South Region
      41         59

# Verify every county mapped correctly and completely
table(participants$county, participants$region)
```

	North Region	South Region
Cedar Falls	20	0
Millbrook	21	0
Oakford	0	17
Riverdale	0	16
Springvale	0	26

Every county correctly collapsed into exactly one of the two intended regions — 41 participants in the North Region, 59 in the South Region — with the cross-tabulation confirming no county was missed or double-counted.

61.5. Interpret the Result

The two-way table's diagonal-block pattern — Cedar Falls and Millbrook entirely in the North Region column, Oakford, Riverdale, and Springvale entirely in the South Region column, with zero counts everywhere else — confirms the regional grouping is complete and correct, exactly matching your supervisor's five-counties-into-two-regions request.

Common Mistakes

- **Using `==` instead of `%in%` for a multi-category match.** `county == c("Cedar Falls", "Millbrook")` does not do what you might expect — it compares element-by-element in a way that rarely matches your intent; `%in%` is the correct tool for “matches any of these.”
- **Leaving a category unmapped by accident.** If you forget to include one of the five counties in either group, those rows keep their original county name in the new `region` column, silently mixed in among the properly grouped values — the cross-tabulation check catches this immediately.
- **Assuming combining categories is the same operation as recoding.** Recoding (previous page) is one-to-one; combining categories is many-to-one — the logic (`%in%` vs. `==`) reflects that real difference.

61.6. Practice Problem

Combine `participants$county` into three regions instead of two: "Cedar Falls" and "Millbrook" as "North", "Oakford" and "Riverdale" as "Central", and "Springvale" alone as "East". Verify with a cross-tabulation.

61.7. Challenge Problem

Your supervisor later asks for a *different* grouping of the same five counties for a separate report. Explain, in a sentence or two, why storing the *first* regional grouping in a new column (`region`) rather than overwriting `county` directly makes it easy to add this second, different grouping (perhaps `region_v2`) without losing either version or re-deriving anything from scratch.

61.8. Key Takeaways

- `%in%` checks membership in a set of values, extending the single-value `==` comparison from the recoding page to “matches any of these.”
- Combining categories is a many-to-one operation — several original categories collapse into one broader group — distinct from recoding’s one-to-one relabeling.
- A two-way `table(old_column, new_column)` cross-tabulation confirms every original category was mapped exactly once, with nothing missed or double-mapped.

Knowledge Check

1. **Multiple choice.** What does `x %in% c("A", "B")` check?
 - a) Whether `x` equals "A" and "B" simultaneously
 - b) Whether `x` matches either "A" or "B"
 - c) Whether `x` is alphabetically between "A" and "B"
 - d) Whether `x` is missing
2. **True or false.** Combining categories and recoding a variable are the same operation.
3. **Predict the output.**

61. Combining Categories

```
x <- c("Red", "Blue", "Green")
x %in% c("Red", "Green")
```

4. **Write the code.** Write the code to combine "Yes" and "Sometimes" into a single category "Any use" in a vector called `alcohol_use`, leaving "No" unchanged.

Show answers

1. b) Whether `x` matches either "A" or "B".
2. **False.** Recoding is one-to-one; combining categories is many-to-one — a genuinely different mapping.
3. [1] TRUE FALSE TRUE
4. `alcohol_use[alcohol_use %in% c("Yes", "Sometimes")] <- "Any use"`

62. Creating Binary Variables

The Problem

Level 11 will teach you logistic regression, which needs its outcome variable coded as a clean binary indicator — conventionally 1 for “has the condition” and 0 for “does not” — not text like "Yes"/"No", and not a raw continuous measurement like blood pressure itself. Your supervisor wants a hypertension indicator: 1 if a participant’s systolic blood pressure is 130 or higher (the standard clinical threshold), 0 otherwise.

Your Mission

Create a binary (0/1) indicator variable from a condition applied to an existing variable.

Think Before Coding

- From Level 1, what does a comparison like `systolic_bp >= 130` produce — `TRUE/FALSE`, or something else?
- From Level 2, what happens if you convert a logical `TRUE/FALSE` vector directly to numeric with `as.numeric()`?

62.1. The R Concept

A logical comparison already produces exactly the right underlying information — `TRUE/FALSE` — and R’s `as.numeric()` converts `TRUE` to 1 and `FALSE` to 0 automatically:

62. Creating Binary Variables

```
as.numeric(condition)
```

```
df$is_hypertensive <- as.numeric(df$systolic_bp >= 130)
```

This is a two-step idea compressed into one line: first, `df$systolic_bp >= 130` produces a TRUE/FALSE vector (Level 1); then `as.numeric()` converts that directly into the 1/0 coding that modeling functions expect.

i Keeping it logical vs. converting to numeric

Some analyses are perfectly happy with a plain logical (TRUE/FALSE) column instead of numeric 1/0 — R treats them equivalently in most arithmetic and modeling contexts. Converting explicitly with `as.numeric()` is mainly about matching a convention (like a data dictionary, or a modeling function's specific expectations) rather than a strict technical requirement.

62.2. Demonstration

```
sbp <- c(118, 132, 145, 121, 138)
```

```
sbp >= 130
```

```
[1] FALSE  TRUE  TRUE FALSE  TRUE
```

```
as.numeric(sbp >= 130)
```

```
[1] 0 1 1 0 1
```

62.3. Explore

Try `sum(as.numeric(sbp >= 130))` — since 1s and 0s sum directly to a count, this immediately tells you how many values met the condition, without needing a separate counting step. Compare this to `sum(sbp >= 130)` (summing the logical vector directly, with no explicit `as.numeric()` at all) — do they give the same answer?

62.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

participants$is_hypertensive <- as.numeric(participants$systolic_bp >= 130)

table(participants$is_hypertensive, useNA = "always")

  0    1 <NA>
50  46    4

sum(participants$is_hypertensive, na.rm = TRUE)

[1] 46
```

The `useNA = "always"` argument to `table()` reveals something important here: 4 participants have NA for `is_hypertensive`, not 0 — because their `systolic_bp` was itself missing, and a comparison against a missing value correctly produces NA rather than a guessed FALSE.

62.5. Interpret the Result

46 of the 96 participants with a recorded systolic blood pressure meet the hypertension threshold of 130 or higher — and critically, the 4 participants with missing blood pressure data correctly show NA for this new indicator too, rather than being silently (and wrongly) counted as 0, “not hypertensive.” This is exactly the kind of correct missing-data propagation Level 1 introduced and this level has reinforced throughout.

Common Mistakes

- **Assuming a binary indicator is automatically complete.** Just like any derived variable (this level’s creating-new-variables page), `is_hypertensive` inherits missingness from `systolic_bp` — a missing blood pressure produces a missing indicator, not a 0.
- **Forgetting `useNA = "always"` when checking a table with potential missing data.** Plain `table(participants$is_hypertensive)` silently omits the NA count entirely, which could make you think the variable is more complete than it actually is.
- **Choosing an arbitrary threshold without a clinical or research justification.** `>= 130` reflects a real, standard clinical hypertension threshold — always be able to justify *why* a particular cutoff was chosen, not just that one was chosen.

62.6. Practice Problem

Create a binary indicator `is_obese`, equal to 1 if `bmi >= 30` and 0 otherwise, correctly propagating missing values. Report how many participants are flagged 1.

62.7. Challenge Problem

Create a binary indicator `is_high_risk`, equal to 1 only if a participant is *both* hypertensive (`systolic_bp >= 130`) *and* a current smoker (`smoker == "Yes"`), using `&` (Level 1). Report how many participants meet both criteria, and explain what happens to a participant missing `systolic_bp` when this combined condition is evaluated.

62.8. Key Takeaways

- `as.numeric(condition)` converts a TRUE/FALSE comparison directly into 1/0, matching the coding convention most modeling functions expect.
- A binary indicator inherits missingness from whatever variable it's built from — a missing input produces a missing indicator, not a default 0.
- `table(x, useNA = "always")` explicitly shows missing-value counts that a plain `table(x)` would silently hide.

Knowledge Check

1. **Multiple choice.** What does `as.numeric(c(TRUE, FALSE, TRUE))` return?
 - a) `c("TRUE", "FALSE", "TRUE")`
 - b) `c(1, 0, 1)`
 - c) `c(0, 1, 0)`
 - d) An error
2. **True or false.** A binary indicator built from a column with missing values will itself have no missing values.
3. **Predict the output.**

62. Creating Binary Variables

```
x <- c(125, NA, 140)
as.numeric(x >= 130)
```

4. **Write the code.** Write the code to create a binary indicator `high_pulse_pressure`, equal to 1 if `pulse_pressure > 50` (using the variable created earlier this level) and 0 otherwise.

Show answers

1. b) `c(1, 0, 1)`
2. **False.** Missingness propagates: a missing input produces a missing (NA) indicator, not a default 0.
3. `[1] 0 NA 1`
4. `participants$high_pulse_pressure <- as.numeric(participants$pulse_pressure > 50)`

63. Conditional Statements: `if`

The Problem

Your supervisor asks you to write a short script that checks whether the study has reached its enrollment target of 100 participants, and prints a different message depending on the answer: "Enrollment complete" if so, or "Still recruiting" if not. This isn't about transforming a column of data — it's about making your *script itself* behave differently depending on a single condition.

Your Mission

Use `if` (and `else`) to make a single decision in your code, executing different blocks depending on whether a condition is `TRUE` or `FALSE`.

Think Before Coding

- From Level 1, what does a comparison like `nrow(participants) == 100` produce?
- Is checking “has enrollment reached 100, yes or no” the same kind of question as the column-wide, per-row conditions you’ve used throughout this level, or is it a single, one-time decision?

63.1. The R Concept

`if` evaluates a single logical condition and runs a block of code only if that condition is `TRUE`; an optional `else` provides an alternative block for when it's `FALSE`:

63. Conditional Statements: *if*

```
if (condition) {  
  # runs only if condition is TRUE  
} else {  
  # runs only if condition is FALSE  
}
```

This is a genuinely different tool from the vectorized comparisons you've used throughout this level (`df$col >= 130`, applied to every row at once). `if` expects a **single** TRUE or FALSE value — not a whole vector — and controls which block of code *runs*, rather than producing a new column of results.

```
if (nrow(participants) == 100) {  
  print("Enrollment complete")  
} else {  
  print("Still recruiting")  
}
```

63.2. Demonstration

```
n_enrolled <- 100  
  
if (n_enrolled == 100) {  
  print("Enrollment complete")  
} else {  
  print("Still recruiting")  
}
```

```
[1] "Enrollment complete"
```

```
n_enrolled <- 87  
  
if (n_enrolled == 100) {
```

```

  print("Enrollment complete")
} else {
  print("Still recruiting")
}

```

```
[1] "Still recruiting"
```

63.3. Explore

Try passing a whole vector's worth of values into an `if` condition, like `if (c(TRUE, FALSE)) { print("test") }`, rather than a single value. In current versions of R, this produces an error — `if`'s condition must be a single, length-one logical value, not a vector — a helpful guardrail that stops you from confusing `if` (a single decision) with the vectorized, per-row tools (`ifelse()`, next page) that this level has used throughout.

63.4. Solve the Problem

```

participants <- read.csv("data/participants.csv")

if (nrow(participants) == 100) {
  print("Enrollment complete")
} else {
  print("Still recruiting")
}

```

```
[1] "Enrollment complete"
```

63.5. Interpret the Result

The script correctly reports "Enrollment complete" because `nrow(participants)` is exactly 100 — a single, one-time decision about the *dataset as a whole*, genuinely different from asking “which individual participants meet a condition,” the per-row question every earlier page in this level has answered with vectorized comparisons.

Common Mistakes

- **Trying to use `if` on an entire column at once.** `if (participants$age >= 65)` fails, because `participants$age >= 65` is a vector of 100 values, not one — `if` needs a single TRUE/FALSE. For per-row decisions across a whole column, you want `ifelse()` (next page), not `if`.
- **Forgetting curly braces for multi-line blocks.** A single-line `if` can skip the braces (`if (x) print("yes")`), but as soon as more than one line needs to run conditionally, `{ }` are required to group them together.
- **Confusing `if/else` (a programming control-flow structure) with `ifelse()` (a vectorized function).** They sound almost identical and solve related but distinct problems — this level’s next three pages exist specifically to make that distinction clear.

63.6. Practice Problem

Write an `if/else` block that prints "Sample size is adequate" if `nrow(participants)` is at least 50, and "Sample size may be too small" otherwise.

63.7. Challenge Problem

Write an `if/else if/else` block (a third option, `else if`, chains an additional condition) that checks `mean(participants$age, na.rm = TRUE)` and prints "Younger sample" if under 40, "Older sample" if 60 or above, and "Middle-aged sample" otherwise.

63.8. Key Takeaways

- `if/else` makes a single, one-time decision in your code based on one TRUE/FALSE condition — not a per-row operation across a whole column.
- `if`'s condition must be a single logical value; passing a vector produces an error, a useful guardrail against confusing it with vectorized tools.
- `if/else` and `ifelse()` (next page) are related but distinct tools, and this level's next few pages build out the full picture of when to use each.

Knowledge Check

1. **Multiple choice.** What kind of condition does `if` expect?
 - a) A vector of any length
 - b) A single TRUE or FALSE value
 - c) A character string
 - d) A data frame
2. **True or false.** `if` can be used directly on an entire column of a data frame to make a per-row decision.
3. **Predict the output.**

63. Conditional Statements: *if*

```
x <- 42
if (x > 100) {
  print("big")
} else {
  print("small")
}
```

4. Find the error.

```
if (participants$age >= 65) {
  print("senior")
}
```

Show answers

1. b) A single TRUE or FALSE value.
2. **False.** `if` requires a single logical value; a whole column is a vector of 100 values, which `if` cannot evaluate directly — that's exactly what `ifelse()` is for.
3. `[1] "small"`
4. `participants$age >= 65` is a vector (100 values), not a single value — `if` cannot evaluate it directly. The fix is to use `ifelse()` for a per-row result, or reduce the condition to a single value (like `any()` or `all()`) if a one-time decision is genuinely intended.

64. `ifelse()`

The Problem

You want to label every one of the 100 participants as "Senior" (age 65+) or "Non-senior" — a per-row decision across an entire column, not the single, one-time decision the last page's `if` was built for. You try wrapping `if` around `participants$age >= 65` directly, and it fails immediately, exactly as the previous page's Common Mistakes warned.

Your Mission

Apply a single condition across every row of a column at once, producing a different value depending on whether each row's condition is `TRUE` or `FALSE`.

Think Before Coding

- From the binary-variables page, you already built `is_hypertensive` using a comparison and `as.numeric()`. What if, instead of 1/0, you wanted actual text labels for each outcome?
- Could a function exist that does what `if/else` does, but automatically, for every element of a vector at once?

64.1. The R Concept

`ifelse()` is the vectorized sibling of `if/else` — it evaluates a condition across an *entire* vector at once, returning one value for every `TRUE` and another for every `FALSE`:

64. `ifelse()`

```
ifelse(condition, value_if_true, value_if_false)
```

```
ifelse(participants$age >= 65, "Senior", "Non-senior")
```

This is exactly the tool the previous page's binary-variables and recoding pages were building toward manually with logical indexing — `ifelse()` compresses that same idea (a condition, a value for TRUE, a value for FALSE) into a single function call.

64.2. Demonstration

```
ages <- c(72, 45, 68, 30, 81)
```

```
ifelse(ages >= 65, "Senior", "Non-senior")
```

```
[1] "Senior"      "Non-senior" "Senior"      "Non-senior" "Senior"
```

```
# Compare to the manual, logical-indexing approach from earlier this level
age_group <- rep(NA_character_, length(ages))
age_group[ages >= 65] <- "Senior"
age_group[ages < 65]  <- "Non-senior"
age_group
```

```
[1] "Senior"      "Non-senior" "Senior"      "Non-senior" "Senior"
```

Both approaches give an identical result — `ifelse()` is simply a faster, more compact way to express exactly what the earlier logical-indexing pattern was already doing.

64.3. Explore

Try `ifelse(c(72, NA, 68) >= 65, "Senior", "Non-senior")` — what happens to the NA position? `ifelse()` correctly propagates missingness: if the condition itself is NA (because the underlying age is missing), the result is NA too, neither "Senior" nor "Non-senior" — exactly the correct behavior, consistent with everything this level has taught about missing data.

64.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

participants$age_group <- ifelse(participants$age >= 65, "Senior", "Non-senior")

table(participants$age_group, useNA = "always")
```

Non-senior	Senior	<NA>
83	11	6

Every participant is now labeled "Senior" or "Non-senior" in one line, with the 6 participants missing `age` correctly showing NA rather than being forced into either category.

64.5. Interpret the Result

11 of the 94 participants with a recorded age are 65 or older, 83 are under 65, and the 6 participants with missing age data correctly remain NA for this new label — `ifelse()` produced, in a single line, exactly the same

64. `ifelse()`

correct, missing-data-aware result the manual logical-indexing approach would have taken several lines to build.

Common Mistakes

- **Using `if` instead of `ifelse()` on a whole column.** This is exactly the error the previous page's Explore and Challenge sections set up — `if` cannot evaluate a multi-element vector; `ifelse()` is built specifically for that.
- **Forgetting `ifelse()` still requires exactly three arguments in order.** `ifelse(condition, value_if_true, value_if_false)` — reversing the last two silently swaps your logic ("Senior" for people *under* 65, and vice versa) with no error to warn you.
- **Assuming `ifelse()` always returns the type you expect.** `ifelse()` can sometimes silently convert types in surprising ways (particularly with dates or factors) — this is precisely the motivation for `if_else()`, covered on the next page, which is stricter about this.

64.6. Practice Problem

Create `bp_group` using `ifelse()`, labeling participants "Elevated" if `systolic_bp >= 130` and "Normal" otherwise. Confirm missing values propagate correctly with `table(..., useNA = "always")`.

64.7. Challenge Problem

Nest two `ifelse()` calls to create a three-category `age_category`: "Youth" (under 18), "Adult" (18–64), and "Senior" (65+) — using the pattern `ifelse(condition1, "Youth", ifelse(condition2, "Adult", "Senior"))`. Test it on `c(15, 45, 70)` and confirm all three labels appear correctly.

64.8. Key Takeaways

- `ifelse(condition, value_if_true, value_if_false)` applies a condition across an entire vector at once, unlike `if`, which requires a single value.
- `ifelse()` correctly propagates NA: a missing input produces a missing (NA) result, never a guessed category.
- Nested `ifelse()` calls can build more than two categories, though this gets unwieldy quickly — a limitation `case_when()` (this level's final page) resolves much more cleanly.

Knowledge Check

1. **Multiple choice.** What does `ifelse(x > 10, "high", "low")` return for `x <- c(5, 15)`?
 - a) `c("low", "high")`
 - b) `c("high", "low")`
 - c) A single value, "high" or "low"
 - d) An error, since `x` has more than one element
2. **True or false.** `ifelse()` requires a single-value condition, exactly like `if`.
3. **Predict the output.**

```
ifelse(c(10, NA, 30) > 20, "big", "small")
```

4. **Find the error.**

```
ifelse(participants$age >= 65, "Non-senior", "Senior")
```

A student intends "Senior" for age 65+.

Show answers

64. `ifelse()`

1. **a)** `c("low", "high")` — `ifelse()` evaluates every element of the vector.
2. **False.** `ifelse()` is specifically vectorized — it works across an entire vector, unlike `if`, which requires exactly one value.
3. `[1] "small" NA "big"`
4. The second and third arguments are swapped — this labels people *under* 65 as "Senior" and 65+ as "Non-senior", backwards from what's intended. It should be `ifelse(participants$age >= 65, "Senior", "Non-senior")`.

65. `if_else()`

The Problem

A colleague's script uses `ifelse(is_followup, visit_date, NA)` to blank out a date column for baseline visits — and it silently converts every proper date into a plain number (the internal representation of a date), breaking every later step that expected an actual date. Nothing in `ifelse()` warned them this would happen.

Your Mission

Understand `if_else()`, the tidyverse's stricter version of `ifelse()`, and why it prevents exactly this kind of silent type-conversion bug.

Think Before Coding

- From Level 2, what does R do when you mix two different types together, like a number and a piece of text, inside `c()`?
- If `ifelse()`'s second and third arguments (the `TRUE` and `FALSE` results) were of two genuinely different types, what do you think should happen — a clear error, or a silent, best-effort guess?

65.1. The R Concept

`if_else()`, from the `dplyr` package (part of the tidyverse, arriving properly in Level 5), works exactly like `ifelse()` for straightforward cases, but is **stricter about types**: if the `TRUE` result and the `FALSE` result aren't

65. `if_else()`

compatible types, `if_else()` raises a clear error immediately, rather than silently coercing one of them into something unexpected.

```
library(dplyr)
if_else(condition, value_if_true, value_if_false)

# ifelse() silently converts dates to plain numbers - a common, hard-to-spot
ifelse(is_followup, visit_date, NA)

# if_else() raises a clear, immediate error instead of silently corrupting the
if_else(is_followup, visit_date, NA)
```

The fix, once you know about the problem, is usually to make the NA explicitly typed to match: `if_else(is_followup, visit_date, as.Date(NA))` for a date column, for instance — `if_else()` forces you to be explicit rather than letting a mismatch pass silently.

65.2. Demonstration

```
library(dplyr)

# A straightforward case - if_else() behaves identically to ifelse()
ages <- c(72, 45, 68)
if_else(ages >= 65, "Senior", "Non-senior")

[1] "Senior"      "Non-senior" "Senior"

# A type-mismatch case - ifelse() silently causes a problem, if_else() catches it
is_followup <- c(TRUE, FALSE, TRUE)
visit_date <- as.Date(c("2026-01-05", "2026-01-06", "2026-01-07"))

ifelse(is_followup, visit_date, NA)
```

```
[1] 20458      NA 20460
```

Notice the dates silently became plain numbers (each date's internal numeric representation) — exactly the bug from the opening problem.

```
if_else(is_followup, visit_date, NA)
```

```
Error in `if_else()`:  
! `false` must be a <Date>, not `NA`.
```

`if_else()` refuses to guess, and instead tells you *exactly* what went wrong — the FALSE result must be a proper `Date`, not a bare, untyped NA.

65.3. Explore

Fix the error by explicitly typing the NA: `if_else(is_followup, visit_date, as.Date(NA))`. Confirm this now works correctly, preserving the `Date` type throughout, with no silent conversion to plain numbers.

65.4. Solve the Problem

```
library(dplyr)

is_followup <- c(TRUE, FALSE, TRUE)
visit_date <- as.Date(c("2026-01-05", "2026-01-06", "2026-01-07"))

visit_date_clean <- if_else(is_followup, visit_date, as.Date(NA))
visit_date_clean
```

```
65. if_else()

[1] "2026-01-05" NA          "2026-01-07"

class(visit_date_clean)

[1] "Date"
```

The colleague's bug is now fixed and, more importantly, made impossible to silently reintroduce — `if_else()` would immediately raise an error again if the types ever mismatched in the future, rather than quietly corrupting the column the way `ifelse()` did.

65.5. Interpret the Result

`class(visit_date_clean)` correctly reports `"Date"`, confirming the column's type was preserved throughout — the baseline visit correctly shows `NA` (a real, typed missing date) rather than the number 20459 (or similar) that `ifelse()` would have silently produced.

Common Mistakes

- **Assuming `if_else()` and `ifelse()` are simply two names for the same thing.** They agree on simple cases, but `if_else()`'s stricter type-checking is a deliberate, meaningful difference — not a stylistic quirk.
- **Being annoyed by an `if_else()` error instead of reading what it's telling you.** The error message in this page's demonstration says exactly what's wrong (`false must be a <Date>`, not `NA`) — it's surfacing a real problem `ifelse()` would have hidden, not being unnecessarily strict for no reason.
- **Forgetting to load `dplyr`.** Like any package function, `if_else()` requires `library(dplyr)` (or `library(tidyverse)`) first.

65.6. Practice Problem

Using `if_else()`, recreate the `age_group` variable from the previous page ("Senior" for 65+, "Non-senior" otherwise) on `participants$age`, and confirm it produces the same result as the `ifelse()` version.

65.7. Challenge Problem

Explain, in two or three sentences, why a strict, error-raising function like `if_else()` is generally *safer* for production analysis code than a permissive, silently-converting one like `ifelse()` — even though the strict version is sometimes more annoying to work with in the moment.

65.8. Key Takeaways

- `if_else()` (from `dplyr`) behaves like `ifelse()` on straightforward cases, but raises a clear error rather than silently converting types when the TRUE and FALSE results don't match.
- This strictness catches real bugs — like a `Date` column silently becoming plain numbers — immediately, rather than letting them surface much later and harder to trace.
- When `if_else()` errors on a type mismatch, the fix is usually to make both results explicitly the same type (like `as.Date(NA)` instead of a bare `NA`).

Knowledge Check

1. **Multiple choice.** What does `if_else()` do differently from `ifelse()`?
 - a) It works only on numeric data

65. `if_else()`

- b) It raises an error rather than silently converting mismatched types
 - c) It cannot handle missing values
 - d) It requires no package to be loaded
2. **True or false.** `ifelse()` can silently convert a `Date` column into plain numbers.
3. **Predict the output.**

```
if_else(c(TRUE, FALSE), "Yes", "No")
```

4. **Find the error.**

```
if_else(is_followup, visit_date, NA)
```

where `visit_date` is a `Date` column.

Show answers

- 1. **b)** It raises an error rather than silently converting mismatched types.
- 2. **True.** This is exactly the bug `if_else()` is designed to catch and prevent.
- 3. `[1] "Yes" "No"`
- 4. The bare `NA` doesn't match the `Date` type of `visit_date`; it needs to be `as.Date(NA)` instead, so both possible results share the same type.

66. `case_when()`

The Problem

Back in Level 2, you built a four-category `bmi_category` variable using three *nested* `ifelse()` calls: `ifelse(bmi < 18.5, "Underweight", ifelse(bmi < 25, "Normal", ifelse(bmi < 30, "Overweight", "Obese")))`. It worked, but even at four categories it was already hard to read at a glance — and your supervisor now wants a five-category blood pressure classification. Nesting `ifelse()` four levels deep would be close to unreadable.

Your Mission

Use `case_when()` to handle several conditions at once, cleanly, without nesting `ifelse()` calls inside each other.

Think Before Coding

- If you had five conditions to check in order, would you rather read them as five separate, parallel lines, or as `ifelse()` nested four levels deep?
- What should happen if a row matches none of your listed conditions — should it get a value at all, or should something else happen?

66.1. The R Concept

`case_when()`, from `dplyr` (arriving properly in Level 5), evaluates a series of conditions **in order**, top to bottom, and assigns the value from the **first**

66. `case_when()`

one that matches:

```
library(dplyr)
case_when(
  condition1 ~ value1,
  condition2 ~ value2,
  condition3 ~ value3,
  TRUE ~ default_value
)
```

Each line reads naturally: “condition ~ (results in) value.” The final `TRUE ~ default_value` line acts as a catch-all — since `TRUE` always matches, it captures any row that didn’t match an earlier condition (equivalent to the final `else` in a nested `ifelse()` chain, but far more readable at a glance).

```
case_when(
  bmi < 18.5 ~ "Underweight",
  bmi < 25   ~ "Normal",
  bmi < 30   ~ "Overweight",
  TRUE      ~ "Obese"
)
```

Notice this reads almost like a sentence: “if BMI is under 18.5, Underweight; if under 25 (having already excluded under-18.5), Normal;” and so on — each condition is only evaluated for rows that didn’t match anything above it.

66.2. Demonstration

```
library(dplyr)

bmi_values <- c(17.2, 22.5, 28.9, 33.1, NA)
```

```
case_when(
  bmi_values < 18.5 ~ "Underweight",
  bmi_values < 25  ~ "Normal",
  bmi_values < 30  ~ "Overweight",
  TRUE           ~ "Obese"
)

[1] "Underweight" "Normal"      "Overweight"  "Obese"      NA
```

Notice the NA BMI correctly produces NA in the result — even the catch-all `TRUE ~ "Obese"` doesn't fire for a missing input, because `case_when()` (like `ifelse()` and `if_else()`) correctly propagates missingness rather than guessing.

Compare this to the equivalent, harder-to-read nested `ifelse()` from Level 2:

```
ifelse(bmi_values < 18.5, "Underweight",
  ifelse(bmi_values < 25, "Normal",
    ifelse(bmi_values < 30, "Overweight", "Obese")))
```

Both produce the same result — `case_when()` simply reads far more clearly once there are more than two categories involved.

66.3. Explore

Try adding a fifth category to the blood pressure classification: "Normal" (< 120), "Elevated" (120–129), "Hypertension Stage 1" (130–139), "Hypertension Stage 2" (140–179), and "Hypertensive Crisis" (180+), using `case_when()` on `participants$systolic_bp`. Notice how each additional category is just one more readable line, rather than another level of nesting.

```
66. case_when()
```

66.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

participants$bp_category <- case_when(
  participants$systolic_bp < 120 ~ "Normal",
  participants$systolic_bp < 130 ~ "Elevated",
  participants$systolic_bp < 140 ~ "Hypertension Stage 1",
  participants$systolic_bp < 180 ~ "Hypertension Stage 2",
  TRUE ~ "Hypertensive Crisis"
)

table(participants$bp_category, useNA = "always")
```

	Elevated	Normal	Hypertension Stage 1	
	18	32		17
Hypertension Stage 2		Hypertensive Crisis		<NA>
	29	0		4

Five clinically meaningful categories, built in five clean, parallel lines — no nested nightmare of nine mismatched parentheses to debug, and the 4 participants with missing `systolic_bp` correctly show NA rather than being forced into any category.

66.5. Interpret the Result

The blood pressure classification is now complete across all five clinical categories, with the missing-data behavior correctly preserved (4 participants with missing `systolic_bp` remain NA, not defaulted into "Normal" or any other category) — exactly the behavior you'd want, and considerably easier to read, extend, or hand off to a colleague than an equivalent nested `ifelse()` chain would have been.

Common Mistakes

- **Forgetting conditions are checked in order, top to bottom.** If you accidentally list a broader condition before a narrower one (like `systolic_bp < 180` before `systolic_bp < 120`), the narrower condition below it will *never* be reached, since the broader one already matched first.
- **Forgetting the final TRUE ~ default catch-all.** Without it, any row that doesn't match any listed condition becomes NA in the result — sometimes exactly what you want, but often an unintentional gap if you forgot a category.
- **Using = instead of ~.** `case_when()`'s syntax specifically requires the tilde (~), not an equals sign — a common typo for anyone used to other assignment-style syntax in R.

66.6. Practice Problem

Using `case_when()`, classify `participants$age` into "Child" (under 18), "Adult" (18–64), and "Senior" (65+). Confirm the result with `table(..., useNA = "always")`.

66.7. Challenge Problem

Rebuild Level 2's four-category `bmi_category` variable using `case_when()` instead of nested `ifelse()`, and confirm with `table()` that it produces exactly the same category counts as Level 2's original version (3 Underweight, 28 Normal, 39 Overweight, 25 Obese).

66. `case_when()`

66.8. Key Takeaways

- `case_when()` evaluates conditions in order and assigns the value from the first match — far more readable than nesting `ifelse()` calls for three or more categories.
- A final `TRUE ~ default_value` line acts as a catch-all for any row that matched nothing above it.
- Like `ifelse()` and `if_else()`, `case_when()` correctly propagates missing values — a missing input never gets forced into the catch-all category.

Knowledge Check

1. **Multiple choice.** In `case_when()`, what does the final `TRUE ~ "default"` line do?
 - a) Nothing; it's optional decoration
 - b) Catches any row that didn't match an earlier condition
 - c) Overrides every other condition
 - d) Only applies to missing values
2. **True or false.** `case_when()` conditions are checked in order, and the first match wins.
3. **Predict the output.**

```
x <- c(5, 15, 25, NA)
case_when(
  x < 10 ~ "low",
  x < 20 ~ "medium",
  TRUE  ~ "high"
)
```

4. **Find the error.**


```
case_when(
  age = "Senior" ~ age >= 65,
  TRUE ~ "Non-senior"
)
```

Show answers

1. **b)** Catches any row that didn't match an earlier condition.
2. **True.** Conditions are evaluated top to bottom, and the first **TRUE** match determines the result.
3. [1] "low" "medium" "high" NA
4. The condition and value are reversed, and = is used instead of ~. It should be `age >= 65 ~ "Senior"`.

66.9. Where you've arrived

You started this level with a vague sense that real data needs fixing before it can be trusted, and a stack of tools scattered across Level 1 through Level 3 for spotting problems. You've now built a complete, systematic toolkit for actually *fixing* what you find: identifying, removing, and replacing missing values; detecting and removing duplicates; renaming and retyping variables in every direction; building new variables and recoding, combining, and binarizing categories; and — across these last four pages — the full progression from a single `if` decision to `ifelse()`'s vectorized version, `if_else()`'s type-safe version, and `case_when()`'s clean, multi-condition version.

That last tool, `case_when()`, is also your bridge into Level 5: it's a **dplyr** function, and Level 5 is where you meet the rest of the tidyverse — `filter()`, `mutate()`, `summarise()`, and the pipe `|>` — that will make everything you've done so far in base R read even more like the sentence you'd say the question in out loud.

Part V.

Level 5: Tidyverse

Level 5 — Tidyverse

Where this level is headed

Base R can do everything you’ve done through Level 4 — but reread some of that code: `participants[participants$age >= 65 & participants$systolic_bp >= 130, ]`. It works, but it doesn’t read like the question you’re actually asking. The tidyverse — `dplyr` especially — lets you write “take the data, filter to seniors with elevated blood pressure, group by county, summarise the mean” almost exactly as you’d say it out loud.

What you’ll build toward

By the end of this level you’ll rewrite nearly everything from Level 4 more readably: selecting and filtering with `select()` and `filter()`, sorting with `arrange()`, building and summarizing variables with `mutate()` and `summarise()`, grouped analysis with `group_by()`, quick counts and unique values with `count()` and `distinct()`, tidyverse-style renaming, applying one operation across many columns with `across()`, and chaining every one of these together with the native pipe `|>` into a single, readable pipeline.

Topics in this level

- `select()`, `filter()`, `arrange()`
- `mutate()` and `summarise()`

Level 5 — Tidyverse

- `group_by()`, `count()`, `distinct()`
- `rename()` and `across()`
- The native pipe `|>`
- Chaining multi-step analyses

A preview mission

Your mission across this level: your professor asks you to identify every participant 65 or older with elevated blood pressure — in one readable pipeline, not a maze of nested brackets and repeated `df$` references.

By the last page, you'll write a single chained pipeline — `import`, `filter`, `group`, `summarise`, `arrange` — that reads, top to bottom, almost exactly like the research question it answers.

67. `select()`

The Problem

Your professor asks for a simplified version of the dataset for a hand-out: just `id`, `age`, and `county` — none of the blood pressure, BMI, or smoking columns. In base R, you'd write `participants[, c("id", "age", "county")]` — functional, but the intent (“just give me these three columns”) isn't especially obvious from the syntax itself.

Your Mission

Choose specific columns from a data frame using `dplyr::select()`, and understand why its syntax reads more directly than base R's bracket indexing.

Think Before Coding

- From Level 2, what did `participants[, c("id", "age", "county")]` do — which part specified rows, and which specified columns?
- If a function were named literally after what it does — “select these columns” — what would you expect its arguments to look like?

67.1. The R Concept

`select()`, from `dplyr` (part of the tidyverse), picks columns by name, with no quotes and no brackets required:

```
67. select()
```

```
library(dplyr)
select(df, col1, col2, col3)

select(participants, id, age, county)
```

You can also **exclude** columns with a minus sign, mirroring Level 2's negative indexing:

```
select(participants, -bmi)          # everything except bmi
select(participants, -c(bmi, smoker)) # everything except bmi and smoker
```

And several helper functions let you select by *pattern* rather than typing every name: `starts_with("sys")`, `ends_with("_bp")`, `contains("bp")` — useful once a dataset has dozens of similarly named columns.

67.2. Demonstration

```
library(dplyr)

participants <- read.csv("data/participants.csv")

simplified <- select(participants, id, age, county)
head(simplified, 3)
```

```
   id age  county
1  1  46  Oakford
2  2  53 Riverdale
3  3  59  Oakford
```

```
without_extras <- select(participants, -bmi, -smoker)
names(without_extras)
```

```
[1] "id"          "age"          "sex"          "county"       "systolic_bp"
[6] "diastolic_bp"
```


67.3. Explore

```
bp_only <- select(participants, ends_with("_bp"))
head(bp_only, 3)
```

	systolic_bp	diastolic_bp
1	125	79
2	123	80
3	130	77

67.3. Explore

Try `select(participants, county, age, id)` — notice the *order* of the columns in the result matches the order you listed them in, not their original order in the dataset. This is a genuinely useful side effect: `select()` can reorder columns, not just filter which ones appear.

67.4. Solve the Problem

```
library(dplyr)

participants <- read.csv("data/participants.csv")

handout_data <- select(participants, id, age, county)

head(handout_data)
```

	id	age	county
1	1	46	Oakford
2	2	53	Riverdale
3	3	59	Oakford
4	4	49	Millbrook

```
67. select()
```

```
5 5 61 Millbrook  
6 6 18 Riverdale
```

```
ncol(handout_data)
```

```
[1] 3
```

Exactly the three columns your professor asked for, in the order requested, with the code itself reading almost like the request: “select `id`, `age`, `county`.”

67.5. Interpret the Result

`handout_data` now contains only the three requested columns, ready to export (Level 3’s `write_csv()`) or print directly for the handout — and reading the `select()` call back, its intent is immediately obvious in a way `participants[, c("id", "age", "county")]` requires a moment more thought to parse.

Common Mistakes

- **Quoting column names inside `select()`.** Unlike base R’s bracket indexing, `select()` uses **unquoted** column names directly — `select(participants, "age")` still works, but `select(participants, age)` is the idiomatic tidyverse style.
- **Forgetting the minus sign drops, rather than keeps, a column.** `select(participants, -age)` returns *every column except age* — a common first-encounter mix-up.
- **Forgetting to load `dplyr`.** Like every package function since Level 1, `select()` requires `library(dplyr)` (or `library(tidyverse)`) first.

67.6. Practice Problem

Use `select()` to create a data frame with just `county`, `smoker`, and `bmi`, in that order, and confirm with `names()` that the columns appear in the order you specified.

67.7. Challenge Problem

Use `select()` with a helper function (`starts_with()`, `ends_with()`, or `contains()`) to select every column whose name contains the text "bp", without typing `systolic_bp` or `diastolic_bp` by name individually.

67.8. Key Takeaways

- `select(df, col1, col2, ...)` picks columns by unquoted name, reading more directly than base R's bracket-and-quotes indexing.
- A minus sign (`-col`) excludes a column rather than selecting it; the order columns are listed in becomes their order in the result.
- Helper functions like `starts_with()`, `ends_with()`, and `contains()` select columns by name pattern, useful for datasets with many similarly named variables.

Knowledge Check

1. **Multiple choice.** What does `select(df, -age)` do?
 - a) Selects only the `age` column
 - b) Selects every column except `age`
 - c) Removes all rows where `age` is missing
 - d) Renames the `age` column
2. **True or false.** Column names inside `select()` must be quoted.

67. `select()`

3. **Predict the output.** What columns (and in what order) would `select(df, county, id)` return, if `df` originally has columns in the order `id`, `age`, `county`?
4. **Write the code.** Write the `select()` call to keep only `id` and `smoker` from `participants`.

Show answers

1. **b)** Selects every column except `age`.
2. **False.** `select()` uses unquoted column names as its idiomatic style, though quoted names also work.
3. `county` first, then `id` — in the order listed in the `select()` call, not the original order.
4. `select(participants, id, smoker)`

68. filter()

The Problem

Your epidemiology professor asks you to identify all participants aged 65 years or older who also have elevated blood pressure (systolic 130 or higher) — the group a clinician would flag for closer monitoring. In base R (Level 2), you’d write `participants[participants$age >= 65 & participants$systolic_bp >= 130, ]` — correct, but dense, with `participants$` repeated for every condition and the row/column comma easy to forget.

Your Mission

Subset rows of a data frame based on one or more conditions, using `dplyr::filter()`.

Think Before Coding

- From Level 2’s logical indexing, what did `participants$age >= 65 & participants$systolic_bp >= 130` produce — and what did you do with that result to subset rows?
- If a function were named directly after the action “filter to rows matching a condition,” would you expect to still need `participants$` in front of every column name?

68. *filter()*

68.1. The R Concept

`filter()` keeps only the rows where a condition is `TRUE`, using column names directly with no `df$` prefix and no comma-based row/column indexing:

```
filter(df, condition)
```

```
filter(participants, age >= 65 & systolic_bp >= 130)
```

Multiple conditions can also be separated by commas instead of `&` — `filter()` treats a comma-separated list of conditions as “all of these must be true,” identical to combining them with `&`:

```
filter(participants, age >= 65, systolic_bp >= 130)    # equivalent to using &
```

68.2. Demonstration

```
library(dplyr)
```

```
participants <- read.csv("data/participants.csv")
```

```
# Base R, from Level 2 - for comparison
```

```
base_r_result <- participants[participants$age >= 65 & participants$systolic_bp >= 130,]  
nrow(base_r_result)
```

```
[1] 4
```

```
# dplyr - same result, more directly readable
```

```
tidy_result <- filter(participants, age >= 65 & systolic_bp >= 130)  
nrow(tidy_result)
```

```
[1] 4
```

```
identical(base_r_result, tidy_result)
```

```
[1] TRUE
```

68.3. Explore

Try `filter(participants, age >= 65, systolic_bp >= 130)` (commas instead of `&`) and confirm it produces the identical result. Then try `filter(participants, county == "Oakford" | county == "Millbrook")` — using `|` (or) to match either of two counties at once, extending Level 1's logical operators directly.

68.4. Solve the Problem

```
library(dplyr)
```

```
participants <- read.csv("data/participants.csv")
```

```
at_risk <- filter(participants, age >= 65 & systolic_bp >= 130)
```

```
at_risk
```

	id	age	sex	county	systolic_bp	diastolic_bp	bmi	smoker
1	31	70	Male	Oakford	145	85	NA	No
2	51	66	Male	Cedar Falls	133	77	34.5	No
3	62	81	Male	Millbrook	153	95	30.3	Yes
4	84	67	Female	Riverdale	134	79	26.8	No

Exactly 4 participants meet both criteria — ready to hand to your professor as the flagged, at-risk subgroup, in one readable line rather than a bracket-and-comma expression.

68. `filter()`

68.5. Interpret the Result

Four participants — ages 70, 66, 81, and 67 — are both 65 or older and have elevated systolic blood pressure (130+), spanning four different counties and including one current smoker. This is exactly the clinically relevant subgroup your professor asked you to identify, produced by a single `filter()` call whose logic is legible at a glance.

Common Mistakes

- **Forgetting `filter()` needs no `df$` prefix.** `filter(participants, participants$age >= 65)` still technically runs, but defeats the point — `filter(participants, age >= 65)` is the idiomatic, intended style.
- **Using `=` instead of `==`.** Exactly the Level 1 trap, still relevant here: `filter(participants, sex = "Female")` is a syntax error inside `filter()` (R interprets `=` as a named-argument assignment) — it must be `sex == "Female"`.
- **Confusing `&` (and) with `|` (or), or forgetting the distinction entirely.** `filter(participants, age >= 65 & systolic_bp >= 130)` requires *both* conditions; `filter(participants, age >= 65 | systolic_bp >= 130)` requires only *either* one — very different result sets.

68.6. Practice Problem

Use `filter()` to find every participant who is a current smoker (`smoker == "Yes"`) **and** has a BMI of 30 or higher. Report how many participants meet both criteria.

68.7. Challenge Problem

Use `filter()` with `%in%` (Level 4) to find every participant living in either Riverdale or Cedar Falls, then separately confirm this matches the count you'd get by adding `table(participants$county)`'s counts for those two counties individually.

68.8. Key Takeaways

- `filter(df, condition)` keeps only rows matching a condition, using column names directly with no `df$` prefix or bracket-comma syntax.
- Multiple conditions can be combined with `&` (and), `|` (or), or by separating them with commas (equivalent to `&`).
- `filter()` produces exactly the same result as base R's logical row indexing (Level 2) — the difference is entirely in readability, not correctness.

Knowledge Check

1. **Multiple choice.** What does `filter(df, age >= 65 & bmi >= 30)` return?
 - a) Every row where either condition is true
 - b) Every row where both conditions are true
 - c) Only the `age` and `bmi` columns
 - d) A single logical value
2. **True or false.** Separating conditions with a comma inside `filter()` behaves like combining them with `&`.
3. **Predict the output.** How many rows would `filter(df, TRUE)` return, for any data frame `df`?

68. `filter()`

4. **Find the error.**

```
filter(participants, sex = "Female")
```

Show answers

1. **b)** Every row where both conditions are true.
2. **True.** A comma-separated list of conditions in `filter()` is equivalent to combining them with `&`.
3. Every row — `TRUE` matches everything, so `filter(df, TRUE)` returns the entire, unfiltered data frame.
4. `filter()` requires `==` for comparison, not `=` (which R interprets as a named-argument assignment inside the function call). It should be `filter(participants, sex == "Female")`.

69. `arrange()`

The Problem

Your professor wants to see the five oldest participants in the study first, to spot-check whether the oldest enrollees have unusually high blood pressure. The dataset isn't sorted by age at all — it's in enrollment order — so you need to reorder the entire dataset by age before you can even look at the top of it.

Your Mission

Sort a data frame's rows by one or more columns, in ascending or descending order.

Think Before Coding

- If you wanted to sort a plain vector of numbers, what base R function (from earlier levels) might you reach for?
- Sorting a whole *data frame* by one column needs to move every other column's values along with it, row by row — is that different from sorting a single, standalone vector?

69.1. The R Concept

`arrange()` sorts a data frame's rows by one or more columns, in ascending order by default:

69. `arrange()`

```
arrange(df, column)          # ascending (smallest first)
arrange(df, desc(column))    # descending (largest first)
```

```
arrange(participants, age)   # youngest first
arrange(participants, desc(age)) # oldest first
```

Unlike sorting a plain vector (where only the values themselves move), `arrange()` reorders *entire rows* together — every column's value for a given participant stays correctly matched to that same participant, just in a new row position.

You can also sort by multiple columns; ties in the first column are broken by the second:

```
arrange(participants, county, desc(age)) # by county, then oldest-first wi
```

69.2. Demonstration

```
library(dplyr)
```

```
participants <- read.csv("data/participants.csv")
```

```
youngest_first <- arrange(participants, age)
head(youngest_first, 3)
```

	id	age	sex	county	systolic_bp	diastolic_bp	bmi	smoker
1	6	18	Female	Riverdale	117	64	24.0	No
2	33	18	Female	Oakford	149	94	30.3	Yes
3	36	18	Male	Springvale	152	88	23.1	No

```
oldest_first <- arrange(participants, desc(age))
head(oldest_first, 3)
```

69.3. Explore

	id	age	sex	county	systolic_bp	diastolic_bp	bmi	smoker
1	62	81	Male	Millbrook	153	95	30.3	Yes
2	19	80	Female	Cedar Falls	109	79	28.4	No
3	17	79	Female	Springvale	117	73	21.1	Yes

69.3. Explore

Try `arrange(participants, county, desc(age))` — sorting first by county (alphabetically), then by age (oldest first) *within* each county. Compare this to `arrange(participants, desc(age), county)` — sorting by age first, with county only breaking ties. Notice these produce genuinely different orderings, since the *first* column listed always takes priority.

69.4. Solve the Problem

```
library(dplyr)
```

```
participants <- read.csv("data/participants.csv")
```

```
oldest_first <- arrange(participants, desc(age))
```

```
head(oldest_first, 5)
```

	id	age	sex	county	systolic_bp	diastolic_bp	bmi	smoker
1	62	81	Male	Millbrook	153	95	30.3	Yes
2	19	80	Female	Cedar Falls	109	79	28.4	No
3	17	79	Female	Springvale	117	73	21.1	Yes
4	50	74	Male	Millbrook	114	75	33.5	No
5	31	70	Male	Oakford	145	85	NA	No

69. `arrange()`

The five oldest participants (ages 81, 80, 79, 74, and 70) are now at the top, ready for your professor to spot-check — and notably, two of the five oldest participants (ages 81 and 70) do have elevated systolic blood pressure, echoing the pattern the previous page’s `filter()` began to surface.

69.5. Interpret the Result

`arrange(participants, desc(age))` reordered every row by age, oldest first, while keeping each participant’s full record intact and correctly matched — participant #62 (Millbrook, age 81) is now row 1, with every one of their other values (blood pressure, BMI, smoking status) correctly still attached to them, not shuffled independently.

Common Mistakes

- **Forgetting `desc()` for descending order.** `arrange(df, age)` sorts smallest-first by default; without wrapping the column in `desc()`, you’ll get the opposite order from what you intended for a “largest first” request.
- **Assuming `arrange()` changes the underlying data.** Like `filter()` and `select()`, `arrange()` returns a *new*, reordered data frame — the original `participants` object is completely unaffected unless you explicitly reassign the result back to it.
- **Not realizing column order matters for multi-column sorts.** `arrange(df, a, b)` and `arrange(df, b, a)` can produce different orderings — the first-listed column always takes sorting priority, with later columns only breaking ties.

69.6. Practice Problem

Use `arrange()` to sort `participants` by `bmi` in ascending order, and identify the participant with the lowest recorded BMI.

69.7. Challenge Problem

Sort `participants` by `county` (alphabetical, ascending) and then by `systolic_bp` (descending) within each county, and report the participant with the highest blood pressure in Springvale specifically.

69.8. Key Takeaways

- `arrange(df, column)` sorts rows in ascending order by default; `arrange(df, desc(column))` sorts descending.
- Sorting a data frame reorders entire rows together, keeping every column correctly matched to its original participant.
- Multiple sort columns break ties in order — the first column listed always takes priority.

Knowledge Check

1. **Multiple choice.** What does `arrange(df, desc(age))` do?
 - a) Sorts by age, youngest first
 - b) Sorts by age, oldest first
 - c) Removes the age column
 - d) Filters out missing ages
2. **True or false.** `arrange()` modifies the original data frame in place, without needing reassignment.
3. **Predict the output.** After `arrange(df, x)`, which row would have the smallest value of `x`?
4. **Write the code.** Write the code to sort `participants` by `bmi`, largest first.

Show answers

69. `arrange()`

1. **b)** Sorts by age, oldest first.
2. **False.** Like other dplyr verbs, `arrange()` returns a new data frame; the original is unchanged unless reassigned.
3. The first row.
4. `arrange(participants, desc(bmi))`

70. mutate()

The Problem

Back in Level 4, you built `pulse_pressure` and `bmi_category` with base R's `df$new_col <- ...` pattern, one assignment line per new variable. Your professor now wants three new variables added at once — pulse pressure, BMI category, and a hypertension flag — and typing three separate `participants$` assignment lines, one after another, feels repetitive for something that's really one coherent step: “add these derived variables to the dataset.”

Your Mission

Create one or more new columns in a single, unified step using `dplyr::mutate()`.

Think Before Coding

- From Level 4, what did `participants$pulse_pressure <- participants$systolic_bp - participants$diastolic_bp` do?
- If a single function could create several new columns at once, each specified by name, would you expect to need `participants$` repeated in front of every column reference inside it?

70.1. The R Concept

`mutate()` creates (or modifies) one or more columns at once, using column names directly with no `df$` prefix — and, crucially, later new columns

70. `mutate()`

within the same `mutate()` call can reference earlier ones:

```
mutate(df,  
  new_col1 = <calculation>,  
  new_col2 = <calculation using new_col1>  
)  
  
mutate(participants,  
  pulse_pressure = systolic_bp - diastolic_bp,  
  is_hypertensive = as.numeric(systolic_bp >= 130)  
)
```

This mirrors Level 4's `case_when()` and `ifelse()` exactly in spirit — `mutate()` is simply the tidyverse's version of “assign a new column,” the same way `filter()` was the tidyverse's version of logical row indexing.

70.2. Demonstration

```
library(dplyr)  
  
participants <- read.csv("data/participants.csv")  
  
participants_new <- mutate(participants,  
  pulse_pressure = systolic_bp - diastolic_bp,  
  is_hypertensive = as.numeric(systolic_bp >= 130)  
)  
  
names(participants_new)  
  
[1] "id"          "age"          "sex"          "county"  
[5] "systolic_bp" "diastolic_bp" "bmi"          "smoker"  
[9] "pulse_pressure" "is_hypertensive"
```

70.3. Explore

```
head(participants_new[, c("systolic_bp", "diastolic_bp", "pulse_pressure", "is_hypertensive")])
```

	systolic_bp	diastolic_bp	pulse_pressure	is_hypertensive
1	125	79	46	0
2	123	80	43	0
3	130	77	53	1

70.3. Explore

Try referencing a just-created column within the same `mutate()` call:

```
mutate(participants, pulse_pressure = systolic_bp - diastolic_bp, high_pp = as.numeric(pulse_pressure > 50)).
```

Confirm `high_pp` correctly uses `pulse_pressure`, even though both were created in the very same call — `mutate()` processes its arguments in order, top to bottom.

70.4. Solve the Problem

```
library(dplyr)
```

```
participants <- read.csv("data/participants.csv")
```

```
participants <- mutate(participants,
  pulse_pressure = systolic_bp - diastolic_bp,
  bmi_category = case_when(
    bmi < 18.5 ~ "Underweight",
    bmi < 25  ~ "Normal",
    bmi < 30  ~ "Overweight",
    TRUE     ~ "Obese"
  ),
  is_hypertensive = as.numeric(systolic_bp >= 130))
```

```
70. mutate()

)

table(participants$bmi_category, useNA = "always")
```

```
Normal Obese Overweight Underweight    <NA>
      28    25         39          3        5
```

All three derived variables from your professor’s request were added in a single `mutate()` call, and the `bmi_category` counts (28 Normal, 25 Obese, 39 Overweight, 3 Underweight, 5 missing) exactly match the counts you first computed with nested `ifelse()` back in Level 2 — the same result, produced more readably.

70.5. Interpret the Result

Three new derived variables — `pulse_pressure`, `bmi_category`, and `is_hypertensive` — now exist in `participants`, all added in one step whose logic is easy to read top to bottom, rather than three disconnected `participants$... <- ...` lines scattered through a script.

Common Mistakes

- **Forgetting `mutate()` returns a new data frame.** Like `filter()`, `select()`, and `arrange()`, `mutate()` doesn’t modify `participants` in place — you must reassign the result (`participants <- mutate(participants, ...)`) to keep the new columns.
- **Assuming column order within `mutate()` doesn’t matter.** It does — a later argument can reference an earlier one *within the same call* (as in the Explore step), but not the reverse; arguments are evaluated top to bottom, in the order written.

- **Recreating Level 4's `case_when()` logic from scratch instead of reusing it.** `mutate()` doesn't replace `case_when()`, `ifelse()`, or arithmetic — it's the *container* they go inside; all of Level 4's conditional tools remain exactly as useful here.

70.6. Practice Problem

Use `mutate()` to add a `bp_ratio` column (`systolic_bp / diastolic_bp`) and an `age_group` column ("Senior" for 65+, "Non-senior" otherwise, using `ifelse()`) in a single call.

70.7. Challenge Problem

Use `mutate()` to add a `risk_score` column that equals 1 for every hypertension criterion a participant meets, summed across being 65+, having BMI 30+, and being a current smoker (hint: this needs three separate `as.numeric()` comparisons added together within one `mutate()` expression). Report the maximum `risk_score` observed in the dataset.

70.8. Key Takeaways

- `mutate(df, new_col = ...)` creates or modifies one or more columns at once, using bare column names with no `df$` prefix.
- Later columns within the same `mutate()` call can reference earlier ones, evaluated top to bottom.
- Like other dplyr verbs, `mutate()` returns a new data frame — reassign the result to keep the changes.

Knowledge Check

70. `mutate()`

1. **Multiple choice.** What does `mutate(df, x2 = x * 2)` do?
 - a) Removes column `x`
 - b) Creates a new column `x2` equal to `x` times 2
 - c) Filters rows where `x` equals 2
 - d) Renames `x` to `x2`
2. **True or false.** Within a single `mutate()` call, a later column can reference one created earlier in the same call.
3. **Predict the output.** After `df <- mutate(df, y = x + 1)`, if the original `df` had no column `y`, does `df` now have one?
4. **Write the code.** Write the `mutate()` call to add a column `bmi_rounded` equal to `round(bmi, 0)`.

Show answers

1. **b)** Creates a new column `x2` equal to `x` times 2.
2. **True.** `mutate()` evaluates its arguments in order, so later ones can reference earlier ones within the same call.
3. Yes — since the result was reassigned back to `df`, the new `y` column now exists.
4. `mutate(df, bmi_rounded = round(bmi, 0))`

71. summarise()

The Problem

Your professor wants a single-line summary of the study: how many participants, and what's the average age, average systolic blood pressure, and average BMI? In base R, you'd write four separate `mean(..., na.rm = TRUE)` calls and manually assemble the results — workable, but no single function expresses “collapse this whole dataset down to one summary row.”

Your Mission

Collapse a data frame down to one or more summary statistics using `dplyr::summarise()`.

Think Before Coding

- From Level 4, what did `mean(participants$age, na.rm = TRUE)` return — a whole data frame, or a single number?
- If you wanted several such numbers (mean age, mean BP, mean BMI) side by side in one tidy row, would you rather compute and print each separately, or build them together?

71.1. The R Concept

`summarise()` collapses a data frame into a single summary row (or one row per group, as you'll see next page), using the same “no `df$` prefix” style as `mutate()`:

71. `summarise()`

```
summarise(df,  
  stat1 = <summary function>(column1),  
  stat2 = <summary function>(column2)  
)  
  
summarise(participants,  
  n_participants = n(),  
  mean_age = mean(age, na.rm = TRUE),  
  mean_bp = mean(systolic_bp, na.rm = TRUE)  
)
```

`n()` is a special `summarise()`-only helper that counts rows — it needs no argument because it always counts whatever rows are currently being summarised.

71.2. Demonstration

```
library(dplyr)  
  
participants <- read.csv("data/participants.csv")  
  
summarise(participants,  
  n_participants = n(),  
  mean_age = mean(age, na.rm = TRUE),  
  mean_bmi = mean(bmi, na.rm = TRUE)  
)  
  
  n_participants mean_age mean_bmi  
1           100 47.42553 27.12316
```

Notice the result is a data frame with exactly **one row** — `summarise()` always collapses, no matter how many rows or columns the input had.

71.3. Explore

Try adding `min_age = min(age, na.rm = TRUE)` and `max_age = max(age, na.rm = TRUE)` to the same `summarise()` call, alongside the existing statistics. Confirm all five summary values appear together in the single output row.

71.4. Solve the Problem

```
library(dplyr)

participants <- read.csv("data/participants.csv")

study_summary <- summarise(participants,
  n_participants = n(),
  mean_age       = mean(age, na.rm = TRUE),
  mean_systolic  = mean(systolic_bp, na.rm = TRUE),
  mean_bmi       = mean(bmi, na.rm = TRUE)
)

study_summary

  n_participants mean_age mean_systolic mean_bmi
1             100  47.42553      129.8438  27.12316
```

Your professor's one-line summary is now a single, tidy row: 100 participants, mean age 47.4, mean systolic blood pressure 129.8, mean BMI 27.1 — every number `na.rm = TRUE` correctly excluded missing values from, exactly as `mean()` did back in Level 2.

71. `summarise()`

71.5. Interpret the Result

`study_summary` collapses all 100 rows of `participants` into one summary row with four statistics. This is the tidyverse’s answer to the same “describe the dataset in a few numbers” task you did with individual `mean()` calls in Level 2 — now assembled together in a single readable step.

Common Mistakes

- **Forgetting `na.rm = TRUE` inside `summarise()`.** This is exactly the Level 2 trap, still fully in force — `mean(age)` returns `NA` the instant `age` contains even one missing value; `summarise()` doesn’t change this base R behavior at all.
- **Confusing `summarise()` with `mutate()`.** `mutate()` keeps the same number of rows and adds columns; `summarise()` collapses down to one row (or one row per group) — mixing them up is one of the most common early tidyverse errors.
- **Expecting `n()` to take an argument.** `n()` always counts the current rows automatically — `n(age)` is an error; just write `n()`.

71.6. Practice Problem

Use `summarise()` to compute the number of participants, the mean `diastolic_bp`, and the maximum `bmi` in a single call.

71.7. Challenge Problem

Use `summarise()` to compute what percentage of participants are current smokers (hint: `mean(smoker == "Yes") * 100` computes this directly, since `TRUE/FALSE` are treated as 1/0). Confirm your percentage matches the smoker count from Level 2’s `table(participants$smoker)` (75 No, 25 Yes).

71.8. Key Takeaways

- `summarise(df, stat = <function>(column))` collapses a data frame down to one summary row.
- `n()` is a `summarise()`-only helper that counts the current rows, with no argument needed.
- `na.rm = TRUE` is still required inside summary functions whenever a column has missing values — `summarise()` doesn't change this base R rule.

Knowledge Check

1. **Multiple choice.** What does `summarise(df, avg = mean(x, na.rm = TRUE))` return?
 - a) The original data frame, unchanged
 - b) A one-row data frame with a single column `avg`
 - c) A vector of every value of `x`
 - d) An error, since `summarise()` needs `group_by()` first
2. **True or false.** `summarise()` always returns the same number of rows as the input data frame.
3. **Predict the output.** What would `summarise(participants, total = n())` return?
4. **Find the error.**

```
summarise(participants, mean_age = mean(age))
```

Show answers

1. **b)** A one-row data frame with a single column `avg`.
2. **False.** `summarise()` collapses down to one row (or one row per group, covered next page) — never the original row count.

71. *summarise()*

3. A one-row data frame with a column `total` equal to 100 (the number of rows in `participants`).
4. `age` has missing values, so `mean(age)` without `na.rm = TRUE` returns `NA` — the same Level 2 rule still applies inside `summarise()`. It should be `mean(age, na.rm = TRUE)`.

72. group_by()

The Problem

Your professor's real question isn't "what's the average blood pressure across the whole study" — it's "how does average blood pressure differ *by county*?" `summarise()` alone collapses everything into one overall row, throwing away exactly the comparison you need. In Level 2 you might have written five separate `mean(participants$systolic_bp[participants$county == "..."], na.rm = TRUE)` calls, one per county — tedious, and easy to mistype a county name.

Your Mission

Split a data frame into groups and compute summary statistics separately for each group, using `dplyr::group_by()` together with `summarise()`.

Think Before Coding

- From Level 4, what did `tapply(participants$systolic_bp, participants$county, mean, na.rm = TRUE)` compute?
- If `summarise()` alone produces one overall row, what would you expect it to produce if the data were first split into five county-sized pieces?

72. `group_by()`

72.1. The R Concept

`group_by()` doesn't change how the data looks when printed — it tags the data frame as “grouped,” so that every subsequent `summarise()` (or `mutate()`) is computed *separately within each group* instead of across the whole dataset:

```
grouped <- group_by(df, grouping_column)
summarise(grouped, stat = <summary function>(column))

by_county <- group_by(participants, county)
summarise(by_county, mean_bp = mean(systolic_bp, na.rm = TRUE))
```

The result has **one row per group** — five rows for five counties — instead of `summarise()`'s usual single overall row.

72.2. Demonstration

```
library(dplyr)

participants <- read.csv("data/participants.csv")

by_county <- group_by(participants, county)

county_summary <- summarise(by_county,
  n_participants = n(),
  mean_bp = round(mean(systolic_bp, na.rm = TRUE), 2)
)

county_summary

# A tibble: 5 x 3
  county      n_participants mean_bp
```

	<chr>	<int>	<dbl>
1	Cedar Falls	20	128.16
2	Millbrook	21	133.30
3	Oakford	17	129.71
4	Riverdale	16	124.36
5	Springvale	26	131.46

Five rows, one per county — `n()` inside a grouped `summarise()` counts rows *within each group*, not the whole dataset.

72.3. Explore

Try grouping by two columns at once: `group_by(participants, county, smoker)`, then `summarise(..., n = n())`. Confirm the result has one row for every county-by-smoking-status combination that actually occurs in the data (not necessarily all 10 possible combinations, if some don't occur).

72.4. Solve the Problem

```
library(dplyr)

participants <- read.csv("data/participants.csv")

by_county <- group_by(participants, county)

county_summary <- summarise(by_county,
  n_participants = n(),
  mean_age = round(mean(age, na.rm = TRUE), 2),
  mean_bp = round(mean(systolic_bp, na.rm = TRUE), 2)
)
```

72. `group_by()`

```
county_summary
```

```
# A tibble: 5 x 4
  county      n_participants mean_age mean_bp
  <chr>          <int>      <dbl>   <dbl>
1 Cedar Falls      20      50.58  128.16
2 Millbrook       21      48.00  133.30
3 Oakford         17      42.20  129.71
4 Riverdale       16      51.27  124.36
5 Springvale      26      45.40  131.46
```

Millbrook has the highest mean blood pressure (133.30) of all five counties, while Riverdale has the lowest (124.36) — exactly the county-by-county comparison your professor asked for, computed in two `group_by()` + `summarise()` lines instead of five repeated subset-and-mean calls.

72.5. Interpret the Result

Splitting `participants` by `county` and summarising within each group reveals real variation base R's single overall mean (129.84 from the previous page) completely hid: Millbrook's average blood pressure (133.30) is nearly nine points higher than Riverdale's (124.36) — the kind of pattern a single ungrouped summary can never surface.

Common Mistakes

- **Forgetting `group_by()` alone doesn't summarise anything.** `group_by(participants, county)` just tags the data as grouped — printing it looks nearly identical to the ungrouped data frame. The grouping only takes visible effect once you call `summarise()` (or `mutate()`) afterward.

72.6. Practice Problem

- **Leaving a data frame grouped after you're done with it.** If you `group_by()` and then later run `mutate()` for something unrelated, the mutation will unexpectedly happen *within groups*. Use `ungroup()` afterward if you need fully ungrouped data again.
- **Forgetting `na.rm = TRUE` still applies per group.** Missing values are excluded within each group's calculation, not just once overall — the rule from Level 2 still holds, just applied five times instead of once.

72.6. Practice Problem

Use `group_by()` and `summarise()` to compute the number of participants and the mean `bmi` for each `smoker` status ("Yes" vs "No").

72.7. Challenge Problem

Use `group_by(participants, county)` followed by `summarise()` to find which county has both the highest mean BMI *and* the highest proportion of smokers. Are they the same county?

72.8. Key Takeaways

- `group_by(df, column)` tags a data frame as grouped, without changing its printed appearance by itself.
- Following `group_by()` with `summarise()` computes statistics separately within each group, producing one row per group.
- Always `ungroup()` (or start a fresh pipeline) once you're done with grouped operations, to avoid accidentally grouping later steps.

Knowledge Check

72. `group_by()`

1. **Multiple choice.** What does `group_by(df, county)` do by itself, before any `summarise()`?
 - a) Immediately computes county means
 - b) Tags the data frame as grouped by county, with no visible summary yet
 - c) Deletes all columns except `county`
 - d) Sorts the data frame by county
2. **True or false.** `summarise()` after `group_by(df, county)` produces one row per county.
3. **Predict the output.** If `participants` has 5 distinct counties, how many rows would `summarise(group_by(participants, county), n = n())` return?
4. **Find the error.**

```
grouped <- group_by(participants, county)
grouped
# expecting to already see county-level means here
```

Show answers

1. **b)** Tags the data frame as grouped by county, with no visible summary yet.
2. **True.** Grouped `summarise()` computes one row per distinct group value.
3. 5 rows — one per distinct county.
4. `group_by()` alone doesn't compute anything — it only sets up the grouping. A `summarise()` call must follow to actually see county-level statistics.

73. `count()`

The Problem

You just used `group_by()` + `summarise(n = n())` to count participants per county — three lines of code for a task as simple as “how many rows have each value of this column?” Back in Level 2, `table(participants$county)` did this in one call. Is there a tidyverse equivalent that’s just as short?

Your Mission

Count how many rows fall into each category of one or more columns, using `dplyr::count()` — the one-line shortcut for `group_by()` + `summarise(n = n())`.

Think Before Coding

- From Level 2, what did `table(participants$county)` return?
- If `count()` is a shortcut for `group_by()` + `summarise(n = n())`, would you expect its output to look like `table()`’s output, or like a `summarise()` data frame’s output?

73.1. The R Concept

`count()` counts rows per group in one step — no `group_by()` needed first:

```
count(df, column)
```

73. `count()`

```
count(participants, county)
```

This is exactly equivalent to `summarise(group_by(participants, county), n = n())` from the previous page, but in a single readable call. You can also count combinations of multiple columns at once, and sort by frequency with `sort = TRUE`:

```
count(participants, county, smoker)      # every county-by-smoker combination
count(participants, county, sort = TRUE)  # counts ordered largest first
```

73.2. Demonstration

```
library(dplyr)
```

```
participants <- read.csv("data/participants.csv")
```

```
count(participants, county)
```

```
      county  n
1 Cedar Falls 20
2 Millbrook  21
3 Oakford    17
4 Riverdale  16
5 Springvale 26
```

Note the column is automatically named `n` — matching exactly what `table(participants$county)` reported back in Level 2, just returned as a proper data frame instead of a table object.

```
count(participants, county, sort = TRUE)
```

73.3. Explore

```
      county  n
1 Springvale 26
2 Millbrook  21
3 Cedar Falls 20
4 Oakford    17
5 Riverdale  16
```

`sort = TRUE` reorders the rows largest-count-first, instantly surfacing that Springvale is the largest county in the study.

73.3. Explore

Try `count(participants, smoker)` and confirm it matches Level 2's `table(participants$smoker)` result (75 “No”, 25 “Yes”). Then try `count(participants, county, smoker)` — notice the result has one row for every county-and-smoking-status combination that actually appears, not all 10 mathematically possible combinations.

73.4. Solve the Problem

```
library(dplyr)

participants <- read.csv("data/participants.csv")

county_smoker_counts <- count(participants, county, smoker)

county_smoker_counts

      county smoker  n
1 Cedar Falls    No  17
```

73. `count()`

2	Cedar Falls	Yes	3
3	Millbrook	No	15
4	Millbrook	Yes	6
5	Oakford	No	11
6	Oakford	Yes	6
7	Riverdale	No	12
8	Riverdale	Yes	4
9	Springvale	No	20
10	Springvale	Yes	6

All 10 county-by-smoking-status combinations appear (every county has at least one smoker and one non-smoker), giving your professor exactly the cross-tabulation `table()` alone (Level 2) could only produce for a single column at a time without extra work.

73.5. Interpret the Result

`county_smoker_counts` breaks the earlier single `smoker` count (75 No, 25 Yes) down further by county — for example, Oakford has the highest proportion of smokers relative to its size (6 of 17, roughly 35%), while Cedar Falls has the lowest (3 of 20, 15%).

Common Mistakes

- **Trying to `count()` a column that doesn't exist or is misspelled.** Like every dplyr verb, `count()` uses bare column names — a typo (`count(participants, contry)`) produces a “column not found” error, not a silent empty result.
- **Forgetting `count()` already includes an implicit `group_by()`.** You don't need to call `group_by()` first — `count(participants, county)` handles the grouping and summarising in one step internally.

- **Confusing `count()`'s output column name.** The `count` column is always named `n` by default (not `count` or `freq`) — referencing `county_smoker_counts$count` will fail; it must be `county_smoker_counts$n`.

73.6. Practice Problem

Use `count()` to find how many participants fall into each combination of `sex` and `smoker`. Which combination is least common?

73.7. Challenge Problem

Use `count(participants, county, sort = TRUE)` to confirm Springvale is the largest county, then use `count(participants, county, smoker)` to determine whether Springvale also has the largest *number* of smokers (not necessarily the same as the largest proportion).

73.8. Key Takeaways

- `count(df, column)` is a one-line shortcut for `group_by()` + `summarise(n = n())`, with the result column always named `n`.
- `count()` can take multiple columns to count every combination that actually occurs in the data.
- `sort = TRUE` reorders the result largest-count-first, useful for quickly spotting the most (or least) common categories.

Knowledge Check

1. **Multiple choice.** What is `count(df, column)` equivalent to?

73. `count()`

- a) `filter(df, column)`
- b) `group_by(df, column) + summarise(n = n())`
- c) `arrange(df, column)`
- d) `select(df, column)`

2. **True or false.** The count column produced by `count()` is always named `n`.
3. **Predict the output.** What would `count(participants, smoker)` return, given 75 “No” and 25 “Yes” values?
4. **Write the code.** Write the `count()` call to count participants per combination of `county` and `sex`, sorted largest-first.

Show answers

1. **b)** `group_by(df, column) + summarise(n = n())`.
2. **True.** `count()`'s result column is always named `n` by default.
3. A two-row data frame: `smoker = "No", n = 75` and `smoker = "Yes", n = 25`.
4. `count(participants, county, sex, sort = TRUE)`

74. `distinct()`

The Problem

Before analyzing `county` any further, your professor wants a quick sanity check: exactly how many *different* counties actually appear in the dataset, and what are their names — without caring yet how many participants are in each. `count()` from the last page tells you counts, but sometimes you just want the unique *values* themselves, nothing more.

Your Mission

List the unique combinations of values in one or more columns, using `dplyr::distinct()`.

Think Before Coding

- From Level 3, what did `unique(participants$county)` return?
- If `count()` gives you unique values *plus* how many rows have each, what would you expect a function that gives you *only* the unique values (no counts) to look like?

74.1. The R Concept

`distinct()` returns the unique rows (or unique combinations of specified columns), with no count attached:

```
distinct(df, column)
```

74. `distinct()`

```
distinct(participants, county)
```

This is the tidyverse counterpart to Level 3's `unique(participants$county)` — but because `distinct()` operates on a data frame rather than a single vector, it can also find unique *combinations* across several columns at once:

```
distinct(participants, county, sex) # every county-sex combination that occurs
```

74.2. Demonstration

```
library(dplyr)
```

```
participants <- read.csv("data/participants.csv")
```

```
distinct(participants, county)
```

```
      county
1   Oakford
2 Riverdale
3 Millbrook
4 Springvale
5 Cedar Falls
```

Five distinct counties — the rows appear in the order they were first encountered in the dataset (row 1 is **Oakford** because participant #1 lives there), not alphabetically.

```
nrow(distinct(participants, county))
```

```
[1] 5
```

74.3. Explore

Try `distinct(participants, county, sex)` and check `nrow()` on the result. With 5 counties and 2 sex categories, there are at most 10 possible combinations — confirm whether all 10 actually occur in the data, or whether some combination is missing entirely.

74.4. Solve the Problem

```
library(dplyr)

participants <- read.csv("data/participants.csv")

unique_counties <- distinct(participants, county)

nrow(unique_counties)

[1] 5

unique_county_sex <- distinct(participants, county, sex)

nrow(unique_county_sex)

[1] 10
```

Five distinct counties confirmed, matching what `unique(participants$county)` reported in Level 3 and what `count(participants, county)` implied by returning 5 rows on the previous page. All 10 possible county-and-sex combinations occur at least once in the data.

74. `distinct()`

74.5. Interpret the Result

`distinct()` confirms there are exactly 5 unique counties in the study — no unexpected typos or extra category hiding in the data (recall Level 3’s messy dataset, where inconsistent capitalization made “13 distinct-looking values” collapse to only 5 real counties after cleaning). This clean dataset’s `county` column has no such problem.

Common Mistakes

- **Confusing `distinct()` with `count()`.** `distinct()` returns only the unique values themselves, with no counts attached; `count()` returns the unique values *plus* how many rows had each. Reach for `distinct()` when you only care “what values exist,” and `count()` when you also care “how many of each.”
- **Expecting `distinct()` to sort its output.** Unlike `arrange()`, `distinct()` returns unique values in the order they first appear in the data, not alphabetically or numerically — combine with `arrange()` afterward if sorted output matters.
- **Forgetting `distinct()` on a whole data frame (no columns specified) checks every column.** `distinct(participants)` with no column names looks for entire duplicate *rows* across all columns — a different (and much rarer) kind of duplicate than a single repeated `county` value.

74.6. Practice Problem

Use `distinct()` to find every unique combination of `smoker` and `sex` that occurs in the dataset, and report how many there are.

74.7. Challenge Problem

Compare `nrow(distinct(participants, county, smoker))` to the number of rows produced by last page’s `count(participants, county, smoker)`. Should these two numbers be the same? Explain why or why not.

74.8. Key Takeaways

- `distinct(df, column)` returns the unique values of a column (or unique combinations of several columns), with no counts attached.
- `distinct()` is the tidyverse counterpart to Level 3’s `unique()`, extended to work across multiple columns at once.
- Rows appear in first-encountered order, not sorted — pair with `arrange()` if sorted output is needed.

Knowledge Check

1. **Multiple choice.** How does `distinct()` differ from `count()`?
 - a) `distinct()` returns unique values with counts; `count()` doesn’t
 - b) `distinct()` returns unique values with no counts; `count()` returns unique values with counts
 - c) They are identical in every way
 - d) `distinct()` only works on numeric columns
2. **True or false.** `distinct(participants, county)` sorts the counties alphabetically.
3. **Predict the output.** If `smoker` has only two possible values (“Yes” and “No”), how many rows would `distinct(participants, smoker)` return?

74. `distinct()`

4. **Write the code.** Write the code to find every unique combination of `county` and `smoker` in `participants`.

Show answers

1. **b)** `distinct()` returns unique values with no counts; `count()` returns unique values with counts.
2. **False.** `distinct()` preserves first-encountered order, not alphabetical order.
3. 2 rows — one for "Yes", one for "No".
4. `distinct(participants, county, smoker)`

75. rename()

The Problem

Your professor asks you to rename `systolic_bp` and `diastolic_bp` to the clearer `sbp` and `dbp` before sharing the dataset with a collaborator. In Level 4's base R, you did this with `names(df)[names(df) == "old"] <- "new"` — correct, but you had to look up the column's *position* indirectly through `names()` matching, rather than just naming old and new directly.

Your Mission

Rename one or more columns directly, using `dplyr::rename()`.

Think Before Coding

- From Level 4, what did `names(participants)[names(participants) == "systolic_bp"] <- "sbp"` actually do, step by step?
- If a function let you just write “new name equals old name” directly, would you still need to go through `names()` matching at all?

75.1. The R Concept

`rename()` renames columns with a direct `new_name = old_name` mapping — no `names()` indirection required:

```
rename(df, new_name = old_name)
```

75. `rename()`

```
rename(participants, sbp = systolic_bp, dbp = diastolic_bp)
```

Read the argument order carefully: it's **new name first, old name second** — the reverse of how you might instinctively say it aloud (“re-name systolic_bp to sbp”), but consistent with every other tidyverse verb’s `new_column = <expression>` pattern (`mutate()`, `summarise()`).

75.2. Demonstration

```
library(dplyr)
```

```
participants <- read.csv("data/participants.csv")
```

```
renamed <- rename(participants, sbp = systolic_bp, dbp = diastolic_bp)
```

```
names(renamed)
```

```
[1] "id"      "age"      "sex"      "county"   "sbp"      "dbp"      "bmi"
[8] "smoker"
```

`systolic_bp` and `diastolic_bp` are gone from the column names; `sbp` and `dbp` have taken their place, in the same column positions.

75.3. Explore

Try `rename(participants, participant_id = id)` and confirm with `names()` that only `id` changed, while every other column name — including its position in the data frame — stayed exactly the same. `rename()` never reorders columns, unlike `select()`.

75.4. Solve the Problem

```
library(dplyr)

participants <- read.csv("data/participants.csv")

for_collaborator <- rename(participants,
  sbp = systolic_bp,
  dbp = diastolic_bp
)

names(for_collaborator)

[1] "id"      "age"      "sex"      "county"  "sbp"      "dbp"      "bmi"
[8] "smoker"

head(for_collaborator[, c("sbp", "dbp")], 3)

  sbp dbp
1 125  79
2 123  80
3 130  77
```

The dataset is ready to share with your professor's collaborator, using the clearer `sbp/dbp` names — with every value untouched, only the column labels changed.

75.5. Interpret the Result

`for_collaborator` has identical data to `participants`, just with two columns relabeled. This is purely cosmetic — no values changed, no rows were reordered, and `nrow()/ncol()` are identical to the original.

75. `rename()`

Common Mistakes

- **Reversing the argument order.** `rename(participants, systolic_bp = sbp)` would try to rename a column called `sbp` (which doesn't exist yet) to `systolic_bp` — the *opposite* of the intended renaming. Always read it as “new = old.”
- **Expecting `rename()` to reorder columns.** Unlike `select()`, `rename()` never changes column order or drops any columns — it only relabels the ones you specify, leaving everything else (including position) untouched.
- **Forgetting to reassign the result.** Like every other dplyr verb covered so far, `rename()` returns a new data frame; `participants` itself is unaffected unless you write `participants <- rename(participants, ...)`.

75.6. Practice Problem

Use `rename()` to rename `bmi` to `body_mass_index` and `county` to `region`, and confirm with `names()` that both changes took effect while all other column names and their order stayed the same.

75.7. Challenge Problem

Compare Level 4's base R approach (`names(df)[names(df) == "old"] <- "new"`) against `rename()` for renaming three columns at once. Which approach scales better as the number of renamed columns grows, and why?

75.8. Key Takeaways

- `rename(df, new_name = old_name)` relabels a column directly — argument order is new name first, old name second.
- `rename()` never reorders or drops columns; it only changes labels, leaving positions and all other columns untouched.
- Like other dplyr verbs, `rename()` returns a new data frame and must be reassigned to keep the change.

Knowledge Check

1. **Multiple choice.** In `rename(df, new_name = old_name)`, which comes first?
 - a) The old name
 - b) The new name
 - c) Order doesn't matter
 - d) Neither — both must be quoted strings in a vector
2. **True or false.** `rename()` can change the order of columns in a data frame.
3. **Predict the output.** After `df2 <- rename(df, patient_id = id)`, does `df` still have a column called `id`?
4. **Find the error.**

```
rename(participants, systolic_bp = sbp)
# intended to rename systolic_bp to sbp
```

Show answers

1. **b)** The new name comes first, then `=`, then the old name.
2. **False.** `rename()` only changes column labels — it never reorders or drops columns.

75. `rename()`

3. Yes — `rename()` returns a new data frame (`df2`); the original `df` is completely unaffected.
4. The argument order is reversed. Since `participants` has a column named `systolic_bp` (not `sbp`), this call tries to rename a nonexistent `sbp` column. It should be `rename(participants, sbp = systolic_bp)`.

76. across()

The Problem

Your professor wants the mean of *four* numeric columns at once — `age`, `systolic_bp`, `diastolic_bp`, and `bmi` — all with `na.rm = TRUE`. Writing `summarise(mean_age = mean(age, na.rm = TRUE), mean_sbp = mean(systolic_bp, na.rm = TRUE), ...)` works, but it's the same `mean(..., na.rm = TRUE)` pattern copy-pasted four times with only the column name changing — exactly the kind of repetition Level 9 (functions) taught you to eliminate.

Your Mission

Apply the same summary function to multiple columns at once, using `dplyr::across()` inside `summarise()` (or `mutate()`).

Think Before Coding

- If you already had four separate `mean(column, na.rm = TRUE)` expressions differing only by column name, what part of the code is truly repeated, and what part actually changes?
- Level 9 taught you to wrap repeated code in a function and call it once per input. Could a *summary column* be built the same way — one instruction, applied across many columns?

76. `across()`

76.1. The R Concept

`across()` applies one function to several columns inside `summarise()` or `mutate()`, instead of writing one summary expression per column:

```
summarise(df, across(c(col1, col2, col3), ~ mean(.x, na.rm = TRUE)))
```

```
summarise(participants, across(c(age, systolic_bp, diastolic_bp, bmi), ~ mean(.x, na.rm = TRUE)))
```

The `~` starts a short **formula-style function**, where `.x` stands in for “whichever column `across()` is currently processing” — the first time through, `.x` is `age`; the second time, `.x` is `systolic_bp`; and so on. This is the same “apply one operation across many things” idea as Level 10’s loops, just expressed as a single dplyr call rather than an explicit `for` loop.

76.2. Demonstration

```
library(dplyr)
```

```
participants <- read.csv("data/participants.csv")
```

```
summarise(participants,
  across(c(age, systolic_bp, diastolic_bp, bmi), ~ mean(.x, na.rm = TRUE))
)
```

	age	systolic_bp	diastolic_bp	bmi
1	47.42553	129.84375	80.44	27.12316

Four means, computed with a single `across()` expression — each result column keeps the original column’s name, since no renaming was requested.

76.3. Explore

Try naming the results by wrapping `across()`'s function argument in a named list: `across(c(age, bmi), list(avg = ~ mean(.x, na.rm = TRUE)))`. Notice the output columns become `age_avg` and `bmi_avg` — `across()` automatically builds column names by combining the original column name with the function's label.

76.4. Solve the Problem

```
library(dplyr)

participants <- read.csv("data/participants.csv")

numeric_summary <- summarise(participants,
  across(c(age, systolic_bp, diastolic_bp, bmi), ~ round(mean(.x, na.rm = TRUE), 2))
)

numeric_summary
```

	age	systolic_bp	diastolic_bp	bmi
1	47.43	129.84	80.44	27.12

All four means computed and rounded in one `across()` call — the same result you'd get from four separate `mean(..., na.rm = TRUE)` expressions, but without repeating the pattern four times.

76.5. Interpret the Result

The four numeric variables average out to age 47.43, systolic BP 129.84, diastolic BP 80.44, and BMI 27.12 — matching exactly the individual

76. `across()`

means computed on earlier pages, now produced by one concise instruction instead of four nearly-identical ones.

Common Mistakes

- **Forgetting the `~` before the function.** `across(cols, mean(.x, na.rm = TRUE))` tries to evaluate `mean(.x, ...)` immediately (before `.x` has any meaning) and errors; the `~` is what tells R “treat this as a function to apply later, once per column.”
- **Using `.` instead of `.x`.** Older tidyverse code sometimes used a single dot (`.`) as the placeholder; modern `across()` uses `.x` — mixing the two conventions is a common source of confusion when reading examples from different tutorial vintages.
- **Applying `across()` to non-numeric columns by accident.** `across(everything(), ~ mean(.x, na.rm = TRUE))` would try to compute a mean of `county` (text) and fail — always specify which columns are actually numeric, or use a selector like `where(is.numeric)`.

76.6. Practice Problem

Use `across()` inside `summarise()` to compute the **maximum** value (not mean) of `age`, `systolic_bp`, and `bmi` in a single call.

76.7. Challenge Problem

Use `across()` combined with `group_by(participants, county)` to compute the mean `age` and mean `bmi` for each county in a single `summarise(across(...))` call, rather than writing `mean_age = ...` and `mean_bmi = ...` as two separate lines.

76.8. Key Takeaways

- `across(c(col1, col2, ...), ~ <function>(.x, ...))` applies one function to several columns inside `summarise()` or `mutate()`.
- The `~` marks a formula-style function, where `.x` stands in for whichever column is currently being processed.
- `across()` eliminates copy-pasting the same summary expression once per column, the same repetition-avoidance principle behind Level 9's functions.

Knowledge Check

1. **Multiple choice.** What does `.x` represent inside `across(cols, ~ mean(.x, na.rm = TRUE))`?
 - a) The entire data frame
 - b) Whichever column `across()` is currently processing
 - c) The number of columns in `cols`
 - d) A fixed column named `x`
2. **True or false.** `across()` can only be used with `mean()`, never other summary functions.
3. **Predict the output.** How many result columns would `across(c(age, bmi, systolic_bp), ~ mean(.x, na.rm = TRUE))` produce inside `summarise()`?
4. **Find the error.**

```
summarise(participants, across(c(age, bmi), mean(.x, na.rm = TRUE)))
```

Show answers

1. **b)** Whichever column `across()` is currently processing.
2. **False.** `across()` works with any summary function — `sum()`, `max()`, `sd()`, or a custom function.

76. `across()`

3. 3 result columns — one for each of `age`, `bmi`, and `systolic_bp`.
4. The function is missing its leading `~`, so R tries to evaluate `mean(.x, na.rm = TRUE)` immediately, before `.x` has meaning. It should be `across(c(age, bmi), ~ mean(.x, na.rm = TRUE))`.

77. The Native Pipe |>

The Problem

Your professor asks you to filter to seniors (65+), then look at just the first 3 rows. Nesting the calls works — `head(filter(participants, age >= 65), 3)` — but notice you have to read it **inside-out**: R evaluates `filter()` first, even though it's written second, buried inside `head()`'s parentheses. As soon as you chain three or four steps together, this inside-out reading becomes genuinely hard to follow.

Your Mission

Chain multiple operations together in the order you actually think about them, using R's native pipe operator `|>`.

Think Before Coding

- In `head(filter(participants, age >= 65), 3)`, which function runs *first* — the one written first, or the one written innermost?
- If you could rewrite that same operation top-to-bottom, left-to-right, in the exact order the steps actually happen, what would that look like?

77.1. The R Concept

The **native pipe**, `|>` (built into base R since version 4.1, no package required), takes whatever is on its left and inserts it as the **first argument** of the function on its right:

77. The Native Pipe |>

```
x |> f()
# is exactly equivalent to:
f(x)
```

```
participants |> filter(age >= 65)
# is exactly equivalent to:
filter(participants, age >= 65)
```

Chained together, a pipe reads top-to-bottom in the exact order the steps happen — no more inside-out nesting:

```
participants |> filter(age >= 65) |> head(3)
```

This is identical to `head(filter(participants, age >= 65), 3)` — same result, same functions, same arguments — but now written in the order you’d describe it aloud: “take `participants`, filter to age 65+, then take the first 3.”

77.2. Demonstration

```
library(dplyr)

participants <- read.csv("data/participants.csv")

# Nested version - reads inside-out
nested_result <- head(filter(participants, age >= 65), 3)

# Piped version - reads top-to-bottom
piped_result <- participants |> filter(age >= 65) |> head(3)

identical(nested_result, piped_result)

[1] TRUE
```

77.3. Explore

```
pipled_result
```

	id	age	sex	county	systolic_bp	diastolic_bp	bmi	smoker
1	17	79	Female	Springvale	117	73	21.1	Yes
2	19	80	Female	Cedar Falls	109	79	28.4	No
3	31	70	Male	Oakford	145	85	NA	No

77.3. Explore

Try adding a third step to the pipeline: `participants |> filter(age >= 65) |> arrange(desc(systolic_bp)) |> head(3)`. Read it aloud as you look at it — “take participants, filter to seniors, sort by blood pressure descending, take the top 3” — and notice the code reads in that exact same order.

77.4. Solve the Problem

```
library(dplyr)
```

```
participants <- read.csv("data/participants.csv")
```

```
at_risk_seniors <- participants |>  
  filter(age >= 65 & systolic_bp >= 130) |>  
  arrange(desc(systolic_bp))
```

```
at_risk_seniors
```

	id	age	sex	county	systolic_bp	diastolic_bp	bmi	smoker
1	62	81	Male	Millbrook	153	95	30.3	Yes
2	31	70	Male	Oakford	145	85	NA	No

77. The Native Pipe |>

3	84	67	Female	Riverdale	134	79	26.8	No
4	51	66	Male	Cedar Falls	133	77	34.5	No

Filtering and sorting are now two clearly ordered steps in one pipeline — read top to bottom, this code says exactly what it does: “take `participants`, filter to at-risk seniors, sort by blood pressure with the highest first.”

77.5. Interpret the Result

The same 4 at-risk seniors identified back on the `filter()` page are now sorted by systolic blood pressure, highest first — participant #62 (age 81, BP 153) stands out as the most urgent case, followed by #31, #84, and #51. The pipe made the two-step process (“filter, then sort”) visibly sequential rather than nested inside one another.

Common Mistakes

- **Confusing the native pipe `|>` with the older `magrittr` pipe `%>%`.** You may see `%>%` in older tidyverse tutorials and Stack Overflow answers — it behaves almost identically, but `|>` is built into base R itself (no `library()` call needed) and is now the standard choice for new code.
- **Forgetting the pipe inserts as the *first* argument only.** `participants |> filter(age >= 65)` becomes `filter(participants, age >= 65)` — `participants` always lands in the first argument slot. If a function’s data argument isn’t first, the plain pipe can’t simply slot it in.
- **Piping into something that isn’t a function call.** `participants |> age` is an error — the right-hand side of `|>` must be a function call like `filter(...)` or `head(...)`, not a bare column reference.

77.6. Practice Problem

Rewrite `arrange(select(participants, id, age, bmi), desc(bmi))` as a piped chain using `|>`, and confirm with `identical()` that both versions produce the same result.

77.7. Challenge Problem

Write a three-step pipeline that starts from `participants`, filters to non-smokers (`smoker == "No"`), sorts by `age` ascending, and keeps only the `id`, `age`, and `county` columns — using `|>` to chain `filter()`, `arrange()`, and `select()` in that order.

77.8. Key Takeaways

- `x |> f()` is exactly equivalent to `f(x)` — the pipe inserts its left-hand side as the function's first argument.
- Chained pipes read top-to-bottom, in the actual order operations happen — the opposite of nested function calls, which read inside-out.
- The native pipe `|>` is built into base R (since R 4.1) and requires no package; you may also encounter the older, package-based `%>%` pipe in existing code.

Knowledge Check

1. **Multiple choice.** What is `participants |> filter(age >= 65)` equivalent to?
 - a) `filter(age >= 65, participants)`
 - b) `filter(participants, age >= 65)`

77. *The Native Pipe |>*

- c) `age >= 65 |> filter(participants)`
 - d) None of these — the pipe changes the result
2. **True or false.** The native pipe `|>` requires `library(magrittr)` or `library(dplyr)` to work.
 3. **Predict the output.** What would `participants |> nrow()` return?
 4. **Find the error.**

```
participants |> age
```

Show answers

1. **b)** `filter(participants, age >= 65)` — the pipe's left side becomes the first argument.
2. **False.** The native pipe `|>` is built into base R (since version 4.1) and needs no package.
3. 100 — the number of rows in `participants`, since `nrow()` takes a data frame and `participants` is piped in as that argument.
4. The right-hand side of `|>` must be a function call, not a bare column reference. `age` alone isn't a function call, so this errors; you'd need something like `participants |> pull(age)` (or simply `participants$age`) instead.

78. Chaining Operations

The Problem

Your professor’s final request for this level: “Among participants aged 50 and older, which county has the highest average blood pressure?” This single question actually requires *four* steps — import the data, filter to age 50+, group by county, and compute + sort mean blood pressure. Doing this in base R (Level 4) would mean four or five separate intermediate objects (`filtered <- ...`, `grouped_means <- tapply(...)`, `sorted <- sort(...)`) cluttering your environment. This is exactly the moment every tool from this level comes together.

Your Mission

Combine `filter()`, `group_by()`, `summarise()`, and `arrange()` into a single chained pipeline using `|>`, answering a real research question in one readable block of code.

Think Before Coding

- In what order would you need to perform these four steps: sort by blood pressure, group by county, filter to age 50+, compute the mean? (Hint: you can’t group data that hasn’t been filtered yet, and you can’t sort a summary that hasn’t been computed yet.)
- Once you know the right order, could you write all four steps as one pipeline using `|>`, the way you did on the previous page?

78. Chaining Operations

78.1. The R Concept

A full analytical pipeline chains every relevant verb together, each one passing its result to the next:

```
result <- df |>
  filter(condition) |>
  group_by(grouping_column) |>
  summarise(stat = <function>(column, na.rm = TRUE)) |>
  arrange(desc(stat))
```

Each step in this chain does exactly one job — `filter` narrows the rows, `group_by()` sets up per-group processing, `summarise()` collapses each group to one row, and `arrange()` orders the final result — and the pipe (`|>`) makes the whole sequence read top-to-bottom, in the order the steps actually happen.

78.2. Demonstration

```
library(dplyr)

participants <- read.csv("data/participants.csv")

older_participants_summary <- participants |>
  filter(age >= 50) |>
  group_by(county) |>
  summarise(
    n = n(),
    mean_bp = round(mean(systolic_bp, na.rm = TRUE), 2)
  ) |>
  arrange(desc(mean_bp))

older_participants_summary
```

```
# A tibble: 5 x 3
  county      n mean_bp
  <chr>    <int>   <dbl>
1 Millbrook     9  130.38
2 Oakford       3  130.00
3 Springvale   13  125.77
4 Cedar Falls  11  125.40
5 Riverdale     9  122.00
```

One pipeline, four chained steps, answering the exact question asked — no intermediate objects needed along the way.

78.3. Explore

Try removing the final `arrange(desc(mean_bp))` step and re-running the pipeline. Notice the same five rows appear, just in whatever order `group_by()` happened to encounter the counties — confirming that `arrange()` only affects *display order*, not which rows or values are present.

78.4. Solve the Problem

```
library(dplyr)

participants <- read.csv("data/participants.csv")

bp_by_county_50plus <- participants |>
  filter(age >= 50) |>
  group_by(county) |>
  summarise(
    n = n(),
```

78. Chaining Operations

```
mean_bp = round(mean(systolic_bp, na.rm = TRUE), 2)
) |>
  arrange(desc(mean_bp))

bp_by_county_50plus
```

```
# A tibble: 5 x 3
  county      n mean_bp
  <chr>   <int>   <dbl>
1 Millbrook     9  130.38
2 Oakford       3  130.00
3 Springvale   13  125.77
4 Cedar Falls  11  125.40
5 Riverdale     9  122.00
```

Among participants aged 50 and older, **Millbrook** has the highest average systolic blood pressure (130.38, across 9 participants), narrowly ahead of Oakford (130.00, but only 3 participants). Riverdale has the lowest (122.00) — your professor’s exact question, answered in one pipeline.

78.5. Interpret the Result

This single chained pipeline — filter, group, summarise, arrange — replaced what would otherwise be four or five separate base R steps with intermediate objects at each stage. Notice too that Oakford’s second-place mean (130.00) rests on only 3 participants, a much shakier estimate than Millbrook’s 9 — a reminder from Level 11 (statistics) that group size matters when interpreting a mean, not just the mean itself.

Common Mistakes

78.6. Practice Problem

- **Grouping before filtering.** `group_by()` should come *after* `filter()` in most pipelines — you generally want to narrow the rows first, then split what remains into groups, not the reverse.
- **Forgetting `arrange()` only changes the final display order.** Placing `arrange()` in the middle of a pipeline (before `summarise()`, for instance) sorts rows that will just get collapsed a step later, having no effect on the final grouped result — `arrange()` belongs at the very end unless you have a specific reason otherwise.
- **Overlooking small group sizes when interpreting grouped summaries.** A mean based on 3 participants (Oakford) and a mean based on 9 (Millbrook) aren't equally trustworthy, even though `summarise()` reports them with the same apparent precision.

78.6. Practice Problem

Write a chained pipeline that starts from `participants`, filters to current smokers (`smoker == "Yes"`), groups by `county`, computes the mean `bmi` and count per county, and sorts by mean `bmi` descending. Which county has the highest mean BMI among smokers?

78.7. Challenge Problem

Write a pipeline that filters to participants with `bmi >= 30` (obese, from Level 4's `case_when()` categories), groups by `county` and `smoker`, computes the count for each county-and-smoking-status combination, and sorts by count descending. Which single county-and-smoking-status combination has the most obese participants?

78.8. Key Takeaways

- A full pipeline chains `filter()` → `group_by()` → `summarise()` → `arrange()` (in that order, generally) using `|>`, replacing multiple intermediate base R objects with one readable block.
- `arrange()` belongs at the end of a pipeline — it only affects display order and has no effect if placed before a `summarise()` that will collapse the rows anyway.
- Small group sizes can produce misleadingly precise-looking summary statistics — always check `n` alongside any grouped mean before drawing conclusions.

Knowledge Check

1. **Multiple choice.** In a pipeline `df |> filter(...) |> group_by(...) |> summarise(...) |> arrange(...)`, what does `arrange()` do?
 - a) Filters rows before grouping
 - b) Changes the final display order of the summarised result
 - c) Groups the data a second time
 - d) Removes duplicate rows
2. **True or false.** Placing `arrange()` in the middle of a pipeline, right before `summarise()`, changes the final grouped-and-summarised result.
3. **Predict the output.** In `bp_by_county_50plus`, which county has the second-highest mean blood pressure, and how many participants does that count rely on?
4. **Find the error.**

```
participants |>
  group_by(county) |>
```

78.8. Key Takeaways

```
filter(age >= 50) |>
  summarise(mean_bp = mean(systolic_bp, na.rm = TRUE))
```

Show answers

1. **b)** Changes the final display order of the summarised result.
2. **False.** `arrange()` before `summarise()` sorts rows that are about to be collapsed anyway — it has no effect on the final grouped result. Only `arrange()` placed *after* `summarise()` affects the final output's order.
3. Oakford, with mean blood pressure 130.00 — but that mean rests on only 3 participants, far fewer than Millbrook's 9.
4. This isn't strictly an error — `group_by()` before `filter()` still works, since `filter()` operates fine on grouped data — but it's misleading style: readers expect a pipeline to narrow rows *before* grouping them, and grouping first can silently change behavior in more complex pipelines involving group-specific filters (e.g. `filter(age == max(age))`), which behaves differently before vs. after grouping).

Part VI.

Level 6: Data Reshaping

Level 6 — Data Reshaping

Where this level is headed

Health data rarely arrives in the shape you need for your specific question. A longitudinal study might store each visit as a new row or a new column depending on who exported it, and two files collected by different teams a year apart need to be combined without losing or duplicating a single participant. Reshaping isn't just about `pivot_longer()` and `pivot_wider()` — it's about understanding what links your datasets together in the first place.

What you'll build toward

You'll start with a repeated-measures dataset — blood pressure and weight recorded across three clinic visits for 28 participants, arriving one row per participant with a column per visit (“wide” format). You'll reshape it to one row per participant-visit combination (“long” format) for tidy analysis, split and combine compound columns, and reshape multiple measurements at once. Then you'll shift to a second, equally common real-world problem: combining two *separate* files — a baseline enrollment file and a one-year follow-up file — using every major type of `dplyr` join, and finish with a capstone that reshapes and joins in a single pipeline.

Topics in this level

- Wide format vs. long format — what each looks like and when each is useful
- `pivot_longer()` and `pivot_wider()`
- `separate()` and `unite()` — splitting and combining columns
- Reshaping multiple measurements at once with `.value`
- Keys, and one-to-one vs. one-to-many relationships
- `left_join()`, `inner_join()`, `right_join()`, `full_join()`
- `semi_join()` and `anti_join()`
- A combined reshaping-and-joining capstone

A preview mission

Your mission across this level: reshape a three-visit blood pressure dataset to compute how much each participant's blood pressure changed over time, then merge a baseline enrollment file with a follow-up visit file — collected a year apart, by different teams — into one analysis-ready dataset without losing or duplicating a single participant.

79. Wide Format vs. Long Format

The Problem

Your professor gives you `data/bp_visits_wide.csv`: 28 participants, one row each, with separate columns `sbp_visit1`, `sbp_visit2`, `sbp_visit3` for blood pressure at three clinic visits. You're asked to plot how blood pressure changes over time, using `visit` as one axis. But there's no single `visit` column to plot against — the visit information is baked into the *column names themselves*, not into any column's values.

Your Mission

Recognize the difference between wide format and long format data, and understand why some tasks require one shape or the other.

Think Before Coding

- In `sbp_visit1`, `sbp_visit2`, `sbp_visit3`, is “which visit” stored as a *value* somewhere, or as part of a *column name*?
- If you wanted to `group_by(visit)` (Level 5) and compute a mean blood pressure per visit, could you do that if “visit” isn't an actual column containing values like 1, 2, 3?

79.1. The R Concept

Wide format stores one row per observational unit (here, one row per participant), with separate columns for each repeated measurement:

79. Wide Format vs. Long Format

id	sbp_visit1	sbp_visit2	sbp_visit3
1	125	120	117
2	123	121	129

Long format stores one row per *combination* of observational unit and measurement occasion, with the measurement occasion itself living in a column:

id	visit	sbp
1	visit1	125
1	visit2	120
1	visit3	117
2	visit1	123
2	visit2	121
2	visit3	129

Both represent the *exact same information* — no data is gained or lost — but long format has an actual `visit` column you can `filter()`, `group_by()`, or plot against, while wide format encodes that same information into column names that dplyr functions can't directly reference as values.

79.2. Demonstration

```
bp_wide <- read.csv("data/bp_visits_wide.csv")
```

```
names(bp_wide)
```

```
[1] "id"          "county"      "clinic_site" "sbp_visit1"
[5] "sbp_visit2"  "sbp_visit3"  "weight_visit1" "weight_visit2"
[9] "weight_visit3"
```

```
nrow(bp_wide)
```

```
[1] 28
```

28 rows — one per participant — with visit information spread across six separate measurement columns (three for `sbp`, three for `weight`).

79.3. Explore

Try `mean(bp_wide$sbp_visit1, na.rm = TRUE)`, then `mean(bp_wide$sbp_visit2, na.rm = TRUE)`, then `mean(bp_wide$sbp_visit3, na.rm = TRUE)` — three separate lines, one per visit, because there’s no single `sbp` column to `group_by(visit)` on. This repetition is exactly the signal that a reshape is needed.

79.4. Solve the Problem

```
bp_wide <- read.csv("data/bp_visits_wide.csv")
```

```
# Wide format: three repeated columns, one row per participant
head(bp_wide[, c("id", "sbp_visit1", "sbp_visit2", "sbp_visit3")], 3)
```

	id	sbp_visit1	sbp_visit2	sbp_visit3
1	1	125	120	117
2	2	123	121	129
3	3	130	128	124

There is no way to `group_by()` a “visit” here, because “visit” isn’t a column of values — it’s spread across three column *names*. The next page’s `pivot_longer()` is exactly the tool that fixes this.

79.5. Interpret the Result

`bp_wide` is a perfectly valid, common way to store repeated-measures data — and it’s often the *easier* format for a human to read at a glance, comparing one participant’s three visits side by side. But it’s the wrong shape for `group_by(visit)`-style analysis or for plotting change over time, which is why reshaping tools exist at all: not to “fix” wide format, but to convert between the shape that’s easy to *read* and the shape that’s easy to *analyze*.

Common Mistakes

- **Assuming one format is objectively “correct.”** Neither wide nor long is inherently better — wide is often better for reading and data entry; long is often better for `group_by()`-style analysis and plotting. The right shape depends on the task.
- **Trying to `group_by()` a value that’s actually a column name.** `group_by(bp_wide, visit)` fails outright — there is no `visit` column in wide format; the visit information lives in the column *names* `sbp_visit1/sbp_visit2/sbp_visit3`, not in any column’s values.
- **Forgetting reshaping doesn’t change the underlying data.** Converting wide to long (or back) doesn’t add, remove, or recalculate anything — it only changes which pieces of information are stored as column names versus column values.

79.6. Practice Problem

Look at `bp_wide`’s `weight_visit1`, `weight_visit2`, `weight_visit3` columns. Without writing any reshaping code yet, describe in words what the long-format equivalent would look like: what would the column names be, and how many rows would it have?

79.7. Challenge Problem

If `bp_wide` has 28 rows and 3 visits' worth of `sbp` columns, how many rows would a long-format version (one row per participant-visit combination, `sbp` only) have? What if you also included the `weight` measurements in the same long format?

79.8. Key Takeaways

- Wide format has one row per unit, with repeated measurements spread across multiple columns.
- Long format has one row per unit-and-occasion combination, with the occasion itself stored as an actual column of values.
- Both formats hold identical information — the right choice depends on whether you're reading data by eye or feeding it into `group_by()`-style analysis or plotting.

Knowledge Check

1. **Multiple choice.** In wide format, how is repeated measurement information (like “visit number”) typically stored?
 - a) In an actual column of values
 - b) Encoded in the column names
 - c) It cannot be stored in wide format at all
 - d) As row names
2. **True or false.** Converting data from wide to long format changes the underlying values.
3. **Predict the output.** If `bp_wide` has 28 rows and 3 visit columns for `sbp`, how many rows would the fully reshaped long format (`sbp` only) have?

79. Wide Format vs. Long Format

4. **Short answer.** Why can't you `group_by(bp_wide, visit)` directly on the wide-format data?

Show answers

1. **b)** Encoded in the column names (e.g., `sbp_visit1`, `sbp_visit2`).
2. **False.** Reshaping only changes the *shape* of the data (which information is a column name vs. a column value) — no values are added, removed, or recalculated.
3. 84 rows (28 participants $\times$ 3 visits).
4. There is no `visit` column in the wide-format data — the visit information is embedded in the column names (`sbp_visit1`, etc.), not stored as values `group_by()` can reference.

80. pivot_longer()

The Problem

You need an actual `visit` column so you can `group_by(visit)` (Level 5) and compute the mean blood pressure at each clinic visit. Right now, “visit” is smeared across three column names (`sbp_visit1`, `sbp_visit2`, `sbp_visit3`) in your wide-format data — there’s no way to `group_by()` a value that doesn’t exist as a value yet.

Your Mission

Convert wide-format columns into long-format rows using `tidyr::pivot_longer()`.

Think Before Coding

- From the previous page, what information is currently trapped inside the column names `sbp_visit1`, `sbp_visit2`, `sbp_visit3`?
- If a function could take several columns and “stack” them into two new columns — one holding what the old column names were, one holding what the old column values were — what would you want those two new columns to be called for this dataset?

80.1. The R Concept

`pivot_longer()`, from the `tidyr` package (part of the tidyverse, alongside `dplyr`), takes multiple columns and stacks them into two new columns: one holding the *former column names*, one holding the *former column values*:

80. `pivot_longer()`

```
pivot_longer(df,
  cols = c(col1, col2, col3), # or a tidyselect helper like starts_with()
  names_to = "name_column",
  values_to = "value_column"
)

pivot_longer(bp_wide,
  cols = starts_with("sbp_visit"),
  names_to = "visit",
  values_to = "sbp"
)
```

`cols` accepts the same `starts_with()` / `ends_with()` / `contains()` helpers from Level 5's `select()` — genuinely the same tidyselect syntax, reused here for a different purpose. Adding `names_prefix = "sbp_"` strips a shared prefix from the resulting names column, turning "sbp_visit1" into the cleaner "visit1".

80.2. Demonstration

```
library(tidyr)

bp_wide <- read.csv("data/bp_visits_wide.csv")

bp_long_sbp <- pivot_longer(bp_wide,
  cols = starts_with("sbp_visit"),
  names_to = "visit",
  names_prefix = "sbp_",
  values_to = "sbp"
)

nrow(bp_long_sbp)
```

```
[1] 84
```

28 participants $\times$ 3 visits = 84 rows, exactly as predicted at the end of the previous page.

```
head(bp_long_sbp[, c("id", "county", "visit", "sbp")], 6)
```

```
   id  county visit sbp
1  1  Oakford visit1 125
2  1  Oakford visit2 120
3  1  Oakford visit3 117
4  2 Riverdale visit1 123
5  2 Riverdale visit2 121
6  2 Riverdale visit3 129
```

Participant #1's three visits are now three separate rows, with an actual `visit` column you could `group_by()` — exactly what wide format couldn't offer.

80.3. Explore

Try `group_by(bp_long_sbp, visit)` followed by `summarise(mean_sbp = mean(sbp, na.rm = TRUE))` (Level 5). Confirm you can now compute a mean blood pressure *per visit* — something that was impossible without first reshaping.

80.4. Solve the Problem

```
library(dplyr)
library(tidyr)
```

80. `pivot_longer()`

```
bp_wide <- read.csv("data/bp_visits_wide.csv")

bp_long_sbp <- pivot_longer(bp_wide,
  cols = starts_with("sbp_visit"),
  names_to = "visit",
  names_prefix = "sbp_",
  values_to = "sbp"
)

visit_means <- bp_long_sbp |>
  group_by(visit) |>
  summarise(mean_sbp = round(mean(sbp, na.rm = TRUE), 2))

visit_means

# A tibble: 3 x 2
  visit mean_sbp
  <chr>   <dbl>
1 visit1  124.96
2 visit2  120.96
3 visit3  120.69
```

Now that “visit” is a real column, `group_by(visit)` works exactly as it did for `county` in Level 5 — mean systolic blood pressure drops from 124.96 at visit 1 to 120.96 at visit 2, then levels off slightly at 120.69 by visit 3.

80.5. Interpret the Result

Blood pressure dropped noticeably between visit 1 and visit 2 on average, with a much smaller further change by visit 3 — a pattern impossible

80.6. Practice Problem

to see from the wide-format data without either writing three separate `mean()` calls or reshaping first. Note also that `visit3`'s mean is based on fewer participants (26, since 2 dropped out — visible if you check `n()` per group), a detail worth flagging before drawing conclusions, echoing Level 5's small-group-size caution.

Common Mistakes

- **Forgetting `cols` needs to select only the columns you want stacked.** If `bp_wide` also has `weight_visit1/weight_visit2/weight_visit3` columns and you only specify `cols = starts_with("sbp_visit")`, those weight columns stay wide — and get **duplicated across every new long row** for a participant, since each of their 3 sbp rows now repeats the same weight values. This isn't wrong, just worth noticing (the next multiple-value page shows how to pivot both measurements together, avoiding the duplication).
- **Mixing up `names_to` and `values_to`.** `names_to` names the column that will hold the *former column names* (like "visit1"); `values_to` names the column that will hold the *former column values* (like 125). Swapping them produces a working but confusingly labeled result.
- **Forgetting `library(tidyr)`.** `pivot_longer()` lives in the `tidyr` package, not `dplyr` — both are part of the tidyverse, but they're technically separate packages.

80.6. Practice Problem

Use `pivot_longer()` to reshape only the `weight_visit1/weight_visit2/weight_visit3` columns into long format, with `names_to = "visit"` and `values_to = "weight"`. Confirm the result has 84 rows.

80. `pivot_longer()`

80.7. Challenge Problem

After reshaping `sbp` to long format, use `group_by(visit)` and `summarise()` to compute both the mean **and** the count (`n()`) of non-missing `sbp` values per visit. Which visit has the smallest sample size, and why might that be (hint: recall the dropout pattern from `data/bp_visits_wide.csv`)?

80.8. Key Takeaways

- `pivot_longer(df, cols = ..., names_to = "...", values_to = "...")` stacks multiple wide columns into two long columns: one for former names, one for former values.
- `cols` accepts the same tidyselect helpers (`starts_with()`, etc.) as Level 5's `select()`.
- Reshaping to long format is often the necessary first step before `group_by()`-based analysis across a repeated measurement like “visit.”

Knowledge Check

1. **Multiple choice.** What does `names_to` specify in `pivot_longer()`?
 - a) The name of the column holding former column values
 - b) The name of the column holding former column names
 - c) Which columns to pivot
 - d) The final row count
2. **True or false.** `pivot_longer()`'s `cols` argument can use `starts_with()` and other tidyselect helpers from Level 5.
3. **Predict the output.** If a wide data frame has 10 rows and 4 columns matching a `pivot_longer()` call's `cols`, how many rows will the long-format result have?

4. Find the error.

```

pivot_longer(bp_wide, cols = starts_with("sbp_visit"), names_to = "sbp", values_to = "v

```

Show answers

1. **b)** The name of the column holding former column names.
2. **True.** `pivot_longer()`'s `cols` argument uses the same tidyselect syntax as `select()`.
3. 40 rows (10 rows $\times$ 4 pivoted columns).
4. `names_to` and `values_to` are swapped — this labels the *column names* ("sbp_visit1", etc.) as `sbp` and the *blood pressure values* (125, etc.) as `visit`, exactly backwards. It should be `names_to = "visit", values_to = "sbp"`.

81. `pivot_wider()`

The Problem

You've reshaped to long format and computed mean blood pressure per county per visit with `group_by(county, visit) + summarise()` — but the result is 15 rows (5 counties $\times$ 3 visits) stacked vertically, which is awkward for a report table. Your professor wants a compact table with one row per county and one column per visit, so all three visits can be compared side by side at a glance.

Your Mission

Convert long-format data back into wide format using `tidyr::pivot_wider()` — the reverse of `pivot_longer()`.

Think Before Coding

- If `pivot_longer()` takes several columns and stacks them into two, what would you expect the *reverse* operation to do with those same two columns?
- Looking at a long-format table with a `visit` column and a `mean_sbp` column, which one should become new column *names*, and which one should fill in the new column *values*?

81.1. The R Concept

`pivot_wider()` takes one column of names and one column of values, and spreads them back out into separate columns:

81. `pivot_wider()`

```
pivot_wider(df,
  names_from = name_column,
  values_from = value_column
)

pivot_wider(county_visit_means,
  names_from = visit,
  values_from = mean_sbp
)
```

This is exactly the reverse of `pivot_longer()`'s `names_to/values_to`: `names_from` says which column's *values* should become new column *names*, and `values_from` says which column's values should fill in those new columns.

81.2. Demonstration

```
library(dplyr)
library(tidyr)

bp_wide <- read.csv("data/bp_visits_wide.csv")

bp_long_sbp <- pivot_longer(bp_wide,
  cols = starts_with("sbp_visit"),
  names_to = "visit",
  names_prefix = "sbp_",
  values_to = "sbp"
)

county_visit_means <- bp_long_sbp |>
  group_by(county, visit) |>
  summarise(mean_sbp = round(mean(sbp, na.rm = TRUE), 2), .groups = "drop")
```

81.2. Demonstration

```
county_visit_means
```

```
# A tibble: 15 x 3
  county      visit mean_sbp
  <chr>      <chr>    <dbl>
1 Cedar Falls visit1    136.00
2 Cedar Falls visit2    130.25
3 Cedar Falls visit3    128.00
4 Millbrook  visit1    122.17
5 Millbrook  visit2    120.33
6 Millbrook  visit3    122.00
7 Oakford    visit1    120.67
8 Oakford    visit2    117.17
9 Oakford    visit3    119.50
10 Riverdale visit1    123.33
11 Riverdale visit2    119.00
12 Riverdale visit3    120.67
13 Springvale visit1    126.33
14 Springvale visit2    121.17
15 Springvale visit3    116.17
```

15 rows, exactly as `group_by(county, visit)` would produce (5 counties $\times$ 3 visits) — readable, but each county’s three visits are spread across three separate rows.

```
report_table <- pivot_wider(county_visit_means,
  names_from = visit,
  values_from = mean_sbp
)
```

```
report_table
```

```
81. pivot_wider()
```

```
# A tibble: 5 x 4
  county      visit1 visit2 visit3
  <chr>      <dbl>  <dbl>  <dbl>
1 Cedar Falls 136.00  130.25 128.00
2 Millbrook  122.17  120.33 122.00
3 Oakford    120.67  117.17 119.50
4 Riverdale  123.33  119.00 120.67
5 Springvale 126.33  121.17 116.17
```

Now 5 rows (one per county) and 3 visit columns — exactly the compact report table your professor asked for.

81.3. Explore

Try `pivot_wider()` on `bp_long_sbp` directly (not the summarised version), using `names_from = visit`, `values_from = sbp`. Confirm this reconstructs something very close to the original `bp_wide` data frame — `pivot_wider()` is the true inverse of `pivot_longer()`, so reshaping long-to-wide and back again returns you to (almost) where you started.

81.4. Solve the Problem

```
library(dplyr)
library(tidyr)

bp_wide <- read.csv("data/bp_visits_wide.csv")

bp_long_sbp <- pivot_longer(bp_wide,
  cols = starts_with("sbp_visit"),
  names_to = "visit",
  names_prefix = "sbp_",
```

81.5. Interpret the Result

```
  values_to = "sbp"
)

county_visit_report <- bp_long_sbp |>
  group_by(county, visit) |>
  summarise(mean_sbp = round(mean(sbp, na.rm = TRUE), 2), .groups = "drop") |>
  pivot_wider(names_from = visit, values_from = mean_sbp)

county_visit_report

# A tibble: 5 x 4
  county      visit1 visit2 visit3
  <chr>      <dbl>  <dbl>  <dbl>
1 Cedar Falls 136.00  130.25 128.00
2 Millbrook  122.17  120.33 122.00
3 Oakford    120.67  117.17 119.50
4 Riverdale  123.33  119.00 120.67
5 Springvale  126.33  121.17 116.17
```

Cedar Falls starts with the highest mean blood pressure (136.00) and shows the clearest downward trend across visits (136.00 → 130.25 → 128.00), while Springvale shows the largest overall drop from visit 1 to visit 3 (126.33 → 116.17).

81.5. Interpret the Result

`county_visit_report` compresses 15 long-format rows into 5 wide-format rows, one per county, with visits laid out side by side — exactly the shape a reader scanning a report wants, even though `group_by()` + `summarise()` (Level 5) only knows how to produce the longer, stacked version. Note that Cedar Falls has only 4 participants in this subset (versus 6 in each other

81. `pivot_wider()`

county), so its more dramatic-looking trend rests on a notably smaller sample.

Common Mistakes

- **Forgetting `pivot_wider()` needs unique name-value combinations.** If `county_visit_means` had two rows with the same county and visit (a duplicate), `pivot_wider()` would need to combine them into a list-column rather than a single value — a common source of confusing “list of lists” output for beginners.
- **Confusing `names_from` and `values_from`.** `names_from` picks which column’s *values* become the new column headers; `values_from` picks which column’s values fill in underneath them. Swapping them produces an error or nonsensical column names.
- **Forgetting small group sizes remain small after reshaping.** Cedar Falls’ `n=4` doesn’t become more reliable just because its numbers are now displayed in a wide table instead of a long one — reshaping changes layout, not sample size.

81.6. Practice Problem

Use `group_by(county, visit)` and `summarise()` to compute the mean weight per county per visit (after first pivoting the weight columns to long format), then use `pivot_wider()` to build a county-by-visit weight report table.

81.7. Challenge Problem

Starting from `bp_long_sbp`, use `pivot_wider(names_from = visit, values_from = sbp)` directly (no `group_by()/summarise()` first) to reconstruct a wide table. Compare its columns to the original

`bp_wide`'s `sbp_visit1/sbp_visit2/sbp_visit3` columns — are the values identical?

81.8. Key Takeaways

- `pivot_wider(df, names_from = ..., values_from = ...)` is the reverse of `pivot_longer()`, spreading long rows back into wide columns.
- `names_from` selects which column's values become new column names; `values_from` selects which column fills them in.
- `pivot_wider()` requires unique name-value combinations per row group — duplicates produce list-columns instead of single values.

Knowledge Check

1. **Multiple choice.** What does `names_from` specify in `pivot_wider()`?
 - a) Which column's values become new column names
 - b) Which column holds the final row count
 - c) Which column to group by
 - d) Which column's values fill in the new columns
2. **True or false.** `pivot_wider()` is the reverse operation of `pivot_longer()`.
3. **Predict the output.** If a long-format table has 5 counties and 3 visits (15 rows total), how many rows would `pivot_wider(names_from = visit, values_from = mean_sbp)` produce?
4. **Find the error.**

```
pivot_wider(county_visit_means, names_from = mean_sbp, values_from = visit)
```

Show answers

81. `pivot_wider()`

1. **a)** Which column's values become new column names.
2. **True.** `pivot_wider()` reverses what `pivot_longer()` does, spreading long rows back into wide columns.
3. 5 rows — one per county, since each county's 3 visit-rows collapse into 1 wide row with 3 visit columns.
4. `names_from` and `values_from` are swapped — this would try to use the numeric `mean_sbp` values as column *names* and the text `visit` values as the data filling them in, which is backwards. It should be `names_from = visit, values_from = mean_sbp`.

82. `separate()`

The Problem

`bp_wide` has a `clinic_site` column like "Oakford-North", "Riverdale-South" — a county name and a clinic direction, joined together with a hyphen. Your professor wants to analyze clinic directions on their own (are North-side clinics recording different blood pressures than South-side ones?), but right now “county” and “direction” are stuck together as one compound piece of text in a single column.

Your Mission

Split one column into two or more columns using `tidyr::separate()`.

Think Before Coding

- Looking at "Oakford-North", what character separates the two pieces of information packed into this single value?
- From Level 4, `case_when()` and `ifelse()` create new columns from existing ones. Would you expect a “split one column into two” function to also require you to describe both what to split by and what to name the new pieces?

82.1. The R Concept

`separate()` splits one column into multiple columns, based on a separator character:

82. `separate()`

```
separate(df, col, into = c("new_col1", "new_col2"), sep = "separator")
```

```
separate(bp_wide, clinic_site, into = c("county_check", "direction"), sep = "-")
```

`into` names the new columns, in the order the split pieces appear (left piece first); `sep` specifies the character (or pattern) to split on — here, a hyphen.

82.2. Demonstration

```
library(tidyr)
```

```
bp_wide <- read.csv("data/bp_visits_wide.csv")
```

```
bp_split <- separate(bp_wide, clinic_site,
  into = c("county_check", "direction"),
  sep = "-"
)
```

```
head(bp_split[, c("id", "county", "county_check", "direction")], 4)
```

	id	county	county_check	direction
1	1	Oakford	Oakford	North
2	2	Riverdale	Riverdale	South
3	3	Oakford	Oakford	East
4	4	Millbrook	Millbrook	West

`clinic_site` is gone from the result; `county_check` and `direction` have taken its place — and notice `county_check` matches the existing `county` column exactly, confirming the split worked correctly.

82.3. Explore

Try `table(bp_split$direction)`. With 28 participants across 4 directions (North, South, East, West), confirm the counts are as you'd expect, then try `identical(bp_split$county, bp_split$county_check)` to formally verify the split matches the original `county` column for every row.

82.4. Solve the Problem

```
library(tidyr)

bp_wide <- read.csv("data/bp_visits_wide.csv")

bp_split <- separate(bp_wide, clinic_site,
  into = c("county_check", "direction"),
  sep = "-"
)

table(bp_split$direction)

  East North South  West
    7     7     7     7

identical(bp_split$county, bp_split$county_check)

[1] TRUE
```

Clinic direction is now its own column, evenly split 7-7-7-7 across North/South/East/West, ready for your professor's direction-based analysis — and the `identical()` check confirms the split extracted exactly the county information already present elsewhere, with nothing lost or corrupted.

82. `separate()`

82.5. Interpret the Result

`direction` can now be used in `group_by()`, `filter()`, or anywhere else a normal column works — something impossible while it was fused into `clinic_site` as compound text. The `county_check` column, having served its verification purpose, would typically be dropped (`select(-county_check)`) once confirmed correct, since keeping a genuine duplicate of `county` around isn't useful going forward.

Common Mistakes

- **Getting into's column order backwards.** `into = c("county_check", "direction")` assumes the *first* split piece (before the separator) is `county` and the *second* is `direction` — matching "Oakford-North"'s actual left-to-right order. If the original text were reversed, you'd need to reverse `into` as well.
- **Choosing a `sep` that appears more than once in the data.** If a county name itself contained a hyphen, `sep = "-"` would split it into more than two pieces, breaking the intended two-column split — always check for separator characters appearing unexpectedly inside the values themselves.
- **Forgetting `separate()` removes the original column by default.** After `separate(df, clinic_site, ...)`, there is no more `clinic_site` column in the result unless you explicitly keep it (with the `remove = FALSE` argument).

82.6. Practice Problem

Use `separate()` to split `clinic_site` into `county_check` and `direction`, then use `filter()` (Level 5) to find every participant seen at a "North" clinic.

82.7. Challenge Problem

After separating `clinic_site`, use `group_by(direction)` and `summarise()` to compute the mean `sbp_visit1` for each clinic direction. Does blood pressure appear to differ meaningfully by clinic direction, or does this look more like random noise given the small group sizes (7 participants per direction)?

82.8. Key Takeaways

- `separate(df, col, into = c("new1", "new2"), sep = "...")` splits one column into multiple columns based on a separator.
- `into`'s column names must be given in the same left-to-right order the split pieces appear in the original values.
- `separate()` removes the original compound column by default, replacing it with the split-out pieces.

Knowledge Check

1. **Multiple choice.** In `separate(df, col, into = c("a", "b"), sep = "-")`, if `col` contains "X-Y", what does `a` become?
 - a) "X-Y"
 - b) "X"
 - c) "Y"
 - d) NA
2. **True or false.** `separate()` keeps the original compound column in the result by default.
3. **Predict the output.** If `clinic_site` contains "Millbrook-East", what would `direction` equal after separating with `sep = "-"`?
4. **Find the error.**

82. `separate()`

```
separate(bp_wide, clinic_site, into = c("direction", "county_check"), se
```

Show answers

1. **b)** "X" — the first split piece (before the separator) becomes the first `into` column.
2. **False.** `separate()` removes the original column by default unless `remove = FALSE` is specified.
3. "East".
4. `into`'s order is backwards — since `clinic_site` values are formatted as "county-direction" (e.g., "Millbrook-East"), the first piece is the county and the second is the direction. This code would incorrectly label "Millbrook" as `direction` and "East" as `county_check`. It should be `into = c("county_check", "direction")`.

83. unite()

The Problem

After `pivot_longer()`, your data has separate `id` and `visit` columns — 84 rows, but no single column that uniquely labels each participant-visit combination as one identifier. Your professor wants a single `record_id` column (like "1_visit1") to use as a unique row label when exporting the long-format data for a database import.

Your Mission

Combine two or more columns into one, using `tidyr::unite()` — the reverse of `separate()`.

Think Before Coding

- From the previous page, `separate()` split one compound column into several. What would you expect the reverse operation — combining several columns into one — to need as arguments?
- If you combined `id` (1) and `visit` ("visit1") with an underscore, what single value would you expect the result to be?

83.1. The R Concept

`unite()` combines multiple columns into one, joined by a separator:

```
unite(df, new_col, col1, col2, sep = "separator")
```

83. `unite()`

```
unite(bp_long_sbp, record_id, id, visit, sep = "_")
```

This is exactly the reverse of `separate()`: instead of naming a `sep` to split *by*, you name a `sep` to join *with*. The new column name (`record_id`) comes first, followed by the columns to combine, in the order you want them joined.

83.2. Demonstration

```
library(tidyr)
```

```
bp_long_sbp <- pivot_longer(bp_wide,
  cols = starts_with("sbp_visit"),
  names_to = "visit",
  names_prefix = "sbp_",
  values_to = "sbp"
)
```

```
bp_united <- unite(bp_long_sbp, record_id, id, visit, sep = "_")
```

```
head(bp_united[, c("record_id", "county", "sbp")], 4)
```

	record_id	county	sbp
1	1_visit1	Oakford	125
2	1_visit2	Oakford	120
3	1_visit3	Oakford	117
4	2_visit1	Riverdale	123

`id` and `visit` are gone from the result; `record_id` has taken their place, combining both pieces of information into one text value.

83.3. Explore

Try `length(unique(bp_united$record_id))` and compare it to `nrow(bp_united)`. Since every participant-visit combination is unique, these two numbers should match exactly — confirming `record_id` works as a genuine unique row identifier.

83.4. Solve the Problem

```
library(tidyr)

bp_long_sbp <- pivot_longer(bp_wide,
  cols = starts_with("sbp_visit"),
  names_to = "visit",
  names_prefix = "sbp_",
  values_to = "sbp"
)

bp_united <- unite(bp_long_sbp, record_id, id, visit, sep = "_")

length(unique(bp_united$record_id))

[1] 84

nrow(bp_united)

[1] 84
```

84 unique `record_id` values for 84 rows — confirming every participant-visit combination produced a genuinely unique identifier, exactly what your professor needs for the database export.

83. `unite()`

83.5. Interpret the Result

`record_id` now serves as a single, human-readable unique key ("1_visit1", "1_visit2", ...) for each row of the long-format data — a common requirement before exporting to external systems that expect one unique identifier column rather than a combination of several.

Common Mistakes

- **Forgetting `unite()` converts everything to text.** Even though `id` started as a number, `record_id` is a character column after uniting — you can no longer do arithmetic on the `id` portion without separating it back out first.
- **Choosing a `sep` that could reappear inside the combined values.** If `visit` values themselves contained an underscore, later trying to `separate()` this `record_id` back apart with `sep = "_"` could split it into more pieces than intended.
- **Forgetting `unite()` removes the original columns by default.** Like `separate()`, `unite()` drops `id` and `visit` from the result unless you specify `remove = FALSE` to keep them alongside the new combined column.

83.6. Practice Problem

Use `unite()` to combine `county` and `visit` into a single column called `county_visit`, separated by a dash. How many unique `county_visit` values result?

83.7. Challenge Problem

Use `unite()` to create `record_id` (as shown above) with `remove = FALSE`, keeping the original `id` and `visit` columns alongside the new combined one. Then use `separate()` to split `record_id` back apart, and confirm with `identical()` that you recover the original `id` and `visit` values (keeping in mind `unite()` converts `id` to text, so you may need `as.character()` on the original for a fair comparison).

83.8. Key Takeaways

- `unite(df, new_col, col1, col2, sep = "...")` combines multiple columns into one text column, joined by a separator — the reverse of `separate()`.
- `unite()` always produces a character (text) column, even if the original columns were numeric.
- Like `separate()`, `unite()` removes the original columns by default unless `remove = FALSE` is specified.

Knowledge Check

1. **Multiple choice.** What does `unite(df, id_visit, id, visit, sep = "_")` produce?
 - a) Two new columns, `id` and `visit`, split apart
 - b) One new column, `id_visit`, combining `id` and `visit` with an underscore
 - c) A count of unique `id` values
 - d) A filtered data frame with only matching rows
2. **True or false.** `unite()` preserves the original numeric type of a column if it was numeric before combining.

83. `unite()`

3. **Predict the output.** What would `unite(df, key, county, visit, sep = "-")` produce for a row where `county = "Oakford"` and `visit = "visit1"`?
4. **Find the error.**

```
unite(bp_long_sbp, id, visit, record_id, sep = "_")
```

Show answers

1. **b)** One new column, `id_visit`, combining `id` and `visit` with an underscore.
2. **False.** `unite()` always converts the combined result to a character (text) column, even if the inputs were numeric.
3. `"Oakford-visit1"`.
4. The new column name must come first, followed by the columns to combine — here `id` and `visit` are existing columns, but `record_id` (the intended new name) is listed last as if it were a third column to combine, which isn't what was likely intended. It should be `unite(bp_long_sbp, record_id, id, visit, sep = "_")`.

84. Pivoting Multiple Measurements at Once

The Problem

Back on the `pivot_longer()` page, a Common Mistake warned that pivoting only `sbp_visit1/sbp_visit2/sbp_visit3` leaves `weight_visit1/weight_visit2/weight_visit3` untouched — duplicating each participant’s weight values across all three of their new long rows. Your professor actually wants **both** measurements reshaped together, correctly: one row per participant-visit, with separate `sbp` and `weight` columns side by side.

Your Mission

Reshape multiple measurement types to long format simultaneously, using `pivot_longer()`’s special `.value` token.

Think Before Coding

- Look at the column name `sbp_visit1`. It actually encodes *two* pieces of information at once — which measurement, and which visit. Can you identify each piece?
- If `pivot_longer()` could split a column name into pieces the same way `separate()` splits a column’s *values*, what would you want the “measurement” piece to become, versus the “visit” piece?

84.1. The R Concept

Column names like `sbp_visit1` and `weight_visit1` encode **two** pieces of information joined by an underscore: which measurement (`sbp` or `weight`), and which visit (`visit1`). The special `.value` token in `names_to`, combined with `names_sep`, tells `pivot_longer()` to split each column name into pieces and use *one specific piece* (wherever `.value` is placed) as the name of a whole new **column**, rather than folding it into a single generic value column:

```
pivot_longer(df,
  cols = matches("visit"),
  names_to = c(".value", "visit"),
  names_sep = "_"
)

pivot_longer(bp_wide,
  cols = c(starts_with("sbp_visit"), starts_with("weight_visit")),
  names_to = c(".value", "visit"),
  names_sep = "_"
)
```

Here, `sbp_visit1` splits at the underscore into `"sbp"` and `"visit1"`. Because `.value` occupies the *first* position in `names_to`, that first piece (`"sbp"`) becomes an actual **column name** in the result, while the second piece (`"visit1"`) becomes a value in a `visit` column — giving you `sbp` and `weight` as two separate, properly reshaped columns, instead of duplicating one across the other.

84.2. Demonstration

```
library(tidyr)
```


84.3. Explore

```
bp_wide <- read.csv("data/bp_visits_wide.csv")

bp_long_both <- pivot_longer(bp_wide,
  cols = c(starts_with("sbp_visit"), starts_with("weight_visit")),
  names_to = c(".value", "visit"),
  names_sep = "_"
)

names(bp_long_both)

[1] "id"          "county"      "clinic_site" "visit"       "sbp"
[6] "weight"
```

Both `sbp` and `weight` now exist as genuine, separate columns — not one duplicated across the other's long rows.

```
head(bp_long_both[, c("id", "visit", "sbp", "weight")], 6)
```

```
   id visit sbp weight
1  1 visit1 125  101.4
2  1 visit2 120  100.8
3  1 visit3 117  101.0
4  2 visit1 123   80.1
5  2 visit2 121   79.6
6  2 visit3 129   79.0
```

84.3. Explore

Try `nrow(bp_long_both)`. It should still be 84 (28 participants $\times$ 3 visits) — reshaping two measurements together doesn't create extra rows, since `.value` correctly keeps `sbp` and `weight` as separate columns within the same row, rather than stacking them into additional rows.

84.4. Solve the Problem

```
library(tidyr)

bp_wide <- read.csv("data/bp_visits_wide.csv")

bp_long_both <- pivot_longer(bp_wide,
  cols = c(starts_with("sbp_visit"), starts_with("weight_visit")),
  names_to = c(".value", "visit"),
  names_sep = "_"
)

nrow(bp_long_both)

[1] 84

sum(!is.na(bp_long_both$sbp))

[1] 82

sum(!is.na(bp_long_both$weight))

[1] 82
```

84 rows (not 168), with both `sbp` and `weight` correctly present as their own columns in every row — exactly the properly reshaped structure your professor asked for, with no duplication of one measurement across the other.

84.5. Interpret the Result

`bp_long_both` is now genuinely “tidy”: one row per participant-visit combination, with each measurement type in its own column. This is the format that would let you `group_by(visit)` and compute means of *both* `sbp` and `weight` in a single `summarise(across(...))` call (Level 5), something the earlier single-measurement reshape couldn’t offer without either duplicating data or reshaping twice.

Common Mistakes

- **Forgetting to include `.value` in `names_to`.** Without it, `pivot_longer()` has no way to know that “`sbp`” and “`weight`” should become separate columns rather than both being lumped into one undifferentiated value column.
- **Choosing the wrong `names_sep`.** `sbp_visit1` splits correctly on the *first* underscore into “`sbp`” and “`visit1`” — but if a measurement name itself contained an underscore, the split would produce more pieces than the two `names_to` expects.
- **Reshaping each measurement separately when they didn’t need to be.** Doing two separate `pivot_longer()` calls (one for `sbp`, one for `weight`) and then trying to `merge` them back together is far more error-prone than using `.value` to reshape both in a single step.

84.6. Practice Problem

Confirm that `bp_long_both`’s `sbp` column, restricted to `visit == "visit1"`, matches `bp_wide$sbp_visit1` exactly (hint: use `filter()` from Level 5, then compare with `identical()` after matching row order by `id`).

84.7. Challenge Problem

Use `bp_long_both`, `group_by(visit)`, and `summarise(across(c(sbp, weight), ~ mean(.x, na.rm = TRUE)))` (Level 5's `across()`) to compute mean blood pressure and mean weight for each visit in a single call.

84.8. Key Takeaways

- The `.value` token in `pivot_longer()`'s `names_to` lets you reshape multiple measurement types into separate columns at once, rather than lumping them all into a single generic value column.
- `names_sep` (or `names_pattern`) tells `pivot_longer()` how to split each compound column name into its `.value` and other pieces.
- Reshaping multiple measurements together in one call avoids both row-count duplication and the awkwardness of merging separately reshaped pieces back together.

Knowledge Check

1. **Multiple choice.** What does the `.value` token do inside `pivot_longer()`'s `names_to`?
 - a) Deletes that piece of the column name
 - b) Uses that piece of the split column name as an actual new column name
 - c) Converts that piece to numeric
 - d) Has no special meaning; it's just a placeholder string
2. **True or false.** Reshaping `sbp` and `weight` together with `.value` produces more rows than reshaping `sbp` alone.
3. **Predict the output.** After the `.value`-based reshape, how many columns would `bp_long_both` have among `id`, `county`, `clinic_site`, `visit`, `sbp`, `weight`?

4. Find the error.

```

pivot_longer(bp_wide,
  cols = c(starts_with("sbp_visit"), starts_with("weight_visit")),
  names_to = "visit",
  names_sep = "_"
)

```

Show answers

1. **b)** Uses that piece of the split column name as an actual new column name.
2. **False.** Both reshapes produce 84 rows — `.value` keeps `sbp` and `weight` as separate columns within the same rows, rather than adding extra rows.
3. 6 columns: `id`, `county`, `clinic_site`, `visit`, `sbp`, `weight`.
4. `names_to` is missing the `.value` token — with just `"visit"`, `pivot_longer()` expects only one piece per column name and can't know that `"sbp"` and `"weight"` should become separate columns. It should be `names_to = c(".value", "visit")`.

85. Reshaping Mini-Capstone: Change Across Visits

The Problem

Your professor’s real question, at last stated plainly: “By county, how much did blood pressure change on average between the first and last visit?” Answering this needs visit 1 and visit 3 side by side in the *same row* so you can subtract them — but you also want the flexibility to pick “first and last visit” through filtering, rather than hardcoding column names like `sbp_visit1` and `sbp_visit3` directly.

Your Mission

Chain `pivot_longer()`, `filter()`, `pivot_wider()`, and `mutate()` together in one pipeline to compute a per-participant change score, then summarise it by county.

Think Before Coding

- To compute `visit3 - visit1` with `mutate()` (Level 5), do both values need to be in the same row, or can they be in different rows?
- If you go to long format first, could you `filter()` down to just the two visits you care about, *before* pivoting back to wide to align them side by side for subtraction?

85.1. The R Concept

A reshaping pipeline often goes **wide** $\rightarrow$ **long** $\rightarrow$ (**filter/analyze**) $\rightarrow$ **wide** again: long format makes it easy to `filter()` down to exactly the visits you want, and pivoting back to wide afterward aligns the remaining visits side by side so you can compute a per-row difference with `mutate()`:

```
df |>
  pivot_longer(cols = ..., names_to = "visit", values_to = "value") |>
  filter(visit %in% c("first_visit", "last_visit")) |>
  pivot_wider(names_from = visit, values_from = value) |>
  mutate(change = last_visit - first_visit)
```

85.2. Demonstration

```
library(dplyr)
library(tidyr)
```

```
bp_wide <- read.csv("data/bp_visits_wide.csv")
```

```
bp_change <- bp_wide |>
  pivot_longer(cols = starts_with("sbp_visit"), names_to = "visit", names_prefix = "sbp_") |>
  filter(visit %in% c("visit1", "visit3")) |>
  pivot_wider(names_from = visit, values_from = sbp) |>
  filter(!is.na(visit3)) |>
  mutate(change = visit3 - visit1)
```

```
head(bp_change[, c("id", "county", "visit1", "visit3", "change")], 5)
```

	id	county	visit1	visit3	change
1	1	Oakford	125	117	-8

2	2	Riverdale	123	129	6
3	3	Oakford	130	124	-6
4	4	Millbrook	117	116	-1
5	5	Millbrook	139	138	-1

Filtering out the 2 participants missing a `visit3` reading before computing `change` (via `filter(!is.na(visit3))`) avoids NA change values contaminating the county averages next.

85.3. Explore

Try skipping the `filter(visit %in% c("visit1", "visit3"))` step and pivoting all three visits wide instead. Confirm you'd then need `mutate(change = visit3 - visit1)` regardless — `visit2` would just sit unused in the result, since `mutate()` only references the columns you actually name.

85.4. Solve the Problem

```
library(dplyr)
library(tidyr)
```

```
bp_wide <- read.csv("data/bp_visits_wide.csv")
```

```
county_change_summary <- bp_wide |>
  pivot_longer(cols = starts_with("sbp_visit"), names_to = "visit", names_prefix = "sbp_", v
  filter(visit %in% c("visit1", "visit3")) |>
  pivot_wider(names_from = visit, values_from = sbp) |>
  filter(!is.na(visit3)) |>
  mutate(change = visit3 - visit1) |>
  group_by(county) |>
```

85. Reshaping Mini-Capstone: Change Across Visits

```
summarise(n = n(), mean_change = round(mean(change), 2)) |>
  arrange(mean_change)
```

```
county_change_summary
```

```
# A tibble: 5 x 3
  county      n mean_change
  <chr>    <int>      <dbl>
1 Springvale     6    -10.17
2 Cedar Falls     4     -8.00
3 Millbrook       4     -3.50
4 Riverdale       6     -2.67
5 Oakford         6     -1.17
```

Springvale shows the largest average blood pressure decrease from visit 1 to visit 3 (-10.17), followed by Cedar Falls (-8.00) — though Cedar Falls' figure rests on just 4 complete participants and Millbrook's on only 4 as well (2 of Millbrook's 6 participants dropped out before visit 3).

85.5. Interpret the Result

Every county in this subset shows a mean *decrease* in blood pressure from visit 1 to visit 3, consistent with the mild simulated treatment effect built into this dataset — but the size of that decrease varies meaningfully by county, from Oakford's modest -1.17 to Springvale's much larger -10.17. As always (Level 5), the smaller counties' averages (Cedar Falls and Millbrook, both n=4 after dropout) deserve more caution than Oakford, Riverdale, or Springvale's more stable n=6 estimates.

Common Mistakes

- **Computing change before removing missing visit3 values.** `mutate(change = visit3 - visit1)` with a missing `visit3` correctly produces NA for `change` — but if you forget to `filter(!is.na(visit3))` (or handle it in `summarise()` with `na.rm = TRUE`), those NAs can silently propagate into a NA group mean.
- **Reshaping all three visits when only two are needed.** Including `visit2` in the `pivot_wider()` step doesn't break anything, but it does create an unused column — filtering down to just the visits you need (as done here) keeps the pipeline focused and avoids confusion.
- **Interpreting a mean change without checking group size.** A -8.00 mean drop in a county of 4 people carries much more uncertainty than a -1.17 mean drop in a county of 6 — reshaping and chaining code correctly is necessary, but not sufficient, for a trustworthy conclusion.

85.6. Practice Problem

Repeat this pipeline for `weight` instead of `sbp` (reshape the `weight_visit1/weight_visit2/weight_visit3` columns, filter to `visit1` and `visit3`, compute `change`, and summarise by county). Does `weight` show a consistent decrease across counties, similar to blood pressure?

85.7. Challenge Problem

Modify the pipeline to compute change between `visit1` and `visit2` instead of `visit1` and `visit3`. Compare the resulting county rankings — does the county with the largest early change (`visit1` to `visit2`) also have the largest overall change (`visit1` to `visit3`)?

85.8. Key Takeaways

- A common reshaping pattern is wide $\rightarrow$ long (to filter flexibly) $\rightarrow$ wide again (to align values side by side for a row-wise calculation).
- Always resolve missing values from dropout or incomplete follow-up *before* they propagate into a downstream group summary.
- A per-county (or per-group) summary is only as trustworthy as its underlying sample size — reshaping code correctly doesn't guarantee the resulting numbers deserve equal confidence.

Knowledge Check

1. **Multiple choice.** Why reshape to long format before filtering to specific visits, rather than just selecting wide columns directly?
 - a) It's required by R syntax
 - b) It lets you filter by visit as an actual value, which generalizes better than hardcoding column names
 - c) Long format uses less memory
 - d) There's no reason; both approaches are identical
2. **True or false.** `mutate(change = visit3 - visit1)` will produce NA for any row where `visit3` is missing.
3. **Predict the output.** In `county_change_summary`, which county has the smallest-magnitude mean change (closest to zero)?
4. **Short answer.** Why does Cedar Falls' -8.00 mean change deserve more caution than Oakford's or Springvale's figures, even though all three are computed the same way?

Show answers

1. **b)** It lets you filter by visit as an actual value (`filter(visit %in% c(...))`), which generalizes to any pair of visits rather than hard-coding specific column names.

85.8. Key Takeaways

2. **True.** Arithmetic involving a missing value produces NA by default, matching the rule from every earlier level.
3. Oakford, with a mean change of -1.17 (closest to zero).
4. Cedar Falls' mean rests on only 4 participants (this subset's smallest county), making its average considerably less stable than Oakford's or Springvale's, both based on 6 participants.

86. Keys and Relationships

The Problem

Your professor now hands you two separate files: `data/baseline_enrollment.csv` (20 participants enrolled at the start of the study) and `data/followup_visit.csv` (20 follow-up visit records, collected a year later by a different field team). Some participants appear in both files; some appear in only one. Before writing any code, you need to understand exactly how these two files relate to each other — otherwise even a correct-looking merge could silently drop or duplicate participants.

Your Mission

Understand what a “key” is, and recognize the different possible relationships between two datasets before attempting to combine them.

Think Before Coding

- Both files have an `id` column. What do you think that column’s job is, compared to every other column in each file?
- If a participant enrolled at baseline but was never seen again at follow-up, what would you expect to happen to that participant’s row in `followup_visit.csv`?

86.1. The R Concept

A **key** is the column (or combination of columns) that identifies which row in one dataset corresponds to which row in another. Here, `id` is

86. Keys and Relationships

the key linking `baseline_enrollment.csv` and `followup_visit.csv` — every row in each file represents one participant, identified by that same `id` value.

Two datasets sharing a key can relate to each other in several ways:

- **Every id in both files matches exactly** (a clean one-to-one relationship) — rare in real longitudinal data.
- **Some ids exist in one file but not the other** — participants who dropped out before follow-up, or new recruits added at follow-up.
- **One id in one file matches multiple rows in the other** (a one-to-many relationship) — for example, if `followup_visit.csv` recorded two visits for some participants.

Before merging anything, always inspect both key columns first.

86.2. Demonstration

```
baseline <- read.csv("data/baseline_enrollment.csv")
followup <- read.csv("data/followup_visit.csv")
```

```
nrow(baseline)
```

```
[1] 20
```

```
nrow(followup)
```

```
[1] 20
```

Both files have 20 rows — but that doesn't mean they contain the same 20 participants.

```
sum(baseline$id %in% followup$id)
```

```
568
```



```
[1] 15

sum(!(baseline$id %in% followup$id))
```

```
[1] 5
```

86.3. Explore

Try `sum(followup$id %in% baseline$id)` and `sum(!(followup$id %in% baseline$id))`. Compare these counts to the ones above for `baseline`. Notice the numbers aren't symmetric in an obvious way until you think it through: some `followup` ids are genuinely new participants who weren't in `baseline` at all.

86.4. Solve the Problem

```
baseline <- read.csv("data/baseline_enrollment.csv")
followup <- read.csv("data/followup_visit.csv")

# ids in baseline but NOT in followup - likely dropouts
setdiff(baseline$id, followup$id)

[1] 16 17 18 19 20

# ids in followup but NOT in baseline - likely new recruits
setdiff(followup$id, baseline$id)

[1] 21 22 23 24 25

# ids present in both files
length(intersect(baseline$id, followup$id))
```

[1] 15

Exactly as your professor’s description hinted: 5 baseline participants (ids 16-20) have no follow-up record at all — likely dropouts — while 5 follow-up ids (21-25) don’t appear in baseline at all — likely participants recruited after the study began. 15 ids (1-15) appear in both files.

86.5. Interpret the Result

Before writing a single `join()` call, you now know exactly what to expect: a merge that keeps only matching ids will produce 15 rows; a merge that keeps every baseline id will produce 20 (with 5 missing follow-up values); a merge that keeps every id from either file will produce 25. Knowing this *ahead of time* is exactly what lets you catch a bug if your eventual join produces an unexpected row count.

Common Mistakes

- **Assuming two files with the same number of rows describe the same units.** `baseline` and `followup` both happen to have 20 rows here, but they represent partially different sets of 20 participants — row count alone tells you nothing about whether the *same* individuals are present.
- **Merging without checking the key column’s uniqueness first.** If `id` had duplicate values within a single file (a one-to-many relationship), a join could produce more rows than either original file — always check `length(unique(df$id)) == nrow(df)` before assuming a one-to-one relationship.
- **Not deciding in advance what should happen to non-matching ids.** Different join types (next several pages) handle non-matches completely differently — deciding whether you want to keep, drop, or investigate them should happen *before* you pick which join function to use, not after.

86.6. Practice Problem

Use `length(unique(baseline$id)) == nrow(baseline)` to confirm `id` is a genuinely unique key in `baseline_enrollment.csv` (no participant appears twice). Do the same check for `followup_visit.csv`.

86.7. Challenge Problem

Without using any `join()` function yet, use `setdiff()` and `intersect()` to predict, in writing, how many rows each of the following would produce: (a) a merge keeping only ids present in both files, (b) a merge keeping every id from `baseline`, (c) a merge keeping every id from `followup`, (d) a merge keeping every id from either file. Confirm your predictions once you reach the following pages.

86.8. Key Takeaways

- A key is the column that identifies which row in one dataset matches which row in another — here, `id`.
- Two datasets can relate as one-to-one, or with unmatched ids on either side (dropouts, new recruits) — always check both directions with `setdiff()` before merging.
- Predicting the expected row count of a merge *before* running it is the best way to catch a join gone wrong.

Knowledge Check

1. **Multiple choice.** What does `setdiff(baseline$id, followup$id)` return?
 - a) ids present in both files

86. Keys and Relationships

- b) ids in **baseline** that are NOT in **followup**
 - c) ids in **followup** that are NOT in **baseline**
 - d) The total number of participants
2. **True or false.** Two data frames with the same number of rows must describe the exact same set of individuals.
 3. **Predict the output.** Given 20 **baseline** ids and 20 **followup** ids, with 15 ids common to both, how many ids are in **baseline** only?
 4. **Short answer.** Why is it useful to compute `setdiff()` and `intersect()` before running any join function?

Show answers

1. **b)** ids in **baseline** that are NOT in **followup**.
2. **False.** Row count alone says nothing about whether the same individuals are represented — as seen here, both files have 20 rows but only 15 individuals overlap.
3. 5 (20 total baseline ids minus 15 shared with followup).
4. Knowing the expected overlap ahead of time lets you predict how many rows each join type should produce, so you can immediately catch a mistake if the actual result doesn't match your prediction.

87. `left_join()`

The Problem

Your professor wants one combined dataset: every baseline-enrolled participant, with follow-up data attached wherever it exists — and participants who dropped out should still appear, just with missing follow-up values, rather than disappearing from the dataset entirely. You need to keep **all 20 baseline participants**, no matter what happened at follow-up.

Your Mission

Combine two data frames, keeping every row from the first (“left”) data frame, using `dplyr::left_join()`.

Think Before Coding

- From the previous page, how many participants are in `baseline_enrollment.csv`, and how many of them also appear in `followup_visit.csv`?
- If a join is designed to keep *every* row from the left-hand data frame, what should happen to the follow-up columns for a participant who has no matching follow-up record?

87.1. The R Concept

`left_join()` keeps every row from the first (“left”) data frame, and attaches matching columns from the second (“right”) data frame wherever a match exists — filling in `NA` where it doesn’t:

87. `left_join()`

```
left_join(left_df, right_df, by = "key_column")
```

```
left_join(baseline, followup, by = "id")
```

The result always has **at least as many rows as `left_df`** (exactly as many, unless the key has duplicates on the right side) — nothing from **`baseline`** is ever dropped, regardless of whether **`followup`** has a matching row.

87.2. Demonstration

```
library(dplyr)
```

```
baseline <- read.csv("data/baseline_enrollment.csv")
```

```
followup <- read.csv("data/followup_visit.csv")
```

```
combined <- left_join(baseline, followup, by = "id")
```

```
nrow(combined)
```

```
[1] 20
```

All 20 baseline participants are present, exactly as many rows as **`baseline`** alone had.

```
combined[combined$id %in% c(16, 17, 18), ]
```

	id	age	sex	county	baseline_bmi	followup_sbp	followup_smoker
16	16	45	Female	Millbrook	30.4	NA	NA
17	17	79	Female	Springvale	21.1	NA	NA
18	18	26	Male	Oakford	29.1	NA	NA

Participants 16, 17, and 18 — who have no follow-up record — still appear in **`combined`**, with NA filling in for **`followup_sbp`** and **`followup_smoker`**.

87.3. Explore

Try `sum(is.na(combined$followup_sbp))`. You might expect exactly 5 (the dropout ids 16-20) — but check `combined[combined$id == 11, ]` too. Participant 11 *does* have a follow-up record, yet still shows NA for `followup_sbp`, because that participant's blood pressure was already missing in the original data (recall Level 4's messiness) — a pre-existing gap the join carries forward, not a sign the join failed to find a match.

87.4. Solve the Problem

```
library(dplyr)

baseline <- read.csv("data/baseline_enrollment.csv")
followup <- read.csv("data/followup_visit.csv")

combined <- left_join(baseline, followup, by = "id")

nrow(combined)

[1] 20

sum(is.na(combined$followup_sbp))

[1] 6

combined[combined$id == 11, c("id", "county", "followup_sbp")]

   id    county followup_sbp
11 11 Riverdale           NA
```

87. `left_join()`

6, not 5: the 5 true dropout ids (16-20, no matching record at all) plus participant 11, who *does* have a follow-up row but whose blood pressure reading was already missing in the source data — a pre-existing gap the join simply carries forward, not a sign of a failed match.

87.5. Interpret the Result

`combined` now represents every participant your study enrolled, whether or not they returned for follow-up — the true dropouts are visible as fully NA follow-up rows rather than silently vanishing, which is exactly the honest, complete picture your professor needs (a disappearing dropout would understate how many participants were actually lost to follow-up). Participant 11’s single missing reading is a different, narrower kind of gap — present in the original `followup_visit.csv` before the join ever ran — and worth distinguishing from true loss to follow-up.

Common Mistakes

- **Confusing “left” with alphabetical or numerical order.** “Left” refers to which data frame is listed *first* in the function call, `left_join(baseline, followup, ...)` — `baseline` is “left” here simply because it’s the first argument, not because of anything about its contents.
- **Forgetting to specify `by`.** Without `by = "id"`, `left_join()` tries to guess a shared column name automatically — usually correct here, but relying on this guess silently can go wrong if both data frames happen to share an unrelated column name too.
- **Assuming NA after a join means the join failed.** An NA in `followup_sbp` after `left_join()` is *expected and correct* for a participant with no follow-up record — it’s information (this person didn’t return), not an error.

87.6. Practice Problem

Use `left_join(followup, baseline, by = "id")` — reversing which data frame is “left.” Compare `nrow()` of this result to the original `left_join(baseline, followup, by = "id")`. Are they the same? Why or why not?

87.7. Challenge Problem

Use `left_join()` and then `filter(!(id %in% followup$id))` (Level 5) to extract just the 5 *true* dropout participants (ids 16-20) into their own data frame, ready for a “lost to follow-up” report — and explain why `filter(is.na(followup_sbp))` alone would incorrectly include participant 11 as well.

87.8. Key Takeaways

- `left_join(left_df, right_df, by = "key")` keeps every row of `left_df`, filling in NA for any right-hand columns where no match exists.
- “Left” simply means “listed first in the function call” — it has nothing to do with the data’s content or order.
- An NA produced by a join is meaningful information (no match found), not necessarily a mistake.

Knowledge Check

1. **Multiple choice.** What happens to a row in `baseline` that has no matching `id` in `followup` after `left_join(baseline, followup, by = "id")`?

87. `left_join()`

- a) It is dropped from the result
 - b) It is kept, with NA for the follow-up columns
 - c) It causes an error
 - d) It is duplicated
2. **True or false.** `left_join(baseline, followup, ...)` and `left_join(followup, baseline, ...)` always produce the same number of rows.
3. **Predict the output.** How many rows would `left_join(baseline, followup, by = "id")` produce, given 20 baseline participants?
4. **Find the error.**

```
combined <- left_join(baseline, followup)
# unsure whether this correctly used "id" as the join key
```

Show answers

- 1. **b)** It is kept, with NA filled in for the follow-up columns.
- 2. **False.** `left_join()` keeps all rows of whichever data frame is listed first — swapping the order can change the row count if the two files have different unmatched-id counts.
- 3. 20 rows — every baseline participant is retained regardless of follow-up match status.
- 4. This isn't necessarily wrong (`left_join()` will guess a shared column automatically), but it's fragile style — always specify `by = "id"` explicitly, so the join key is unambiguous and documented in the code itself.

88. `inner_join()`

The Problem

For your professor's actual statistical analysis — comparing baseline BMI to follow-up blood pressure — participants with a completely missing follow-up record (the `NA` rows from `left_join()`) are useless; there's nothing to compare them against. You want only participants who have **both** a baseline and a follow-up record, with no `NA`s to work around.

Your Mission

Combine two data frames, keeping only rows whose key appears in **both**, using `dplyr::inner_join()`.

Think Before Coding

- From two pages ago, how many ids appear in both `baseline_enrollment.csv` and `followup_visit.csv`?
- If `left_join()` keeps every left-hand row (filling in `NA`s for non-matches), what would you expect a join that keeps *only* matches to do with those same non-matching rows instead?

88.1. The R Concept

`inner_join()` keeps only the rows whose key exists in **both** data frames — any row without a match on either side is dropped entirely, so the join itself introduces no *new* `NA`s (though `NA`s already present in either file's own columns, from earlier missingness, still come along for the ride):

88. `inner_join()`

```
inner_join(df1, df2, by = "key_column")  
  
inner_join(baseline, followup, by = "id")
```

Unlike `left_join()`, `inner_join()`'s result can have *fewer* rows than either original data frame — exactly the count of ids common to both.

88.2. Demonstration

```
library(dplyr)  
  
baseline <- read.csv("data/baseline_enrollment.csv")  
followup <- read.csv("data/followup_visit.csv")  
  
complete_cases <- inner_join(baseline, followup, by = "id")  
  
nrow(complete_cases)
```

```
[1] 15
```

15 rows — exactly the number of ids found in both files with `intersect()` two pages ago.

```
sum(is.na(complete_cases))
```

```
[1] 3
```

Not zero — `inner_join()` guarantees every remaining row has a **matching id on both sides**, but it does nothing to clean up missingness that already existed *within* either file's own columns (recall `baseline_bmi` and `age` each had one pre-existing missing value from Level 3-4's messiness). The join only controls whether rows appear at all, not whether every field inside them is complete.

88.3. Explore

Try `range(complete_cases$id)`. Confirm the ids present are exactly 1 through 15 — the ids identified as common to both files — with none of the dropouts (16-20) or new recruits (21-25) appearing anywhere in the result.

88.4. Solve the Problem

```
library(dplyr)

baseline <- read.csv("data/baseline_enrollment.csv")
followup <- read.csv("data/followup_visit.csv")

complete_cases <- inner_join(baseline, followup, by = "id")

nrow(complete_cases)

[1] 15

cor(complete_cases$baseline_bmi, complete_cases$followup_sbp, use = "complete.obs")

[1] -0.1682
```

15 participant records with matching baseline and follow-up ids, ready for your professor's comparison — `use = "complete.obs"` (Level 11 preview) further narrows the correlation itself to the 13 participants with *both* a non-missing `baseline_bmi` and `followup_sbp`, showing a weak negative correlation not worth drawing strong conclusions from at this sample size.

88. `inner_join()`

88.5. Interpret the Result

`complete_cases` intentionally excludes the 5 dropouts (no follow-up record at all) and the 5 new recruits (no baseline record at all) — leaving only participants where a genuine before-and-after comparison is *possible*. It doesn't, however, guarantee every column within those 15 rows is filled in — `id10`'s `baseline_bmi` and `id13`'s `age` remain `NA`, exactly as they were in the original files, a reminder that matching keys and complete data are two separate concerns. It's also worth remembering *who* got excluded by the join itself, since dropouts are rarely random (Level 12 will return to this idea).

Common Mistakes

- **Using `inner_join()` when you actually wanted to preserve everyone.** If your goal was “show me the full enrolled cohort, follow-up or not,” `inner_join()` silently discards the very dropout information `left_join()` would have preserved — pick the join type based on what you actually need to see.
- **Not reporting how many rows were dropped.** Going from 20 baseline participants to 15 complete cases is itself an important finding (25% attrition) — an `inner_join()` without also reporting the original counts can hide a meaningful loss of data from a reader.
- **Assuming `inner_join()` and `left_join()` differ only when there's missing data.** If every key matched perfectly in both directions, `inner_join()` and `left_join()` would return identical results — the difference only becomes visible once you have non-matching rows, exactly the case here.
- **Assuming `inner_join()` guarantees a fully complete dataset.** `inner_join()` only guarantees every remaining row has a matching key on both sides — it says nothing about whether the individual columns within those matched rows are themselves free of `NA`, as `id10`'s missing `baseline_bmi` shows here.

88.6. Practice Problem

Compute `nrow(baseline) - nrow(complete_cases)` and confirm it equals the 5 dropouts identified two pages ago with `setdiff()`.

88.7. Challenge Problem

Use `inner_join()` and then `group_by(county) + summarise()` (Level 5) to compute the mean `followup_sbp` for each county among only the 15 complete-case participants. Compare this to what you'd get if you mistakenly used all 20 baseline rows via `left_join()` and forgot to handle the NAs.

88.8. Key Takeaways

- `inner_join(df1, df2, by = "key")` keeps only rows whose key exists in both data frames — no NAs are produced, but rows without a match are silently dropped.
- `inner_join()`'s result can have fewer rows than either input, exactly the count of matching keys.
- Always report how many rows a join dropped — an `inner_join()`'s missing participants are themselves often an important finding (like attrition), not just a technical detail.

Knowledge Check

1. **Multiple choice.** What does `inner_join(baseline, followup, by = "id")` keep?
 - a) Every row from `baseline`, regardless of a match
 - b) Every row from `followup`, regardless of a match

88. `inner_join()`

- c) Only rows whose `id` exists in both data frames
 - d) Every row from either data frame
2. **True or false.** `inner_join()` can never produce a result containing any NA values.
 3. **Predict the output.** Given 20 `baseline` ids and 15 common to both files, how many rows would `inner_join(baseline, followup, by = "id")` produce?
 4. **Short answer.** Why might reporting only the `inner_join()` result, without mentioning the original 20 baseline participants, be misleading in a study report?

Show answers

1. **c)** Only rows whose `id` exists in both data frames.
2. **False.** `inner_join()` prevents *new* NAs from being introduced by non-matching keys, but NAs already present in either file's own columns before the join (like `id10`'s missing `baseline_bmi`) still appear in the result.
3. 15 rows — exactly the ids common to both `baseline` and `followup`.
4. It would hide the fact that 5 of the original 20 enrolled participants (25%) had no follow-up data at all — an attrition rate that's often clinically and statistically important to report alongside any complete-case analysis.

89. `right_join()` and `full_join()`

The Problem

Your professor now wants two more views of the same data. First: a dataset centered on the *follow-up* file — every follow-up participant, including the 5 new recruits (ids 21-25) who joined the study after baseline and therefore have no baseline record at all. Second: a single dataset with **every participant who ever appeared in either file**, baseline or follow-up, with no one excluded.

Your Mission

Keep every row from the second (“right”) data frame using `dplyr::right_join()`, and keep every row from **either** data frame using `dplyr::full_join()`.

Think Before Coding

- If `left_join()` keeps every row of the *first* data frame listed, what would you expect a function named `right_join()` to keep instead?
- If you wanted a dataset with absolutely nobody excluded — every id from `baseline` and every id from `followup` — would `left_join()` or `right_join()` alone be enough on their own?

89.1. The R Concept

`right_join()` is `left_join()` with the roles reversed — it keeps every row of the **second** data frame, filling in `NA` for any first-data-frame columns that don't have a match:

89. *right_join()* and *full_join()*

```
right_join(df1, df2, by = "key_column")
# equivalent to:
left_join(df2, df1, by = "key_column")    # with column order preserved differ

right_join(baseline, followup, by = "id")
```

full_join() goes further still, keeping **every** row from **both** data frames
— matched where possible, NA-filled on whichever side lacks a match:

```
full_join(df1, df2, by = "key_column")

full_join(baseline, followup, by = "id")
```

89.2. Demonstration

```
library(dplyr)

baseline <- read.csv("data/baseline_enrollment.csv")
followup <- read.csv("data/followup_visit.csv")

right_result <- right_join(baseline, followup, by = "id")

nrow(right_result)

[1] 20

right_result[right_result$id %in% c(21, 22), ]

   id age  sex county baseline_bmi followup_sbp followup_smoker
21  21  NA <NA>   <NA>          NA         129             Yes
22  22  NA <NA>   <NA>          NA         124             No
```

89.3. Explore

Participants 21 and 22 — new recruits with no baseline record — still appear, with NA filling in every baseline column.

```
full_result <- full_join(baseline, followup, by = "id")
```

```
nrow(full_result)
```

```
[1] 25
```

89.3. Explore

Try `sort(unique(full_result$id))`. Confirm it runs from 1 to 25 with no gaps — every id that appeared in *either* file is present exactly once, whether it was a dropout, a new recruit, or a complete case.

89.4. Solve the Problem

```
library(dplyr)
```

```
baseline <- read.csv("data/baseline_enrollment.csv")
```

```
followup <- read.csv("data/followup_visit.csv")
```

```
right_result <- right_join(baseline, followup, by = "id")
```

```
nrow(right_result)
```

```
[1] 20
```

```
sum(is.na(right_result$baseline_bmi))
```

```
[1] 6
```

89. *right_join()* and *full_join()*

```
full_result <- full_join(baseline, followup, by = "id")
nrow(full_result)
```

```
[1] 25
```

right_join() produces 20 rows (every follow-up participant), with 6 missing *baseline_bmi* values — the 5 genuinely new recruits (21-25) plus participant 10, whose baseline BMI was already missing in the source data. *full_join()* produces 25 rows — the complete union of every id that appeared anywhere across both files.

89.5. Interpret the Result

right_result answers “who showed up at follow-up, and what do we know about them from baseline (if anything)?” — useful when the follow-up file is your primary population of interest. *full_result* answers a broader question: “who was ever part of this study, at any point?” — the union of all 25 distinct participants across both files, useful for a complete accounting of the whole study’s reach, dropouts and new recruits alike.

Common Mistakes

- **Forgetting *right_join(df1, df2)* and *left_join(df2, df1)* aren’t quite identical.** They keep the same *rows*, but the resulting column order differs (whichever data frame is listed first keeps its columns first) — the row-level result is equivalent, but the layout isn’t automatically identical.
- **Reaching for *full_join()* by default “just to be safe.”** *full_join()* guarantees no one is excluded, but it also guarantees more NAs to handle — if your analysis genuinely needs complete cases (Page 10’s *inner_join()*), a *full_join()* just adds cleanup work rather than solving the problem.

- **Losing track of which NAs mean “no matching record” versus “record exists but this specific field was missing.”** As seen throughout this level, both kinds of NA can appear in the same joined column, and conflating them can lead to miscounting genuine non-participation versus ordinary data entry gaps.

89.6. Practice Problem

Use `right_join(baseline, followup, by = "id")` and then `filter(is.na(age))` (Level 5) to extract the new recruits who have no baseline record. How many are there, and does that match the 5 identified with `setdiff()` on the keys-and-relationships page?

89.7. Challenge Problem

Compute `nrow(full_join(baseline, followup, by = "id"))` using only `nrow(baseline)`, `nrow(followup)`, and the `intersect()/setdiff()` counts from the keys-and-relationships page (hint: total unique ids = baseline count + followup count – ids common to both). Confirm your formula matches the actual `full_join()` row count.

89.8. Key Takeaways

- `right_join(df1, df2, by = "key")` keeps every row of `df2`, mirroring `left_join()` with the roles reversed.
- `full_join(df1, df2, by = "key")` keeps every row from either data frame — the complete union of both files’ ids.
- Choose the join type based on which population you actually want represented in the result, not by defaulting to whichever seems “safest.”

89. `right_join()` and `full_join()`

Knowledge Check

1. **Multiple choice.** What does `right_join(baseline, followup, by = "id")` keep?
 - a) Every row from `baseline`
 - b) Every row from `followup`
 - c) Only rows present in both
 - d) No rows — it always errors
2. **True or false.** `full_join()` can produce a result with more rows than either original data frame alone.
3. **Predict the output.** Given 20 baseline ids, 20 followup ids, and 15 common to both, how many rows would `full_join(baseline, followup, by = "id")` produce?
4. **Short answer.** Why might `right_join()` be more useful than `left_join()` if your analysis is primarily about the follow-up visit itself, rather than the original enrolled cohort?

Show answers

1. b) Every row from `followup`.
2. **True.** `full_join()`'s row count equals the number of *distinct* ids across both files combined — which can exceed either individual file's row count whenever there are any non-matching ids on either side.
3. 25 rows ($20 + 20 - 15$ common = 25 distinct ids total).
4. `right_join()` centers the result on the follow-up population (including new recruits not in baseline), whereas `left_join()` would center it on the original baseline cohort (including dropouts) — the right choice depends on which group is the actual subject of the analysis.

90. `semi_join()` and `anti_join()`

The Problem

Your professor asks two more specific questions, neither of which needs any *columns* combined from the second file — just a **filtered view** of the first file, based on whether a match exists elsewhere. First: “Show me only baseline participants who *do* have a follow-up record — but keep just the baseline columns, nothing from follow-up.” Second: “Show me exactly which baseline participants were lost to follow-up — again, baseline columns only.”

Your Mission

Filter one data frame based on whether its key matches another, without adding any of the second data frame’s columns, using `dplyr::semi_join()` and `dplyr::anti_join()`.

Think Before Coding

- Every join so far (`left_join()`, `inner_join()`, `right_join()`, `full_join()`) *added columns* from the second data frame. If a function only *filtered rows* of the first data frame instead, would its result need any new columns at all?
- If `semi_join()` keeps rows *with* a match, what would you expect its natural counterpart to keep instead?

90. *semi_join()* and *anti_join()*

90.1. The R Concept

`semi_join()` keeps only the rows of the first data frame that **have** a matching key in the second — but adds none of the second data frame’s columns:

```
semi_join(df1, df2, by = "key_column")
```

```
semi_join(baseline, followup, by = "id")
```

`anti_join()` does the opposite: it keeps only the rows of the first data frame that **do not** have a matching key in the second — again, adding no new columns:

```
anti_join(df1, df2, by = "key_column")
```

```
anti_join(baseline, followup, by = "id")
```

Both are **filtering joins** — unlike `left_join()/inner_join()/right_join()/full_join()` (all “mutating joins” that add columns), `semi_join()` and `anti_join()` only ever return a subset of the first data frame’s own rows and columns.

90.2. Demonstration

```
library(dplyr)
```

```
baseline <- read.csv("data/baseline_enrollment.csv")
```

```
followup <- read.csv("data/followup_visit.csv")
```

```
has_followup <- semi_join(baseline, followup, by = "id")
```

```
nrow(has_followup)
```

```
[1] 15
```


90.3. Explore

```
names(has_followup)
```

```
[1] "id"          "age"          "sex"          "county"       "baseline_bmi"
```

15 rows — the participants with a follow-up match — but notice: only `baseline`'s own 5 columns, with nothing from `followup` attached (unlike `inner_join()`, which would have added `followup_sbp` and `followup_smoker` too).

```
lost_to_followup <- anti_join(baseline, followup, by = "id")
```

```
lost_to_followup$id
```

```
[1] 16 17 18 19 20
```

90.3. Explore

Try `anti_join(followup, baseline, by = "id")` — reversing which file is first. Confirm this returns the 5 *new recruits* (ids 21-25) instead of the 5 dropouts — `anti_join()`'s result depends entirely on which data frame is listed first, exactly like every join so far.

90.4. Solve the Problem

```
library(dplyr)
```

```
baseline <- read.csv("data/baseline_enrollment.csv")
```

```
followup <- read.csv("data/followup_visit.csv")
```

```
lost_to_followup <- anti_join(baseline, followup, by = "id")
```

90. `semi_join()` and `anti_join()`

```
new_recruits <- anti_join(followup, baseline, by = "id")
```

```
lost_to_followup[, c("id", "county", "age")]
```

	id	county	age
16	16	Millbrook	45
17	17	Springvale	79
18	18	Oakford	26
19	19	Cedar Falls	80
20	20	Oakford	55

```
new_recruits$id
```

```
[1] 21 22 23 24 25
```

Two clean, separate reports for your professor: exactly the 5 baseline participants lost to follow-up (ids 16-20, ages ranging from 26 to 80), and exactly the 5 new recruits who joined only at follow-up (ids 21-25) — each extracted with a single filtering join, no unwanted columns attached.

90.5. Interpret the Result

`semi_join()` and `anti_join()` answer “which rows match (or don’t)?” without the complexity of merged, NA-filled columns — often exactly what’s needed when the goal is identifying *who* falls into a group, not analyzing their combined data. Compare this to Page 10’s `inner_join()`, which answers a related but different question (“give me the matched data, combined”) by adding columns rather than just filtering rows.

Common Mistakes

90.6. Practice Problem

- **Reaching for `inner_join()` when you only needed a filter.** If your goal is just “which participants match,” `semi_join()` gives a cleaner result than `inner_join()` followed by manually dropping the unwanted follow-up columns.
- **Forgetting `anti_join()`’s direction matters enormously.** `anti_join(baseline, followup, ...)` (dropouts) and `anti_join(followup, baseline, ...)` (new recruits) answer completely different questions — mixing them up reverses which group you’re actually reporting on.
- **Expecting `semi_join()` to add any columns at all.** Unlike every other join in this level, `semi_join()`’s result never has more columns than its *first* argument — if you need `followup`’s columns too, `semi_join()` is the wrong tool; reach for `inner_join()` instead.

90.6. Practice Problem

Use `semi_join(followup, baseline, by = "id")` to find which follow-up participants also have a baseline record. How many are there, and does this match `inner_join()`’s row count from Page 10?

90.7. Challenge Problem

Confirm that `nrow(semi_join(baseline, followup, by = "id")) + nrow(anti_join(baseline, followup, by = "id"))` equals `nrow(baseline)` exactly. Explain in words why this must always be true for any two data frames and any key column.

90. `semi_join()` and `anti_join()`

90.8. Key Takeaways

- `semi_join(df1, df2, by = "key")` keeps only `df1`'s rows that have a match in `df2`, with none of `df2`'s columns added.
- `anti_join(df1, df2, by = "key")` keeps only `df1`'s rows that have **no** match in `df2` — again, no new columns.
- Both are filtering joins, distinct from the “mutating joins” (`left_join()`, `inner_join()`, `right_join()`, `full_join()`) that combine columns from both data frames.

Knowledge Check

1. **Multiple choice.** How many columns does `semi_join(baseline, followup, by = "id")` return, if `baseline` has 5 columns and `followup` has 3?
 - a) 8 ($5 + 3$)
 - b) 5 (only `baseline`'s columns)
 - c) 3 (only `followup`'s columns)
 - d) 2 (only the shared key and one other)
2. **True or false.** `anti_join(baseline, followup, by = "id")` and `anti_join(followup, baseline, by = "id")` return the same rows.
3. **Predict the output.** Given 20 baseline ids and 15 with a follow-up match, how many rows would `anti_join(baseline, followup, by = "id")` return?
4. **Short answer.** When would you prefer `semi_join()` over `inner_join()`, even though both only keep matching rows?

Show answers

1. **b) 5** — `semi_join()` never adds columns from the second data frame, regardless of how many it has.

90.8. Key Takeaways

2. **False.** The two calls answer different questions entirely — dropouts (baseline-only) versus new recruits (followup-only) — and generally return completely different rows.
3. 5 rows — the baseline ids (16-20) with no matching follow-up record.
4. When you only need to know *which* rows match, not to actually combine data from both files — `semi_join()` avoids adding unwanted columns that you'd otherwise have to `select()` away after an `inner_join()`.

91. Level 6 Capstone: Reshaping and Joining Together

The Problem

Your professor’s final question for this level ties everything together: “Does blood pressure change from visit 1 to visit 3 differ between younger and older participants?” Answering this needs **two** separate skills from this level at once — reshaping `bp_visits_wide.csv` to compute each participant’s change score (Page 7), and then joining that result against `baseline_enrollment.csv` to bring in each participant’s `age`, since `age` was never part of the visits file to begin with.

Your Mission

Chain reshaping and joining together in a single analysis: reshape to compute a per-participant value, then join in demographic information from a separate file to analyze it further.

Think Before Coding

- `bp_visits_wide.csv` and `baseline_enrollment.csv` are two entirely separate files. What column do they share that could serve as a join key?
- Not every participant in `bp_visits_wide.csv` will necessarily have a matching row in `baseline_enrollment.csv` (they’re different studies’ worth of ids). Which join type keeps every visits-file participant regardless of whether a baseline match exists?

91. Level 6 Capstone: Reshaping and Joining Together

91.1. The R Concept

Real analyses frequently need both reshaping *and* joining — reshape one dataset into an analysis-ready shape, then join in additional variables from an entirely different source before the real analysis can happen:

```
df |>
  pivot_longer(...) |>           # reshape to long
  filter(...) |>
  pivot_wider(...) |>           # reshape back to wide, aligned for calculation
  mutate(change = ...) |>       # compute the per-unit value
  left_join(other_df, by = "id") # bring in variables the first file never had
```

91.2. Demonstration

```
library(dplyr)
library(tidyr)

bp_wide <- read.csv("data/bp_visits_wide.csv")
baseline <- read.csv("data/baseline_enrollment.csv")

bp_change <- bp_wide |>
  pivot_longer(cols = starts_with("sbp_visit"), names_to = "visit", names_prefix = "sbp_") |>
  filter(visit %in% c("visit1", "visit3")) |>
  pivot_wider(names_from = visit, values_from = sbp) |>
  filter(!is.na(visit3)) |>
  mutate(change = visit3 - visit1)

nrow(bp_change)
```

```
[1] 26
```

```
600
```


91.3. Explore

```
bp_change_with_age <- left_join(bp_change, baseline[, c("id", "age")], by = "id")

sum(is.na(bp_change_with_age$age))
```

```
[1] 10
```

10 of the 26 complete-case participants have no usable `age` — 9 simply aren't in `baseline_enrollment.csv` at all (this visits study enrolled a broader group than the baseline file covers), and 1 (participant 13) is in `baseline_enrollment.csv` but has a missing `age` value there.

91.3. Explore

Try `filter(bp_change_with_age, is.na(age))` and inspect which ids these are. Confirm 9 of them are ids outside `baseline_enrollment.csv`'s range entirely (using `setdiff()`), and 1 (id 13) is inside the range but has its own pre-existing missing `age`.

91.4. Solve the Problem

```
library(dplyr)
library(tidyr)

bp_wide <- read.csv("data/bp_visits_wide.csv")
baseline <- read.csv("data/baseline_enrollment.csv")

bp_change <- bp_wide |>
  pivot_longer(cols = starts_with("sbp_visit"), names_to = "visit", names_prefix = "sbp_", v
  filter(visit %in% c("visit1", "visit3")) |>
  pivot_wider(names_from = visit, values_from = sbp) |>
```

91. Level 6 Capstone: Reshaping and Joining Together

```
filter(!is.na(visit3)) |>
mutate(change = visit3 - visit1)

bp_change_with_age <- bp_change |>
left_join(baseline[, c("id", "age")], by = "id") |>
filter(!is.na(age)) |>
mutate(age_group = ifelse(age >= 50, "50+", "Under 50"))

age_group_summary <- bp_change_with_age |>
group_by(age_group) |>
summarise(n = n(), mean_change = round(mean(change), 2)) |>
arrange(age_group)
```

```
age_group_summary
```

```
# A tibble: 2 x 3
  age_group      n mean_change
  <chr>      <int>      <dbl>
1 50+         7      -1.86
2 Under 50     9      -8.56
```

Among the 16 participants with usable age and complete visit data, those under 50 show a much larger average blood pressure decrease (-8.56) than those 50 and older (-1.86) — the opposite of what you might have guessed, and a good reminder to report a surprising pattern rather than dismiss it, while still treating it cautiously (n=7 and n=9 are both small groups).

91.5. Interpret the Result

Answering your professor's question required chaining nearly every tool from this level: `pivot_longer()` and `pivot_wider()` to align `visit1` and `visit3` for a per-participant change score, `filter()` to handle dropouts

(Page 7), and finally `left_join()` to bring in `age` from a completely separate file — with 10 participants necessarily excluded from the final age-based comparison because that information simply doesn't exist for them. The under-50 group's larger average blood pressure drop is a genuine pattern in this small dataset, but with 7 and 9 participants per group, it's a hypothesis to investigate further (Level 11), not a settled conclusion.

Common Mistakes

- **Forgetting that joining across studies often means partial coverage, not full coverage.** `bp_visits_wide.csv` and `baseline_enrollment.csv` weren't guaranteed to describe the same participants — 9 of 26 simply had no baseline record at all, which is normal and expected, not a data error to “fix.”
- **Silently dropping unmatched rows without reporting how many.** The move from 26 complete-case participants to 16 with usable `age` is a meaningful reduction — always state it explicitly rather than letting the final summary table's row count speak for itself.
- **Treating a striking small-sample pattern as a confirmed finding.** The under-50 group's larger blood pressure drop, based on just 9 participants, is worth flagging and investigating further — not yet reported as if it were a validated result.

91.6. Practice Problem

Redo this pipeline joining in `sex` instead of `age` from `baseline_enrollment.csv`, and compare mean blood pressure change between male and female participants (among those with a usable `sex` value).

91.7. Challenge Problem

Redo the join using `full_join()` instead of `left_join()` between `bp_change` and `baseline`. Does `nrow()` of the result change? Explain what `full_join()` would add here that `left_join()` didn't, given that `baseline` might contain enrolled participants who aren't in `bp_visits_wide.csv` at all.

91.8. Key Takeaways

- Real analyses often require reshaping *and* joining together — reshaping to compute a per-unit value, then joining to bring in variables that weren't part of the original file.
- Joining across two independently collected files frequently produces partial, not full, coverage — always report how many rows lack a match, and why that's expected rather than an error.
- A striking pattern found in a small, joined subset should be reported as a lead worth investigating, not treated as a final answer.

Knowledge Check

1. **Multiple choice.** Why was `left_join()` (not `inner_join()`) used to bring `age` into `bp_change`?
 - a) `left_join()` runs faster
 - b) To keep every complete-case visits participant, even those without a baseline match, rather than silently dropping them
 - c) `inner_join()` doesn't exist in `dplyr`
 - d) There is no difference between the two here
2. **True or false.** Every participant in `bp_visits_wide.csv` also appears in `baseline_enrollment.csv`.

3. **Predict the output.** If 26 participants have complete visit data and 10 lack a usable `age`, how many remain in the final age-group summary?
4. **Short answer.** Why should the under-50 group's larger average blood pressure drop be reported cautiously, even though the calculation itself is correct?

Show answers

1. **b)** To keep every complete-case visits participant, even those without a baseline match, rather than silently dropping them (an `inner_join()` would have removed them immediately, hiding the coverage gap).
2. **False.** 9 of the 26 complete-case participants have no matching id in `baseline_enrollment.csv` at all.
3. 16 participants (26 minus the 10 lacking a usable age).
4. The comparison rests on only 7 (50+) and 9 (under 50) participants — small enough that the pattern could easily be due to chance rather than a real, generalizable effect; it's worth investigating further, not yet reporting as a confirmed finding.

Part VII.

Level 7: Descriptive Statistics

Level 7 — Descriptive Statistics

Where this level is headed

Before you model anything, you have to describe it: what's the average age, how spread out is the blood pressure data, what proportion of participants smoke? This level covers the summary statistics every health report opens with — by hand, in base R, and in the tidyverse.

What you'll build toward

You'll return to `data/participants.csv` — the same 100-participant dataset from every earlier level — and this time describe it numerically rather than clean or reshape it: central tendency, spread, quantiles, frequency tables, cross-tabulations, and correlation. You'll finish by assembling everything into a single descriptive report, the kind of numeric summary that opens almost every health study.

Topics in this level

- Mean, median, and mode — and a base R trap around the word “mode” itself
- Range, variance, and standard deviation
- Quantiles, percentiles, and the interquartile range (IQR)
- `summary()` and the five-number summary
- Frequency tables and proportions with `table()` and `prop.table()`

Level 7 — Descriptive Statistics

- Cross-tabulations between two categorical variables
- Correlation with `cor()`
- A capstone: assembling a full descriptive statistics report

A preview mission

Your mission across this level: write the first paragraph of a health report — the “Table 1” description of your study sample that every published study opens with — entirely from numbers you compute yourself, correctly handling the missing values that have run through this dataset since Level 1.

92. Mean, Median, and Mode

The Problem

Your professor asks for “the average age” of your 100 participants. You know `mean(participants$age, na.rm = TRUE)` from Level 2 — but a colleague mentions that with 6 missing ages and a few unusually young or old participants, the mean alone might not tell the whole story. They suggest also reporting the median, and the *mode* — but when you type `mode(participants$age)` into R, the result looks nothing like what you expected.

Your Mission

Compute and correctly interpret the three measures of central tendency — mean, median, and mode — and understand why R’s built-in `mode()` function is not what it sounds like.

Think Before Coding

- From Level 2, what does `mean(x, na.rm = TRUE)` compute, and why is `na.rm = TRUE` necessary for `age`?
- The *median* is “the middle value when sorted.” If `age` has 94 non-missing values (an even number), how would you expect the median to be computed — is there a single middle value, or two?

92.1. The R Concept

The **mean** is the sum of all values divided by how many there are — sensitive to unusually large or small values (“outliers”). The **median** is the middle value once the data is sorted (or the average of the two middle values, for an even count) — much less affected by outliers. The **mode** is the most frequently occurring value:

```
mean(x, na.rm = TRUE)
median(x, na.rm = TRUE)
```

R has **no built-in function for the statistical mode**. The function called `mode()` in base R does something completely unrelated — it reports an object’s storage type (“numeric”, “character”, etc.), not its most frequent value:

```
mode(participants$age)
```

```
[1] "numeric"
```

That’s `age`’s storage type, not its most common value — a well-known trap for anyone coming from other statistical software. Finding the actual statistical mode requires `table()` (Level 4) instead:

```
table(x)[which.max(table(x))]
```

92.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
mean(participants$age, na.rm = TRUE)
```

```
[1] 47.42553
```

```
median(participants$age, na.rm = TRUE)
```

```
[1] 49
```

The mean (47.43) and median (49) are reasonably close here, suggesting `age` isn't wildly skewed by extreme outliers — though they're not identical, so it's still worth reporting both.

```
mode(participants$age)
```

```
[1] "numeric"
```

Exactly the trap described above — `mode()` reports the storage type, telling you nothing about which age value is most common.

92.3. Explore

Try `table(participants$age)` directly and scan the output for the largest count. Then try `sort(table(participants$age), decreasing = TRUE)[1:3]` to see the three most frequent ages at a glance, without scanning the whole table by eye.

92.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")
```

```
mean_age <- mean(participants$age, na.rm = TRUE)
```

```
median_age <- median(participants$age, na.rm = TRUE)
```

92. Mean, Median, and Mode

```
age_table <- table(participants$age)
mode_age <- names(age_table)[which.max(age_table)]

mean_age

[1] 47.42553

median_age

[1] 49

mode_age

[1] "53"

age_table[mode_age]

53
5
```

Mean age 47.43, median age 49, and the single most common age is 53 (occurring 5 times) — three different, complementary answers to “what’s typical,” none of which is simply “more correct” than the others.

92.5. Interpret the Result

The mean pulls slightly lower than the median here, hinting at a slight leftward skew (a few younger participants pulling the average down relative to the middle value) — consistent with what you’d expect from a health study spanning ages 18 to 81. The mode (53) tells a different kind of story:

it doesn't describe "the middle" or "the average," just which single value happens to repeat most often — useful for categorical data, but often less informative than the mean or median for a wide-ranging numeric variable like age.

Common Mistakes

- **Using `mode()` expecting the statistical mode.** As demonstrated above, `mode()` in R reports an object's storage type — this is one of the most common traps for beginners, especially those coming from Excel or other statistical software where "mode" always means "most frequent value."
- **Forgetting `na.rm = TRUE` for mean and median alike.** Both `mean()` and `median()` return NA by default if any value is missing — the Level 2 rule applies to both identically, not just to `mean()`.
- **Assuming mean and median should always be close together.** They coincide closely for roughly symmetric data, but can diverge substantially for skewed data (like income, where a few very high earners pull the mean well above the median) — a difference between them is itself useful information, not an error.

92.6. Practice Problem

Compute the mean, median, and mode of `bmi`. Are the mean and median close together, or do they diverge notably? What might that suggest about the shape of the BMI distribution?

92.7. Challenge Problem

Write your own small function (Level 9 preview) called `get_mode()` that takes a vector and returns its most frequent value using `table()` and `which.max()`, handling NA values appropriately. Test it on

92. Mean, Median, and Mode

`participants$county` (a categorical variable, where “mode” is a much more natural summary than it is for `age`).

92.8. Key Takeaways

- Mean, median, and mode each describe “what’s typical” differently — mean is sensitive to outliers, median is not, and mode identifies the single most frequent value.
- R’s built-in `mode()` function reports storage type, not the statistical mode — use `table()` combined with `which.max()` instead.
- A meaningful difference between mean and median can itself be an informative finding about a variable’s distribution shape.

Knowledge Check

1. **Multiple choice.** What does `mode(participants$age)` actually return in base R?
 - a) The most frequent age
 - b) The middle age value
 - c) The storage type of the `age` column (e.g., “`numeric`”)
 - d) An error, since `mode()` doesn’t exist
2. **True or false.** `median()` is generally less sensitive to extreme outliers than `mean()`.
3. **Predict the output.** If `age` has 94 non-missing values (an even count), how is the median calculated?
4. **Find the error.**

```
most_common_age <- mode(participants$age)
```

Show answers

92.8. Key Takeaways

1. **c)** The storage type of the `age` column — `mode()` in base R is unrelated to the statistical mode.
2. **True.** The median only depends on the middle value(s) once sorted, so extreme values don't pull it the way they pull the mean.
3. As the average of the two middle values once all 94 non-missing ages are sorted.
4. `mode()` doesn't compute the statistical mode in R — it returns the object's storage type. Finding the actual most frequent age requires `table(participants$age)` combined with `which.max()`.

93. Range, Variance, and Standard Deviation

The Problem

Two counties could have the exact same mean systolic blood pressure, yet look completely different in practice — one where almost everyone clusters tightly around that average, another where readings swing wildly from very low to very high. The mean alone (Level 2) can't tell these two situations apart. Your professor wants a number that captures **how spread out** the data is, not just its center.

Your Mission

Compute and interpret the three most common measures of spread: range, variance, and standard deviation.

Think Before Coding

- If every participant had exactly the same blood pressure reading, what would the “spread” of that data be?
- The **range** is the distance between the smallest and largest value. Is a single very extreme outlier likely to affect the range a little, or a lot?

93. Range, Variance, and Standard Deviation

93.1. The R Concept

The **range** is simply the difference between the maximum and minimum values:

```
max(x, na.rm = TRUE) - min(x, na.rm = TRUE)
# or, more directly:
range(x, na.rm = TRUE)    # returns c(min, max)
```

Variance measures the average squared distance of each value from the mean — squaring makes every deviation positive and emphasizes larger deviations more heavily:

```
var(x, na.rm = TRUE)
```

Standard deviation is the square root of variance, which brings the measure back into the *same units* as the original data (unlike variance, which is in squared units) — making it the far more commonly reported of the two:

```
sd(x, na.rm = TRUE)
```

93.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
range(participants$age, na.rm = TRUE)
```

```
[1] 18 81
```

```
var(participants$age, na.rm = TRUE)
```

```
[1] 226.0105
```

93.3. Explore

```
sd(participants$age, na.rm = TRUE)
```

```
[1] 15.0336
```

Ages range from 18 to 81 (a 63-year span), with a standard deviation of about 15 years — meaning a “typical” participant’s age deviates from the mean (47.43) by roughly 15 years in either direction.

93.3. Explore

Try computing `sd(participants$systolic_bp, na.rm = TRUE)` and `sd(participants$bmi, na.rm = TRUE)`. Compare their relative sizes to their means — a standard deviation of 15 means something very different for a variable averaging 47 than it would for one averaging 130.

93.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

age_range <- range(participants$age, na.rm = TRUE)
age_sd <- sd(participants$age, na.rm = TRUE)

bp_range <- range(participants$systolic_bp, na.rm = TRUE)
bp_sd <- sd(participants$systolic_bp, na.rm = TRUE)

age_range

[1] 18 81

age_sd
```

93. Range, Variance, and Standard Deviation

```
[1] 15.0336
```

```
bp_range
```

```
[1] 90 179
```

```
bp_sd
```

```
[1] 18.8709
```

Age spans 63 years with an SD of about 15; systolic blood pressure spans 89 points (90 to 179) with an SD of about 18.9 — both meaningfully spread out, but on entirely different scales, which is exactly why reporting the units alongside any spread measure matters.

93.5. Interpret the Result

A standard deviation of 15.03 years for `age` tells you that most participants fall within roughly 32 to 62 years of age (mean $\pm$ 1 SD) — a genuinely wide range for this study. Systolic blood pressure’s SD of 18.87 similarly indicates substantial participant-to-participant variation, not a tightly clustered sample. Neither the range nor the SD by itself tells the complete story — the range is entirely determined by just two extreme values, while the SD reflects the spread of *every* value at once, which is why SD is usually the more informative of the two for describing a full distribution.

Common Mistakes

- **Reporting variance instead of standard deviation in a written summary.** Variance is in *squared* units (e.g., “years²”), which is rarely meaningful to a reader — always convert to standard deviation (or report both, with SD emphasized) before writing up results.

93.6. Practice Problem

- **Relying on the range alone to describe spread.** The range depends entirely on the two most extreme values in the entire dataset — a single unusually high or low measurement can make the range look far more dramatic than the data's typical spread actually is.
- **Forgetting `na.rm = TRUE` for `var()` and `sd()`, not just `mean()`.** Every one of these functions follows the same Level 2 rule: any missing value makes the result `NA` unless explicitly told to remove it first.

93.6. Practice Problem

Compute the range, variance, and standard deviation of `bmi`. Given BMI's mean (27.12) and SD, would a BMI of 40 be unusually far from typical, or well within the range you'd expect for this sample?

93.7. Challenge Problem

Compute the standard deviation of `systolic_bp` separately for smokers and non-smokers (hint: use `participants$systolic_bp[participants$smoker == "Yes"]` from Level 2's logical indexing, or `filter()` from Level 5). Does blood pressure appear more variable within one smoking group than the other?

93.8. Key Takeaways

- Range is the distance between the minimum and maximum values — simple, but sensitive to just two extreme data points.
- Variance measures average squared distance from the mean; standard deviation (its square root) restates that spread in the data's original units, making it far easier to interpret.

93. Range, Variance, and Standard Deviation

- A measure of central tendency (Page 1) and a measure of spread (this page) together give a far more complete picture than either alone.

Knowledge Check

1. **Multiple choice.** What units is standard deviation expressed in, relative to the original data?
 - a) Squared units
 - b) The same units as the original data
 - c) Always a percentage
 - d) Unitless
2. **True or false.** The range is affected only by the two most extreme values in a dataset, regardless of how the rest of the data is distributed.
3. **Predict the output.** If every value in a vector were identical, what would `sd(x)` equal?
4. **Find the error.**

```
cat("The typical age varies by about", var(participants$age, na.rm = TRUE))
```

Show answers

1. **b)** The same units as the original data — this is precisely why SD is reported far more often than variance.
2. **True.** The range depends entirely on the minimum and maximum; every other value in between has no effect on it whatsoever.
3. 0 — with no variation among identical values, there is no spread at all.

93.8. Key Takeaways

4. This reports `var()` (variance, in squared years) as if it were a spread measured in years — but variance and standard deviation are not interchangeable. It should use `sd(participants$age, na.rm = TRUE)` instead, which is in the same units (years) as the original data.

94. Quantiles and the Interquartile Range

The Problem

Standard deviation (previous page) describes spread using every value at once, but it's sensitive to extreme outliers pulling it around. Your professor wants a spread measure based on the *middle* of the data — one that ignores unusually high or low values entirely — and also wants to know: is a BMI of 40 unusual for this study, or does a meaningful chunk of participants have BMIs that high?

Your Mission

Compute quantiles (including quartiles) with `quantile()`, and use the interquartile range (IQR) to describe spread and flag potential outliers.

Think Before Coding

- From Level 4, what did `summary()` report as the “1st Qu.” and “3rd Qu.” values for `age`?
- If the middle 50% of a dataset spans a narrow range, but the full range (min to max) is much wider, what does that suggest about the data beyond the middle?

94. Quantiles and the Interquartile Range

94.1. The R Concept

A **quantile** is a value below which a given proportion of the data falls. The **quantiles** are the 25th, 50th, and 75th percentiles — dividing the data into four equal-sized groups:

```
quantile(x, na.rm = TRUE)           # 0%, 25%, 50%, 75%, 100% by default
quantile(x, probs = 0.9, na.rm = TRUE) # just the 90th percentile
```

The **interquartile range (IQR)** is the distance between the 1st and 3rd quartiles — the span of the *middle 50%* of the data, deliberately ignoring the most extreme quarter on either end:

```
IQR(x, na.rm = TRUE)
```

A common rule of thumb flags a value as a potential outlier if it falls more than $1.5 \times \text{IQR}$ below Q1 or above Q3.

94.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
quantile(participants$bmi, na.rm = TRUE)
```

0%	25%	50%	75%	100%
15.60	23.35	27.70	30.20	38.60

```
IQR(participants$bmi, na.rm = TRUE)
```

```
[1] 6.85
```

The middle 50% of BMI values falls between 23.35 and 30.20 — a fairly narrow band — while the full range (15.6 to 38.6) is much wider, hinting that a modest number of participants sit further out toward either extreme.

94.3. Explore

Try `quantile(participants$age, probs = c(0.1, 0.9), na.rm = TRUE)` to find the 10th and 90th percentiles directly, rather than the default quartiles. This tells you the age range containing the *middle 80%* of participants, ignoring the most extreme 10% on either end.

94.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

bmi_q1 <- quantile(participants$bmi, 0.25, na.rm = TRUE)
bmi_q3 <- quantile(participants$bmi, 0.75, na.rm = TRUE)
bmi_iqr <- IQR(participants$bmi, na.rm = TRUE)

lower_bound <- bmi_q1 - 1.5 * bmi_iqr
upper_bound <- bmi_q3 + 1.5 * bmi_iqr

lower_bound

      25%
13.075

upper_bound
```

94. Quantiles and the Interquartile Range

```
75%  
40.475
```

```
sum(participants$bmi > upper_bound | participants$bmi < lower_bound, na.rm =  
  
[1] 0
```

No participants fall outside the $1.5 \times \text{IQR}$ outlier bounds (13.075 to 40.475) — a BMI of 40 would actually just barely exceed the upper bound, but the highest BMI actually recorded in this study is 38.6, comfortably within the “not unusual” range by this rule.

94.5. Interpret the Result

BMI’s middle 50% (23.35 to 30.20) is fairly tightly clustered, and even the full range (15.6 to 38.6) doesn’t produce any values flagged as outliers by the standard $1.5 \times \text{IQR}$ rule. This is a genuinely useful finding to report: it tells your professor there’s no single extreme participant unduly influencing the mean or standard deviation reported on the previous two pages.

Common Mistakes

- **Confusing quartiles with percentiles.** Quartiles are simply specific percentiles (25th, 50th, 75th) — `quantile(x, 0.25)` and the “1st quartile” are the exact same number, just described with different vocabulary.
- **Forgetting the $1.5 \times \text{IQR}$ rule identifies *potential* outliers, not certain errors.** A value outside these bounds deserves a closer look (was it a data entry mistake, or a genuinely extreme but valid measurement?) — it should never be deleted automatically just for falling outside the rule.

- Applying `na.rm = TRUE` inconsistently across `quantile()` and `IQR()`. Both functions follow the same missing-value rule as every other summary statistic in this level — omit `na.rm = TRUE` and a single NA in the data makes the whole result NA.

94.6. Practice Problem

Compute the 1st and 3rd quartiles and the IQR for `systolic_bp`, then use the $1.5 \times \text{IQR}$ rule to check whether any participants have unusually high or low blood pressure readings.

94.7. Challenge Problem

Compute the 5th and 95th percentiles of `age` using `quantile(participants$age, probs = c(0.05, 0.95), na.rm = TRUE)`. How many participants fall outside this middle-90% range? Compare this count to what the $1.5 \times \text{IQR}$ rule would flag for the same variable.

94.8. Key Takeaways

- `quantile(x, probs, na.rm = TRUE)` computes any percentile you specify; the default call returns the quartiles (0%, 25%, 50%, 75%, 100%).
- `IQR(x, na.rm = TRUE)` measures the spread of the middle 50% of the data, ignoring the most extreme quarter on either end.
- The $1.5 \times \text{IQR}$ rule flags potential outliers for further investigation — it's a starting point for judgment, not an automatic deletion rule.

Knowledge Check

94. Quantiles and the Interquartile Range

1. **Multiple choice.** What does `quantile(x, 0.5, na.rm = TRUE)` return?
 - a) The mean
 - b) The median (50th percentile)
 - c) The mode
 - d) The standard deviation
2. **True or false.** The IQR uses the full range of the data, from minimum to maximum.
3. **Predict the output.** If $Q1 = 20$ and $Q3 = 40$, what is the IQR?
4. **Find the error.**

```
# "Any value outside 1.5xIQR is a data entry error and should be deleted"
```

Show answers

1. **b)** The median — the 50th percentile is exactly the median by definition.
2. **False.** The IQR uses only the 1st and 3rd quartiles (the middle 50%), deliberately excluding the most extreme values at either end.
3. 20 ($40 - 20$).
4. The $1.5 \times \text{IQR}$ rule flags *potential* outliers worth investigating, not confirmed errors — a value outside these bounds could be a genuine, valid, if unusual, measurement. It should never be deleted automatically without first checking whether it's a real observation.

95. The `summary()` Function

The Problem

You’ve now computed the mean, median, quartiles, and missing-value count of `age` across three separate pages, using `mean()`, `median()`, `quantile()`, and `sum(is.na(...))` one at a time (Level 4). Your professor wants all of these numbers at once, in one glance, without four separate function calls every time you look at a new variable.

Your Mission

Use `summary()` to get a complete numeric snapshot — minimum, quartiles, mean, maximum, and missing count — in a single call.

Think Before Coding

- From Level 4, what six or seven numbers did `summary(participants$age)` report?
- If `summary()` already reports the median as “Median” and the quartiles as “1st Qu.” / “3rd Qu.”, is there anything this level has covered so far that `summary()` *doesn’t* include?

95.1. The R Concept

`summary()` produces the “five-number summary” (minimum, 1st quartile, median, 3rd quartile, maximum) plus the mean, all in one call — and automatically reports a count of missing values if any exist, with no `na.rm` needed:

95. The `summary()` Function

```
summary(x)
```

95.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
summary(participants$age)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
18.00	36.25	49.00	47.43	58.75	81.00	6

One call reports the minimum (18), 1st quartile (36.25), median (49), mean (47.43), 3rd quartile (58.75), maximum (81), and missing count (6) — everything from the previous three pages, condensed into a single readable line.

95.3. Explore

Try `summary(participants)` — calling `summary()` on the **entire data frame** at once, not just one column. Notice it produces a compact summary of every column simultaneously: numeric columns get the five-number-summary treatment, while character columns (like `county`) get a simpler length/class/mode description instead.

95.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")
```

```
summary(participants$bmi)
```

95.5. Interpret the Result

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
15.60	23.35	27.70	27.12	30.20	38.60	5

```
summary(participants$systolic_bp)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
90.00	116.75	128.50	129.84	145.25	179.00	4

Both variables' complete descriptive profiles — spread, center, and missingness all at once — ready to drop directly into your professor's report without needing four separate function calls per variable.

95.5. Interpret the Result

`summary()` confirms what the earlier pages found piece by piece: BMI's mean (27.12) and median (27.70) sit close together, suggesting a roughly symmetric distribution, while systolic blood pressure's mean (129.84) and median (128.50) are similarly close. Both variables report a handful of missing values (5 and 4 respectively) automatically — no separate `sum(is.na(...))` call needed.

Common Mistakes

- **Forgetting `summary()` needs no `na.rm` argument.** Unlike `mean()`, `median()`, `var()`, `sd()`, and `quantile()`, `summary()` handles missing values automatically, reporting them in a dedicated NA's row rather than requiring `na.rm = TRUE`.
- **Applying `summary()` to a character column and expecting numeric output.** `summary(participants$county)` reports length, class, and mode (the *storage type* trap from Page 1) rather than quartiles — `summary()` behaves differently depending on the column's type.

95. The `summary()` Function

- **Treating `summary()`'s output as a data frame you can immediately manipulate.** `summary()` returns a special `table`-like object formatted for reading, not something you'd typically pipe into `filter()` or `mutate()` — for further computation, go back to the individual functions (`mean()`, `quantile()`, etc.).

95.6. Practice Problem

Run `summary(participants)` on the whole data frame and identify which columns show an NA's count, and which don't.

95.7. Challenge Problem

Compare `summary(participants$age)`'s reported mean to the value from `mean(participants$age, na.rm = TRUE)` computed on Page 1. Confirm they match exactly, then explain in words why `summary()` doesn't need an explicit `na.rm` argument to produce this correct result.

95.8. Key Takeaways

- `summary(x)` reports the minimum, quartiles, mean, maximum, and missing-value count for a numeric column in one call.
- `summary()` automatically excludes missing values from its calculations and reports their count separately — no `na.rm` argument required.
- `summary()` behaves differently for character or factor columns than for numeric ones, and can also summarize an entire data frame at once.

Knowledge Check

1. **Multiple choice.** Which of these does `summary()` NOT report for a numeric column?
 - a) Mean
 - b) Standard deviation
 - c) Median
 - d) Maximum
2. **True or false.** `summary()` requires an explicit `na.rm = TRUE` argument to handle missing values correctly.
3. **Predict the output.** If a column has no missing values at all, would `summary()` still show an `NA's` row?
4. **Short answer.** Why might you still want to compute `sd()` separately, even after running `summary()`?

Show answers

1. **b)** Standard deviation is not part of `summary()`'s output — only the five-number summary plus the mean.
2. **False.** `summary()` handles missing values automatically, reporting their count in a dedicated row without any `na.rm` argument needed.
3. No — the `NA's` row only appears when there is at least one missing value to report.
4. `summary()` doesn't report standard deviation at all, even though it's one of the most commonly cited spread measures alongside the mean — `sd()` must still be called separately to get it.

96. Frequency Tables and Proportions

The Problem

Your professor wants to know: “What percentage of participants are current smokers?” You know `table(participants$smoker)` from Level 4 gives raw counts (75 No, 25 Yes) — but a raw count of 25 doesn’t directly answer a question phrased in terms of *percentage*. You need to convert those counts into proportions yourself.

Your Mission

Convert a frequency table’s raw counts into proportions and percentages using `prop.table()`.

Think Before Coding

- If 25 out of 100 participants smoke, what proportion (a number between 0 and 1) does that represent? What percentage?
- From Level 4, what does `table(participants$smoker)` return — and could you divide that result by the total number of participants yourself, without a dedicated function?

96.1. The R Concept

`prop.table()` takes a table of counts (from `table()`) and converts every count into a proportion of the total:

96. Frequency Tables and Proportions

```
counts <- table(x)
prop.table(counts)
```

Multiplying by 100 converts proportions into percentages:

```
round(prop.table(counts) * 100, 1)
```

96.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
smoker_counts <- table(participants$smoker)
smoker_counts
```

```
No Yes
75  25
```

```
prop.table(smoker_counts)
```

```
No  Yes
0.75 0.25
```

```
round(prop.table(smoker_counts) * 100, 1)
```

```
No  Yes
75.0 25.0
```

75% of participants are non-smokers, 25% are current smokers — the exact percentage your professor asked for, computed directly from the raw counts you already knew how to produce.

96.3. Explore

Try `table(participants$county)` followed by `round(prop.table(table(participants$county)) * 100, 1)`. With 5 counties, confirm the percentages sum to (approximately) 100% — small rounding differences can occur when percentages don't divide evenly.

96.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

county_counts <- table(participants$county)
county_percentages <- round(prop.table(county_counts) * 100, 1)
```

county_counts

Cedar Falls	Millbrook	Oakford	Riverdale	Springvale
20	21	17	16	26

county_percentages

Cedar Falls	Millbrook	Oakford	Riverdale	Springvale
20.0	21.0	17.0	16.0	26.0

Springvale is both the largest county by raw count (26 participants) and by percentage (26.0%) — with Riverdale the smallest at 16 participants (16.0%). Since these percentages happen to divide evenly (all counts are whole percentages of 100), they sum to exactly 100%.

96.5. Interpret the Result

Reporting “26% of participants are from Springvale” is far more interpretable to most readers than “26 participants are from Springvale” without knowing the total — `prop.table()` is what turns Level 4’s raw `table()` counts into the percentages that actually appear in published health reports.

Common Mistakes

- **Forgetting `prop.table()` operates on the *table*, not the original vector.** `prop.table(participants$county)` (passing the raw column) doesn’t work as intended — `prop.table()` expects a `table()` object as input, not the original data.
- **Not rounding before displaying percentages.** `prop.table(counts) * 100` alone can produce long, hard-to-read decimals (like 16.666666...) — `round(..., 1)` keeps a report readable without implying false precision.
- **Reporting percentages without also reporting the underlying counts.** A percentage alone (“25% smoke”) hides the sample size behind it — 25% of 100 participants carries very different weight than 25% of just 8, so both numbers usually belong together in a report.

96.6. Practice Problem

Compute the percentage of participants who are "Female" versus "Male" using `table()` and `prop.table()` on the `sex` column.

96.7. Challenge Problem

Compute `table(participants$smoker)` percentages both including and excluding any missing values (check whether `smoker` has any NAs first with `sum(is.na(participants$smoker))`). Does handling missing values change the reported percentages meaningfully in this case?

96.8. Key Takeaways

- `prop.table(table(x))` converts a frequency table's raw counts into proportions (summing to 1).
- Multiplying by 100 and rounding produces reader-friendly percentages.
- Always report percentages alongside their underlying counts, so a reader can judge how much data the percentage is actually based on.

Knowledge Check

1. **Multiple choice.** What does `prop.table()` expect as its input?
 - a) A raw data frame
 - b) A `table()` object of counts
 - c) A single numeric value
 - d) A character vector
2. **True or false.** Proportions from `prop.table()` always sum to exactly 1.
3. **Predict the output.** If 40 out of 100 participants are male, what would `prop.table(table(sex))["Male"]` return?
4. **Find the error.**

96. *Frequency Tables and Proportions*

```
prop.table(participants$smoker)
```

Show answers

1. **b)** A `table()` object of counts — `prop.table()` converts counts to proportions, so it needs a table as input.
2. **True.** By definition, every category's count divided by the total will sum to exactly 1 (aside from potential floating-point rounding at many decimal places).
3. 0.4.
4. `prop.table()` was given the raw `smoker` column directly, rather than a `table()` of its counts. It should be `prop.table(table(participants$smoker))`.

97. Cross-Tabulations

The Problem

You know 25% of all participants smoke (previous page) — but your professor’s actual question is more specific: “Does the smoking rate differ between men and women?” A single overall percentage can’t answer this; you need to look at smoking status *separately within each sex* — exactly the kind of two-variable breakdown `table()` alone (with one argument) can’t produce.

Your Mission

Build a two-way frequency table — a cross-tabulation — showing how two categorical variables relate to each other.

Think Before Coding

- From Level 5, `count(participants, sex, smoker)` produced counts for every sex-and-smoking-status combination as separate *rows*. Could `table()` produce something similar, but shaped like a grid instead?
- If you wanted to know “what percentage of *women* smoke” specifically, would you divide by the total number of participants, or just the total number of women?

97.1. The R Concept

`table()` with two arguments builds a cross-tabulation — a grid with one variable’s categories as rows and the other’s as columns:

```
table(row_variable, column_variable)
```

```
table(participants$sex, participants$smoker)
```

To convert this into row-wise percentages (e.g., “of all women, what percent smoke?”), use `prop.table()` with `margin = 1` (percentages *within each row*):

```
prop.table(table(participants$sex, participants$smoker), margin = 1)
```

97.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
sex_smoker_table <- table(participants$sex, participants$smoker)
sex_smoker_table
```

	No	Yes
Female	42	10
Male	33	15

Now you can see, at a glance, that among the 52 women, 42 don’t smoke and 10 do — and among the 48 men, 33 don’t smoke and 15 do.

```
round(prop.table(sex_smoker_table, margin = 1) * 100, 1)
```

	No	Yes
Female	80.8	19.2
Male	68.8	31.2

`margin = 1` computes each row's percentages *within that row* — 80.8% and 19.2% for women sum to 100%, and 68.8% and 31.2% for men sum to 100% separately, rather than all four percentages summing to 100% together.

97.3. Explore

Try `prop.table(sex_smoker_table)` with **no** `margin` argument at all. Notice all four percentages now sum to 100% *overall*, answering a different question (“what percent of the entire study is female non-smokers?”) than the row-wise version above (“what percent of women don’t smoke?”).

97.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

sex_smoker_table <- table(participants$sex, participants$smoker)
sex_smoker_pct <- round(prop.table(sex_smoker_table, margin = 1) * 100, 1)

sex_smoker_table
```

	No	Yes
Female	42	10
Male	33	15

```
sex_smoker_pct
```

97. Cross-Tabulations

	No	Yes
Female	80.8	19.2
Male	68.8	31.2

Yes — the smoking rate does differ meaningfully by sex in this sample: 31.2% of men smoke, compared to 19.2% of women, a 12-percentage-point difference directly answering your professor’s question.

97.5. Interpret the Result

Reporting the overall 25% smoking rate (previous page) alone would have hidden this sex difference entirely — the cross-tabulation reveals that men in this sample smoke at nearly double the rate of women. As always, this is a descriptive pattern in $n=48$ men and $n=52$ women, not yet a tested statistical claim (Level 11 covers how to test whether a difference like this is likely to reflect a real, generalizable pattern versus chance).

Common Mistakes

- **Forgetting to specify `margin` and misreading the resulting percentages.** Without `margin = 1` or `margin = 2`, `prop.table()` computes percentages relative to the *entire table*, not within each row or column — a very different (and often not what you intended) question.
- **Confusing `margin = 1` (row percentages) with `margin = 2` (column percentages).** `margin = 1` makes each *row* sum to 100%; `margin = 2` makes each *column* sum to 100% instead — mixing these up flips which variable you’re computing percentages “within.”
- **Drawing a causal conclusion from a descriptive cross-tabulation.** “Men smoke more than women in this sample” is a valid descriptive finding — it is not, by itself, evidence that

being male *causes* higher smoking rates; cross-tabulations describe patterns, they don't establish causation.

97.6. Practice Problem

Build a cross-tabulation of `county` and `smoker`, then compute column percentages (`margin = 2`) to answer: “of all current smokers, what percentage live in each county?”

97.7. Challenge Problem

Use `table(participants$sex, participants$smoker)` together with `margin = 2` (instead of `margin = 1`) to compute: “of all smokers, what percentage are male versus female?” Explain in words why this answers a different question than the row-percentage version already computed above.

97.8. Key Takeaways

- `table(var1, var2)` builds a two-way cross-tabulation, showing how two categorical variables relate.
- `prop.table(cross_tab, margin = 1)` computes row-wise percentages; `margin = 2` computes column-wise percentages; omitting `margin` computes percentages relative to the whole table.
- A cross-tabulation can reveal patterns (like a sex difference in smoking rate) that an overall single-variable summary completely hides — but it remains a descriptive finding, not a causal claim.

Knowledge Check

97. Cross-Tabulations

1. **Multiple choice.** What does `margin = 1` mean in `prop.table(tab, margin = 1)`?
 - a) Percentages computed within each column
 - b) Percentages computed within each row
 - c) Percentages computed relative to the whole table
 - d) It has no effect
2. **True or false.** A cross-tabulation can reveal patterns that an overall single-variable frequency table would hide.
3. **Predict the output.** In `sex_smoker_table`, do the row percentages for “Female” (80.8, 19.2) sum to 100?
4. **Short answer.** Why is “men smoke more than women in this sample” a safer statement than “being male causes higher smoking rates,” based on this cross-tabulation alone?

Show answers

1. **b)** Percentages computed within each row.
2. **True.** The overall 25% smoking rate hid a substantial sex difference (19.2% vs. 31.2%) that only the cross-tabulation revealed.
3. Yes — $80.8 + 19.2 = 100$, exactly as expected for row percentages.
4. A cross-tabulation only describes an association observed in this particular sample — it doesn’t rule out other explanations (like differences in age, county, or other unmeasured factors between the male and female participants) that a purely descriptive table can’t account for.

98. Correlation with `cor()`

The Problem

Your professor asks: “Does blood pressure tend to rise with BMI in this sample? What about with age?” So far, every statistic in this level has described *one* variable at a time. Answering “does X relate to Y” needs a way to summarize how **two numeric variables move together** — not a cross-tabulation (previous page, which is for categorical variables), but its numeric counterpart.

Your Mission

Compute and interpret a correlation coefficient using `cor()`, measuring the strength and direction of a linear relationship between two numeric variables.

Think Before Coding

- If two variables moved in perfect lockstep — every increase in one matched by a proportional increase in the other — what would you expect a “relationship strength” number to look like at its most extreme?
- Correlation coefficients range from -1 to 1. What do you think a value near 0 would indicate about the relationship between two variables?

98. Correlation with `cor()`

98.1. The R Concept

`cor()` computes the Pearson correlation coefficient between two numeric variables, a single number between **-1** and **1**:

```
cor(x, y, use = "complete.obs")
```

- **+1** means a perfect positive relationship (as one variable increases, so does the other, exactly proportionally).
- **-1** means a perfect negative relationship (as one increases, the other decreases, exactly proportionally).
- **0** means no linear relationship at all.

Values in between describe weaker versions of these patterns — a common rough guide treats values below about 0.3 in magnitude as “weak,” 0.3-0.7 as “moderate,” and above 0.7 as “strong,” though these thresholds vary by field. The `use = "complete.obs"` argument tells `cor()` to use only rows where **both** variables are non-missing — `cor()` has no separate `na.rm` argument like `mean()` does.

98.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
cor(participants$bmi, participants$systolic_bp, use = "complete.obs")
```

```
[1] 0.1631
```

A weak positive correlation: higher BMI is very slightly associated with higher systolic blood pressure in this sample, but the relationship is far from strong.

```
cor(participants$age, participants$systolic_bp, use = "complete.obs")
```

```
[1] -0.2211
```

A weak negative correlation between age and blood pressure — the opposite direction you might have expected, and a reminder that real (and simulated) data doesn't always match intuition.

98.3. Explore

Try `cor(participants$systolic_bp, participants$diastolic_bp, use = "complete.obs")`. Since both are blood pressure readings taken from the same participants at the same time, you'd expect a much stronger relationship than either of the two above — confirm whether that's the case.

98.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

cor_bmi_bp <- cor(participants$bmi, participants$systolic_bp, use = "complete.obs")
cor_age_bp <- cor(participants$age, participants$systolic_bp, use = "complete.obs")
cor_sbp_dbp <- cor(participants$systolic_bp, participants$diastolic_bp, use = "complete.obs")

cor_bmi_bp

[1] 0.1631

cor_age_bp

[1] -0.2211
```

98. Correlation with `cor()`

```
cor_sbp_dbp
```

```
[1] 0.9204
```

Systolic and diastolic blood pressure correlate very strongly (0.92) — participants with higher systolic readings consistently have higher diastolic readings too, exactly as clinically expected — while BMI (0.16) and age (-0.22) show only weak relationships with systolic blood pressure in this particular sample.

98.5. Interpret the Result

The very strong systolic-diastolic correlation (0.92) makes clinical sense: both numbers describe the same underlying cardiovascular measurement taken at the same moment, so it would be surprising if they *didn't* move together closely. The weak BMI-blood pressure and age-blood pressure correlations are worth reporting honestly rather than downplaying — this dataset simply doesn't show the strong BMI-BP or age-BP relationship sometimes assumed by default, a useful (if less dramatic) finding in its own right, and a genuine contrast against just how strong a correlation *can* look when two variables are this closely related.

Common Mistakes

- **Assuming a weak correlation means “no relationship at all.”**
A correlation of 0.16 is weak, but not necessarily zero — `cor()` measures *linear* relationship strength specifically; a strong non-linear relationship could still produce a low `cor()` value.
- **Interpreting correlation as causation.** A positive BMI-blood pressure correlation doesn't establish that higher BMI *causes* higher blood pressure — other explanations (age, genetics, unmeasured factors) could produce the same pattern.

- Forgetting `cor()`'s missing-value argument is `use`, not `na.rm`. Unlike `mean()`, `sd()`, or `quantile()`, `cor()` uses `use = "complete.obs"` (or other options) rather than `na.rm = TRUE` — a different argument name for a related idea.

98.6. Practice Problem

Compute the correlation between `age` and `bmi`. Is it stronger, weaker, or similar in magnitude to the age-blood pressure correlation computed above?

98.7. Challenge Problem

Compute `cor(participants$age, participants$systolic_bp, use = "complete.obs")` separately for smokers and non-smokers (hint: use `filter()` from Level 5 to split the data first). Does the age-blood pressure relationship look different within each smoking group?

98.8. Key Takeaways

- `cor(x, y, use = "complete.obs")` computes a correlation coefficient between -1 and 1, describing the strength and direction of a linear relationship between two numeric variables.
- Values near 0 indicate a weak (or no) linear relationship; values near ± 1 indicate a strong one, with the sign indicating direction.
- Correlation, however strong, never by itself establishes that one variable causes changes in another.

Knowledge Check

98. *Correlation with cor()*

1. **Multiple choice.** What does a correlation of -0.85 indicate?
 - a) A strong positive relationship
 - b) A strong negative relationship
 - c) No relationship at all
 - d) An invalid result, since correlations can't be negative
2. **True or false.** A high correlation between two variables proves that one causes the other.
3. **Predict the output.** If two variables have absolutely no linear relationship, what value would `cor()` return, approximately?
4. **Find the error.**

```
cor(participants$bmi, participants$systolic_bp, na.rm = TRUE)
```

Show answers

1. **b)** A strong negative relationship — the sign indicates direction (negative means as one variable rises, the other tends to fall), and 0.85 in magnitude indicates a strong relationship.
2. **False.** Correlation never establishes causation by itself — a strong relationship could arise from a third, unmeasured factor influencing both variables.
3. Approximately 0.
4. `cor()` doesn't accept `na.rm` — its missing-value argument is called `use`. It should be `cor(participants$bmi, participants$systolic_bp, use = "complete.obs")`.

99. Level 7 Capstone: A Descriptive Statistics Report

The Problem

Your professor’s final request for this level: write the opening paragraph of a study report — the “Table 1” every published health study begins with, describing the sample before any deeper analysis. It needs to state, clearly and correctly: how many participants, their age and BMI (center and spread), the sex and smoking breakdown, and how blood pressure varies by county — pulling together every tool from this entire level.

Your Mission

Assemble a complete descriptive statistics report from `participants.csv`, combining central tendency, spread, frequency tables, and grouped summaries into one coherent set of findings.

Think Before Coding

- Looking back over this level’s seven pages, which functions would you need to describe a single *numeric* variable like `age` completely (center, spread, and missingness)?
- Which functions would you need to describe a single *categorical* variable like `sex` or `smoker` completely (counts and percentages)?

99.1. The R Concept

A complete descriptive report typically combines, for each numeric variable: `mean()`, `median()`, `sd()`, `range()`, and a missing-value count; and for each categorical variable: `table()` and `prop.table()`. `dplyr`'s `group_by()` + `summarise()` (Level 5) extends this to grouped numeric summaries, like blood pressure by county.

```
numeric_summary <- function(x) {  
  c(  
    n_valid = sum(!is.na(x)),  
    n_missing = sum(is.na(x)),  
    mean = round(mean(x, na.rm = TRUE), 2),  
    sd = round(sd(x, na.rm = TRUE), 2),  
    median = median(x, na.rm = TRUE),  
    min = min(x, na.rm = TRUE),  
    max = max(x, na.rm = TRUE)  
  )  
}
```

99.2. Demonstration

```
library(dplyr)  
  
participants <- read.csv("data/participants.csv")  
  
nrow(participants)  
  
[1] 100  
  
round(mean(participants$age, na.rm = TRUE), 2)
```

```
[1] 47.43
```

```
round(sd(participants$age, na.rm = TRUE), 2)
```

```
[1] 15.03
```

```
round(prop.table(table(participants$sex)) * 100, 1)
```

```
Female  Male
  52.0   48.0
```

99.3. Explore

Try wrapping the numeric-summary calculations above into a small reusable function (a Level 9 preview), taking a column as input and returning a named vector of statistics — so you don't have to retype `mean()`, `sd()`, `median()`, `min()`, and `max()` separately for every variable in the report.

99.4. Solve the Problem

```
library(dplyr)
```

```
participants <- read.csv("data/participants.csv")
```

```
# Sample size
```

```
n_total <- nrow(participants)
```

```
# Age: center, spread, missingness
```

```
age_mean <- round(mean(participants$age, na.rm = TRUE), 2)
```

```
age_sd <- round(sd(participants$age, na.rm = TRUE), 2)
```

99. Level 7 Capstone: A Descriptive Statistics Report

```
age_median <- median(participants$age, na.rm = TRUE)
age_missing <- sum(is.na(participants$age))

# BMI: center and spread
bmi_mean <- round(mean(participants$bmi, na.rm = TRUE), 2)
bmi_sd <- round(sd(participants$bmi, na.rm = TRUE), 2)

# Sex breakdown
sex_pct <- round(prop.table(table(participants$sex)) * 100, 1)

# Smoking status
smoker_pct <- round(prop.table(table(participants$smoker)) * 100, 1)

# Blood pressure by county
bp_by_county <- participants |>
  group_by(county) |>
  summarise(n = n(), mean_sbp = round(mean(systolic_bp, na.rm = TRUE), 2)) |>
  arrange(desc(mean_sbp))

n_total

[1] 100

age_mean

[1] 47.43

sex_pct

Female    Male
    52.0    48.0

bp_by_county

660
```

99.5. Interpret the Result

```
# A tibble: 5 x 3
  county      n mean_sbp
  <chr>    <int>    <dbl>
1 Millbrook    21   133.30
2 Springvale   26   131.46
3 Oakford     17   129.71
4 Cedar Falls  20   128.16
5 Riverdale   16   124.36
```

Every number your professor’s Table 1 paragraph needs is now computed: sample size, age center and spread, sex breakdown, smoking rate, and blood pressure ranked by county.

99.5. Interpret the Result

Putting it into a written paragraph:

“The study sample included 100 participants, with a mean age of 47.4 years ($SD = 15.0$, 6 missing) and mean BMI of 27.1 ($SD = 4.6$). The sample was 52% female and 48% male, and 25% of participants reported current smoking. Mean systolic blood pressure ranged from 124.4 mmHg in Riverdale to 133.3 mmHg in Millbrook across the study’s five counties.”

Every number in that paragraph traces directly back to a specific function call from this level — nothing was estimated or eyeballed. This is exactly the kind of opening paragraph a published health study begins with, built entirely from tools covered across eight pages.

Common Mistakes

- **Reporting a mean without its missing-value count.** “Mean age was 47.4” is incomplete without also noting that 6 of 100 ages were missing — a reader needs to know how much of the sample that mean actually reflects.

99. Level 7 Capstone: A Descriptive Statistics Report

- **Rounding at different points inconsistently.** Rounding `age_mean` to 2 decimals but leaving `bp_by_county`'s means unrounded (or rounded differently) makes a report look inconsistent — decide on a rounding convention and apply it uniformly.
- **Writing the narrative paragraph before double-checking every number against its source calculation.** It's easy to transpose two percentages, or misremember which county had the highest blood pressure, when translating numbers into prose — always re-verify each stated number against the code that produced it before finalizing a report.

99.6. Practice Problem

Extend the report to include the smoking percentage by sex (from Level 7's cross-tabulation page) as an additional sentence in the paragraph.

99.7. Challenge Problem

Write a complete “Table 1” for just the subset of participants aged 65 and older (Level 5's `filter()`), reporting sample size, mean age, mean BMI, sex breakdown, and smoking rate for this older subgroup specifically. How does this subgroup's profile compare to the full sample's?

99.8. Key Takeaways

- A complete descriptive report combines central tendency, spread, and missingness for numeric variables, and counts with percentages for categorical variables.

- `group_by()` + `summarise()` (Level 5) extends single-variable descriptive statistics into grouped comparisons, like blood pressure by county.
- Every number in a written report should be traceable back to a specific, reproducible calculation — never eyeballed or estimated.

Knowledge Check

1. **Multiple choice.** Which piece of information should always accompany a reported mean for a variable with missing data?
 - a) The variable's storage type
 - b) The count (or percentage) of missing values
 - c) The variable's alphabetical position in the data frame
 - d) Nothing extra is needed
2. **True or false.** A complete descriptive report only needs numeric summaries; categorical variables don't need to be described.
3. **Predict the output.** Given the reported percentages (52% female, 48% male), would you expect these two numbers to sum to 100?
4. **Short answer.** Why is it good practice to re-verify every number in a written report paragraph against its source calculation before finalizing it?

Show answers

1. **b)** The count or percentage of missing values, so a reader knows how much of the sample the reported mean actually reflects.
2. **False.** Categorical variables (like `sex` and `smoker`) need their own counts and percentages — a complete report describes every relevant variable, numeric and categorical alike.
3. Yes — since every participant has exactly one recorded sex in this dataset, the two percentages should sum to 100.

99. Level 7 Capstone: A Descriptive Statistics Report

4. Numbers are easy to transpose, mis-round, or misremember when translating from code output into prose — a final verification pass catches these errors before they reach a reader who may take the report at face value.

Part VIII.

Level 8: Data Visualization with ggplot2

Level 8 — Data Visualization with `ggplot2`

Where this level is headed

A well-designed figure can say more than a page of tables — and a poorly designed one can mislead a whole room of stakeholders. This level teaches the grammar of graphics through `ggplot2`, building from a single scatter-plot to publication-ready, multi-panel figures.

What you'll build toward

You'll return to `participants.csv` (and revisit the multi-visit data from Level 6) to build every major chart type in `ggplot2`: scatter plots, trend lines, bar charts, histograms and density curves, boxplots, and line plots for change over time. You'll then learn to control how a plot looks — titles, labels, themes, colors, legends, and faceting into multiple panels — before saving a finished figure to a file and combining everything into one publication-style capstone figure.

Topics in this level

- The grammar of graphics: `ggplot()` and `aes()`
- `geom_point()` for scatter plots, and `geom_smooth()` for trend lines
- `geom_bar()` for categorical counts

- `geom_histogram()` and `geom_density()` for distributions
- `geom_boxplot()` for comparing groups
- `geom_line()` for change over time
- Titles, labels, and themes
- Colors, grouping, and legends
- Faceting into multi-panel figures
- Saving plots with `ggsave()`

A preview mission

Your mission across this level: show your research team how blood pressure differs by age and smoking status — in one figure clear enough to stand on its own in a report, built entirely from the grammar of graphics this level teaches.

100. The Grammar of Graphics

The Problem

Level 7 found a weak positive correlation (0.16) between BMI and systolic blood pressure using `cor()` — a single number. Your professor wants to actually **see** that relationship: a picture showing every participant’s BMI plotted against their blood pressure, so the pattern (or lack of one) is visible at a glance rather than summarized into one coefficient.

Your Mission

Build your first plot using `ggplot2`’s “grammar of graphics” — a system for describing any chart as data, a coordinate mapping, and a visual representation.

Think Before Coding

- To plot BMI against blood pressure, which variable would naturally belong on the x-axis, and which on the y-axis?
- A correlation number (Level 7) summarizes a relationship in one value. What might a plot show you that a single number like 0.16 cannot?

100.1. The R Concept

`ggplot2` (part of the tidyverse) builds every plot from the same three ingredients, specified in layers:

100. The Grammar of Graphics

```
ggplot(data = df, mapping = aes(x = x_column, y = y_column)) +  
  geom_point()
```

- `ggplot(data = df)` says *which data frame* to plot.
- `aes(x = ..., y = ...)` (“aesthetic mapping”) says *which columns* map to *which visual properties* — here, position on the x- and y-axes.
- `geom_point()` says *how* to represent each row — as a point, for a scatter plot.

These pieces are combined with `+`, not `|>` — `ggplot2` predates the tidyverse’s pipe-based style and uses its own layering syntax, adding one visual element at a time.

100.2. Demonstration

```
library(ggplot2)
```

```
participants <- read.csv("data/participants.csv")
```

```
ggplot(data = participants, mapping = aes(x = bmi, y = systolic_bp)) +  
  geom_point()
```

This produces a scatter plot: one point per participant, BMI along the x-axis and systolic blood pressure along the y-axis. Consistent with Level 7’s weak correlation (0.16), the points form a loose, scattered cloud without a strong visible upward or downward trend — a picture that confirms, rather than contradicts, what `cor()` already reported numerically.

100.3. Explore

Try swapping which variable goes on which axis: `aes(x = systolic_bp, y = bmi)`. The overall shape of the scattered cloud looks similar either way

(correlation doesn't have a "direction" in that sense) — but by convention, the variable you consider more likely to *predict* or *explain* the other (here, BMI) usually goes on the x-axis.

100.4. Solve the Problem

```
library(ggplot2)

participants <- read.csv("data/participants.csv")

bmi_bp_plot <- ggplot(data = participants, mapping = aes(x = bmi, y = systolic_bp)) +
  geom_point()

bmi_bp_plot
```

Saving the plot to a named object (`bmi_bp_plot`) — rather than only ever creating and immediately displaying it — lets you keep building on it in later pages (adding a title, a trend line, or color grouping) without retyping the base layer each time.

100.5. Interpret the Result

The scatter plot visually confirms Level 7's numeric finding: BMI and systolic blood pressure show only a weak relationship in this sample — points are spread broadly across the plot rather than clustering along a clear diagonal line. A plot like this is often more convincing to a non-technical audience than a bare correlation coefficient, since the absence of a strong pattern is immediately visible rather than requiring the reader to interpret what "0.16" means.

Common Mistakes

- **Forgetting `library(ggplot2)`.** Like every tidyverse package, `ggplot2` must be loaded before use — `ggplot()` alone won't be recognized otherwise.
- **Using `%>` or `|>` between `ggplot2` layers instead of `+`.** `ggplot2` layers combine with `+`, not the pipe — a very common point of confusion for anyone who has just learned Level 5's and 6's `|>`-based chaining.
- **Putting `aes()` mappings outside `aes()`.** Writing `geom_point(x = bmi, y = systolic_bp)` (without `aes()`) tries to treat `bmi` and `systolic_bp` as fixed values, not column mappings, and typically produces an error — column mappings always belong inside `aes()`.

100.6. Practice Problem

Build a scatter plot of `age` (x-axis) against `systolic_bp` (y-axis). Based on Level 7's correlation (-0.22), would you expect a stronger or weaker visible pattern than the BMI-blood pressure plot above?

100.7. Challenge Problem

Build a scatter plot of `systolic_bp` against `diastolic_bp`. Given Level 7's much stronger correlation (0.92) between these two variables, describe in words how you'd expect this plot's point pattern to look different from the BMI-blood pressure plot above.

100.8. Key Takeaways

- Every `ggplot2` plot starts with `ggplot(data = df, aes(x = ..., y = ...))`, then adds a `geom_*()` layer describing how to represent the data visually.

- `aes()` maps data columns to visual properties (position, color, size, etc.); `ggplot2` layers are combined with `+`, not the pipe.
- A plot can make a numeric relationship (like a correlation coefficient) immediately visible in a way a single number cannot.

Knowledge Check

1. **Multiple choice.** What does `aes(x = bmi, y = systolic_bp)` specify?
 - a) That `bmi` and `systolic_bp` should be added together
 - b) Which columns map to the x- and y-axis positions
 - c) The plot's title
 - d) The colors of each point
2. **True or false.** `ggplot2` layers are combined using the pipe operator (`|>`).
3. **Predict the output.** What geometry (`geom`) would you add to represent each row as a single point on a scatter plot?
4. **Find the error.**

```
ggplot(data = participants) +  
  geom_point(x = bmi, y = systolic_bp)
```

Show answers

1. **b)** Which columns map to the x- and y-axis positions.
2. **False.** `ggplot2` layers are combined with `+`, not the pipe.
3. `geom_point()`.
4. The column mappings are missing `aes()` — writing `geom_point(x = bmi, y = systolic_bp)` doesn't tell `ggplot2` these are column names to map. It should be `geom_point(mapping = aes(x = bmi, y = systolic_bp))`, or better, specify `aes()` once in the initial `ggplot()` call.

101. Adding Trend Lines with `geom_smooth()`

The Problem

The BMI-blood pressure scatter plot from the previous page shows a scattered cloud of points, but your professor asks a harder question to answer just by squinting at the cloud: “Is there an overall upward trend hiding in that scatter, even if it’s weak?” You need a line summarizing the general direction of the relationship, layered right on top of the raw points.

Your Mission

Add a trend line to a scatter plot using `geom_smooth()`, layered on top of `geom_point()`.

Think Before Coding

- If `ggplot2` layers stack using `+`, what would you expect happens if you add a second `geom_*()` layer after `geom_point()`, using the exact same `aes()` mapping?
- Level 7’s correlation between BMI and blood pressure was positive (0.16), though weak. What direction would you expect a trend line summarizing that relationship to slope?

101.1. The R Concept

`geom_smooth()` adds a fitted trend line (with a shaded confidence band by default) on top of existing data, layered with `+` exactly like any other geom:

```
ggplot(data = df, aes(x = x_col, y = y_col)) +  
  geom_point() +  
  geom_smooth(method = "lm")
```

`method = "lm"` fits a straight line (linear model, previewing Level 11); omitting `method` lets `ggplot2` choose a curved, more flexible trend line instead, which can be useful for spotting non-linear patterns a straight line would miss.

101.2. Demonstration

```
library(ggplot2)
```

```
participants <- read.csv("data/participants.csv")
```

```
ggplot(data = participants, mapping = aes(x = bmi, y = systolic_bp)) +  
  geom_point() +  
  geom_smooth(method = "lm")
```

This adds a straight trend line through the scattered points, sloping gently upward — consistent with the weak positive correlation (0.16) from Level 7 — along with a shaded gray band showing the uncertainty around that line. Because the underlying relationship is weak, the line's upward slope is subtle, and the confidence band is fairly wide relative to the data's spread.

101.3. Explore

Try the same plot with `age` and `systolic_bp` instead of `bmi`. Given Level 7's correlation there was **negative** (-0.22, both weak but opposite in sign from the BMI relationship), predict which direction the trend line will slope before running the code, then confirm.

101.4. Solve the Problem

```
library(ggplot2)

participants <- read.csv("data/participants.csv")

age_bp_trend <- ggplot(data = participants, mapping = aes(x = age, y = systolic_bp)) +
  geom_point() +
  geom_smooth(method = "lm")

age_bp_trend
```

The trend line here slopes gently *downward* as age increases — matching the negative correlation (-0.22) found in Level 7. As with the BMI plot, the relationship is weak enough that the line's slope is subtle and the surrounding confidence band is fairly wide, visually reinforcing that this isn't a strong, dependable pattern.

101.5. Interpret the Result

Both trend lines make Level 7's correlation coefficients visually concrete: a gentle upward line for the weak positive BMI-BP relationship, and a gentle downward line for the weak negative age-BP relationship. In both cases, the *direction* of the trend line matches the sign of the correlation

101. Adding Trend Lines with `geom_smooth()`

exactly, while its *subtlety* (a nearly flat slope, a wide confidence band) reflects just how weak both relationships are — exactly consistent with correlation magnitudes both under 0.25 in absolute value.

Common Mistakes

- **Interpreting a fitted trend line as proof of a strong relationship.** A `geom_smooth()` line can be drawn through *any* scatter of points, even ones with essentially no real relationship — always look at how tightly the points hug the line, and how wide the confidence band is, not just whether a line was drawn at all.
- **Forgetting `method = "lm"` fits a straight line specifically.** Without it, `geom_smooth()` defaults to a flexible curved fit (`loess`), which can look more dramatic than a straight-line relationship actually warrants — be explicit about which you intend.
- **Adding `geom_smooth()` without `geom_point()` first and expecting to see the raw data.** `geom_smooth()` alone only shows the fitted line and confidence band — without `geom_point()`, the underlying scatter of individual observations disappears entirely.

101.6. Practice Problem

Build a scatter plot with a `geom_smooth(method = "lm")` trend line for `systolic_bp` against `diastolic_bp`. Given Level 7's much stronger correlation (0.92) here, describe how you'd expect the points to hug the trend line more tightly than in either plot above.

101.7. Challenge Problem

Build the same BMI-blood pressure plot twice: once with `geom_smooth(method = "lm")` and once with `geom_smooth()` (no `method` specified, defaulting to `loess`). Compare the two trend lines — do they tell a meaningfully

different story for this particular relationship, or do they look similar given how weak the underlying pattern is?

101.8. Key Takeaways

- `geom_smooth()` adds a fitted trend line (with a confidence band) on top of a scatter plot, layered with `+`.
- `method = "lm"` fits a straight line; omitting `method` fits a more flexible curve, useful for spotting non-linear patterns.
- A trend line's slope direction should match the sign of the corresponding `cor()` value — always check the two against each other.

Knowledge Check

1. **Multiple choice.** What does `method = "lm"` specify inside `geom_smooth()`?
 - a) A flexible curved fit
 - b) A straight-line (linear) fit
 - c) The plot's color scheme
 - d) The confidence level
2. **True or false.** A `geom_smooth()` trend line can be drawn even when the underlying relationship between two variables is very weak.
3. **Predict the output.** If two variables have a correlation of -0.22, which direction would you expect their `geom_smooth(method = "lm")` trend line to slope?
4. **Find the error.**

```
ggplot(data = participants, aes(x = bmi, y = systolic_bp)) +
  geom_smooth(method = "lm")
# intended to show both the raw data points and the trend line
```

101. Adding Trend Lines with `geom_smooth()`

Show answers

1. **b)** A straight-line (linear) fit.
2. **True.** A trend line can be fit through any scatter of points, regardless of how weak the actual relationship is — the confidence band's width and how loosely points scatter around the line are what reveal that weakness.
3. Downward (negative slope), since a negative correlation means the variables tend to move in opposite directions.
4. `geom_point()` is missing — this code only shows the trend line and confidence band, not the individual data points. It should be `geom_point() + geom_smooth(method = "lm")`.

102. Bar Charts with `geom_bar()`

The Problem

Level 7's `table(participants$county)` printed five county names next to five counts — accurate, but a wall of numbers your professor wants turned into something a reader can absorb in half a second: a chart showing at a glance which county contributed the most participants, and which the fewest.

Your Mission

Build a bar chart of category counts using `geom_bar()`.

Think Before Coding

- Level 7's `table(participants$county)` already counted rows per county for you. Does `geom_bar()` need you to count rows yourself first, or can it count them automatically?
- If `county` goes on the x-axis, what would you expect the y-axis to represent, even though you never explicitly mapped anything to `y`?

102.1. The R Concept

`geom_bar()` counts the number of rows for each category **automatically** — you only map the categorical variable to `x`; `ggplot2` handles the counting internally, similar to how `table()` did in Level 4:

102. Bar Charts with `geom_bar()`

```
ggplot(data = df, aes(x = category_column)) +  
  geom_bar()
```

This is different from `geom_point()` and `geom_smooth()`, which plot values that already exist in your data — `geom_bar()` computes a new value (the count) as part of drawing the plot.

102.2. Demonstration

```
library(ggplot2)  
  
participants <- read.csv("data/participants.csv")  
  
ggplot(data = participants, mapping = aes(x = county)) +  
  geom_bar()
```

This produces five bars, one per county, with height equal to that county's participant count — Springvale's bar is the tallest (26 participants), and Riverdale's is the shortest (16 participants), matching Level 5's `count(participants, county)` exactly.

102.3. Explore

Try adding `+ coord_flip()` to the plot above. This rotates the chart so county names run along the y-axis and bars extend horizontally — often more readable when category names (like “Cedar Falls”) are long enough to overlap when printed underneath vertical bars.

102.4. Solve the Problem

```
library(ggplot2)

participants <- read.csv("data/participants.csv")

county_bar_chart <- ggplot(data = participants, mapping = aes(x = county)) +
  geom_bar()

county_bar_chart
```

Five bars in the order R’s default alphabetical sorting places county names: Cedar Falls (20), Millbrook (21), Oakford (17), Riverdale (16), Springvale (26) — Springvale’s bar visibly the tallest, Riverdale’s the shortest, exactly matching the raw counts from Level 4’s `table()`.

102.5. Interpret the Result

The bar chart makes the county size ranking immediately visible without requiring a reader to compare five separate numbers by eye — Springvale’s larger contribution to the study (26 of 100 participants) and Riverdale’s smaller one (16 of 100) are now a visual fact rather than a row in a table. This is exactly the kind of quick, scannable summary a report’s opening figure often needs.

Common Mistakes

- **Mapping y to a count that doesn’t exist yet.** `geom_bar()` computes counts internally — writing `aes(x = county, y = count)` looks for a column literally named `count` in your data frame, which doesn’t exist, and produces an error. Just map `x`; let `geom_bar()` handle the rest.

102. Bar Charts with `geom_bar()`

- **Confusing `geom_bar()` with `geom_col()`.** `geom_bar()` counts rows automatically; `geom_col()` expects you to have *already* computed the heights yourself (for example, with Level 5's `summarise()`) and maps a column directly to bar height instead of counting.
- **Leaving bars in default alphabetical order when a ranked order would communicate more clearly.** Sorting bars by height (largest to smallest) is often more readable than alphabetical order — a refinement covered later in this level alongside labels and themes.

102.6. Practice Problem

Build a bar chart of `smoker` status. With only two categories (“No” and “Yes”), confirm the relative bar heights visually match the 75/25 split found in Level 7.

102.7. Challenge Problem

Level 5's `count(participants, county, sort = TRUE)` already sorts counties by size. Research (or predict from what you already know about `ggplot2`'s “factor” handling) how you might reorder `geom_bar()`'s x-axis categories by count rather than alphabetically, without changing the underlying data.

102.8. Key Takeaways

- `geom_bar()` automatically counts rows per category — map only `x` to the categorical column, and let `ggplot2` compute the bar heights.

- `geom_bar()` (auto-counting) differs from `geom_col()` (plotting pre-computed values) — choosing the right one depends on whether your data already has the heights you want to plot.
- A bar chart turns a `table()`-style count comparison into an instantly scannable visual ranking.

Knowledge Check

1. **Multiple choice.** What does `geom_bar()` do when you map only `x` to a categorical column?
 - a) It errors, requiring a `y` mapping too
 - b) It automatically counts rows per category and uses that as bar height
 - c) It plots every row as a separate point
 - d) It computes the mean of `x` per category
2. **True or false.** `geom_bar()` and `geom_col()` work identically in every situation.
3. **Predict the output.** Given county counts of 20, 21, 17, 16, and 26, which county's bar would be tallest?
4. **Find the error.**

```
ggplot(data = participants, aes(x = county, y = count)) +  
  geom_bar()
```

Show answers

1. **b)** It automatically counts rows per category and uses that count as the bar height.
2. **False.** `geom_bar()` counts rows automatically; `geom_col()` requires the bar heights to already exist as a column in the data — they serve different purposes.
3. Springvale, with 26 participants — the largest count among the five counties.

102. Bar Charts with `geom_bar()`

4. There is no column literally named `count` in `participants` — `geom_bar()` computes counts internally and doesn't need (or expect) a `y` mapping at all. It should simply be `aes(x = county)`.

103. Histograms and Density Plots

The Problem

Level 7's `summary(participants$age)` reported a minimum of 18, a maximum of 81, a mean of 47.43, and a median of 49 — but none of those five numbers show you the actual *shape* of the age distribution. Are ages spread evenly across that range? Clustered in the middle? Your professor wants to see the full distribution, not just its five-number summary.

Your Mission

Visualize the distribution of a numeric variable using `geom_histogram()` and `geom_density()`.

Think Before Coding

- A histogram groups continuous values into “bins” (like “40-49 years old”) and counts how many observations fall in each. What single argument would you expect controls how wide those bins are?
- If a histogram shows bar *counts* per bin, what would a smoothed curve tracing the same overall shape, without discrete bins, look like instead?

103.1. The R Concept

`geom_histogram()` divides a numeric variable into bins and counts how many observations fall into each — visually, like `geom_bar()` but for continuous rather than categorical data:

103. Histograms and Density Plots

```
ggplot(data = df, aes(x = numeric_column)) +  
  geom_histogram(binwidth = 10)
```

`binwidth` controls how wide each bin is — too wide, and you lose detail; too narrow, and the shape becomes noisy and hard to read.

`geom_density()` shows the same kind of information as a smoothed curve instead of discrete bars — useful for seeing the overall shape without committing to a specific bin width:

```
ggplot(data = df, aes(x = numeric_column)) +  
  geom_density()
```

103.2. Demonstration

```
library(ggplot2)
```

```
participants <- read.csv("data/participants.csv")
```

```
ggplot(data = participants, mapping = aes(x = age)) +  
  geom_histogram(binwidth = 10)
```

With 10-year bins, this produces bars roughly following this pattern: a small bar for ages 10-19 (5 participants), rising through 20-29 (8), 30-39 (16), peaking around 40-49 (20) and 50-59 (24), then declining through 60-69 (16), 70-79 (3), and 80-89 (2) — a distribution that rises to a peak in the 40s-50s and tapers off on both sides, roughly consistent with the mean (47.43) and median (49) sitting near that peak.

103.3. Explore

Try the same plot with `binwidth = 5` instead of 10. Finer bins reveal more detail (and more visual noise) — compare how much the overall

“rises then falls, peaking in the 40s-50s” shape still holds versus how much finer fluctuation appears.

103.4. Solve the Problem

```
library(ggplot2)

participants <- read.csv("data/participants.csv")

age_histogram <- ggplot(data = participants, mapping = aes(x = age)) +
  geom_histogram(binwidth = 10)

age_histogram

age_density <- ggplot(data = participants, mapping = aes(x = age)) +
  geom_density()

age_density
```

The histogram shows discrete 10-year bars rising to a peak around 40-59 and tapering at both ends; the density plot shows the same overall shape as a single smooth curve, without the bin-width choice affecting how it looks.

103.5. Interpret the Result

Age in this sample isn’t uniformly spread across 18-81 — it’s concentrated in the 40s and 50s, with progressively fewer participants at the youngest and oldest ends. This is exactly the kind of shape information Level 7’s five-number summary alone couldn’t show: two datasets could share an identical mean, median, and range while having completely different distribution shapes (one peaked in the middle, one flat, one with two separate

103. Histograms and Density Plots

peaks) — only a histogram or density plot reveals which one you actually have.

Common Mistakes

- **Choosing a binwidth without trying a few options first.** Too wide a bin can hide real structure (like a second peak); too narrow can make the plot look noisy and uninterpretable — always try 2-3 different bin widths before settling on one for a final report.
- **Forgetting `geom_histogram()` needs only `x`, not `y`.** Like `geom_bar()`, `geom_histogram()` computes bin counts (the y-axis) automatically — mapping a `y` variable yourself isn't necessary and usually isn't what you want.
- **Treating a density plot's y-axis as a literal count.** A density curve's y-axis is scaled so the total area under the curve equals 1 — it should be read as showing *relative* likelihood across values, not as a raw count the way a histogram's y-axis is.

103.6. Practice Problem

Build a histogram of `bmi` with `binwidth = 5`. Based on Level 7's finding that BMI's mean (27.12) and median (27.70) sit close together, would you expect a roughly symmetric shape, or a heavily skewed one?

103.7. Challenge Problem

Build a histogram of `systolic_bp` with three different bin widths (5, 10, and 20). Describe in words how the visual impression of the distribution's shape changes as bin width increases — at what point does the histogram start losing meaningful detail?

103.8. Key Takeaways

- `geom_histogram(binwidth = ...)` groups a numeric variable into bins and shows counts per bin, revealing distribution shape that a five-number summary alone cannot.
- `geom_density()` shows the same kind of shape information as a smooth curve, without needing to choose a bin width.
- Two variables can share identical means, medians, and ranges while having completely different distribution shapes — always check the shape directly rather than assuming it from summary statistics alone.

Knowledge Check

1. **Multiple choice.** What does `binwidth` control in `geom_histogram()`?
 - a) The color of the bars
 - b) How wide each bin (group of values) is
 - c) The number of rows in the data
 - d) The plot's title
2. **True or false.** A density plot's y-axis represents a raw count, exactly like a histogram's.
3. **Predict the output.** If age is concentrated in the 40s and 50s with fewer participants at the extremes, what general shape would you expect the histogram to show?
4. **Find the error.**

```
ggplot(data = participants, aes(x = age, y = count)) +  
  geom_histogram(binwidth = 10)
```

Show answers

1. **b)** How wide each bin (group of values) is.

103. Histograms and Density Plots

2. **False.** A density plot's y-axis is scaled so the total area under the curve equals 1, representing relative likelihood rather than a raw count.
3. A shape that rises toward a peak in the 40s-50s and tapers off toward both younger and older ages.
4. `geom_histogram()` computes bin counts automatically — there is no `count` column to map, and specifying `y` isn't necessary. It should simply be `aes(x = age)`.

104. Boxplots with `geom_boxplot()`

The Problem

Your professor wants to compare BMI between smokers and non-smokers — not just their means, but their full distributions side by side: medians, middle 50%, and any extreme values in each group. A histogram (previous page) only shows one variable at a time; comparing two groups would mean building two separate histograms and eyeballing them against each other.

Your Mission

Compare a numeric variable's distribution across groups using `geom_boxplot()`.

Think Before Coding

- From Level 7, a box in a boxplot typically represents the middle 50% of the data. Which two values from Level 7's `quantile()` page define the edges of that box?
- If you wanted one boxplot per smoking status, side by side, which variable would you map to `x`, and which to `y`?

104.1. The R Concept

`geom_boxplot()` draws a box spanning the 1st to 3rd quartile (the IQR, Level 7), a line at the median, and “whiskers” extending to the most

104. Boxplots with `geom_boxplot()`

extreme non-outlier values — with individual points beyond the whiskers flagged as potential outliers, echoing Level 7's $1.5 \times \text{IQR}$ rule:

```
ggplot(data = df, aes(x = group_column, y = numeric_column)) +  
  geom_boxplot()
```

Mapping a categorical variable to `x` and a numeric variable to `y` produces one boxplot per category, side by side for direct comparison.

104.2. Demonstration

```
library(ggplot2)
```

```
participants <- read.csv("data/participants.csv")
```

```
ggplot(data = participants, mapping = aes(x = smoker, y = bmi)) +  
  geom_boxplot()
```

This produces two boxplots side by side: non-smokers' box spans roughly 23.3 to 29.65 (median 27.8), while smokers' box spans roughly 24.12 to 30.32 (median 26.25) — visibly similar in spread, with the non-smoker group's median sitting slightly higher than the smoker group's.

104.3. Explore

Try `ggplot(data = participants, aes(x = county, y = systolic_bp)) + geom_boxplot()` — five boxplots side by side instead of two. This extends the same idea from Level 5's `group_by(county) + summarise()` grouped means, but shows each county's *full* blood pressure distribution rather than collapsing it to a single number.

104.4. Solve the Problem

```
library(ggplot2)

participants <- read.csv("data/participants.csv")

bmi_smoker_boxplot <- ggplot(data = participants, mapping = aes(x = smoker, y = bmi)) +
  geom_boxplot()

bmi_smoker_boxplot
```

Non-smokers (n=71): median BMI 27.8, middle 50% spanning roughly 23.3-29.65, full range 15.6-38.6. Smokers (n=24): median BMI 26.25, middle 50% spanning roughly 24.12-30.32, full range 18.1-35.8. The two boxes overlap substantially, visually confirming that BMI doesn't differ dramatically by smoking status in this sample.

104.5. Interpret the Result

The boxplot shows the two groups' BMI distributions are broadly similar — overlapping boxes and comparable medians — rather than one group being dramatically higher or lower than the other. This is a more complete picture than comparing two means alone: it also shows that non-smokers have a slightly wider full range (down to 15.6) than smokers (down to 18.1), a detail a bare mean-comparison would miss entirely.

Common Mistakes

- **Reading the line inside the box as the mean.** The line inside a `geom_boxplot()` box is the **median**, not the mean — the two can differ, especially for skewed data, and conflating them leads to a misreading of the plot.

104. Boxplots with `geom_boxplot()`

- **Assuming points beyond the whiskers are always errors.** Points plotted individually beyond a boxplot's whiskers are flagged as *potential* outliers by the $1.5 \times \text{IQR}$ rule (Level 7) — worth investigating, not automatically wrong or removable.
- **Comparing boxplots without checking group sizes.** A boxplot for a group of 5 participants and one for a group of 95 look visually similar in a plot, even though the smaller group's summary is far less stable — always report each group's `n` alongside a boxplot comparison.

104.6. Practice Problem

Build a boxplot comparing `systolic_bp` between smokers and non-smokers. Does blood pressure appear to differ more between smoking groups than BMI did?

104.7. Challenge Problem

Build a boxplot of `bmi` across all five counties (`aes(x = county, y = bmi)`). Combine this with Level 5's `group_by(county) + summarise()` output for mean BMI per county, and check whether the boxplot's visual median ranking matches the numeric mean ranking — they don't always agree, especially for skewed distributions.

104.8. Key Takeaways

- `geom_boxplot()` shows the median, interquartile range, and potential outliers for a numeric variable, one box per category when a categorical variable is mapped to `x`.

- The line inside the box is the median, not the mean — don't conflate the two when reading a boxplot.
- Comparing full distributions (via boxplots) reveals more than comparing single summary statistics like means alone.

Knowledge Check

1. **Multiple choice.** What does the line inside a `geom_boxplot()` box represent?
 - a) The mean
 - b) The median
 - c) The mode
 - d) The maximum value
2. **True or false.** Points plotted beyond a boxplot's whiskers are always data entry errors.
3. **Predict the output.** If you map a categorical variable with 5 categories to `x` and a numeric variable to `y`, how many boxes would `geom_boxplot()` draw?
4. **Find the error.**

```
ggplot(data = participants, aes(x = bmi, y = smoker)) +
  geom_boxplot()
# intended: one BMI boxplot per smoking status, side by side
```

Show answers

1. **b)** The median.
2. **False.** Points beyond the whiskers are *potential* outliers worth investigating, not automatically confirmed errors.
3. 5 boxes, one per category.

104. Boxplots with `geom_boxplot()`

4. The `x` and `y` mappings are reversed — `smoker` (the categorical grouping variable) should be on `x`, and `bmi` (the numeric variable being compared) should be on `y`. It should be `aes(x = smoker, y = bmi)`.

105. Line Plots with `geom_line()`

The Problem

Level 6's reshaped visit data showed mean systolic blood pressure dropping from 124.96 at visit 1, to 120.96 at visit 2, to 120.69 at visit 3 — three numbers in a table. Your professor wants this change over time as a picture: a line tracing the trend across visits, immediately showing whether the drop leveled off or kept falling.

Your Mission

Visualize a trend over an ordered sequence (like visit number) using `geom_line()`.

Think Before Coding

- `geom_point()` (Page 1) draws unconnected points. What would you expect `geom_line()` to add that a scatter plot alone doesn't show?
- For a line plot to draw a single connected line across visits, does the data need to already be sorted by visit, or does `ggplot2` handle that automatically based on the x-axis values?

105.1. The R Concept

`geom_line()` connects data points in order along the x-axis — appropriate when there's a natural sequence (like time or visit number) connecting the values:

105. Line Plots with `geom_line()`

```
ggplot(data = df, aes(x = ordered_variable, y = numeric_column)) +  
  geom_line()
```

Unlike `geom_point()`, `geom_line()` assumes consecutive x-axis values belong to the same continuous trend — so it's the right choice for a variable like “visit number,” but not for an unordered category like `county`.

105.2. Demonstration

```
library(ggplot2)  
library(dplyr)  
library(tidyr)  
  
bp_wide <- read.csv("data/bp_visits_wide.csv")  
  
bp_long <- pivot_longer(bp_wide,  
  cols = starts_with("sbp_visit"),  
  names_to = "visit",  
  names_prefix = "sbp_",  
  values_to = "sbp"  
)  
  
visit_means <- bp_long |>  
  group_by(visit) |>  
  summarise(mean_sbp = mean(sbp, na.rm = TRUE))  
  
ggplot(data = visit_means, mapping = aes(x = visit, y = mean_sbp, group = 1)) +  
  geom_line() +  
  geom_point()
```

This draws a line connecting three points — one per visit — dropping from about 124.96 at visit 1 to 120.96 at visit 2, then leveling off only

slightly further to 120.69 at visit 3: a clear visual of the pattern Level 6 first found in table form.

105.3. Explore

Try removing `group = 1` from the `aes()` mapping and re-running the plot. Since `visit` is a character column (not a single continuous numeric sequence), `ggplot2` doesn't automatically know all three points belong to one connected line without `group = 1` telling it so explicitly.

105.4. Solve the Problem

```
library(ggplot2)
library(dplyr)
library(tidyr)

bp_wide <- read.csv("data/bp_visits_wide.csv")

bp_long <- pivot_longer(bp_wide,
  cols = starts_with("sbp_visit"),
  names_to = "visit",
  names_prefix = "sbp_",
  values_to = "sbp"
)

visit_means <- bp_long |>
  group_by(visit) |>
  summarise(mean_sbp = round(mean(sbp, na.rm = TRUE), 2))

visit_means
```

105. Line Plots with `geom_line()`

```
# A tibble: 3 x 2
  visit mean_sbp
  <chr>   <dbl>
1 visit1  124.96
2 visit2  120.96
3 visit3  120.69
```

```
visit_trend_plot <- ggplot(data = visit_means, mapping = aes(x = visit, y = r
  geom_line() +
  geom_point()
```

```
visit_trend_plot
```

A clearly declining line from visit 1 to visit 2, followed by a much flatter segment from visit 2 to visit 3 — the shape of the improvement isn’t linear across all three visits; most of the drop happens early, then levels off.

105.5. Interpret the Result

The line plot reveals something the three raw numbers alone made you compute mentally: blood pressure didn’t decline at a steady rate across visits — most of the improvement happened between visit 1 and visit 2, with much less additional change by visit 3. This “front-loaded improvement, then plateau” shape is a genuinely useful clinical observation that’s far easier to see in a line plot than in a table of three numbers.

Common Mistakes

- **Forgetting `group = 1` when the x-axis variable is categorical (like “visit1”, “visit2”, “visit3”).** Without it, `ggplot2` may not connect the points into a single line, since it doesn’t automatically assume unrelated categorical labels form one continuous sequence.

- **Using `geom_line()` for genuinely unordered categories.** A line plot implies a meaningful order and connection between consecutive x-axis values — using it for something like `county` (which has no natural order) would visually imply a trend that doesn't exist.
- **Plotting raw individual-participant data with `geom_line()` instead of the summarised means.** Connecting each of the 28 individual participants' three visits with `geom_line()` (rather than the grouped mean) would produce 28 overlapping lines — useful for some purposes, but a very different (and much busier) plot than the clean mean trend shown here.

105.6. Practice Problem

Reshape and plot the mean `weight` (instead of `sbp`) across the three visits using `geom_line()`. Does weight show a similar “front-loaded change, then plateau” pattern?

105.7. Challenge Problem

Build a line plot showing each of the 28 individual participants' blood pressure across the three visits (map `id` to `group`, and consider adding `alpha = 0.3` to `geom_line()` to make the overlapping lines more readable). How does this “spaghetti plot” of individual trajectories compare to the single clean mean-trend line above?

105.8. Key Takeaways

- `geom_line()` connects points in x-axis order, appropriate for a naturally sequential variable like visit number or time.

105. Line Plots with `geom_line()`

- `group = 1` (or another grouping variable) tells `ggplot2` which points belong to the same connected line, especially important when the x-axis is categorical.
- A line plot can reveal a trend's *shape* (front-loaded change, a plateau, a steady decline) that a table of the same numbers requires more effort to notice.

Knowledge Check

1. **Multiple choice.** When is `geom_line()` an appropriate choice?
 - a) For any categorical variable, regardless of order
 - b) When there's a meaningful sequence connecting the x-axis values, like time or visit number
 - c) Only for scatter plots
 - d) Only when there are more than 100 data points
2. **True or false.** `geom_line()` automatically knows to connect all points into a single line, even when the x-axis is a character column, without any extra arguments.
3. **Predict the output.** Given mean blood pressure of 124.96, 120.96, and 120.69 across three visits, does the largest single-visit drop happen between visit 1-2, or visit 2-3?
4. **Short answer.** Why would `geom_line()` be a poor choice for plotting mean BMI across the five counties?

Show answers

1. **b)** When there's a meaningful sequence connecting the x-axis values, like time or visit number.
2. **False.** For a categorical (character) x-axis, `ggplot2` typically needs an explicit `group` mapping (like `group = 1`) to know all the points belong to one connected line.

105.8. Key Takeaways

3. Visit 1 to visit 2 ($124.96 \rightarrow 120.96$, a drop of about 4.0), compared to visit 2 to visit 3 ($120.96 \rightarrow 120.69$, a drop of only about 0.27).
4. Counties have no natural order or sequential relationship to each other — connecting them with a line would visually imply a trend or progression from one county to the next that doesn't actually exist; a bar chart is the more honest choice for unordered categories.

106. Titles, Labels, and Themes

The Problem

Every plot built so far works, but none of them would survive being dropped into an actual report: the axes are labeled with raw column names (`bmi`, `systolic_bp`) instead of readable descriptions, there's no title explaining what the plot shows, and the default gray background isn't what most published figures use. Your professor's report needs plots a reader can understand without also reading your R code.

Your Mission

Add readable titles, axis labels, and a cleaner visual theme to a plot using `labs()` and `theme_*`() functions.

Think Before Coding

- Looking at any plot from this level so far, what would a reader unfamiliar with your dataset's column names find confusing about the axis labels?
- If `ggplot2` layers combine with `+`, would you expect adding a title to also just be one more `+`-connected layer?

106.1. The R Concept

`labs()` sets a plot's title, subtitle, axis labels, and caption, all in one layer:

106. Titles, Labels, and Themes

```
+ labs(  
  title = "Plot Title",  
  subtitle = "Optional subtitle",  
  x = "Readable X-Axis Label",  
  y = "Readable Y-Axis Label",  
  caption = "Optional footnote"  
)
```

`theme_*`() functions replace `ggplot2`'s default gray-background theme with alternatives — `theme_minimal()`, `theme_bw()`, and `theme_classic()` are common, cleaner choices for reports:

```
+ theme_minimal()
```

106.2. Demonstration

```
library(ggplot2)
```

```
participants <- read.csv("data/participants.csv")
```

```
ggplot(data = participants, mapping = aes(x = bmi, y = systolic_bp)) +  
  geom_point() +  
  labs(  
    title = "BMI and Systolic Blood Pressure",  
    subtitle = "Weak positive relationship (r = 0.16)",  
    x = "Body Mass Index (BMI)",  
    y = "Systolic Blood Pressure (mmHg)"  
  ) +  
  theme_minimal()
```

The same scatter plot from Page 1, now with a clear title stating what's being shown, a subtitle previewing the finding, readable axis labels with units, and a cleaner white background in place of the default gray.

106.3. Explore

Try swapping `theme_minimal()` for `theme_bw()` or `theme_classic()`. All three remove the default gray background, but with subtly different grid-line and border styles — try each and decide which reads most clearly for your own report.

106.4. Solve the Problem

```
library(ggplot2)

participants <- read.csv("data/participants.csv")

county_bar_chart <- ggplot(data = participants, mapping = aes(x = county)) +
  geom_bar() +
  labs(
    title = "Number of Participants by County",
    subtitle = "n = 100 total participants across 5 counties",
    x = "County",
    y = "Number of Participants"
  ) +
  theme_minimal()

county_bar_chart
```

The county bar chart from Page 3, now clearly labeled: a reader immediately knows this shows participant counts by county, out of 100 total, without needing to guess what “county” and the unlabeled y-axis numbers meant.

106.5. Interpret the Result

A plot’s raw content doesn’t change when you add `labs()` and a theme — the same five bars, the same heights — but its *readability* to someone who isn’t you changes enormously. A title and readable axis labels turn a plot that requires you to explain it out loud into one that explains itself.

Common Mistakes

- **Leaving default column names as axis labels in a final report figure.** `bmi` and `systolic_bp` mean nothing to a general reader — always replace them with readable labels and units via `labs(x = ..., y = ...)` before sharing a plot outside your own analysis.
- **Adding a title that describes the chart type instead of the finding.** “Scatter Plot of BMI vs Blood Pressure” restates what’s visually obvious; “Weak Positive Relationship Between BMI and Blood Pressure” tells the reader what to notice — titles are more useful when they communicate the finding, not just the chart type.
- **Forgetting `theme_*`() replaces the *entire* theme, not just one element.** If you want to change just the background but keep other default styling, `theme_minimal()` isn’t a targeted tweak — it’s a wholesale theme swap; individual style tweaks use `theme(...)` (lowercase, no suffix) instead.

106.6. Practice Problem

Add a title, readable axis labels, and `theme_minimal()` to the age histogram from Page 4. Include a subtitle noting the mean and median age found in Level 7.

106.7. Challenge Problem

Take the boxplot comparing BMI by smoking status (Page 5) and add a title, axis labels, and a caption noting the sample sizes in each group (n=71 non-smokers, n=24 smokers). Consider what additional context a reader would need to correctly interpret the plot without seeing your code.

106.8. Key Takeaways

- `labs(title = ..., x = ..., y = ...)` replaces raw column names and adds context, making a plot interpretable without needing to see the underlying code.
- `theme_minimal()`, `theme_bw()`, and `theme_classic()` replace `ggplot2`'s default gray background with cleaner alternatives suited for reports.
- A good title communicates the finding, not just the chart type — “what should the reader notice,” not merely “what kind of chart this is.”

Knowledge Check

1. **Multiple choice.** What does `labs(x = "Body Mass Index (BMI)")` do?
 - a) Filters the data to only BMI values
 - b) Replaces the x-axis label with readable text
 - c) Changes the color of the x-axis
 - d) Removes the x-axis entirely
2. **True or false.** `theme_minimal()` only changes the plot's background color and leaves everything else untouched.

3. **Predict the output.** Would `labs(title = "My Plot") + labs(subtitle = "A subtitle")` (two separate `labs()` calls) work, or does `labs()` need to be called only once?
4. **Short answer.** Why is “Weak Positive Relationship Between BMI and Blood Pressure” a more useful title than “Scatter Plot of BMI and Blood Pressure”?

Show answers

1. **b)** Replaces the x-axis label with readable text.
2. **False.** `theme_*()` functions replace the entire visual theme (grid-lines, borders, fonts, background), not just one specific element.
3. Both work — multiple `labs()` calls combine their arguments, though it’s more common (and clearer) to set all label arguments in a single `labs()` call.
4. The first title tells the reader what pattern to notice in the data; the second only restates the chart type, which is already visually obvious from looking at the plot itself.

107. Colors, Grouping, and Legends

The Problem

Level 7's cross-tabulation found smoking rates differ notably by sex (19.2% women, 31.2% men), but blood pressure doesn't differ much by smoking status on its own (129.61 mmHg for non-smokers, 130.54 for smokers). Your professor wants to see the BMI-blood pressure scatter plot from Page 1 again — but this time with smokers and non-smokers visually distinguished, to check by eye whether the two groups occupy different regions of the plot.

Your Mission

Map a third variable to color, adding a visual grouping dimension to a plot, complete with an automatically generated legend.

Think Before Coding

- `aes()` has mapped variables to `x` and `y` so far. Could a third variable be mapped to a different visual property — like color — using that same `aes()` function?
- If two groups (smokers and non-smokers) are colored differently on the same plot, would you expect `ggplot2` to automatically generate a legend explaining which color means what?

107.1. The R Concept

Mapping a categorical variable to `color` (for points and lines) or `fill` (for bars and boxes) inside `aes()` adds a grouping dimension to a plot, with a legend generated automatically:

```
ggplot(data = df, aes(x = x_col, y = y_col, color = grouping_column)) +  
  geom_point()
```

No extra step is needed for the legend — `ggplot2` builds it automatically from the color mapping, labeled with the grouping column’s actual category values.

107.2. Demonstration

```
library(ggplot2)  
  
participants <- read.csv("data/participants.csv")  
  
ggplot(data = participants, mapping = aes(x = bmi, y = systolic_bp, color = smoker)) +  
  geom_point()
```

This produces the same scatter plot as Page 1, but now every point is colored by smoking status, with a legend on the side labeled “smoker” showing which color corresponds to “Yes” and which to “No.” Visually, the two colors appear thoroughly intermixed across the plot rather than occupying separate regions — consistent with smokers’ and non-smokers’ mean blood pressure (130.54 vs. 129.61) being nearly identical.

107.3. Explore

Try mapping `color = county` instead of `color = smoker` on the same plot. With 5 counties instead of 2 smoking categories, `ggplot2` automatically assigns 5 distinct colors and builds a legend with 5 entries — no additional code needed beyond changing which column is mapped.

107.4. Solve the Problem

```
library(ggplot2)

participants <- read.csv("data/participants.csv")

bmi_bp_by_smoker <- ggplot(data = participants, mapping = aes(x = bmi, y = systolic_bp, color = smoker)) +
  geom_point() +
  labs(
    title = "BMI and Blood Pressure by Smoking Status",
    x = "Body Mass Index (BMI)",
    y = "Systolic Blood Pressure (mmHg)",
    color = "Current Smoker"
  ) +
  theme_minimal()

bmi_bp_by_smoker
```

Note `labs(color = "Current Smoker")` renames the legend title itself (from the default “smoker” to a more readable “Current Smoker”) — `labs()` can relabel any aesthetic, not just axes.

107.5. Interpret the Result

Coloring by smoking status doesn't reveal any visually obvious separation between the two groups on this plot — smokers and non-smokers scatter across the same overall region of BMI and blood pressure values. This visual finding is consistent with Level 7's numeric result: smoking status and blood pressure showed little relationship in this sample. A color mapping doesn't manufacture a pattern that isn't there — it simply makes an existing (or absent) pattern easier to check by eye.

Common Mistakes

- **Using color for a continuous numeric variable when a distinct palette is expected.** Mapping `color` to a categorical variable (like `smoker`) gives discrete colors with a labeled legend; mapping it to a continuous variable (like `age`) instead produces a color *gradient* — a genuinely different kind of legend and visual encoding.
- **Confusing color and fill.** `color` controls outlines and point colors (used for `geom_point()`, `geom_line()`); `fill` controls the interior color of shapes with area (used for `geom_bar()`, `geom_boxplot()`) — using the wrong one for a given geom often produces no visible change at all.
- **Adding a color mapping and assuming it reveals a meaningful group difference.** Sometimes, as here, coloring by a group reveals *no* visible separation — that's a legitimate and useful finding (echoing Level 7's weak smoking-blood pressure relationship), not a failed plot.

107.6. Practice Problem

Recreate the BMI-blood pressure scatter plot with `color = sex` instead of `color = smoker`. Does mapping sex reveal any more visible separation than smoking status did?

107.7. Challenge Problem

Build a boxplot of `bmi` by `county` (Page 5’s technique), adding `fill = county` to color each box differently. Since `county` is already mapped to `x`, is a `fill` legend actually necessary here, or would it be redundant with the x-axis labels? Explain your reasoning.

107.8. Key Takeaways

- Mapping a variable to `color` (or `fill`) inside `aes()` adds a grouping dimension to any plot, with a legend generated automatically.
- `color` applies to outlines and points; `fill` applies to the interior of shapes with area, like bars and boxes.
- A color mapping that reveals no visible group separation is still a legitimate and informative result — it doesn’t mean the plot failed.

Knowledge Check

1. **Multiple choice.** What does mapping `color = smoker` inside `aes()` do?
 - a) Changes the plot’s background color
 - b) Colors points differently based on smoking status, with an automatic legend
 - c) Removes non-smokers from the plot
 - d) Requires a separate `legend()` function call
2. **True or false.** `color` and `fill` always produce the same visual effect, regardless of which geom is used.
3. **Predict the output.** If you map `color = county` (5 categories) instead of `color = smoker` (2 categories), how many legend entries would you expect?

107. Colors, Grouping, and Legends

4. Find the error.

```
ggplot(data = participants, aes(x = smoker, y = bmi)) +  
  geom_boxplot(color = county)
```

Show answers

1. **b)** Colors points differently based on smoking status, with an automatic legend.
2. **False.** `color` affects outlines/points, while `fill` affects the interior of shapes with area — for a `geom_boxplot()`, `fill` typically produces a more visible effect than `color`.
3. 5 legend entries, one per county.
4. `color = county` is placed outside `aes()`, so `ggplot2` tries to interpret `county` as a literal fixed value rather than a column mapping, which errors. It should be inside `aes()`: `geom_boxplot(aes(fill = county))` (using `fill`, since boxplots have an interior area).

108. Faceting into Multiple Panels

The Problem

The previous page colored a single scatter plot by smoking status — fine for 2 groups, but your professor now wants to see the BMI distribution *separately for each of the 5 counties*. Cramming 5 differently colored histograms onto one set of axes would be a cluttered mess. What you actually want is 5 small, identical histograms side by side, one per county — not 5 colors overlaid on one plot.

Your Mission

Split a single plot into multiple small panels, one per category, using `facet_wrap()`.

Think Before Coding

- Page 8's `color` mapping put multiple groups' data *on the same axes*, distinguished only by color. What would you expect happens instead if each group got its *own separate* set of axes?
- With 5 counties, how many small panels would you expect `facet_wrap(~ county)` to produce?

108.1. The R Concept

`facet_wrap()` splits a plot into multiple small panels, one per category of a specified variable — each panel shares the same x- and y-axis scale by default, making the panels directly comparable to each other:

108. Faceting into Multiple Panels

```
ggplot(data = df, aes(x = numeric_column)) +  
  geom_histogram(binwidth = ...) +  
  facet_wrap(~ grouping_column)
```

The ~ (tilde) before the grouping column is required syntax, borrowed from R's formula notation (also used in modeling, previewed in Level 11).

108.2. Demonstration

```
library(ggplot2)  
  
participants <- read.csv("data/participants.csv")  
  
ggplot(data = participants, mapping = aes(x = bmi)) +  
  geom_histogram(binwidth = 5) +  
  facet_wrap(~ county)
```

This produces 5 small histogram panels, one per county, arranged in a grid — each panel showing that county's BMI distribution specifically, all on the same shared x-axis scale so panel-to-panel comparisons remain fair.

108.3. Explore

Try adding `nrow = 1` inside `facet_wrap(~ county, nrow = 1)`, forcing all 5 panels into a single horizontal row instead of `ggplot2`'s default grid arrangement. Compare readability between the default layout and the single-row version.

108.4. Solve the Problem

```
library(ggplot2)

participants <- read.csv("data/participants.csv")

bmi_by_county_facets <- ggplot(data = participants, mapping = aes(x = bmi)) +
  geom_histogram(binwidth = 5) +
  facet_wrap(~ county) +
  labs(
    title = "BMI Distribution by County",
    x = "Body Mass Index (BMI)",
    y = "Number of Participants"
  ) +
  theme_minimal()

bmi_by_county_facets
```

Five panels, each shaped roughly like the overall BMI histogram from Page 4 — but Springvale’s panel visibly shifts further left (lower BMI values) than the other four counties, consistent with Springvale’s mean BMI (25.24) sitting noticeably below Cedar Falls (28.42), Oakford (28.02), Millbrook (27.37), and Riverdale (27.38).

108.5. Interpret the Result

Faceting reveals a pattern the single combined histogram (Page 4) couldn’t show: Springvale’s BMI distribution is shifted toward lower values relative to the other four counties, which look broadly similar to each other. This is exactly the kind of subgroup pattern that a single aggregated plot averages away — faceting trades one plot for several smaller, directly comparable

108. Faceting into Multiple Panels

ones, at the cost of each individual panel having less data (and therefore looking noisier) than the combined version would.

Common Mistakes

- **Forgetting the `~` before the grouping variable.** `facet_wrap(county)` (without the tilde) produces an error — `facet_wrap()` requires formula syntax, `facet_wrap(~ county)`.
- **Faceting by a variable with too many categories.** Faceting by `id` (100 distinct participants) would produce 100 tiny, nearly unreadable panels — faceting works best with a handful of meaningful categories, not a variable that's nearly unique per row.
- **Forgetting that facets share the same axis scale by default, which can hide within-panel detail for smaller groups.** A county with only 16 participants (Riverdale) gets the same y-axis range as one with 26 (Springvale) by default — `scales = "free"` inside `facet_wrap()` allows each panel its own scale, at the cost of making direct height comparisons across panels less straightforward.

108.6. Practice Problem

Facet the `systolic_bp` histogram (instead of `bmi`) by `smoker` (2 panels instead of 5). Does blood pressure's shape look meaningfully different between the two panels?

108.7. Challenge Problem

Facet a scatter plot of `bmi` against `systolic_bp` by `county` (`facet_wrap(~ county)` applied to Page 1's plot). With Level 7's weak overall BMI-blood pressure correlation (0.16) in mind, would you expect any single county's facet to show a visibly stronger or weaker pattern than the others, given how few points each individual county panel now contains?

108.8. Key Takeaways

- `facet_wrap(~ grouping_column)` splits one plot into multiple small panels, one per category, sharing the same axis scale by default for fair comparison.
- Faceting can reveal subgroup patterns (like Springvale's lower BMI) that a single combined plot averages away.
- Faceting works best for a handful of meaningful categories — too many categories produce panels too small and numerous to read.

Knowledge Check

1. **Multiple choice.** What is required syntax when specifying the faceting variable?
 - a) A pipe (`|>`) before the variable name
 - b) A tilde (`~`) before the variable name
 - c) Quotation marks around the variable name
 - d) A `group =` argument
2. **True or false.** Facets always use independent, separately scaled axes for each panel by default.
3. **Predict the output.** If `county` has 5 distinct values, how many panels would `facet_wrap(~ county)` produce?
4. **Short answer.** Why might faceting by `id` (100 distinct participants) be a poor choice, even though it's technically valid code?

Show answers

1. **b)** A tilde (`~`) before the variable name — required formula syntax for `facet_wrap()`.
2. **False.** By default, facets share the same axis scale across all panels; `scales = "free"` must be specified to give each panel its own independent scale.

108. *Faceting into Multiple Panels*

3. 5 panels, one per county.
4. With 100 distinct ids, faceting would produce 100 tiny panels, each containing just a single data point — far too many small panels to read meaningfully, and no more informative than looking at the raw data table itself.

109. Saving Plots with `ggsave()`

The Problem

Every plot built in this level so far only exists inside your R session — visible while you're working, but gone the moment you close RStudio, and impossible to paste into a Word document or slide deck your professor asked for. You need an actual image file on disk: a `.png` or `.pdf` you can attach to an email or drop into a report.

Your Mission

Save a `ggplot2` plot to an image file using `ggsave()`.

Think Before Coding

- From Level 3, what did `write_csv()` need to know in order to save a data frame to disk? Would you expect a plot-saving function to need similar information?
- If you don't tell `ggsave()` which plot to save, what do you think it saves instead?

109.1. The R Concept

`ggsave()` writes the most recently displayed (or a specifically named) plot to an image file, with the format determined automatically by the file extension:

```
ggsave("filename.png", plot = my_plot, width = 8, height = 6, dpi = 300)
```

109. Saving Plots with *ggsave()*

- "filename.png" — the file to create; .png, .pdf, .jpg, and .svg are all supported, chosen by the extension.
- plot = my_plot — which saved plot object to write; if omitted, ggsave() saves whatever plot was most recently displayed.
- width, height — the image's dimensions, typically in inches.
- dpi — resolution ("dots per inch"); 300 is a common standard for print-quality output.

109.2. Demonstration

```
library(ggplot2)

participants <- read.csv("data/participants.csv")

bmi_bp_plot <- ggplot(data = participants, mapping = aes(x = bmi, y = systol)) +
  geom_point() +
  labs(
    title = "BMI and Systolic Blood Pressure",
    x = "Body Mass Index (BMI)",
    y = "Systolic Blood Pressure (mmHg)"
  ) +
  theme_minimal()

ggsave("bmi_bp_scatter.png", plot = bmi_bp_plot, width = 8, height = 6, dpi = 300)
```

This creates a file called `bmi_bp_scatter.png` in your working directory (Level 1) — a standalone image file, ready to attach to an email, insert into a Word document, or embed in a slide deck.

109.3. Explore

Try saving the same plot twice: once as "bmi_bp_scatter.png" and once as "bmi_bp_scatter.pdf". Both calls use the exact same `ggsave()` syntax — only the file extension changes, and `ggsave()` picks the appropriate image format based on it.

109.4. Solve the Problem

```
library(ggplot2)

participants <- read.csv("data/participants.csv")

county_bar_chart <- ggplot(data = participants, mapping = aes(x = county)) +
  geom_bar() +
  labs(
    title = "Number of Participants by County",
    x = "County",
    y = "Number of Participants"
  ) +
  theme_minimal()

ggsave("county_bar_chart.png", plot = county_bar_chart, width = 8, height = 5, dpi = 300)

file.exists("county_bar_chart.png")

[1] TRUE
```

`file.exists()` (Level 3) confirms the image file was actually written to disk — ready to attach to your professor's report exactly as requested.

109.5. Interpret the Result

`county_bar_chart.png` now exists as a genuine file on disk, independent of your R session — closing RStudio, restarting your computer, or emailing the file to someone without R installed at all doesn't affect it, unlike a plot that only exists as an object in your current session's memory.

Common Mistakes

- **Forgetting to specify `plot = ...` when the intended plot isn't the most recently displayed one.** `ggsave()` defaults to saving whatever was last shown on screen — if you've displayed a different plot since building the one you actually want saved, omitting `plot = ...` silently saves the wrong image.
- **Choosing a `dpi` too low for print use.** The default `dpi = 300` is standard for print-quality output; much lower values (like 72, common for web images) can look blurry or pixelated when printed or zoomed into.
- **Not checking the file actually saved where expected.** Like Level 3's `write_csv()`, `ggsave()` writes relative to your current working directory (Level 1) unless given a full file path — always confirm with `file.exists()` or by checking your file browser.

109.6. Practice Problem

Save the BMI-by-smoking-status boxplot (Page 5) as `"bmi_smoker_boxplot.png"`, with `width = 7`, `height = 5`, and `dpi = 300`. Confirm the file exists afterward.

109.7. Challenge Problem

Save the same plot in three different formats (`.png`, `.pdf`, `.svg`) using three separate `ggsave()` calls, and research (or predict from what you know about file formats generally) one practical difference between a `.png` and an `.svg` file that might matter for a report intended to be resized or printed at large scale.

109.8. Key Takeaways

- `ggsave("filename.ext", plot = my_plot, width = ..., height = ..., dpi = ...)` writes a `ggplot2` plot to an image file, with format determined by the file extension.
- Without an explicit `plot = ...` argument, `ggsave()` saves whichever plot was most recently displayed — not necessarily the one you intend.
- A saved image file persists independently of your R session, unlike a plot object that exists only in memory until saved.

Knowledge Check

1. **Multiple choice.** What determines the image format `ggsave()` writes?
 - a) The `dpi` argument
 - b) The file extension in the filename
 - c) The `width` and `height` arguments
 - d) `ggsave()` always writes `.png` regardless
2. **True or false.** If `plot = ...` is omitted, `ggsave()` saves whichever plot object appears first (alphabetically) in your environment.

109. Saving Plots with `ggsave()`

3. **Predict the output.** After `ggsave("my_plot.png", plot = my_plot)`, what would `file.exists("my_plot.png")` return, assuming the save succeeded?
4. **Find the error.**

```
ggplot(data = participants, aes(x = bmi)) +  
  geom_histogram(binwidth = 5)  
  
plot1 <- ggplot(data = participants, aes(x = age)) +  
  geom_histogram(binwidth = 10)  
  
ggsave("age_histogram.png", width = 8, height = 6)  
# intended to save plot1 (the age histogram)
```

Show answers

1. **b)** The file extension in the filename (`.png`, `.pdf`, `.jpg`, `.svg`, etc.).
2. **False.** Without `plot = ...`, `ggsave()` saves whichever plot was most recently *displayed*, not the first one alphabetically in the environment.
3. **TRUE.**
4. The BMI histogram was displayed last (even though `plot1`, the age histogram, was assigned afterward but never explicitly shown), so `ggsave()` without `plot = ...` would save the wrong plot. It should explicitly specify `ggsave("age_histogram.png", plot = plot1, width = 8, height = 6)`.

110. Level 8 Capstone: One Figure, Two Variables, One Story

The Problem

This level's mission from the very first page: show your research team how blood pressure differs by age *and* smoking status — in one figure, not several separate ones. You now have every ingredient — scatter plots, trend lines, color grouping, titles, labels, and themes — but you've only ever used them one or two at a time. Your professor wants them combined into a single, polished figure that tells the complete story at a glance.

Your Mission

Combine `geom_point()`, `geom_smooth()`, color grouping, and full labeling into one finished, presentation-ready figure.

Think Before Coding

- To show blood pressure by age *and* smoking status simultaneously, how many variables need to be mapped in `aes()` at once?
- If you add `geom_smooth(method = "lm")` to a plot where `color` is already mapped to `smoker`, would you expect one trend line for the whole dataset, or one trend line per smoking group?

110.1. The R Concept

When `color` (or `fill`) is mapped inside the initial `ggplot()` call, every subsequent layer — including `geom_smooth()` — automatically applies **per group**, without any extra code:

```
ggplot(data = df, aes(x = x_col, y = y_col, color = grouping_col)) +  
  geom_point() +  
  geom_smooth(method = "lm")
```

This produces one trend line *per category* of the grouping variable, each colored to match its group's points — because the `color` mapping set in `ggplot()` applies to every layer that follows, not just `geom_point()`.

110.2. Demonstration

```
library(ggplot2)  
  
participants <- read.csv("data/participants.csv")  
  
ggplot(data = participants, mapping = aes(x = age, y = systolic_bp, color = s  
  geom_point() +  
  geom_smooth(method = "lm", se = FALSE)
```

This produces two trend lines instead of one — a separate line fit only to smokers' data, and another fit only to non-smokers' data — both layered over the same color-coded scatter of points, with `se = FALSE` hiding the confidence bands to keep the figure less cluttered with two groups already present.

110.3. Explore

Try removing `se = FALSE` and comparing how much busier the plot looks with both groups' confidence bands shown. With relatively small group sizes (22 smokers, 68 non-smokers) and weak underlying correlations, decide for yourself whether the added confidence-band detail helps or clutters this particular figure.

110.4. Solve the Problem

```
library(ggplot2)

participants <- read.csv("data/participants.csv")

bp_by_age_smoking <- ggplot(data = participants, mapping = aes(x = age, y = systolic_bp, color = "Current Smoker")) +
  geom_point(alpha = 0.7) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(
    title = "Blood Pressure by Age and Smoking Status",
    subtitle = "Weak downward trend in both groups; non-smokers show a slightly steeper decline",
    x = "Age (years)",
    y = "Systolic Blood Pressure (mmHg)",
    color = "Current Smoker"
  ) +
  theme_minimal()

bp_by_age_smoking
```

Two gently downward-sloping trend lines, one per smoking group: non-smokers' line slopes slightly more steeply downward (correlation -0.25,

110. Level 8 Capstone: One Figure, Two Variables, One Story

n=68) than smokers' line (correlation -0.14, n=22), though both relationships remain weak. `alpha = 0.7` makes overlapping points partially transparent, helping visually dense regions of the plot stay readable.

110.5. Interpret the Result

This single figure answers the level's opening mission directly: blood pressure shows a weak negative relationship with age in both smoking groups, with non-smokers showing a marginally steeper downward trend than smokers — though given both groups' correlations are well below 0.3 in magnitude, and the smoker group is fairly small (n=22), this difference between the two slopes shouldn't be over-interpreted as a confirmed, meaningfully different pattern. What the figure does successfully avoid is any *false* impression of a strong relationship in either group — the loosely scattered points and shallow trend lines honestly represent how weak these patterns actually are.

Common Mistakes

- **Building an impressively multi-layered figure that overstates a weak finding.** Combining many `ggplot2` techniques into one figure doesn't make an underlying weak relationship (correlations of -0.14 and -0.25) look stronger than it is — a well-built figure should still honestly represent uncertainty and weak effects, not disguise them with visual polish.
- **Forgetting that color mapped once in `ggplot()` cascades to every later layer.** This is what makes `geom_smooth()` automatically fit separate lines per group here — a useful shortcut, but one that can surprise you if you expected a single combined trend line instead.
- **Overcrowding a figure with too many simultaneous elements.** Adding confidence bands, multiple colors, dense point overlap, and heavy labeling all at once can make a figure harder to read,

not easier — `alpha` (transparency) and `se = FALSE` (hiding confidence bands) are both tools for keeping a multi-layered figure legible.

110.6. Practice Problem

Build the same style of combined figure (`geom_point()` + `geom_smooth()` + `color` + `labels` + `theme`) for BMI and blood pressure by sex instead of age and smoking status. Does mapping `color = sex` reveal a more or less visible difference between groups than smoking status did?

110.7. Challenge Problem

Combine faceting (Page 9) with color grouping in a single figure: facet the age-blood pressure scatter plot by `county` (5 panels) while still coloring points by `smoker` within each panel. With county-level sample sizes as small as 16, discuss whether this much subdivision (5 counties $\times$ 2 smoking groups) still produces a figure with enough data per group to draw any meaningful visual conclusions.

110.8. Key Takeaways

- Mapping `color` (or `fill`) once in the initial `ggplot()` call cascades automatically to every subsequent layer, including `geom_smooth()`, producing one trend line per group without extra code.
- A polished, multi-layered figure should still honestly represent the strength (or weakness) of the underlying pattern — visual sophistication is not a substitute for accurately representing uncertainty.
- `alpha` (point transparency) and `se = FALSE` (hiding confidence bands) are both tools for keeping a busy, multi-group figure legible rather than cluttered.

Knowledge Check

1. **Multiple choice.** If `color = smoker` is mapped in the initial `ggplot()` call, what happens when you add `geom_smooth(method = "lm")`?
 - a) One trend line is fit to the entire dataset, ignoring smoking status
 - b) One trend line is fit separately for each smoking group
 - c) `geom_smooth()` ignores the color mapping entirely
 - d) The plot errors, since `geom_smooth()` can't be combined with color mapping
2. **True or false.** A visually polished, multi-layered figure can make a weak relationship look stronger than it actually is if built carelessly.
3. **Predict the output.** With smokers' correlation at -0.14 and non-smokers' at -0.25, which group's trend line would slope more steeply downward?
4. **Short answer.** Why does `se = FALSE` help keep a two-group trend line figure more readable?

Show answers

1. **b)** One trend line is fit separately for each smoking group, since the `color` mapping set in `ggplot()` cascades to every subsequent layer.
2. **True.** Visual polish (multiple colors, trend lines, clean themes) doesn't change the underlying data — a careless figure could visually overstate a weak pattern's strength if built without care.
3. Non-smokers, with the stronger (more negative) correlation of -0.25.
4. `se = FALSE` hides each group's confidence band — with two overlapping trend lines already present, adding two shaded confidence regions on top could make the figure visually cluttered and harder to read.

Part IX.

Level 9: Functions and Programming

Level 9 — Functions and Programming

Where this level is headed

Copy-pasting the same three lines of code for fifty variables is how errors sneak into real research. This level teaches you to write your own functions — turning a repeated task into a single, reliable, reusable tool.

What you'll build toward

You've called dozens of functions since Level 1 — `mean()`, `filter()`, `pivot_longer()`, `ggplot()` — without ever writing one yourself. This level flips that around: you'll write your own functions from scratch, give them default arguments, add conditional logic and validation inside them, and then apply a single function across many columns at once with base R's `sapply()/lapply()` and the tidyverse's `purrr::map()` family.

Topics in this level

- What a function actually is, and how R's built-in functions you already use are no different in kind from ones you'll write yourself
- Writing your first function: inputs (arguments), outputs (return values)
- Default argument values

Level 9 — Functions and Programming

- Conditional logic inside a function
- Variable scope — what a function can and can't see or change outside itself
- Anonymous (“throwaway”) functions
- `sapply()` and `lapply()` for applying a function across many columns
- `purrr::map()`, `map_dbl()`, and `map_chr()`
- Basic error handling with `stop()` and `warning()`

A preview mission

Your mission across this level: `participants.csv` has 8 columns, several with their own missing-value rate (6% for `age`, 4% for `systolic_bp`, 5% for `bmi`). Write a single function that computes “percent missing” for one column, then apply it across every column in the dataset at once — write it once, run it on all 8 columns automatically, rather than retyping `sum(is.na(...)) / nrow(...) * 100` eight separate times.

111. What Is a Function?

The Problem

Since Level 1, you've typed `mean(participants$age, na.rm = TRUE)` without ever asking what `mean()` actually *is* — you just trust it takes a vector in and gives a number back. Now your professor asks you to compute something `mean()` doesn't do: the percentage of missing values in a column. There's no built-in `pct_missing()` function in R. You'll have to build the tool yourself.

Your Mission

Understand what a function fundamentally is — a reusable, named piece of code that takes input and produces output — as preparation for writing your own.

Think Before Coding

- When you call `mean(participants$age, na.rm = TRUE)`, what goes *in* (the input), and what comes *out* (the output)?
- If you had to explain `sum(is.na(x)) / length(x) * 100` to a colleague as a single reusable idea, what would you want to name it?

111.1. The R Concept

A **function** is a named, reusable block of code that takes zero or more **inputs** (called *arguments* or *parameters*), does something with them, and produces an **output** (called the *return value*). Every function you've used

111. What Is a Function?

since Level 1 — `mean()`, `sum()`, `read.csv()`, `filter()`, `ggplot()` — follows this exact same pattern:

```
function_name(input1, input2, ...)  # produces some output
```

`mean(participants$age, na.rm = TRUE)` takes a vector (`participants$age`) and a setting (`na.rm = TRUE`) as input, and returns a single number as output. There is nothing magical about built-in functions — they're written in exactly the same style you're about to learn to write your own functions in.

111.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
# Using a built-in function - input goes in, output comes out
mean(participants$age, na.rm = TRUE)
```

```
[1] 47.42553
```

```
# The pattern you'll replicate: some calculation, wrapped up and named
sum(is.na(participants$age)) / length(participants$age) * 100
```

```
[1] 6
```

The second line computes the percentage of missing `age` values by hand — exactly the calculation you'd want to name and reuse, the same way someone once wrote `mean()` so you'd never have to type `sum(x) / length(x)` yourself.

111.3. Explore

Try running `sum(is.na(participants$systolic_bp)) / length(participants$systolic_bp) * 100` and `sum(is.na(participants$bmi)) / length(participants$bmi) * 100`. Notice you're retyping the same pattern with only the column name changing — exactly the repetition a function is meant to eliminate.

111.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

age_pct_missing <- sum(is.na(participants$age)) / length(participants$age) * 100
bp_pct_missing <- sum(is.na(participants$systolic_bp)) / length(participants$systolic_bp) *
bmi_pct_missing <- sum(is.na(participants$bmi)) / length(participants$bmi) * 100

age_pct_missing

[1] 6

bp_pct_missing

[1] 4

bmi_pct_missing

[1] 5
```

Three correct answers (6%, 4%, 5% missing) — but three nearly identical lines of code, differing only in which column is referenced. This is precisely the kind of repetition the next page's custom function will eliminate.

111.5. Interpret the Result

`age`, `systolic_bp`, and `bmi` are each missing a small percentage of their values (6%, 4%, and 5% respectively) — useful individually, but the *process* of computing each one was identical. Recognizing “I keep typing the same pattern with only one thing changing” is the signal that a function belongs here — the same signal that should have applied a hundred times over across every prior level, now finally something you can act on.

Common Mistakes

- **Assuming built-in and custom functions work fundamentally differently.** They don’t — `mean()` and a function you write yourself both take input and produce output the exact same way; the only difference is who wrote them and when.
- **Not noticing repeated code as a signal to write a function.** Copy-pasting the same calculation three times with one small change (Page 1’s `age/systolic_bp/bmi` example) is the clearest sign a function is overdue — the next page turns exactly this repetition into one reusable tool.
- **Confusing a function’s *definition* with *calling* it.** Writing a function (next page) and calling it are two separate steps — defining `pct_missing <- function(x) {...}` doesn’t run anything by itself; you still have to call `pct_missing(participants$age)` afterward to get a result.

111.6. Practice Problem

Compute the percentage missing for `diastolic_bp` and `smoker` by hand, using the same `sum(is.na(x)) / length(x) * 100` pattern. Confirm both are 0%, since Level 7 found no missing values in either column.

111.7. Challenge Problem

Before reading the next page, try to write out in plain English (not R code) the steps a function called `pct_missing` would need to perform, given any column as input. What would its single input be? What single output would it produce?

111.8. Key Takeaways

- A function takes input (arguments) and produces output (a return value) — every function you’ve used since Level 1 follows this same basic pattern.
- Built-in functions like `mean()` and functions you write yourself work identically in kind — the only difference is authorship.
- Retyping the same calculation with only one small change is the clearest signal that code belongs in a function instead.

Knowledge Check

1. **Multiple choice.** What are a function’s inputs called?
 - a) Return values
 - b) Arguments (or parameters)
 - c) Comments
 - d) Vectors
2. **True or false.** Built-in functions like `mean()` work fundamentally differently from functions a user writes.
3. **Predict the output.** What would `sum(is.na(participants$diastolic_bp)) / length(participants$diastolic_bp) * 100` return, given `diastolic_bp` has no missing values?

111. *What Is a Function?*

4. **Short answer.** What's the clearest signal that a repeated piece of code should be turned into a function?

Show answers

1. **b)** Arguments (or parameters).
2. **False.** Built-in and user-written functions work identically in kind — both take input and produce output the same way.
3. 0 — with no missing values, the calculation gives 0%.
4. Typing the same (or nearly the same) calculation more than once, with only a small piece — like a column name — changing between repetitions.

112. Writing Your First Function

The Problem

You've identified the repeated pattern (previous page): `sum(is.na(x)) / length(x) * 100`. Now you need to actually turn it into a real, callable tool — something you can invoke as `pct_missing(participants$age)` the same way you'd call `mean(participants$age)`, instead of retyping the full calculation every time.

Your Mission

Write your own function using R's `function()` keyword, with a defined input and a returned output.

Think Before Coding

- Looking at `sum(is.na(x)) / length(x) * 100`, what single input (`x`) does this calculation need to run?
- If a function's last line is a value (not assigned to anything), do you think R automatically treats that as the function's output?

112.1. The R Concept

A function is created with the `function()` keyword, listing its argument names in parentheses, followed by a block of code in curly braces `{}`:

112. Writing Your First Function

```
function_name <- function(argument1, argument2) {  
  # code using argument1 and argument2  
  result    # the last evaluated expression is returned automatically  
}
```

R automatically returns whatever the **last line** of the function evaluates to — no explicit `return()` needed, though it's allowed and sometimes used for clarity or to exit early:

```
pct_missing <- function(x) {  
  sum(is.na(x)) / length(x) * 100  
}
```

112.2. Demonstration

```
pct_missing <- function(x) {  
  sum(is.na(x)) / length(x) * 100  
}
```

```
participants <- read.csv("data/participants.csv")
```

```
pct_missing(participants$age)
```

```
[1] 6
```

```
pct_missing(participants$systolic_bp)
```

```
[1] 4
```

```
pct_missing(participants$bmi)
```

```
[1] 5
```

The exact same three answers as the previous page (6%, 4%, 5%) — but now produced by calling one short, named function three times, instead of retyping the full calculation each time.

112.3. Explore

Try `pct_missing(participants$diastolic_bp)`. Confirm it correctly returns 0, exactly matching your hand calculation from the previous page's practice problem — your function works on *any* vector passed to it, not just the ones you tested first.

112.4. Solve the Problem

```
pct_missing <- function(x) {
  sum(is.na(x)) / length(x) * 100
}
```

```
participants <- read.csv("data/participants.csv")
```

```
pct_missing(participants$id)
```

```
[1] 0
```

```
pct_missing(participants$smoker)
```

```
[1] 0
```

```
pct_missing(participants$county)
```

```
[1] 0
```

112. Writing Your First Function

`id`, `smoker`, and `county` all correctly return 0 — none of these columns have any missing values, and `pct_missing()` handles them exactly as correctly as it handled the columns that *did* have missing data, without needing any special-casing.

112.5. Interpret the Result

`pct_missing()` now works as a genuine, general-purpose tool — callable on any column in `participants`, or in any other dataset entirely, without ever needing to see or modify its internal code again. This is the core payoff of writing a function: the calculation was written once, correctly, and now every future use of it is just a short function call.

Common Mistakes

- **Forgetting the curly braces `{}` for multi-line functions.** A single-line function body can skip the braces (`pct_missing <- function(x) sum(is.na(x)) / length(x) * 100`), but any function with more than one line needs `{}` to group them together.
- **Assuming a function needs an explicit `return()` to produce output.** R automatically returns the last evaluated expression — `return()` is optional and mainly useful for exiting a function early (covered later in this level) or for clarity in longer functions.
- **Confusing the argument name (`x`) with any specific column.** `x` inside the function body is just a placeholder name — it takes on whatever value is passed in when the function is called (`participants$age`, `participants$bmi`, or anything else), and has no meaning outside the function itself.

112.6. Practice Problem

Write a function called `range_width` that takes a numeric vector `x` and returns the difference between its maximum and minimum values (using `na.rm = TRUE` inside), then test it on `participants$bmi`.

112.7. Challenge Problem

Write a function called `mean_and_sd` that takes a numeric vector `x` and returns *both* the mean and standard deviation as a named vector (hint: `c(mean = ..., sd = ...)`), handling missing values appropriately. Test it on `participants$age` and confirm both values match Level 7's earlier findings (mean 47.43, SD 15.03).

112.8. Key Takeaways

- A function is created with `function(arguments) { code }`, and is called later by name with actual values in place of its arguments.
- R automatically returns the last evaluated expression from a function — an explicit `return()` is optional, not required.
- Once written and tested, a function becomes a reusable tool that works on any appropriate input, not just the specific examples used while building it.

Knowledge Check

1. **Multiple choice.** What does R return from a function by default, with no explicit `return()`?
 - a) Nothing — an error occurs
 - b) The first evaluated expression

112. Writing Your First Function

- c) The last evaluated expression
 - d) NULL, always
2. **True or false.** Curly braces `{}` are required for every function, even one-line ones.
 3. **Predict the output.** Given `pct_missing <- function(x) sum(is.na(x)) / length(x) * 100`, what would `pct_missing(c(1, NA, 3, NA))` return?
 4. **Find the error.**

```
pct_missing <- function(x)
  sum(is.na(x)) / length(x) * 100
  cat("Done!")
```

Show answers

1. **c)** The last evaluated expression.
2. **False.** A single-line function body can omit the curly braces; they're only required when the body spans multiple lines that need to be grouped together.
3. 50 — 2 of the 4 values are NA, so $2 / 4 * 100 = 50$.
4. Without curly braces, only the very next single line (`sum(is.na(x)) / length(x) * 100`) is treated as the function's body — the `cat("Done!")` line runs separately, outside the function, every time this code is sourced, rather than being part of `pct_missing()`'s behavior. Wrapping both lines in `{}` would fix this ambiguity.

113. Default Argument Values

The Problem

You want a function that reports a numeric variable's mean and standard deviation, rounded for a clean report — but different reports need different precision: a quick internal check might want whole numbers, while a final publication table wants 2 decimal places. Writing two separate functions (`describe_rounded` and `describe_precise`) for what's really the same calculation feels wasteful.

Your Mission

Give a function argument a default value, so it works sensibly when called with fewer arguments, while still allowing that default to be overridden.

Think Before Coding

- If most of the time you want 2 decimal places, but occasionally want more or fewer, would you rather type `digits = 2` every single time, or have 2 be assumed unless you say otherwise?
- From Level 2, `mean(x, na.rm = TRUE)` — is `na.rm` required, or does it have some default value if you omit it entirely?

113.1. The R Concept

A **default argument value** is specified with `=` directly in the function definition, and is used automatically whenever that argument is omitted from the call:

113. Default Argument Values

```
function_name <- function(x, digits = 2) {  
  # digits is 2 unless the caller specifies otherwise  
}  
  
describe_numeric <- function(x, digits = 2) {  
  c(  
    mean = round(mean(x, na.rm = TRUE), digits),  
    sd = round(sd(x, na.rm = TRUE), digits)  
  )  
}
```

Calling `describe_numeric(participants$age)` uses `digits = 2` automatically; calling `describe_numeric(participants$age, digits = 0)` overrides it explicitly. This is exactly the same pattern behind `mean(x, na.rm = TRUE)`'s `na.rm` argument, which defaults to `FALSE` unless you explicitly set it to `TRUE`.

113.2. Demonstration

```
describe_numeric <- function(x, digits = 2) {  
  c(  
    mean = round(mean(x, na.rm = TRUE), digits),  
    sd = round(sd(x, na.rm = TRUE), digits)  
  )  
}  
  
participants <- read.csv("data/participants.csv")  
  
describe_numeric(participants$age)  
  
      mean      sd  
47.43 15.03
```

```
describe_numeric(participants$age, digits = 0)
```

```
mean  sd
  47   15
```

Calling the function with no `digits` argument at all uses the default (2 decimal places); explicitly passing `digits = 0` overrides that default, rounding to whole numbers instead.

113.3. Explore

Try `describe_numeric(participants$bmi, digits = 4)`. Confirm the function happily accepts any value for `digits`, not just 0 or 2 — the default is only ever a *fallback*, never a restriction on what values are allowed.

113.4. Solve the Problem

```
describe_numeric <- function(x, digits = 2) {
  c(
    mean = round(mean(x, na.rm = TRUE), digits),
    sd = round(sd(x, na.rm = TRUE), digits)
  )
}
```

```
participants <- read.csv("data/participants.csv")
```

```
describe_numeric(participants$systolic_bp)
```

```
mean  sd
129.84 18.87
```

113. Default Argument Values

```
describe_numeric(participants$systolic_bp, digits = 1)
```

```
   mean    sd  
129.8  18.9
```

Both calls compute the exact same underlying mean and SD — only the rounding precision changes, controlled entirely by whether `digits` is left at its default (2) or explicitly overridden (1).

113.5. Interpret the Result

`describe_numeric()` now serves both the quick-internal-check case (default 2 decimals) and the more precise or more rounded reporting case (any `digits` value explicitly requested), without needing two separate functions or repeating the `round()`-wrapped calculation twice. A well-chosen default lets the common case stay simple, while still leaving full flexibility available when needed.

Common Mistakes

- **Choosing a default that silently produces wrong-looking results for typical use.** A poorly chosen default (like `digits = 10`) could produce overly precise, hard-to-read output for the common case — defaults should reflect what’s usually wanted, not just any arbitrary value.
- **Forgetting default arguments can reference other arguments.** More advanced function definitions can have one default depend on another argument’s value — while not needed here, it’s worth knowing defaults aren’t limited to fixed constants.
- **Assuming a default value means an argument can never be changed.** A default is only what’s used *if omitted* — every default-valued argument remains fully overridable at the calling site, exactly as shown with `digits = 0` and `digits = 1` above.

113.6. Practice Problem

Modify `pct_missing()` from Page 2 to accept a `digits` argument with a default of 1, rounding its result before returning it. Test it on `participants$age` both with and without specifying `digits`.

113.7. Challenge Problem

Write a function called `flag_high_bp` that takes a numeric vector of systolic blood pressure values and a `threshold` argument defaulting to 130 (Level 4's hypertension cutoff), returning `TRUE/FALSE` for each value indicating whether it exceeds the threshold. Test it once with the default threshold, and once with `threshold = 140` (a stricter cutoff), and compare how many values are flagged under each.

113.8. Key Takeaways

- A default argument value (`argument = default_value` in the function definition) is used automatically whenever that argument is omitted from a call.
- Defaults make the common case simple to call, while every default remains fully overridable when a different value is needed.
- This is the exact same pattern behind built-in functions like `mean(x, na.rm = TRUE)`, whose `na.rm` argument defaults to `FALSE`.

Knowledge Check

1. **Multiple choice.** In `function(x, digits = 2)`, when is `digits` equal to 2?
 - a) Always, regardless of how the function is called

113. Default Argument Values

- b) Only when the caller explicitly writes `digits = 2`
 - c) Whenever the caller omits the `digits` argument entirely
 - d) Never — defaults are just documentation, not actual behavior
2. **True or false.** A default argument value can always be overridden by the caller.
 3. **Predict the output.** Given `describe_numeric <- function(x, digits = 2) {...}`, what would `digits` equal inside a call to `describe_numeric(participants$bmi)`?
 4. **Find the error.**

```
describe_numeric <- function(x, digits) {  
  c(mean = round(mean(x, na.rm = TRUE), digits))  
}  
describe_numeric(participants$age)
```

Show answers

1. **c)** Whenever the caller omits the `digits` argument entirely.
2. **True.** A default is only ever a fallback for when an argument is omitted — it never prevents the caller from specifying a different value.
3. **2** — the default value, since the call didn't specify `digits`.
4. **`digits`** has no default value here, so calling `describe_numeric(participants$age)` without specifying `digits` produces an error (`argument "digits" is missing, with no default`). Adding `digits = 2` (or any default) to the function definition would fix this.

114. Conditional Logic Inside Functions

The Problem

Level 4's `case_when()` classified blood pressure into categories (Normal, Elevated, Stage 1, Stage 2) directly inside a `mutate()` call on the whole `participants` data frame. Your professor now wants to classify a *single* blood pressure value on the spot — during a live conversation, not by re-running a full data-cleaning pipeline — using the exact same category boundaries.

Your Mission

Write a function containing `if/else if/else` conditional logic (Level 4), reusable for classifying any single value.

Think Before Coding

- From Level 4, what were the four systolic blood pressure category boundaries (Normal, Elevated, Stage 1, Stage 2)?
- If a function's body contains an `if/else if/else` chain, and each branch's last line is a category name, would you expect the function to return whichever branch actually ran?

114.1. The R Concept

A function's body can contain the exact same `if/else if/else` logic (Level 4) you'd use outside a function — the only difference is that the result becomes the function's return value:

```
classify <- function(x) {  
  if (x < threshold1) {  
    "Category A"  
  } else if (x < threshold2) {  
    "Category B"  
  } else {  
    "Category C"  
  }  
}
```

```
bp_category <- function(sbp) {  
  if (sbp < 120) {  
    "Normal"  
  } else if (sbp < 130) {  
    "Elevated"  
  } else if (sbp < 140) {  
    "Stage 1"  
  } else {  
    "Stage 2"  
  }  
}
```

Whichever branch actually runs becomes the function's return value automatically — R returns the last evaluated expression, exactly as on Page 2, and here that last expression differs depending on which `if/else if` branch was taken.

114.2. Demonstration

```
bp_category <- function(sbp) {
  if (sbp < 120) {
    "Normal"
  } else if (sbp < 130) {
    "Elevated"
  } else if (sbp < 140) {
    "Stage 1"
  } else {
    "Stage 2"
  }
}
```

```
bp_category(115)
```

```
[1] "Normal"
```

```
bp_category(145)
```

```
[1] "Stage 2"
```

`bp_category()` correctly classifies both a normal reading (115) and a high one (145) — usable instantly for any single value, without needing a data frame or a `mutate()` call at all.

114.3. Explore

Try `bp_category(120)` and `bp_category(129)` — both should return "Elevated", since the boundaries use `<` (strictly less than), meaning the *lower* bound of each category belongs to that category, not the one before it. Confirm this matches Level 4's `case_when()` boundary logic exactly.

114.4. Solve the Problem

```

bp_category <- function(sbp) {
  if (sbp < 120) {
    "Normal"
  } else if (sbp < 130) {
    "Elevated"
  } else if (sbp < 140) {
    "Stage 1"
  } else {
    "Stage 2"
  }
}

participants <- read.csv("data/participants.csv")

# Apply it to every non-missing systolic_bp value using sapply (previewed previously)
bp_categories <- sapply(participants$systolic_bp[!is.na(participants$systolic_bp)], bp_category)

table(bp_categories)

```

```

bp_categories
Elevated   Normal   Stage 1   Stage 2
      18       32       17       29

```

Exactly the same category counts (18 Elevated, 32 Normal, 17 Stage 1, 29 Stage 2) as Level 4's `case_when()`-based `mutate()` produced on the full data frame — confirming the function-based, single-value approach and the data-frame-wide `case_when()` approach agree perfectly.

114.5. Interpret the Result

`bp_category()` reproduces Level 4's classification logic exactly, but packaged as a reusable tool that works on one value at a time — useful in contexts where you don't want to build (or don't have) a full data frame, like classifying a single new reading as it comes in. The matching counts (18/32/17/29) confirm the function was written correctly, cross-checked against a completely different implementation from an earlier level.

Common Mistakes

- **Getting boundary conditions backwards inside a function, same as in Level 4.** `sbp < 120` versus `sbp <= 120` produces different classifications right at the boundary value 120 — the same care needed for `case_when()` conditions applies identically inside a function's `if` statements.
- **Forgetting `else` needs to handle every remaining case.** If the final `else` branch were missing, a systolic BP of 150 (falling through every `if/else if` condition) would return `NULL` instead of a category — always ensure the final `else` genuinely catches everything not handled above it.
- **Passing a vector to a function written for a single value.** `bp_category()` as written expects one number at a time — passing an entire column directly (`bp_category(participants$systolic_bp)`) doesn't work as intended, because `if` in R expects a single logical value, not a vector; `sapply()` (Page 8) is what applies a single-value function across an entire column correctly.

114.6. Practice Problem

Write a function called `age_group` that takes a single age value and returns "Senior" for ages 65 and older, "Adult" for ages 18-64, using `if/else`. Test it on a few individual values.

114.7. Challenge Problem

Modify `bp_category()` to also handle a "Low" category for `sbp < 90` (add this as the very first condition checked), and re-run it across `participants$systolic_bp`. Given the minimum recorded systolic BP is exactly 90, would you expect this new category to ever actually appear in the results?

114.8. Key Takeaways

- A function's body can contain the same `if/else if/else` logic used outside a function (Level 4) — whichever branch runs becomes the function's automatic return value.
- Boundary conditions (`<` versus `<=`) require the same care inside a function as they do in any standalone conditional.
- Cross-checking a function's output against a previously verified result (like Level 4's `case_when()` counts) is a reliable way to confirm the function was written correctly.

Knowledge Check

1. **Multiple choice.** In `bp_category()`, what does the function return for `sbp = 135`?
 - a) "Normal"
 - b) "Elevated"
 - c) "Stage 1"
 - d) "Stage 2"
2. **True or false.** A missing final `else` branch could cause a function to return `NULL` for values not covered by any earlier condition.
3. **Predict the output.** What would `bp_category(119)` and `bp_category(120)` each return?

4. **Short answer.** Why does cross-checking `bp_category()`'s counts (18/32/17/29) against Level 4's `case_when()` counts provide good evidence the function is correct?

Show answers

1. **c)** "Stage 1" — 135 is at least 130 but less than 140.
2. **True.** If no `if/else` if condition matches and there's no final `else`, the function returns `NULL` for that input, rather than a meaningful category.
3. `bp_category(119)` returns "Normal" (since `119 < 120`); `bp_category(120)` returns "Elevated" (since 120 is not less than 120, so it falls to the next condition).
4. Two independently written implementations (Level 4's `case_when()` inside `mutate()`, and this level's `if/else` if function) producing identical counts on the same data is strong evidence neither has a logic error — if either had a boundary mistake, the two counts would very likely disagree.

115. Functions with Multiple Arguments

The Problem

Level 5’s flagship mission flagged participants aged 65+ *and* with systolic blood pressure of 130+ — a condition depending on **two** separate values at once, not one. `bp_category()` (previous page) only ever looked at a single value. Your professor now wants a reusable check that takes both age and blood pressure together and decides, in one call, whether a participant counts as “at risk.”

Your Mission

Write a function that accepts multiple arguments, combining them with logical operators (Level 1) to make a single combined decision.

Think Before Coding

- From Level 5, what exact condition (using `&`) identified the 4 at-risk participants (age 65+ and systolic BP 130+)?
- If a function needs two separate pieces of information to make one decision, how many arguments would you expect its `function()` definition to list?

115.1. The R Concept

A function can accept as many arguments as it needs, listed in order, separated by commas:

```
function_name <- function(arg1, arg2) {  
  # code using both arg1 and arg2  
}  
  
is_at_risk <- function(age, sbp, age_cutoff = 65, bp_cutoff = 130) {  
  age >= age_cutoff & sbp >= bp_cutoff  
}
```

Notice `is_at_risk()` combines two *required* arguments (`age`, `sbp`) with two *optional*, default-valued arguments (`age_cutoff`, `bp_cutoff`, Page 3) — a function can freely mix both kinds.

115.2. Demonstration

```
is_at_risk <- function(age, sbp, age_cutoff = 65, bp_cutoff = 130) {  
  age >= age_cutoff & sbp >= bp_cutoff  
}
```

```
is_at_risk(70, 145)
```

```
[1] TRUE
```

```
is_at_risk(50, 145)
```

```
[1] FALSE
```


The first participant (age 70, BP 145) meets both thresholds and is flagged **TRUE**; the second (age 50, BP 145) fails the age condition alone, so the combined result is **FALSE** — exactly the **&** (“and”) logic from Level 1, now packaged inside a reusable function.

115.3. Explore

Try `is_at_risk(70, 145, age_cutoff = 75)`. Since 70 no longer meets a stricter age cutoff of 75, this should return **FALSE** — confirming the default-valued arguments (Page 3) remain fully overridable even in a function with multiple parameters.

115.4. Solve the Problem

```
is_at_risk <- function(age, sbp, age_cutoff = 65, bp_cutoff = 130) {
  age >= age_cutoff & sbp >= bp_cutoff
}

participants <- read.csv("data/participants.csv")

at_risk_flags <- is_at_risk(participants$age, participants$systolic_bp)

sum(at_risk_flags, na.rm = TRUE)

[1] 4
```

Because `age >= age_cutoff & sbp >= bp_cutoff` uses R’s vectorized **&** operator (Level 2), `is_at_risk()` works correctly whether called on two single numbers or two entire columns at once — no `sapply()` or loop required here, since **&** already compares element-by-element automatically.

115. Functions with Multiple Arguments

The result (4 at-risk participants) exactly matches Level 5's `filter(age >= 65 & systolic_bp >= 130)` count.

115.5. Interpret the Result

`is_at_risk()` reproduces Level 5's flagship finding (4 at-risk participants) exactly, but as a named, reusable, customizable tool — the age and blood pressure cutoffs can be adjusted per call without editing the function's internals, unlike Level 5's `filter()` call, which would need its condition rewritten by hand for a different threshold.

Common Mistakes

- **Forgetting argument order matters when calling a function positionally.** `is_at_risk(145, 70)` (blood pressure first, age second) silently swaps the two values' roles — always double check argument order, or use named arguments (`is_at_risk(age = 70, sbp = 145)`) to avoid ambiguity entirely.
- **Assuming every function automatically vectorizes like `is_at_risk()` did here.** This function happened to work correctly on whole columns because `&`, `>=` are already vectorized operators (Level 2) — not every function behaves this way automatically, especially ones containing `if` (Page 4), which expects a single value, not a vector.
- **Mixing required and default arguments in the wrong order.** In R, arguments with defaults should generally come after arguments without defaults in the function definition — `function(age_cutoff = 65, age, sbp)` would still technically work when called with named arguments, but is much more error-prone and confusing to read.

115.6. Practice Problem

Write a function called `is_high_risk_smoker` that takes `bmi`, `smoker`, and a `bmi_cutoff` defaulting to 30, returning `TRUE` if BMI meets or exceeds the cutoff **and** the participant is a current smoker. Test it on the full `participants` data frame and report how many are flagged.

115.7. Challenge Problem

Modify `is_at_risk()` to accept a third required argument, `smoker`, and flag a participant only if **all three** conditions hold: age 65+, systolic BP 130+, **and** current smoker status. How many participants meet this stricter combined definition, compared to the original 4?

115.8. Key Takeaways

- A function can take multiple arguments, listed in order and separated by commas, freely mixing required and default-valued arguments.
- Whether a function automatically works on whole vectors depends on whether its internal operations (like `&`, `>=`) are already vectorized — this isn't automatic for every function.
- Named arguments (`is_at_risk(age = 70, sbp = 145)`) avoid the ambiguity of positional argument order, especially as a function's argument count grows.

Knowledge Check

1. **Multiple choice.** In `is_at_risk(age, sbp, age_cutoff = 65, bp_cutoff = 130)`, which arguments are optional?
 - a) age and sbp

115. Functions with Multiple Arguments

- b) `age_cutoff` and `bp_cutoff`
 - c) All four are required
 - d) None — all four have defaults
2. **True or false.** Calling `is_at_risk(145, 70)` (with arguments in the wrong order) will produce an error rather than a silently wrong answer.
 3. **Predict the output.** What would `is_at_risk(80, 150, age_cutoff = 70, bp_cutoff = 140)` return?
 4. **Find the error.**

```
is_at_risk(sbp = 70, 145)
# intended: age = 70, sbp = 145
```

Show answers

1. **b) `age_cutoff` and `bp_cutoff`** — both have default values, making them optional.
2. **False.** R doesn't detect that the arguments are “backwards” — it silently treats the first positional value as `age` and the second as `sbp`, producing a plausible-looking but incorrect result rather than an error.
3. **TRUE** — age 80 > 70 and blood pressure 150 > 140, so both conditions hold.
4. **`sbp` was named explicitly as 70**, so the remaining positional value 145 fills the next unfilled argument, `age` — meaning this call actually evaluates as `age = 145, sbp = 70`, the reverse of what was intended. Mixing named and positional arguments is legal in R, but it's easy to misread; naming both arguments explicitly (`is_at_risk(age = 70, sbp = 145)`) avoids this kind of silent mix-up entirely.

116. Variable Scope

The Problem

Inside `describe_numeric()` (Page 3), you used a variable name `x` for the input, and inside `is_at_risk()` (Page 5), you used `age` and `sbp`. But `participants` also has actual columns called `age`! After calling these functions, you check `age` in your R console — and it doesn't exist as a standalone object at all, even though you just used that exact name inside a function. What's going on?

Your Mission

Understand variable *scope* — which parts of your code can see, and be affected by, a given variable — and why functions don't leak their internal variables into your main workspace.

Think Before Coding

- If a function's argument is named `age`, and you also have a column called `participants$age`, do you think R gets confused about which “age” you mean inside the function?
- After a function finishes running, would you expect variables created *inside* it to still exist afterward, in your main R session?

116.1. The R Concept

Every function call creates its own **local scope** — a private workspace for variables created or received as arguments inside that function. Variables

116. Variable Scope

inside a function's local scope:

- Are **isolated** from your main (“global”) workspace — a variable named **age** inside a function has no connection to any **age** object outside it, even if they share the exact same name.
- **Disappear** once the function finishes running — they exist only for the duration of that one function call.
- Can **read** variables from the global environment if not shadowed by a local one of the same name, but cannot **modify** the global version by simply assigning to a same-named local variable.

```
age <- 100    # global variable

check_age <- function(age) {
  age <- age + 1  # this only changes the LOCAL copy of age
  age
}

check_age(30)  # returns 31
age           # still 100 - completely unaffected
```

116.2. Demonstration

```
age <- 100

check_age <- function(age) {
  age <- age + 1
  age
}

check_age(30)

[1] 31
```

```
age
```

```
[1] 100
```

Even though the function's argument is also named `age`, and the function reassigns `age` internally, the *global* `age` (100) is completely untouched — the function's `age` was always a separate, local copy that ceased to exist the moment the function finished.

116.3. Explore

Try defining `local_total <- 0` inside a function body, running the function, and then checking whether `local_total` exists in your main environment afterward (it won't — `exists("local_total")` will return `FALSE`). Variables created inside a function's body are just as local as its arguments.

116.4. Solve the Problem

```
pct_missing <- function(x) {
  n_missing <- sum(is.na(x))      # local variable, exists only inside this call
  n_total <- length(x)           # also local
  n_missing / n_total * 100
}
```

```
participants <- read.csv("data/participants.csv")
```

```
pct_missing(participants$age)
```

```
[1] 6
```

116. Variable Scope

```
exists("n_missing")
```

```
[1] FALSE
```

`n_missing` and `n_total` were created and used entirely inside `pct_missing()`'s local scope — after the function call finishes, `exists("n_missing")` confirms they never leaked into your main workspace at all.

116.5. Interpret the Result

This isolation is a genuine safety feature, not a limitation: it means you can freely reuse short, convenient variable names like `x`, `n`, or `age` inside any function without ever worrying about accidentally overwriting an important variable of the same name in your main script. Functions act as self-contained units — what happens inside, stays inside, unless you explicitly return a value out.

Common Mistakes

- **Expecting a function to modify a global variable by assigning to a same-named local one.** `check_age()`'s internal `age <- age + 1` only ever changes the local copy — to actually update something in your main workspace, the function would need to `return()` the new value and you'd need to reassign it yourself (`age <- check_age(age)`).
- **Assuming variables created inside a function are available for debugging afterward.** If a function errors partway through, its local variables (like `n_missing` above) are simply gone — this is why printing intermediate values *during* development (before wrapping code in a function) is often easier for debugging than trying to inspect a function's internals after the fact.

- **Reusing a global variable's exact name as a function argument and assuming this causes a conflict.** As shown above, this is actually completely safe — R always prioritizes the local (argument) version inside the function body, with no naming conflict at all.

116.6. Practice Problem

Write a small function that takes an argument called `bmi`, adds 1 to it internally, and returns the result. Call it with `participants$bmi[1]` as the argument, then confirm that `participants$bmi[1]` itself is completely unchanged afterward.

116.7. Challenge Problem

Predict, without running any code, what this snippet prints for each line:

```
total <- 10
add_five <- function(total) {
  total <- total + 5
  total
}
print(add_five(total))
print(total)
```

Then run it to confirm your prediction.

116.8. Key Takeaways

- Every function call creates its own local scope — variables inside it (arguments and internally created ones alike) are isolated from your main workspace.
- A function can read global variables it hasn't shadowed, but assigning to a same-named local variable never modifies the global version.
- Local variables disappear once a function finishes running, which is a safety feature protecting your main workspace from accidental modification.

Knowledge Check

1. **Multiple choice.** After `check_age(30)` runs (as defined above), what is the value of the global `age`?
 - a) 31
 - b) 30
 - c) 100 (unchanged)
 - d) NULL
2. **True or false.** Variables created inside a function's body continue to exist in your main workspace after the function finishes running.
3. **Predict the output.** If a function argument is named `x`, and your main workspace also has a variable called `x`, does the function's internal use of `x` affect the global one?
4. **Short answer.** Why is it considered safe practice to reuse short variable names like `x` or `n` inside many different functions?

Show answers

1. **c)** 100 — the global `age` is completely unaffected by the function's local reassignment.

2. **False.** Local variables exist only for the duration of the function call and disappear once it finishes.
3. No — the function's `x` is entirely local and isolated from any global variable of the same name, unless the function explicitly returns a value that you then reassign to the global `x` yourself.
4. Because each function call creates its own isolated local scope, reusing a name like `x` in one function has no effect on any variable of the same name used elsewhere, including in your main workspace or inside other functions.

117. Anonymous Functions

The Problem

You want to quickly compute the range (max minus min) of `bmi`, just this once, without needing to reuse the calculation anywhere else afterward. Writing a full named function (`range_width <- function(x) {...}`, Page 2’s practice problem) just to use it a single time feels like overkill — you don’t need a name for something you’ll never call again.

Your Mission

Write a throwaway, unnamed (“anonymous”) function for a one-off calculation, without assigning it a name first.

Think Before Coding

- Every function you’ve written so far has followed the pattern `name <- function(...) {...}`. What part of that pattern is actually *required* for a function to work, versus just a convenience for reusing it later?
- If you never plan to call a function more than once, does it still need a name at all?

117.1. The R Concept

A function doesn’t need to be assigned a name to be used immediately — this is called an **anonymous function**:

117. Anonymous Functions

```
function(x) max(x, na.rm = TRUE) - min(x, na.rm = TRUE)
```

R also provides a shorter syntax for anonymous functions, using a backslash `\` instead of the full word `function`:

```
\(x) max(x, na.rm = TRUE) - min(x, na.rm = TRUE)
```

Both versions define the exact same function — the `\(x)` shorthand is simply less typing, and is especially common when passing a quick function directly into another function (like `sapply()`, previewed next page), rather than defining it separately first.

117.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
# Anonymous function, called immediately - no name assigned  
(function(x) max(x, na.rm = TRUE) - min(x, na.rm = TRUE))(participants$bmi)
```

```
[1] 23
```

```
# The shorthand \() syntax - identical result  
(\() max(x, na.rm = TRUE) - min(x, na.rm = TRUE))(participants$bmi)
```

```
[1] 23
```

Both versions compute the exact same range width for `bmi` ($38.6 - 15.6 = 23$) without ever creating a named, reusable function object — appropriate here, since this calculation is only needed this one time.

117.3. Explore

Try wrapping the same anonymous function around `participants$age` instead of `participants$bmi`. Notice you can reuse the *pattern* freely by copy-pasting it — what you’re avoiding isn’t reuse of the code itself, but the overhead of formally naming and storing a function object you’ll only call once.

117.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

age_range_width <- (\(x) max(x, na.rm = TRUE) - min(x, na.rm = TRUE))(participants$age)
bmi_range_width <- (\(x) max(x, na.rm = TRUE) - min(x, na.rm = TRUE))(participants$bmi)

age_range_width

[1] 63

bmi_range_width

[1] 23
```

Both range widths (63 for age, 23 for BMI) computed correctly using the shorthand anonymous function syntax, with no named function ever created or stored for later reuse.

117.5. Interpret the Result

Anonymous functions are most useful exactly in situations like this: a one-off calculation, or (as the next page shows) a quick calculation passed directly as an argument into another function like `sapply()`. Using a full named function definition for something this disposable would add unnecessary ceremony to genuinely one-time code.

Common Mistakes

- **Using an anonymous function for something you actually need to call repeatedly.** If you find yourself copy-pasting the same anonymous function multiple times, that's a signal (echoing Page 1) it should become a properly named function instead — anonymous functions are for genuine one-off use, not disguised repetition.
- **Forgetting the parentheses needed to call an anonymous function immediately.** Writing `\(x) x + 1(5)` doesn't call the function with 5 as intended — the whole anonymous function definition needs to be wrapped in its own parentheses first, then called: `(\ (x) x + 1)(5)`.
- **Confusing the modern `\(x)` shorthand with older R syntax.** The backslash shorthand was introduced in R 4.1 — code written for older R versions will only use the full `function(x)` form; both are equivalent in current R, so either is fine to use going forward.

117.6. Practice Problem

Write an anonymous function (using the `\(x)` shorthand) that computes the coefficient of variation (`sd(x, na.rm = TRUE) / mean(x, na.rm = TRUE) * 100`) and apply it immediately to `participants$systolic_bp`.

117.7. Challenge Problem

Explain, in your own words, the difference between when you'd choose a named function (Pages 2-6) versus an anonymous one (this page). Give one example scenario from this course so far where each would be the more appropriate choice.

117.8. Key Takeaways

- An anonymous function has no name and can be defined and called immediately, appropriate for genuinely one-off calculations.
- R's `\(x)` shorthand is equivalent to the full `function(x)` syntax, just less typing — especially handy when passing a quick function directly as an argument to another function.
- Repeated use of the same anonymous function is a signal it should become a properly named, reusable function instead.

Knowledge Check

1. **Multiple choice.** What does `\(x)` mean in modern R syntax?
 - a) A comment
 - b) Shorthand for defining an anonymous function
 - c) A file path separator
 - d) An error-checking operator
2. **True or false.** An anonymous function must always be assigned to a variable name before it can be called.
3. **Predict the output.** What would `\(x) x * 2`(10) return?
4. **Short answer.** What's the clearest signal that an anonymous function should instead be rewritten as a named one?

117. *Anonymous Functions*

Show answers

1. **b)** Shorthand for defining an anonymous function (equivalent to `function(x)`).
2. **False.** An anonymous function can be defined and called immediately, with no name ever assigned.
3. 20.
4. If the same anonymous function is being copy-pasted and reused in multiple places, that repetition is the same signal from Page 1 indicating the code should become one properly named, reusable function instead.

118. Applying a Function Across Columns: `sapply()` and `lapply()`

The Problem

You have `pct_missing()` (Page 2), a working function for one column at a time. But `participants` has 8 columns, and typing `pct_missing(participants$id)`, `pct_missing(participants$age)`, `pct_missing(participants$sex)` ... eight separate times is exactly the repetition functions were supposed to eliminate in the first place. You need to apply `pct_missing()` to *every column at once*, automatically.

Your Mission

Apply a function across every column of a data frame at once using `sapply()` and `lapply()`.

Think Before Coding

- Level 2 introduced vectorized operations like `x + 1`, which apply automatically to every element of a vector. Could a *function* similarly be applied automatically to every *column* of a data frame?
- A data frame is really a list of columns underneath. If a function could be applied to every element of a list, would that also apply it to every column of a data frame?

118. Applying a Function Across Columns: `sapply()` and `lapply()`

118.1. The R Concept

`sapply()` (“simplify apply”) applies a function to every element of a vector or list — including every column of a data frame — and simplifies the result into a vector when possible:

```
sapply(df, function_name)
```

`lapply()` (“list apply”) does the exact same thing, but always returns a **list** instead of simplifying — useful when the results don’t simplify neatly into a single vector:

```
lapply(df, function_name)
```

118.2. Demonstration

```
pct_missing <- function(x) {  
  sum(is.na(x)) / length(x) * 100  
}
```

```
participants <- read.csv("data/participants.csv")
```

```
sapply(participants, pct_missing)
```

id	age	sex	county	systolic_bp	diastolic_bp
0	6	0	0	4	0
bmi	smoker				
5	0				

One call, `sapply(participants, pct_missing)`, applied `pct_missing()` to all 8 columns automatically — no more retyping `pct_missing(participants$...)` eight separate times.

118.3. Explore

```
lapply(participants, pct_missing)

$id
[1] 0

$age
[1] 6

$sex
[1] 0
... (output continues as a list, one entry per column)
```

`lapply()` produces the same underlying values, but as a list (one named entry per column) rather than `sapply()`'s simplified named vector.

118.3. Explore

Try `sapply(participants, class)` — applying the built-in `class()` function (Level 2) across every column instead of `pct_missing()`. This confirms `sapply()` works with *any* function, built-in or custom, as long as it's designed to take a single column as input.

118.4. Solve the Problem

```
pct_missing <- function(x) {
  sum(is.na(x)) / length(x) * 100
}

participants <- read.csv("data/participants.csv")
```

118. Applying a Function Across Columns: `sapply()` and `lapply()`

```
missing_report <- sapply(participants, pct_missing)
```

```
missing_report
```

id	age	sex	county	systolic_bp	diastolic_bp
0	6	0	0	4	0
bmi	smoker				
5	0				

```
missing_report[missing_report > 0]
```

age	systolic_bp	bmi
6	4	5

`sapply()` computed the missing-value percentage for all 8 columns in one call, and a follow-up logical filter (`missing_report > 0`, Level 2) immediately narrows the result down to just the 3 columns that actually have any missingness — `age` (6%), `systolic_bp` (4%), and `bmi` (5%).

118.5. Interpret the Result

This is exactly the level’s opening mission delivered: `pct_missing()` was written once (Page 2), and `sapply()` applied it across the entire dataset automatically — no repetition, no retyping the calculation eight times, and the result immediately usable for further filtering (`missing_report > 0`) to focus attention on only the columns that need it.

Common Mistakes

- **Forgetting `sapply()` and `lapply()` pass one column at a time to the function, not the whole data frame at once.** `pct_missing()` receives `participants$age` as its `x` on one call,

then `participants$systolic_bp` on the next — never the entire `participants` data frame in a single call.

- **Expecting `sapply()` to always return a clean vector.** `sapply()` simplifies when it can, but if a function's results don't have a consistent length or type across every column, `sapply()` falls back to returning a list — the same result `lapply()` would have given directly.
- **Applying a function written for a single value (like `bp_category()`, Page 4) directly to an entire column without `sapply()`.** `bp_category()`'s `if` statement expects one number, not a vector — calling `bp_category(participants$systolic_bp)` directly errors; wrapping it as `sapply(participants$systolic_bp, bp_category)` (or vectorizing the function itself) is what correctly applies it across every value.

118.6. Practice Problem

Use `sapply(participants, class)` to check the storage type of every column. Which columns are "character", and which are "integer" or "numeric"?

118.7. Challenge Problem

Write a function called `n_unique` that returns the number of distinct non-missing values in a vector (hint: use `length(unique(...))` combined with removing NAs first). Apply it with `sapply()` across every column of `participants`, and confirm `county` shows exactly 5 distinct values, matching Level 4's finding.

118.8. Key Takeaways

- `sapply(df, function_name)` applies a function to every column of a data frame, simplifying the result into a vector when possible.
- `lapply(df, function_name)` does the same thing but always returns a list, useful when results don't simplify neatly.
- Both `sapply()` and `lapply()` pass one column at a time to the function — the function itself is still written exactly as if handling a single vector, exactly as on Pages 2-4.

Knowledge Check

1. **Multiple choice.** What does `sapply(participants, pct_missing)` do?
 - a) Applies `pct_missing()` to the entire data frame as one single input
 - b) Applies `pct_missing()` separately to each column, simplifying the results into a vector
 - c) Filters `participants` to only rows with missing values
 - d) Modifies `participants` in place
2. **True or false.** `lapply()` always returns a simplified vector, exactly like `sapply()`.
3. **Predict the output.** Given `sapply(participants, pct_missing)`'s result, what would `missing_report["bmi"]` return?
4. **Find the error.**

```
bp_category(participants$systolic_bp)
# intended: classify every participant's blood pressure at once
```

Show answers

118.8. Key Takeaways

1. **b)** Applies `pct_missing()` separately to each column, simplifying the results into a vector.
2. **False.** `lapply()` always returns a list; `sapply()` attempts to simplify the result into a vector (or matrix) when possible.
3. 5 — the percentage of missing `bmi` values.
4. `bp_category()` (Page 4) was written expecting a single value, using `if`, which errors on a vector input. It should be wrapped with `sapply()`: `sapply(participants$systolic_bp, bp_category)`, applying the single-value function across every element correctly.

119. The Tidyverse Way: `purrr::map()`

The Problem

`sapply()` (previous page) works, but it has a quirk: its return type depends on the data. Sometimes it simplifies to a vector, sometimes it silently returns a list instead, depending on what the function produced for each column — code that works today could silently break tomorrow if the data changes shape slightly. Your professor, having read a modern tidyverse-based analysis script, asks why it uses `map()` instead of `sapply()`.

Your Mission

Apply a function across a data frame's columns using `purrr::map()` and its type-specific variants, `map_dbl()` and `map_chr()`.

Think Before Coding

- If `sapply()`'s return type can vary unpredictably depending on the data, would a function that always promises a specific return type (like “always a number”) be more predictable to build further code on top of?
- `pct_missing()` always returns a single number. Which `purrr` function do you suspect is named for guaranteeing a “double” (numeric) result every time?

119.1. The R Concept

`purrr::map()` (part of the tidyverse) applies a function across a vector or data frame's columns, always returning a **list** — predictable, but not simplified:

```
library(purrr)
map(df, function_name)
```

Its type-specific variants guarantee a specific output type, erroring loudly instead of silently returning the wrong shape if a function doesn't produce what's expected:

```
map_dbl(df, function_name) # always returns a numeric vector, or errors
map_chr(df, function_name) # always returns a character vector, or errors
```

This predictability is `purrr`'s core advantage over base R's `sapply()`: you know exactly what type of result to expect before you even run the code, and get an immediate, clear error if a function doesn't honor that promise — rather than a silently different-shaped result to debug later.

119.2. Demonstration

```
library(purrr)

pct_missing <- function(x) {
  sum(is.na(x)) / length(x) * 100
}

participants <- read_csv("data/participants.csv")

map_dbl(participants, pct_missing)
```

119.3. Explore

id	age	sex	county	systolic_bp	diastolic_bp
0	6	0	0	4	0
bmi	smoker				
5	0				

The exact same values `sapply()` produced on the previous page — but `map_dbl()` *guarantees* this is a numeric vector, since `pct_missing()` always returns a single number for any input.

119.3. Explore

Try `map_chr(participants, class)` — applying the built-in `class()` function (which always returns a character value) across every column. Since `class()` always produces text, `map_chr()` is the appropriately type-guaranteed choice here, rather than the more general `map()` or the numeric-specific `map_dbl()`.

119.4. Solve the Problem

```
library(purrr)
library(dplyr)

pct_missing <- function(x) {
  sum(is.na(x)) / length(x) * 100
}

participants <- read.csv("data/participants.csv")

missing_report <- map_dbl(participants, pct_missing)

missing_report[missing_report > 0]
```

119. The Tidyverse Way: `purrr::map()`

```
      age systolic_bp      bmi
      6         4         5

column_types <- map_chr(participants, class)

column_types

      id      age      sex      county systolic_bp diastolic_bp
"integer" "integer" "character" "character"  "integer"  "integer"
      bmi      smoker
"numeric" "character"
```

Both the missing-value percentages and the column types are computed with guaranteed, predictable output types — `map_dbl()` promising numbers, `map_chr()` promising text — matching `sapply()`'s results from the previous page exactly, just with an explicit type contract attached.

119.5. Interpret the Result

The results are identical to `sapply()`'s (Page 8) — the practical difference only shows up when something unexpected happens. If `pct_missing()` accidentally returned text for one column (a bug), `map_dbl()` would immediately error, pointing you straight to the problem; `sapply()` might instead silently return a mismatched list, leaving you to notice the problem much later, and less obviously.

Common Mistakes

- **Choosing `map()` when a type-specific variant is available and appropriate.** Plain `map()` always returns a list, even when every result is a single number — `map_dbl()` is almost always the better choice when you know every result should be numeric, since it also validates that promise.

119.6. Practice Problem

- **Mismatching the variant to what the function actually returns.** Using `map_chr()` on a function that returns numbers (like `pct_missing()`) causes an immediate error — the variant you choose must match what the applied function actually produces.
- **Forgetting library(purrr).** Like every tidyverse package, `purrr`'s functions must be loaded first — though `map()`, `map_dbl()`, and `map_chr()` are also available automatically if `library(tidyverse)` was used instead of loading individual packages.

119.6. Practice Problem

Use `map_dbl()` to apply the `range_width` function (Page 2's practice problem) across every *numeric* column of `participants` (hint: you may need to first `select()` (Level 5) only the numeric columns, since `range_width()` won't work on character columns like `county`).

119.7. Challenge Problem

Compare `sapply(participants, pct_missing)` and `map_dbl(participants, pct_missing)` side by side using `identical()`. Are their results identical in this case? Then research (or predict) a scenario in which `sapply()`'s output would differ in *type* from `map_dbl()`'s, even if the underlying values were the same.

119.8. Key Takeaways

- `purrr::map()` and its type-specific variants (`map_dbl()`, `map_chr()`) apply a function across a data frame's columns, similar to `sapply()/lapply()`.

119. The Tidyverse Way: `purrr::map()`

- Type-specific variants guarantee a specific output type, erroring clearly if a function doesn't honor that promise — more predictable than `sapply()`'s sometimes-simplifies, sometimes-doesn't behavior.
- Choosing the right variant (`map_dbl()` for numbers, `map_chr()` for text) should match what the applied function is actually known to return.

Knowledge Check

1. **Multiple choice.** What does `map_dbl()` guarantee about its return value?
 - a) Nothing — it behaves identically to `sapply()`
 - b) It always returns a numeric vector, or errors if the function doesn't produce numbers
 - c) It always returns a list
 - d) It always returns a character vector
2. **True or false.** `map()` (with no suffix) always returns a list, regardless of what the applied function produces.
3. **Predict the output.** If `pct_missing()` always returns a single number, would `map_dbl(participants, pct_missing)` ever error?
4. **Find the error.**

```
map_chr(participants, pct_missing)
# pct_missing() returns numbers, not text
```

Show answers

1. **b)** It always returns a numeric vector, or errors if the applied function doesn't produce numbers.
2. **True.** Plain `map()` always returns a list, unlike `sapply()`, which sometimes simplifies and sometimes doesn't.
3. No — since `pct_missing()` reliably returns a single number for any valid input, `map_dbl()` would never error on it.

119.8. Key Takeaways

4. `map_chr()` expects the applied function to return character (text) values, but `pct_missing()` returns numbers — this mismatch causes `map_chr()` to error immediately. It should be `map_dbl(participants, pct_missing)` instead, matching the function's actual (numeric) return type.

120. Basic Error Handling: `stop()` and `warning()`

The Problem

A colleague uses your `pct_missing()` function, but accidentally passes it an empty vector (`pct_missing(numeric(0))`) — and it silently returns `NaN` (“not a number”, from dividing by zero) instead of a clear error explaining what went wrong. They spend twenty minutes confused about why their report shows `NaN` before realizing the actual mistake was upstream. Your function should have caught this immediately.

Your Mission

Add basic input validation to a function using `stop()` (for errors that should halt execution) and `warning()` (for concerning-but-not-fatal situations).

Think Before Coding

- If a function receives input that makes its calculation meaningless (like an empty vector), would you rather it fail immediately with a clear message, or silently return a confusing result like `NaN`?
- What’s the difference between a problem serious enough to stop a function entirely, versus one worth flagging but not necessarily fatal?

120.1. The R Concept

`stop()` immediately halts a function's execution and raises an error with a custom message — appropriate when the function genuinely cannot produce a meaningful result:

```
if (some_bad_condition) {  
  stop("A clear message explaining what went wrong")  
}
```

`warning()` prints a caution message but lets the function **continue running** — appropriate for situations worth flagging but not necessarily fatal:

```
if (some_concerning_condition) {  
  warning("A message noting something worth double-checking")  
}
```

120.2. Demonstration

```
pct_missing <- function(x) {  
  if (length(x) == 0) {  
    stop("x has length 0 - cannot compute percent missing")  
  }  
  sum(is.na(x)) / length(x) * 100  
}  
  
pct_missing(numeric(0))
```

```
Error in pct_missing(numeric(0)) : x has length 0 - cannot compute percent m.
```

Instead of silently returning a confusing `NaN`, the function now stops immediately with a message explaining exactly what went wrong — much easier to debug than discovering an unexplained `NaN` several steps later in a report.

120.3. Explore

Try calling the updated `pct_missing()` on a valid column, like `participants$age`. Confirm it still works exactly as before for legitimate input — `stop()` only interrupts execution when the specific bad condition (`length(x) == 0`) is actually met, not for every call.

120.4. Solve the Problem

```
pct_missing <- function(x) {
  if (length(x) == 0) {
    stop("x has length 0 - cannot compute percent missing")
  }
  if (all(is.na(x))) {
    warning("Every value in x is missing - result will be 100")
  }
  sum(is.na(x)) / length(x) * 100
}

participants <- read.csv("data/participants.csv")

pct_missing(participants$age)

[1] 6

pct_missing(c(NA, NA, NA))
```

120. Basic Error Handling: `stop()` and `warning()`

Warning message:

```
In pct_missing(c(NA, NA, NA)) : Every value in x is missing - result will be
[1] 100
```

```
pct_missing(numeric(0))
```

```
Error in pct_missing(numeric(0)) : x has length 0 - cannot compute percent m
```

Three different inputs, three different appropriate responses: a valid column runs normally (6%), an all-missing vector runs but with a warning attached (100%, flagged as worth double-checking), and a genuinely invalid empty vector stops execution entirely with a clear explanation.

120.5. Interpret the Result

`pct_missing()` now protects both the function's caller and anyone reading its output: a `stop()`-triggered error message makes an invalid call immediately obvious rather than producing a confusing silent result, while a `warning()` flags a technically valid but concerning result (100% missing) without halting a larger script that might process many columns in a loop.

Common Mistakes

- **Using `warning()` when the situation actually warrants `stop()`.** A `warning()` lets execution continue — if the function truly cannot produce a meaningful result (like the empty-vector case), continuing anyway just produces a bad result further downstream; `stop()` is the right choice whenever continuing would be actively misleading.

120.6. Practice Problem

- **Writing a vague error message.** `stop("Error!")` tells a caller nothing about what actually went wrong or how to fix it — a good `stop()` message states the specific problem, as in `"x has length 0 - cannot compute percent missing"`.
- **Adding so much validation that a function becomes hard to use for legitimate edge cases.** Not every unusual input needs a `stop()` or `warning()` — reserve them for situations that are genuinely invalid or genuinely worth flagging, not just unusual.

120.6. Practice Problem

Add a `stop()` check to `bp_category()` (Page 4) that halts execution with a clear message if `sbp` is negative (an impossible blood pressure reading). Test it with a normal value and a negative one.

120.7. Challenge Problem

Modify `is_at_risk()` (Page 5) to `warning()` (not `stop()`) if `age_cutoff` is set below 0 or above 120 (an implausible age threshold), while still allowing the function to run. Explain why `warning()` is more appropriate than `stop()` for this particular situation, compared to the empty-vector case above.

120.8. Key Takeaways

- `stop()` immediately halts a function with a custom error message, appropriate when the function truly cannot produce a meaningful result.
- `warning()` prints a caution message but allows the function to continue running, appropriate for concerning-but-not-fatal situations.

120. Basic Error Handling: `stop()` and `warning()`

- A good error or warning message states the specific problem clearly, helping whoever encounters it fix the actual issue quickly.

Knowledge Check

1. **Multiple choice.** What is the key difference between `stop()` and `warning()`?
 - a) `stop()` prints a message and continues; `warning()` halts execution
 - b) `stop()` halts execution; `warning()` prints a message and continues
 - c) They behave identically
 - d) `warning()` can only be used inside `if` statements
2. **True or false.** A function should use `stop()` for every unusual or unexpected input, no matter how minor.
3. **Predict the output.** What would `pct_missing(numeric(0))` do, given the version of the function with the `length(x) == 0` check added?
4. **Short answer.** Why is a specific, descriptive error message (like "x has length 0 - cannot compute percent missing") more useful than a generic one (like "Error!")?

Show answers

1. **b)** `stop()` halts execution with an error; `warning()` prints a message but lets the function continue running.
2. **False.** Overusing `stop()/warning()` for minor or merely unusual inputs makes a function harder to use — reserve them for genuinely invalid or genuinely concerning situations.
3. It immediately stops with the error message "x has length 0 - cannot compute percent missing", rather than returning a confusing NaN.

120.8. Key Takeaways

4. A specific message tells whoever encounters the error exactly what went wrong and often hints at how to fix it, saving significant debugging time compared to a generic message that provides no clue about the actual problem.

121. Level 9 Capstone: A Complete Data Quality Report

The Problem

Your professor’s final request for this level: a single, reusable “data quality report” tool — one function that, applied to any column, reports what type it is, what percentage is missing, and either its mean/SD (numeric columns) or its number of distinct categories (character columns). It needs to validate its input, handle both column types correctly, and apply across every column in `participants` in one call.

Your Mission

Combine everything from this level — conditionals, default arguments, scope, and function application (`map_chr()`) — into one polished, reusable function.

Think Before Coding

- From Page 4, how would you write a conditional that behaves differently for a numeric column versus a character column, using `is.numeric()`?
- From Page 10, what check would you add first, before any calculation, to make sure the function fails clearly on invalid input rather than silently?

121.1. The R Concept

A complete, well-built function combines every idea from this level: input validation (Page 10) first, then conditional logic (Page 4) branching on the input's type, using default arguments (Page 3) for adjustable behavior, all wrapped as one named, reusable tool (Page 2) that can be applied across many columns at once (Page 8-9):

```
column_report <- function(x, digits = 2) {  
  if (length(x) == 0) {  
    stop("x has length 0 - cannot build a report")  
  }  
  pct_miss <- round(sum(is.na(x)) / length(x) * 100, digits)  
  
  if (is.numeric(x)) {  
    paste0("numeric | ", pct_miss, "% missing | mean = ",  
          round(mean(x, na.rm = TRUE), digits), ", sd = ",  
          round(sd(x, na.rm = TRUE), digits))  
  } else {  
    paste0("character | ", pct_miss, "% missing | ",  
          length(unique(x[!is.na(x)])), " unique values")  
  }  
}
```

121.2. Demonstration

```
column_report <- function(x, digits = 2) {  
  if (length(x) == 0) {  
    stop("x has length 0 - cannot build a report")  
  }  
  pct_miss <- round(sum(is.na(x)) / length(x) * 100, digits)
```

```

if (is.numeric(x)) {
  paste0("numeric | ", pct_miss, "% missing | mean = ",
        round(mean(x, na.rm = TRUE), digits), ", sd = ",
        round(sd(x, na.rm = TRUE), digits))
} else {
  paste0("character | ", pct_miss, "% missing | ",
        length(unique(x[!is.na(x)])), " unique values")
}
}

participants <- read.csv("data/participants.csv")

column_report(participants$age)

[1] "numeric | 6% missing | mean = 47.43, sd = 15.03"

column_report(participants$county)

[1] "character | 0% missing | 5 unique values"

```

One function, correctly branching between numeric and character summaries depending entirely on the column's type — exactly the behavior your professor asked for.

121.3. Explore

Try `column_report(participants$smoker)` and `column_report(participants$bmi)`. Confirm the numeric branch and character branch each produce sensible, correctly formatted output — and try `column_report(numeric(0))` to confirm the `stop()` check still works exactly as it did on Page 10.

121.4. Solve the Problem

```
library(purrr)

column_report <- function(x, digits = 2) {
  if (length(x) == 0) {
    stop("x has length 0 - cannot build a report")
  }
  pct_miss <- round(sum(is.na(x)) / length(x) * 100, digits)

  if (is.numeric(x)) {
    paste0("numeric | ", pct_miss, "% missing | mean = ",
           round(mean(x, na.rm = TRUE), digits), ", sd = ",
           round(sd(x, na.rm = TRUE), digits))
  } else {
    paste0("character | ", pct_miss, "% missing | ",
           length(unique(x[!is.na(x)])), " unique values")
  }
}

participants <- read.csv("data/participants.csv")

full_report <- map_chr(participants, column_report)

full_report

              id              age              sex
"numeric | 0% missing... "numeric | 6% missing... "character | 0% mi...
              county          systolic_bp          diastolic_bp
"character | 0% mi... "numeric | 4% missing... "numeric | 0% missing...
              bmi              smoker
"numeric | 5% missing... "character | 0% mi...

full_report["age"]
```

121.5. Interpret the Result

```
age
"numeric | 6% missing | mean = 47.43, sd = 15.03"

full_report["county"]
```

```
county
"character | 0% missing | 5 unique values"
```

One `map_chr()` call generated a complete, correctly branched report for all 8 columns at once — the exact tool your professor asked for, built entirely from pieces developed one at a time across this level.

121.5. Interpret the Result

`column_report()` correctly identifies `age`, `systolic_bp`, `diastolic_bp`, and `bmi` as numeric (reporting mean and SD), and `sex`, `county`, and `smoker` as character (reporting the count of distinct categories) — with `id`, though numeric, also handled correctly by the same branch, even though a mean and SD of an ID number isn't particularly meaningful. This is worth noting as a real limitation: the function correctly determines *type*, but doesn't know which numeric columns are meaningfully continuous (like `age`) versus arbitrary identifiers (like `id`) — a distinction still requiring your own judgment as the analyst.

Common Mistakes

- **Assuming a function that runs without errors is automatically producing meaningful output.** `column_report(participants$id)` runs successfully and reports a mean and SD, but an ID column's "average" isn't a meaningful statistic — a function correctly executing its logic isn't the same as that logic being the right thing to compute for every column.

121. Level 9 Capstone: A Complete Data Quality Report

- **Forgetting `is.numeric()` and `is.character()` aren't the only possible column types.** This function's `if (is.numeric(x)) {...} else {...}` treats everything non-numeric as the character branch — a logical (TRUE/FALSE) or factor column would fall into that same `else` branch, potentially producing a less appropriate summary than a dedicated branch for that type would.
- **Building one big function instead of composing several small, tested ones.** `column_report()` reuses ideas (validation, conditionals, rounding) developed and tested separately across this level's earlier pages — building complex functionality out of smaller, already-verified pieces is more reliable than writing one large function from scratch in a single attempt.

121.6. Practice Problem

Modify `column_report()` to accept a `digits` value of 1 instead of the default 2, and confirm the numeric branch's mean/SD values change precision accordingly for `participants$systolic_bp`.

121.7. Challenge Problem

Extend `column_report()` with a third branch specifically for logical (TRUE/FALSE) columns, reporting the percentage TRUE instead of a mean/SD or unique-value count. Since `participants` doesn't currently have a logical column, first create one with `mutate(is_senior = age >= 65)` (Level 5) to test your new branch.

121.8. Key Takeaways

- A well-built function combines input validation, conditional branching, sensible defaults, and clear return values — each individually simple, together forming a genuinely useful tool.
- A function running without error doesn't guarantee its output is meaningful for every possible input — type-checking logic (`is.numeric()`) doesn't know which numeric columns are conceptually continuous versus arbitrary identifiers.
- Building complex functionality by composing smaller, already-tested pieces (as this capstone did, reusing ideas from Pages 2-10) is more reliable than writing one large function from scratch.

Knowledge Check

1. **Multiple choice.** What does `column_report()`'s `if (is.numeric(x))` branch handle?
 - a) Only columns explicitly named "numeric"
 - b) Any column where `is.numeric()` returns `TRUE`, like `age` or `bmi`
 - c) Character columns only
 - d) Columns with no missing values
2. **True or false.** A function running without producing an error guarantees its output is meaningful for every input.
3. **Predict the output.** What would `column_report(participants$sex)` report, given `sex` has 2 distinct values and no missing data?
4. **Short answer.** Why is `id`'s mean and SD, while technically computed correctly, not a particularly meaningful piece of information?

Show answers

1. **b)** Any column where `is.numeric()` returns `TRUE` — this includes `id`, `age`, `systolic_bp`, `diastolic_bp`, and `bmi`.

121. *Level 9 Capstone: A Complete Data Quality Report*

2. **False.** A function can execute successfully and still produce output that isn't meaningful for a particular input, as `column_report(participants$id)` demonstrates.
3. `"character | 0% missing | 2 unique values"`.
4. `id` is an arbitrary identifier, not a meaningfully continuous measurement — its “average” value (50.5) doesn't describe anything about the participants themselves, unlike `age`'s mean, which genuinely summarizes a real characteristic of the sample.

Part X.

**Level 10: Loops and
Automation**

Level 10 — Loops and Automation

Where this level is headed

Some tasks — cleaning ten site files the same way, running the same table for twelve subgroups — are naturally repetitive. This level covers `for` loops, `while` loops, and the `apply()` family so you never have to do the same thing twice by hand.

What you'll build toward

Level 9 already covered *applying a function you write* across many columns at once (`sapply()`, `purrr::map()`). This level covers the more general tool underneath: the explicit loop — `for` and `while` — plus two base R loop-replacements Level 9 didn't reach (`vapply()`, `tapply()`), and, crucially, when a loop is actually the right tool versus when a vectorized approach (Level 2 onward) is faster and more readable. You'll finish by automating a genuinely repetitive real task: your data arrives as five separate, identically structured clinic files, and you'll write the loop that processes all five in one run.

Topics in this level

- Why explicit loops matter, even after learning `sapply()/map()` in Level 9
- `for` loops: iterating over a vector or sequence

Level 10 — Loops and Automation

- Accumulating results across loop iterations
- `while` loops: repeating until a condition is no longer true
- Looping over a data frame's columns
- Nested loops (a loop inside a loop)
- `vapply()` (a stricter `sapply()`) and `tapply()` (grouped summaries without `group_by()`)
- When to loop, and when to prefer a vectorized approach instead
- A capstone: automating the same cleaning steps across five separate files

A preview mission

Your mission across this level: your team collects data from five clinics every month, each landing in an identically structured file (`data/clinics/clinic_*.csv`). Automate reading, checking, and combining all five into one dataset with a single loop, so the process takes one run instead of five separate ones — and confirm the combined result exactly reproduces the familiar 100-participant `participants.csv` you've used since Level 1.

122. for Loops

The Problem

Level 9's `sapply()` and `map_dbl()` applied a function across columns automatically, hiding the actual mechanics of “do this once per item” from view. Your professor asks a more basic question: before you had `sapply()`, how would you print each of the five county names in `participants$county`'s distinct values, one at a time, explicitly controlling each step yourself?

Your Mission

Write a `for` loop that repeats a block of code once for each element of a vector.

Think Before Coding

- If you wanted to print each of 5 county names one at a time, how many times would the code inside your loop need to run?
- Does a `for` loop need to know in advance exactly how many times it will run, before it starts?

122.1. The R Concept

A **for loop** repeats a block of code once for every element in a vector, giving each element a temporary name you choose:

122. *for* Loops

```
for (item in vector) {  
  # code using item, repeated once per element of vector  
}
```

Each time through the loop, `item` takes on the next value in `vector`, in order, until every element has been used once.

122.2. Demonstration

```
counties <- c("Cedar Falls", "Millbrook", "Oakford", "Riverdale", "Springvale")
```

```
for (county in counties) {  
  print(county)  
}
```

```
[1] "Cedar Falls"  
[1] "Millbrook"  
[1] "Oakford"  
[1] "Riverdale"  
[1] "Springvale"
```

The loop ran exactly 5 times — once per county name — printing each one in the same order it appeared in `counties`, with `county` taking on each successive value automatically.

122.3. Explore

Try looping over `1:5` instead of the county names: `for (i in 1:5) { print(i * 10) }`. Confirm the loop runs exactly 5 times, with `i` taking values 1 through 5 in order — the loop variable doesn't have to be a meaningful real-world value; it's often just a counter.

122.4. Solve the Problem

```

participants <- read.csv("data/participants.csv")

distinct_counties <- unique(participants$county)

for (county in distinct_counties) {
  n_participants <- sum(participants$county == county)
  print(paste(county, ":", n_participants, "participants"))
}

[1] "Oakford : 17 participants"
[1] "Riverdale : 16 participants"
[1] "Millbrook : 21 participants"
[1] "Springvale : 26 participants"
[1] "Cedar Falls : 20 participants"

```

The loop ran once per distinct county (in the order `unique()` first encountered them in the data, not alphabetically), computing and printing each county's participant count — matching exactly the counts Level 4's `table()` and Level 5's `count()` already established.

122.5. Interpret the Result

This `for` loop reproduces information you already had from `table(participants$county)` (Level 4) — but explicitly, one county at a time, with full control over exactly what happens on each iteration (here, a custom formatted sentence rather than a bare count). This explicitness is both the loop's strength (you can do anything inside it) and its verbosity (Level 9's `sapply()/map()` would get you the raw counts in one line, without a loop at all).

122. *for* Loops

Common Mistakes

- **Forgetting a *for* loop's results disappear unless explicitly saved.** `print()` inside the loop displays each value as it runs, but doesn't save anything — if you need the results *after* the loop finishes, you must store them somewhere (Page 2 covers exactly this).
- **Looping over the wrong thing.** `for (county in nrow(participants))` loops over a single number (the row count), not over each county — a common typo-like mistake is looping over a summary of the data instead of the data (or its distinct values) itself.
- **Assuming a *for* loop is always the best tool.** As Level 9 showed, `sapply()/map()` often replace a *for* loop entirely, more concisely — *for* loops remain most useful when you need explicit step-by-step control, or when later steps depend on earlier ones (Page 7 explores this distinction further).

122.6. Practice Problem

Write a *for* loop over `unique(participants$smoker)` that prints each smoking status alongside its count, similar to the county example above.

122.7. Challenge Problem

Write a *for* loop over `1:5` that prints the squares of 1 through 5 (i^2). Then rewrite the same task using `sapply(1:5, function(i) i^2)` (Level 9) and confirm both approaches produce the same 5 values, just via different mechanics.

122.8. Key Takeaways

- `for (item in vector) { ... }` repeats a block of code once per element of `vector`, with `item` taking each successive value in turn.
- A `for` loop's results are not automatically saved anywhere — anything you want to keep must be explicitly stored (next page).
- `for` loops offer full explicit control at the cost of verbosity — Level 9's `sapply()/map()` often replace simple loops more concisely, but loops remain useful for more complex, step-dependent tasks.

Knowledge Check

1. **Multiple choice.** In `for (county in counties) { print(county) }`, how many times does the loop run?
 - a) Exactly once
 - b) Once per element of `counties`
 - c) Infinitely, until manually stopped
 - d) Zero times, since nothing is returned
2. **True or false.** Values printed inside a `for` loop are automatically saved to a vector for later use.
3. **Predict the output.** How many times would `for (i in 1:5) { print(i) }` print a value?
4. **Find the error.**

```
for (county in nrow(participants)) {
  print(county)
}
# intended: print each distinct county name
```

Show answers

1. **b)** Once per element of `counties` — 5 times, for 5 county names.

122. *for* Loops

2. **False.** `print()` only displays values as the loop runs; nothing is saved unless you explicitly store it (Page 2).
3. 5 times, printing 1, 2, 3, 4, 5 in order.
4. `nrow(participants)` is a single number (100), so the loop runs only once, printing 100 — not once per distinct county. It should loop over `unique(participants$county)` instead.

123. Accumulating Results in a Loop

The Problem

The previous page's loop printed each county's participant count, but the moment it finished, all five printed values were gone — nothing was saved. Your professor wants the actual counts stored somewhere usable afterward: for further calculation, for plotting, for anything beyond just looking at printed text.

Your Mission

Store, or “accumulate,” results produced inside a loop into a vector you can use after the loop finishes.

Think Before Coding

- If you wanted to save one number per loop iteration, and the loop runs 5 times, how many “slots” would you need to store all the results?
- Would it make sense to create an empty results container *before* the loop starts, or only after it finishes?

123.1. The R Concept

To accumulate results, create an empty (or appropriately sized) vector **before** the loop starts, then assign into a specific position of it during each iteration:

123. Accumulating Results in a Loop

```
results <- vector("numeric", length(some_vector)) # pre-allocate

for (i in seq_along(some_vector)) {
  results[i] <- some_calculation(some_vector[i])
}
```

`seq_along(x)` generates the sequence 1, 2, 3, ..., `length(x)` — useful for looping by *position* (needed here, since you must know which slot in `results` to fill on each iteration) rather than by value directly.

123.2. Demonstration

```
counties <- unique(participants$county)

county_counts <- vector("numeric", length(counties)) # pre-allocate 5 empty slots
names(county_counts) <- counties                     # label each slot

for (i in seq_along(counties)) {
  county_counts[i] <- sum(participants$county == counties[i])
}

county_counts
```

Oakford	Riverdale	Millbrook	Springvale	Cedar Falls
17	16	21	26	20

Unlike the previous page's `print()`-only loop, `county_counts` now genuinely exists as a named vector after the loop finishes — usable for further calculations, exactly like any other vector from Level 2.

123.3. Explore

Try `sum(county_counts)` after running the loop above. It should equal 100, confirming the accumulated counts across all five counties correctly sum to the full sample size — a useful sanity check whenever accumulating results across groups that should account for the whole dataset.

123.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

numeric_columns <- c("age", "systolic_bp", "diastolic_bp", "bmi")

pct_missing_results <- vector("numeric", length(numeric_columns))
names(pct_missing_results) <- numeric_columns

for (i in seq_along(numeric_columns)) {
  col_name <- numeric_columns[i]
  col_values <- participants[[col_name]]
  pct_missing_results[i] <- sum(is.na(col_values)) / length(col_values) * 100
}

pct_missing_results
```

age	systolic_bp	diastolic_bp	bmi
6	4	0	5

The exact same percentages Level 9's `sapply(participants, pct_missing)` produced — but built here explicitly, one column at a time, with the result accumulated into a pre-allocated vector rather than handled automatically.

123.5. Interpret the Result

`pct_missing_results` now exists as a genuine, reusable vector, matching the missing-value percentages already established in Level 9 (6% age, 4% systolic BP, 0% diastolic BP, 5% BMI) — confirming the explicit loop-and-accumulate approach and Level 9's `sapply()` shortcut agree exactly, as they should, since they're computing the identical thing by different means.

Common Mistakes

- **Growing a vector inside the loop instead of pre-allocating it.** Writing `results <- c(results, new_value)` inside a loop (rather than pre-allocating `results` at the correct size beforehand) works, but is slower and considered poor style for larger loops — pre-allocating with `vector("numeric", n)` first is the more robust habit to build.
- **Looping over values instead of positions when position matters.** `for (col_name in numeric_columns)` (looping by value) makes it harder to know *which slot* of a pre-allocated results vector to fill — `for (i in seq_along(numeric_columns))` (looping by position) solves this directly, since `i` gives you both the current position and (via indexing) the current value.
- **Forgetting to name the results vector.** `pct_missing_results` without `names()` assigned would just be an unlabeled vector of four numbers — much harder to interpret than one clearly labeled by column name.

123.6. Practice Problem

Write a loop that accumulates the mean value of each numeric column (`age`, `systolic_bp`, `diastolic_bp`, `bmi`) into a named results vector, using `seq_along()` and pre-allocation as shown above.

123.7. Challenge Problem

Modify the county-count loop above to also accumulate each county's mean `systolic_bp` into a second pre-allocated vector, alongside the participant counts. Confirm your accumulated means match Level 5's `group_by(county) + summarise()` results (Cedar Falls 128.16, Millbrook 133.30, Oakford 129.71, Riverdale 124.36, Springvale 131.46).

123.8. Key Takeaways

- Accumulating results requires creating a container (typically pre-allocated with `vector()`) before the loop starts, then assigning into a specific position during each iteration.
- `seq_along(x)` generates position indices (1, 2, ..., `length(x)`), useful whenever you need to know *which slot* to fill, not just the current value.
- Pre-allocating a results vector at the correct size is more efficient and more robust than growing a vector one element at a time inside a loop.

Knowledge Check

1. **Multiple choice.** What does `seq_along(c("a", "b", "c"))` return?
 - a) `c("a", "b", "c")`
 - b) `c(1, 2, 3)`
 - c) 3
 - d) `c(0, 1, 2)`
2. **True or false.** Values printed with `print()` inside a loop are automatically available in a vector after the loop finishes.

123. Accumulating Results in a Loop

3. **Predict the output.** If `results <- vector("numeric", 5)` before a loop runs, what are the initial values of `results`, before any assignment happens?
4. **Find the error.**

```
pct_missing_results <- vector("numeric", length(numeric_columns))
for (col_name in numeric_columns) {
  pct_missing_results[col_name] <- sum(is.na(participants[[col_name]]))
}
```

Show answers

1. **b)** `c(1, 2, 3)` — the position indices, not the values themselves.
2. **False.** `print()` only displays values as the loop runs — nothing is saved to a vector unless explicitly assigned into one.
3. All zeros: `c(0, 0, 0, 0, 0)` — `vector("numeric", 5)` pre-allocates 5 slots, each initialized to 0.
4. This isn't strictly wrong — R allows indexing a vector by name (`pct_missing_results[col_name]`), which would actually work correctly here despite looping by value instead of position — but it's a more fragile pattern than looping with `seq_along()`, since `results[col_name]` silently creates a new named element if `col_name` doesn't already exist as one of the vector's names, rather than erroring on a genuine mismatch.

124. while Loops

The Problem

Your professor asks a strange but genuine question: “If we kept removing our oldest participant, one at a time, how many would we need to remove before the remaining sample’s mean age drops below 40?” A **for** loop (Page 1) needs to know in advance exactly how many times to run — but here, you don’t know that number ahead of time. You only know the *condition* you’re waiting for (“mean age is still 40 or above”), not the exact iteration count.

Your Mission

Repeat a block of code until a condition is no longer true, using a **while** loop.

Think Before Coding

- A **for** loop runs a fixed, known number of times. If you don’t know in advance how many iterations you’ll need, is a **for** loop still the right tool?
- Every **while** loop needs something inside it that eventually makes its condition **FALSE** — what would that “something” be for the mean-age-removal question above?

124.1. The R Concept

A **while loop** repeats a block of code for as long as a condition remains TRUE, checking that condition again before every iteration:

```
while (condition) {  
  # code repeated as long as condition is TRUE  
  # something inside here must eventually make condition FALSE  
}
```

Unlike a **for** loop, a **while** loop doesn't know in advance how many times it will run — it simply keeps going until its condition becomes FALSE. This makes **while** loops appropriate for exactly the kind of “keep going until...” question **for** loops aren't naturally suited for.

124.2. Demonstration

```
count <- 0  
  
while (count < 5) {  
  print(count)  
  count <- count + 1  
}
```

```
[1] 0  
[1] 1  
[1] 2  
[1] 3  
[1] 4
```

The loop checks `count < 5` before every iteration, stopping the moment `count` reaches 5 — note that `count <- count + 1` inside the loop is what

eventually makes the condition `FALSE`; without it, this loop would run forever (an “infinite loop,” a common bug to watch for).

124.3. Explore

Try removing the `count <- count + 1` line from the loop above (don’t actually run it, just consider it). Since `count` would never change, `count < 5` would remain `TRUE` forever — this is exactly why every `while` loop needs something inside it that moves its condition toward becoming `FALSE`.

124.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

ages <- sort(participants$age[!is.na(participants$age)], decreasing = TRUE)

removed_count <- 0

while (mean(ages) >= 40) {
  ages <- ages[-1]          # remove the current oldest (first, since sorted descending)
  removed_count <- removed_count + 1
}

removed_count

[1] 29

mean(ages)

[1] 39.95385
```

124. *while* Loops

```
length(ages)
```

```
[1] 65
```

The loop removed the oldest participant, one at a time, checking the mean age after each removal — stopping after exactly 29 removals, once the remaining 65 participants' mean age dropped to 39.95, just below the 40 threshold.

124.5. Interpret the Result

Nearly a third of the original 94 non-missing ages (29 of 94) had to be removed — specifically, the oldest third — before the sample's mean age dropped below 40. This is a genuinely different kind of question than anything answerable with a `for` loop directly: you needed to keep going until a *condition* was met, with no way to know in advance exactly how many iterations that would take.

Common Mistakes

- **Forgetting to include something that changes the condition inside the loop.** This is the single most common `while` loop bug — an infinite loop, where the condition never becomes `FALSE` because nothing inside the loop actually moves toward that outcome.
- **Using `while` when a `for` loop (or vectorized approach) would be simpler.** If you already know exactly how many iterations you need, a `for` loop is usually clearer — `while` loops are best reserved for situations where the stopping point genuinely depends on a condition evaluated during the loop, not a known count in advance.
- **Not having an escape plan for a `while` loop that might never terminate.** If there's any chance the condition could never become `FALSE` (a data problem, a logic error), consider adding a safety

counter (e.g., stop after 1000 iterations regardless) to prevent a truly infinite loop from freezing your session.

124.6. Practice Problem

Write a `while` loop that removes the *youngest* participant, one at a time, until the remaining sample's mean age exceeds 55. How many participants need to be removed?

124.7. Challenge Problem

Modify the blood-pressure equivalent of this question: starting from all non-missing `systolic_bp` values, repeatedly remove the single highest reading until the mean drops below 125. How many removals are required, and what does this suggest about how much a handful of high readings are pulling up the overall mean?

124.8. Key Takeaways

- `while (condition) { ... }` repeats a block of code for as long as `condition` remains `TRUE`, without needing to know the iteration count in advance.
- Every `while` loop must contain something that eventually makes its condition `FALSE` — omitting this creates an infinite loop.
- `while` loops are the right tool specifically when a stopping point depends on a condition evaluated during the loop, unlike `for` loops (Page 1), which need a known number of iterations up front.

Knowledge Check

124. *while* Loops

1. **Multiple choice.** When is a **while** loop the more appropriate choice over a **for** loop?
 - a) When you know exactly how many iterations you need in advance
 - b) When the number of iterations depends on a condition that's only known while the loop runs
 - c) Never — **for** loops can always replace **while** loops
 - d) Only when looping over character vectors
2. **True or false.** Every **while** loop is guaranteed to eventually stop on its own, without any special care needed.
3. **Predict the output.** If `count <- 0` and the loop is `while (count < 3) { print(count); count <- count + 1 }`, how many times does it print?
4. **Find the error.**

```
count <- 0
while (count < 5) {
  print(count)
}
```

Show answers

1. **b)** When the number of iterations depends on a condition that's only known while the loop runs, not a fixed count decided in advance.
2. **False.** A **while** loop only stops if something inside it actually changes the condition toward **FALSE** — without that, it runs forever (an infinite loop).
3. 3 times, printing 0, 1, 2.
4. `count` is never updated inside the loop, so `count < 5` remains **TRUE** forever, creating an infinite loop that would need to be manually interrupted. It should include `count <- count + 1` inside the loop body.

125. Looping Over a Data Frame's Columns

The Problem

Level 9's `sapply(participants, pct_missing)` computed missing-value percentages for every column in one call — convenient, but it hid exactly *how* R visits each column in turn. Your professor asks you to write out that same process explicitly with a `for` loop, printing a formatted summary line for each column — partly to build your understanding of what `sapply()` is actually doing underneath.

Your Mission

Loop over every column of a data frame explicitly, using `names()` and double-bracket indexing.

Think Before Coding

- From Level 2, what does `names(participants)` return?
- If `col_name` holds a column name as text (like "age"), how would you access that actual column's values — `participants$col_name`, or something else?

125.1. The R Concept

Looping over a data frame's columns means looping over its **column names**, then using **double-bracket indexing** (`df[[col_name]]`) to ac-

125. Looping Over a Data Frame's Columns

cess each column's actual values — `df$col_name` doesn't work here, because `$` requires the literal column name typed directly, not a variable holding that name as text:

```
for (col_name in names(df)) {  
  column_values <- df[[col_name]]  
  # code using column_values  
}
```

125.2. Demonstration

```
participants <- read.csv("data/participants.csv")  
  
for (col_name in names(participants)) {  
  print(col_name)  
}
```

```
[1] "id"  
[1] "age"  
[1] "sex"  
[1] "county"  
[1] "systolic_bp"  
[1] "diastolic_bp"  
[1] "bmi"  
[1] "smoker"
```

The loop visits all 8 column names in order — but printing just the *names* isn't very useful yet; the next step is using each name to access that column's actual values.

125.3. Explore

Try `participants$col_name` inside the loop (instead of `participants[[col_name]]`) and see what happens. Since `col_name` is a variable holding text like "age", `$` looks for a column *literally named* `col_name`, which doesn't exist — `df[[col_name]]` is what correctly uses the *value* of `col_name` to select a column.

125.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

for (col_name in names(participants)) {
  column_values <- participants[[col_name]]
  pct_miss <- round(sum(is.na(column_values)) / length(column_values) * 100, 1)
  print(paste0(col_name, ": ", pct_miss, "% missing"))
}
```

```
[1] "id: 0% missing"
[1] "age: 6% missing"
[1] "sex: 0% missing"
[1] "county: 0% missing"
[1] "systolic_bp: 4% missing"
[1] "diastolic_bp: 0% missing"
[1] "bmi: 5% missing"
[1] "smoker: 0% missing"
```

Exactly the same missing-value percentages Level 9's `sapply(participants, pct_missing)` produced — now built with an explicit `for` loop over column names, using `[[ ]]` to access each column's actual values by name.

125.5. Interpret the Result

This confirms what `sapply()` was doing internally all along: visiting each column by name, applying a calculation, and (in `sapply()`'s case) automatically collecting the results into a vector. Writing the loop out explicitly makes every step visible — useful for understanding, debugging, or handling situations more complex than a single function call per column can express cleanly.

Common Mistakes

- **Using `$` with a variable holding a column name.** `participants$col_name` looks for a literal column called `col_name`, not the column whose name is *stored in* the `col_name` variable — this is one of the most common loop-over-columns mistakes, and `[[ ]]` is the fix.
- **Forgetting `names()` returns column names as text, not the columns themselves.** `for (col_name in names(participants))` gives you `col_name` as a string like `"age"` — you still need `participants[[col_name]]` to get the actual values.
- **Writing a loop when `sapply()/map_dbl()` (Level 9) would be simpler for a single, uniform calculation.** This page's loop is intentionally the “long way” to build understanding — for straightforward one-function-per-column tasks, Level 9's shortcuts remain the more practical everyday choice.

125.6. Practice Problem

Write a `for` loop over `names(participants)` that prints each column's `class()` (Level 2), similar to how Page 4's `sapply(participants, class)` worked in Level 9.

125.7. Challenge Problem

Modify the missing-value loop above to only print a line for columns that actually have at least one missing value (hint: wrap the `print()` line in an `if (pct_miss > 0) {...}` check, combining this level's loops with Level 4's conditionals). Confirm only `age`, `systolic_bp`, and `bmi` produce output.

125.8. Key Takeaways

- Looping over a data frame's columns means looping over `names(df)`, then using `df[[col_name]]` (not `df$col_name`) to access each column's values by a name stored in a variable.
- `$` requires the literal column name typed directly; `[[ ]]` works with a variable holding that name as text — an essential distinction for this kind of loop.
- Writing a loop explicitly (rather than using `sapply()/map()`) trades conciseness for step-by-step visibility, useful for understanding or for tasks too complex for a single function call per column.

Knowledge Check

1. **Multiple choice.** Inside `for (col_name in names(df))`, how do you correctly access the column named by `col_name`?
 - a) `df$col_name`
 - b) `df[[col_name]]`
 - c) `df(col_name)`
 - d) `col_name(df)`
2. **True or false.** `df$col_name` and `df[[col_name]]` always produce the same result, regardless of whether `col_name` is a variable or a literal name.

125. Looping Over a Data Frame's Columns

3. **Predict the output.** What would `names(participants)` return as its very first element?
4. **Find the error.**

```
for (col_name in names(participants)) {  
  print(participants$col_name)  
}
```

Show answers

1. **b)** `df[[col_name]]`.
2. **False.** `df$col_name` looks for a column literally named `col_name`; `df[[col_name]]` uses the *value* stored in the `col_name` variable to select the column — these behave very differently when `col_name` is a variable.
3. `"id"` — the first column in `participants`.
4. `participants$col_name` looks for a column literally named `"col_name"`, which doesn't exist, rather than using the value stored in the loop variable `col_name`. It should be `participants[[col_name]]`.

126. Nested Loops

The Problem

Level 5's `count(participants, county, smoker)` produced a cross-tabulation of every county-by-smoking-status combination in one call. Your professor wants to see how that same result could be built manually, one combination at a time — county by county, and *within* each county, smoking status by smoking status — to understand exactly how many comparisons are actually happening underneath a single `count()` call.

Your Mission

Write a loop inside another loop (a “nested loop”) to iterate over every combination of two variables.

Think Before Coding

- With 5 counties and 2 smoking statuses, how many total county-and-smoking-status combinations exist?
- If an outer loop runs 5 times, and an inner loop runs 2 times *for each* outer iteration, how many times does the inner loop's code run in total?

126.1. The R Concept

A **nested loop** places one `for` loop entirely inside another — the inner loop runs completely, start to finish, for *every single* iteration of the outer loop:

126. Nested Loops

```
for (outer_item in outer_vector) {  
  for (inner_item in inner_vector) {  
    # runs once for every combination of outer_item and inner_item  
  }  
}
```

If the outer loop has n items and the inner loop has m items, the innermost code runs $n * m$ times in total.

126.2. Demonstration

```
counties <- c("Cedar Falls", "Millbrook")  
smoking_statuses <- c("No", "Yes")  
  
for (county in counties) {  
  for (status in smoking_statuses) {  
    print(paste(county, "-", status))  
  }  
}
```

```
[1] "Cedar Falls - No"  
[1] "Cedar Falls - Yes"  
[1] "Millbrook - No"  
[1] "Millbrook - Yes"
```

With 2 counties and 2 smoking statuses, the inner loop's code ran $2 \times 2 = 4$ times total — notice the inner loop (`status`) completes both of its values *before* the outer loop (`county`) advances to its next value.

126.3. Explore

Try adding a third county to the `counties` vector and re-running. With 3 counties and 2 smoking statuses, confirm the total number of printed lines becomes $3 \times 2 = 6$, following the same pattern.

126.4. Solve the Problem

```

participants <- read.csv("data/participants.csv")

counties <- unique(participants$county)
smoking_statuses <- unique(participants$smoker)

for (county in counties) {
  for (status in smoking_statuses) {
    n_matching <- sum(participants$county == county & participants$smoker == status)
    print(paste0(county, " - ", status, ": ", n_matching))
  }
}

[1] "Oakford - No: 11"
[1] "Oakford - Yes: 6"
[1] "Riverdale - No: 12"
[1] "Riverdale - Yes: 4"
[1] "Millbrook - No: 15"
[1] "Millbrook - Yes: 6"
[1] "Springvale - No: 20"
[1] "Springvale - Yes: 6"
[1] "Cedar Falls - No: 17"
[1] "Cedar Falls - Yes: 3"

```

126. Nested Loops

With 5 counties and 2 smoking statuses, the nested loop ran $5 \times 2 = 10$ times total, correctly reproducing every county-by-smoking-status count exactly as Level 5's `count(participants, county, smoker)` did.

126.5. Interpret the Result

Every one of these 10 combination counts exactly matches Level 5's `count()`-based cross-tabulation — Oakford has 11 non-smokers and 6 smokers, Cedar Falls has 17 non-smokers and just 3 smokers, and so on. This confirms the nested loop correctly reproduces, one manual combination at a time, exactly what `count()` computes in a single tidyverse call — useful for understanding the underlying mechanics, even though `count()` remains the far more practical everyday tool.

Common Mistakes

- **Confusing loop order.** In `for (county in counties) { for (status in ...) {...} }`, the *inner* loop completes entirely before the *outer* loop advances — reversing the nesting order changes which variable “moves slowest,” though the final set of combinations covered is the same either way.
- **Not accounting for how quickly nested loops grow.** 5 counties $\times$ 2 statuses is only 10 combinations, but nesting a third loop inside (say, by age group) could multiply the total iterations substantially — nested loops can become slow or unwieldy as the number of nesting levels grows.
- **Reinventing `count()` or `table()` unnecessarily in everyday work.** Like Page 4's column loop, this nested loop is valuable for building understanding of what a cross-tabulation actually computes — but Level 5's `count(participants, county, smoker)` remains the simpler, more maintainable choice for routine analysis.

126.6. Practice Problem

Write a nested loop over `unique(participants$county)` and `unique(participants$sex)`, printing the count for every county-by-sex combination.

126.7. Challenge Problem

Extend the nested loop to accumulate results into a data frame (one row per combination, with columns `county`, `smoker`, and `n`) instead of just printing them, using `rbind()` to append a new row during each innermost iteration. Compare your accumulated data frame to Level 5's `count(participants, county, smoker)` output directly.

126.8. Key Takeaways

- A nested loop places one `for` loop inside another, running the inner loop's code once for every combination of outer and inner values — $n * m$ total iterations for outer length `n` and inner length `m`.
- The inner loop completes entirely before the outer loop advances to its next value.
- Nested loops are valuable for understanding what a cross-tabulation computes, but tools like `count()` (Level 5) remain the more practical everyday choice for this kind of task.

Knowledge Check

1. **Multiple choice.** If an outer loop has 5 items and an inner loop has 2 items, how many total times does the innermost code run?
 - a) 5

126. *Nested Loops*

- b) 2
 - c) $7 (5 + 2)$
 - d) $10 (5 \times 2)$
2. **True or false.** In a nested loop, the outer loop advances to its next value before the inner loop finishes all of its own iterations.
 3. **Predict the output.** With 3 counties and 2 smoking statuses in a nested loop, how many total combinations would be printed?
 4. **Short answer.** Why might a nested loop become impractical if you added a third level of nesting (say, by age group with 5 categories) to the county-by-smoking-status loop?

Show answers

1. **d)** $10 (5 \times 2)$ — once for every combination of outer and inner values.
2. **False.** The inner loop always completes all of its iterations before the outer loop advances to its next value.
3. 6 combinations (3×2).
4. Adding a third nesting level multiplies the total iterations again ($5 \text{ counties} \times 2 \text{ smoking statuses} \times 5 \text{ age groups} = 50 \text{ combinations}$) — nested loops can grow unwieldy and hard to read as more levels are added, which is exactly why tools like `count()` with multiple grouping columns (Level 5) are usually preferred for this kind of task in practice.

127. `vapply()` and `tapply()`

The Problem

Level 9's `sapply()` is convenient but has a quirk (also raised by `purrr::map()`, Level 9): its return type isn't guaranteed in advance. Separately, you want each county's mean blood pressure — Level 5 solved this with `group_by()` + `summarise()`, but your professor asks whether base R (predating the tidyverse) has an equally direct way to compute a grouped statistic without loading `dplyr` at all.

Your Mission

Use `vapply()` for a type-safe alternative to `sapply()`, and `tapply()` for a grouped summary statistic without `group_by()`.

Think Before Coding

- Level 9's `map_dbl()` guarantees a numeric return type, erroring if a function doesn't comply. Would you expect a base R equivalent to `sapply()` with the same kind of guarantee?
- From Level 5, `group_by(participants, county)` followed by `summarise(mean_bp = mean(systolic_bp, na.rm = TRUE))` computed one mean per county. What three pieces of information would a single base R function need to do the same thing directly?

127. *vapply()* and *tapply()*

127.1. The R Concept

vapply() works like **sapply()**, but requires you to specify the expected return type upfront (**FUN.VALUE**) — exactly the base R equivalent of Level 9's **map_dbl()**/**map_chr()** guarantee:

```
vapply(df, function_name, FUN.VALUE = numeric(1))
```

If **function_name** ever returns something other than a single number for any column, **vapply()** errors immediately, rather than silently producing unexpected output.

tapply() computes a summary statistic for a numeric vector, split by a grouping vector — in one call, without **group_by()**:

```
tapply(numeric_vector, grouping_vector, function_name, na.rm = TRUE)
```

127.2. Demonstration

```
pct_missing <- function(x) {  
  sum(is.na(x)) / length(x) * 100  
}
```

```
participants <- read.csv("data/participants.csv")
```

```
vapply(participants, pct_missing, FUN.VALUE = numeric(1))
```

id	age	sex	county	systolic_bp	diastolic_bp
0	6	0	0	4	0
bmi	smoker				
5	0				

127.3. Explore

The same results as Level 9's `sapply()` and `map_dbl()` — but `vapply()` additionally guarantees, and checks, that `pct_missing()` returns exactly one number per column, erroring immediately if that promise were ever broken.

```
tapply(participants$systolic_bp, participants$county, mean, na.rm = TRUE)
```

Cedar Falls	Millbrook	Oakford	Riverdale	Springvale
128.1579	133.3000	129.7059	124.3571	131.4615

One line, no `group_by()` or `summarise()` needed — `tapply()` splits `systolic_bp` by `county` and applies `mean(..., na.rm = TRUE)` to each group directly, matching Level 5's grouped means exactly.

127.3. Explore

Try `vapply(participants, class, FUN.VALUE = character(1))` — note `FUN.VALUE = character(1)` here, since `class()` returns text, not a number. Using the wrong `FUN.VALUE` type (say, `numeric(1)` for a function that returns text) produces an immediate, clear error.

127.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")
```

```
county_bp_means <- tapply(participants$systolic_bp, participants$county, mean, na.rm = TRUE)
```

```
round(county_bp_means, 2)
```

Cedar Falls	Millbrook	Oakford	Riverdale	Springvale
128.16	133.30	129.71	124.36	131.46

127. `vapply()` and `tapply()`

```
county_age_means <- tapply(participants$age, participants$county, mean, na.rm = TRUE)
round(county_age_means, 2)
```

Cedar Falls	Millbrook	Oakford	Riverdale	Springvale
50.58	48.00	42.20	51.27	45.40

Both grouped means computed with a single `tapply()` call each — matching Level 5’s `group_by()` + `summarise()` results exactly (blood pressure) and Level 5’s earlier county age findings (mean age), all without loading `dplyr` at all.

127.5. Interpret the Result

`tapply()` proves the tidyverse isn’t strictly necessary for grouped summaries — base R could always do this, just with a less pipe-friendly single-function-call style. `vapply()` similarly proves base R had a type-safety mechanism (`FUN.VALUE`) analogous to `purrr`’s type-specific `map_*()` variants long before `purrr` existed. Neither replaces the tidyverse’s readability advantages for longer, multi-step pipelines — but both are worth recognizing when reading older R code that predates the tidyverse’s popularity.

Common Mistakes

- **Forgetting `FUN.VALUE` must match what the function actually returns.** `vapply(df, pct_missing, FUN.VALUE = character(1))` would error immediately, since `pct_missing()` returns numbers, not text — the mismatch is exactly the point of `vapply()`’s stricter checking, but it does mean you must know your function’s output type correctly.

- **Confusing `tapply()`'s argument order.** `tapply(x, INDEX, FUN)` takes the *values* first, then the *grouping* vector, then the *function* — reversing the first two arguments (grouping by the wrong variable) is an easy mistake when moving quickly.
- **Expecting `tapply()`'s output to be a data frame.** `tapply()` returns a named array (here, effectively a named vector) — not a data frame like `summarise()` produces, so operations expecting data-frame structure (like further `filter()`-ing) won't work directly on its result without additional conversion.

127.6. Practice Problem

Use `tapply()` to compute the mean `bmi` for each smoking status (`participants$smoker`), and compare the result to Page 5's boxplot-based finding from Level 8 (non-smokers median 27.8, smokers median 26.25 — note `tapply()` here computes the *mean*, not median, so some difference is expected).

127.7. Challenge Problem

Use `vapply()` with `FUN.VALUE = numeric(1)` to apply a function that computes each numeric column's range width (`max - min`, Level 9's practice problem) across `participants`. You'll need to first restrict to only the numeric columns (Level 5's `select()` with `where(is.numeric)`, or manual column selection), since `vapply()`'s strict type-checking will error on character columns where range width isn't meaningful.

127. `vapply()` and `tapply()`

127.8. Key Takeaways

- `vapply(df, function_name, FUN.VALUE = ...)` works like `sapply()` but requires specifying the expected return type, erroring immediately if that promise is broken — the base R equivalent of `purrr`'s `map_dbl()`/`map_chr()`.
- `tapply(values, grouping, function_name)` computes a summary statistic split by group in one call, without needing `group_by()`.
- Both functions predate the tidyverse and prove grouped, type-safe operations were always possible in base R — the tidyverse mainly improves readability and pipe-friendliness, not raw capability.

Knowledge Check

1. **Multiple choice.** What does `FUN.VALUE = numeric(1)` tell `vapply()`?
 - a) To convert every column to numeric first
 - b) To expect the applied function to return exactly one number per element
 - c) To skip any column that isn't numeric
 - d) Nothing — it's an optional, unused argument
2. **True or false.** `tapply()` requires the tidyverse's `dplyr` package to work.
3. **Predict the output.** What would `tapply(participants$age, participants$smoker, mean, na.rm = TRUE)` compute?
4. **Find the error.**

```
vapply(participants, class, FUN.VALUE = numeric(1))
```

Show answers

127.8. Key Takeaways

1. **b)** To expect the applied function to return exactly one number per element, erroring if it doesn't.
2. **False.** `tapply()` is base R, requiring no additional packages at all.
3. The mean age for each smoking status ("No" and "Yes"), split by group in one call.
4. `class()` returns character (text) values, but `FUN.VALUE = numeric(1)` tells `vapply()` to expect numbers — this mismatch causes an immediate error. It should be `FUN.VALUE = character(1)` instead.

128. When to Loop, and When to Vectorize

The Problem

You now know several ways to repeat a calculation: `for` loops, `while` loops, `sapply()/map()` (Level 9), and vectorized arithmetic (Level 2, `x + y` applied to entire vectors at once). A new team member asks you: “Should I write a `for` loop to compute `pulse_pressure` for every participant, or is there a better way?” You realize you’ve never explicitly compared these options against each other.

Your Mission

Understand when a `for` loop is genuinely necessary, and when R’s vectorized operations make a loop unnecessary and slower.

Think Before Coding

- From Level 2 and Level 4, how did you compute `pulse_pressure <- participants$systolic_bp - participants$diastolic_bp`? Did that line need a loop at all?
- If each row’s calculation doesn’t depend on any *other* row’s result, does the calculation genuinely need to happen one row at a time, in order?

128.1. The R Concept

R's arithmetic and comparison operators (+, -, *, /, >=, ==, and more) are **vectorized** — they automatically apply to every element of a vector at once, internally implemented in fast, compiled code rather than R's slower loop interpreter:

```
participants$pulse_pressure <- participants$systolic_bp - participants$diastolic_bp
```

Writing an explicit for loop to do the exact same thing works, but is both slower and more verbose:

```
pulse_pressure <- vector("numeric", nrow(participants))
for (i in seq_len(nrow(participants))) {
  pulse_pressure[i] <- participants$systolic_bp[i] - participants$diastolic_bp[i]
}
```

The general rule: **if a calculation for one row never depends on another row's result, it can be vectorized** — and should be, since R's vectorized operations are both faster and more readable than an equivalent loop.

128.2. Demonstration

```
participants <- read.csv("data/participants.csv")

# Vectorized - one line, no loop
pulse_pressure_vectorized <- participants$systolic_bp - participants$diastolic_bp

# Explicit loop - same result, more code
pulse_pressure_loop <- vector("numeric", nrow(participants))
for (i in seq_len(nrow(participants))) {
  pulse_pressure_loop[i] <- participants$systolic_bp[i] - participants$diastolic_bp[i]
}
```

```

}

identical(pulse_pressure_vectorized, pulse_pressure_loop)

[1] TRUE

```

Both approaches produce the exact same result — `identical()` confirms it — but the vectorized version is one readable line, while the loop needed pre-allocation (Page 2) and manual indexing to achieve the same outcome.

128.3. Explore

Think back to Page 3’s `while` loop question: “how many oldest participants must be removed before the mean age drops below 40?” Could that particular question be answered with a single vectorized line, with no loop at all? Consider why the answer depends on a *changing* dataset at each step (each removal affects what the “current mean” even is) — a genuine case where a loop is actually necessary.

128.4. Solve the Problem

```

participants <- read.csv("data/participants.csv")

# This CAN be vectorized - each row's flag doesn't depend on any other row
at_risk_vectorized <- participants$age >= 65 & participants$systolic_bp >= 130
sum(at_risk_vectorized, na.rm = TRUE)

[1] 4

```

128. When to Loop, and When to Vectorize

```
# This CANNOT be simply vectorized - each step depends on the previous step's
ages <- sort(participants$age[!is.na(participants$age)], decreasing = TRUE)
removed_count <- 0
while (mean(ages) >= 40) {
  ages <- ages[-1]
  removed_count <- removed_count + 1
}
removed_count
```

```
[1] 29
```

The at-risk flag (Level 5’s flagship mission) is naturally vectorized, since each participant’s flag depends only on their own age and blood pressure. The removal question (Page 3) genuinely requires a loop, since each iteration’s “current mean” depends entirely on what was removed in every *previous* iteration — there’s no way to compute “removals needed” for all rows independently and simultaneously.

128.5. Interpret the Result

The distinction is exactly this: vectorize whenever each result is independent of every other result (the vast majority of data cleaning and analysis tasks across this entire course); reach for an explicit loop only when a calculation is genuinely **sequential** — each step depending on the state left behind by the previous one, as in Page 3’s iterative removal question.

Common Mistakes

- **Writing a for loop out of habit for tasks that are naturally vectorized.** Nearly every calculation from Levels 1-9 (pulse_pressure, bmi_category, is_at_risk) was vectorized without ever needing an explicit loop — defaulting to a loop for these tasks adds unnecessary code and reduced performance.

- **Assuming loops are always “wrong” and should never be used.** Page 3’s mean-age removal question is a genuine counterexample — some calculations are inherently sequential and cannot be vectorized away, no matter how much R idiom is applied.
- **Not recognizing the signal that a loop is genuinely needed.** The clearest sign: does computing row *i*’s result require knowing the result from row *i*-1 (or an evolving running total/condition)? If yes, a loop (or a specialized sequential function) is likely necessary; if no, vectorize.

128.6. Practice Problem

Is computing `bmi_category` for every participant (Level 4’s `case_when()` logic) naturally vectorizable, or does it require a loop? Justify your answer using the “does row *i* depend on any other row” test from this page.

128.7. Challenge Problem

Consider a hypothetical running total: “the cumulative number of participants enrolled so far, in the order they appear in the dataset.” Is this naturally vectorized, or does it require a loop (or a specialized function like `cumsum()`)? Research `cumsum()` and explain how it relates to this level’s loop-versus-vectorize distinction.

128.8. Key Takeaways

- R’s vectorized operators (`+`, `-`, `>=`, `==`, and more) automatically apply to every element of a vector at once, and are both faster and more readable than an equivalent explicit loop.

128. *When to Loop, and When to Vectorize*

- The test for whether a task can be vectorized: does each row's result depend only on that row's own values, or does it depend on another row's result (or an evolving state across iterations)?
- Loops remain genuinely necessary for sequential calculations, like Page 3's iterative removal question — but the majority of everyday data tasks across this course have been, and should be, vectorized.

Knowledge Check

1. **Multiple choice.** Which of these is the best test for whether a calculation can be vectorized?
 - a) Whether the data frame has more than 10 rows
 - b) Whether each row's result depends only on that row's own values
 - c) Whether the column is numeric or character
 - d) Whether a loop would be shorter to write
2. **True or false.** A `for` loop and a vectorized operation can produce identical results, differing mainly in speed and code length.
3. **Predict the output.** Is computing `is_at_risk(participants$age, participants$systolic_bp)` (Level 9) naturally vectorizable?
4. **Short answer.** Why does Page 3's "remove oldest participants until mean age drops below 40" question require a loop rather than a vectorized one-line solution?

Show answers

1. **b)** Whether each row's result depends only on that row's own values (versus depending on another row's result or an evolving state).
2. **True.** As demonstrated with `pulse_pressure`, both approaches can produce `identical()` results — the choice affects performance and readability, not correctness.

128.8. Key Takeaways

3. Yes — each participant's at-risk flag depends only on their own `age` and `systolic_bp`, with no dependency on any other participant's values.
4. Each iteration's "current mean" depends entirely on which participants were removed in every previous iteration — there's no way to know in advance, independently for each row, whether it will end up removed, since that depends on the evolving state of the shrinking dataset as the loop proceeds.

129. Level 10 Capstone: Automating Five Clinic Files

The Problem

This level's opening mission, finally in front of you: `data/clinics/` contains five separate files — one per county's clinic — each with the exact same column structure as `participants.csv`. Reading, checking, and combining them by hand would mean writing (and re-running) the same three lines five separate times. Your professor wants one script that handles all five files in a single run, however many files there happen to be.

Your Mission

Use `list.files()` to discover every clinic file automatically, then loop through them, reading and combining all five into one dataset with a single script.

Think Before Coding

- From Level 3, what did `read.csv()` need as its argument — a fixed, hardcoded filename, or could that filename come from a variable instead?
- If you had a vector containing all five file paths, would a `for` loop (Page 1) be a natural way to read each one in turn?

129.1. The R Concept

`list.files()` returns a character vector of filenames found in a directory, optionally filtered by pattern — exactly what’s needed to discover “however many files happen to exist” without hardcoding a count:

```
file_list <- list.files("data/clinics", pattern = "\\*.csv$", full.names = TRUE)
```

Combined with a `for` loop and result accumulation (Page 2), this reads and combines every discovered file automatically:

```
all_data <- list() # a list to accumulate each file's data frame

for (i in seq_along(file_list)) {
  all_data[[i]] <- read.csv(file_list[i])
}

combined <- do.call(rbind, all_data) # stack all data frames into one
```

129.2. Demonstration

```
clinic_files <- list.files("data/clinics", pattern = "\\*.csv$", full.names = TRUE)

clinic_files

[1] "data/clinics/clinic_cedarfalls.csv" "data/clinics/clinic_millbrook.csv"
[3] "data/clinics/clinic_oakford.csv"    "data/clinics/clinic_riverdale.csv"
[5] "data/clinics/clinic_springvale.csv"
```

`list.files()` found all five clinic files automatically — if a sixth clinic file were added tomorrow, this same line would discover it too, with no code changes needed.

129.3. Explore

Try `length(clinic_files)`. Confirm it returns 5, matching the number of counties in the original study — and consider what would happen to this same code if a clinic closed (4 files) or a new one opened (6 files): nothing in the loop itself would need to change.

129.4. Solve the Problem

```
clinic_files <- list.files("data/clinics", pattern = "\\*.csv$", full.names = TRUE)

all_clinics <- list()

for (i in seq_along(clinic_files)) {
  all_clinics[[i]] <- read.csv(clinic_files[i])
}

combined_participants <- do.call(rbind, all_clinics)

nrow(combined_participants)

[1] 100

table(combined_participants$county)

Cedar Falls   Millbrook   Oakford   Riverdale   Springvale
          20           21           17           16           26

round(mean(combined_participants$age, na.rm = TRUE), 2)

[1] 47.43
```

129. Level 10 Capstone: Automating Five Clinic Files

100 total rows, county counts of 20/21/17/16/26, and a mean age of 47.43 — every one of these exactly matches the original `participants.csv` used since Level 1, confirming the five separate clinic files, read and combined automatically, reconstruct the identical dataset.

129.5. Interpret the Result

The automation succeeded completely: a single script, running once, correctly discovered, read, and combined all five clinic files into a dataset statistically identical to the original — same row count (100), same county breakdown, same mean age (47.43). If your team's five clinics send updated files next month, this exact same code would run again with zero changes, reading whatever files exist in `data/clinics/` at that time.

Common Mistakes

- **Hardcoding file names instead of using `list.files()`.** Writing five separate `read.csv("data/clinics/clinic_cedarfalls.csv")`-style lines defeats the purpose of automation — if a clinic's filename or count changes, hardcoded code needs manual updates every time; `list.files()` adapts automatically.
- **Forgetting `full.names = TRUE`.** Without it, `list.files()` returns just the filenames (like `"clinic_cedarfalls.csv"`) without the folder path — `read.csv()` would then fail to find the file unless your working directory (Level 1) happened to already be `data/clinics/`.
- **Using `rbind()` directly on a list instead of `do.call(rbind, list)`.** `rbind(all_clinics)` (without `do.call()`) doesn't correctly stack a list of data frames — `do.call(rbind, all_clinics)` is what correctly applies `rbind()` across every element of the list at once.

129.6. Practice Problem

Modify the loop to also print each file's row count as it's read (e.g., `"clinic_oakford.csv: 17 rows"`), confirming each file's contribution before they're all combined.

129.7. Challenge Problem

Add a validation step after combining: use `sum(is.na(combined_participants$age))` to confirm the combined dataset has exactly 6 missing ages, matching the original `participants.csv` exactly. Then write a general-purpose function (Level 9) called `read_and_combine` that takes a folder path and file pattern as arguments, returning the combined data frame — so this entire capstone becomes reusable for any future folder of identically structured files, not just `data/clinics/`.

129.8. Key Takeaways

- `list.files(folder, pattern = ..., full.names = TRUE)` discovers files automatically, adapting to however many exist without hardcoding names or counts.
- Combining a loop (Page 1), result accumulation into a list (Page 2), and `do.call(rbind, list)` automates reading and stacking any number of identically structured files into one dataset.
- True automation means the same code keeps working correctly even as the underlying number or names of files change — exactly the property that made this capstone's approach worth building, instead of hardcoding five separate `read.csv()` calls.

Knowledge Check

129. Level 10 Capstone: Automating Five Clinic Files

1. **Multiple choice.** What does `list.files("data/clinics", pattern = "\\*.csv$", full.names = TRUE)` return?
 - a) The contents of every CSV file in the folder
 - b) A character vector of full file paths for every CSV file in the folder
 - c) The number of CSV files in the folder
 - d) A single combined data frame
2. **True or false.** If a sixth clinic file were added to `data/clinics/`, the capstone's loop-based code would need to be rewritten to include it.
3. **Predict the output.** After combining all five clinic files, what would `nrow(combined_participants)` equal?
4. **Find the error.**

```
all_clinics <- list()
for (i in seq_along(clinic_files)) {
  all_clinics[[i]] <- read.csv(clinic_files[i])
}
combined_participants <- rbind(all_clinics)
```

Show answers

1. **b)** A character vector of full file paths for every CSV file in the folder.
2. **False.** `list.files()` automatically discovers however many matching files exist — a sixth file would be picked up with no code changes needed.
3. 100 — the same total row count as the original `participants.csv`.
4. `rbind(all_clinics)` doesn't correctly stack a list of data frames — it should be `do.call(rbind, all_clinics)`, which applies `rbind()` across every element of the list at once.

Part XI.

Level 11: Statistical Analysis in R

Level 11 — Statistical Analysis in R

Where this level is headed

This is where the course turns into the analysis you'll actually publish: confidence intervals, t-tests, chi-square tests, correlation, and regression — linear, logistic, Poisson, and Cox — each introduced through a real research question, with assumptions, code, output, and interpretation every time.

What you'll build toward

Every level so far has described this dataset — means, correlations, plots, patterns. This level asks a harder question of each of those patterns: is it statistically significant, or could it plausibly be due to chance in a sample this size? You'll move from probability foundations through confidence intervals, t-tests, chi-square, correlation testing, simple linear regression, ANOVA, and logistic regression — testing many of the exact patterns Levels 5, 7, and 8 already described numerically and visually, now with a formal answer to “is this real, or noise?” Multivariable models, confounding, and more advanced regression families are covered in Level 12, building directly on the foundation this level establishes.

Topics in this level

- Probability and the normal distribution

Level 11 — Statistical Analysis in R

- The Central Limit Theorem, demonstrated by simulation
- Confidence intervals
- One-sample, two-sample, and paired t-tests
- The chi-square test of independence
- Testing a correlation for statistical significance
- Simple linear regression with `lm()`
- One-way ANOVA
- Logistic regression for a binary outcome
- A capstone: formally testing several of this course's earlier descriptive findings

A preview mission

Your mission across this level: revisit Level 7's finding that smoking rates differ by sex (19.2% vs. 31.2%) and Level 8's finding that blood pressure trends gently downward with age — and formally test both. Are they statistically significant, or could patterns this size plausibly arise by chance in a sample of 100 participants?

130. Probability and the Normal Distribution

The Problem

Level 8's histogram of `bmi` looked roughly bell-shaped — clustered around the mean, tapering off symmetrically on both sides. Your professor asks: if BMI genuinely follows something close to a **normal distribution**, could you predict what proportion of participants have BMI over 30, using only the mean and standard deviation — without directly counting rows in the data at all?

Your Mission

Use R's normal distribution functions (`pnorm()`, `qnorm()`, `dnorm()`) to compute probabilities from a theoretical distribution, and compare them against what the actual data shows.

Think Before Coding

- A normal distribution is fully described by just two numbers. Which two summary statistics from Level 7 do you think those are?
- If BMI followed a normal distribution exactly, would you expect the proportion of participants above any given cutoff to be *exactly* predictable from the mean and SD alone?

130.1. The R Concept

The **normal distribution** (“bell curve”) is defined entirely by its mean and standard deviation. R’s `pnorm()` computes the probability of a value falling below a cutoff, under a normal distribution with a given mean and SD:

```
pnorm(cutoff, mean = mu, sd = sigma)          # P(X <= cutoff)
1 - pnorm(cutoff, mean = mu, sd = sigma)      # P(X > cutoff)
```

`qnorm()` works in reverse — given a probability, it returns the corresponding cutoff value (“quantile function,” extending Level 7’s `quantile()` to a theoretical distribution rather than actual data):

```
qnorm(0.95, mean = mu, sd = sigma)  # the value below which 95% of a normal
```

130.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
bmi_mean <- mean(participants$bmi, na.rm = TRUE)
```

```
bmi_sd <- sd(participants$bmi, na.rm = TRUE)
```

```
bmi_mean
```

```
[1] 27.12316
```

```
bmi_sd
```

```
[1] 4.639157
```

```
# theoretical probability of BMI > 30, assuming a normal distribution
```

```
1 - pnorm(30, mean = bmi_mean, sd = bmi_sd)
```



```
[1] 0.2676
```

130.3. Explore

Try comparing this theoretical prediction against the actual data: `mean(participants$bmi > 30, na.rm = TRUE)`. The two numbers won't match exactly — real data never follows a theoretical distribution perfectly — but a close match suggests the normal approximation is reasonable for this variable.

130.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

bmi_mean <- mean(participants$bmi, na.rm = TRUE)
bmi_sd <- sd(participants$bmi, na.rm = TRUE)

theoretical_prop <- 1 - pnorm(30, mean = bmi_mean, sd = bmi_sd)
empirical_prop <- mean(participants$bmi > 30, na.rm = TRUE)

theoretical_prop

[1] 0.2676

empirical_prop

[1] 0.2632
```

The theoretical prediction (26.76%) and the actual empirical proportion (26.32%) are remarkably close — under a one-percentage-point difference — strong evidence that BMI in this sample is well-approximated by a normal distribution with mean 27.12 and SD 4.64.

130.5. Interpret the Result

This close agreement matters beyond curiosity: many of the statistical tests in the rest of this level (t-tests, linear regression) formally *assume* the data (or, more precisely, certain quantities derived from it) follows something close to a normal distribution. Confirming that assumption is reasonable here — rather than blindly trusting it — is good statistical practice, not a formality to skip.

Common Mistakes

- **Forgetting `pnorm()` gives a cumulative probability ($P(X \leq \text{cutoff})$), not $P(X = \text{cutoff})$ or $P(X > \text{cutoff})$ directly.** To get “greater than,” you must subtract from 1: `1 - pnorm(cutoff, ...)`, exactly as done above.
- **Assuming every numeric variable is normally distributed without checking.** Some variables (income, hospital length of stay, and many others) are heavily skewed and poorly approximated by a normal distribution — always compare the theoretical prediction against the actual data, as this page did, rather than assuming normality by default.
- **Mixing up `pnorm()` and `qnorm()`.** `pnorm()` takes a value and returns a probability; `qnorm()` takes a probability and returns a value — they’re inverses of each other, and using the wrong one produces a nonsensical result (e.g., `qnorm()` given a value over 1 errors, since probabilities can’t exceed 1).

130.6. Practice Problem

Use `pnorm()` to compute the theoretical probability that a participant’s systolic blood pressure is below 110, using the sample mean and SD from Level 7. Compare it against the actual empirical proportion in the data.

130.7. Challenge Problem

Use `qnorm(0.90, mean = bmi_mean, sd = bmi_sd)` to find the BMI value below which 90% of a normal distribution with this sample's mean and SD would fall. Then check what percentage of participants in the actual data fall below that same cutoff, using `mean(participants$bmi < cutoff, na.rm = TRUE)`.

130.8. Key Takeaways

- The normal distribution is fully described by its mean and standard deviation; `pnorm()` computes probabilities and `qnorm()` computes cutoff values, inverses of each other.
- Comparing a theoretical (normal-distribution) prediction against the actual empirical proportion in real data is a simple, useful check of whether the normal approximation is reasonable.
- Many statistical tests later in this level assume approximate normality — checking this assumption, rather than blindly trusting it, is good practice.

Knowledge Check

1. **Multiple choice.** What does `pnorm(30, mean = 27.12, sd = 4.64)` compute?
 - a) The probability that X equals exactly 30
 - b) The probability that X is less than or equal to 30
 - c) The value below which 30% of the distribution falls
 - d) The standard deviation of the distribution
2. **True or false.** A normal distribution is fully described by its mean and standard deviation alone.

130. *Probability and the Normal Distribution*

3. **Predict the output.** What would `1 - pnorm(30, mean = 27.12, sd = 4.64)` represent?
4. **Find the error.**

```
pnorm(0.95, mean = bmi_mean, sd = bmi_sd)
# intended: find the BMI value below which 95% of the distribution falls
```

Show answers

1. **b)** The probability that X is less than or equal to 30.
2. **True.** Unlike many other distributions, a normal distribution requires only these two parameters to be fully specified.
3. The probability that a value from this distribution exceeds 30 ($P(X > 30)$).
4. `pnorm()` takes a value and returns a probability — the intended calculation (finding a value from a probability) requires the inverse function, `qnorm()`. It should be `qnorm(0.95, mean = bmi_mean, sd = bmi_sd)`.

131. The Central Limit Theorem

The Problem

Every test in the rest of this level relies on one deep idea: even if a variable itself isn't perfectly normal, the *sample mean* computed from repeated samples of that variable tends to follow a normal distribution anyway, especially as sample size grows. Your professor asks you to actually demonstrate this with a simulation, rather than just accepting it as a rule — is it really true, or just statistical folklore?

Your Mission

Demonstrate the Central Limit Theorem (CLT) by simulation: repeatedly sample from `age`, compute the mean of each sample, and examine the distribution of those sample means.

Think Before Coding

- If you drew 30 random participants (with replacement) from `age` and computed their mean, then repeated this 1,000 times, would you expect every one of those 1,000 means to be identical?
- Would you expect the *spread* of those 1,000 sample means to be wider, narrower, or the same as the spread of the original individual ages?

131.1. The R Concept

The **Central Limit Theorem** states that the distribution of sample means (computed from repeated samples of a given size) approaches a normal distribution as sample size grows — regardless of the original variable’s own distribution shape — with a spread (“standard error”) shrinking predictably as sample size increases:

```
set.seed(42)
sample_means <- replicate(n_draws, mean(sample(population, size = n, replace = TRUE)))
```

The theoretical **standard error** of the mean is the population standard deviation divided by the square root of the sample size:

$$SE = \frac{\sigma}{\sqrt{n}}$$

131.2. Demonstration

```
set.seed(42)

participants <- read.csv("data/participants.csv")
ages <- participants$age[!is.na(participants$age)]

sample_means <- replicate(1000, mean(sample(ages, size = 30, replace = TRUE)))

mean(sample_means)

[1] 47.5    # your exact value will vary slightly - this is randomness-dependent

sd(sample_means)

[1] 2.75    # again, expect a value close to this, not identical to the decimal
```

131.3. Explore

The mean of 1,000 sample means lands almost exactly on the true population mean (47.43) — and the spread of those sample means is far *narrower* than the original data's spread ($SD = 15.03$), exactly as the CLT predicts. Note: because this involves R's own random number generator, your exact decimal values will differ slightly from anyone else's (and from what's printed here) even with the same `set.seed(42)` — what matters, and what will always hold, is that the mean lands close to 47.43 and the spread lands close to the theoretical prediction computed next.

131.3. Explore

Try computing the theoretical standard error directly: `sd(ages) / sqrt(30)`. Compare it to `sd(sample_means)` from the simulation above — the two should be close, confirming the CLT's formula matches what actually happens when you simulate it, even though the simulated value itself will vary slightly from run to run and system to system.

131.4. Solve the Problem

```
set.seed(42)

participants <- read.csv("data/participants.csv")
ages <- participants$age[!is.na(participants$age)]

sample_means <- replicate(1000, mean(sample(ages, size = 30, replace = TRUE)))

empirical_se <- sd(sample_means)
theoretical_se <- sd(ages) / sqrt(30)

empirical_se
```

131. The Central Limit Theorem

```
[1] 2.75    # a representative value - simulation-dependent, will vary slightly
```

```
theoretical_se
```

```
[1] 2.744756
```

`theoretical_se` is an exact, deterministic number — computed directly from the actual data (`sd(ages)`, 15.033646, divided by `sqrt(30)`) with no randomness involved at all. `empirical_se`, by contrast, comes from a random simulation, so it will land *close to* 2.744756 but essentially never exactly equal to it — the whole point of the simulation is to confirm that closeness, not to reproduce the theoretical number exactly.

131.5. Interpret the Result

This simulation demonstrates something genuinely powerful: no matter what shape `age`'s own distribution takes, the distribution of its *sample means* (across repeated samples of size 30) clusters tightly and symmetrically around the true population mean, with a spread landing close to what `sd(ages) / sqrt(n)` predicts. This is the theoretical foundation underneath every t-test and confidence interval in the rest of this level — they all rely on the sample mean behaving this predictably.

Common Mistakes

- **Confusing the standard deviation of the original data with the standard error of the mean.** `sd(ages)` (15.03) describes how spread out individual ages are; `sd(sample_means)` (2.757) describes how spread out *sample means* are — these are fundamentally different quantities, and confusing them is one of the most common statistical mistakes.

- **Forgetting `set.seed()` for reproducible simulations.** Without it, `sample()` and `replicate()` produce different random results every time the code runs — `set.seed(42)` (or any fixed number) ensures the exact same “random” sequence every time, essential for a reproducible demonstration.
- **Assuming the CLT requires the original variable to already be normal.** The power of the CLT is precisely that it applies even when the original variable *isn't* normal — sample means still tend toward a normal distribution as sample size grows, though how large a sample size is “enough” depends on how far from normal the original variable is.

131.6. Practice Problem

Repeat the simulation with a larger sample size (`size = 60` instead of 30) and compare the resulting standard error to the original. Does it shrink or grow as sample size increases, and does the direction of change match the `sd(ages) / sqrt(n)` formula’s prediction?

131.7. Challenge Problem

Repeat the simulation using `systolic_bp` instead of `age`. Given `systolic_bp`’s different mean and SD (Level 7), predict the expected standard error using the formula, then confirm it against your simulation’s result.

131.8. Key Takeaways

- The Central Limit Theorem states that sample means, from repeated sampling, tend toward a normal distribution with a predictable stan-

131. The Central Limit Theorem

standard error ($\text{population SD} / \sqrt{n}$), regardless of the original variable's own distribution shape.

- The standard error of the mean (spread of sample means) is a fundamentally different, and much smaller, quantity than the standard deviation of the original data.
- This theorem is the foundation underlying t-tests and confidence intervals, both covered in the rest of this level.

Knowledge Check

1. **Multiple choice.** According to the CLT, as sample size increases, what happens to the standard error of the sample mean?
 - a) It stays the same
 - b) It increases
 - c) It decreases
 - d) It becomes undefined
2. **True or false.** The Central Limit Theorem requires the original variable to already follow a normal distribution.
3. **Predict the output.** If `sd(ages) = 15.03` and sample size is 100 (instead of 30), what would the theoretical standard error be (approximately)?
4. **Short answer.** Why is `set.seed()` important when demonstrating a concept through simulation?

Show answers

1. **c)** It decreases — larger samples produce sample means clustered more tightly around the true population mean.
2. **False.** The CLT's key strength is that it applies even when the original variable isn't normally distributed, as sample size grows sufficiently large.
3. Approximately 1.503 ($15.03 / \sqrt{100} = 15.03 / 10$).

131.8. Key Takeaways

4. `set.seed()` makes a “random” simulation fully reproducible — anyone re-running the exact same code gets the exact same results, essential for verifying, debugging, or teaching from a simulation’s specific output.

132. Confidence Intervals

The Problem

Level 7 reported the mean systolic blood pressure as 129.84 — a single number, computed from 96 non-missing participants out of a much larger population of people this study could have sampled. Your professor asks: if you'd sampled a different 96 participants, would you expect to get exactly 129.84 again, or somewhere nearby? How far off might the true population mean plausibly be from this one sample's estimate?

Your Mission

Compute a 95% confidence interval for a sample mean, expressing the uncertainty around a single point estimate like 129.84.

Think Before Coding

- From Page 2's Central Limit Theorem, what did the standard error of a sample mean describe?
- If a sample mean has some uncertainty around it, would you expect a *range* of plausible values to be more informative than a single number alone?

132.1. The R Concept

A **confidence interval** gives a range of plausible values for a population parameter (like the true mean), built around a sample estimate using its standard error. R's `t.test()` — primarily a hypothesis test (next page)

132. Confidence Intervals

— conveniently also reports a confidence interval for the mean as part of its output:

```
t.test(x)$conf.int
```

The interval is built from the sample mean, the standard error (Page 2), and a critical value from the **t-distribution** (a close relative of the normal distribution, adjusted for the extra uncertainty of estimating the standard deviation from a limited sample):

$$CI = \bar{x} \pm t_{crit} \times SE$$

132.2. Demonstration

```
participants <- read.csv("data/participants.csv")

sbp <- participants$systolic_bp[!is.na(participants$systolic_bp)]

t.test(sbp)$conf.int

[1] 126.0201 133.6674
attr(,"conf.level")
[1] 0.95
```

The sample mean (129.84) comes with a 95% confidence interval of roughly 126.02 to 133.67 — a range of plausible values for the true population mean, not just the single point estimate alone.

132.3. Explore

Try `t.test(sbp, conf.level = 0.99)$conf.int` — a 99% confidence interval instead of the default 95%. Notice the interval widens: a *higher* confidence level requires a *wider* range to remain equally confident of capturing the true population mean.

132.4. Solve the Problem

```

participants <- read.csv("data/participants.csv")

sbp <- participants$systolic_bp[!is.na(participants$systolic_bp)]

sbp_mean <- mean(sbp)
sbp_se <- sd(sbp) / sqrt(length(sbp))
t_critical <- qt(0.975, df = length(sbp) - 1)

lower_bound <- sbp_mean - t_critical * sbp_se
upper_bound <- sbp_mean + t_critical * sbp_se

c(lower_bound, upper_bound)

[1] 126.0201 133.6674

t.test(sbp)$conf.int

[1] 126.0201 133.6674
attr(,"conf.level")
[1] 0.95

```

132. Confidence Intervals

Building the interval manually ($\text{mean} \pm \text{critical value} \times \text{SE}$) produces exactly the same result as `t.test()$conf.int` — confirming the formula and the built-in function agree, and showing explicitly what `t.test()` computes behind the scenes.

132.5. Interpret the Result

You can now say something more precise than “the mean systolic blood pressure was 129.84”: you’re 95% confident the true population mean lies somewhere between 126.02 and 133.67 mmHg. This is a meaningfully different, more honest statement than reporting the single point estimate alone — it explicitly communicates the uncertainty that comes from having only 96 participants rather than the entire population.

Common Mistakes

- **Interpreting a 95% CI as “95% probability the true mean is in this range.”** The more precise interpretation: if you repeated this sampling process many times, about 95% of the resulting confidence intervals would contain the true population mean — a subtle, but real, distinction from a probability statement about this one specific interval.
- **Assuming a narrower interval always means a better estimate.** A narrower interval usually comes from a *larger* sample size, not necessarily a “better” one — comparing intervals from very different sample sizes without accounting for this can be misleading.
- **Forgetting confidence intervals depend on the same normality assumptions checked on Page 1.** For very small samples with heavily skewed data, `t.test()`’s confidence interval can become less reliable — always sanity-check the underlying assumptions, not just run the function blindly.

132.6. Practice Problem

Compute a 95% confidence interval for `bmi`'s mean using `t.test()`. Does the interval contain the value 27 (roughly the boundary between “Normal” and “Overweight,” Level 4's `case_when()` categories)?

132.7. Challenge Problem

Compute confidence intervals for `age`'s mean at three different confidence levels (90%, 95%, 99%) using `t.test(age, conf.level = ...)`. Describe the pattern in how the interval's width changes as the confidence level increases, and explain why that pattern makes sense.

132.8. Key Takeaways

- A confidence interval expresses a range of plausible values for a population parameter, built from a sample estimate and its standard error.
- `t.test(x)$conf.int` computes this directly; the same interval can be built manually as `mean ± critical_value × SE`.
- A higher confidence level produces a wider interval — there's an inherent trade-off between how confident you are and how precise (narrow) the range can be.

Knowledge Check

1. **Multiple choice.** What does a 95% confidence interval represent?
 - a) There is a 95% probability the sample mean is correct
 - b) A range of plausible values for the population parameter, with a procedure that captures it about 95% of the time across repeated sampling

132. Confidence Intervals

- c) 95% of the individual data points fall within this range
 - d) The interval within which all future samples must fall
2. **True or false.** A higher confidence level (say, 99% instead of 95%) always produces a narrower interval.
 3. **Predict the output.** If the sample mean is 129.84 and the 95% CI is (126.02, 133.67), would a 90% CI be wider or narrower than this range?
 4. **Find the error.**

"There's a 95% chance the true population mean is 129.84."

Show answers

1. **b)** A range of plausible values for the population parameter, with a procedure that captures the true value about 95% of the time across repeated sampling.
2. **False.** A higher confidence level requires a *wider* interval to maintain that higher level of confidence, not a narrower one.
3. Narrower — a lower confidence level (90%) requires a smaller range to achieve that lower degree of confidence.
4. This statement misinterprets both the point estimate and the confidence interval — the sample mean (129.84) is a single computed value, not a probabilistic claim, and the correct interpretation of a 95% CI is about the interval-generating *procedure* capturing the true mean across repeated sampling, not a 95% probability attached to any single specific value.

133. The One-Sample t-Test

The Problem

A clinical guideline states that normal systolic blood pressure is around 120 mmHg. Your professor asks: does this *study sample's* mean (129.84) differ from 120 by more than could plausibly be due to chance alone — or could the true population this sample represents actually have a mean of 120, with 129.84 just being one somewhat-higher-than-typical sample?

Your Mission

Test whether a sample's mean differs significantly from a specific reference value, using a one-sample t-test.

Think Before Coding

- If chance alone could easily produce a sample mean of 129.84 from a true population mean of 120, would you expect that to happen often, or rarely?
- From Page 3, does the 95% confidence interval for mean systolic blood pressure (126.02 to 133.67) contain the reference value 120?

133.1. The R Concept

A **one-sample t-test** compares a sample's mean against a specific hypothesized value (μ), producing a **p-value**: the probability of observing a difference at least this large, purely by chance, if the true population mean actually equaled the hypothesized value:

133. The One-Sample *t*-Test

```
t.test(x, mu = hypothesized_value)
```

A small p-value (conventionally, below 0.05) suggests the observed difference is unlikely to be due to chance alone — evidence *against* the hypothesized value being the true population mean.

133.2. Demonstration

```
participants <- read.csv("data/participants.csv")  
  
sbp <- participants$systolic_bp[!is.na(participants$systolic_bp)]  
  
t.test(sbp, mu = 120)
```

One Sample t-test

```
data: sbp  
t = 5.111, df = 95, p-value = 1.655e-06  
alternative hypothesis: true mean is not equal to 120  
95 percent confidence interval:  
 126.0201 133.6674  
sample estimates:  
mean of x  
 129.8438
```

133.3. Explore

Notice the confidence interval reported here (126.02 to 133.67) is exactly the same interval from Page 3 — `t.test()` always reports both a hypothesis test *and* a confidence interval together, since they're built from

133.4. Solve the Problem

the same underlying calculation. Try `t.test(sbp, mu = 130)` instead — since 130 falls *inside* the confidence interval, you’d expect a much larger p-value (failing to reject that particular hypothesized value).

133.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

sbp <- participants$systolic_bp[!is.na(participants$systolic_bp)]

test_result <- t.test(sbp, mu = 120)

test_result$statistic

      t
5.110964

test_result$p.value

[1] 1.655098e-06
```

The t-statistic (5.11) is large, and the p-value (0.0000017, far below 0.05) is extremely small — strong evidence that this sample’s mean genuinely differs from 120, not merely due to random chance.

133.5. Interpret the Result

The sample’s mean systolic blood pressure (129.84) is statistically significantly different from the reference value of 120 ($p < 0.001$) — consistent with the fact that 120 doesn’t even fall inside the 95% confidence interval

133. The One-Sample *t*-Test

(126.02 to 133.67) computed on Page 3. This is genuine evidence, not just a numeric coincidence: a sample mean this far from 120, in a sample of 96, would be extremely unlikely to occur by chance alone if the true population mean actually were 120.

Common Mistakes

- **Confusing statistical significance with practical (clinical) importance.** A p-value this small confirms the difference from 120 is unlikely to be chance — but whether a roughly 10 mmHg difference is *clinically* meaningful is a separate, non-statistical judgment that depends on domain knowledge, not just the p-value.
- **Treating $p < 0.05$ as a magic threshold rather than a convention.** 0.05 is a widely used convention, not a law of nature — a p-value of 0.049 and 0.051 represent nearly identical evidence, despite falling on opposite sides of this threshold.
- **Forgetting the confidence interval and the p-value are two views of the same information.** If a hypothesized value falls *outside* the confidence interval (as 120 does here), the corresponding t-test will always produce $p < 0.05$ — the two are mathematically linked, not independent pieces of evidence.

133.6. Practice Problem

Test whether mean `bmi` differs significantly from 25 (the conventional upper boundary of “Normal” BMI, Level 4). Report the t-statistic, p-value, and your conclusion.

133.7. Challenge Problem

Test whether mean `age` differs significantly from 45. Then check whether 45 falls inside or outside the 95% confidence interval for `age`’s mean (Page

3's practice problem) — confirm the two approaches (hypothesis test and confidence interval) lead to the same conclusion.

133.8. Key Takeaways

- A one-sample t-test compares a sample mean against a specific hypothesized value, producing a p-value indicating how likely such a difference would be purely by chance.
- A small p-value (conventionally < 0.05) is evidence against the hypothesized value; it does not by itself indicate the size or practical importance of the difference.
- A hypothesized value falling outside a confidence interval and a t-test producing $p < 0.05$ always agree — they're two expressions of the same underlying calculation.

Knowledge Check

1. **Multiple choice.** What does the p-value from `t.test(sbp, mu = 120)` represent?
 - a) The probability the true mean is exactly 120
 - b) The probability of observing a difference this large purely by chance, if the true mean were actually 120
 - c) The sample mean itself
 - d) The percentage of participants with blood pressure above 120
2. **True or false.** A statistically significant result automatically means the difference is large enough to matter clinically.
3. **Predict the output.** If a hypothesized value falls inside a variable's 95% confidence interval, would you expect its corresponding one-sample t-test to produce $p < 0.05$ or $p \geq 0.05$?

133. *The One-Sample t-Test*

4. **Short answer.** Why is treating $p = 0.05$ as a rigid pass/fail cutoff considered poor statistical practice?

Show answers

1. **b)** The probability of observing a difference this large purely by chance, if the true population mean were actually 120.
2. **False.** Statistical significance indicates the difference is unlikely to be due to chance — it says nothing about whether that difference is large enough to matter practically or clinically.
3. $p = 0.05$ — a hypothesized value inside the confidence interval indicates the data is consistent with that value, corresponding to a non-significant test result.
4. Because 0.05 is an arbitrary convention, not a meaningful boundary in the underlying evidence — a p-value of 0.049 and 0.051 reflect nearly identical strength of evidence, so treating one as “significant” and the other as “not significant” overstates the importance of a fairly arbitrary line.

134. The Two-Sample t-Test

The Problem

Level 8's scatter plot colored by smoking status showed smokers and non-smokers scattered across similar blood pressure ranges, with means of 130.54 and 129.61 respectively — a difference of under one point. Your professor asks the natural next question: is that roughly one-point difference statistically meaningful, or is it exactly the kind of small difference you'd expect to see between any two randomly split groups, purely by chance?

Your Mission

Compare the means of two independent groups using a two-sample t-test.

Think Before Coding

- If smoking status had absolutely no relationship with blood pressure, would you expect the two groups' means to be *exactly* identical, or just *close*?
- With only 24 smokers compared to 72 non-smokers, would you expect a real underlying difference to be easier or harder to detect statistically, compared to two equally large groups?

134. The Two-Sample *t*-Test

134.1. The R Concept

A **two-sample t-test** compares the means of two independent groups, testing whether their difference is larger than what chance alone would typically produce:

```
t.test(numeric_variable ~ grouping_variable, data = df)
```

By default, R's `t.test()` uses **Welch's t-test**, which doesn't assume the two groups have equal variance — a safer default than the classic “Student's t-test,” which does assume equal variance.

134.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
t.test(systolic_bp ~ smoker, data = participants)
```

Welch Two Sample t-test

data: systolic_bp by smoker

t = -0.21305, df = 41.106, p-value = 0.8323

alternative hypothesis: true difference in means between group No and group Yes

95 percent confidence interval:

-9.750700 7.889589

sample estimates:

mean in group No mean in group Yes

129.6111

130.5417

134.3. Explore

Notice the 95% confidence interval for the difference in means (-9.75 to 7.89) **contains 0** — meaning “no difference at all” is well within the range of plausible values. Compare this to Page 4’s one-sample test, where the hypothesized value (120) fell *outside* the confidence interval — here, the hypothesized “no difference” value falls comfortably inside.

134.4. Solve the Problem

```

participants <- read.csv("data/participants.csv")

test_result <- t.test(systolic_bp ~ smoker, data = participants)

test_result$p.value

[1] 0.8323397

test_result$estimate

mean in group No mean in group Yes
      129.6111      130.5417

```

A p-value of 0.83 is very large — far above any conventional significance threshold — indicating essentially no evidence that smoking status is associated with a real difference in mean systolic blood pressure in this sample.

134.5. Interpret the Result

The roughly one-point difference in means (130.54 vs. 129.61) is entirely consistent with random sampling variation — not evidence of a real underlying relationship between smoking status and blood pressure in this dataset. This matches Level 8’s visual finding exactly: the two groups’ points were thoroughly intermixed on the scatter plot, with no visible separation, and the formal test now confirms that visual impression numerically.

Common Mistakes

- **Treating a non-significant result as proof there’s no effect at all.** A large p-value (0.83) means this sample provides no strong evidence of a difference — it does not prove smoking has *zero* true effect on blood pressure; a larger, more carefully designed study could still detect a real but small effect this sample was too limited to find.
- **Ignoring unequal group sizes when interpreting results.** With only 24 smokers versus 72 non-smokers, this test has less power to detect a small real difference than if both groups were equally large — a non-significant result here is somewhat less conclusive than the same result from two equally sized, larger groups.
- **Using the classic equal-variance t-test by default instead of Welch’s.** R’s default (`t.test()` with no `var.equal` argument) already uses the safer Welch’s version — explicitly setting `var.equal = TRUE` (the classic Student’s t-test) requires justifying the equal-variance assumption first, which shouldn’t be assumed without checking.

134.6. Practice Problem

Test whether mean `bmi` differs significantly between smokers and non-smokers. Compare your p-value to this page’s blood pressure result — is the evidence for a smoking-BMI relationship stronger or weaker?

134.7. Challenge Problem

Test whether mean `age` differs significantly between males and females (`age ~ sex`). Report the confidence interval for the difference, and state in a full sentence (as you would in a report) what the result means, following the “not proof of zero effect” caution from this page’s Common Mistakes.

134.8. Key Takeaways

- A two-sample t-test compares two independent groups’ means, testing whether their observed difference exceeds what chance alone would plausibly produce.
- R’s default `t.test()` uses Welch’s t-test, which doesn’t assume equal variance between groups — a safer default than the classic equal-variance version.
- A non-significant result (large p-value) means the sample lacks strong evidence of a difference — it does not prove no real difference exists at all.

Knowledge Check

1. **Multiple choice.** What does `t.test(systolic_bp ~ smoker, data = participants)` test?
 - a) Whether `systolic_bp` is normally distributed

134. *The Two-Sample t-Test*

- b) Whether the mean `systolic_bp` differs between smokers and non-smokers
 - c) Whether `smoker` predicts `systolic_bp` using regression
 - d) The correlation between `systolic_bp` and `smoker`
2. **True or false.** A p-value of 0.83 proves that smoking has no effect on blood pressure whatsoever.
 3. **Predict the output.** If a confidence interval for a difference in means contains 0, would you expect the corresponding p-value to be above or below 0.05?
 4. **Short answer.** Why might unequal group sizes (24 smokers vs. 72 non-smokers) make it harder to detect a real but modest difference between groups?

Show answers

1. b) Whether the mean `systolic_bp` differs between smokers and non-smokers.
2. **False.** A large p-value indicates a lack of strong evidence for a difference in this particular sample — it does not prove the true difference is exactly zero.
3. Above 0.05 — a confidence interval containing 0 (no difference) corresponds to a non-significant result.
4. Statistical power to detect a real difference depends heavily on sample size in each group — with only 24 smokers, there's more sampling variability in that group's mean than in the 72 non-smokers, making a modest true difference harder to distinguish reliably from chance.

135. The Paired t-Test

The Problem

Level 6 and Level 8 both found that blood pressure appeared to drop across the three-visit study — mean systolic BP fell from 124.96 at visit 1 to 120.69 by visit 3. But that comparison used *different means* across all participants at each visit. Your professor wants something stronger: for the *same* 26 participants who completed both visit 1 and visit 3, is the average within-person drop statistically significant?

Your Mission

Test whether paired, before-and-after measurements from the same individuals differ significantly, using a paired t-test.

Think Before Coding

- A paired t-test compares two measurements taken on the *same* individuals. How is this fundamentally different from Page 5's two-sample t-test, which compared two separate *groups* of different people?
- If every single participant's blood pressure dropped by exactly 5 points from visit 1 to visit 3, would that be stronger or weaker evidence than if some dropped a lot, some rose slightly, but the *average* drop was still 5?

135.1. The R Concept

A **paired t-test** analyzes the *differences* between two measurements taken on the same individuals, testing whether the mean difference is significantly different from zero:

```
t.test(measurement_time1, measurement_time2, paired = TRUE)
```

The `paired = TRUE` argument is essential — without it, R would (incorrectly) treat the two measurements as independent groups (Page 5’s test), losing the crucial information that each pair of values comes from the *same* participant.

135.2. Demonstration

```
library(tidyr)
library(dplyr)
```

```
bp_wide <- read.csv("data/bp_visits_wide.csv")
```

```
# Keep only participants with both visit 1 and visit 3 recorded
complete_visits <- bp_wide[bp_wide$sbp_visit3 != "", ]
```

```
t.test(complete_visits$sbp_visit1, complete_visits$sbp_visit3, paired = TRUE)
```

Paired t-test

```
data: complete_visits$sbp_visit1 and complete_visits$sbp_visit3
t = 3.3639, df = 25, p-value = 0.002479
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
```



```

1.938809 8.061191
sample estimates:
mean difference
          5

```

135.3. Explore

Try running the same two visit-1 and visit-3 values through Page 5's *unpaired* `t.test(..., paired = FALSE)` instead. Notice the p-value changes — a paired test is typically more powerful when the measurements are genuinely linked (the same person's own before-and-after change), since it removes person-to-person variability that has nothing to do with the change itself.

135.4. Solve the Problem

```

bp_wide <- read.csv("data/bp_visits_wide.csv")

complete_visits <- bp_wide[bp_wide$sbp_visit3 != "", ]

paired_result <- t.test(complete_visits$sbp_visit1, complete_visits$sbp_visit3, paired = TRUE)

paired_result$estimate

mean difference
          5

paired_result$p.value

[1] 0.002479479

```

135. The Paired *t*-Test

Among the 26 participants with complete visit 1 and visit 3 data, blood pressure dropped by an average of exactly 5 points — and this drop is highly statistically significant ($p = 0.0025$), providing strong evidence this isn't just random fluctuation.

135.5. Interpret the Result

Unlike Page 5's smoking comparison ($p = 0.83$, no significant difference), this paired result shows genuine statistical evidence: the average 5-point systolic blood pressure decrease from visit 1 to visit 3, among the same 26 participants, is very unlikely to have arisen purely by chance. This formally confirms what Level 6's reshaping analysis and Level 8's line plot both suggested visually and numerically — a real within-person improvement across the study's visits.

Common Mistakes

- **Forgetting `paired = TRUE`.** Running `t.test(complete_visits$sbp_visit1, complete_visits$sbp_visit3)` without `paired = TRUE` incorrectly analyzes the two visits as independent groups, discarding the fact that each visit-1 value is linked to a specific visit-3 value from the *same* participant — this typically produces a different (and less appropriately powered) result.
- **Including participants with a missing visit in a paired test.** A paired test requires a complete pair for every included participant — the 2 participants who dropped out before visit 3 (Level 6) must be excluded, exactly as `complete_visits` does here by filtering first.
- **Confusing “statistically significant improvement” with “clinically meaningful improvement.”** A 5-point average drop is statistically well-established here — but whether a 5 mmHg change matters clinically is a separate, domain-specific judgment, echoing Page 4's significance-versus-importance distinction.

135.6. Practice Problem

Run a paired t-test comparing `sbp_visit1` and `sbp_visit2` (rather than `sbp_visit3`) for all 28 participants (no filtering needed, since neither visit 1 nor visit 2 has any missing values in this dataset). Is the visit 1-to-visit 2 change also statistically significant?

135.7. Challenge Problem

Compare the paired t-test's p-value (visit 1 vs. visit 3) to what you'd get treating the same two columns as an *unpaired* two-sample comparison (Page 5's approach, ignoring the pairing). Which test produces stronger evidence of a difference, and explain why pairing typically increases a test's statistical power when measurements are genuinely linked.

135.8. Key Takeaways

- A paired t-test analyzes within-individual differences between two linked measurements, testing whether the mean difference significantly differs from zero.
- `paired = TRUE` is essential — omitting it treats linked measurements as independent groups, losing statistical power and answering a different question.
- Pairing typically increases a test's power to detect a real effect, since it removes person-to-person variability unrelated to the actual change being measured.

Knowledge Check

1. **Multiple choice.** When is a paired t-test the appropriate choice over a two-sample t-test?

135. *The Paired t-Test*

- a) When comparing two entirely different groups of people
 - b) When comparing two measurements taken on the same individuals
 - c) When the sample size is very large
 - d) Never — they are always interchangeable
2. **True or false.** Omitting `paired = TRUE` when comparing two linked measurements will still produce a mathematically valid, equally appropriate result.
3. **Predict the output.** If every one of the 26 participants had exactly the same 5-point blood pressure drop (no variation at all), would you expect the p-value to be smaller or larger than what was actually found?
4. **Short answer.** Why must participants with a missing visit 3 be excluded from this paired analysis, rather than simply included with a missing value?

Show answers

- 1. **b)** When comparing two measurements taken on the same individuals.
- 2. **False.** Omitting `paired = TRUE` treats the two sets of measurements as independent groups, discarding the linkage between each pair and typically changing (and often weakening) the statistical result.
- 3. Smaller — with zero variation in the difference across participants, the evidence for a genuine, consistent effect would be even stronger, producing a smaller p-value than the actual result (which has some participant-to-participant variability in the size of the drop).
- 4. A paired test requires a complete pair (both visit 1 and visit 3) for every included participant — a participant with a missing visit 3 has no difference to compute at all, so they cannot contribute a valid pair to the analysis.

136. The Chi-Square Test of Independence

The Problem

Level 7's cross-tabulation found a striking-looking descriptive pattern: 19.2% of women smoke, compared to 31.2% of men — a 12-percentage-point gap. But Level 7 also cautioned that this was a *descriptive* finding, not yet a tested one. Your professor now asks the question Level 7 deliberately left open: is this sex difference in smoking rate statistically significant, or could a gap this size easily arise by chance in a sample of only 100 participants?

Your Mission

Test whether two categorical variables are statistically associated, using a chi-square test of independence.

Think Before Coding

- If sex and smoking status were completely unrelated, would you still expect the exact same smoking rate in both groups, or just *roughly similar* rates?
- From Level 7, how many women and men were in this sample (52 and 48)? Does a 12-point gap in a sample this size feel like a lot of data to draw a firm conclusion from?

136.1. The R Concept

The **chi-square test of independence** compares a cross-tabulation's *observed* counts against the counts you'd *expect* if the two variables were completely unrelated, producing a p-value:

```
chisq.test(table(var1, var2))
```

By default, R applies a continuity correction (Yates' correction) for 2×2 tables, which slightly adjusts the test to be more conservative for small samples.

136.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
sex_smoker_table <- table(participants$sex, participants$smoker)
sex_smoker_table
```

	No	Yes
Female	42	10
Male	33	15

```
chisq.test(sex_smoker_table)
```

Pearson's Chi-squared test with Yates' continuity correction

```
data: sex_smoker_table
X-squared = 1.3355, df = 1, p-value = 0.2478
```

136.3. Explore

Try `chisq.test(sex_smoker_table)$expected` to see the counts that *would* be expected if sex and smoking status were completely independent: roughly 39 female non-smokers and 13 female smokers (compared to the actual 42 and 10). The chi-square statistic measures exactly how far the observed counts (42, 10, 33, 15) deviate from these expected ones.

136.4. Solve the Problem

```

participants <- read.csv("data/participants.csv")

sex_smoker_table <- table(participants$sex, participants$smoker)

test_result <- chisq.test(sex_smoker_table)

test_result$statistic

X-squared
1.335470

test_result$p.value

[1] 0.2478344

test_result$expected

      No Yes
Female 39 13
Male   36 12

```

136. The Chi-Square Test of Independence

A p-value of 0.248 is well above the conventional 0.05 threshold — despite the 12-percentage-point descriptive gap Level 7 found, this chi-square test provides no strong statistical evidence that sex and smoking status are genuinely associated in this sample.

136.5. Interpret the Result

This is a genuinely important, humbling result: Level 7's 19.2%-versus-31.2% gap *looked* substantial when described in percentages, but with only 52 women and 48 men, that gap is well within the range chance alone could plausibly produce. This is exactly why Level 7 cautioned against treating a cross-tabulation's pattern as a confirmed finding — the formal test now confirms that caution was warranted, not just a reflexive disclaimer.

Common Mistakes

- **Treating a large descriptive gap (12 percentage points) as automatically meaningful.** Percentage-point differences can look dramatic without accounting for sample size — a chi-square test (or any formal significance test) directly addresses whether the gap is larger than chance would typically produce, not just whether it looks big.
- **Applying a chi-square test to cell counts that are too small.** `chisq.test()` can become unreliable when expected cell counts are very small (a common rule of thumb: below 5) — always check `$expected` before trusting the result; here, all expected counts (39, 13, 36, 12) are comfortably above that threshold.
- **Confusing “not statistically significant” with “definitely no association exists.”** As with the t-tests earlier in this level, a non-significant chi-square result means this sample doesn't provide strong evidence of an association — it doesn't prove sex and smoking are truly unrelated in the broader population this sample represents.

136.6. Practice Problem

Run a chi-square test on `county` and `smoker` (a larger, 5×2 table). Check `$expected` first to confirm no expected cell count falls below 5, then interpret the resulting p-value.

136.7. Challenge Problem

Compare `chisq.test(sex_smoker_table)` (with Yates' correction, R's default) to `chisq.test(sex_smoker_table, correct = FALSE)` (without it). How much does the p-value change? Research why Yates' correction tends to matter more for smaller samples and 2×2 tables specifically.

136.8. Key Takeaways

- The chi-square test of independence compares observed cross-tabulation counts against what would be expected under complete independence, producing a p-value.
- A descriptive percentage-point gap can look substantial while still being entirely consistent with chance, especially in smaller samples — always follow up a descriptive cross-tabulation (Level 7) with a formal test before drawing conclusions.
- Check `$expected` cell counts before trusting a chi-square result — very small expected counts (conventionally below 5) can make the test unreliable.

Knowledge Check

1. **Multiple choice.** What does the chi-square test of independence evaluate?

136. *The Chi-Square Test of Independence*

- a) Whether two numeric variables are correlated
 - b) Whether two categorical variables are associated, beyond what chance would produce
 - c) Whether a sample mean differs from a reference value
 - d) Whether a variable is normally distributed
2. **True or false.** A large descriptive percentage-point gap between two groups always corresponds to a statistically significant chi-square test result.
 3. **Predict the output.** Given a p-value of 0.248 for the sex-smoking chi-square test, would you conclude there's strong evidence of an association?
 4. **Short answer.** Why is it important to check expected cell counts before trusting a chi-square test's result?

Show answers

1. **b)** Whether two categorical variables are associated, beyond what chance would produce.
2. **False.** As shown here, a 12-percentage-point gap produced a non-significant result ($p = 0.248$) — the size of a descriptive gap and its statistical significance depend heavily on sample size, and don't always align.
3. No — a p-value of 0.248 is well above the conventional 0.05 threshold, indicating no strong evidence of an association in this sample.
4. The chi-square test's approximation becomes less reliable when expected cell counts are very small (conventionally below 5) — trusting a result built on unreliable expected counts risks drawing an incorrect conclusion from a test that wasn't appropriate for that data in the first place.

137. Testing a Correlation for Significance

The Problem

Level 7's `cor()` reported a correlation of 0.16 between BMI and systolic blood pressure — a weak relationship, but `cor()` alone never told you whether even that weak relationship was statistically distinguishable from a true correlation of exactly zero. Your professor asks directly: is this 0.16 genuine evidence of a relationship, or could a correlation this size easily appear by chance alone in a sample of 91 participants?

Your Mission

Test whether a correlation coefficient is statistically significantly different from zero, using `cor.test()`.

Think Before Coding

- If BMI and blood pressure were truly completely unrelated in the broader population, would you expect a sample correlation of *exactly* 0.00, or something close to but not exactly 0?
- Level 7's `cor()` gave you a single number (0.16). What additional piece of information would you need to know whether that number is statistically meaningful?

137. Testing a Correlation for Significance

137.1. The R Concept

`cor.test()` tests whether a correlation coefficient is significantly different from zero, producing both a p-value and a confidence interval — exactly the same hypothesis-testing logic as the t-tests earlier in this level, applied to a correlation instead of a mean:

```
cor.test(x, y)
```

137.2. Demonstration

```
participants <- read.csv("data/participants.csv")  
  
cor.test(participants$bmi, participants$systolic_bp)
```

Pearson's product-moment correlation

```
data: participants$bmi and participants$systolic_bp  
t = 1.5596, df = 89, p-value = 0.1224  
alternative hypothesis: true correlation is not equal to 0  
95 percent confidence interval:  
 -0.04432528  0.35705926  
sample estimates:  
      cor  
0.1631085
```

137.3. Explore

Notice the 95% confidence interval for the correlation (-0.044 to 0.357) **contains 0** — meaning “no correlation at all” remains a plausible value

given this data. Compare this to a hypothetical situation where the interval didn't contain 0 at all (as you'll see with a different variable pair in the Solve section) — that would indicate a statistically significant relationship.

137.4. Solve the Problem

```

participants <- read.csv("data/participants.csv")

bmi_bp_test <- cor.test(participants$bmi, participants$systolic_bp)
sbp_dbp_test <- cor.test(participants$systolic_bp, participants$diastolic_bp)

bmi_bp_test$p.value

[1] 0.1223918

sbp_dbp_test$p.value

[1] 4.119909e-40

sbp_dbp_test$estimate

      cor
0.9203873

```

The BMI-blood pressure correlation (0.16) is *not* statistically significant ($p = 0.122$) — consistent with Level 8's plot showing no clear visual trend. The systolic-diastolic correlation (0.92), by contrast, is overwhelmingly significant (p is a vanishingly small number, effectively zero for practical purposes — R's printed test output would show this as $p\text{-value} < 2.2e-16$), exactly as expected for two blood pressure readings taken from the same measurement.

137.5. Interpret the Result

This confirms, formally, what Level 7 and Level 8 already suggested visually and numerically: the weak BMI-blood pressure relationship in this sample ($r = 0.16$) is not statistically distinguishable from no relationship at all, given only 91 data points. The much stronger systolic-diastolic relationship ($r = 0.92$), on the other hand, is statistically significant beyond any reasonable doubt — a useful contrast for recognizing what a genuinely strong, clearly non-zero correlation's test result looks like.

Common Mistakes

- **Interpreting `cor()`'s raw coefficient alone as sufficient evidence.** A correlation of 0.16, by itself, says nothing about statistical significance — Level 7 intentionally stopped short of this conclusion, since `cor()` alone cannot tell you whether the true population correlation could plausibly be zero.
- **Assuming a non-significant correlation of 0.16 and an even weaker one of 0.05 provide identical evidence.** They don't necessarily — sample size matters enormously; a correlation of 0.16 in a much larger sample could easily be statistically significant, while the same 0.16 here ($n=91$) is not.
- **Forgetting `cor.test()` assumes a roughly linear relationship (Pearson correlation).** If two variables have a strong *curved* (non-linear) relationship, Pearson's correlation and its significance test can both understate the true strength of that relationship — always look at a scatter plot (Level 8) alongside any correlation test.

137.6. Practice Problem

Run `cor.test()` on `age` and `bmi`. Is this correlation statistically significant? Compare the confidence interval to the BMI-blood pressure result above.

137.7. Challenge Problem

Run `cor.test()` on `age` and `systolic_bp` (Level 7 found $r = -0.22$). Determine whether this correlation is statistically significant, and compare its p-value and confidence interval to the BMI-blood pressure test in this page's Solve section. Which relationship has stronger statistical evidence behind it, despite both being described as “weak” by Level 7's rough correlation-strength guide?

137.8. Key Takeaways

- `cor.test(x, y)` tests whether a correlation coefficient is statistically significantly different from zero, providing both a p-value and a confidence interval.
- A weak-looking correlation (like 0.16) can still be reported alongside its significance test — a small coefficient and a non-significant result often go together, but sample size determines exactly where that boundary falls.
- Always pair a correlation test with a scatter plot (Level 8) — Pearson correlation and its significance test both assume a roughly linear relationship.

Knowledge Check

1. **Multiple choice.** What does `cor.test()` add beyond what `cor()` (Level 7) provides?
 - a) A different correlation coefficient
 - b) A p-value and confidence interval for the correlation
 - c) A scatter plot of the two variables
 - d) Nothing — they are identical

137. *Testing a Correlation for Significance*

2. **True or false.** A correlation coefficient of 0.16 is always statistically significant, regardless of sample size.
3. **Predict the output.** If a correlation's 95% confidence interval is $(-0.044, 0.357)$, would you expect the corresponding p-value to be above or below 0.05?
4. **Short answer.** Why is it possible for two “weak” correlations (say, 0.16 and -0.22) to differ in whether they're statistically significant?

Show answers

1. **b)** A p-value and confidence interval for the correlation, indicating whether it's statistically distinguishable from zero.
2. **False.** Statistical significance depends on both the correlation's size and the sample size — the same coefficient can be significant in a larger sample and non-significant in a smaller one.
3. Above 0.05 — a confidence interval containing 0 corresponds to a non-significant result.
4. Statistical significance for a correlation depends on both its magnitude and the sample size (and specifically the resulting t-statistic, which combines both) — two correlations of similar rough magnitude can still produce different p-values if their underlying sample sizes differ, or if one happens to sit closer to the boundary of significance than the other.

138. Simple Linear Regression with `lm()`

The Problem

Level 8's trend line (`geom_smooth(method = "lm")`) showed blood pressure sloping gently downward with age, and Level 7's correlation confirmed a weak negative relationship (-0.22). But neither told you the actual *equation* of that line — how many mmHg blood pressure is predicted to change per additional year of age — or whether that specific slope is itself statistically significant. Your professor wants the full regression model, not just the plot and the correlation.

Your Mission

Fit a simple linear regression model using `lm()`, and interpret its coefficients, R-squared, and significance.

Think Before Coding

- Level 8's `geom_smooth(method = "lm")` fit a straight line through the age-blood pressure scatter plot. A straight line is described by two numbers — which two?
- If age has a statistically significant relationship with blood pressure, would you expect the slope's p-value to be small or large?

138.1. The R Concept

`lm()` (“linear model”) fits a straight-line equation predicting one numeric variable from another:

```
model <- lm(outcome ~ predictor, data = df)
summary(model)
```

The fitted equation takes the form $\hat{y} = \text{intercept} + \text{slope} \times x$. `summary()` reports the intercept and slope (each with their own standard error, t-statistic, and p-value), plus overall model statistics: **R-squared** (the proportion of variance in the outcome explained by the predictor) and an overall **F-statistic** with its p-value.

138.2. Demonstration

```
participants <- read.csv("data/participants.csv")

bp_age_model <- lm(systolic_bp ~ age, data = participants)

summary(bp_age_model)
```

Call:

```
lm(formula = systolic_bp ~ age, data = participants)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-36.251	-14.104	-1.394	13.232	43.949

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
--	----------	------------	---------	----------

138.3. Explore

```
(Intercept) 142.1669      6.4332  22.099 < 2e-16 ***
age          -0.2762      0.1299  -2.127  0.0363 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 18.7 on 88 degrees of freedom
(10 observations deleted due to missingness)
Multiple R-squared:  0.04888,    Adjusted R-squared:  0.03807
F-statistic: 4.522 on 1 and 88 DF,  p-value: 0.03626
```

138.3. Explore

Try `coef(bp_age_model)` to extract just the intercept and slope directly, and `confint(bp_age_model)` for their 95% confidence intervals. Notice `age`'s confidence interval doesn't contain 0 — consistent with its p-value (0.036) being below 0.05.

138.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

bp_age_model <- lm(systolic_bp ~ age, data = participants)

coef(bp_age_model)

(Intercept)      age
 142.166864   -0.276224

summary(bp_age_model)$r.squared

[1] 0.04887722
```

138. Simple Linear Regression with `lm()`

The fitted equation is **predicted blood pressure = 142.17 – 0.276 × age**: for every additional year of age, predicted systolic blood pressure decreases by about 0.276 mmHg, on average. The slope’s p-value (0.036) is just below 0.05 — statistically significant, though the model’s R-squared (0.049) shows age alone explains under 5% of the variation in blood pressure across participants.

138.5. Interpret the Result

Age has a statistically significant, but practically modest, linear relationship with systolic blood pressure in this sample: a 20-year age difference predicts only about a 5.5 mmHg difference in blood pressure (20×0.276), and the vast majority of blood-pressure variation (95%+) is explained by factors *other than* age alone. This is a genuinely useful, honest way to report the finding — significant, but far from the whole story, exactly consistent with Level 7’s “weak” correlation label and Level 8’s subtle trend line.

Common Mistakes

- **Reporting the slope’s significance without also reporting R-squared.** A statistically significant slope ($p = 0.036$) doesn’t mean age is a *strong* predictor — R-squared (0.049) shows age explains less than 5% of blood pressure’s variation, an essential piece of context a p-value alone doesn’t convey.
- **Interpreting the intercept (142.17) as a meaningful real-world prediction.** The intercept represents predicted blood pressure at age 0 — a nonsensical age for this dataset (Level 4’s realistic age range starts at 18) — the intercept here is a mathematical anchor for the line, not a meaningful standalone prediction.
- **Forgetting `lm()` silently drops rows with missing values.** The output notes “10 observations deleted due to missingness” — `age` and `systolic_bp`’s combined missingness (Level 3-4) reduced the usable

sample from 100 to 90, a detail easy to overlook if you don't read the model's fine print.

138.6. Practice Problem

Fit `lm(systolic_bp ~ bmi, data = participants)` and compare its slope, p-value, and R-squared to the age model above. Which predictor has a larger R-squared?

138.7. Challenge Problem

Fit `lm(systolic_bp ~ diastolic_bp, data = participants)`, given Level 7's much stronger correlation (0.92) between these two variables. Compare this model's R-squared to the age model's — does a much stronger correlation translate into a much higher R-squared, and can you explain the mathematical relationship between a simple linear regression's R-squared and its predictor's correlation coefficient?

138.8. Key Takeaways

- `lm(outcome ~ predictor, data = df)` fits a straight-line equation; `summary()` reports the intercept, slope, their significance, and overall model R-squared.
- A statistically significant slope doesn't imply a practically strong predictor — always report R-squared alongside significance to convey how much variation is actually explained.
- `lm()` silently drops rows with missing values in any variable used in the model — always check the reported sample size against your expectations.

Knowledge Check

1. **Multiple choice.** In `predicted blood pressure = 142.17 - 0.276 × age`, what does the -0.276 represent?
 - a) The predicted blood pressure at age 0
 - b) The change in predicted blood pressure for each additional year of age
 - c) The correlation between age and blood pressure
 - d) The R-squared of the model
2. **True or false.** A statistically significant slope automatically means the predictor explains most of the outcome's variation.
3. **Predict the output.** If R-squared is 0.049, approximately what percentage of blood pressure's variation is explained by age alone?
4. **Find the error.**

```
# "Age is a strong predictor of blood pressure since its slope is statist
```

Show answers

1. **b)** The change in predicted blood pressure for each additional year of age (the slope).
2. **False.** Statistical significance indicates the slope is unlikely to be exactly zero — it says nothing about how much of the outcome's total variation the predictor actually explains (that's what R-squared measures).
3. About 4.9%.
4. This statement conflates statistical significance with predictive strength — a p-value of 0.036 indicates the slope is unlikely to be exactly zero, but the model's R-squared (0.049) shows age explains under 5% of blood pressure's variation, meaning it is a statistically detectable but practically weak predictor, not a “strong” one.

139. One-Way ANOVA

The Problem

Level 5's `group_by(county) + summarise()` found blood pressure means ranging from 124.36 (Riverdale) to 133.30 (Millbrook) across the study's five counties — real-looking differences. But Page 5's two-sample t-test only ever compares *two* groups at a time. With five counties, comparing every possible pair (10 separate t-tests) would be tedious, and would inflate the chance of a false positive from running so many tests. Your professor asks for one single test of whether blood pressure differs across *all five* counties at once.

Your Mission

Test whether a numeric variable's mean differs across more than two groups simultaneously, using one-way ANOVA.

Think Before Coding

- If you wanted to compare blood pressure across all 5 counties using only two-sample t-tests (Page 5), how many separate pairwise comparisons would that require?
- If blood pressure genuinely didn't differ by county at all, would you still expect the 5 counties' sample means to be spread out a little, or to be exactly identical?

139. One-Way ANOVA

139.1. The R Concept

One-way ANOVA (“analysis of variance”) tests whether the means of **three or more** groups differ significantly, using a single overall test rather than multiple pairwise comparisons:

```
model <- aov(numeric_variable ~ grouping_variable, data = df)
summary(model)
```

ANOVA compares how much variation exists *between* group means to how much variation exists *within* each group — producing an F-statistic and a single overall p-value.

139.2. Demonstration

```
participants <- read.csv("data/participants.csv")
```

```
bp_county_model <- aov(systolic_bp ~ county, data = participants)
```

```
summary(bp_county_model)
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
county	4	783	195.7	0.539	0.708
Residuals	91	33048	363.2		

139.3. Explore

Try `TukeyHSD(bp_county_model)` — a follow-up test that compares every pair of counties individually, *while* adjusting for the fact that multiple comparisons are being made. This is the statistically appropriate way to

139.4. Solve the Problem

answer “which specific counties differ,” rather than running 10 separate, unadjusted two-sample t-tests.

139.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

bp_county_model <- aov(systolic_bp ~ county, data = participants)

summary(bp_county_model)
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
county	4	783	195.7	0.539	0.708
Residuals	91	33048	363.2		

```
summary(lm(systolic_bp ~ county, data = participants))$r.squared

[1] 0.02313655
```

The F-statistic (0.539) is small, and the p-value (0.708) is far above 0.05 — no statistically significant evidence that mean systolic blood pressure differs across the five counties. County membership explains only about 2.3% of the variation in blood pressure (R-squared = 0.023).

139.5. Interpret the Result

Despite Level 5’s grouped means ranging from 124.36 to 133.30 — a roughly 9-point spread that might look meaningful in a table — the ANOVA shows this spread is entirely consistent with random sampling variation across five groups of modest size (16 to 26 participants each).

139. One-Way ANOVA

This doesn't mean county is *irrelevant* to blood pressure in the broader population; it means this particular sample doesn't provide strong statistical evidence of a real county-level difference.

Common Mistakes

- **Running multiple unadjusted two-sample t-tests instead of one ANOVA.** Testing every pair of 5 counties individually (10 tests) substantially increases the chance of at least one false-positive result appearing purely by chance — ANOVA's single overall test avoids this problem; `TukeyHSD()` handles pairwise follow-up comparisons correctly if needed afterward.
- **Assuming a non-significant ANOVA means every group is identical.** It means the *overall* test found no significant difference across all groups combined — a follow-up test could still reveal one specific pair with a real difference, though `TukeyHSD()` would be needed to check this properly rather than assumed from the overall ANOVA result alone.
- **Confusing ANOVA's F-statistic with a t-test's t-statistic.** They're related (an F-statistic here is testing multiple groups simultaneously) but not directly comparable in magnitude — always interpret the p-value, not the raw statistic's size, when judging significance.

139.6. Practice Problem

Run a one-way ANOVA testing whether `bmi` differs across the five counties. Report the F-statistic, p-value, and R-squared, and compare to the blood pressure result above.

139.7. Challenge Problem

Run `TukeyHSD(bp_county_model)` and examine the adjusted p-value for the Millbrook-Riverdale comparison specifically (the two counties with the largest raw mean difference, Level 5). Is this specific pairwise comparison statistically significant after the multiple-comparison adjustment, even though the overall ANOVA was not?

139.8. Key Takeaways

- One-way ANOVA tests whether three or more groups' means differ significantly, using a single overall F-test rather than multiple pairwise comparisons.
- A non-significant ANOVA result means the sample lacks strong evidence of an overall difference across groups — it doesn't rule out every possible pairwise difference without a proper follow-up test like `TukeyHSD()`.
- Running many unadjusted pairwise t-tests instead of one ANOVA inflates the risk of a false-positive finding purely from the number of comparisons made.

Knowledge Check

1. **Multiple choice.** When is one-way ANOVA the appropriate choice over a two-sample t-test?
 - a) When comparing exactly two groups
 - b) When comparing three or more groups' means simultaneously
 - c) When testing a correlation coefficient
 - d) When the outcome variable is categorical
2. **True or false.** Running 10 separate two-sample t-tests to compare 5 groups pairwise is statistically equivalent to running one ANOVA.

139. *One-Way ANOVA*

3. **Predict the output.** Given an ANOVA p-value of 0.708, would you conclude blood pressure significantly differs across the five counties?
4. **Short answer.** Why does running many pairwise comparisons instead of one overall test increase the risk of a false-positive finding?

Show answers

1. **b)** When comparing three or more groups' means simultaneously.
2. **False.** Multiple unadjusted pairwise tests inflate the overall chance of at least one false-positive result — ANOVA's single test (or a properly adjusted follow-up like `TukeyHSD()`) accounts for this in a way that separate, unadjusted t-tests do not.
3. No — a p-value of 0.708 is far above 0.05, indicating no statistically significant evidence of a difference across counties.
4. Each individual test carries some chance (conventionally 5%) of a false-positive result purely by chance — running many such tests compounds this risk, making it increasingly likely that at least one test shows “significance” purely due to chance, even if no real differences exist anywhere.

140. Logistic Regression for a Binary Outcome

The Problem

Page 9's `lm()` predicted a continuous outcome (blood pressure) from age. But `smoker` is a **binary** outcome ("Yes"/"No") — not a continuous number `lm()` is designed to predict. Your professor asks whether age and BMI together predict the *probability* of being a smoker — a question `lm()` isn't built to answer directly, since a straight line can predict values below 0 or above 1, which make no sense as probabilities.

Your Mission

Fit a logistic regression model using `glm()` to predict a binary outcome, and interpret its coefficients as odds ratios.

Think Before Coding

- Why might a straight line (`lm()`) be a poor fit for predicting something that must always be between 0 and 1, like a probability?
- Level 5's `is_at_risk()` and `is.numeric(x)` used `TRUE/FALSE` values. If R can treat `TRUE` as 1 and `FALSE` as 0 (Level 2), could `smoker == "Yes"` become a numeric 0/1 outcome for modeling?

140.1. The R Concept

`glm()` (“generalized linear model”) with `family = binomial` fits a **logistic regression** — a model for a binary (0/1) outcome that always predicts a valid probability between 0 and 1:

```
model <- glm(outcome_binary ~ predictor1 + predictor2, data = df, family = b
summary(model)
```

Logistic regression coefficients are on a “log-odds” scale, which is hard to interpret directly — exponentiating them (`exp(coef(model))`) converts them into **odds ratios**, a much more interpretable quantity: an odds ratio above 1 means higher values of the predictor are associated with higher odds of the outcome; below 1 means lower odds.

140.2. Demonstration

```
participants <- read.csv("data/participants.csv")

participants$smoker_binary <- as.numeric(participants$smoker == "Yes")

smoker_model <- glm(smoker_binary ~ age + bmi, data = participants, family =
summary(smoker_model)
```

Call:

```
glm(formula = smoker_binary ~ age + bmi, family = binomial,
    data = participants)
```

Coefficients:

Estimate Std. Error z value Pr(>|z|)

140.3. Explore

```
(Intercept) -0.913984    1.596931   -0.572    0.567
age          0.008919    0.016554    0.539    0.590
bmi         -0.023321    0.052168   -0.447    0.655
```

(Dispersion parameter for binomial family taken to be 1)

```
Null deviance: 99.543  on 88  degrees of freedom
Residual deviance: 99.072  on 86  degrees of freedom
AIC: 105.07
```

140.3. Explore

Try `exp(coef(smoker_model))` to convert the coefficients into odds ratios. An odds ratio near 1.00 (for either `age` or `bmi`) indicates that predictor has little association with the odds of being a smoker — very different from an odds ratio of, say, 2.0 (doubled odds) or 0.5 (halved odds).

140.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

participants$smoker_binary <- as.numeric(participants$smoker == "Yes")

smoker_model <- glm(smoker_binary ~ age + bmi, data = participants, family = binomial)

exp(coef(smoker_model))

(Intercept)          age          bmi
  0.4009245    1.0089595    0.9769487

exp(confint(smoker_model))
```

140. Logistic Regression for a Binary Outcome

	2.5 %	97.5 %
(Intercept)	0.01698651	8.9573952
age	0.97654895	1.0430263
bmi	0.88132002	1.0821256

Neither `age` (OR = 1.009, 95% CI: 0.977-1.043) nor `bmi` (OR = 0.977, 95% CI: 0.881-1.082) has an odds ratio confidence interval excluding 1 — both intervals comfortably contain 1.0, meaning neither predictor shows a statistically significant association with smoking status in this sample.

140.5. Interpret the Result

Neither age nor BMI meaningfully predicts smoking status in this sample: both odds ratios sit very close to 1.00 (1.009 and 0.977), and both p-values (0.590 and 0.655) are far from significant. This is consistent with everything found earlier in the course — Level 7’s cross-tabulations and Page 7’s chi-square test both suggested weak, non-significant relationships involving smoking status, and this logistic regression confirms the same conclusion using two predictors simultaneously rather than one categorical comparison at a time.

Common Mistakes

- **Interpreting logistic regression coefficients directly, without exponentiating first.** The raw coefficients (like `age`’s 0.0089) are on a log-odds scale, not directly interpretable as “probability change per unit” — `exp(coef(model))` is required to get the more interpretable odds ratio.
- **Forgetting `family = binomial` is required.** Without it, `glm()` defaults to ordinary linear regression (equivalent to `lm()`), which is inappropriate for a 0/1 outcome and can predict nonsensical values like -0.3 or 1.4 as a “probability.”

- **Treating a non-significant model as proof age and BMI have no relationship to smoking whatsoever, in any population.** As with every non-significant result in this level, this only means *this particular sample* of 89 participants doesn't provide strong evidence of an association — not a universal claim about the true, broader relationship.

140.6. Practice Problem

Fit a logistic regression predicting `smoker_binary` from `age` alone (no `bmi`). Compare its age coefficient and p-value to the two-predictor model above — does adding `bmi` change age's estimated association much?

140.7. Challenge Problem

Fit a logistic regression predicting a **hypertension** binary outcome (`systolic_bp` $\geq$ 130, Level 4's threshold) from `age` and `bmi`. Compare the resulting odds ratios and p-values to the smoker model in this page — does either predictor show a stronger (or weaker) association with hypertension than with smoking status?

140.8. Key Takeaways

- `glm(outcome ~ predictors, data = df, family = binomial)` fits a logistic regression for a binary (0/1) outcome, always predicting a valid probability between 0 and 1.
- Logistic regression coefficients are on a log-odds scale — `exp(coef(model))` converts them into odds ratios, a far more interpretable quantity.

140. Logistic Regression for a Binary Outcome

- An odds ratio's 95% confidence interval containing 1.0 indicates no statistically significant association, exactly analogous to a correlation's confidence interval containing 0 (Page 8).

Knowledge Check

1. **Multiple choice.** Why is `lm()` inappropriate for predicting a binary (0/1) outcome directly?
 - a) `lm()` cannot handle numeric predictors
 - b) `lm()` can predict values outside the valid 0-to-1 probability range
 - c) `lm()` requires exactly one predictor
 - d) There is no actual problem with using `lm()` this way
2. **True or false.** An odds ratio of exactly 1.00 indicates no association between the predictor and the outcome's odds.
3. **Predict the output.** If a predictor's odds ratio 95% confidence interval is (0.88, 1.08), would you conclude that predictor is statistically significantly associated with the outcome?
4. **Find the error.**

```
# "The age coefficient of 0.0089 means each additional year increases th
```

Show answers

1. **b)** `lm()` can predict values outside the valid 0-to-1 probability range, which makes no sense for a binary outcome.
2. **True.** An odds ratio of exactly 1.00 means the predictor has no association with the odds of the outcome.
3. No — a confidence interval containing 1.0 (as (0.88, 1.08) does) indicates no statistically significant association.

4. Raw logistic regression coefficients are on a log-odds scale, not a direct percentage-probability scale — the coefficient must first be exponentiated ($\exp(0.0089) = 1.009$) to get the odds ratio, and even then, an odds ratio doesn't translate directly into a simple percentage-point change in probability the way this statement implies.

141. Level 11 Capstone: Testing This Course's Earlier Findings

The Problem

This level's opening mission, finally answerable: Level 7 found smoking rates differ by sex (19.2% women, 31.2% men), and Level 8's trend line showed blood pressure declining gently with age. Both were purely descriptive findings at the time — plots and percentages, without a formal test behind them. Your professor wants a final, side-by-side verdict: which of these two patterns holds up to formal statistical testing, and which doesn't?

Your Mission

Formally test both of this course's earlier descriptive findings, using the tools from this entire level, and report a clear, honest verdict on each.

Think Before Coding

- Which test from this level is appropriate for “does smoking rate differ by sex” — a chi-square test, a t-test, or a correlation test?
- Which test is appropriate for “does blood pressure decline with age” — the same test, or a different one?

141.1. The R Concept

Choosing the right test depends entirely on the *type* of variables involved:

141. Level 11 Capstone: Testing This Course's Earlier Findings

```
# Two categorical variables -> chi-square test
chisq.test(table(categorical1, categorical2))

# One numeric, one numeric (linear relationship) -> correlation test or regression
cor.test(numeric1, numeric2)
lm(numeric1 ~ numeric2)
```

Sex and smoking status are both categorical — a chi-square test (Page 7) is the right tool. Age and blood pressure are both numeric — a correlation test (Page 8) or simple linear regression (Page 9) are both appropriate, and (as Page 9's challenge problem hinted) mathematically equivalent for a single predictor.

141.2. Demonstration

```
participants <- read.csv("data/participants.csv")

# Finding 1: sex and smoking status (Level 7's cross-tabulation)
sex_smoker_table <- table(participants$sex, participants$smoker)
chisq.test(sex_smoker_table)

# Finding 2: age and blood pressure (Level 8's trend line)
cor.test(participants$age, participants$systolic_bp)
```

Pearson's Chi-squared test with Yates' continuity correction

```
data: sex_smoker_table
X-squared = 1.3355, df = 1, p-value = 0.2478
```

Pearson's product-moment correlation

141.3. Explore

```
data: participants$age and participants$systolic_bp
t = -2.1266, df = 88, p-value = 0.03626
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.4094278 -0.0146621
sample estimates:
      cor
-0.2210819
```

141.3. Explore

Notice the two p-values point in opposite directions: 0.248 (not significant) for sex-and-smoking, but 0.036 (significant) for age-and-blood-pressure. Both were originally reported in this course as comparably “modest” descriptive patterns — only the formal tests reveal that one holds up statistically and the other doesn’t.

141.4. Solve the Problem

```
participants <- read.csv("data/participants.csv")

sex_smoker_test <- chisq.test(table(participants$sex, participants$smoker))
age_bp_test <- cor.test(participants$age, participants$systolic_bp)

sex_smoker_test$p.value

[1] 0.2478344

age_bp_test$p.value
```

141. Level 11 Capstone: Testing This Course's Earlier Findings

```
[1] 0.03625678
```

```
age_bp_test$estimate
```

```
cor
```

```
-0.2210819
```

Finding 1 (sex and smoking status): not statistically significant ($p = 0.248$). Despite a 12-percentage-point descriptive gap, this sample of 52 women and 48 men doesn't provide strong evidence of a true association between sex and smoking status.

Finding 2 (age and blood pressure): statistically significant ($p = 0.036$, $r = -0.221$). Older participants do show a modest but statistically real tendency toward lower systolic blood pressure in this sample — confirmed by both the correlation test here and the regression slope test on Page 9.

141.5. Interpret the Result

This side-by-side verdict is exactly the kind of honest, complete conclusion a real analysis should reach: one of Level 7 and Level 8's two earlier patterns turned out to be statistically robust (age and blood pressure), while the other — despite looking similarly notable in a plot or percentage table — did not hold up (sex and smoking status). Neither conclusion is a “failure”: correctly identifying which patterns are statistically supported and which aren't is precisely the value this entire level adds on top of Levels 5, 7, and 8's descriptive and visual work.

Common Mistakes

141.6. Practice Problem

- **Reporting only the significant finding and quietly dropping the non-significant one.** A complete, honest analysis reports both results — a non-significant finding is still a real, useful piece of information (it tells you a suspected pattern isn't well-supported by this data), not something to omit because it's less exciting.
- **Assuming a smaller p-value always means a “more important” finding.** The age-blood pressure relationship ($p = 0.036$, $r = -0.221$) is statistically significant but still numerically weak (R-squared under 5%, Page 9) — statistical significance and practical importance remain two separate questions, exactly as emphasized throughout this level.
- **Forgetting that “significant” and “not significant” are both provisional conclusions from one particular sample.** A larger, independently collected sample could, in principle, find sex and smoking status significantly associated after all, or fail to replicate the age-blood pressure finding — every result in this level describes this specific sample of 100 participants, not an eternal truth about the wider population.

141.6. Practice Problem

Choose one more descriptive finding from earlier in this course (for example, Level 5's blood pressure ranking by county, or Level 8's BMI-by-county faceted histogram) and select the appropriate formal test from this level to check whether it holds up statistically.

141.7. Challenge Problem

Write a short paragraph (as if for the final section of a real report) summarizing both findings from this capstone: state each result, its statistical significance (or lack of it), its effect size (correlation or percentage-point

141. Level 11 Capstone: Testing This Course's Earlier Findings

gap), and an honest, appropriately cautious interpretation of what both findings do and don't establish about this dataset.

141.8. Key Takeaways

- Choosing the correct statistical test depends on variable type: chi-square for two categorical variables, correlation/regression for two numeric variables, t-tests for comparing means across groups.
- A complete analysis reports both significant and non-significant results honestly — omitting inconvenient non-significant findings misrepresents the evidence.
- Statistical significance and practical/clinical importance remain distinct: a significant result can still describe a weak relationship, and every conclusion in this level is provisional to this specific sample of 100 participants.

Knowledge Check

1. **Multiple choice.** Which test correctly evaluates whether sex and smoking status (both categorical) are associated?
 - a) A t-test
 - b) A chi-square test
 - c) A correlation test
 - d) Simple linear regression
2. **True or false.** Both of this course's earlier descriptive findings (sex-smoking, age-blood pressure) turned out to be statistically significant when formally tested.
3. **Predict the output.** Given a p-value of 0.036 for the age-blood pressure correlation, would you conclude this relationship is statistically significant at the conventional 0.05 threshold?

4. **Short answer.** Why is it important to report a non-significant finding (like the sex-smoking result) rather than omitting it from a final report?

Show answers

1. **b)** A chi-square test — the appropriate tool for testing association between two categorical variables.
2. **False.** Only the age-blood pressure relationship ($p = 0.036$) was statistically significant; the sex-smoking association ($p = 0.248$) was not.
3. Yes — 0.036 is below the conventional 0.05 threshold, indicating statistical significance.
4. Omitting non-significant findings misrepresents the true state of evidence — a reader deserves to know that a plausible-looking descriptive pattern (like the sex-smoking gap) did not hold up to formal testing, just as much as they deserve to know which patterns did.

Part XII.

Level 12 — Advanced Data Analysis

Level 12 — Advanced Data Analysis

Where this level is headed

Level 11 fit models with a single predictor at a time: one t-test, one correlation, one simple regression line. Real analyses almost never stop there — a professor asking “does age predict blood pressure” almost always follows up with “yes, but what about BMI and smoking status too? Could one of those be behind the age effect instead?” This level extends `lm()` and `glm()` to multiple predictors at once, and gives you the tools to answer exactly that kind of follow-up honestly.

What this level covers

- Fitting multiple regression with more than one numeric predictor at once
- Using a categorical variable (like `county`) directly as a regression predictor, and reading its dummy-coded coefficients
- Interaction terms — testing whether one predictor’s effect depends on the value of another
- Confounding and adjustment — checking whether a predictor’s effect changes once another variable is controlled for
- Formally comparing nested models with `anova()` and AIC, instead of eyeballing R-squared
- Basic regression diagnostics: checking residual normality and constant variance
- Multicollinearity — what it is, how to detect it, and why it matters

- Making predictions for new, unobserved data with `predict()`

A preview mission

Level 11's simple regression found age predicts systolic blood pressure (Page 9), on its own. Your mission across this level: build up a fuller model — age, BMI, and smoking status together — and find out whether age's effect survives once those other variables are accounted for, or whether it was confounded all along.

i A note on scope

This level focuses on the multiple-regression toolkit every analyst reaches for first: multiple predictors, categorical predictors, interactions, confounding, diagnostics, and prediction — all built on the same 100-participant dataset used throughout this course. Topics like multiple imputation, propensity score matching, survey weighting, mixed-effects models, and time-to-event (survival) analysis are genuinely important in public health research, but each requires either specialized data this course's dataset doesn't have (repeated survey waves, censored event times) or a level of mathematical background beyond this course's scope. Consider this level a solid on-ramp to those topics, not the final word on them.

Part XIII.

**Level 13 — Advanced R
Programming**

Level 13 — Advanced R Programming

Where this level is headed

To build tools other researchers rely on — not just one-off scripts — you need to understand how R actually works under the hood: environments, object systems, error handling, and version control.

What's coming

- Environments, scoping, and closures
- S3, S4, and R6 object systems
- Debugging and `tryCatch()`
- Functional programming patterns
- Package development and documentation
- Git and GitHub for reproducible collaboration

A preview mission

Your mission: turn a script you've rerun ten times into a small, tested, documented R package your whole team can install and trust.

i Status

This level's lessons are being built out next, following the same Problem → Think → Learn → Try It → Solve → Interpret → Practice → Challenge structure as Level 1. If you've just finished the level before this one, you're exactly on schedule — keep going.

Part XIV.

Level 14 — Reproducible Research

Level 14 — Reproducible Research

Where this level is headed

A result nobody else can reproduce isn't finished research. This level is about Quarto itself: turning code, figures, tables, and citations into a single reproducible document, website, or presentation — including the very book you're reading now.

What's coming

- R Markdown vs. Quarto, and YAML front matter
- Code chunks, inline code, tables, and figures
- Cross-references, citations, and bibliographies
- Parameterized reports
- HTML, PDF, and Word output
- Building websites, books, and presentations

A preview mission

Your mission: turn a messy analysis script into a polished, reproducible report that regenerates itself the moment the underlying data changes.

i Status

This level's lessons are being built out next, following the same Problem → Think → Learn → Try It → Solve → Interpret → Practice

Level 14 — Reproducible Research

→ Challenge structure as Level 1. If you've just finished the level before this one, you're exactly on schedule — keep going.

Part XV.

Level 15 — Real-World Capstone Projects

Level 15 — Real-World Capstone Projects

Where this level is headed

Everything up to this point was scaffolded. Here, you get a research question and raw data, and you're expected to design the entire workflow yourself — from cleaning through modeling to a reproducible final report.

What's coming

- Physical activity and obesity
- Factors associated with hypertension
- Smoking and cardiovascular risk
- Predictors of cognitive decline
- Neighborhood deprivation and health outcomes
- Longitudinal and survival analysis projects
- A reproducible research report and interactive dashboard
- An independent project from raw data to publication-ready results

A preview mission

Your mission: pick a real public-health question, and take it from a raw CSV file all the way to a report you'd be comfortable putting your name on.

Level 15 — Real-World Capstone Projects

i Status

This level's lessons are being built out next, following the same Problem → Think → Learn → Try It → Solve → Interpret → Practice → Challenge structure as Level 1. If you've just finished the level before this one, you're exactly on schedule — keep going.